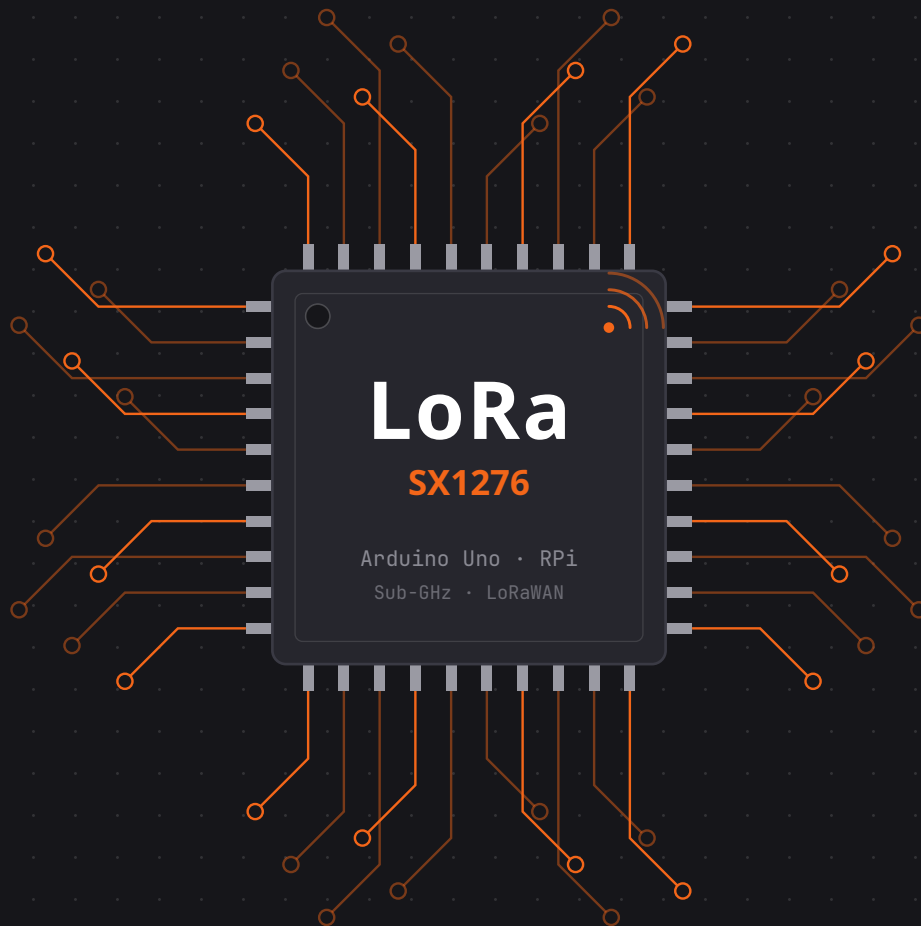


LORA COMMUNICATION LAB · BUKU KERJA LABORATORIUM

Petunjuk Praktikum Komunikasi Jarak Jauh dengan **LoRa**

Arduino Uno & Raspberry Pi Version - Buku Pegangan untuk Pengajar



LoRa P2P

ACK

Master-Slave

ALOHA

CSMA/CA

TDMA

LoRaWAN

PENULIS

Akhmad Hendriawan

TAHUN

2026

Lisensi Dokumen

Copyright © 2026 oleh Akhmad Hendriawan.

Dokumen ini disusun sebagai petunjuk praktikum mata kuliah Wireless Sensor Network dan Internet of Things, seri komunikasi LoRa. Seluruh isinya dapat digunakan dan disebarluaskan secara bebas untuk tujuan bukan komersial (nonprofit), dengan syarat tidak menghapus atau mengubah atribut penulis dan pernyataan copyright yang disertakan di dalamnya.

Seluruh berkas kode sumber setiap modul dimuat lengkap di dalam buku ini, sehingga buku dapat dipakai tanpa akses jaringan. Salinan daringnya berada pada repositori praktikum github.com/hendrieepis/WSN-IOT-prak-Lora, di dalam folder ModulNN_nama yang disebut pada awal tiap modul.

Departemen Teknik Elektro, Politeknik Elektronika Negeri Surabaya

Daftar Isi

Pendahuluan	18
Tentang Lab Ini	18
Struktur Setiap Modul	18
Keterkaitan Antar-Modul	19
Daftar Modul	20
Perangkat Keras	22
Arduino Uno + Dragino LoRa Shield v1.2	22
Raspberry Pi + Dragino LoRa GPS HAT v1.4	23
Perangkat Lunak	24
Board Bercampur: Uno Asli dan Klon	25
Menjalankan	26
Status Verifikasi	26
Kode Sumber dan Repositori	28
Referensi	28
1 Modul 01 – Tautan LoRa Satu Arah	29
1.1 Pendahuluan	29
1.2 Capaian Pembelajaran	29
1.3 Dasar Teori (Secukupnya)	30
1.4 Topologi	31
1.5 Alat yang Digunakan	31
1.6 Kode Program	32
1.7 Build dan Flash	40
1.8 Percobaan	41
1.8.1 EXP-01 – Inisialisasi Radio	41
1.8.2 EXP-02 – Aliran Data Satu Arah	41
1.8.3 EXP-03 – Parameter Harus Seragam	42
1.8.4 Verifikasi Perangkat Keras (Log Referensi)	43
1.8.5 Log Serial Terverifikasi	43
1.9 Pengukuran	46
1.10 Analisis	47
1.11 Concept Check	47
1.12 Challenge (Tugas Modifikasi)	47
1.13 Laporan	48
2 Modul 02 – Penerimaan Non-Blocking dan Indikator LED	49
2.1 Pendahuluan	49
2.2 Capaian Pembelajaran	50
2.3 Dasar Teori (Secukupnya)	50

2.4	Topologi	51
2.5	Alat yang Digunakan	51
2.6	Kode Program	52
2.7	Build dan Flash	60
2.8	Percobaan	61
2.8.1	EXP-01 – Penerimaan Tanpa Polling	61
2.8.2	EXP-02 – LED Sebagai Instrumen	62
2.8.3	EXP-03 – Bukti Non-Blocking	62
2.8.4	Verifikasi Perangkat Keras (Log Referensi)	63
2.8.5	Log Serial Terverifikasi	63
2.9	Pengukuran	65
2.10	Analisis	66
2.11	Concept Check	66
2.12	Challenge (Tugas Modifikasi)	66
2.13	Laporan	67
3	Modul 03 – Peer-to-Peer Ping-Pong	68
3.1	Pendahuluan	68
3.2	Capaian Pembelajaran	68
3.3	Dasar Teori (Secukupnya)	69
3.4	Topologi	70
3.5	Alat yang Digunakan	70
3.6	Kode Program	70
3.7	Build dan Flash	78
3.8	Percobaan	79
3.8.1	EXP-01 – Percakapan Berkelanjutan	79
3.8.2	EXP-02 – Waktu Pulang-Pergi	80
3.8.3	EXP-03 – Percakapan Terputus dan Pulih	80
3.8.4	Verifikasi Perangkat Keras (Log Referensi)	81
3.8.5	Log Serial Terverifikasi	82
3.9	Pengukuran	84
3.10	Analisis	85
3.11	Concept Check	85
3.12	Challenge (Tugas Modifikasi)	85
3.13	Laporan	86
4	Modul 04 – Keandalan Terukur dengan ACK	87
4.1	Pendahuluan	87
4.2	Capaian Pembelajaran	88
4.3	Dasar Teori (Secukupnya)	88
4.4	Topologi	89
4.5	Alat yang Digunakan	89
4.6	Kode Program	90
4.7	Build dan Flash	100
4.8	Percobaan	100

4.8.1	EXP-01 — Siklus ACK yang Sehat	100
4.8.2	EXP-02 — Batas Waktu Bekerja	101
4.8.3	EXP-03 — Nomor Urut Harus Cocok	102
4.8.4	Verifikasi Perangkat Keras (Log Referensi)	102
4.8.5	Log Serial Terverifikasi	103
4.9	Pengukuran	105
4.10	Analisis	106
4.11	Concept Check	106
4.12	Challenge (Tugas Modifikasi)	106
4.13	Laporan	106
5	Modul 05 — Master-Slave 3 Node (Round-Robin Polling)	108
5.1	Pendahuluan	108
5.2	Capaian Pembelajaran	109
5.3	Dasar Teori (Secukupnya)	109
5.4	Topologi	110
5.5	Alat yang Digunakan	110
5.6	Kode Program	111
5.7	Build dan Flash	128
5.8	Percobaan	128
5.8.1	EXP-01 — Slave Menyaring Panggilan	128
5.8.2	EXP-02 — Siklus Round-Robin	129
5.8.3	EXP-03 — Satu Node Hilang	130
5.8.4	EXP-04 — Tabrakan yang Disengaja	130
5.8.5	Verifikasi Perangkat Keras (Log Referensi)	131
5.8.6	Log Serial Terverifikasi	132
5.9	Pengukuran	137
5.10	Analisis	138
5.11	Concept Check	138
5.12	Challenge (Tugas Modifikasi)	138
5.13	Laporan	139
6	Modul 06 — LoRa dari Linux (Python, tanpa Library)	140
6.1	Pendahuluan	140
6.2	Capaian Pembelajaran	141
6.3	Dasar Teori (Secukupnya)	141
6.4	Topologi	142
6.5	Alat yang Digunakan	143
6.6	Kode Program	145
6.7	Persiapan dan Menjalankan	152
6.8	Percobaan	152
6.8.1	EXP-01 — Chip Menjawab Namanya	152
6.8.2	EXP-02 — Aliran Data Satu Arah	153
6.8.3	EXP-03 — Uji Silang Platform	153
6.8.4	EXP-04 — Membaca Kembali Apa yang Ditulis	154
6.8.5	Catatan Verifikasi	155
6.9	Pengukuran	155

6.10	Analisis	156
6.11	Concept Check	156
6.12	Challenge (Tugas Modifikasi)	157
6.13	Laporan	157
7	Modul 07 – Gateway Linux Menjadwalkan Node Arduino . . .	158
7.1	Pendahuluan	158
7.2	Capaian Pembelajaran	159
7.3	Dasar Teori (Secukupnya)	159
7.4	Topologi	160
7.5	Alat yang Digunakan	161
7.6	Kode Program	162
7.7	Build, Flash, dan Menjalankan	179
7.8	Percobaan	179
7.8.1	EXP-01 – Siklus Pertama Lintas Platform	179
7.8.2	EXP-02 – Statistik dan Lama Siklus Lintas Mesin	180
7.8.3	EXP-03 – Satu Node Hilang	181
7.8.4	EXP-04 – Menekan Batas Waktu Sampai Rusak	182
7.8.5	Verifikasi Radio (Dijalankan Sebelum EXP-01)	183
7.8.6	Verifikasi Perangkat Keras (Log Referensi)	183
7.8.7	Log Serial Terverifikasi	184
7.9	Pengukuran	192
7.10	Analisis	193
7.11	Concept Check	194
7.12	Challenge (Tugas Modifikasi)	194
7.13	Laporan	195
8	Modul 07B – Varian CRC Payload Aktif	196
8.1	Pendahuluan	196
8.2	Capaian Pembelajaran	196
8.3	Dasar Teori (Secukupnya)	197
8.4	Topologi	197
8.5	Alat yang Digunakan	198
8.6	Kode Program	198
8.7	Build, Flash, dan Menjalankan	215
8.8	Percobaan	216
8.8.1	EXP-01 – Siklus Pertama dengan CRC Aktif	216
8.8.2	EXP-02 – Statistik dan Lama Siklus Lintas Mesin	217
8.8.3	EXP-03 – Satu Node Hilang	218
8.8.4	EXP-04 – Menekan Batas Waktu Sampai Rusak	218
8.8.5	Verifikasi Radio (Dijalankan Sebelum EXP-01)	219
8.8.6	Verifikasi Perangkat Keras (Log Referensi)	219
8.8.7	Log Serial Terverifikasi	221
8.9	Pengukuran	232
8.10	Analisis	233
8.11	Concept Check	233

8.12	Challenge (Tugas Modifikasi)	234
8.13	Laporan	234
9	Modul 08 – ALOHA Tanpa Umpan Balik: Bebas Bicara	235
9.1	Pendahuluan	235
9.2	Capaian Pembelajaran	236
9.3	Dasar Teori (Secukupnya)	236
9.4	Topologi	237
9.5	Alat yang Digunakan	238
9.6	Kode Program	238
9.7	Build dan Flash	252
9.8	Percobaan	253
9.8.1	EXP-01 – Dua Node Mengirim Bebas	253
9.8.2	EXP-02 – Memaksa Tabrakan	253
9.8.3	EXP-03 – Payload Dua Ruangan	254
9.8.4	Verifikasi Perangkat Keras	255
9.8.5	Log Serial Terverifikasi	255
9.9	Pengukuran	256
9.10	Analisis	257
9.11	Concept Check	257
9.12	Challenge (Tugas Modifikasi)	257
9.13	Laporan	258
10	Modul 08B – ALOHA + ACK: Kini Tahu	259
10.1	Pendahuluan	259
10.2	Capaian Pembelajaran	260
10.3	Dasar Teori (Secukupnya)	260
10.4	Topologi	261
10.5	Alat yang Digunakan	261
10.6	Kode Program	262
10.7	Build dan Flash	276
10.8	Percobaan	277
10.8.1	EXP-01 – Siklus ACK Sehat, Dua Node	277
10.8.2	EXP-02 – Dibandingkan dengan M08 (Kegagalan Senyap vs Diketahui)	278
10.8.3	EXP-03 – Node Tanpa Retry Tetap Berjalan	278
10.8.4	Verifikasi Perangkat Keras	279
10.8.5	Log Serial Terverifikasi	279
10.9	Pengukuran	281
10.10	Analisis	282
10.11	Concept Check	282
10.12	Challenge (Tugas Modifikasi)	282
10.13	Laporan	283
11	Modul 08C – Random Backoff dan Retry: Coba Lagi	284
11.1	Pendahuluan	284
11.2	Capaian Pembelajaran	285

11.3	Dasar Teori (Secukupnya)	285
11.4	Topologi	287
11.5	Alat yang Digunakan	287
11.6	Kode Program	288
11.7	Build dan Flash	303
11.8	Percobaan	303
11.8.1	EXP-01 — Retry Sehat, Jarak Dekat	303
11.8.2	EXP-02 — Memaksa Retry: M08B vs M08C pada Interval Sama	304
11.8.3	EXP-03 — Duplicate Betul-Betul Tidak Dihitung Ganda	305
11.8.4	Verifikasi Perangkat Keras	305
11.8.5	Log Serial Terverifikasi	305
11.9	Pengukuran	308
11.10	Analisis	308
11.11	Concept Check	308
11.12	Challenge (Tugas Modifikasi)	309
11.13	Laporan	309
12	Modul 09 — CSMA/CA: Dengar Dulu, Baru Bicara	310
12.1	Pendahuluan	310
12.2	Capaian Pembelajaran	311
12.3	Dasar Teori (Secukupnya)	312
12.4	Topologi	313
12.5	Alat yang Digunakan	313
12.6	Kode Program	315
12.7	Build dan Flash	334
12.8	Percobaan	334
12.8.1	EXP-01 — Kalibrasi Telinga: Berapa Nilai “Sibuk”?	334
12.8.2	EXP-02 — Dua Node Sopan	335
12.8.3	EXP-03 — Memaksa Berebut	336
12.8.4	EXP-04 — Matikan Telinganya	337
12.8.5	EXP-05 — RSSI dibanding CAD	338
12.8.6	Verifikasi Perangkat Keras	339
12.8.7	Log Serial Terverifikasi	339
12.9	Pengukuran	343
12.10	Analisis	344
12.11	Concept Check	344
12.12	Challenge (Tugas Modifikasi)	345
12.13	Laporan	345
13	Modul 10 — Slot Terjadwal: Slotted ALOHA dan TDMA	346
13.1	Pendahuluan	346
13.2	Capaian Pembelajaran	347
13.3	Dasar Teori (Secukupnya)	348
13.4	Topologi	349
13.5	Alat yang Digunakan	349
13.6	Kode Program	350

13.7	Build dan Flash	366
13.8	Percobaan	366
13.8.1	EXP-01 — Mode A (Random Slot) = Slotted ALOHA	366
13.8.2	EXP-02 — Mode B (Assigned Slot) = TDMA	368
13.8.3	EXP-03 — Slot Observed dibanding Assigned	368
13.8.4	Verifikasi Perangkat Keras	369
13.8.5	Log Serial Terverifikasi	369
13.9	Pengukuran	372
13.10	Analisis	373
13.11	Concept Check	374
13.12	Challenge (Tugas Modifikasi)	374
13.13	Laporan	374
14	Modul 11 — LoRaWAN: Protokol Mengambil Alih	376
14.1	Pendahuluan	376
14.2	Capaian Pembelajaran	377
14.3	Dasar Teori (Secukupnya)	377
14.4	Topologi	380
14.5	Alat yang Digunakan	380
14.6	Kode Program	381
14.6.1	Sisi Node (Arduino)	382
14.6.2	Sisi Gateway dan Server (Raspberry Pi)	389
14.7	Perintah Deploy	408
14.8	Percobaan	408
14.8.1	EXP-01 — Register Device: Menyiapkan Jaringan Sebelum Ada Satu Pun Paket	408
14.8.2	EXP-02 — OTAA Join, dan Satu Kesalahan yang Sengaja Dibuat	409
14.8.3	EXP-03 — Uplink Dua Node: Mengenali Asal Data Tanpa Melihat DevEUI	410
14.8.4	EXP-04 — Kanal Tunggal: Harga yang Dibayar	411
14.8.5	Verifikasi Perangkat Keras	412
14.8.6	Log Serial Terverifikasi	412
14.9	Pengukuran	416
14.10	Analisis	416
14.11	Concept Check	417
14.12	Challenge (Tugas Modifikasi)	417
14.13	Laporan	417
14.14	Lampiran A — Docker Seperlunya	418
14.14.1	A.1 Kenapa Harus Ada Docker	418
14.14.2	A.2 Lima Perintah yang Dipakai	418
14.14.3	A.3 Membaca docker compose ps	419
14.14.4	A.4 Perlu sudo atau Tidak	419
14.14.5	A.5 Sesudah Raspberry Pi Dimatikan atau Reboot	420
14.14.6	A.6 Kalau Ada yang Tidak Beres	420
14.14.7	A.7 Tiga Hal yang Sebaiknya Tidak Dilakukan	421
14.15	Lampiran B — Menjalankan Lab untuk Beberapa Kelompok Sekaligus	421
14.15.1	B.1 Satu Server untuk Sekelas	421
14.15.2	B.2 Yang Membedakan Tiap Kelompok	422
14.15.3	B.3 Langkah Tiap Kelompok	422
14.15.4	B.4 Hal-Hal yang Perlu Diantisipasi	423
14.15.5	B.5 Kalau Tetap Ingin Meng-clone Kartu SD	423

14.15.6 B.6 Memindahkan Image Lewat Flashdisk	423
14.15.7 B.7 Yang Sudah Diuji	424
14.16 Lampiran C – Menjalankan Gateway Otomatis Lewat systemd	424
14.16.1 C.1 Memasang	424
14.16.2 C.2 Perintah Harian	425
14.16.3 C.3 Perilaku yang Sudah Diatur	425
14.16.4 C.4 Yang Sudah Diuji	425
14.17 Lampiran D – Menyiapkan Raspberry Pi 5 Lain dari Nol	425
14.17.1 D.1 Yang Perlu Ada Lebih Dulu	426
14.17.2 D.2 Jalur Cepat – Pi Punya Internet	426
14.17.3 D.3 Jalur Tanpa Unduhan Besar – Image dari Arsip	426
14.17.4 D.4 Menjadikan Pi Itu Gateway Kelompok	427
14.17.5 D.5 Yang Sudah Diuji	427
Bagian Pelengkap	428
A Lampiran A – Perkakas Lintas Modul	428
A.1 Mengenali Port Sebelum Mengunggah	428
A.2 Kode Program	428
A.3 Peta Perkakas Tiap Modul	430
A.4 Salinan Daring	430

Daftar Gambar

P.1	Arduino Uno dengan Dragino LoRa Shield v1.2 terpasang	22
P.2	Dragino LoRa GPS HAT v1.4 terpasang di Raspberry Pi	23
6.1	Dragino LoRa GPS HAT v1.4 terpasang di Raspberry Pi	143
6.2	Pemetaan pin Dragino LoRa GPS HAT v1.4	144

Daftar Tabel

P.1	Sebelas bagian baku pada setiap modul praktikum	18
P.2	Kemampuan yang ditambahkan tiap modul pada arc pertama	19
P.3	Kontrak data yang dipertahankan lintas modul	19
P.4	Mission roster seluruh modul praktikum LoRa	20
P.5	Perlakuan terhadap tabrakan sepanjang arc kedua	21
P.6	Spesifikasi Arduino Uno dengan Dragino LoRa Shield v1.2	22
P.7	Pemetaan pin Dragino LoRa Shield v1.2 terhadap Arduino Uno	23
P.8	Spesifikasi Raspberry Pi dengan Dragino LoRa GPS HAT v1.4	23
P.9	Pemetaan pin LoRa GPS HAT v1.4 dan padanannya di shield Arduino	24
P.10	Library Arduino yang dipakai seri ini	24
P.11	Paket Python yang dipakai di sisi Raspberry Pi	25
P.12	Hasil verifikasi kesetaraan board Uno asli dan klon	25
P.13	Jembatan USB dan penamaan port tiap jenis board	25
P.14	Status build dan pemakaian flash tiap environment	26
P.15	Perubahan yang dilakukan terhadap kode asli	27
1.1	Peta modul pada arc pertama seri LoRa	29
1.2	Istilah kerja Modul 01	30
1.3	Peran tiap node Modul 01	31
1.4	Alat dan bahan Modul 01	31
1.5	Pemetaan pin shield Modul 01	31
1.6	Lembar pengamatan EXP-01	41
1.7	Lembar pengamatan EXP-02	41
1.8	Lembar pengamatan EXP-03 — keseragaman parameter radio	42
1.9	Hasil verifikasi perangkat keras Modul 01	43
1.10	Log serial Modul 01: Board & Port	43
1.11	Log serial Modul 01: EXP-01 — Inisialisasi Radio	44
1.12	Log serial Modul 01: EXP-02 — Aliran Data Satu Arah (62 detik)	44
1.13	Log serial Modul 01: EXP-03 — Parameter Harus Seragam	44
1.14	Log serial Modul 01: Riwayat: satu shield sempat gagal init	45
1.15	Log serial Modul 01: Uji kompatibilitas board asli vs klon	45
1.16	Log serial Modul 01: Uji kompatibilitas board asli vs klon	46
1.17	Lembar pengukuran A — jarak terhadap kualitas tautan	46
1.18	Lembar pengukuran B — pengaruh penghalang pada jarak 10 m	46
1.19	Lembar pengukuran C — pengaruh spreading factor	47
2.1	Peta modul pada arc pertama seri LoRa	49
2.2	Istilah kerja Modul 02	50
2.3	Peran tiap node Modul 02	51
2.4	Alat dan bahan Modul 02	51
2.5	Pemetaan pin Modul 02	51
2.6	Lembar pengamatan EXP-01	61
2.7	Lembar pengamatan EXP-02 — LED sebagai instrumen	62
2.8	Lembar pengamatan EXP-03 — bukti non-blocking	62
2.9	Angka rujukan EXP-03 — loss interrupt dibanding polling	63
2.10	Hasil verifikasi perangkat keras Modul 02	63
2.11	Log serial Modul 02: Board & Port	64
2.12	Log serial Modul 02: EXP-01 — Penerimaan Tanpa Polling	64
2.13	Log serial Modul 02: EXP-03 — Bukti Non-Blocking	64
2.14	Lembar pengukuran A — ketahanan terhadap kesibukan loop()	65
2.15	Lembar pengukuran B — jarak terhadap kualitas tautan	66

3.1	Peta modul pada arc pertama seri LoRa	68
3.2	Istilah kerja Modul 03	69
3.3	Peran tiap node Modul 03	70
3.4	Alat dan bahan Modul 03	70
3.5	Lembar pengamatan EXP-01	79
3.6	Lembar pengamatan EXP-02 — waktu pulang-pergi sepuluh siklus	80
3.7	Lembar pengamatan EXP-03 — skenario terputus dan pulih	80
3.8	Lembar pengamatan EXP-03 — data pemulihan	81
3.9	Angka rujukan EXP-03 — deteksi hilangnya lawan bicara	81
3.10	Hasil verifikasi perangkat keras Modul 03	81
3.11	Log serial Modul 03: Board & Port	82
3.12	Log serial Modul 03: EXP-02 — Waktu Pulang-Pergi (30 detik)	82
3.13	Log serial Modul 03: Skenario 1-2: responder hilang lalu kembali	83
3.14	Log serial Modul 03: Skenario 3-4: initiator hilang lalu kembali	83
3.15	Log serial Modul 03: Skenario 3-4: initiator hilang lalu kembali	84
3.16	Lembar pengukuran A — jarak terhadap kelangsungan percakapan	84
3.17	Lembar pengukuran B — tautan asimetris	84
3.18	Lembar pengukuran C — waktu pulang-pergi terhadap spreading factor	85
4.1	Peta modul pada arc pertama seri LoRa	87
4.2	Istilah kerja Modul 04	88
4.3	Peran tiap node Modul 04	89
4.4	Alat dan bahan Modul 04	89
4.5	Lembar pengamatan EXP-01	101
4.6	Lembar pengamatan EXP-02	101
4.7	Perlakuan EXP-03 — pencocokan nomor urut	102
4.8	Lembar pengamatan EXP-03	102
4.9	Hasil verifikasi perangkat keras Modul 04	102
4.10	Log serial Modul 04: Board & Port	103
4.11	Log serial Modul 04: EXP-01 — Siklus ACK yang Sehat (45 detik)	103
4.12	Log serial Modul 04: EXP-02 — Batas Waktu Bekerja	104
4.13	Log serial Modul 04: 03-b — penerima membalas OKE	104
4.14	Lembar pengukuran A — keberhasilan terhadap jarak	105
4.15	Lembar pengukuran B — ketidaksepakatan kedua sisi	105
4.16	Lembar pengukuran C — pengaruh nilai timeout	105
5.1	Peta modul pada arc pertama seri LoRa	108
5.2	Istilah kerja Modul 05	109
5.3	Peran tiap node Modul 05	110
5.4	Alat dan bahan Modul 05	110
5.5	Lembar pengamatan EXP-01	129
5.6	Lembar pengamatan EXP-02	129
5.7	Skenario EXP-03 — satu node hilang	130
5.8	Lembar pengamatan EXP-03 — data	130
5.9	Lembar pengamatan EXP-04 — tabrakan yang disengaja	131
5.10	Hasil verifikasi perangkat keras Modul 05	131
5.11	Log serial Modul 05: Board & Port	132
5.12	Log serial Modul 05: EXP-01 — Slave Menyaring Panggilan	133
5.13	Log serial Modul 05: EXP-02 — Siklus Round-Robin (40 detik)	133
5.14	Log serial Modul 05: EXP-03 — Satu Node Hilang	133
5.15	Log serial Modul 05: EXP-04 — Tabrakan yang Disengaja	134
5.16	Log serial Modul 05: Sesi verifikasi ulang — 21 Agustus 2026	135
5.17	Log serial Modul 05: Verifikasi anomali SNR — sesi lanjutan 21 Agustus 2026	136
5.18	Lembar pengukuran A — keberhasilan per node terhadap jarak	137
5.19	Lembar pengukuran B — skenario asimetris	137
5.20	Lembar pengukuran C — lama siklus terhadap jumlah node hidup	137
5.21	Lembar pengukuran D — pengaruh batas waktu polling	138
6.1	Peta modul pada arc pertama seri LoRa	140

6.2	Istilah kerja Modul 06	141
6.3	Peran tiap node Modul 06	142
6.4	Alat dan bahan Modul 06	143
6.5	Pemetaan pin HAT terhadap Raspberry Pi dan padanannya di shield Arduino	144
6.6	Lembar pengamatan EXP-01	152
6.7	Lembar pengamatan EXP-02	153
6.8	Lembar pengamatan EXP-03 — uji silang platform	154
6.9	Lembar pengamatan EXP-04 — membaca kembali register	154
6.10	Lembar pengukuran A — jarak terhadap kualitas tautan	155
6.11	Lembar pengukuran B — perbandingan platform	156
6.12	Lembar pengukuran C — ketepatan waktu kirim	156
7.1	Peta modul pada arc pertama seri LoRa	158
7.2	Istilah kerja Modul 07	159
7.3	Peran tiap node Modul 07	161
7.4	Alat dan bahan Modul 07	161
7.5	Pemetaan pin HAT sisi master Modul 07	161
7.6	Lembar pengamatan EXP-01	180
7.7	Lembar pengamatan EXP-02	181
7.8	Lembar pengamatan EXP-03	181
7.9	Lembar pengamatan EXP-04 — menekan batas waktu	182
7.10	Hasil verifikasi perangkat keras Modul 07	183
7.11	Log serial Modul 07: Board & Port	184
7.12	Log serial Modul 07: Verifikasi radio sebelum percobaan	184
7.13	Log serial Modul 07: EXP-01 — Siklus Pertama Lintas Platform	185
7.14	Log serial Modul 07: EXP-02 — Statistik dan Lama Siklus	186
7.15	Log serial Modul 07: Lama siklus jauh lebih rapat daripada dugaan README	186
7.16	Log serial Modul 07: Lama siklus jauh lebih rapat daripada dugaan README	187
7.17	Log serial Modul 07: EXP-03 — Satu Node Hilang	187
7.18	Log serial Modul 07: EXP-04 — Menekan Batas Waktu Sampai Rusak	188
7.19	Log serial Modul 07: EXP-04 — Menekan Batas Waktu Sampai Rusak	189
7.20	Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati	189
7.21	Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati	190
7.22	Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati	190
7.23	Log serial Modul 07: Verifikasi ulang — 21 Agustus 2026	191
7.24	Lembar pengukuran A — keberhasilan terhadap jarak	192
7.25	Lembar pengukuran B — perbandingan platform master	193
7.26	Lembar pengukuran C — ambang POLL_TIMEOUT	193
7.27	Lembar pengukuran D — rasio payload rusak lolos	193
8.1	Kedudukan Modul 07B terhadap modul sebelumnya	196
8.2	Istilah yang berubah pada Modul 07B	197
8.3	Lembar pengamatan EXP-01	216
8.4	Lembar pengamatan EXP-02	217
8.5	Lembar pengamatan EXP-03	218
8.6	Lembar pengamatan EXP-04 — menekan batas waktu dengan CRC aktif	219
8.7	Hasil verifikasi perangkat keras Modul 07B	220
8.8	Log serial Modul 07B: Board & Port	221
8.9	Log serial Modul 07B: Verifikasi radio sebelum percobaan	221
8.10	Log serial Modul 07B: EXP-01 — Siklus Pertama Lintas Platform	222
8.11	Log serial Modul 07B: EXP-02 — Statistik dan Lama Siklus	223
8.12	Log serial Modul 07B: Lama siklus jauh lebih rapat daripada dugaan README	223
8.13	Log serial Modul 07B: Lama siklus jauh lebih rapat daripada dugaan README	224
8.14	Log serial Modul 07B: EXP-03 — Satu Node Hilang	224
8.15	Log serial Modul 07B: EXP-04 — Menekan Batas Waktu Sampai Rusak	225
8.16	Log serial Modul 07B: EXP-04 — Menekan Batas Waktu Sampai Rusak	226
8.17	Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati	226
8.18	Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati	227

8.19	Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati	227
8.20	Log serial Modul 07B: Verifikasi ulang — 21 Agustus 2026	228
8.21	Log serial Modul 07B: Modifikasi CRC — 21 Agustus 2026	231
8.22	Lembar pengukuran A — keberhasilan terhadap jarak	232
8.23	Lembar pengukuran B — perbandingan platform master dan biaya CRC	232
8.24	Lembar pengukuran C — ambang POLL_TIMEOUT dengan CRC aktif	232
8.25	Lembar pengukuran D — rasio payload rusak yang lolos	233
9.1	Peta modul pada arc kedua seri LoRa	235
9.2	Istilah kerja Modul 08	236
9.3	Peran tiap node Modul 08	238
9.4	Alat dan bahan Modul 08	238
9.5	Hasil pengamatan EXP-01 pada sesi verifikasi perangkat	253
9.6	Lembar pengamatan EXP-02 — memaksa tabrakan	254
9.7	Hasil pengamatan EXP-03 — payload dua ruangan	254
9.8	Log serial Modul 08: Board & Port	255
9.9	Log serial Modul 08: EXP-01 — Dua Node Mengirim Bebas	256
9.10	Log serial Modul 08: EXP-03 — Payload Dua Ruangan	256
9.11	Lembar pengukuran A — kedatangan paket terhadap kepadatan kirim	257
9.12	Lembar pengukuran B — RSSI dan SNR per node (sesi verifikasi)	257
10.1	Peta modul pada arc kedua seri LoRa	259
10.2	Istilah kerja Modul 08B	260
10.3	Peran tiap node Modul 08B	261
10.4	Alat dan bahan Modul 08B	261
10.5	Hasil pengamatan EXP-01 pada sesi verifikasi perangkat	277
10.6	Lembar pengamatan EXP-02 — M08 dibanding M08B	278
10.7	Lembar pengamatan EXP-03	278
10.8	Log serial Modul 08B: Board & Port	279
10.9	Log serial Modul 08B: EXP-01 — Siklus ACK Sehat, Dua Node	279
10.10	Log serial Modul 08B: Temuan Tambahan A — Tabrakan Nyata Tertangkap Langsung.	280
10.11	Log serial Modul 08B: Temuan Tambahan B — Ketidaksepakatan Node vs Gateway (DATA sampai, ACK hilang)	280
10.12	Lembar pengukuran A — tingkat keberhasilan per node (sesi verifikasi)	281
10.13	Lembar pengukuran B — ketidaksepakatan node dan gateway	281
10.14	Lembar pengukuran C — M08 dibanding M08B	282
11.1	Peta modul pada arc kedua seri LoRa	284
11.2	Istilah kerja Modul 08C	285
11.3	Peran tiap node Modul 08C	287
11.4	Alat dan bahan Modul 08C	287
11.5	Hasil pengamatan EXP-01 pada sesi verifikasi perangkat	304
11.6	Lembar pengamatan EXP-02 — M08B dibanding M08C	304
11.7	Lembar pengamatan EXP-03 — deteksi duplicate	305
11.8	Log serial Modul 08C: Board & Port	306
11.9	Log serial Modul 08C: EXP-01 — Retry Sehat, Jarak Dekat	306
11.10	Log serial Modul 08C: Temuan Tambahan — Perbandingan Langsung dengan M08B (Tanpa vs Dengan Retry)	306
11.11	Log serial Modul 08C: Temuan Tambahan — Duplicate Tidak Dihitung Ganda	307
11.12	Lembar pengukuran A — distribusi retry per siklus sukses	308
11.13	Lembar pengukuran B — M08B dibanding M08C	308
11.14	Lembar pengukuran C — duplicate dibanding data baru	308
12.1	Peta modul pada arc kedua seri LoRa	311
12.2	Istilah kerja Modul 09	312
12.3	Peran tiap node Modul 09	313
12.4	Alat dan bahan Modul 09	313
12.5	Pemetaan pin Modul 09	314
12.6	Parameter CSMA/CA Modul 09	314

12.7	Lembar pengamatan EXP-01 — kalibrasi ambang carrier sense	334
12.8	Lembar pengamatan EXP-02 — dua node sopan	336
12.9	Lembar pengamatan EXP-03 — memaksa berebut kanal	336
12.10	Hasil EXP-04 — CSMA/CA dibanding kirim buta	337
12.11	Hasil EXP-05 — carrier sense berbasis RSSI dibanding CAD	338
12.12	Log serial Modul 09: Board & Port	339
12.13	Log serial Modul 09: Ringkasan keempat sesi	339
12.14	Log serial Modul 09: Statistik akses kanal per node	340
12.15	Log serial Modul 09: RSSI & SNR di gateway	340
12.16	Lembar pengukuran A — beban kanal terhadap perilaku CSMA/CA	343
12.17	Lembar pengukuran B — RSSI, SNR, dan tunda akses per node	343
12.18	Lembar pengukuran C — CSMA/CA dibanding kirim buta	344
13.1	Peta modul pada arc kedua seri LoRa	347
13.2	Istilah kerja Modul 10	348
13.3	Peran tiap node Modul 10	349
13.4	Alat dan bahan Modul 10	349
13.5	Hasil pengamatan EXP-01 — Mode A (Slotted ALOHA)	367
13.6	Hubungan tabrakan slot dengan kegagalan pada Mode A	367
13.7	Hasil pengamatan EXP-02 — Mode B (TDMA)	368
13.8	Hasil pengamatan EXP-03 — slot teramati dibanding slot assignment	368
13.9	Log serial Modul 10: Board & Port	369
13.10	Log serial Modul 10: EXP-01 — Mode A (Random Slot) = Slotted ALOHA	370
13.11	Log serial Modul 10: EXP-01 — Mode A (Random Slot) = Slotted ALOHA	370
13.12	Log serial Modul 10: EXP-02 — Mode B (Assigned Slot) = TDMA	371
13.13	Log serial Modul 10: Perbandingan langsung	371
13.14	Log serial Modul 10: Tabel D — sapuan SLOT_COUNT (Mode A)	371
13.15	Log serial Modul 10: Tabel D — sapuan SLOT_COUNT (Mode A)	372
13.16	Lembar pengukuran A — Slotted ALOHA dibanding TDMA	372
13.17	Lembar pengukuran B — distribusi slot pada Mode A	372
13.18	Lembar pengukuran C — tabrakan dan akibatnya pada Mode A	372
13.19	Lembar pengukuran D — pengaruh jumlah slot pada Mode A	373
14.1	Kedudukan Modul 11 terhadap modul sebelumnya	376
14.2	Rentang data dummy tiap node Modul 11	377
14.3	Istilah kerja Modul 11	377
14.4	Lapisan buatan tangan M04–M10 dan padanannya di LoRaWAN	378
14.5	Peran tiap perangkat Modul 11	380
14.6	Alat dan bahan Modul 11	380
14.7	Pemetaan pin sisi node Modul 11	381
14.8	Pemetaan pin sisi gateway Modul 11	381
14.9	Hasil pengamatan EXP-01 — pendaftaran perangkat	408
14.10	Hasil pengamatan EXP-02 — OTAA join	409
14.11	Hasil pengamatan EXP-03 — uplink dua node	410
14.12	Hasil pengamatan EXP-04 — harga gateway kanal tunggal	411
14.13	Log serial Modul 11: Board & Port	412
14.14	Log serial Modul 11: EXP-03 — Uplink dua node, 5 menit	414
14.15	Log serial Modul 11: EXP-03 — Uplink dua node, 5 menit	414
14.16	Log serial Modul 11: Uji reboot (Lampiran A.5 dan C.4)	416
14.17	Lembar pengukuran A — join OTAA	416
14.18	Lembar pengukuran B — uplink selama 5 menit	416
14.19	Lembar pengukuran C — rentang data dummy	416
14.20	Komponen ChirpStack v4 dan tugasnya	418
14.21	Istilah Docker dan padanannya	418
14.22	Lima perintah Docker yang dipakai modul ini	419
14.23	Port yang dipakai tiap komponen	419
14.24	Perilaku sistem sesudah Raspberry Pi dimatikan	420
14.25	Gejala umum dan cara menanganinya	420

14.26	Server bersama dibanding server per kelompok	422
14.27	Pembagian kanal dan kunci tiap kelompok	422
14.28	Dua berkas yang dipasang oleh opsi --service	424
14.29	Perintah systemd yang dipakai sehari-hari	425
A.1	Perkakas yang tersedia pada tiap modul	430

Pendahuluan

Tentang Lab Ini

Buku ini bukan kumpulan tutorial Arduino, melainkan buku kerja laboratorium. Setiap modul merupakan satu misi rekayasa dengan target sukses yang terukur, prosedur eksperimen yang tertulis, dan data yang harus dikumpulkan sendiri oleh praktikan.

Fokusnya adalah **LoRa mentah** — modulasi radio tanpa lapisan jaringan di atasnya. Tidak ada proses join, tidak ada alamat, tidak ada koneksi, dan tidak ada penjadwalan. Semua yang biasanya disediakan protokol harus dibangun sendiri di lapisan aplikasi, satu per satu, dan setiap penambahan diukur akibatnya. Justru di situlah nilai seri ini: praktikan menyaksikan sendiri masalah yang selama ini disembunyikan protokol, sebelum memakai protokol yang menyembunyikannya.

Lima modul pertama dikerjakan di atas Arduino Uno bershield Dragino LoRa v1.2. Dua modul berikutnya berpindah ke Raspberry Pi bershield Dragino LoRa GPS HAT v1.4 — bukan sekadar berganti papan, melainkan dua pertanyaan lanjutan. Modul 06 melepas library dan memegang register SX1276 langsung dari Python, sehingga isi `LoRa.begin()` terlihat baris per baris. Modul 07 memindahkan master jaringan Modul 05 ke Raspberry Pi tanpa mengubah firmware slave sama sekali, membentuk topologi gateway yang lazim di dunia nyata. Chip radionya tetap sama di seluruh seri, dan kesamaan itulah yang membuat kedua sisi dapat dipasangkan silang.

Seri ini melengkapi lab WSN-IOT-prak yang membahas BLE, Zigbee, dan Thread. Perbandingannya disengaja: di sana jaringan mengurus dirinya sendiri, di sini tidak ada jaringan sama sekali.

Struktur Setiap Modul

Seluruh modul memakai format yang sama, terdiri atas sebelas bagian seperti pada Tabel P.1.

No	Bagian	Isi
1	Pendahuluan	Identitas modul, keterkaitan dengan modul lain, prasyarat, dan apa yang dipakai lagi sesudahnya — seluruhnya dalam bentuk kalimat, ditutup peta modul dan kontrak data
2	Capaian Pembelajaran	Lima capaian terukur beserta kriteria keberhasilan
3	Dasar Teori (secukupnya)	Hanya istilah yang dipakai di percobaan, beserta sekuens yang diamati
4	Topologi	Diagram bernama board, peran tiap node
5	Alat yang Digunakan	Platform, alat dan bahan, pemetaan pin, <code>platformio.ini</code> , pre-flight, perintah deploy
6	Percobaan	EXP-01 sampai EXP-04 dengan CHECKPOINT di tiap tahap
7	Pengukuran	Tabel yang diisi sendiri oleh praktikan
8	Analisis	Pertanyaan yang hanya bisa dijawab dari tabel Pengukuran
9	Concept Check	Pertanyaan konseptual, bukan hafalan
10	Challenge	Tugas modifikasi kode , bukan “jelaskan hasilnya”
11	Laporan	Daftar deliverable

Tabel P.1: Sebelas bagian baku pada setiap modul praktikum

Tiga hal membedakan format ini dari panduan praktikum biasa.

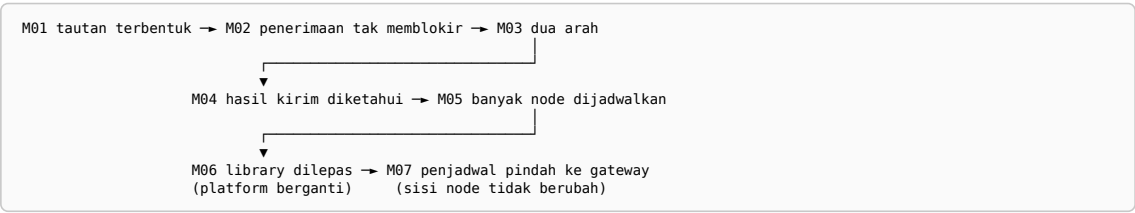
CHECKPOINT di tengah percobaan Praktikan memverifikasi progres sebelum lanjut, bukan baru ketahuan salah di akhir sesi.

“Buka abstraksinya” Satu kotak per modul yang menyuruh praktikan membongkar satu baris kode yang tampak sepele – menghubungkan API dengan apa yang sebenarnya terjadi di udara.

Percobaan yang sengaja dirusak Beberapa modul meminta parameter diubah sampai komunikasi gagal (Modul 01 EXP-03, Modul 05 EXP-04), karena kegagalan yang terkendali mengajarkan lebih banyak daripada keberhasilan yang mulus.

Keterkaitan Antar-Modul

Tiap modul menambahkan **satu** kemampuan yang hilang dari modul sebelumnya.



Modul	Yang hilang di modul sebelumnya	Yang ditambahkan
01	—	Tautan radio, parameter, RSSI dan SNR
02	Penerima sibuk menunggu, paket terlewat	Interrupt DIO0 + flag pattern
03	Data hanya mengalir satu arah	Percakapan bergantian + auto-retry
04	Nasib paket tidak pernah diketahui	ACK, timeout, statistik keberhasilan
05	Dua node tidak pernah berebut bicara	Pengalamatan aplikasi + penjadwalan
06	Isi LoRa.begin() tidak pernah terlihat	Register SX1276 langsung, Python di Linux, uji silang platform
07	Master mikrokontroler buntu di Serial Monitor	Gateway Linux menjadwalkan node Arduino tanpa mengubah firmware-nya

Tabel P.2: Kemampuan yang ditambahkan tiap modul pada arc pertama

Kontrak data yang konsisten. Beberapa keputusan sengaja dipertahankan lintas modul supaya datanya dapat dibandingkan, seperti tercantum pada Tabel P.3.

Kontrak	Diperkenalkan	Dipakai lagi di
Nomor urut di dalam payload untuk menghitung loss	M01 (Hello LoRa #n)	M03, M04 (DATA:n), M05 (S1:DATA:n)
Parameter radio baku SF7 / BW 125 kHz / CR 4/5 / 17 dBm	M01	M02–M05
Identitas pengirim di dalam payload	M03 (DeviceA:)	M05 (S1:, S2:)
Pencocokan permintaan dengan balasan	M04 (DATA:n ↔ ACK:n)	M05, M07 (POLL:n ↔ S<n>:DATA:m)
RSSI dan SNR dicatat berpasangan	M01	seluruh modul
Payload dua ruangan NODE=, SEQ=, R1T=, R1H=, R2T=, R2H=	M08	M08B, M08C, M10
Balasan ACK=<id>, SEQ=<n>	M08B	M08C, M10

Tabel P.3: Kontrak data yang dipertahankan lintas modul

Satu pengecualian yang disengaja: M09. Modul CSMA/CA memakai payload yang lebih pendek (NODE=, SEQ=, T=, H= — satu sensor per node) dan **tanpa ACK sama sekali**, meski M08B dan M08C sudah memilikinya. Itu bukan kelalaian: M09 mengukur satu variabel saja, yaitu efek mendengarkan kanal sebelum bicara, dan ACK atau retry akan menutupi kegagalan yang justru sedang diukur. Pembandingnya karena itu M08, bukan M08C — dan M09 menyediakan pembanding itu di dalam dirinya sendiri lewat CS_MODE=2 (carrier sense dimatikan).

Konsekuensinya, **angka pengukuran modul awal dipakai lagi di modul akhir.** Loss terhadap jarak dari M01 menjadi pembanding tingkat keberhasilan ACK di M04; waktu pulang-pergi M03 menjadi dasar penentuan batas waktu di M04 dan M05.

Kontrak yang sama itu pula yang membuat dua modul terakhir arc pertama dapat disambungkan silang ke modul awal. Payload M06 identik dengan M01 (Hello LoRa #n, SF7, BW 125 kHz), sehingga **sender Raspberry Pi dapat diuji langsung terhadap receiver Arduino M01** dan sebaliknya — percobaan inti M06. Firmware slave M07 pun identik dengan slave M05 kecuali satu baris pesan pembuka, dan kesamaan itu diperiksa dengan diff sebagai bagian dari laporan, bukan sekadar dinyatakan.

Daftar Modul

Modul	Folder	Misi	Mekanisme RX	Level
01	Modul01_lora_uart	Establish the Link	polling	Basic
02	Modul02_lora_led_notif	Stop arah Waiting for Packets	interrupt + flag	Basic
03	Modul03_lora_p2p	Take arah Turns Talking	polling	Intermediate
04	Modul04_lora_ack	Know arah If It Arrived	interrupt + timeout	Intermediate
05	Modul05_lora_master_slave	Simulate 3 node the Airwaves	polling terjadwal	Advanced
06	Modul06_rpi_lora_python	Enter arah the Library	polling register	Intermediate
07	Modul07_rpi_master_slave	Monitor 3 node the Scheduler to Linux	polling terjadwal	Advanced
07B	Modul07b_rpi_master_slave_crc	Monitor 3 node the Scheduler to Linux — varian CRC	polling terjadwal + CRC payload	Advanced
08	Modul08_lora_aloha_tanpa_ack	Speak 2 node Freely,	interrupt, tanpa balasan	Intermediate

Modul	Folder	Mission	Mekanisme RX	Level
		Collide Silently		
08B	Modul08b_lora_aloha_ack	Binary, 2 node Know	interrupt + timeout	Intermediate
08C	Modul08c_lora_aloha_retry	Binary, 2 node Again, Randomly	interrupt + timeout + retry	Advanced
09	Modul09_lora_csma_ca	Binary, 2 node Before You Talk	carrier sense + backoff	Advanced
10	Modul10_lora_slotted_aloha_tdma	Binary, 2 node a Number	interrupt + SYNC + slot	Advanced
11	Modul11_lorawan_chirpstack	Binary, 2 node + server Protocol Take Over	LoRaWAN Class A (LMIC)	Advanced

Tabel P.4: Mission roster seluruh modul praktikum LoRa

Modul 01–05 memakai Arduino Uno dengan Dragino LoRa Shield v1.2. Modul 06 memakai dua Raspberry Pi dengan LoRa GPS HAT v1.4. Modul 07 mencampur keduanya: Raspberry Pi sebagai master, dua Arduino Uno sebagai slave.

Cara membaca penomoran. Angka adalah nomor modul; huruf di belakangnya berarti modul itu **menumpuk langsung** di atas modul bernomor sama, bukan pertemuan yang berdiri sendiri. M08 → M08B → M08C adalah tiga langkah berturut-turut di atas topologi dan kontrak data yang sama, sedangkan M07B adalah varian M07 dengan CRC payload diaktifkan.

Arc kedua – akses kanal tanpa penjadwal terpusat. Modul 01–07 adalah satu seri utuh yang berakhir di M07. Modul 08 dan seterusnya membuka arc baru di atas Arduino Uno dengan Dragino LoRa Shield v1.2 (topologi sama seperti M05, dua node dan satu gateway), yang sengaja **membalik** premis M05 dan M07: alih-alih menjadwalkan giliran bicara dari pusat, node dibiarkan mengirim data dummy suhu dan kelembaban dua ruangan kapan saja, dan setiap pertemuan menambah **satu** lapisan kendali kanal. M08 mengirim bebas tanpa umpan balik sama sekali, M08B menambahkan ACK, M08C menambahkan random backoff dan retry sekaligus membuat gateway mengenali paket duplicate lewat SEQ (barulah di sini ALOHA lengkap seperti yang dirumuskan Abramson), M09 menerapkan carrier sense, lalu M10 menutup arc ini dengan SYNC dan slot waktu, dengan retry dihapus dan digantikan penjadwalan. Urutannya mengikuti sejarah protokol akses kanal: **ALOHA → CSMA → slot terjadwal**, dan yang bertambah tiap pertemuan adalah cara memperlakukan tabrakan, seperti pada Tabel P.5.

Modul	Tabrakan diperlakukan bagaimana
M08	dibiarkan, bahkan tidak terlihat
M08B	dideteksi – ACK membuat kegagalan terbaca
M08C	dipulihkan – retry dengan jeda acak
M09	dihindari sebelum terjadi – dengar dulu sebelum bicara
M10	dicegah secara struktural – tiap node punya jatah waktunya

Tabel P.5: Perlakuan terhadap tabrakan sepanjang arc kedua

Dua mode dibandingkan langsung pada M10, dan keduanya protokol yang berbeda: Random Slot adalah **Slotted ALOHA** yang sesungguhnya (tabrakan berkurang tetapi belum hilang, throughput puncak $\approx 36,8$

%), sedangkan Assigned Slot sudah menjadi **TDMA** — tiap node punya slot tetap dan tabrakan hilang secara struktural.

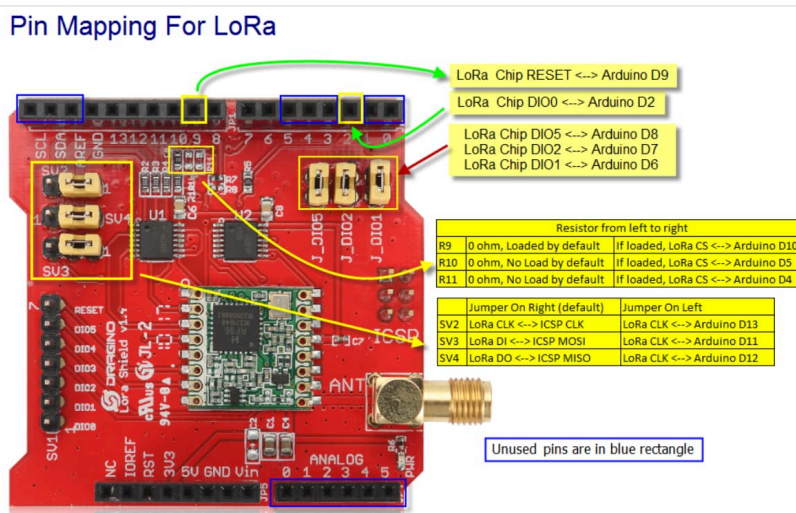
Arc ketiga — protokol mengambil alih. Modul 11 menutup pertanyaan yang menggantung sejak M04: semua yang dibangun sendiri di lapisan aplikasi — alamat node, ACK, nomor urut, retry, giliran bicara — sudah lama distandarkan orang lain, dan namanya LoRaWAN. Di modul ini kedua Arduino Uno menjalankan stack LoRaWAN Class A (MCCI LMIC) dan join lewat OTAA ke ChirpStack v4 yang berjalan di Raspberry Pi 5, dengan Pi yang sama merangkap **single channel gateway** memakai driver register lanjutan M06. Payload-nya sengaja dibiarkan sesederhana mungkin ($T=27.4$, $H=68$, ASCII, tanpa encoding) supaya perhatian tertuju pada alur Register Device → OTAA Join → Uplink → data terlihat di ChirpStack, bukan pada pengemasan byte. Dua hal baru yang tidak pernah muncul di sepuluh modul sebelumnya: payload terenkripsi sehingga **gateway sendiri tidak bisa membacanya**, dan alamat perangkat yang **diberikan server**, bukan ditentukan build flag.

Perangkat Keras

Dua papan Dragino dipakai di seri ini. Keduanya membawa chip radio yang **sama**, SX1276; yang berbeda hanya papan pembawa dan pin mana yang tersambung ke mana. Perbedaan itu tidak terasa sama sekali di udara — itulah yang dibuktikan M06 dan M07.

Arduino Uno + Dragino LoRa Shield v1.2

Dipakai pada Modul 01–05, sisi slave M07 dan M07B, seluruh arc kedua (M08 sampai M10), serta sisi node M11.



Gambar P.1: Arduino Uno dengan Dragino LoRa Shield v1.2 terpasang

Parameter	Nilai
Board	Arduino Uno (ATmega328P), flash 32 KB, RAM 2 KB
Shield	Dragino LoRa Shield v1.2
Chip radio	Semtech SX1276
Frekuensi	433 MHz — shield yang dipakai lab ini; varian 868 / 915 / 920 MHz juga beredar
Daya pancar	maksimum +20 dBm; program memakai 17 dBm
Sensitivitas	hingga - 148 dBm
Antarmuka	SPI perangkat keras + 3 pin kendali (NSS, RST, DIO0)
Antena	konektor SMA eksternal — wajib terpasang

Tabel P.6: Spesifikasi Arduino Uno dengan Dragino LoRa Shield v1.2

PERINGATAN

Jangan menyalakan shield tanpa antena. Daya pancar yang tidak menemukan beban dipantulkan kembali ke penguat SX1276 dan dapat merusaknya permanen.

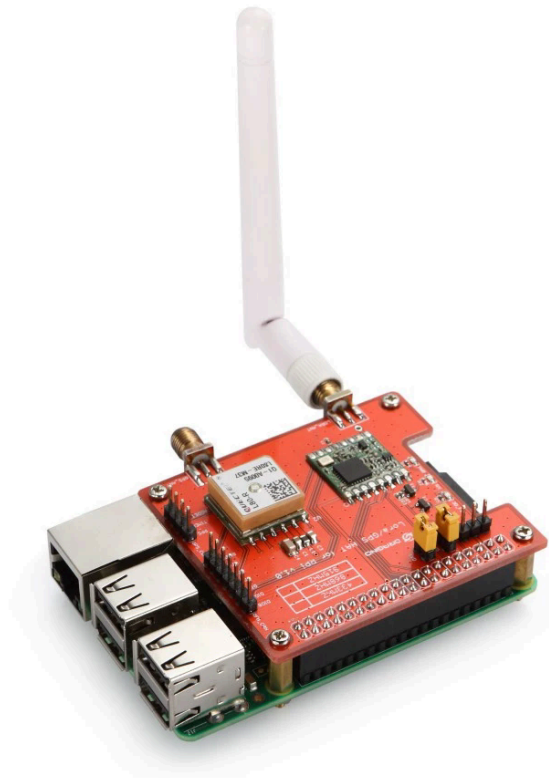
Pin	Fungsi	Boleh dipakai program?
D10	NSS / CS (jumper R9)	Tidak
D11, D12, D13	MOSI, MISO, SCK	Tidak — D13 juga LED bawaan
D9	RST SX1276	Tidak
D2	DIO0 (interrupt)	Tidak
D6, D7, D8	DIO1, DIO2, DIO5	Jangan dijadikan output
D3, D4, D5, A0–A5	bebas	Ya — D3 direkomendasikan untuk LED indikator

Tabel P.7: Pemetaan pin Dragino LoRa Shield v1.2 terhadap Arduino Uno

Skematik resmi shield ada pada folder skematik/ di repositori praktikum.

Raspberry Pi + Dragino LoRa GPS HAT v1.4

Dipakai pada Modul 06, sisi master M07 dan M07B, serta gateway dan server ChirpStack M11.



Gambar P.2: Dragino LoRa GPS HAT v1.4 terpasang di Raspberry Pi

Parameter	Nilai
Board	Raspberry Pi 2 / 3 / 4 — diuji pada Pi 4 oleh penyusun kode aslinya
HAT	Dragino LoRa GPS HAT v1.4

Parameter	Nilai
Chip radio	Semtech SX1276 — sama persis dengan shield Arduino
GPS	modul onboard, UART 9600 bps, NMEA 0183 — tidak dipakai di seri ini
Antarmuka	SPI0 perangkat keras + 3 jalur GPIO (NSS, RESET, DIO0)
Antena	konektor SMA eksternal — wajib terpasang , sama seperti shield

Tabel P.8: Spesifikasi Raspberry Pi dengan Dragino LoRa GPS HAT v1.4

LoRa GPS HAT	WiringPi	BCM GPIO	Padanan di shield Arduino
LoRa_NSS	GPIO6	25	D10
RESET	GPIO0	17	D9
DIO0	GPIO7	4	D2
SCK / MOSI / MISO	GPIO14/12/13	11 / 10 / 9	D13 / D11 / D12
GPS_RX / GPS_TX / 1PPS	GPIO15/16/1	14 / 15 / 18	—

Tabel P.9: Pemetaan pin LoRa GPS HAT v1.4 dan padanannya di shield Arduino

Kolom **WiringPi** adalah penomoran yang dipakai dokumentasi resmi Dragino; kolom **BCM** adalah yang dipakai seluruh kode Python di seri ini (`GPIO.setmode(GPIO.BCM)`). Keduanya menunjuk pin fisik yang sama, dan tertukarnya keduanya adalah penyebab kegagalan yang paling sering terjadi.

PENTING

NSS bukan CE0. HAT memakai GPIO 25 biasa sebagai chip select, bukan jalur CE0 bawaan SPI. Akibatnya kode Python membuka SPI pada (0, 0) tetapi menggerakkan NSS sendiri di sekitar tiap transaksi — persis seperti driver Arduino menggerakkan D10.

CATATAN

Raspberry Pi 5 memakai chip GPIO baru (RP1) yang tidak didukung `RPi.GPIO`. Pasang `rpi-lgpio` sebagai gantinya; nama modulnya sama, sehingga tidak ada baris kode yang perlu diubah. Jangan memasang keduanya sekaligus.

Skematik HAT dan user manual resminya ada pada folder `skematik/` dan `dokumen/` di repositori praktikum.

Perangkat Lunak

Library	Versi	Fungsi
LoRa (sandeepmistry)	0.8.x	Driver SX1276: init, TX blocking, RX polling/interrupt, RSSI, SNR
SPI	bawaan framework	Komunikasi SPI ke SX1276

Tabel P.10: Library Arduino yang dipakai seri ini

PlatformIO mengunduh keduanya otomatis lewat `lib_deps`; tidak ada pemasangan manual.

CATATAN

Mengapa bukan RadioLib? RadioLib terkompilasi menjadi lebih dari 33 KB pada Arduino Uno — melampaui flash 32 KB yang tersedia. Library `sandeepmistry` hanya memakai 18–30 % flash pada seluruh modul seri ini, sebagaimana terlihat pada Tabel P.14.

Sisi Raspberry Pi (M06 dan master M07) tidak memakai library LoRa sama sekali.

Paket	Fungsi
spidev	Akses /dev/spidev0.0 — satu-satunya jalan bicara ke SX1276
RPi.GPIO	Kendali jalur NSS dan RESET, pembacaan DIO0
rpi-lgpio	Pengganti RPi.GPIO khusus Raspberry Pi 5 — nama modul sama, kode tidak berubah

Tabel P.11: Paket Python yang dipakai di sisi Raspberry Pi

Driver SX1276-nya ditulis di dalam berkas program itu sendiri, langsung di atas register. Nama fungsinya sengaja dibuat menyerupai API sandeepmistry (beginPacket, endPacket, parsePacket, packetRssi) agar kedua platform dapat dibandingkan baris per baris — dan agar terlihat bahwa library Arduino tidak melakukan apa pun yang ajaib, hanya menulis register yang sama. Pemasangannya lewat requirements.txt di masing-masing folder modul.

Board Bercampur: Uno Asli dan Klon

Catatan ini berlaku untuk seluruh Arduino di lab. Lab ini memakai Arduino Uno asli maupun klon secara bercampur. **Firmware kedua jenis board identik** — tidak ada satu baris pun yang perlu diubah, dan tidak ada environment terpisah. Hal itu sudah diverifikasi di perangkat, bukan sekadar diasumsikan.

Yang diperiksa	Hasil
Mikrokontroler	Device signature = 0x1e950f (m328p) — sama pada kedua jenis
Protokol unggah	arduino pada kedua jenis
Berkas .hex untuk upload_port berbeda	md5 identik — port bukan masukan kompilasi
Modul 01 dengan peran ditukar antar-jenis	berjalan normal di kedua arah, RSSI – 54,0 vs – 53,8 dBm
Modul 05 dengan master asli + satu slave klon	53 siklus, keberhasilan 100 % di kedua slave

Tabel P.12: Hasil verifikasi kesetaraan board Uno asli dan klon

Yang berbeda hanya **chip jembatan USB-ke-serial** di atas board, dan itu hanya mengubah nama port di sistem operasi.

Jenis board	Jembatan USB	Nama port di Linux
Uno asli	ATmega16U2 (2341:0043)	/dev/ttyACM*
Klon	CH340 (1a86:7523), CH343 (1a86:55d3), FTDI, CP2102	/dev/ttyUSB*

Tabel P.13: Jembatan USB dan penamaan port tiap jenis board

Di Windows keduanya sama-sama muncul sebagai COMx, sehingga perbedaan ini tidak terasa sama sekali. **Kenali port sebelum mengunggah.**

```
python3 tools/deteksi_port.py          # daftar port + jenis board
python3 tools/deteksi_port.py --ini    # potongan platformio.ini siap tempel
```

Port	VID:PID	Jenis	Jembatan USB
/dev/ttyACM0	2341:0043	Uno asli	ATmega16U2 (Arduino LLC)
/dev/ttyACM1	2341:0043	Uno asli	ATmega16U2 (Arduino LLC)
/dev/ttyUSB0	1a86:7523	klon	CH340/CH341

PERINGATAN

Satu hal yang benar-benar berbeda perilakunya, dan hanya di sisi perkakas: skrip Python yang menyatel jalur DTR/RTS sebelum `open()` ditolak oleh CDC ATmega16U2 pada Uno asli dengan [Errno 110] Connection timed out, sementara pada klon CH340 hal itu lolos. Seluruh `monitor_serial.py` pada seri ini sudah tidak menyentuh jalur tersebut. Bila menulis skrip serial sendiri, buka port apa adanya — jangan mengatur DTR/RTS sebelum membukanya.

Menjalankan

Modul 01–05, sisi slave M07, dan seluruh arc kedua — Arduino, lewat PlatformIO:

```
pio device list                # catat port tiap board
pio run -d Modul01_lora_uart -e receiver -t upload  # penerima dahulu
pio run -d Modul01_lora_uart -e sender -t upload -t monitor
```

Modul 06 dan sisi master Modul 07 — Raspberry Pi, langsung dengan Python:

```
sudo raspi-config              # Interface Options > SPI > Yes, lalu reboot
ls /dev/spi*                  # harus muncul spidev0.0

pip3 install -r Modul06_rpi_lora_python/requirements.txt
python3 Modul06_rpi_lora_python/src/receiver.py    # penerima dahulu
python3 Modul06_rpi_lora_python/src/sender.py      # di Pi kedua
```

Tidak ada yang dikompilasi di sisi Raspberry Pi, sehingga tidak ada `platformio.ini` untuknya. Modul07_rpi_master_slave/platformio.ini hanya memuat kedua environment slave; masternya dijalankan sebagai `python3 src/master.py`.

Port di tiap `platformio.ini` masih memakai nilai contoh untuk tiga Uno asli. Jalankan `tools/deteksi_port.py` lebih dahulu, lalu sesuaikan `upload_port` dan `monitor_port` sesuai board yang benar-benar terpasang.

PENTING

Urutan unggah penting di sebagian besar modul: pihak yang **menunggu** diunggah lebih dahulu, pihak yang **memulai** belakangan. Tiap modul menyebutkan urutannya.

Baud Serial Monitor: 9600 untuk M01–M04, **115200 untuk M05 dan slave M07**. Modul 06 tidak memakai Serial Monitor sama sekali — keluarannya langsung ke terminal Raspberry Pi.

Status Verifikasi

Seluruh modul Arduino dikompilasi ulang setelah dikonversi ke PlatformIO. Modul 05 dan Modul 07 sudah diuji langsung di perangkat keras (tiga Arduino Uno bershield Dragino, satu Raspberry Pi 5 bershield LoRa GPS HAT) — rincian sesi dan angka terukurnya ada pada `logserial.md` masing-masing folder. Modul 01–04 dan Modul 06 belum diuji ulang pada konversi ini; perilaku yang dijelaskan berasal dari kode sumber asli beserta dokumentasinya, bukan dari pengamatan ulang. Modul 08, 08B, 09, dan 10 baru dikompilasi (`pio run`), **belum diuji di perangkat keras sama sekali** — belum ada log serial, belum ada RSSI atau SNR nyata. Angka pada tabel pengukuran modul yang belum diuji tetap disediakan kosong untuk diisi praktikan.

Modul	Environment	Build	Flash (dari 32.256 B)
01	sender	OK	18,3 % (5.890 B)
01	receiver	OK	22,7 % (7.312 B)
02	sender	OK	22,9 % (7.380 B)
02	receiver	OK	24,5 % (7.916 B)
03	devicea	OK	26,3 % (8.486 B)
03	deviceb	OK	25,4 % (8.206 B)
04	sender	OK	23,0 % (7.424 B)

Modul	Environment	Build	Flash (dari 32.256 B)
04	receiver	OK	26,9 % (8.686 B)
05	master	OK	29,6 % (9.560 B)
05	slave1	OK	26,3 % (8.492 B)
05	slave2	OK	26,3 % (8.492 B)
07	slave1	OK	26,4 % (8.522 B)
07	slave2	OK	26,4 % (8.522 B)
08	gateway	OK	33,7 % (10.868 B)
08	node1 / node2	OK	29,5 % (9.526 B)
08B	gateway	OK	35,9 % (11.566 B)
08B	node1 / node2	OK	33,2 % (10.702 B)
09	gateway	OK	37,1 % (11.978 B)
09	node1 / node2	OK	34,6 % (11.176 B)
10	gateway	OK	37,0 % (11.922 B)
10	node1 / node2	OK	35,2 % (11.366 B)

Tabel P.14: Status build dan pemakaian flash tiap environment

Modul 06 dan master Modul 07 tidak muncul pada tabel di atas karena tidak ada yang dikompilasi: keduanya Python yang dijalankan langsung. Yang diperiksa pada keduanya hanya kesahihan sintaksis (python3 -m py_compile), sebab spidev dan RPi.GPIO hanya dapat dipasang di Raspberry Pi. Kode aslinya berasal dari repositori Dragino LoRa GPS HAT yang menyatakan telah diuji berjalan pada Raspberry Pi 4.

Perubahan terhadap kode asli. Yang berubah hampir seluruhnya cara membangun dan menamai berkas, bukan isi programnya; satu-satunya perubahan perilaku adalah baris terakhir Tabel P.15.

Sebelum	Sesudah	Alasan
Berkas .ino per folder	src/<peran>/main.cpp + platformio.ini	Mengikuti alur PlatformIO seperti lab WSN-IOT-prak
#define DEVICE_A disunting manual (M03)	build flag -DDEVICE_A pada environment	Satu source untuk dua board; menghilangkan risiko lupa mengembalikan
slave1.ino dan slave2.ino terpisah (M05)	satu slave/main.cpp + -DSLAVE_ID=n	Kedua slave identik kecuali nomornya
Port ditulis COM8/COM9 di komentar	upload_port di platformio.ini	Port terkumpul di satu tempat, tidak tersebar di komentar
01a-sender.py / 01b-receiver.py (M06)	src/sender.py / src/receiver.py	Tata letak src/ seragam dengan seluruh modul lain
Nama modul disebut di docstring sebagai contoh lepas	Docstring menunjuk modul lab yang dicerminkannya	Tiap berkas Python menyebut padanan Arduino-nya secara langsung
Pesan pembuka slave M07 tertulis mati POLL:1	Menyebut SLAVE_ID yang sesungguhnya	Slave 2 tidak lagi mencetak Menunggu POLL:1. Cacat yang sama sempat ada juga di M05, ditemukan dan diperbaiki lewat pengujian perangkat 21 Agustus 2026

Tabel P.15: Perubahan yang dilakukan terhadap kode asli

Kode Sumber dan Repositori

Seluruh berkas kode sumber setiap modul dimuat **lengkap** di dalam buku ini, pada bagian “Kode Program” masing-masing bab, sehingga buku dapat dipakai tanpa akses jaringan. Salinan daringnya berada pada repositori praktikum github.com/hendrieepis/WSN-IOT-prak-Lora.

Kotak **KODE SUMBER** di awal tiap bagian Kode Program menyediakan tautan langsung ke folder dan berkas yang bersangkutan, sehingga kode dapat diunduh tanpa mengetik ulang. Perkakas yang dipakai lintas modul berada di [tools/](#) dan dimuat pada Lampiran A.

Referensi

Arduino — shield (M01–M05, slave M07, M08–M11)

- [Dragino LoRa Shield — repositori resmi](#)
- Skematik Shield v1.2 (PDF) — [skematik/Lora Shield v1.2.sch.pdf](#)
- [Library LoRa by sandeepmistry](#)

Raspberry Pi — HAT (M06, master M07, gateway M11)

- [Wiki Dragino — Lora/GPS HAT](#)
- [Dragino LoRa GPS HAT v1.4 — repositori resmi](#)
- Skematik LoRa GPS HAT v1.4 (PDF) — [skematik/Lora GPS HAT for RPi v1.4.pdf](#)
- User Manual LoRa GPS HAT v1.0 (PDF) — [dokumen/LoRa_GPS_HAT_UserManual_v1.0.pdf](#)

Umum

- [Datasheet SX1276](#)

1. Modul 01 — Tautan LoRa Satu Arah

LoRa COMMUNICATION LAB · MODUL 01

Establish the Link — Tautan LoRa Satu Arah

Arduino Uno + Dragino LoRa Shield v1.2 · LoRa mentah · satu arah · RX polling · level Basic · 2 × 50 menit · folder kode Modul01_lora_uart

PENGANTAR MODUL · Gambaran Umum

Modul 01 dirancang untuk dua pertemuan (2 × 50 menit) pada tingkat dasar. Misinya membentuk tautan LoRa satu arah yang stabil: satu board mengirim pesan berkala, satu board lain menerimanya beserta ukuran kualitas sinyal. Percobaan memakai dua Arduino Uno bershield Dragino LoRa v1.2, diamati melalui dua Serial Monitor pada 9600 baud.

1.1 Pendahuluan

LoRa berbeda mendasar dari BLE, Zigbee, maupun Thread yang berbasis jaringan. Di sini **tidak ada jaringan sama sekali** — tidak ada proses join, tidak ada alamat, tidak ada koneksi. Yang ada hanya modulasi radio: siapa pun yang menyetel frekuensi, spreading factor, bandwidth, dan coding rate yang sama akan mendengar apa yang dikirim. Semua yang biasanya disediakan protokol — identitas, penyaringan tujuan, konfirmasi, penanganan tabrakan — harus dibangun sendiri di lapisan aplikasi, dan itulah yang dikerjakan modul-modul berikutnya.

Prasyaratnya hanya dasar bahasa C dan alur build PlatformIO; tidak ada modul LoRa yang mendahuluinya. Yang dibangun di sini adalah inisialisasi SX1276 melalui SPI, penyetelan empat parameter radio yang menentukan tautan, pengiriman paket secara blocking, penerimaan dengan cara polling, serta pembacaan RSSI dan SNR sebagai instrumen ukur. Semuanya dipakai lagi pada M02 ketika penerimaan berpindah ke interrupt, M03 ketika arah data menjadi dua arah, M04 ketika keandalan diukur dengan ACK, dan M05 ketika jumlah node bertambah.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01 (ini)	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan tanpa memblokir loop (interrupt) + indikator LED
03	Dua arah — kedua board bergantian mengirim dan menerima
04	Keandalan diukur: ACK, timeout, dan hitungan gagal
05	Banyak node — satu master menjadwalkan giliran bicara

Tabel 1.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Payload berupa string ASCII pendek dengan **nomor urut** di dalamnya (Hello LoRa #7). Nomor itulah yang membuat paket hilang dapat dihitung — cukup mencari lompatan angka di log penerima. Format bernomor yang sama dipertahankan sampai M05 (DATA:n, S1:DATA:n), sehingga hasil pengukuran antar-modul dapat dibandingkan.

1.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membentuk dan mengukur tautan LoRa satu arah

1. Menjelaskan peran empat parameter radio LoRa — frekuensi, spreading factor, bandwidth, dan coding rate — serta akibatnya bila salah satu berbeda antara pengirim dan penerima.
2. Menginisialisasi SX1276 melalui SPI dengan pemetaan pin shield yang benar, dan mendiagnosis kegagalan `LoRa.begin()`.
3. Menjelaskan perbedaan pengiriman blocking dan penerimaan berbasis polling, beserta konsekuensinya pada `loop()`.
4. Membaca RSSI dan SNR sebagai dua besaran yang berbeda, dan menjelaskan mengapa keduanya perlu dicatat bersama.
5. Menghitung packet loss dari nomor urut pada payload.

Kriteria keberhasilan

- ☐ Kedua board mencetak `Init LoRa ... OK` setelah reset.
- ☐ Penerima mencetak setiap paket beserta RSSI dan SNR.
- ☐ Nomor urut yang diterima berurutan tanpa lompatan pada jarak dekat.
- ☐ Tabel jarak-RSSI-SNR-loss terisi dari pengukuran sendiri, minimal empat jarak.

1.3 Dasar Teori (Secukupnya)

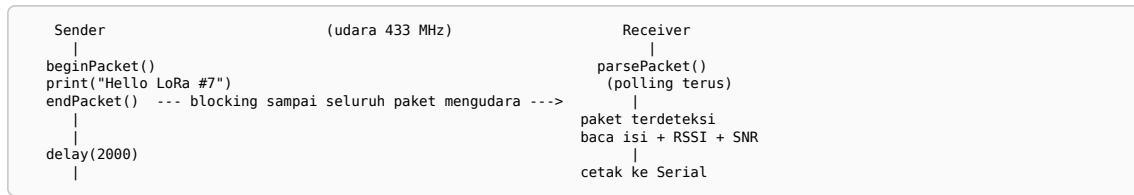
Teori dibatasi pada apa yang dipakai di percobaan. Istilah kerja yang diperlukan dirangkum pada Tabel 1.2.

Istilah	Definisi kerja di lab ini
LoRa	Teknik modulasi <i>chirp spread spectrum</i> untuk jangkauan jauh dengan laju data rendah. Modul ini memakai LoRa mentah, bukan LoRaWAN.
SX1276	Chip radio pada shield Dragino, dikendalikan mikrokontroler lewat SPI.
Frekuensi kerja	Kanal radio yang dipakai. Program disetel 433 MHz sesuai shield yang dipakai di lab ini; kedua board wajib sama persis.
Spreading Factor (SF)	Lama satu simbol. SF besar menambah jangkauan dan ketahanan, tetapi memperlambat data dan memperpanjang waktu udara. Modul ini memakai SF7.
Bandwidth (BW)	Lebar kanal. Semakin sempit semakin sensitif, tetapi semakin lambat. Modul ini memakai 125 kHz.
Coding Rate (CR)	Rasio kode koreksi galat. 4/5 berarti tiap 4 bit data disertai 1 bit koreksi.
RSSI	Kuat sinyal terima dalam dBm — seberapa keras sinyal terdengar. Semakin mendekati nol semakin kuat.
SNR	Selisih sinyal terhadap derau dalam dB — seberapa jernih sinyal terdengar. LoRa masih dapat memecahkan sinyal pada SNR negatif.
Polling	Penerima memeriksa sendiri secara berulang apakah ada paket (<code>LoRa.parsePacket()</code>).

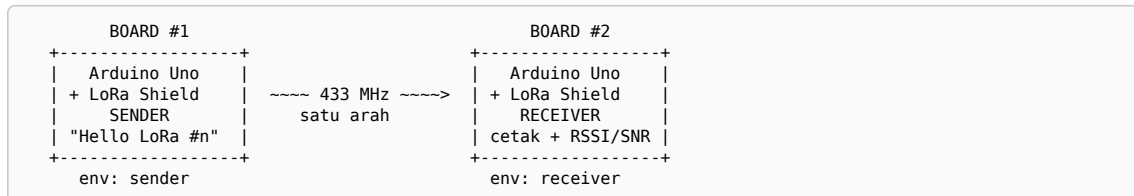
Tabel 1.2: Istilah kerja Modul 01

Mengapa RSSI saja tidak cukup. RSSI mengukur daya yang tiba di penerima, termasuk derau. Sebuah sinyal bisa terdengar keras (RSSI – 70 dBm) tetapi tenggelam dalam gangguan sehingga gagal dipecahkan, dan sebaliknya sinyal lemah (RSSI – 120 dBm) tetap terbaca jika lingkungan sunyi. Keunggulan LoRa justru terletak di sana: modulasinya sanggup bekerja pada **SNR negatif**, yaitu ketika sinyal lebih lemah daripada derau di sekitarnya. Karena itu kedua angka dicatat bersama, dan pada M05 keduanya dipakai untuk menjelaskan mengapa sebuah node tidak menjawab.

Sekuens yang diamati



1.4 Topologi



Node	Environment	Peran	Payload / interval
Sender	sender	Pengirim	Hello LoRa #n tiap 2000 ms
Receiver	receiver	Penerima (polling)	—

Tabel 1.3: Peran tiap node Modul 01

Tidak ada alamat maupun identitas node pada modul ini. Setiap penerima yang menyetel parameter radio sama akan menerima paket yang sama — sifat yang akan menjadi masalah nyata di M05, dan diselesaikan di sana dengan penomoran di lapisan aplikasi.

1.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P, flash 32 KB	2
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
3	Antena SMA	sesuai band shield — wajib terpasang sebelum diberi daya	2
4	Kabel USB tipe B	kabel data	2
5	PC/Laptop	PlatformIO Core/IDE, 2 port USB bebas	1

Tabel 1.4: Alat dan bahan Modul 01

PERINGATAN

Jangan menyalakan shield tanpa antena. Daya pancar yang tidak menemukan beban akan dipantulkan kembali ke penguat SX1276 dan dapat merusaknya secara permanen. Pasang antena lebih dahulu, baru sambungkan USB.

Pin yang dipakai shield (tidak boleh dipakai program)

Pin Arduino	Fungsi LoRa	Keterangan
D10	NSS / CS	Chip select SX1276, ditentukan jumper R9 (0 ohm, terpasang dari pabrik)

Pin Arduino	Fungsi LoRa	Keterangan
D11	MOSI	SPI data keluar
D12	MISO	SPI data masuk
D13	SCK	SPI clock — juga LED bawaan Arduino
D9	RST	Reset SX1276
D2	DIO0	Interrupt TX-done / RX-done (INT0)

Tabel 1.5: Pemetaan pin shield Modul 01

Pin D6, D7, dan D8 tersambung ke DIO1, DIO2, dan DIO5 pada shield. Ketiganya tidak dipakai program ini, tetapi jangan dijadikan output agar tidak beradu dengan keluaran chip. Pin yang bebas dipakai: **D3, D4, D5, dan A0–A5**.

Struktur proyek

```

Modul01_lora_uart/
├── platformio.ini
├── monitor_serial.py    ← pantau TX & RX sekaligus, ringkas loss/RSSI/SNR
├── upload_auto.py       ← deteksi port otomatis saat unggah
├── logserial.md         ← log referensi hasil uji perangkat
└── src/
    ├── sender/main.cpp  ← kirim "Hello LoRa #n" tiap 2 detik
    └── receiver/main.cpp ← terima, cetak isi + RSSI + SNR

```

1.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul01_lora_uart/](#)

- [Modul01_lora_uart/platformio.ini](#)
- [Modul01_lora_uart/src/sender/main.cpp](#)
- [Modul01_lora_uart/src/receiver/main.cpp](#)
- [Modul01_lora_uart/monitor_serial.py](#)
- [Modul01_lora_uart/upload_auto.py](#)

```

[platformio]
default_envs = receiver

[env]
platform = atmelavr
board = uno
framework = arduino
monitor_speed = 9600
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
[env:sender]
build_src_filter = +<sender/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:receiver]
build_src_filter = +<receiver/*.cpp>
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

```

Kode 1.1: platformio.ini Modul 01 — environment sender dan receiver

```

/*
LoRa Sender - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: sender

Pin mapping (hardwired on shield):
  NSS/CS -> D10 (R9 0-ohm populated)
  DI00    -> D2
  RST     -> D9
  SCK     -> D13, MOSI -> D11, MISO -> D12 (ICSP header)

Ganti FREQUENCY sesuai versi modul:
  433E6 -> 433 MHz
  868E6 -> 868 MHz (EU)
  915E6 -> 915 MHz (US/AS)
  923E6 -> 923 MHz (Indonesia AS923)
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN    10
#define RST_PIN    9
#define DI00_PIN   2

#define FREQUENCY  433E6

int counter = 0;

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println(F("=== LoRa SENDER ==="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);

  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek kabel/modul."));
    while (true);
  }

  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);
  LoRa.setTxPower(17);

  Serial.println(F("OK"));
  Serial.print(F("Frekuensi : ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
  Serial.println(F("SF=7, BW=125kHz, CR=4/5, Power=17dBm"));
  Serial.println(F("Kirim tiap 2 detik...\n"));
}

void loop() {
  String msg = "Hello LoRa #" + String(counter);

  Serial.print(F("[TX] \n"));
  Serial.print(msg);
  Serial.print(F("\n ... "));

  LoRa.beginPacket();
  LoRa.print(msg);
  LoRa.endPacket();

  Serial.println(F("terkirim"));
  counter++;

  delay(2000);
}

```

Kode 1.2: src/sender/main.cpp — kirim Hello LoRa #n tiap 2 detik

```

/*
LoRa Receiver - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: receiver

```

```

Pin mapping (hardwired on shield):
NSS/CS -> D10 (R9 0-ohm populated)
DIO0 -> D2
RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12 (ICSP header)

FREQUENCY harus sama persis dengan sender.
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN 10
#define RST_PIN 9
#define DIO0_PIN 2

#define FREQUENCY 433E6

void setup() {
  Serial.begin(9600);
  while (!Serial);

  Serial.println(F("=== LoRa RECEIVER ==="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);

  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek kabel/modul."));
    while (true);
  }

  LoRa.setSpreadingFactor(7);
  LoRa.setSignalBandwidth(125E3);
  LoRa.setCodingRate4(5);

  Serial.println(F("OK"));
  Serial.print(F("Frekuensi : ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
  Serial.println(F("SF=7, BW=125kHz, CR=4/5"));
  Serial.println(F("Menunggu paket...\n"));
}

void loop() {
  int packetSize = LoRa.parsePacket();

  if (packetSize > 0) {
    String received = "";
    while (LoRa.available()) {
      received += (char)LoRa.read();
    }

    Serial.println(F("--- Paket Diterima ---"));
    Serial.print(F(" Data : \n"));
    Serial.print(received);
    Serial.println(F("\n"));
    Serial.print(F(" RSSI : "));
    Serial.print(LoRa.packetRssi());
    Serial.println(F(" dBm"));
    Serial.print(F(" SNR : "));
    Serial.print(LoRa.packetSnr());
    Serial.println(F(" dB"));
    Serial.println();
  }
}

```

Kode 1.3: src/receiver/main.cpp — terima paket, cetak isi beserta RSSI dan SNR

```

#!/usr/bin/env python3
"""Monitor pengirim dan penerima LoRa sekaligus untuk Modul 01.

Menampilkan kedua aliran serial dalam SATU jendela dengan timestamp bersama,
lalu meringkasnya saat berhenti: berapa paket dikirim, berapa yang tiba, dan
sebaran RSSI serta SNR-nya. Membandingkan nomor urut di dua jendela terpisah
jauh lebih sulit, karena tiap jendela punya sumbu waktunya sendiri.

python3 Modul01_lora_uart/monitor_serial.py

```

```
python3 Modul01_lora_uart/monitor_serial.py --log sesil.txt
python3 Modul01_lora_uart/monitor_serial.py --port TX=/dev/ttyUSB0
python3 Modul01_lora_uart/monitor_serial.py --durasi 60      # berhenti sendiri
```

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C, atau pakai `--durasi` agar berhenti otomatis.

PORT OTOMATIS. Port TX/RX tidak lagi ditulis tetap di kode: setiap kali dijalankan, skrip memindai `/dev/ttyACM\*` dan `/dev/ttyUSB\*` yang sedang aktif dan memakai yang benar-benar tersambung,urut sesuai nama device. Ini penting karena nama port bisa berubah tiap kali board dicabut-pasang atau board asli/klon bercampur (lihat tools/deteksi_port.py). Bila urutan otomatisnya keliru (mis. board TX ternyata terdeteksi sebagai RX), timpa lewat --port, mis. --port TX=/dev/ttyACM1 --port RX=/dev/ttyACM0.

SOAL RESET OTOMATIS. Pada Arduino Uno, jalur DTR terhubung ke pin RESET lewat kapasitor. Membuka port serial **selalu me-reset board**, dan hal itu justru dimanfaatkan di sini: pesan `setup()` beserta baris `Init LoRa ... OK` ikut terekam tanpa perlu menekan tombol reset. Konsekuensinya, penghitung paket kedua board kembali ke nol setiap kali monitor dijalankan – jalankan monitor lebih dahulu, baru mulai mengukur, dan jangan membukanya di tengah percobaan panjang.

Pada Uno, DTR hanya menyentuh jalur RESET dan tidak tersambung ke GPIO mana pun, sehingga membuka port tidak pernah membuat sebuah pin terbaca seolah ditekan.

```
"""
import argparse
import glob
import re
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

# Nama peran bawaan (urutan) dan warna yang dipasangkan ke port yang
# terdeteksi aktif, bukan ke nama /dev tertentu – lihat deteksi_port_aktif().
NAMA_PERAN_BAWAAN = ["TX", "RX"]
PALET_WARNA = ["\033[33m", "\033[36m", "\033[35m", "\033[32m", "\033[34m"]

def deteksi_port_aktif():
    """Kumpulkan port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* – pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil ulang tiap program start supaya daftarnya selalu mengikuti apa
    yang benar-benar tersambung saat itu, bukan port tetap di kode.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def bangun_port_bawaan():
    """Pasangkan tiap port aktif ke nama peran (TX, RX, lalu P3, P4, ...)."""
    hasil = []
    for i, dev in enumerate(deteksi_port_aktif()):
        nama = NAMA_PERAN_BAWAAN[i] if i < len(NAMA_PERAN_BAWAAN) else f"P{i + 1}"
        warna = PALET_WARNA[i % len(PALET_WARNA)]
        hasil.append((nama, dev, warna))
    return hasil

# Pola yang dipanen dari baris serial untuk menyusun ringkasan
RE_NOMOR = re.compile(r"#(\d+)")
RE_RSSI = re.compile(r"RSSI\s*:\s*(-?\d+)")
RE_SNR = re.compile(r"SNR\s*:\s*(-?\d.+)")
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
# Bahan ringkasan akhir; diisi reader, dibaca sekali saat program berhenti
nomor_tx, nomor_rx, daftar_rssi, daftar_snr = [], [], [], []
stop = threading.Event()
t0 = time.time()

def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
```

```

stamp = f"{time.time() - t0:8.3f}"
plain = f"[{stamp}] {name:<7} | {text}"
with print_lock:
    if use_color:
        print(f"[DIM][{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
    else:
        print(plain, flush=True)
    if logfile:
        logfile.write(plain + "\n")
        logfile.flush()

def open_port(port, baud):
    """Buka port serial menuju board Arduino.

    Jalur DTR/RTS sengaja TIDAK disentuh. Menyetelnya sebelum open() ditolak
    oleh CDC ATmega16U2 pada Uno asli dan berakhir sebagai
    `[Errno 110] Connection timed out`; pada klon ber-bridge CH340 hal itu
    kebetulan lolos, sehingga kesalahannya mudah tidak terlihat bila diuji
    hanya pada satu jenis board.

    Membuka port tetap me-reset board sekali - itu memang yang diinginkan,
    karena pesan setup() ikut terekam - dan reset itu berupa pulsa dari
    kapasitor, bukan penahanan, sehingga board langsung berjalan normal.
    """
    return serial.Serial(port, baud, timeout=0.2)

def panen(name, text):
    """Ambil nomor paket, RSSI, dan SNR dari sebuah baris serial.

    Nomor dipanen dari kedua sisi supaya paket yang hilang dapat dihitung
    sebagai selisih, bukan diperkirakan dari jumlah baris.
    """
    m = RE_NOMOR.search(text)
    if m:
        (nomor_tx if name == "TX" else nomor_rx).append(int(m.group(1)))
    if name != "TX":
        if (m := RE_RSSI.search(text)):
            daftar_rssi.append(int(m.group(1)))
        if (m := RE_SNR.search(text)):
            daftar_snr.append(float(m.group(1)))

def ringkasan():
    """Cetak hasil ukur tautan: loss, RSSI, dan SNR."""
    print("\n" + "-" * 60)
    print(f"Durasi: {time.time() - t0:.1f} s")
    for nama in counts:
        print(f" {nama:<7} : {counts.get(nama, 0)} baris")

    if not nomor_tx and not nomor_rx:
        print("\nTidak ada paket bernomor yang terbaca - periksa tautan.")
        return

    kirim, terima = set(nomor_tx), set(nomor_rx)
    print(f"\n Paket dikirim : {len(kirim)}"
          + (f" (nomor {min(kirim)}..{max(kirim)})" if kirim else ""))
    print(f" Paket diterima : {len(terima)}"
          + (f" (nomor {min(terima)}..{max(terima)})" if terima else ""))

    # Hanya nomor yang benar-benar terlihat dikirim yang dihitung sebagai hilang,
    # sehingga paket sebelum monitor dijalankan tidak ikut dianggap gagal.
    if kirim:
        hilang = sorted(kirim - terima)
        loss = len(hilang) / len(kirim) * 100
        print(f" Paket hilang : {len(hilang)} ({loss:.1f} %)")
        if hilang:
            print(f" Nomor hilang : {' '.join(str(n) for n in hilang[:20])}"
                  + (" ..." if len(hilang) > 20 else ""))

    if daftar_rssi:
        print(f" RSSI min/maks/rata-rata : {min(daftar_rssi)} / {max(daftar_rssi)} / "
              f"{sum(daftar_rssi) / len(daftar_rssi):.1f} dBm")
    if daftar_snr:
        print(f" SNR min/maks/rata-rata : {min(daftar_snr):.2f} / {max(daftar_snr):.2f} / "
              f"{sum(daftar_snr) / len(daftar_snr):.2f} dB")

def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:

```



```

    ser = open_port(port, baud)
except Exception as e:
    show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
    return

show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
buf = b""
while not stop.is_set():
    try:
        data = ser.read(256)
    except Exception as e:
        show(name, color, f"!! port terputus: {e}", use_color, logfile)
        break
    if not data:
        continue
    buf += data
    # Pesan dipecah per baris agar output tiga board tidak saling menyisip
    # di tengah kalimat.
    *lines, buf = buf.split(b"\n")
    for line in lines:
        text = line.decode("utf-8", "replace").rstrip("\r")
        if text.strip():
            counts[name] += 1
            panen(name, text)
            show(name, color, text, use_color, logfile)
ser.close()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyXXX",
                    help="ganti port TX atau RX, mis. RX=/dev/ttyACM0; boleh diulang")
    # Modul 01-04 memakai 9600; Modul 05 memakai 115200.
    ap.add_argument("--baud", type=int, default=9600)
    ap.add_argument("--durasi", type=float, metavar="DETIK",
                    help="berhenti otomatis setelah sekian detik (mis. --durasi 60)")
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    port_bawaan = bangun_port_bawaan()
    if not port_bawaan and not args.port:
        sys.exit("Tidak ada port serial USB yang terdeteksi (/dev/ttyACM*, /dev/ttyUSB*). \n"
                "Periksa kabel data (bukan charge-only) dan hak akses grup dialout, \n"
                "atau jalankan tools/deteksi_port.py untuk diagnosis lebih lanjut, \n"
                "atau tentukan port manual lewat --port TX=/dev/ttyACM0 --port RX=/dev/ttyACM1")

    dipakai = port_bawaan[:len(NAMA_PERAN_BAWAAN)]
    sisa = port_bawaan[len(NAMA_PERAN_BAWAAN):]
    print(f"Port aktif terdeteksi: {'', '.join(p for _, p, _ in port_bawaan)}")
    if port_bawaan else "Tidak ada port aktif terdeteksi otomatis – memakai --port saja.")
    for nama, port, _ in dipakai:
        print(f" -> {nama} = {port}")
    if sisa:
        print(f" (tidak dipakai: {'', '.join(p for _, p, _ in sisa)}")
        f" – pakai --port NAMA=/dev/... untuk memakainya")

    ports = {name: (port, color) for name, port, color in dipakai}
    for item in args.port or []:
        if "=" not in item:
            sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyXXX)")
        name, port = item.split("=", 1)
        _, color = ports.get(name.upper(), (None, "\033[35m"))
        ports[name.upper()] = (port, color)

    if not ports:
        sys.exit("Tidak ada port untuk dipantau – tentukan lewat --port NAMA=/dev/ttyXXX")

    use_color = not args.no_color and sys.stdout.isatty()
    logfile = open(args.log, "w") if args.log else None

    # SIGTERM (mis. dijalankan lewat `timeout 30 ...`) cukup menyalakan flag
    # `stop` yang sama dipakai reader thread, BUKAN melempar exception --
    # melempar exception dari signal handler bisa mendarat di tengah blok
    # `finally` (mis. saat t.join() atau ringkasan() berjalan) dan menyebabkan
    # crash tak tertangkap alih-alih mencetak ringkasan dengan rapi.
    signal.signal(signal.SIGTERM, lambda _: stop.set())

    batas = f" · berhenti otomatis {args.durasi:.0f} s" if args.durasi else ""
    print(f"Monitor LoRa TX/RX @ {args.baud} baud – Ctrl-C untuk berhenti{batas}")
    f"{' · log: ' + args.log if args.log else ''}")

```

```

print("-" * 60)

threads = [threading.Thread(target=reader,
                             args=(name, port, color, args.baud, use_color, logfile),
                             daemon=True)
            for name, (port, color) in ports.items()]
for t in threads:
    t.start()

batasWaktu = t0 + args.durasi if args.durasi else None
try:
    while any(t.is_alive() for t in threads) and not stop.is_set():
        if batasWaktu and time.time() >= batasWaktu:
            break
        time.sleep(0.2)
except KeyboardInterrupt:
    pass
finally:
    stop.set()
    for t in threads:
        t.join(timeout=1.0)
    ringkasan()
    if logfile:
        logfile.close()
        print(f"\nLog tersimpan di {args.log}")

if __name__ == "__main__":
    main()

```

Kode 1.4: monitor_serial.py — pantau TX dan RX pada satu sumbu waktu

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 01 (LoRa UART) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul01_lora_uart -e sender -t upload -t monitor
    pio run -d Modul01_lora_uart -e receiver -t upload -t monitor

Skrup ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `sender` (port pertama) dan `receiver` (port kedua), lalu
menjalankan `pio run` dengan --upload-port sesuai hasil deteksi --
platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua
board selesai di-upload, skrip bertanya apakah mau langsung menjalankan
monitor_serial.py -- bila ya, dijalankan dengan --port TX=... --port RX=...
persis memakai port sender/receiver yang barusan dipakai upload (bukan
deteksi ulang), supaya tidak mungkin salah pasang.

python3 Modul01_lora_uart/upload_auto.py
-> upload kedua board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul01_lora_uart/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk sender, lanjut receiver
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti monitor_serial.py

python3 Modul01_lora_uart/upload_auto.py --only sender
-> upload satu environment saja

python3 Modul01_lora_uart/upload_auto.py --sender /dev/ttyACM0 --receiver /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual

"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["sender", "receiver"]
# Nama environment PlatformIO -> nama peran yang dipakai monitor_serial.py
ENV_KE_PERAN = {"sender": "TX", "receiver": "RX"}

def deteksi_port_aktif():

```

```

"""Port serial USB yang sedang tersambung, menurut nama device.

Pola yang sama dipakai tools/deteksi_port.py dan monitor_serial.py: Uno
asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
sebagai /dev/ttyUSB*.
"""
return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $ ", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan monitor_serial.py
    dengan port sender/receiver PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang barusan
    di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan monitor_serial.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "monitor_serial.py")]
    for env, peran in ENV_KE_PERAN.items():
        if env in port_env:
            cmd += ["--port", f"{peran}={port_env[env]}"]

    print(" $ ", " ".join(cmd))
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) monitor_serial.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--sender", metavar="/dev/ttyXXX",
                    help="tiapa port sender (lewati deteksi otomatis)")
    ap.add_argument("--receiver", metavar="/dev/ttyXXX",
                    help="tiapa port receiver (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (sender atau receiver)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                    "(bawaan: upload saja -- pakai monitor_serial.py untuk memantau keduanya"
                    sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"sender": args.sender, "receiver": args.receiver}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:

```

```

if nama in port_env:
    sumber = "manual" if manual[nama] else "otomatis"
    print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
else:
    print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
          f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

daftar = [args.only] if args.only else ENV_URUTAN
hilang = [n for n in daftar if n not in port_env]
if hilang:
    sys.exit(f"\nTidak bisa lanjut: port untuk {' '.join(hilang)} tidak ditemukan.\n"
            f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

for nama in daftar:
    if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
        sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

print("\nSelesai.")
tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 1.5: upload_auto.py — pemilih port otomatis saat unggah

1.7 Build dan Flash

Penerima diunggah lebih dahulu, agar paket pertama pengirim tidak terbuang.

```

pio run -d Modul01_lora_uart -e receiver -t upload -t monitor
pio run -d Modul01_lora_uart -e sender -t upload -t monitor

```

Memantau kedua board sekaligus. Dua Serial Monitor terpisah menyulitkan perbandingan nomor urut, karena tiap jendela memiliki sumbu waktunya sendiri. Skrip monitor_serial.py (Kode 1.4) menggabungkan keduanya pada satu sumbu waktu, lalu meringkas hasil ukur tautannya saat berhenti.

```

python3 Modul01_lora_uart/monitor_serial.py
python3 Modul01_lora_uart/monitor_serial.py --durasi 60 --log sesil.txt
python3 Modul01_lora_uart/monitor_serial.py --port RX=/dev/ttyACM0

```

Keluarannya berisi kedua aliran berdampingan, ditutup ringkasan yang langsung mengisi tabel EXP-02.

```

[ 2.012] RX      | --- Paket Diterima ---
[ 2.012] RX      | Data : "Hello LoRa #0"
[ 2.012] RX      | RSSI : -41 dBm
[ 2.023] TX      | [TX] "Hello LoRa #0" ... terkirim
-----
Paket dikirim   : 10 (nomor 0..9)
Paket diterima  : 10 (nomor 0..9)
Paket hilang    : 0 (0.0 %)
RSSI min/maks/rata-rata : -46 / -40 / -41.4 dBm
SNR min/maks/rata-rata : 9.25 / 9.75 / 9.45 dB

```

Paket hilang dihitung dari **selisih nomor urut** yang benar-benar terlihat dikirim, bukan dari jumlah baris, sehingga paket yang lewat sebelum monitor dijalankan tidak ikut dianggap gagal.

PENTING

Membuka monitor me-reset kedua board. Pada Arduino Uno, jalur DTR terhubung ke pin RESET, sehingga penghitung paket kembali ke nol setiap kali monitor dijalankan. Sifat itu justru berguna — baris Init LoRa ... OK ikut terekam — tetapi berarti monitor harus dijalankan **lebih dahulu**, bukan di tengah percobaan yang sedang berlangsung.

Pre-flight checklist

- ☐ Antena terpasang pada kedua shield.
- ☐ pio device list dijalankan, port kedua board dicatat dan diisikan ke platformio.ini.
- ☐ Nilai FREQUENCY pada kedua source **sama persis** dan sesuai band shield yang dipakai.

☐ Dua Serial Monitor 9600 baud siap, atau `monitor_serial.py` dijalankan lebih dahulu.

1.8 Percobaan

1.8.1 EXP-01 — Inisialisasi Radio

Unggah kedua firmware, buka Serial Monitor keduanya, dan amati pesan awal.

Expected output — sender

```
=== LoRa SENDER ===
Init LoRa ... OK
Frekuensi : 433.00 MHz
SF=7, BW=125kHz, CR=4/5, Power=17dBm
Kirim tiap 2 detik...
```

Data capture

Parameter	Hasil
Pesan init pada sender	
Pesan init pada receiver	
Frekuensi yang tercetak (MHz)	
Pemakaian Flash dari ringkasan build	

Tabel 1.6: Lembar pengamatan EXP-01

CHECKPOINT

Kedua board mencetak OK. Munculnya GAGAL! Cek kabel/modul. berarti SX1276 tidak menjawab lewat SPI: periksa shield benar-benar duduk di header, jumper R9 terpasang, dan tidak ada program lain memakai D10.

1.8.2 EXP-02 — Aliran Data Satu Arah

Biarkan sistem berjalan dua menit, lalu amati keterhubungan antara nomor urut di kedua sisi.

Expected output — receiver

```
--- Paket Diterima ---
Data : "Hello LoRa #2"
RSSI : -55 dBm
SNR : 9.50 dB

--- Paket Diterima ---
Data : "Hello LoRa #3"
RSSI : -54 dBm
SNR : 9.00 dB
```

Data capture

Parameter	Hasil
Nomor urut terakhir di sender	
Nomor urut terakhir di receiver	
Paket hilang (selisih)	
RSSI rata-rata (dBm)	

Parameter	Hasil
SNR rata-rata (dB)	

Tabel 1.7: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA

Di `src/receiver/main.cpp`, `LoRa.parsePacket()` dipanggil pada setiap putaran `loop()` tanpa jeda sama sekali. Tambahkan `delay(1000)` di akhir `loop()` penerima, unggah ulang, lalu amati apa yang terjadi pada paket yang tiba selama penerima sedang tertidur. Jelaskan hasilnya, lalu kembalikan kodenya. Pengamatan ini adalah alasan M02 berpindah ke interrupt.

CHECKPOINT

Nomor urut yang diterima naik satu per satu tanpa lompatan pada jarak dekat. Lompatan angka pada jarak 1 meter menandakan gangguan atau parameter yang tidak seragam, bukan keterbatasan jangkauan — telusuri sebelum melanjutkan ke pengukuran jarak.

1.8.3 EXP-03 — Parameter Harus Seragam

Ubah **satu** parameter di penerima saja, unggah ulang, dan amati akibatnya. Kembalikan ke nilai semula setelah tiap uji.

Uji	Perubahan di receiver	Hasil yang teramati
03-a	<code>setSpreadingFactor(8)</code>	
03-b	<code>setSignalBandwidth(250E3)</code>	
03-c	<code>setCodingRate4(6)</code>	
03-d	<code>FREQUENCY 868E6</code>	

Tabel 1.8: Lembar pengamatan EXP-03 — keseragaman parameter radio

CHECKPOINT

Tiga dari empat uji menghentikan penerimaan **sepenuhnya**, bukan sekadar memperburuknya: untuk SF, bandwidth, dan frekuensi, radio LoRa tidak mengenal “hampir cocok”. Satu uji berperilaku berbeda — temukan yang mana, lalu jelaskan sebabnya sebelum melanjutkan. Petunjuk: periksa apa saja yang dibawa header PHY sebuah paket LoRa, dan apa yang menentukan bentuk gelombangnya secara fisik.

Mengapa coding rate berbeda sendiri. Uji 03-c tetap menerima paket seperti biasa. Penyebabnya, program memakai *explicit header mode* (bawaan library): tiap paket membawa header PHY berisi panjang payload dan **coding rate yang dipakai**, dan header itu sendiri selalu dikirim dengan CR 4/8. Penerima membaca CR dari header lalu menyesuaikan diri, sehingga `setCodingRate4()` di sisi penerima hanya berlaku ketika penerima **mengirim**. Sebaliknya SF, bandwidth, dan frekuensi menentukan bentuk gelombang: bila salah satu berbeda, sinyal tidak dikenali sebagai paket LoRa sama sekali dan header pun tidak pernah terbaca. Hasil uji ini terekam di `logserial.md`.

1.8.4 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada dua Arduino Uno bershield Dragino LoRa v1.2, frekuensi 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

```
[ 5.851] TX | [TX] "Hello LoRa #2" ... terkirim
[ 6.051] RX | --- Paket Diterima ---
[ 6.051] RX | Data : "Hello LoRa #2"
[ 6.051] RX | RSSI : -55 dBm
[ 6.051] RX | SNR : 9.50 dB
```

Parameter	Hasil terukur
Paket dikirim / diterima dalam 62 s	28 / 28 (loss 0 %)
RSSI: min / maks / rata-rata	- 56 / - 43 / - 53,9 dBm
SNR: min / maks / rata-rata	9,00 / 9,75 / 9,45 dB
Selang TX → RX tercetak	0,002–0,200 s
Interval kirim terukur	2,00–2,01 s
EXP-03 diulang pada dua pasangan board	pola hasil identik
EXP-03: SF / BW / frekuensi berbeda	penerimaan berhenti total
EXP-03: coding rate berbeda	tetap diterima — lihat penjelasan header PHY di atas

Tabel 1.9: Hasil verifikasi perangkat keras Modul 01

```
Environment Status Flash RAM
sender SUCCESS 18.3% (5890 B) ~14%
receiver SUCCESS 22.7% (7312 B) ~15%
```

Keduanya jauh di bawah batas 32 KB Arduino Uno — inilah alasan library sandeepmistry dipilih alih-alih RadioLib yang melampaui kapasitas flash.

1.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud 9600, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Jarak antar-board ± 30 cm.

Board & Port

Peran	Environment	Port	Board	Status radio
Sender	sender	/dev/ttyACM0	Uno asli (2341:0043)	✓ init berhasil
Receiver	receiver	/dev/ttyACM1	Uno asli (2341:0043)	✓ init berhasil

Tabel 1.10: Log serial Modul 01: Board & Port

Pemetaan ini sesuai platformio. ini apa adanya, sehingga --upload-port tidak diperlukan. Kedua aliran serial direkam bersamaan memakai `monitor_serial.py`, sehingga stempel waktunya berasal dari satu sumbu yang sama dan hitungan paket hilang berasal dari selisih nomor urut, bukan perkiraan.

EXP-01 — Inisialisasi Radio

```
[ 5.049] RX | -- tersambung ke /dev/ttyACM1 @ 9600 --
[ 5.050] TX | -- tersambung ke /dev/ttyACM0 @ 9600 --
[ 5.851] TX | [TX] "Hello LoRa #2" ... terkirim
[ 6.051] RX | --- Paket Diterima ---
[ 6.051] RX | Data : "Hello LoRa #2"
[ 6.051] RX | RSSI : -55 dBm
[ 6.051] RX | SNR : 9.50 dB
```

```
[ 8.054] TX | [TX] "Hello LoRa #3" ... terkirim
[ 8.056] RX | --- Paket Diterima ---
[ 8.056] RX | Data : "Hello LoRa #3"
[ 8.056] RX | RSSI : -54 dBm
[ 8.056] RX | SNR : 9.00 dB
```

Pesan awal kedua board saat reset:

```
=== LoRa SENDER ===          === LoRa RECEIVER ===
Init LoRa ... OK              Init LoRa ... OK
Frekuensi : 433.00 MHz        Frekuensi : 433.00 MHz
SF=7, BW=125kHz, CR=4/5,      SF=7, BW=125kHz, CR=4/5
Power=17dBm                   Menunggu paket...
Kirim tiap 2 detik...
```

Parameter	Hasil
Pesan init sender	OK
Pesan init receiver	OK
Frekuensi yang tercetak	433.00 MHz di kedua board
Selang [TX] → paket tercetak di penerima	0,002–0,200 s
Flash sender / receiver	18,3 % (5.890 B) / 22,7 % (7.312 B)

Tabel 1.11: Log serial Modul 01: EXP-01 — Inisialisasi Radio

CHECKPOINT terpenuhi. Kedua board mencetak OK.

EXP-02 — Aliran Data Satu Arah (62 detik)

Parameter	Hasil
Paket dikirim / diterima	28 / 28 (nomor 2..29)
Paket hilang	0 (loss 0 %)
RSSI: min / maks / rata-rata	-56 / -43 / -53,9 dBm
SNR: min / maks / rata-rata	9,00 / 9,75 / 9,45 dB
Interval kirim terukur	2,00–2,01 s (sesuai delay (2000))

Tabel 1.12: Log serial Modul 01: EXP-02 — Aliran Data Satu Arah (62 detik)

RSSI pada sesi ini (-53,9 dBm) lebih lemah daripada sesi sebelumnya yang memakai board berbeda (-41,6 dBm), meskipun jaraknya sama. Selisih ± 12 dB itu berasal dari perbedaan antenna dan posisi board di meja, bukan dari jarak — pengingat bahwa RSSI hanya dapat dibandingkan bila kondisi ukurnya benar-benar sama. SNR keduanya praktis identik (9,45 dB vs 9,53 dB), karena derau lingkungannya sama.

EXP-03 — Parameter Harus Seragam

Satu parameter diubah **di penerima saja**, diunggah ulang, lalu penerimaan diamati 16 detik. Pengirim tidak disentuh.

Uji	Perubahan di receiver	Paket diterima / 16 s	Hasil
baseline	SF7, BW 125 kHz, CR 4/5, 433 MHz	5	tautan normal
03-a	setSpreadingFactor(8)	0	komunikasi berhenti total
03-b	setSignalBandwidth(250E3)	0	komunikasi berhenti total
03-c	setCodingRate4(6)	5	tetap diterima normal
03-d	FREQUENCY 868E6	0	komunikasi berhenti total

Tabel 1.13: Log serial Modul 01: EXP-03 — Parameter Harus Seragam

Temuan: coding rate tidak harus sama

Uji 03-c membantah dugaan awal bahwa keempat parameter sama-sama wajib seragam. Coding rate yang berbeda **tidak** menghentikan penerimaan — jumlah paket yang tertangkap sama persis dengan baseline.

Hasil ini **tereplikasi pada dua pasangan board yang berbeda**: pengujian pertama memakai Uno klon ber-bridge CH340 sebagai pengirim, pengujian kedua memakai dua Uno asli. Keduanya memberi pola yang sama persis, sehingga temuan ini bukan sifat satu perangkat tertentu.

Penjelasannya ada pada struktur paket LoRa. Program memakai *explicit header mode*, bawaan library. Pada mode itu setiap paket membawa **header PHY** berisi panjang payload dan **coding rate yang dipakai**, dan header itu sendiri selalu dikirim dengan CR 4/8. Penerima membaca coding rate dari header lalu menyesuaikan diri untuk memecahkan payload. Nilai `setCodingRate4()` di sisi penerima karena itu hanya menentukan CR ketika penerima **mengirim**, bukan ketika menerima.

Berbeda halnya dengan SF, bandwidth, dan frekuensi. Ketiganya menentukan bentuk gelombang secara fisik: penerima yang menyetel SF atau bandwidth berbeda tidak akan mengenali sinyal sebagai paket LoRa sama sekali, sehingga header pun tidak pernah terbaca. Itulah alasan uji 03-a, 03-b, dan 03-d berhenti total sementara 03-c berjalan seperti biasa.

Konsekuensi praktisnya: bila suatu saat *implicit header mode* dipakai — biasanya untuk menghemat waktu udara pada payload tetap — coding rate wajib disepakati kedua sisi, karena tidak ada lagi header yang mengabarkannya.

Riwayat: satu shield sempat gagal init

Pada sesi pengujian sebelumnya, board Uno di `/dev/ttyACM0` konsisten gagal:

```
=== LoRa SENDER ===
Init LoRa ... GAGAL! Cek kabel/modul.
```

Firmware kedua board ditukar untuk memastikan penyebabnya:

Board fisik	Sebagai sender	Sebagai receiver
di <code>/dev/ttyACM0</code>	GAGAL	GAGAL
di <code>/dev/ttyACM1</code>	OK	OK

Tabel 1.14: Log serial Modul 01: Riwayat: satu shield sempat gagal init

Kegagalan mengikuti **board fisik**, bukan firmware, bukan port, dan bukan pengaturan frekuensi — `LoRa.begin()` memeriksa register versi SX1276 **sebelum** frekuensi diterapkan, sehingga perubahan 920 MHz ke 433 MHz tidak mengubah gejalanya. Setelah shield diperbaiki, board yang sama menginisialisasi dengan normal dan dipakai pada seluruh pengukuran di atas.

Catatan ini dipertahankan karena gejalanya khas dan langkah diagnosis — menukar firmware untuk memisahkan kesalahan perangkat dari kesalahan program — berlaku untuk seluruh modul dalam seri ini.

Uji kompatibilitas board asli vs klon

Lab memakai Uno asli dan klon bercampur, sehingga perlu dipastikan firmware yang sama benar-benar berjalan di keduanya.

Yang diperiksa	Hasil
Device signature board asli (<code>tttyACM0</code>)	0x1e950f (m328p)
Device signature board klon (<code>tttyUSB0</code>)	0x1e950f (m328p) — sama
Protokol unggah	arduino pada keduanya
md5 .hex saat upload_port diubah	f263400e... pada keduanya — identik

Tabel 1.15: Log serial Modul 01: Uji kompatibilitas board asli vs klon

Perilaku dengan peran ditukar antar-jenis board, 26 detik per konfigurasi:

Konfigurasi	Paket	Hilang	RSSI rata-rata
TX asli (ttyACM0) → RX klon (ttyUSB0)	12	1 (nomor 0)	-54,0 dBm
TX klon (ttyUSB0) → RX asli (ttyACM0)	12	0	-53,8 dBm

Tabel 1.16: Log serial Modul 01: Uji kompatibilitas board asli vs klon

Satu-satunya paket hilang adalah **#0**, yaitu paket yang dikirim ketika penerima masih menyelesaikan boot — bukan gejala ketidakcocokan board. Membuka port menyebabkan kedua board reset bersamaan, dan pengirim siap $\pm 0,2$ detik lebih cepat daripada penerima. Inilah alasan README meminta penerima diunggah lebih dahulu, dan alasan paket pertama sebaiknya tidak dihitung dalam pengukuran.

Kesimpulan: **tidak ada penyesuaian kode apa pun** yang diperlukan untuk board klon. Yang berbeda hanya nama port di sistem operasi.

Catatan pengambilan log

- Arduino Uno melakukan reset otomatis setiap kali port serial dibuka, karena jalur DTR terhubung ke pin RESET lewat kapasitor. Sifat ini dipakai agar pesan `setup()` ikut terekam tanpa menekan tombol reset.
- `monitor_serial.py` sempat gagal membuka port kedua Uno asli dengan [Errno 110] Connection timed out. Penyebabnya, skrip menyetel DTR/RTS **sebelum** `open()` — hal yang ditolak CDC ATmega16U2 pada Uno asli, tetapi kebetulan lolos pada klon ber-bridge CH340 yang dipakai di sesi sebelumnya. Skrip diperbaiki agar tidak menyentuh jalur itu sama sekali, dan perbaikannya diterapkan ke seluruh salinan monitor pada Modul 01–04.
- Baris [TX] ... terkirim **bukan** bukti paket mengudara. Program mencetaknya setelah `LoRa.endPacket()` kembali, tanpa memeriksa status radio. Pada board yang `LoRa.begin()`-nya gagal, baris itu tetap muncul.
- Seluruh perubahan sumber pada EXP-03 dikembalikan ke kondisi semula setelah pengujian, dan baseline diukur ulang untuk memastikannya.
- Tabel pengukuran jarak, penghalang, dan spreading factor pada README belum terisi: percobaan ini dijalankan di satu meja pada jarak tetap ± 30 cm.

1.9 Pengukuran

A. Jarak terhadap kualitas tautan — sender dan receiver dipisahkan pada empat jarak, masing-masing diamati 60 detik.

Jarak	RSSI (dBm)	SNR (dB)	Dikirim	Diterima	Loss (%)
1 m					
10 m					
50 m					
100 m					

Tabel 1.17: Lembar pengukuran A — jarak terhadap kualitas tautan

B. Pengaruh penghalang — pada jarak tetap 10 m.

Kondisi	RSSI (dBm)	SNR (dB)	Loss (%)
Garis pandang bebas			
Terhalang satu dinding			

Kondisi	RSSI (dBm)	SNR (dB)	Loss (%)
Terhalang dua dinding			
Antena menempel di logam			

Tabel 1.18: Lembar pengukuran B — pengaruh penghalang pada jarak 10 m

C. Pengaruh spreading factor — ubah SF pada **kedua** board, ukur pada jarak yang sama.

SF	Waktu udara per paket (perkiraan)	RSSI (dBm)	SNR (dB)	Loss (%)
7				
9				
12				

Tabel 1.19: Lembar pengukuran C — pengaruh spreading factor

1.10 Analisis

1. Bagaimana hubungan jarak terhadap RSSI pada data Tabel 1.17? Apakah penurunannya sebanding dengan jarak, dan mengapa demikian?
2. Pada baris mana RSSI masih baik tetapi SNR sudah memburuk? Jelaskan apa yang terjadi secara fisik pada kondisi itu.
3. Dari Tabel 1.19, apa yang dipertukarkan ketika spreading factor dinaikkan? Kaitkan dengan waktu udara dan konsumsi energi.
4. Tiga parameter pada EXP-03 membuat komunikasi gagal total, satu lagi tidak berpengaruh sama sekali. Kelompokkan keempatnya berdasarkan apakah parameter itu menentukan bentuk gelombang atau hanya cara payload dikodekan, lalu jelaskan mengapa pengelompokan itu memprediksi hasilnya.
5. Modul ini tidak memiliki alamat maupun identitas node. Sebutkan dua masalah yang pasti muncul bila ada tiga board menyala bersamaan dengan firmware yang sama.

1.11 Concept Check

1. Apa perbedaan RSSI dan SNR, dan mengapa keduanya dicatat bersama?
2. Mengapa antena wajib terpasang sebelum shield diberi daya?
3. Apa arti “TX blocking” pada `LoRa.endPacket()`, dan apa yang tidak dapat dikerjakan Arduino selama pengiriman berlangsung?
4. D13 dipakai sebagai SCK sekaligus LED bawaan. Apa akibatnya bila LED bawaan dipakai sebagai indikator komunikasi?
5. Mengapa LoRa pada modul ini disebut LoRa mentah dan bukan LoRaWAN? Sebutkan satu hal yang disediakan LoRaWAN tetapi tidak ada di sini.

1.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Membuat penerima menghitung sendiri

CH-1 — Hitung loss otomatis Uraikan nomor urut dari payload di penerima, bandingkan dengan nomor sebelumnya, dan cetak LOSS: n paket setiap kali terjadi lompatan.

CH-2 — Statistik berjalan Tampilkan RSSI minimum, maksimum, dan rata-rata pada penerima setiap 10 paket.

Level 3 — Tantangan · Mencari batas tautan

CH-3 — Uji jangkauan maksimum Turunkan `setTxPower()` bertahap dari 17 dBm ke 2 dBm pada jarak tetap, catat pada daya berapa paket mulai hilang, lalu bandingkan dengan hasil menaikkan SF.

CH-4 — Payload lebih besar Naikkan panjang payload bertahap sampai 200 byte, ukur pengaruhnya terhadap loss dan waktu kirim. Jelaskan mengapa paket panjang lebih rapuh.

1.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — parameter radio, RSSI vs SNR, polling.
3. Konfigurasi — pemetaan pin shield, nilai SF/BW/CR/frekuensi, environment PlatformIO.
4. Hasil eksperimen — log serial kedua board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 1.17, Tabel 1.18, dan Tabel 1.19.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri berdasarkan hasil pengujian.

2. Modul 02 — Penerimaan Non-Blocking dan Indikator LED

LoRa COMMUNICATION LAB · MODUL 02

Stop Waiting for Packets — Penerimaan Non-Blocking dan Indikator LED

Arduino Uno + Dragino LoRa Shield v1.2 · LoRa mentah · satu arah · RX interrupt + flag · level Basic · 2 × 50 menit · folder kode Modul02_lora_led_notif

PENGANTAR MODUL · Gambaran Umum

Modul 02 dirancang untuk dua pertemuan (2 × 50 menit) pada tingkat dasar. Misinya membebaskan `loop()` penerima dari tugas menunggu: paket dikabarkan oleh interrupt, bukan dicari terus-menerus. Sebagai bukti visual bahwa radio benar-benar bekerja, kedua board menyalakan LED saat mengirim dan menerima. Percobaan memakai dua Arduino Uno bershield Dragino LoRa v1.2, diamati melalui dua Serial Monitor pada 9600 baud.

2.1 Pendahuluan

M01 memakai polling: penerima memanggil `parsePacket()` berulang kali, sehingga seluruh waktu prosesor habis untuk bertanya “sudah ada paket belum”. Selama `loop()` sibuk mengerjakan hal lain, paket yang tiba akan terlewat. Modul ini memindahkan pemberitahuan ke jalur perangkat keras: pin DIO0 SX1276 menarik interrupt INT0 Arduino begitu paket selesai diterima, dan `loop()` cukup memeriksa sebuah penanda. Pola inilah yang membuat penerima tetap responsif sambil mengerjakan tugas lain — dan menjadi dasar M04, tempat penerima harus menunggu ACK sambil menghitung waktu.

Prasyaratnya adalah M01: inisialisasi SX1276, parameter radio, serta pembacaan RSSI dan SNR. Yang dibangun di sini adalah pemasangan callback `LoRa.onReceive()`, pemakaian variabel `volatile` sebagai jembatan antara ISR dan `loop()`, pemahaman mengapa pekerjaan berat tidak boleh dilakukan di dalam ISR, serta indikator LED yang tidak mengganggu jalur SPI. Semuanya dipakai lagi pada M04 ketika ACK ditunggu dengan batas waktu, dan M05 ketika master menjadwalkan giliran bicara.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02 (ini)	Penerimaan lewat interrupt — <code>loop()</code> tidak lagi menunggu
03	Dua arah — kedua board bergantian mengirim dan menerima
04	Keandalan diukur: ACK, timeout, dan hitungan gagal
05	Banyak node — satu master menjadwalkan giliran bicara

Tabel 2.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Payload tetap bernomor (Hello #n), sama seperti M01, sehingga hasil pengukuran kedua modul dapat dibandingkan langsung. Yang berubah bukan datanya, melainkan **cara penerima mengetahui data itu tiba**.

2.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Memindahkan penerimaan dari polling ke interrupt

1. Menjelaskan perbedaan polling dan interrupt pada penerimaan paket LoRa, beserta jalur perangkat kerasnya (DIO0 → D2/INT0).
2. Memasang `LoRa.onReceive()` dan `LoRa.receive()` dengan benar, serta menjelaskan mengapa `LoRa.receive()` harus dipanggil ulang setelah sebuah paket diolah.
3. Menjelaskan fungsi kata kunci `volatile` dan akibatnya bila dihilangkan.
4. Menyebutkan pekerjaan yang tidak boleh dilakukan di dalam ISR beserta alasannya.
5. Menjelaskan mengapa LED indikator sebaiknya tidak memakai D13 pada board bershield LoRa.

Kriteria keberhasilan

- ☐ Penerima mencetak paket tanpa memanggil `parsePacket()` sama sekali di `loop()`.
- ☐ LED berkedip pada pengirim setiap kali TX selesai, dan pada penerima setiap kali paket tiba.
- ☐ Penerima tetap menerima paket meskipun `loop()` diberi pekerjaan tambahan (EXP-03).
- ☐ Perbedaan perilaku polling dan interrupt dibuktikan dengan data, bukan sekadar dijelaskan.

2.3 Dasar Teori (Secukupnya)

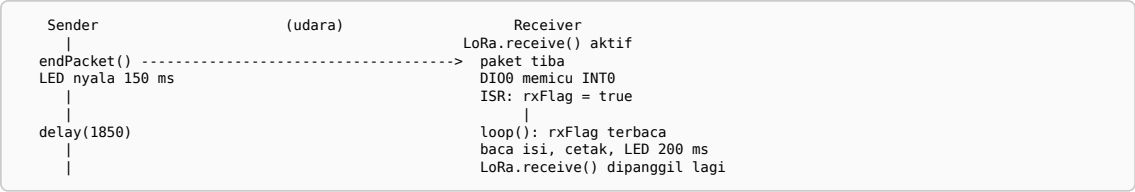
Istilah	Definisi kerja di lab ini
Interrupt	Sinyal perangkat keras yang menghentikan sementara program utama untuk menjalankan fungsi khusus.
ISR	<i>Interrupt Service Routine</i> — fungsi yang dijalankan saat interrupt terjadi. Harus sangat singkat.
DIO0	Pin keluaran SX1276 yang berubah keadaan saat paket selesai diterima atau dikirim. Tersambung ke D2 (INT0) Arduino.
<code>volatile</code>	Penanda bagi compiler bahwa sebuah variabel dapat berubah di luar alur program utama, sehingga nilainya tidak boleh disimpan di register.
Mode receive kontinu	Keadaan SX1276 setelah <code>LoRa.receive()</code> : radio terus mendengarkan tanpa campur tangan prosesor.
Flag pattern	Pola baku: ISR hanya menyalakan penanda, seluruh pekerjaan berat dikerjakan <code>loop()</code> .

Tabel 2.2: Istilah kerja Modul 02

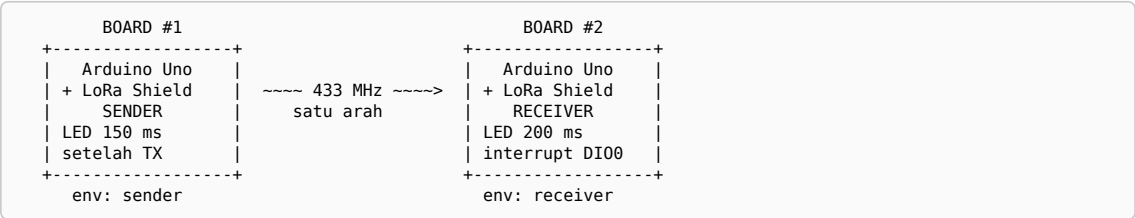
Mengapa ISR harus singkat. Selama ISR berjalan, interrupt lain tertunda dan `millis()` berhenti bertambah pada AVR. Mencetak ke Serial di dalam ISR — operasi yang memakan milidetik — dapat membuat pewaktuan kacau, paket berikutnya terlewat, bahkan program membeku. Karena itu `onReceive()` pada modul ini hanya mengerjakan satu hal: `rxFlag = true`. Pembacaan isi paket, pencetakan, dan penyalan LED semuanya dikerjakan `loop()` setelah interrupt selesai.

Mengapa bukan D13 untuk LED. D13 berbagi jalur dengan SCK, denyut clock SPI menuju SX1276. Setiap kali radio berkomunikasi, LED bawaan ikut berkedip dengan sendirinya, sehingga tidak dapat dipercaya sebagai indikator kejadian. Program ini memakai `LED_BUILTIN` agar dapat langsung dijalankan tanpa komponen tambahan, tetapi untuk indikator yang bersih ganti `LED_PIN` menjadi **D3** dan pasang LED dengan resistor 220 Ω ke GND — hal ini menjadi bagian dari EXP-02.

Sekuens yang diamati



2.4 Topologi



Node	Environment	Peran	Mekanisme RX	LED
Sender	sender	Pengirim, Hello #n tiap 2 detik	—	Kedip 150 ms setelah TX
Receiver	receiver	Penerima	Interrupt DIO0 + rxFlag	Kedip 200 ms saat RX

Tabel 2.3: Peran tiap node Modul 02

2.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	2
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
3	Antena SMA	wajib terpasang sebelum diberi daya	2
4	LED + resistor 220 Ω	opsional, untuk indikator bersih di D3	2
5	Kabel USB tipe B	kabel data	2

Tabel 2.4: Alat dan bahan Modul 02

Pin yang dipakai

Pin	Fungsi	Keterangan
D10, D11, D12, D13	SPI ke SX1276	NSS, MOSI, MISO, SCK
D9	RST	Reset SX1276
D2	DIO0	Interrupt RX-done — inti modul ini
D13	LED bawaan	Dipakai program sebagai indikator awal, berbagi jalur dengan SCK
D3	LED eksternal	Pin bebas yang direkomendasikan untuk indikator bersih

Tabel 2.5: Pemetaan pin Modul 02

Struktur proyek

```

Modul02_lora_led_notif/
├── platformio.ini
├── monitor_serial.py      ← pantau TX & RX sekaligus, ringkas loss/RSSI/SNR
├── upload_auto.py        ← deteksi port otomatis saat unggah
├── logserial.md           ← log referensi hasil uji perangkat
└── src/
    ├── sender/main.cpp    ← kirim + kedip LED setelah TX
    └── receiver/main.cpp  ← interrupt DI00, rxFlag, kedip LED saat RX

```

2.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul02_lora_led_notif/](#)

- [Modul02_lora_led_notif/platformio.ini](#)
- [Modul02_lora_led_notif/src/sender/main.cpp](#)
- [Modul02_lora_led_notif/src/receiver/main.cpp](#)
- [Modul02_lora_led_notif/monitor_serial.py](#)
- [Modul02_lora_led_notif/upload_auto.py](#)

```

[platformio]
default_envs = receiver

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 9600

[env:sender]
build_src_filter = +<sender/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:receiver]
build_src_filter = +<receiver/*.cpp>
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

```

Kode 2.1: platformio.ini Modul 02 — environment sender dan receiver

```

/*
LoRa Sender + LED Notif - Dragino LoRa Shield v1.2 + Arduino Uno
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: sender

Pin Mapping Dragino Shield:
NSS/CS -> D10 (R9 0-ohm populated)
DI00    -> D2
RST     -> D9
SCK     -> D13 / ICSP, MOSI -> D11, MISO -> D12

LED:
LED_BUILTIN (D13) = built-in LED Arduino, tapi shared dengan SPI SCK.
LED akan berkedip saat SPI aktif - untuk notifikasi setelah TX selesai, OK.
Ganti LED_PIN ke D3 jika ingin LED eksternal tanpa gangguan SPI.
*/

#include <Arduino.h>
#include <SPI.h>

```



```

#include <LoRa.h>

#define NSS_PIN    10
#define RST_PIN    9
#define DI00_PIN   2

#define FREQUENCY    433E6
#define BANDWIDTH     125E3
#define SPREADING_FACTOR 7
#define CODING_RATE   5
#define TX_POWER      17

#define LED_PIN LED_BUILTIN // D13. Ganti ke 3 untuk LED eksternal di D3.

int counter = 0;

void setup() {
  Serial.begin(9600);
  while (!Serial);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.println(F("=== LoRa SENDER (Dragino) ==="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
  }

  LoRa.setSpreadingFactor(SPREADING_FACTOR);
  LoRa.setSignalBandwidth(BANDWIDTH);
  LoRa.setCodingRate4(CODING_RATE);
  LoRa.setTxPower(TX_POWER);

  Serial.println(F("OK"));
  Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.print(F(" MHz | "));
  Serial.print(F("BW: ")); Serial.print(BANDWIDTH / 1E3); Serial.print(F(" kHz | "));
  Serial.print(F("SF: ")); Serial.print(SPREADING_FACTOR); Serial.print(F(" | "));
  Serial.print(F("Power: ")); Serial.print(TX_POWER); Serial.println(F(" dBm"));
  Serial.println(F("Mulai kirim tiap 2 detik...\n"));
}

void loop() {
  String msg = "Hello #" + String(counter);

  Serial.print(F("[TX] Kirim: \n"));
  Serial.print(msg);
  Serial.print(F("\n ... "));

  LoRa.beginPacket();
  LoRa.print(msg);
  LoRa.endPacket(); // blocking – SPI aktif sampai sini

  // LED blink setelah TX selesai (SPI sudah idle)
  digitalWrite(LED_PIN, HIGH);
  delay(150);
  digitalWrite(LED_PIN, LOW);

  Serial.println(F("OK"));
  counter++;

  delay(1850); // total interval ~2 detik
}

```

Kode 2.2: src/sender/main.cpp — kirim Hello #n disertai kedip LED setelah TX

```

/*
  LoRa Receiver Non-Blocking + LED Notif - Dragino LoRa Shield v1.2 + Arduino Uno
  Library : LoRa by sandeepmistry v0.8.x
  Environment PlatformIO: receiver

  Interrupt-driven receive via DI00 -> D2 (INT0 Arduino Uno).
  Loop() tidak terblokir – bisa kerjakan tugas lain sambil menunggu paket.

  Pin Mapping Dragino Shield:

```

```

NSS/CS -> D10 (R9 0-ohm populated)
DI00 -> D2 (interrupt RX-done)
RST -> D9
SCK -> D13 / ICSP, MOSI -> D11, MISO -> D12

LED:
LED_BUILTIN (D13) shared dengan SPI SCK – nyala sebentar saat paket tiba.
Ganti LED_PIN ke 3 untuk LED eksternal di D3 (tanpa gangguan SPI).
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN 10
#define RST_PIN 9
#define DI00_PIN 2

#define FREQUENCY 433E6
#define BANDWIDTH 125E3
#define SPREADING_FACTOR 7
#define CODING_RATE 5

#define LED_PIN LED_BUILTIN // D13. Ganti ke 3 untuk LED eksternal di D3.

volatile bool rxFlag = false;

void onReceive(int packetSize) {
    if (packetSize > 0) rxFlag = true;
}

void setup() {
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.println(F("=== LoRa RECEIVER (Dragino) ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);

    LoRa.onReceive(onReceive); // pasang callback ISR via DI00 (D2)
    LoRa.receive();           // masuk mode receive kontinyu (non-blocking)

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.print(F(" MHz | "));
    Serial.print(F("BW: ")); Serial.print(BANDWIDTH / 1E3); Serial.print(F(" kHz | "));
    Serial.print(F("SF")); Serial.println(SPREADING_FACTOR);
    Serial.println(F("Menunggu paket (non-blocking)...\n"));
}

void loop() {
    if (rxFlag) {
        rxFlag = false;

        digitalWrite(LED_PIN, HIGH); // LED nyala – paket diterima

        String received = "";
        while (LoRa.available()) {
            received += (char)LoRa.read();
        }

        Serial.println(F("====="));
        Serial.print(F("[RX] Pesan : "));
        Serial.println(received);
        Serial.print(F("[RX] RSSI : "));
        Serial.print(LoRa.packetRssi());
        Serial.println(F(" dBm"));
        Serial.print(F("[RX] SNR : "));
        Serial.print(LoRa.packetSnr());
        Serial.println(F(" dB"));
        Serial.println(F("=====\\n"));
    }
}

```

```

    delay(200);                // LED nyala 200ms
    digitalWrite(LED_PIN, LOW);

    LoRa.receive();            // kembali ke mode receive
}

// Di sini bisa tambah kode lain – loop tidak terblokir saat menunggu paket
}

```

Kode 2.3: src/receiver/main.cpp — interrupt DIO0, rxFlag, kedip LED saat RX

```

#!/usr/bin/env python3
"""Monitor pengirim dan penerima LoRa sekaligus untuk Modul 02.

Menampilkan kedua aliran serial dalam SATU jendela dengan timestamp bersama,
lalu meringkasnya saat berhenti: berapa paket dikirim, berapa yang tiba, dan
sebaran RSSI serta SNR-nya. Membandingkan nomor urut di dua jendela terpisah
jauh lebih sulit, karena tiap jendela punya sumbu waktunya sendiri.

Pada modul ini ringkasan tersebut dipakai untuk membuktikan keunggulan
penerimaan berbasis interrupt: jalankan monitor sambil `loop()` penerima
diberi pekerjaan tiruan (EXP-03), lalu bandingkan angka paket hilangnya
dengan penerima polling Modul 01 pada pekerjaan tiruan yang sama.

python3 Modul02_lora_led_notif/monitor_serial.py
python3 Modul02_lora_led_notif/monitor_serial.py --log sesil.txt
python3 Modul02_lora_led_notif/monitor_serial.py --port RX=/dev/ttyACM0
python3 Modul02_lora_led_notif/monitor_serial.py --durasi 60 # berhenti sendiri

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C, atau pakai `--durasi` agar berhenti otomatis.

PORT OTOMATIS. Port TX/RX tidak lagi ditulis tetap di kode: setiap kali
dijalankan, skrip memindai /dev/ttyACM* dan /dev/ttyUSB* yang sedang aktif
dan memakai yang benar-benar tersambung, urut sesuai nama device. Ini
penting karena nama port bisa berubah tiap kali board dicabut-pasang atau
board asli/klon bercampur (lihat tools/deteksi_port.py). Bila urutan
otomatisnya keliru (mis. board TX ternyata terdeteksi sebagai RX), timpa
lewat --port, mis. --port TX=/dev/ttyUSB0 --port RX=/dev/ttyACM1.

SOAL RESET OTOMATIS. Pada Arduino Uno, jalur DTR terhubung ke pin RESET lewat
kapasitor. Membuka port serial **selalu me-reset board**, dan hal itu justru
dimanfaatkan di sini: pesan `setup()` beserta baris `Init LoRa ... OK` ikut
terekam tanpa perlu menekan tombol reset. Konsekuensinya, penghitung paket
kedua board kembali ke nol setiap kali monitor dijalankan – jalankan monitor
lebih dahulu, baru mulai mengukur, dan jangan membukanya di tengah percobaan
panjang.

Pada Uno, DTR hanya menyentuh jalur RESET dan tidak tersambung ke GPIO mana pun,
sehingga membuka port tidak pernah membuat sebuah pin terbaca seolah ditekan.
"""
import argparse
import glob
import re
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

# Nama peran bawaan (urutan) dan warna yang dipasangkan ke port yang
# terdeteksi aktif, bukan ke nama /dev tertentu – lihat deteksi_port_aktif().
NAMA_PERAN_BAWAAN = ["TX", "RX"]
PALET_WARNA = ["\033[33m", "\033[36m", "\033[35m", "\033[32m", "\033[34m"]

def deteksi_port_aktif():
    """Kumpulkan port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* – pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil ulang tiap program start supaya daftarnya selalu mengikuti apa
    yang benar-benar tersambung saat itu, bukan port tetap di kode.
    """

```

```

"""
return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def bangun_port_bawaan():
    """Pasangkan tiap port aktif ke nama peran (TX, RX, lalu P3, P4, ...)."""
    hasil = []
    for i, dev in enumerate(deteksi_port_aktif()):
        nama = NAMA_PERAN_BAWAAN[i] if i < len(NAMA_PERAN_BAWAAN) else f"P{i + 1}"
        warna = PALET_WARNA[i % len(PALET_WARNA)]
        hasil.append((nama, dev, warna))
    return hasil

# Pola yang dipanen dari baris serial untuk menyusun ringkasan
RE_NOMOR = re.compile(r"##(\d+)")
RE_RSSI = re.compile(r"RSSI\s*:\s*(-?\d+)")
RE_SNR = re.compile(r"SNR\s*:\s*(-?\d.]+)")
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
# Bahan ringkasan akhir; diisi reader, dibaca sekali saat program berhenti
nomor_tx, nomor_rx, daftar_rssi, daftar_snr = [], [], [], []
stop = threading.Event()
t0 = time.time()

def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"{DIM}[{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()

def open_port(port, baud):
    """Buka port serial menuju board Arduino.

    Jalur DTR/RTS sengaja TIDAK disentuh. Menyetelnya sebelum open() ditolak
    oleh CDC ATmega16U2 pada Uno asli dan berakhir sebagai
    `[Errno 110] Connection timed out`; pada klon ber-bridge CH340 hal itu
    kebetulan lolos, sehingga kesalahannya mudah tidak terlihat bila diuji
    hanya pada satu jenis board.

    Membuka port tetap me-reset board sekali - itu memang yang diinginkan,
    karena pesan setup() ikut terekam - dan reset itu berupa pulsa dari
    kapasitor, bukan penahanan, sehingga board langsung berjalan normal.
    """
    return serial.Serial(port, baud, timeout=0.2)

def panen(name, text):
    """Ambil nomor paket, RSSI, dan SNR dari sebuah baris serial.

    Nomor dipanen dari kedua sisi supaya paket yang hilang dapat dihitung
    sebagai selisih, bukan diperkirakan dari jumlah baris.
    """
    m = RE_NOMOR.search(text)
    if m:
        (nomor_tx if name == "TX" else nomor_rx).append(int(m.group(1)))
    if name != "TX":
        if (m := RE_RSSI.search(text)):
            daftar_rssi.append(int(m.group(1)))
        if (m := RE_SNR.search(text)):
            daftar_snr.append(float(m.group(1)))

def ringkasan():
    """Cetak hasil ukur tautan: loss, RSSI, dan SNR."""
    print("\n" + "-" * 60)
    print(f"Durasi: {time.time() - t0:.1f} s")
    for nama in counts:
        print(f" {nama:<7} : {counts.get(nama, 0)} baris")

    if not nomor_tx and not nomor_rx:

```

```

        print("\nTidak ada paket bernomor yang terbaca – periksa tautan.")
        return

    kirim, terima = set(nomor_tx), set(nomor_rx)
    print(f"\n  Paket dikirim   : {len(kirim)}"
          + (f"   (nomor {min(kirim)}..{max(kirim)})" if kirim else ""))
    print(f"  Paket diterima : {len(terima)}"
          + (f"   (nomor {min(terima)}..{max(terima)})" if terima else ""))

    # Hanya nomor yang benar-benar terlihat dikirim yang dihitung sebagai hilang,
    # sehingga paket sebelum monitor dijalankan tidak ikut dianggap gagal.
    if kirim:
        hilang = sorted(kirim - terima)
        loss = len(hilang) / len(kirim) * 100
        print(f"  Paket hilang   : {len(hilang)} ({loss:.1f} %)")
        if hilang:
            print(f"    Nomor hilang   : {' '.join(str(n) for n in hilang[:20])}"
                  + (" ..." if len(hilang) > 20 else ""))

    if daftar_rssi:
        print(f"  RSSI  min/maks/rata-rata : {min(daftar_rssi)} / {max(daftar_rssi)} / "
              f"{sum(daftar_rssi) / len(daftar_rssi):.1f} dBm")
    if daftar_snr:
        print(f"  SNR   min/maks/rata-rata : {min(daftar_snr):.2f} / {max(daftar_snr):.2f} / "
              f"{sum(daftar_snr) / len(daftar_snr):.2f} dB")

def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip
        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:
            text = line.decode("utf-8", "replace").rstrip("\r")
            if text.strip():
                counts[name] += 1
                panen(name, text)
                show(name, color, text, use_color, logfile)
    ser.close()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyXXX",
                    help="ganti port TX atau RX, mis. RX=/dev/ttyACM0; boleh diulang")
    # Modul 01-04 memakai 9600; Modul 05 memakai 115200.
    ap.add_argument("--baud", type=int, default=9600)
    ap.add_argument("--durasi", type=float, metavar="DETIK",
                    help="berhenti otomatis setelah sekian detik (mis. --durasi 60)")
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    port_bawaan = bangun_port_bawaan()
    if not port_bawaan and not args.port:
        sys.exit("Tidak ada port serial USB yang terdeteksi (/dev/ttyACM*, /dev/ttyUSB*). \n"
                 "Periksa kabel data (bukan charge-only) dan hak akses grup dialout, \n"
                 "atau jalankan tools/deteksi_port.py untuk diagnosis lebih lanjut, \n"
                 "atau tentukan port manual lewat --port TX=/dev/ttyUSB0 --port RX=/dev/ttyACM1")

    dipakai = port_bawaan[:len(NAMA_PERAN_BAWAAN)]
    sisa = port_bawaan[len(NAMA_PERAN_BAWAAN):]
    print(f"Port aktif terdeteksi: {' '.join(p for _, p, _ in port_bawaan)}"
          + (f" if port_bawaan else \"Tidak ada port aktif terdeteksi otomatis – memakai --port saja.\""))

```

```

for nama, port, _ in dipakai:
    print(f" -> {nama} = {port}")
if sisa:
    print(f" (tidak dipakai: {' '.join(p for _, p, _ in sisa)})"
          f" - pakai --port NAMA=/dev/... untuk memakainya)")

ports = {name: (port, color) for name, port, color in dipakai}
for item in args.port or []:
    if "=" not in item:
        sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyXXX)")
    name, port = item.split("=", 1)
    _, color = ports.get(name.upper(), (None, "\033[35m"))
    ports[name.upper()] = (port, color)

if not ports:
    sys.exit("Tidak ada port untuk dipantau - tentukan lewat --port NAMA=/dev/ttyXXX")

use_color = not args.no_color and sys.stdout.isatty()
logfile = open(args.log, "w") if args.log else None

# SIGTERM (mis. dijalankan lewat `timeout 30 ...`) cukup menyalakan flag
# `stop` yang sama dipakai reader thread, BUKAN melempar exception --
# melempar exception dari signal handler bisa mendarat di tengah blok
# `finally` (mis. saat t.join() atau ringkasan() berjalan) dan menyebabkan
# crash tak tertangkap alih-alih mencetak ringkasan dengan rapi.
signal.signal(signal.SIGTERM, lambda *_: stop.set())

batas = f" · berhenti otomatis {args.durasi:.0f} s" if args.durasi else ""
print(f"Monitor LoRa TX/RX @ {args.baud} baud - Ctrl-C untuk berhenti{batas}")
f"{' · log: ' + args.log if args.log else ''}"
print("-" * 60)

threads = [threading.Thread(target=reader,
                             args=(name, port, color, args.baud, use_color, logfile),
                             daemon=True)
            for name, (port, color) in ports.items()]
for t in threads:
    t.start()

batasWaktu = t0 + args.durasi if args.durasi else None
try:
    while any(t.is_alive() for t in threads) and not stop.is_set():
        if batasWaktu and time.time() >= batasWaktu:
            break
        time.sleep(0.2)
except KeyboardInterrupt:
    pass
finally:
    stop.set()
    for t in threads:
        t.join(timeout=1.0)
    ringkasan()
    if logfile:
        logfile.close()
        print(f"\nLog tersimpan di {args.log}")

if __name__ == "__main__":
    main()

```

Kode 2.4: monitor_serial.py — pantau TX dan RX pada satu sumbu waktu

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 02 (LoRa LED Notifikasi) - port serial dideteksi sendiri.

```

Alternatif untuk menjalankan manual:

```

pio run -d Modul02_lora_led_notif -e sender -t upload -t monitor
pio run -d Modul02_lora_led_notif -e receiver -t upload -t monitor

```

Skrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya ke environment `sender` (port pertama) dan `receiver` (port kedua), lalu menjalankan `pio run` dengan --upload-port sesuai hasil deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua board selesai di-upload, skrip bertanya apakah mau langsung menjalankan monitor_serial.py -- bila ya, dijalankan dengan --port TX=... --port RX=... persis memakai port sender/receiver yang barusan dipakai upload (bukan

```

deteksi ulang), supaya tidak mungkin salah pasang.

python3 Modul02_lora_led_notif/upload_auto.py
-> upload kedua board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul02_lora_led_notif/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk sender, lanjut receiver
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti monitor_serial.py

python3 Modul02_lora_led_notif/upload_auto.py --only sender
-> upload satu environment saja

python3 Modul02_lora_led_notif/upload_auto.py --sender /dev/ttyUSB0 --receiver /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["sender", "receiver"]
# Nama environment PlatformIO -> nama peran yang dipakai monitor_serial.py
ENV_KE_PERAN = {"sender": "TX", "receiver": "RX"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan monitor_serial.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan monitor_serial.py
    dengan port sender/receiver PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang barusan
    di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan monitor_serial.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "monitor_serial.py")]
    for env, peran in ENV_KE_PERAN.items():
        if env in port_env:
            cmd += ["--port", f"{peran}={port_env[env]}"]

    print(" $", " ".join(cmd))
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) monitor_serial.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

```

```

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--sender", metavar="/dev/ttyXXX",
                    help="tiapa port sender (lewati deteksi otomatis)")
    ap.add_argument("--receiver", metavar="/dev/ttyXXX",
                    help="tiapa port receiver (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (sender atau receiver)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                         "(bawaan: upload saja -- pakai monitor_serial.py untuk memantau keduanya "
                         "sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"sender": args.sender, "receiver": args.receiver}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f"  -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f"  -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"  -> sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 2.5: upload_auto.py — pilih port otomatis saat unggah

2.7 Build dan Flash

Penerima diunggah lebih dahulu.

```

pio run -d Modul02_lora_led_notif -e receiver -t upload -t monitor
pio run -d Modul02_lora_led_notif -e sender -t upload -t monitor

```

Memantau kedua board sekaligus. Percobaan inti modul ini (EXP-03) menuntut perbandingan jumlah paket hilang antara dua firmware penerima. Menghitungnya dari dua jendela terpisah rawan salah, karena tiap jendela punya sumbu waktu sendiri. Skrip `monitor_serial.py` menggabungkan keduanya dan langsung meringkas hasil ukurnya.

```

python3 Modul02_lora_led_notif/monitor_serial.py
python3 Modul02_lora_led_notif/monitor_serial.py --durasi 40 --log sesil.txt

```



```
Paket dikirim : 9 (nomor 0..8)
Paket diterima : 9 (nomor 0..8)
Paket hilang : 0 (0.0 %)
RSSI min/maks/rata-rata : -38 / -38 / -38.0 dBm
SNR min/maks/rata-rata : 9.00 / 9.50 / 9.28 dB
```

PENTING

Membuka monitor me-reset kedua board. Pada Arduino Uno, DTR terhubung ke pin RESET, sehingga penghitung paket kembali ke nol setiap monitor dijalankan. Jalankan monitor lebih dahulu, baru mulai mengukur.

Pre-flight checklist

- ☐ Antena terpasang pada kedua shield.
- ☐ Port kedua board dicatat lewat `pio device list` dan diisikan ke `platformio.ini`.
- ☐ Dua Serial Monitor 9600 baud siap.
- ☐ Bila memakai LED eksternal: kaki panjang ke D3 lewat resistor 220 Ω , kaki pendek ke GND.

2.8 Percobaan

2.8.1 EXP-01 — Penerimaan Tanpa Polling

Unggah kedua firmware dan amati aliran data seperti pada M01.

Expected output — receiver

```
=== LoRa RECEIVER (Dragino) ===
Init LoRa ... OK
Freq: 433.00 MHz | BW: 125.00 kHz | SF7
Menunggu paket (non-blocking)...

=====
[RX] Pesan : Hello #0
[RX] RSSI : -38 dBm
[RX] SNR : 9.50 dB
=====
```

Data capture

Parameter	Hasil
Apakah <code>parsePacket()</code> ada di <code>loop()</code> penerima?	
Nomor urut terakhir sender / receiver	
RSSI rata-rata (dBm)	
Lama kedip LED penerima (ms)	

Tabel 2.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA

Di `src/receiver/main.cpp`, cari fungsi `onReceive()`. Isinya hanya satu baris pemberian nilai. Jawab: mengapa pembacaan `LoRa.available()` dan pencetakan Serial tidak diletakkan di sana padahal itu terasa lebih ringkas? Lalu telusuri di mana `LoRa.receive()` dipanggil untuk kedua kalinya, dan jelaskan apa yang terjadi bila baris itu dihapus.

CHECKPOINT

Penerima mencetak paket **tanpa** satu pun panggilan `parsePacket()` di `loop()`. Bila baris itu masih ada, yang diuji bukan mekanisme interrupt.

2.8.2 EXP-02 — LED Sebagai Instrumen

Amati LED bawaan (D13) pada kedua board, lalu ganti LED_PIN menjadi 3, pasang LED eksternal, dan bandingkan.

Data capture

Parameter	LED bawaan (D13)	LED eksternal (D3)
Berkedip saat paket tiba?		
Berkedip juga saat radio diam?		
Dapat dipercaya sebagai indikator kejadian?		

Tabel 2.7: Lembar pengamatan EXP-02 — LED sebagai instrumen

CHECKPOINT

LED di D13 berkedip tidak beraturan mengikuti lalu lintas SPI, sedangkan LED di D3 hanya berkedip saat kejadian yang dimaksud. Pengamatan inilah alasan pin indikator dipisahkan dari jalur komunikasi.

2.8.3 EXP-03 — Bukti Non-Blocking

Tambahkan pekerjaan tiruan pada `loop()` penerima, tepat sebelum pemeriksaan `rxFlag`, seperti pada Kode 2.6.

```
// pekerjaan tiruan: seolah-olah penerima sedang mengolah sensor
delay(1500);
```

Kode 2.6: Pekerjaan tiruan yang disisipkan pada `loop()` penerima

Unggah ulang, amati 2 menit, lalu ulangi percobaan yang sama pada penerima **M01** yang berbasis polling.

Data capture

Firmware	Paket dikirim	Paket diterima	Loss (%)
M02 (interrupt) + <code>delay(1500)</code>			
M01 (polling) + <code>delay(1500)</code>			

Tabel 2.8: Lembar pengamatan EXP-03 — bukti non-blocking

Gunakan **pengirim yang sama** untuk kedua pengukuran, dan ganti firmware penerimanya saja — dengan begitu satu-satunya variabel adalah mekanisme penerimaan.

CHECKPOINT

Penerima M02 tetap menangkap **seluruh** paket pada `delay(1500)`, sedangkan penerima M01 kehilangan hampir semuanya. Selisih inilah nilai sesungguhnya dari mekanisme interrupt. Hapus `delay()` setelah percobaan selesai.

Angka rujukan (hasil ukur nyata, 40 detik per baris, pengirim tiap 2 detik — lihat `logserial.md`).

<code>delay()</code> di <code>loop()</code>	M02 interrupt	M01 polling
0 ms	0 %	0 %
500 ms	0 %	68,4 %
1500 ms	0 %	94,7 %
3000 ms	42,1 %	100 %

Tabel 2.9: Angka rujukan EXP-03 — loss interrupt dibanding polling

Perhatikan bahwa interrupt pun akhirnya runtuh, dan batasnya dapat dihitung: penahanan 3000 ms melampaui interval kirim 2000 ms, sehingga paket baru tiba sebelum paket sebelumnya diambil dan `LoRa.receive()` dipanggil ulang — SX1276 hanya menyimpan satu paket pada satu waktu. Interrupt memindahkan pemberitahuan ke perangkat keras, tetapi tidak membuat penerima kebal.

2.8.4 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada dua Arduino Uno bershield Dragino LoRa v1.2, 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

```
[ 1.813] RX | Init LoRa ... OK
[ 1.813] RX | Freq: 433.00 MHz | BW: 125.00 kHz | SF7
[ 2.013] RX | [RX] Pesan : Hello #0
[ 2.013] RX | [RX] RSSI  : -38 dBm
[ 2.022] TX | [TX] Kirim: "Hello #0" ... OK
```

Parameter	Hasil terukur
Paket dikirim / diterima (20 s)	9 / 9 (loss 0 %)
RSSI rata-rata	– 38,0 dBm
SNR rata-rata	9,28 dB
EXP-03 pada <code>delay(1500)</code>	interrupt 0 % vs polling 94,7 % hilang
EXP-02 (pengamatan LED)	belum diverifikasi — bersifat visual, tidak terekam serial

Tabel 2.10: Hasil verifikasi perangkat keras Modul 02

Environment	Status	Flash
sender	SUCCESS	22.9% (7380 B)
receiver	SUCCESS	24.5% (7916 B)

Penerima berbasis interrupt hanya menambah ± 600 byte dibanding penerima polling M01 (7312 B) — biaya yang sangat kecil dibanding manfaatnya.

2.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud 9600, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Jarak antar-board ± 30 cm.

Board & Port

Peran	Environment	Port	Board
Sender	sender	/dev/ttyUSB0	Uno klon, bridge CH340 (1a86:7523)
Receiver	receiver	/dev/ttyACM1	Uno asli (2341:0043)

Tabel 2.11: Log serial Modul 02: Board & Port

Board Uno asli di /dev/ttyACM0 tidak dipakai: shield-nya gagal `LoRa.begin()`. Diagnosisnya tercatat pada `../Modul01_lora_uart/logserial.md`.

Kedua aliran serial direkam bersamaan memakai `monitor_serial.py`, sehingga stempel waktunya berasal dari satu sumbu yang sama dan hitungan paket hilang berasal dari selisih nomor urut.

EXP-01 — Penerimaan Tanpa Polling

```
[ 1.813] RX | === LoRa RECEIVER (Dragino) ===
[ 1.813] RX | Init LoRa ... OK
[ 1.813] RX | Freq: 433.00 MHz | BW: 125.00 kHz | SF7
[ 1.813] RX | Menunggu paket (non-blocking)...
[ 1.822] TX | === LoRa SENDER (Dragino) ===
[ 1.822] TX | Init LoRa ... OK
[ 2.013] RX | =====
[ 2.013] RX | [RX] Pesan : Hello #0
[ 2.013] RX | [RX] RSSI : -38 dBm
[ 2.013] RX | [RX] SNR : 9.50 dB
[ 2.013] RX | =====
[ 2.022] TX | Freq: 433.00 MHz | BW: 125.00 kHz | SF7 | Power: 17 dBm
[ 2.022] TX | Mulai kirim tiap 2 detik...
[ 2.022] TX | [TX] Kirim: "Hello #0" ... OK
```

Parameter	Hasil
Apakah <code>parsePacket()</code> ada di <code>loop()</code> penerima?	tidak — hanya if (<code>rxFlag</code>)
Paket dikirim / diterima (20 s)	9 / 9 (loss 0 %)
RSSI: min / maks / rata-rata	-38 / -38 / -38,0 dBm
SNR: min / maks / rata-rata	9,00 / 9,50 / 9,28 dB
Flash sender / receiver	22,9 % (7.380 B) / 24,5 % (7.916 B)

Tabel 2.12: Log serial Modul 02: EXP-01 — Penerimaan Tanpa Polling

Penerima mencetak paket pertama **0,2 detik** setelah init, tanpa satu pun panggilan `parsePacket()` — pemberitahuan datang dari DIO0 lewat interrupt.

EXP-02 — LED Sebagai Instrumen

Belum diverifikasi. Pengamatan LED bersifat visual dan tidak muncul pada aliran serial, sehingga tidak dapat direkam oleh perangkat lunak. Yang dapat dipastikan dari kode: `LED_PIN` disetel ke `LED_BUILTIN`, yaitu **D13**, pin yang sama dengan SCK jalur SPI menuju SX1276. Perbandingan D13 dengan LED eksternal di D3 perlu diamati langsung dengan mata sesuai prosedur pada README.

EXP-03 — Bukti Non-Blocking

Percobaan inti modul ini. Pekerjaan tiruan `delay(n)` disisipkan pada awal `loop()` penerima, lalu jumlah paket hilang diukur selama 40 detik untuk tiap nilai.

Pengirim tidak diubah sama sekali — pengirim Modul 02 yang sama dipakai untuk kedua firmware penerima, sehingga satu-satunya variabel adalah mekanisme penerimaannya.

<code>delay()</code> di <code>loop()</code>	M02 interrupt — hilang	M01 polling — hilang
0 ms	0 / 19 (0 %)	0 / 19 (0 %)
500 ms	0 / 19 (0 %)	13 / 19 (68,4 %)
1500 ms	0 / 19 (0 %)	18 / 19 (94,7 %)

delay() di loop()	M02 interrupt — hilang	M01 polling — hilang
3000 ms	8 / 19 (42,1 %)	19 / 19 (100 %)

Tabel 2.13: Log serial Modul 02: EXP-03 — Bukti Non-Blocking

Data mentahnya:

```
EXP-03 — penerima INTERRUPT (M02):
delay=0      dikirim 19 (0..18) diterima 19 (0..18) hilang 0 (0.0 %)
delay=500    dikirim 19 (0..18) diterima 19 (0..18) hilang 0 (0.0 %)
delay=1500   dikirim 19 (0..18) diterima 19 (0..18) hilang 0 (0.0 %)
delay=3000   dikirim 19 (0..18) diterima 11 (1..17) hilang 8 (42.1 %)

EXP-03 — penerima POLLING (M01):
delay=0      dikirim 19 (0..18) diterima 19 (0..18) hilang 0 (0.0 %)
delay=500    dikirim 19 (0..18) diterima 6 (11..16) hilang 13 (68.4 %)
delay=1500   dikirim 19 (0..18) diterima 1 (11..11) hilang 18 (94.7 %)
delay=3000   dikirim 19 (0..18) diterima 0          hilang 19 (100.0 %)
```

Bacaan hasil

Pada beban ringan keduanya setara. Tanpa pekerjaan tiruan, kedua mekanisme menerima seluruh paket. Perbedaannya tidak akan terlihat bila diuji hanya pada kondisi ideal — inilah alasan percobaan ini memerlukan beban buatan.

Polling runtuh sejak beban terkecil. Dengan delay(500) saja, penerima polling sudah kehilangan 68 % paket, padahal pengirim hanya mengirim tiap 2 detik. Penyebabnya: `parsePacket()` hanya membaca paket yang **sedang** tersedia saat dipanggil. Selama `loop()` tertahan, paket yang tiba tidak diambil dari FIFO, dan paket berikutnya menyimpannya.

Interrupt bertahan sampai batas yang dapat dihitung. Penerima interrupt tetap sempurna sampai delay(1500), lalu runtuh pada delay(3000). Batas itu bukan kebetulan: pengirim mengirim tiap **2000 ms**, sehingga selama penahanan 1500 ms masih ada sela untuk memproses paket dan memanggil `LoRa.receive()` kembali. Pada penahanan 3000 ms, paket baru sudah tiba sebelum penerima sempat mengambil paket sebelumnya dan mempersenjatai ulang radio — dan SX1276 hanya menyimpan satu paket pada satu waktu.

Kesimpulan yang dapat diuji ulang: interrupt memindahkan pemberitahuan ke perangkat keras, tetapi **tidak** membuat penerima kebal. Batas ketahanannya kira-kira sepanjang interval kirim, bukan tak berhingga.

Catatan pengambilan log

- Seluruh penyisipan `delay()` pada EXP-03 dikembalikan ke kondisi semula setelah pengujian, dan firmware Modul 02 diunggah ulang untuk memastikan board kembali ke keadaan baku.
- Nilai Paket dikirim dihitung dari nomor urut yang benar-benar terlihat pada aliran serial pengirim, bukan diperkirakan dari lama pengujian.
- Baris [TX] Kirim: ... OK pada pengirim dicetak setelah `LoRa.endPacket()` kembali, tanpa memeriksa status radio — bukan bukti paket diterima siapa pun.
- Tabel pengukuran jarak pada README belum terisi: percobaan ini dijalankan di satu meja pada jarak tetap ± 30 cm.

2.9 Pengukuran

A. Ketahanan terhadap kesibukan `loop()` — ulangi EXP-03 dengan beberapa lama pekerjaan tiruan.

delay() pada loop	Loss M01 polling (%)	Loss M02 interrupt (%)
0 ms		
500 ms		
1500 ms		

delay() pada loop	Loss M01 polling (%)	Loss M02 interrupt (%)
3000 ms		

Tabel 2.14: Lembar pengukuran A — ketahanan terhadap kesibukan loop()

B. Jarak terhadap kualitas tautan — untuk dibandingkan langsung dengan lembar pengukuran A pada M01.

Jarak	RSSI (dBm)	SNR (dB)	Loss (%)
1 m			
10 m			
50 m			

Tabel 2.15: Lembar pengukuran B — jarak terhadap kualitas tautan

2.10 Analisis

1. Dari Tabel 2.14, pada lama `delay()` berapa perbedaan polling dan interrupt mulai menonjol? Jelaskan mengapa interrupt pun akhirnya ikut kehilangan paket.
2. Mengapa ISR hanya boleh mengubah penanda? Sebutkan dua akibat konkret bila `Serial.println()` dipanggil di dalamnya.
3. Apa yang terjadi bila kata kunci `volatile` dihapus dari `rxFlag`? Jelaskan dari sisi optimasi compiler.
4. Bandingkan hasil Tabel 2.15 dengan lembar pengukuran A pada M01. Apakah mekanisme penerimaan memengaruhi jangkauan? Jelaskan alasannya.
5. LED penerima menyala 200 ms dengan `delay()`. Sebutkan kelemahan cara itu, dan pada kondisi apa kelemahannya menjadi nyata.

2.11 Concept Check

1. Apa perbedaan polling dan interrupt dalam menerima paket?
2. Jalur perangkat keras apa yang menghubungkan SX1276 ke interrupt Arduino, dan lewat pin mana?
3. Mengapa `LoRa.receive()` harus dipanggil lagi setelah sebuah paket selesai diolah?
4. Mengapa `rxFlag` harus `volatile`?
5. Mengapa D13 kurang tepat dijadikan indikator kejadian pada board bershield LoRa?

2.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Indikator yang tidak menghalangi

CH-1 — LED tanpa `delay()` Ganti kedip LED penerima dengan penjadwalan berbasis `millis()`, lalu buktikan dengan EXP-03 bahwa penerima menjadi lebih tahan terhadap paket yang datang beruntun.

CH-2 — Dua LED Pakai D3 untuk paket diterima dan D4 untuk paket ditolak (misalnya payload yang tidak diawali `Hello`). Jelaskan mengapa penyaringan seperti ini akan diperlukan pada M05.

Level 3 – Tantangan · Kualitas tautan yang terlihat tanpa Serial Monitor

CH-3 – Intensitas mengikuti sinyal Kedipkan LED dengan PWM yang terangnya mengikuti RSSI, sehingga kualitas tautan terlihat tanpa membaca Serial Monitor.

CH-4 – Hitung paket terlewat Uraikan nomor urut dari payload, dan tampilkan jumlah paket yang hilang secara berjalan pada penerima.

2.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas – interrupt, ISR, *volatile*, mode receive kontinu.
3. Konfigurasi – pin DIO0, pilihan pin LED, parameter radio.
4. Hasil eksperimen – log serial kedua board (EXP-01 sampai EXP-03 beserta checkpoint), foto atau video LED.
5. Data pengukuran – Tabel 2.14 dan Tabel 2.15.
6. Analisis dan concept check.
7. Challenge – minimal CH-1.
8. Kesimpulan yang disusun sendiri, khususnya mengenai kapan interrupt sepadan dengan tambahan kerumitannya.

3. Modul 03 — Peer-to-Peer Ping-Pong

LoRa COMMUNICATION LAB · MODUL 03

Take Turns Talking — Peer-to-Peer Ping-Pong

Arduino Uno + Dragino LoRa Shield v1.2 · LoRa mentah · dua arah · RX polling · level Intermediate · 2 × 50 menit · folder kode Modul03_lora_p2p

PENGANTAR MODUL · Gambaran Umum

Modul 03 dirancang untuk dua pertemuan (2 × 50 menit) pada tingkat menengah. Misinya membuat kedua board bergantian berbicara di atas satu radio yang sama: Device A memulai Ping, Device B membalas Pong, dan seterusnya tanpa henti. Percobaan memakai dua Arduino Uno bershield Dragino LoRa v1.2, diamati melalui dua Serial Monitor pada 9600 baud.

3.1 Pendahuluan

Dua modul pertama hanya memindahkan data satu arah, sehingga peran tiap board tetap: yang satu selalu bicara, yang lain selalu mendengar. Begitu arah dibalik, muncul kenyataan yang tidak ada pada BLE maupun Zigbee — radio LoRa bersifat **half-duplex**: satu chip tidak dapat mengirim dan menerima pada saat bersamaan. Setiap board harus bergiliran, dan giliran itu tidak diatur protokol mana pun, melainkan oleh kode aplikasi. Di sinilah muncul persoalan klasik komunikasi dua arah: bagaimana bila balasan tidak pernah datang, dan siapa yang bertanggung jawab memulai ulang percakapan.

Prasyaratnya adalah M01 untuk parameter radio dan pembacaan RSSI, serta M02 untuk pemahaman kapan `loop()` terblokir. Yang dibangun di sini adalah percakapan bergantian di atas radio half-duplex, penguraian isi paket untuk menentukan balasan, penanda identitas pengirim di dalam payload, serta pemulihan otomatis melalui pengiriman ulang berkala. Semuanya dipakai lagi pada M04 ketika balasan diformalkan menjadi ACK dengan batas waktu, dan M05 ketika giliran bicara dijadwalkan satu master untuk banyak node.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan lewat interrupt — <code>loop()</code> tidak lagi menunggu
03 (ini)	Dua arah bergantian di atas radio half-duplex + pemulihan otomatis
04	Keandalan diukur: ACK, timeout, dan hitungan gagal
05	Banyak node — satu master menjadwalkan giliran bicara

Tabel 3.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Payload mulai membawa **identitas pengirim** di depan isinya: DeviceA:Ping, DeviceB:Pong. Tanpa penanda itu, sebuah board tidak dapat membedakan gema pesannya sendiri dari balasan lawan bicara. Pola identitas-di-dalam-payload ini berkembang menjadi `S1:DATA:n` pada M05, dan merupakan padanan LoRa dari prefiks A: dan B: pada modul BLE multi-node.

3.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membangun percakapan dua arah yang pulih sendiri

1. Menjelaskan sifat half-duplex radio LoRa dan akibatnya pada rancangan percakapan dua arah.
2. Membangun pertukaran Ping-Pong yang berkelanjutan, termasuk penguraian isi paket untuk menentukan balasan.
3. Menjelaskan perlunya penanda identitas pada payload ketika beberapa board memakai parameter radio yang sama.
4. Menerapkan pemulihan otomatis berbasis batas waktu, dan menjelaskan mengapa hanya satu pihak yang boleh memegang peran itu.
5. Mengukur waktu pulang-pergi (*round-trip*) satu siklus Ping-Pong dan menjelaskan penyusunnya.

Kriteria keberhasilan

- ☐ Kedua board bergantian mencetak [RX] dan [TX] tanpa henti.
- ☐ Identitas pengirim pada payload sesuai dengan board asalnya.
- ☐ Ketika Device B dimatikan, Device A mencetak [RETRY] dan pulih sendiri setelah B dinyalakan lagi.
- ☐ Waktu pulang-pergi terukur dari log, minimal 10 siklus.

3.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Half-duplex	Radio dapat mengirim atau menerima, tetapi tidak keduanya sekaligus. SX1276 harus berpindah mode.
Initiator	Pihak yang memulai percakapan dan bertanggung jawab mengulang bila macet. Di sini Device A.
Responder	Pihak yang hanya membalas ketika menerima. Di sini Device B.
Round-trip time	Selang dari sebuah paket dikirim sampai balasannya diterima.
Auto-retry	Pengiriman ulang otomatis bila tidak ada balasan dalam selang tertentu (5 detik pada modul ini).
Deadlock percakapan	Keadaan macet ketika kedua pihak sama-sama menunggu — pasti terjadi bila tidak ada pihak yang memegang peran initiator.

Tabel 3.2: Istilah kerja Modul 03

Mengapa hanya satu pihak yang boleh melakukan retry. Andaikan kedua board sama-sama mengirim ulang setiap 5 detik ketika sunyi, keduanya berpeluang memancar pada saat yang hampir bersamaan. Karena radio half-duplex, board yang sedang memancar tidak mungkin mendengar lawannya, sehingga kedua paket saling menutupi dan tidak satu pun diterima. Percakapan justru semakin sulit pulih ketika semakin banyak pihak berinisiatif. Pembagian peran initiator dan responder menyelesaikannya dengan cara paling sederhana: hanya satu pihak yang berhak membuka suara.

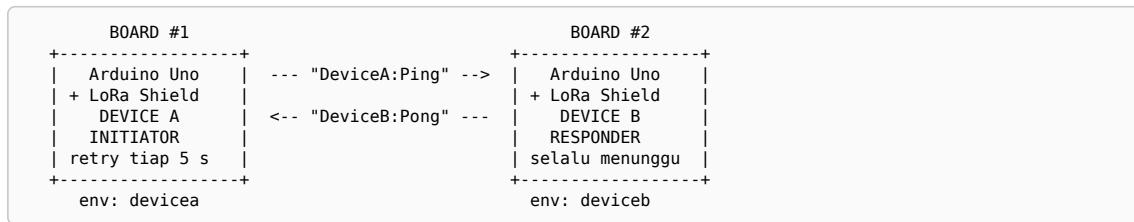
Sekuens yang diamati

```

Device A (initiator)      (udara)      Device B (responder)
|                               |
"DeviceA:Ping" ----->  paket tiba
|                               |
|                               | cetak + RSSI
paket tiba <-----      "DeviceB:Ping"
|                               |
cetak + RSSI
balas "DeviceA:Ping" ----->  |
|                               |
|                               | ...
[ bila sunyi > 5 detik ]
[ RETRY ] kirim ulang Ping

```

3.4 Topologi



Node	Environment	Build flag	Peran	Pemulihan
Device A	devicea	-DDEVICE_A	Initiator, memulai Ping	Kirim ulang tiap 5 detik bila sunyi
Device B	deviceb	—	Responder, membalas tiap paket	Tidak perlu, selalu menunggu

Tabel 3.3: Peran tiap node Modul 03

Kedua board memakai **file source yang sama**, `src/peer/main.cpp`. Perannya ditentukan build flag di `platformio.ini`, bukan dengan menyunting `#define` lalu mengembalikannya — cara yang sering menjadi sumber kesalahan ketika satu board diunggah firmware peran yang keliru.

3.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	2
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
3	Antena SMA	wajib terpasang sebelum diberi daya	2
4	Kabel USB tipe B	kabel data	2

Tabel 3.4: Alat dan bahan Modul 03

Struktur proyek

Modul03_lora_p2p/	
├─ platformio.ini	← peran ditentukan build flag -DDEVICE_A
├─ monitor_serial.py	← pantau kedua peer, ringkas siklus/retry/RTT
├─ upload_auto.py	← deteksi port otomatis saat unggah
├─ logserial.md	← log referensi hasil uji perangkat
├─ src/	
│ └─ peer/main.cpp	← satu source untuk kedua board

3.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul03_lora_p2p/](#)

- [Modul03_lora_p2p/platformio.ini](#)
- [Modul03_lora_p2p/src/peer/main.cpp](#)
- [Modul03_lora_p2p/monitor_serial.py](#)
- [Modul03_lora_p2p/upload_auto.py](#)

```
[platformio]
default_envs = deviceb

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 9600
build_src_filter = +<peer/*.cpp>

; Kedua board memakai source yang sama; perannya ditentukan build flag.
[env:devicea]
build_flags = -DDEVICE_A
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:deviceb]
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1
```

Kode 3.1: platformio.ini Modul 03 — peran ditentukan build flag -DDEVICE_A

```
/*
LoRa Peer-to-Peer (Ping-Pong) - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: devicea / deviceb

Satu file ini di-upload ke DUA board.
Peran dibedakan lewat environment PlatformIO, bukan dengan menyunting #define.

Mekanisme:
TX : blocking (endPacket) – sederhana dan andal di AVR
RX : polling via LoRa.parsePacket() – tidak perlu ISR, bebas race condition
Device A kirim ulang Ping tiap PING_INTERVAL ms jika tidak ada balasan (auto-recovery)

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10 (R9 0-ohm terpasang)
DIO0 -> D2
RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DIO0_PIN         2

#define FREQUENCY        433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_PIN          LED_BUILTIN // D13. Ganti ke 3 untuk LED eksternal di D3.
#define PING_INTERVAL    5000        // ms – Device A kirim ulang Ping jika tidak ada balasan

// Peran board ditentukan build flag di platformio.ini:
// [env:devicea] -> -DDEVICE_A (initiator, memulai Ping)
// [env:deviceb] -> (tanpa flag) responder, membalas tiap paket masuk
// Cara ini menggantikan penyuntingan #define secara manual, sehingga satu
// source dipakai kedua board tanpa risiko lupa mengembalikannya.

#ifdef DEVICE_A
```

```

#define MY_NAME "DeviceA"
#else
#define MY_NAME "DeviceB"
#endif

unsigned long lastTxMs = 0;

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket(); // blocking – selesai TX baru lanjut
}

void setup() {
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.println(F("=== LoRa PEER-TO-PEER ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);

    Serial.println(F("OK"));
    Serial.print(F("Freq : ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("Peran : "));

#ifdef DEVICE_A
    Serial.println(F("INITIATOR (Device A, env devicea)"));
    Serial.println();
    delay(1000); // beri Device B waktu siap
    transmit(MY_NAME ":Ping");
    lastTxMs = millis();
    Serial.println(F("[TX] DeviceA:Ping\n"));
#else
    Serial.println(F("RESPONDER (Device B, env deviceb)"));
    Serial.println(F("Menunggu paket...\n"));
#endif
}

void loop() {
#ifdef DEVICE_A
    // Auto-recovery: kirim ulang Ping jika tidak ada balasan dalam PING_INTERVAL ms
    if (millis() - lastTxMs > PING_INTERVAL) {
        Serial.println(F("[RETRY] Tidak ada balasan, kirim ulang Ping..."));
        transmit(MY_NAME ":Ping");
        lastTxMs = millis();
        Serial.println(F("[TX] DeviceA:Ping\n"));
    }
#endif

    // Cek paket masuk – polling, tidak perlu ISR
    int ps = LoRa.parsePacket();
    if (ps == 0) return;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    lastTxMs = millis(); // reset retry timer (Device A); diabaikan Device B

    digitalWrite(LED_PIN, HIGH);
    Serial.println(F("====="));
    Serial.print(F("[RX] Pesan : ")); Serial.println(received);
    Serial.print(F("[RX] RSSI : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
    Serial.print(F("[RX] SNR : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));
    Serial.println(F("====="));
    digitalWrite(LED_PIN, LOW);

    delay(50);

```

```
String reply = String(MY_NAME) + ":" +
    (received.indexOf("Ping") >= 0 ? "Pong" : "Ping");

transmit(reply);
Serial.print(F("[TX] ")); Serial.println(reply);
Serial.println();
}
```

Kode 3.2: src/peer/main.cpp — satu source untuk initiator dan responder

```
#!/usr/bin/env python3
"""Monitor kedua peer LoRa sekaligus untuk Modul 03.

Menampilkan kedua aliran serial dalam SATU jendela dengan timestamp bersama,
lalu meringkasnya saat berhenti: berapa siklus Ping-Pong yang selesai, berapa
kali initiator terpaksa mengulang, dan berapa waktu pulang-perginya.

Payload modul ini tidak bernomor (`DeviceA:Ping`, `DeviceB:Pong`), sehingga
paket hilang tidak dapat dihitung dari lompatan angka seperti pada Modul 01.
Yang diukur di sini adalah **waktu pulang-pergi** – selang dari sebuah paket
dikirim sampai balasannya tiba – dan jumlah baris `[RETRY]`, yaitu berapa kali
percakapan sempat macet lalu dipulihkan sendiri oleh initiator.

python3 Modul03_lora_p2p/monitor_serial.py
python3 Modul03_lora_p2p/monitor_serial.py --log sesil.txt
python3 Modul03_lora_p2p/monitor_serial.py --port A=/dev/ttyACM0
python3 Modul03_lora_p2p/monitor_serial.py --durasi 60 # berhenti sendiri

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C, atau pakai `--durasi` agar berhenti otomatis.

PORT OTOMATIS. Port A/B tidak lagi ditulis tetap di kode: setiap kali
dijalankan, skrip memindai /dev/ttyACM* dan /dev/ttyUSB* yang sedang aktif
dan memakai yang benar-benar tersambung, urut sesuai nama device. Ini
penting karena nama port bisa berubah tiap kali board dicabut-pasang atau
board asli/klon bercampur (lihat tools/deteksi_port.py). Bila urutan
otomatisnya keliru (mis. Device A ternyata terdeteksi sebagai B), timpa
lewat --port, mis. --port A=/dev/ttyACM1 --port B=/dev/ttyACM0.

SOAL RESET OTOMATIS. Pada Arduino Uno, jalur DTR terhubung ke pin RESET lewat
kapasitor. Membuka port serial **selalu me-reset board**, dan hal itu justru
dimanfaatkan di sini: pesan `setup()` beserta baris `Init LoRa ... OK` ikut
terekam tanpa perlu menekan tombol reset. Konsekuensinya, penghitung paket
kedua board kembali ke nol setiap kali monitor dijalankan – jalankan monitor
lebih dahulu, baru mulai mengukur, dan jangan membukanya di tengah percobaan
panjang.

Pada Uno, DTR hanya menyentuh jalur RESET dan tidak tersambung ke GPIO mana pun,
sehingga membuka port tidak pernah membuat sebuah pin terbaca seolah ditekan.
"""

import argparse
import glob
import re
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

# Nama peran bawaan (urutan) dan warna yang dipasangkan ke port yang
# terdeteksi aktif, bukan ke nama /dev tertentu – lihat deteksi_port_aktif().
NAMA_PERAN_BAWAAN = ["A", "B"]
PALET_WARNA = ["\033[33m", "\033[36m", "\033[35m", "\033[32m", "\033[34m"]

def deteksi_port_aktif():
    """Kumpulkan port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* – pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil ulang tiap program start supaya daftarnya selalu mengikuti apa
    yang benar-benar tersambung saat itu, bukan port tetap di kode.
    """
```

```

return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def bangun_port_bawaan():
    """Pasangkan tiap port aktif ke nama peran (A, B, lalu P3, P4, ...)."""
    hasil = []
    for i, dev in enumerate(deteksi_port_aktif()):
        nama = NAMA_PERAN_BAWAAN[i] if i < len(NAMA_PERAN_BAWAAN) else f"P{i + 1}"
        warna = PALET_WARNA[i % len(PALET_WARNA)]
        hasil.append((nama, dev, warna))
    return hasil

# Pola yang dipanen dari baris serial untuk menyusun ringkasan
RE_TX = re.compile(r"^\[TX\]\s*(\S+)")
RE_RX = re.compile(r"^\[RX\] Pesan\s*:\s*(\S+)")
RE_RETRY = re.compile(r"^\[RETRY\]")
RE_RSSI = re.compile(r"RSSI\s*:\s*(-?\d+)")
RE_SNR = re.compile(r"SNR\s*:\s*(-?\d.]+)")
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
# Bahan ringkasan akhir; diisi reader, dibaca sekali saat program berhenti
 kirim = {} # nama -> jumlah paket dikirim
 terima = {} # nama -> jumlah paket diterima
 retry = {} # nama -> jumlah pengiriman ulang
 rtt = [] # waktu pulang-pergi Device A, detik
 rssi_per = {} # nama -> daftar RSSI yang dilihat node itu
 snr_per = {}
 tx_terakhir = {} # nama -> waktu [TX] terakhir, untuk menghitung rtt
 stop = threading.Event()
 t0 = time.time()

def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"[DIM][{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()

def open_port(port, baud):
    """Buka port serial menuju board Arduino.

    Jalur DTR/RTS sengaja TIDAK disentuh. Menyetelnya sebelum open() ditolak
    oleh CDC ATmega16U2 pada Uno asli dan berakhir sebagai
    `[Errno 110] Connection timed out`; pada klon ber-bridge CH340 hal itu
    kebetulan lolos, sehingga kesalahannya mudah tidak terlihat bila diuji
    hanya pada satu jenis board.

    Membuka port tetap me-reset board sekali - itu memang yang diinginkan,
    karena pesan setup() ikut terekam - dan reset itu berupa pulsa dari
    kapasitor, bukan penahanan, sehingga board langsung berjalan normal.
    """
    return serial.Serial(port, baud, timeout=0.2)

def panen(name, text):
    """Ambil kejadian TX, RX, RETRY, RSSI, dan SNR dari sebuah baris serial.

    Waktu pulang-pergi dihitung sebagai selang antara [TX] sebuah node dengan
    [RX] berikutnya pada node yang sama - itulah satu putaran penuh Ping-Pong.
    """
    sekarang = time.time()
    if RE_TX.match(text):
        kirim[name] = kirim.get(name, 0) + 1
        tx_terakhir[name] = sekarang
    elif RE_RX.match(text):
        terima[name] = terima.get(name, 0) + 1
        if name in tx_terakhir:
            rtt.append(sekarang - tx_terakhir.pop(name))
    elif RE_RETRY.match(text):
        retry[name] = retry.get(name, 0) + 1
        # Retry berarti balasan sebelumnya tidak pernah datang; jangan dihitung

```

```

# sebagai waktu pulang-pergi.
tx_terakhir.pop(name, None)

if (m := RE_RSSI.search(text)):
    rssi_per.setdefault(name, []).append(int(m.group(1)))
if (m := RE_SNR.search(text)):
    snr_per.setdefault(name, []).append(float(m.group(1)))

def ringkasan():
    """Cetak hasil ukur tautan: loss, RSSI, dan SNR."""
    print("\n" + "-" * 60)
    print(f"Durasi: {time.time() - t0:.1f} s")
    for nama in counts:
        print(f" {nama:<7} : {counts.get(nama, 0)} baris")

    if not kirim and not terima:
        print("\nTidak ada lalu lintas Ping-Pong yang terbaca - periksa tautan.")
        return

    print()
    for nama in counts:
        print(f" {nama:<7} kirim {kirim.get(nama, 0):<4} terima {terima.get(nama, 0):<4}"
              f" retry {retry.get(nama, 0)}")
        r, sn = rssi_per.get(nama), snr_per.get(nama)
        if r:
            print(f"          RSSI min/maks/rata-rata : {min(r)} / {max(r)} / "
                  f" {sum(r) / len(r):.1f} dBm")
        if sn:
            print(f"          SNR min/maks/rata-rata : {min(sn):.2f} / {max(sn):.2f} / "
                  f" {sum(sn) / len(sn):.2f} dB")

    if rtt:
        print(f"\n Siklus Ping-Pong selesai : {len(rtt)}")
        print(f" Waktu pulang-pergi min/maks/rata-rata : "
              f" {min(rtt) * 1000:.0f} / {max(rtt) * 1000:.0f} / "
              f" {sum(rtt) / len(rtt) * 1000:.0f} ms")
    else:
        print("\n Tidak ada siklus penuh yang terukur.")

def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip
        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:
            text = line.decode("utf-8", "replace").rstrip("\r")
            if text.strip():
                counts[name] += 1
                panen(name, text)
                show(name, color, text, use_color, logfile)
    ser.close()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                 formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyXXX",
                    help="ganti port A atau B, mis. A=/dev/ttyACM0; boleh diulang")
    # Modul 01-04 memakai 9600; Modul 05 memakai 115200.
    ap.add_argument("--baud", type=int, default=9600)
    ap.add_argument("--durasi", type=float, metavar="DETIK",
                    help="berhenti otomatis setelah sekian detik (mis. --durasi 60)")
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")

```

```

ap.add_argument("--no-color", action="store_true")
args = ap.parse_args()

port_bawaan = bangun_port_bawaan()
if not port_bawaan and not args.port:
    sys.exit("Tidak ada port serial USB yang terdeteksi (/dev/ttyACM*, /dev/ttyUSB*).\\n"
            "Periksa kabel data (bukan charge-only) dan hak akses grup dialout,\\n"
            "atau jalankan tools/deteksi_port.py untuk diagnosis lebih lanjut,\\n"
            "atau tentukan port manual lewat --port A=/dev/ttyACM0 --port B=/dev/ttyACM1")

dipakai = port_bawaan[:len(NAMA_PERAN_BAWAAN)]
sisas = port_bawaan[len(NAMA_PERAN_BAWAAN):]
print(f"Port aktif terdeteksi: {'', '.join(p for _, p, _ in port_bawaan)}")
    if port_bawaan else "Tidak ada port aktif terdeteksi otomatis – memakai --port saja.")
for nama, port, _ in dipakai:
    print(f" -> {nama} = {port}")
if sisas:
    print(f" (tidak dipakai: {'', '.join(p for _, p, _ in sisas)}")
        f" – pakai --port NAMA=/dev/... untuk memakainya)")

ports = {name: (port, color) for name, port, color in dipakai}
for item in args.port or []:
    if "=" not in item:
        sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyXXX)")
    name, port = item.split("=", 1)
    _, color = ports.get(name.upper(), (None, "\033[35m"))
    ports[name.upper()] = (port, color)

if not ports:
    sys.exit("Tidak ada port untuk dipantau – tentukan lewat --port NAMA=/dev/ttyXXX")

use_color = not args.no_color and sys.stdout.isatty()
logfile = open(args.log, "w") if args.log else None

# SIGTERM (mis. dijalankan lewat `timeout 30 ...`) cukup menyalakan flag
# `stop` yang sama dipakai reader thread, BUKAN melempar exception --
# melempar exception dari signal handler bisa mendarat di tengah blok
# `finally` (mis. saat t.join() atau ringkasan() berjalan) dan menyebabkan
# crash tak tertangkap alih-alih mencetak ringkasan dengan rapi.
signal.signal(signal.SIGTERM, lambda *_: stop.set())

batas = f" · berhenti otomatis {args.durasi:.0f} s" if args.durasi else ""
print(f"Monitor LoRa TX/RX @ {args.baud} baud – Ctrl-C untuk berhenti{batas}")
    f"{' · log: ' + args.log if args.log else ''}")
print("-" * 60)

threads = [threading.Thread(target=reader,
                            args=(name, port, color, args.baud, use_color, logfile),
                            daemon=True)
            for name, (port, color) in ports.items()]
for t in threads:
    t.start()

batasWaktu = t0 + args.durasi if args.durasi else None
try:
    while any(t.is_alive() for t in threads) and not stop.is_set():
        if batasWaktu and time.time() >= batasWaktu:
            break
        time.sleep(0.2)
except KeyboardInterrupt:
    pass
finally:
    stop.set()
    for t in threads:
        t.join(timeout=1.0)
    ringkasan()
    if logfile:
        logfile.close()
        print(f"\\nLog tersimpan di {args.log}")

if __name__ == "__main__":
    main()

```

Kode 3.3: monitor_serial.py — pantau kedua peer, ringkas siklus, retry, dan RTT

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 03 (LoRa P2P) – port serial dideteksi sendiri.

```


Alternatif untuk menjalankan manual:

```
pio run -d Modul03_lora_p2p -e devicea -t upload -t monitor
pio run -d Modul03_lora_p2p -e deviceb -t upload -t monitor
```

Skrip ini memindai port `/dev/ttyACM*/ttyUSB*` yang sedang aktif, memetakannya ke environment ``devicea`` (port pertama) dan ``deviceb`` (port kedua), lalu menjalankan ``pio run`` dengan `--upload-port` sesuai hasil deteksi `--platformio.ini` tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua board selesai di-upload, skrip bertanya apakah mau langsung menjalankan `monitor_serial.py` -- bila ya, dijalankan dengan `--port A=...` `--port B=...` persis memakai port `devicea/deviceb` yang barusan dipakai upload (bukan deteksi ulang), supaya tidak mungkin salah pasang.

```
python3 Modul03_lora_p2p/upload_auto.py
-> upload kedua board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul03_lora_p2p/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk devicea, lanjut deviceb
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti monitor_serial.py

python3 Modul03_lora_p2p/upload_auto.py --only devicea
-> upload satu environment saja

python3 Modul03_lora_p2p/upload_auto.py --devicea /dev/ttyACM0 --deviceb /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["devicea", "deviceb"]
# Nama environment PlatformIO -> nama peran yang dipakai monitor_serial.py
ENV_KE_PERAN = {"devicea": "A", "deviceb": "B"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan monitor_serial.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan monitor_serial.py
    dengan port devicea/deviceb PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang barusan
    di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan monitor_serial.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return
```

```

cmd = [sys.executable, str(PROJECT_DIR / "monitor_serial.py")]
for env, peran in ENV_KE_PERAN.items():
    if env in port_env:
        cmd += ["--port", f"{peran}={port_env[env]}"]

print(" $", " ".join(cmd))
try:
    subprocess.call(cmd)
except KeyboardInterrupt:
    # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
    # ikut menerima SIGINT selagi menunggu (os.waitpid) monitor_serial.py
    # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
    # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
    print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--devicea", metavar="/dev/ttyXXX",
                    help="tampa port devicea (lewati deteksi otomatis)")
    ap.add_argument("--deviceb", metavar="/dev/ttyXXX",
                    help="tampa port deviceb (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (devicea atau deviceb)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                        "(bawaan: upload saja -- pakai monitor_serial.py untuk memantau keduanya"
                        "sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"devicea": args.devicea, "deviceb": args.deviceb}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"-- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' -- proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 3.4: upload_auto.py — pemilih port otomatis saat unggah

3.7 Build dan Flash

Responder lebih dahulu, agar sudah siap ketika initiator mengirim Ping pertama pada detik pertama setelah reset.

```

pio run -d Modul03_lora_p2p -e deviceb -t upload -t monitor
pio run -d Modul03_lora_p2p -e devicea -t upload -t monitor

```

Memantau kedua peer sekaligus. Waktu pulang-pergi tidak dapat diukur dari dua jendela terpisah, karena tiap jendela punya sumbu waktunya sendiri. Skrip `monitor_serial.py` menggabungkan keduanya dan langsung menghitung siklus, retry, serta waktu pulang-pergi.

```
python3 Modul03_lora_p2p/monitor_serial.py
python3 Modul03_lora_p2p/monitor_serial.py --durasi 30 --log sesil.txt
```

```
A      kirim 79  terima 78  retry 0
      RSSI min/maks/rata-rata : -46 / -40 / -40.2 dBm
B      kirim 78  terima 78  retry 0
Siklus Ping-Pong selesai : 156
Waktu pulang-pergi min/maks/rata-rata : 200 / 401 / 220 ms
```

PERINGATAN

Jangan memantau port board yang hendak diunggah. Monitor menahan port itu, sehingga `pio run -t upload` tidak dapat membukanya dan unggahan gagal tanpa pesan yang jelas. Saat menguji EXP-03, pantau node yang **diamati** saja dan biarkan port node yang dimatikan-hidupkan tetap bebas.

Pre-flight checklist

- ☐ Antena terpasang pada kedua shield.
- ☐ Port kedua board dicatat lewat `pio device list` dan diisikan ke `platformio.ini`.
- ☐ Dua Serial Monitor 9600 baud siap, keduanya terlihat bersamaan.
- ☐ Urutan unggah dipahami: `deviceb` dahulu, `devicea` kemudian.

3.8 Percobaan

3.8.1 EXP-01 — Percakapan Berkelanjutan

Unggah kedua firmware sesuai urutan, lalu amati kedua Serial Monitor bersamaan.

Expected output — Device A

```
=== LoRa PEER-T0-PEER ===
Init LoRa ... OK
Freq : 433.00 MHz
Peran : INITIATOR (Device A, env devicea)

[TX] DeviceA:Ping

=====
[RX] Pesan : DeviceB:Pong
[RX] RSSI : -40 dBm
[RX] SNR : 9.75 dB
=====
[TX] DeviceA:Ping
```

Data capture

Parameter	Hasil
Isi payload yang diterima Device A	
Isi payload yang diterima Device B	
Jumlah siklus dalam 60 detik	
RSSI arah A→ B / B→ A (dBm)	

Tabel 3.5: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA

Di `src/peer/main.cpp`, balasan disusun dari pemeriksaan `received.indexOf("Ping") >= 0`. Jawab: apa yang terjadi bila kedua board diunggah ke environment yang sama, misalnya devicea pada keduanya? Ramalkan hasilnya lebih dahulu, baru buktikan dengan mencobanya, lalu kembalikan konfigurasi semula.

CHECKPOINT

Kedua Serial Monitor menampilkan [RX] dan [TX] bergantian tanpa henti. Bila salah satu board hanya mencetak [TX] berulang tanpa pernah [RX], paketnya tidak pernah tiba — periksa parameter radio dan antena sebelum melanjutkan.

3.8.2 EXP-02 — Waktu Pulang-Pergi

Ukur selang antara [TX] dan [RX] berikutnya pada Device A, sebanyak sepuluh siklus.

Siklus	Waktu pulang-pergi (ms)	RSSI balasan (dBm)
1		
2		
3		
4		
5		
6		
7		
8		
9		
10		
Rata-rata		

Tabel 3.6: Lembar pengamatan EXP-02 — waktu pulang-pergi sepuluh siklus

CHECKPOINT

Waktu pulang-pergi harus stabil, tidak berbeda jauh antar-siklus pada jarak tetap. Sebaran yang lebar menandakan sebagian paket hilang lalu dipulihkan retry — periksa apakah ada baris [RETRY] di antaranya.

3.8.3 EXP-03 — Percakapan Terputus dan Pulih

Uji perilaku sistem ketika salah satu pihak menghilang.

#	Skenario	Langkah	Hasil di Device A	Hasil di Device B
1	Responder hilang	Cabut USB Device B		—

#	Skenario	Langkah	Hasil di Device A	Hasil di Device B
2	Responder kembali	Pasang lagi Device B		
3	Initiator hilang	Cabut USB Device A	—	
4	Initiator kembali	Pasang lagi Device A		

Tabel 3.7: Lembar pengamatan EXP-03 — skenario terputus dan pulih

Data capture

Parameter	Hasil
Selang antar-baris [RETRY] (detik)	
Waktu pemulihan setelah Device B kembali (detik)	
Apakah percakapan pulih tanpa mereset Device A?	
Apakah percakapan pulih tanpa mereset Device B?	

Tabel 3.8: Lembar pengamatan EXP-03 — data pemulihan

CHECKPOINT

Skenario 3 adalah yang paling penting. Ketika **initiator** yang hilang, Device B diam selamanya tanpa satu pun pesan kesalahan: ia memang dirancang hanya membalas. Pada pengujian rujukan, B mencetak **nol baris** selama 23,5 detik tanpa initiator — dari sisi B, “lawan bicara mati” tidak dapat dibedakan dari “belum ada yang mengajak bicara”. Catat berapa lama keadaan itu bertahan, lalu kaitkan dengan pertanyaan nomor 4 pada bagian Analisis.

Angka rujukan (hasil ukur nyata, lihat `logserial.md`).

Yang hilang	Terdeteksi?	Gejala	Waktu pemulihan
Responder (B)	ya	[RETRY] tiap 5,04 s	4,86 s — dibatasi PING_INTERVAL
Initiator (A)	tidak	B diam total, 0 baris	1,1 s setelah A kembali

Tabel 3.9: Angka rujukan EXP-03 — deteksi hilangnya lawan bicara

3.8.4 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada dua Arduino Uno bershield Dragino LoRa v1.2, 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

Parameter	Hasil terukur
Siklus Ping-Pong (30 detik)	156 — $\pm 5,2$ siklus/detik
Waktu pulang-pergi min/maks/rata-rata	200 / 401 / 220 ms
Retry selama percakapan normal	0
RSSI di A / di B (rata-rata)	– 40,2 / – 40,7 dBm — tautan hampir simetris
Responder hilang → terdeteksi?	ya, [RETRY] tiap 5,04 s; pulih 4,86 s

Parameter	Hasil terukur
Initiator hilang → terdeteksi?	tidak — responder diam total, 0 baris selama 23,5 detik

Tabel 3.10: Hasil verifikasi perangkat keras Modul 03

Environment	Status	Flash
devicea	SUCCESS	26.3% (8486 B)
deviceb	SUCCESS	25.4% (8206 B)

Selisih ± 280 byte berasal dari blok `#ifdef DEVICE_A` yang hanya ikut terkompilasi pada initiator — bukti bahwa build flag benar-benar mengubah firmware, bukan sekadar penanda.

3.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen. Hasil aktual dari perangkat. Baud 9600, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Jarak antar-board ± 30 cm.

Board & Port

Peran	Environment	Port	Board
Device A — initiator	devicea	/dev/ttyUSB0	Uno klon, bridge CH340 (1a86:7523)
Device B — responder	deviceb	/dev/ttyACM1	Uno asli (2341:0043)

Tabel 3.11: Log serial Modul 03: Board & Port

Board Uno asli di `/dev/ttyACM0` tidak dipakai: shield-nya gagal `LoRa.begin()` (lihat `../Modul01_lora_uart/logserial.md`).

EXP-01 — Percakapan Berkelanjutan

```
[ 1.811] B | === LoRa PEER-TO-PEER ===
[ 1.811] B | Init LoRa ... OK
[ 1.811] B | Freq : 433.00 MHz
[ 1.811] B | Peran : RESPONDER (Device B, env deviceb)
[ 1.811] B | Menunggu paket...
[ 1.822] A | Peran : INITIATOR (Device A, env devicea)
[ 2.824] A | [TX] DeviceA:Ping
[ 3.013] B | [RX] Pesan : DeviceA:Ping
[ 3.013] B | [RX] RSSI : -41 dBm
[ 3.013] B | [RX] SNR : 9.75 dB
[ 3.013] B | [TX] DeviceB:Pong
```

Device A menunggu 1 detik sesudah init sebelum mengirim Ping pertama, sesuai `delay(1000)` di `setup()` yang memberi waktu responder bersiap.

EXP-02 — Waktu Pulang-Pergi (30 detik)

Diukur dengan `monitor_serial.py`, yang menghitung selang antara [TX] sebuah node dan [RX] berikutnya pada node yang sama.

Parameter	Hasil
Siklus Ping-Pong selesai	156 dalam 30 detik ($\pm 5,2$ siklus/detik)
Waktu pulang-pergi min / maks / rata-rata	200 / 401 / 220 ms
Device A: kirim / terima / retry	79 / 78 / 0
Device B: kirim / terima / retry	78 / 78 / —
RSSI di A (dari B) min/maks/rata-rata	-46 / -40 / -40,2 dBm
RSSI di B (dari A) min/maks/rata-rata	-41 / -40 / -40,7 dBm

Parameter	Hasil
SNR di A / di B (rata-rata)	9,86 dB / 9,35 dB
Flash devicea / deviceb	26,3 % (8.486 B) / 25,4 % (8.206 B)

Tabel 3.12: Log serial Modul 03: EXP-02 — Waktu Pulang-Pergi (30 detik)

Percakapan berjalan **tanpa satu pun retry** selama 30 detik. Waktu pulang-pergi 220 ms terdiri dari dua kali waktu udara paket SF7 ditambah waktu proses kedua board — jauh lebih besar daripada delay(50) yang disisipkan sebelum membalas.

Tautan bersifat hampir simetris: RSSI yang dilihat A (−40,2 dBm) dan yang dilihat B (−40,7 dBm) berselisih di bawah 1 dB.

EXP-03 — Percakapan Terputus dan Pulih

Ketiadaan node ditirukan dengan mengunggah **sketsa diam** (setup() dan loop() kosong, radio tidak pernah diinisialisasi), bukan dengan mencabut USB — agar seluruh rangkaian percobaan dapat diulang tanpa campur tangan tangan dan waktunya tercatat tepat.

Skenario 1–2: responder hilang lalu kembali

Device A dipantau terus-menerus; hanya Device B yang dimatikan dan dinyalakan.

```
[ 13.90] * | >>> Device B kini diam
[ 17.42] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 22.46] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 27.49] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 32.54] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 37.60] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 42.64] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 47.68] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 48.07] * | >>> Device B kembali aktif
[ 52.72] A | [RETRY] Tidak ada balasan, kirim ulang Ping...
[ 52.72] A | [TX] DeviceA:Ping
[ 52.93] A | [RX] Pesan : DeviceB:Pong          <-- percakapan pulih
[ 52.96] A | [RX] RSSI   : -45 dBm
```

Parameter	Hasil
Selang B diam → [RETRY] pertama	3,5 s
Selang antar-[RETRY]	5,04 s (sesuai PING_INTERVAL 5000)
Jumlah retry selama 34 detik sunyi	8
Waktu pemulihan setelah B kembali	4,86 s
Perlu mereset Device A?	tidak
Perlu mereset Device B?	tidak

Tabel 3.13: Log serial Modul 03: Skenario 1–2: responder hilang lalu kembali

Pemulihan memakan 4,86 detik bukan karena tautan lambat, melainkan karena Device A hanya mencoba lagi **pada jadwal retry berikutnya**. Batas atas waktu pemulihan sistem ini adalah satu PING_INTERVAL, yaitu 5 detik.

Skenario 3–4: initiator hilang lalu kembali

Kebalikannya: Device B dipantau, Device A yang dimatikan.

Parameter	Hasil
Baris yang dicetak B selama 23,5 detik tanpa initiator	0 baris
Pesan kesalahan dari B	tidak ada
Waktu pemulihan setelah A kembali	1,1 s

Tabel 3.14: Log serial Modul 03: Skenario 3–4: initiator hilang lalu kembali

Inilah temuan terpenting modul ini: ketika **initiator** yang hilang, responder tidak mencetak apa pun sama sekali — tidak ada peringatan, tidak ada retry, hanya sunyi. Dari sisi Device B, keadaan “lawan bicara mati” tidak dapat dibedakan dari “belum ada yang mengajak bicara”. Responder memang dirancang hanya membalas, dan tidak memiliki pewaktu apa pun untuk menyadari kesunyian.

Perbandingan kedua arah kegagalan:

Yang hilang	Yang terjadi	Terdeteksi?	Waktu pemulihan
Responder (B)	A mencetak [RETRY] tiap 5 detik	ya	≤ 5 s (terukur 4,86 s)
Initiator (A)	B diam total tanpa pesan	tidak	1,1 s setelah A kembali

Tabel 3.15: Log serial Modul 03: Skenario 3–4: initiator hilang lalu kembali

Catatan pengambilan log

- Percobaan pertama EXP-03 gagal karena `monitor_serial.py` menahan `/dev/ttyACM1`, sehingga unggah pemulihan ke Device B tidak dapat membuka port itu dan Device B tetap diam. Rangkaian diulang dengan hanya memantau node yang datanya dibutuhkan, sehingga port node lainnya bebas untuk diunggah. Ini juga berlaku saat praktikum: jangan menjalankan monitor pada port board yang hendak diunggah.
- Payload modul ini tidak bernomor, sehingga paket hilang tidak dapat dihitung dari lompatan angka seperti Modul 01. Yang diukur adalah waktu pulang-pergi dan jumlah retry. Penomoran payload menjadi bahan CH-2.
- Kode sumber sempat memuat sisa penamaan port Windows (COM8, COM9) pada pesan `setup()`, warisan dari repositori asal. Keduanya diganti nama environment PlatformIO agar tidak menyesatkan.
- Tabel pengukuran jarak, tautan asimetris, dan spreading factor pada README belum terisi: percobaan ini dijalankan di satu meja pada jarak tetap ± 30 cm.

3.9 Pengukuran

A. Jarak terhadap kelangsungan percakapan

Jarak	RSSI A $\rightarrow$ B (dBm)	RSSI B $\rightarrow$ A (dBm)	Siklus per menit	[RETRY] per menit
1 m				
10 m				
50 m				
100 m				

Tabel 3.16: Lembar pengukuran A — jarak terhadap kelangsungan percakapan

B. Tautan asimetris — sering terjadi di lapangan dan mudah dilewatkan.

Kondisi	RSSI A $\rightarrow$ B	RSSI B $\rightarrow$ A	Percakapan berlanjut?
Kedua antenna tegak			
Antena B direbahkan			
Antena B dekat logam			

Tabel 3.17: Lembar pengukuran B — tautan asimetris

C. Waktu pulang-pergi terhadap spreading factor — ubah SF pada **kedua** board.

SF	Waktu pulang-pergi rata-rata (ms)	Siklus per menit
7		
9		
12		

Tabel 3.18: Lembar pengukuran C — waktu pulang-pergi terhadap spreading factor

3.10 Analisis

1. Dari Tabel 3.18, berapa kali waktu pulang-pergi memanjang ketika SF dinaikkan dari 7 ke 12? Bandingkan dengan perkiraan waktu udara secara teori.
2. Pada Tabel 3.17, mungkinkah A mendengar B tetapi B tidak mendengar A? Jelaskan penyebab fisiknya dan akibatnya pada percakapan dua arah.
3. Mengapa hanya Device A yang melakukan retry? Ramalkan apa yang terjadi bila kedua board sama-sama melakukannya, dan kaitkan dengan sifat half-duplex.
4. Device B tidak dapat mendeteksi bahwa lawan bicaranya hilang. Rancang mekanisme paling sederhana yang membuatnya sadar, dan sebutkan biayanya.
5. Sistem ini menganggap setiap balasan yang tiba pasti berasal dari lawan bicara yang benar. Sebutkan apa yang terjadi bila ada kelompok lain memakai frekuensi dan SF yang sama di ruangan yang sama.

3.11 Concept Check

1. Apa arti half-duplex, dan bagian mana dari kode yang memaksa kedua board bergiliran?
2. Mengapa payload perlu memuat identitas pengirim?
3. Apa fungsi PING_INTERVAL, dan apa akibatnya bila nilainya diperkecil menjadi 200 ms?
4. Mengapa responder diunggah lebih dahulu?
5. Mengapa satu file source dapat menghasilkan dua firmware yang berbeda, dan di mana perbedaannya ditentukan?

3.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Percakapan yang mengukur dirinya sendiri

CH-1 — Ukur sendiri waktu pulang-pergi Catat `millis()` saat Ping dikirim, hitung selisihnya saat Pong tiba, dan tampilkan langsung di Serial Monitor beserta rata-rata berjalan.

CH-2 — Nomor urut percakapan Ubah payload menjadi `DeviceA:Ping:n`, dan buat kedua board mendeteksi paket yang hilang dari lompatan nomor.

Level 3 – Tantangan · Pemulihan yang lebih cerdas

CH-3 – Responder yang sadar Buat Device B mencetak peringatan bila tidak menerima apa pun selama 15 detik, lalu jelaskan mengapa ia tetap tidak boleh mulai mengirim sendiri.

CH-4 – Retry dengan jeda menaik Ganti retry tetap 5 detik dengan jeda yang membesar (5, 10, 20 detik) dan kembali normal setelah berhasil. Jelaskan keuntungannya bila ada banyak pasangan board di satu ruangan.

3.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas – half-duplex, initiator dan responder, auto-retry.
3. Konfigurasi – build flag peran, parameter radio, format payload.
4. Hasil eksperimen – log serial kedua board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran – Tabel 3.16, Tabel 3.17, dan Tabel 3.18.
6. Analisis dan concept check.
7. Challenge – minimal CH-1 dan CH-2.
8. Kesimpulan yang disusun sendiri, khususnya mengenai siapa yang bertanggung jawab memulihkan percakapan.

4. Modul 04 — Keandalan Terukur dengan ACK

LoRa COMMUNICATION LAB · MODUL 04

Know If It Arrived — Keandalan Terukur dengan ACK

Arduino Uno + Dragino LoRa Shield v1.2 · LoRa mentah · dua arah · RX interrupt + timeout · level Intermediate · 2 × 50 menit · folder kode Modul04_loRa_ack

PENGANTAR MODUL · Gambaran Umum

Modul 04 dirancang untuk dua pertemuan (2 × 50 menit) pada tingkat menengah. Misinya mengubah pengiriman yang “mudah-mudahan sampai” menjadi pengiriman yang **diketahui** sampai atau gagal: pengirim menunggu balasan ACK: n dalam batas waktu tertentu, dan mencatat setiap kegagalan. Percobaan memakai dua Arduino Uno bershield Dragino LoRa v1.2, diamati melalui dua Serial Monitor pada 9600 baud.

4.1 Pendahuluan

Tiga modul sebelumnya memindahkan data tanpa pernah tahu nasibnya. Pada M01 dan M02 pengirim tidak pernah mendapat kabar apa pun; pada M03 balasan memang ada, tetapi tidak ada yang menghitung berapa kali balasan itu tidak datang. Modul ini melengkapi bagian yang hilang tersebut, dan bersamanya memperkenalkan besaran yang menjadi tolok ukur seluruh sistem komunikasi: **berapa persen pengiriman yang berhasil**. Angka itulah yang membedakan pernyataan “alatnya jalan” dari “alatnya andal pada jarak sekian”.

Prasyaratnya adalah M02 untuk penerimaan berbasis interrupt, dan M03 untuk percakapan dua arah di atas radio half-duplex. Yang dibangun di sini adalah protokol permintaan-balasan sederhana dengan nomor urut, penungguan berbatas waktu tanpa memblokir pewaktu, penolakan balasan yang tidak sesuai harapan, serta statistik berhasil dan gagal yang berjalan terus. Semuanya dipakai lagi pada M05 ketika master memakai batas waktu untuk memutuskan sebuah slave dianggap tidak menjawab.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan lewat interrupt — loop() tidak lagi menunggu
03	Dua arah bergantian di atas radio half-duplex
04 (ini)	Setiap pengiriman diketahui hasilnya: ACK, timeout, statistik
05	Banyak node — satu master menjadwalkan giliran bicara

Tabel 4.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Permintaan berbentuk DATA: n dan balasan ACK: n dengan **nomor yang sama persis**. Kecocokan nomor itulah yang membuat pengirim tahu balasan yang tiba benar-benar milik paket yang sedang ditunggu, bukan sisa balasan lama atau milik pasangan board lain. Prinsip pencocokan permintaan-balasan ini muncul lagi di M05 sebagai POLL: 1 yang harus dijawab S1: DATA: n.

4.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mengukur keandalan pengiriman, bukan sekadar mengirim

1. Menjelaskan mekanisme ACK sebagai alat ukur keandalan, dan membedakannya dari sekadar balasan pada M03.
2. Menerapkan penungguan berbatas waktu (*timeout*) yang tidak menghentikan pewaktu maupun interrupt.
3. Menjelaskan perlunya mencocokkan nomor urut antara permintaan dan balasan, serta akibat bila pencocokan itu dihilangkan.
4. Menghitung tingkat keberhasilan pengiriman dan menyajikannya sebagai fungsi jarak.
5. Menjelaskan mengapa ACK menaikkan keandalan sekaligus menurunkan laju data, dan kapan pertukaran itu sepadan.

Kriteria keberhasilan

- ☐ Pengirim mencetak [OK] ACK diterima! pada jarak dekat, dengan penghitung FAIL tetap nol.
- ☐ Penerima membalas setiap DATA:n dengan ACK:n bernomor sama.
- ☐ Ketika penerima dimatikan, pengirim mencetak [FAIL] tepat setelah batas waktu, bukan membeku.
- ☐ Tabel keberhasilan terhadap jarak terisi, minimal empat jarak.

4.3 Dasar Teori (Secukupnya)

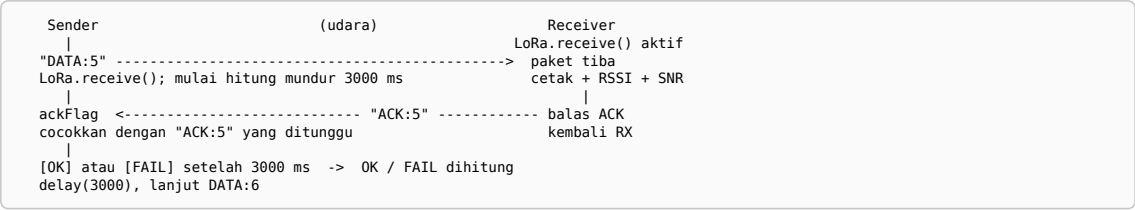
Istilah	Definisi kerja di lab ini
ACK	<i>Acknowledgement</i> — balasan yang menyatakan sebuah paket benar-benar diterima.
Timeout	Batas waktu menunggu balasan. Modul ini memakai 3000 ms.
Nomor urut	Angka pada payload yang mengikat permintaan dengan balasannya.
Tingkat keberhasilan	$OK / (OK + FAIL) \times 100 \%$ — besaran utama yang diukur modul ini.
Kegagalan senyap	Paket hilang tanpa ada pihak yang menyadarinya. Inilah yang dihapus oleh ACK.
Waktu udara	Lama sebuah paket mengudara. Menentukan batas bawah nilai timeout yang masuk akal.

Tabel 4.2: Istilah kerja Modul 04

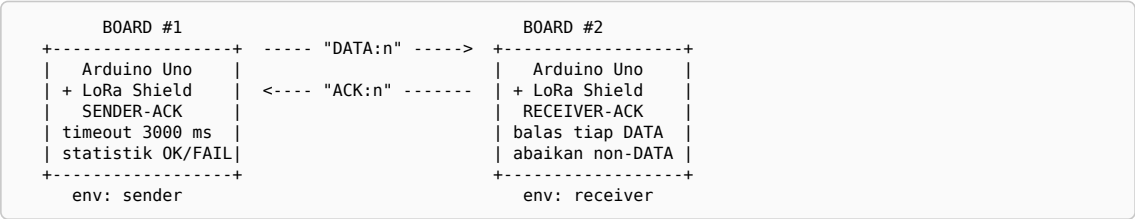
Mengapa timeout tidak boleh terlalu pendek. Satu putaran ACK memerlukan waktu udara paket data, waktu proses penerima, dan waktu udara paket ACK. Pada SF7 dan BW 125 kHz, paket pendek memakan puluhan milidetik; pada SF12 waktu itu membengkak menjadi lebih dari satu detik untuk sekali jalan. Timeout yang lebih pendek daripada waktu pulang-pergi akan mencatat kegagalan padahal ACK sedang dalam perjalanan — dan yang lebih buruk, ACK yang terlambat tiba akan tertinggal di antrian dan mengacaukan siklus berikutnya. Karena itu program ini menolak balasan yang nomornya tidak cocok, alih-alih menerimanya begitu saja.

Mengapa ACK bukan jaminan sempurna. ACK hanya membuktikan paket data sampai ke penerima; ia tidak membuktikan ACK-nya sendiri sampai kembali ke pengirim. Bila DATA:5 diterima dengan baik tetapi ACK:5 hilang di jalan, penerima mencatat keberhasilan sedangkan pengirim mencatat kegagalan. Keadaan tidak sepakat ini nyata dan akan terlihat pada pengukuran jarak jauh — bandingkan penghitung di kedua sisi pada EXP-03.

Sekuens yang diamati



4.4 Topologi



Node	Environment	Peran	Mekanisme RX	Batas waktu
Sender	sender	Kirim DATA:n, tunggu ACK:n	Interrupt DIO0 + ackFlag	3000 ms
Receiver	receiver	Terima DATA:n, balas ACK:n	Interrupt DIO0 + rxFlag	—

Tabel 4.3: Peran tiap node Modul 04

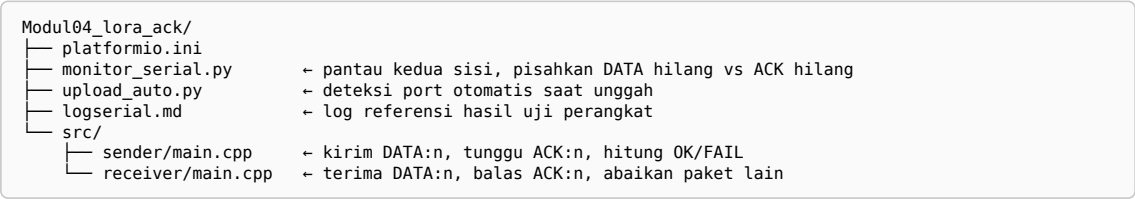
4.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	2
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
3	Antena SMA	wajib terpasang sebelum diberi daya	2
4	Kabel USB tipe B	kabel data	2

Tabel 4.4: Alat dan bahan Modul 04

Struktur proyek



4.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul04_lora_ack/](#)

- [Modul04_lora_ack/platformio.ini](#)
- [Modul04_lora_ack/src/sender/main.cpp](#)
- [Modul04_lora_ack/src/receiver/main.cpp](#)
- [Modul04_lora_ack/monitor_serial.py](#)
- [Modul04_lora_ack/upload_auto.py](#)

```
[platformio]
default_envs = receiver

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 9600

[env:sender]
build_src_filter = +<sender/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:receiver]
build_src_filter = +<receiver/*.cpp>
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1
```

Kode 4.1: platformio.ini Modul 04 — environment sender dan receiver

```
/*
  LoRa ACK Sender - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
  Library : LoRa by sandeepmistry v0.8.x
  Environment PlatformIO: sender

  Mekanisme:
  1. Kirim "DATA:N" tiap 3 detik (TX blocking)
  2. Langsung masuk RX mode, tunggu "ACK:N" max ACK_TIMEOUT ms
  3. Catat statistik ACK OK / NO ACK ke Serial

  Pin Mapping Dragino Shield v1.2:
  NSS/CS -> D10, DIO0 -> D2, RST -> D9
  SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN      10
#define RST_PIN      9
#define DIO0_PIN     2

#define FREQUENCY     433E6
#define BANDWIDTH     125E3
#define SPREADING_FACTOR 7
#define CODING_RATE   5
#define TX_POWER      17

#define LED_PIN       LED_BUILTIN // D13. Ganti ke 3 untuk LED eksternal di D3.
#define LED_DURATION  200
```

```

#define ACK_TIMEOUT 3000

int counter = 0;
int ackOK = 0;
int ackFail = 0;

unsigned long ledOnTime = 0;
bool ledActive = false;

volatile bool ackFlag = false;

void onAckReceived(int size) {
    if (size > 0) ackFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

void setup() {
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.println(F("=== LoRa ACK SENDER ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.onReceive(onAckReceived); // ISR untuk terima ACK

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.print(F(" MHz | "));
    Serial.print(F("SF")); Serial.print(SPREADING_FACTOR); Serial.print(F(" | "));
    Serial.print(F("ACK timeout: ")); Serial.print(ACK_TIMEOUT); Serial.println(F(" ms\n"));
}

void loop() {
    updateLED();

    // --- 1. Kirim DATA (TX blocking) ---
    String msg = "DATA:" + String(counter);
    Serial.print(F("[TX] Kirim: "));
    Serial.print(msg);
    Serial.print(F(" ... "));

    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket(); // blocking - selesai TX baru lanjut

    Serial.println(F("selesai"));

    // --- 2. Masuk RX, tunggu ACK ---
    String expectedACK = "ACK:" + String(counter);
    ackFlag = false;
    LoRa.receive(); // non-blocking RX via interrupt DI00

    unsigned long waitStart = millis();
    bool gotACK = false;

    while (millis() - waitStart < ACK_TIMEOUT) {
        updateLED();
    }
}

```

```

    if (ackFlag) {
        ackFlag = false;

        String reply = "";
        while (LoRa.available()) {
            reply += (char)LoRa.read();
        }

        Serial.print(F("[RX] Balasan: "));
        Serial.println(reply);

        if (reply == expectedACK) {
            gotACK = true;
            break;
        }

        // Paket datang tapi bukan ACK yang ditunggu – tetap tunggu
        Serial.println(F("[RX] WARN: bukan ACK yang diharapkan, tetap tunggu..."));
    }
}

// --- 3. Evaluasi hasil ---
if (gotACK) {
    ackOK++;
    triggerLED();
    Serial.print(F("[OK] ACK diterima!"));
} else {
    ackFail++;
    Serial.print(F("[FAIL] Tidak ada ACK!"));
}

Serial.print(F(" | OK: ")); Serial.print(ackOK);
Serial.print(F(" | FAIL: ")); Serial.println(ackFail);
Serial.println();

counter++;
delay(3000);
}

```

Kode 4.2: src/sender/main.cpp — kirim DATA:n, tunggu ACK:n, hitung OK dan FAIL

```

/*
  LoRa ACK Receiver - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
  Library : LoRa by sandeepmistry v0.8.x
  Environment PlatformIO: receiver

  Mekanisme:
  1. Tunggu paket "DATA:N" via interrupt DI00 (non-blocking RX)
  2. Segera kirim "ACK:N" (TX blocking)
  3. Kembali ke RX mode

  Pin Mapping Dragino Shield v1.2:
  NSS/CS -> D10, DI00 -> D2, RST -> D9
  SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN      10
#define RST_PIN      9
#define DI00_PIN     2

#define FREQUENCY     433E6
#define BANDWIDTH     125E3
#define SPREADING_FACTOR 7
#define CODING_RATE    5
#define TX_POWER      17

#define LED_PIN      LED_BUILTIN // D13. Ganti ke 3 untuk LED eksternal di D3.
#define LED_DURATION 200

int rxCount = 0;

unsigned long ledOnTime = 0;
bool ledActive = false;

```



```

volatile bool rxFlag = false;

void onReceive(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

void setup() {
    Serial.begin(9600);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.println(F("=== LoRa ACK RECEIVER ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.onReceive(onReceive);
    LoRa.receive(); // langsung masuk RX mode

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.println(F("Menunggu paket DATA...\n"));
}

void loop() {
    updateLED();

    if (!rxFlag) return;
    rxFlag = false;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    // Abaikan paket yang bukan format DATA:
    if (!received.startsWith("DATA:")) {
        Serial.print(F("[WARN] Abaikan paket: "));
        Serial.println(received);
        LoRa.receive();
        return;
    }

    float rssi = LoRa.packetRssi();
    float snr = LoRa.packetSnr();
    rxCount++;
    triggerLED();

    Serial.println(F("=== PAKET DITERIMA ==="));
    Serial.print(F(" Data : ")); Serial.println(received);
    Serial.print(F(" RSSI : ")); Serial.print(rssi); Serial.println(F(" dBm"));
    Serial.print(F(" SNR : ")); Serial.print(snr); Serial.println(F(" dB"));
    Serial.print(F(" Total : ")); Serial.println(rxCount);
    Serial.println(F("====="));

    // Kirim ACK (TX blocking)
    String ackMsg = "ACK:" + received.substring(5);
    Serial.print(F("[TX] ACK: "));
    Serial.println(ackMsg);
}

```

```

LoRa.beginPacket();
LoRa.print(ackMsg);
LoRa.endPacket(); // blocking – pastikan ACK terkirim sempurna

// Kembali ke RX mode
LoRa.receive();
Serial.println(F("[RX] Menunggu paket berikutnya...\n"));
}

```

Kode 4.3: src/receiver/main.cpp — terima DATA:n, balas ACK:n, abaikan paket lain

```

#!/usr/bin/env python3
"""Monitor pengirim dan penerima LoRa sekaligus untuk Modul 04.

Menampilkan kedua aliran serial dalam SATU jendela dengan timestamp bersama,
lalu meringkasnya saat berhenti: berapa DATA dikirim, berapa yang berbalas ACK,
berapa yang kehabisan waktu, dan berapa lama satu putaran DATA-ACK memakan
waktu.

Ringkasan ini menghitung ulang keberhasilan dari **kedua sisi**: berapa DATA
yang benar-benar tiba di penerima, dan berapa ACK yang benar-benar kembali ke
pengirim. Selisih keduanya adalah ketidaksepakatan yang dibahas pada tabel B
bagian Pengukuran - keadaan ketika penerima merasa berhasil sedangkan pengirim
mencatat gagal karena ACK-nya yang hilang.

python3 Modul04_lora_ack/monitor_serial.py
python3 Modul04_lora_ack/monitor_serial.py --log sesil.txt
python3 Modul04_lora_ack/monitor_serial.py --port RX=/dev/ttyACM0
python3 Modul04_lora_ack/monitor_serial.py --durasi 60 # berhenti sendiri

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C, atau pakai `--durasi` agar berhenti otomatis.

PORT OTOMATIS. Port TX/RX tidak lagi ditulis tetap di kode: setiap kali
dijalankan, skrip memindai /dev/ttyACM* dan /dev/ttyUSB* yang sedang aktif
dan memakai yang benar-benar tersambung, urut sesuai nama device. Ini
penting karena nama port bisa berubah tiap kali board dicabut-pasang atau
board asli/klon bercampur (lihat tools/deteksi_port.py). Bila urutan
otomatisnya keliru (mis. board TX ternyata terdeteksi sebagai RX), timpa
lewat --port, mis. --port TX=/dev/ttyUSB0 --port RX=/dev/ttyACM1.

SOAL RESET OTOMATIS. Pada Arduino Uno, jalur DTR terhubung ke pin RESET lewat
kapasitor. Membuka port serial **selalu me-reset board**, dan hal itu justru
dimanfaatkan di sini: pesan `setup()` beserta baris `Init LoRa ... OK` ikut
terekam tanpa perlu menekan tombol reset. Konsekuensinya, penghitung paket
kedua board kembali ke nol setiap kali monitor dijalankan – jalankan monitor
lebih dahulu, baru mulai mengukur, dan jangan membukanya di tengah percobaan
panjang.

Pada Uno, DTR hanya menyentuh jalur RESET dan tidak tersambung ke GPIO mana pun,
sehingga membuka port tidak pernah membuat sebuah pin terbaca seolah ditekan.
"""
import argparse
import glob
import re
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

# Nama peran bawaan (urutan) dan warna yang dipasangkan ke port yang
# terdeteksi aktif, bukan ke nama /dev tertentu – lihat deteksi_port_aktif().
NAMA_PERAN_BAWAAN = ["TX", "RX"]
PALET_WARNA = ["\033[33m", "\033[36m", "\033[35m", "\033[32m", "\033[34m"]

def deteksi_port_aktif():
    """Kumpulkan port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* – pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil ulang tiap program start supaya daftarnya selalu mengikuti apa
    """

```

```

yang benar-benar tersambung saat itu, bukan port tetap di kode.
"""
return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def bangun_port_bawaan():
    """Pasangkan tiap port aktif ke nama peran (TX, RX, lalu P3, P4, ...)."""
    hasil = []
    for i, dev in enumerate(deteksi_port_aktif()):
        nama = NAMA_PERAN_BAWAAN[i] if i < len(NAMA_PERAN_BAWAAN) else f"P{i + 1}"
        warna = PALET_WARNA[i % len(PALET_WARNA)]
        hasil.append((nama, dev, warna))
    return hasil

# Pola yang dipanen dari baris serial untuk menyusun ringkasan
RE_KIRIM = re.compile(r"\[TX\] Kirim:\s*DATA:(\d+)") # pengirim: DATA keluar
RE_TIBA = re.compile(r"Data\s*:\s*DATA:(\d+)") # penerima: DATA masuk
RE_ACK_KIRIM = re.compile(r"\[TX\] ACK:\s*ACK:(\d+)") # penerima: ACK dibalas
RE_ACK_TIBA = re.compile(r"\[RX\] Balasan:\s*ACK:(\d+)") # pengirim: ACK kembali
RE_OK = re.compile(r"\[OK\]")
RE_FAIL = re.compile(r"\[FAIL\]")
RE_RSSI = re.compile(r"RSSI\s*:\s*(-?\d\.\d+)")
RE_SNR = re.compile(r"SNR\s*:\s*(-?\d\.\d+)")
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
# Bahan ringkasan akhir; diisi reader, dibaca sekali saat program berhenti
data_kirim, data_tiba = set(), set() # nomor DATA di sisi pengirim / penerima
ack_kirim, ack_tiba = set(), set() # nomor ACK di sisi penerima / pengirim
jml_ok, jml_fail = [0], [0] # penghitung [OK] dan [FAIL] di pengirim
rtt = [] # DATA keluar -> ACK kembali, detik
waktu_kirim = {} # nomor DATA -> saat dikirim
daftar_rssi, daftar_snr = [], []
stop = threading.Event()
t0 = time.time()

def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"{DIM}[{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()

def open_port(port, baud):
    """Buka port serial menuju board Arduino.

    Jalur DTR/RTS sengaja TIDAK disentuh. Menyetelnya sebelum open() ditolak
    oleh CDC ATmega16U2 pada Uno asli dan berakhir sebagai
    `[Errno 110] Connection timed out`; pada klon ber-bridge CH340 hal itu
    kebetulan lolos, sehingga kesalahannya mudah tidak terlihat bila diuji
    hanya pada satu jenis board.

    Membuka port tetap me-reset board sekali - itu memang yang diinginkan,
    karena pesan setup() ikut terekam - dan reset itu berupa pulsa dari
    kapasitor, bukan penahanan, sehingga board langsung berjalan normal.
    """
    return serial.Serial(port, baud, timeout=0.2)

def panen(name, text):
    """Ambil kejadian DATA, ACK, OK/FAIL, RSSI, dan SNR dari sebuah baris.

    Nomor dipanen dari empat titik berbeda - DATA keluar, DATA tiba, ACK
    dibalas, ACK kembali - sehingga dapat dibedakan mana yang hilang: paket
    datanya, atau justru balasannya.
    """
    if (m := RE_KIRIM.search(text)):
        n = int(m.group(1)); data_kirim.add(n); waktu_kirim[n] = time.time()
    elif (m := RE_TIBA.search(text)):
        data_tiba.add(int(m.group(1)))
    elif (m := RE_ACK_KIRIM.search(text)):
        ack_kirim.add(int(m.group(1)))

```

```

elif (m := RE_ACK_TIBA.search(text)):
    n = int(m.group(1)); ack_tiba.add(n)
    if n in waktu_kirim:
        rtt.append(time.time() - waktu_kirim.pop(n))
elif RE_OK.search(text):
    jml_ok[0] += 1
elif RE_FAIL.search(text):
    jml_fail[0] += 1

if name != "TX":
    if (m := RE_RSSI.search(text)):
        daftar_rssi.append(float(m.group(1)))
    if (m := RE_SNR.search(text)):
        daftar_snr.append(float(m.group(1)))

def ringkasan():
    """Cetak hasil ukur tautan: loss, RSSI, dan SNR."""
    print("\n" + "-" * 60)
    print(f"Durasi: {time.time() - t0:.1f} s")
    for nama in counts:
        print(f" {nama:<7} : {counts.get(nama, 0)} baris")

    if not data_kirim:
        print("\nTidak ada DATA yang terbaca - periksa tautan.")
        return

    print(f"\n DATA dikirim pengirim : {len(data_kirim)}"
          f" (nomor {min(data_kirim)}..{max(data_kirim)})")
    print(f" DATA tiba di penerima : {len(data_tiba & data_kirim)}")
    print(f" ACK dibalas penerima : {len(ack_kirim & data_kirim)}")
    print(f" ACK kembali ke pengirim: {len(ack_tiba & data_kirim)}")

    # Dua jenis kegagalan yang berbeda akibatnya, sengaja dipisah: DATA yang
    # tidak pernah tiba, dan DATA yang tiba tetapi ACK-nya hilang di jalan.
    data_hilang = sorted(data_kirim - data_tiba)
    ack_hilang = sorted((data_tiba & data_kirim) - ack_tiba)
    n = len(data_kirim)
    print(f"\n DATA hilang : {len(data_hilang)} ({len(data_hilang) / n * 100:.1f} %)")
    print(f" DATA tiba tapi ACK hilang: {len(ack_hilang)} ({len(ack_hilang) / n * 100:.1f} %)")
    print(f" Keberhasilan menurut pengirim : "
          f"{len(ack_tiba & data_kirim) / n * 100:.1f} %")
    print(f" Keberhasilan menurut penerima : "
          f"{len(data_tiba & data_kirim) / n * 100:.1f} %")
    if jml_ok[0] or jml_fail[0]:
        print(f" Penghitung firmware : OK {jml_ok[0]} / FAIL {jml_fail[0]}")

    if rtt:
        print(f"\n Waktu DATA->ACK min/maks/rata-rata : "
              f"{min(rtt) * 1000:.0f} / {max(rtt) * 1000:.0f} / "
              f"{sum(rtt) / len(rtt) * 1000:.0f} ms")
    if daftar_rssi:
        print(f" RSSI min/maks/rata-rata : {min(daftar_rssi):.0f} / {max(daftar_rssi):.0f} / "
              f"{sum(daftar_rssi) / len(daftar_rssi):.1f} dBm")
    if daftar_snr:
        print(f" SNR min/maks/rata-rata : {min(daftar_snr):.2f} / {max(daftar_snr):.2f} / "
              f"{sum(daftar_snr) / len(daftar_snr):.2f} dB")

def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip
        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:

```

```

        text = line.decode("utf-8", "replace").rstrip("\r")
        if text.strip():
            counts[name] += 1
            panen(name, text)
            show(name, color, text, use_color, logfile)
    ser.close()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyXXX",
                    help="ganti port TX atau RX, mis. RX=/dev/ttyACM0; boleh diulang")
    # Modul 01-04 memakai 9600; Modul 05 memakai 115200.
    ap.add_argument("--baud", type=int, default=9600)
    ap.add_argument("--durasi", type=float, metavar="DETIK",
                    help="berhenti otomatis setelah sekian detik (mis. --durasi 60)")
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    port_bawaan = bangun_port_bawaan()
    if not port_bawaan and not args.port:
        sys.exit("Tidak ada port serial USB yang terdeteksi (/dev/ttyACM*, /dev/ttyUSB*).\\n"
                "Periksa kabel data (bukan charge-only) dan hak akses grup dialout,\\n"
                "atau jalankan tools/deteksi_port.py untuk diagnosis lebih lanjut,\\n"
                "atau tentukan port manual lewat --port TX=/dev/ttyACM0 --port RX=/dev/ttyACM1")

    dipakai = port_bawaan[:len(NAMA_PERAN_BAWAAN)]
    sisa = port_bawaan[len(NAMA_PERAN_BAWAAN):]
    print(f"Port aktif terdeteksi: {'', '.join(p for _, p, _ in port_bawaan)}")
    if port_bawaan else "Tidak ada port aktif terdeteksi otomatis – memakai --port saja.")
    for nama, port, _ in dipakai:
        print(f" -> {nama} = {port}")
    if sisa:
        print(f" (tidak dipakai: {'', '.join(p for _, p, _ in sisa)}")
        f" - pakai --port NAMA=/dev/... untuk memakainya")

    ports = {name: (port, color) for name, port, color in dipakai}
    for item in args.port or []:
        if "=" not in item:
            sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyXXX)")
        name, port = item.split("=", 1)
        _, color = ports.get(name.upper(), (None, "\033[35m"))
        ports[name.upper()] = (port, color)

    if not ports:
        sys.exit("Tidak ada port untuk dipantau – tentukan lewat --port NAMA=/dev/ttyXXX")

    use_color = not args.no_color and sys.stdout.isatty()
    logfile = open(args.log, "w") if args.log else None

    # SIGTERM (mis. dijalankan lewat `timeout 30 ...`) cukup menyalakan flag
    # `stop` yang sama dipakai reader thread, BUKAN melempar exception --
    # melempar exception dari signal handler bisa mendarat di tengah blok
    # `finally` (mis. saat t.join() atau ringkasan() berjalan) dan menyebabkan
    # crash tak tertangkap alih-alih mencetak ringkasan dengan rapi.
    signal.signal(signal.SIGTERM, lambda *_: stop.set())

    batas = f" · berhenti otomatis {args.durasi:.0f} s" if args.durasi else ""
    print(f"Monitor LoRa TX/RX @ {args.baud} baud – Ctrl-C untuk berhenti{batas}")
    f"{' · log: ' + args.log if args.log else ''}"
    print("-" * 60)

    threads = [threading.Thread(target=reader,
                                args=(name, port, color, args.baud, use_color, logfile),
                                daemon=True)
                for name, (port, color) in ports.items()]
    for t in threads:
        t.start()

    batasWaktu = t0 + args.durasi if args.durasi else None
    try:
        while any(t.is_alive() for t in threads) and not stop.is_set():
            if batasWaktu and time.time() >= batasWaktu:
                break
            time.sleep(0.2)
    except KeyboardInterrupt:
        pass
    finally:
        stop.set()
        for t in threads:

```

```

        t.join(timeout=1.0)
    ringkasan()
    if logfile:
        logfile.close()
        print(f"\nLog tersimpan di {args.log}")

if __name__ == "__main__":
    main()

```

Kode 4.4: `monitor_serial.py` — pisahkan kegagalan DATA hilang dari ACK hilang

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 04 (LoRa ACK) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul04_lora_ack -e sender -t upload -t monitor
    pio run -d Modul04_lora_ack -e receiver -t upload -t monitor

Skrrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `sender` (port pertama) dan `receiver` (port kedua), lalu
menjalankan `pio run` dengan --upload-port sesuai hasil deteksi --
platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua
board selesai di-upload, skrip bertanya apakah mau langsung menjalankan
monitor_serial.py -- bila ya, dijalankan dengan --port TX=... --port RX=...
persis memakai port sender/receiver yang barusan dipakai upload (bukan
deteksi ulang), supaya tidak mungkin salah pasang.

python3 Modul04_lora_ack/upload_auto.py
-> upload kedua board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul04_lora_ack/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk sender, lanjut receiver
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti monitor_serial.py

python3 Modul04_lora_ack/upload_auto.py --only sender
-> upload satu environment saja

python3 Modul04_lora_ack/upload_auto.py --sender /dev/ttyACM0 --receiver /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["sender", "receiver"]
# Nama environment PlatformIO -> nama peran yang dipakai monitor_serial.py
ENV_KE_PERAN = {"sender": "TX", "receiver": "RX"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan monitor_serial.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $ ", " ".join(cmd))
    return subprocess.call(cmd) == 0

```

```

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan monitor_serial.py
    dengan port sender/receiver PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang barusan
    di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan monitor_serial.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "monitor_serial.py")]
    for env, peran in ENV_KE_PERAN.items():
        if env in port_env:
            cmd += ["--port", f"{peran}={port_env[env]}"]

    print(" $", " ".join(cmd))
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) monitor_serial.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--sender", metavar="/dev/ttyXXX",
                    help="tampa port sender (lewati deteksi otomatis)")
    ap.add_argument("--receiver", metavar="/dev/ttyXXX",
                    help="tampa port receiver (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (sender atau receiver)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                    "(bawaan: upload saja -- pakai monitor_serial.py untuk memantau keduanya"
                    " sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"sender": args.sender, "receiver": args.receiver}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {' '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {' '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

```

```
if __name__ == "__main__":
    main()
```

Kode 4.5: upload_auto.py — pilih port otomatis saat unggah

4.7 Build dan Flash

Penerima lebih dahulu, agar paket pertama pengirim tidak langsung tercatat gagal.

```
pio run -d Modul04_lora_ack -e receiver -t upload -t monitor
pio run -d Modul04_lora_ack -e sender -t upload -t monitor
```

Memantau kedua board sekaligus. Lembar pengukuran B menuntut pembandingan penghitung kedua sisi. Skrip monitor_serial.py melakukannya otomatis, dan memisahkan dua jenis kegagalan yang akibatnya berbeda: DATA yang tidak pernah tiba, dan DATA yang tiba tetapi ACK-nya hilang di jalan pulang.

```
python3 Modul04_lora_ack/monitor_serial.py
python3 Modul04_lora_ack/monitor_serial.py --durasi 60 --log sesil.txt
```

```
DATA dikirim pengirim : 14 (nomor 0..13)
DATA tiba di penerima : 14
ACK dibalas penerima : 14
ACK kembali ke pengirim: 14
DATA hilang : 0 (0.0 %)
DATA tiba tapi ACK hilang: 0 (0.0 %)
Keberhasilan menurut pengirim : 100.0 %
Keberhasilan menurut penerima : 100.0 %
Waktu DATA->ACK min/maks/rata-rata : 0 / 201 / 143 ms
```

PERINGATAN

Jangan memantau port board yang hendak diunggah. Monitor menahan port itu, sehingga pio run -t upload gagal membukanya. Saat menguji EXP-02, pantau pengirim saja dan biarkan port penerima bebas.

Pre-flight checklist

- ☐ Antena terpasang pada kedua shield.
- ☐ Port kedua board dicatat lewat pio device list dan diisikan ke platformio.ini.
- ☐ Dua Serial Monitor 9600 baud siap, keduanya terlihat bersamaan.
- ☐ Penghitung OK dan FAIL pada pengirim diamati sejak baris pertama.

4.8 Percobaan

4.8.1 EXP-01 — Siklus ACK yang Sehat

Unggah kedua firmware sesuai urutan dan amati sepuluh siklus pertama.

Expected output — sender

```
=== LoRa ACK SENDER ===
Init LoRa ... OK
Freq: 433.00 MHz | SF7 | ACK timeout: 3000 ms

[TX] Kirim: DATA:0 ... selesai
[RX] Balasan: ACK:0
[OK] ACK diterima! | OK: 1 | FAIL: 0
```

Expected output — receiver


```

=== PAKET DITERIMA ===
Data : DATA:0
RSSI : -32.00 dBm
SNR : 9.00 dB
Total : 1
=====
[TX] ACK: ACK:0
[RX] Menunggu paket berikutnya...

```

Data capture

Parameter	Hasil
Nilai OK setelah 10 siklus	
Nilai FAIL setelah 10 siklus	
Tingkat keberhasilan (%)	
RSSI arah data / arah ACK (dBm)	

Tabel 4.5: Lembar pengamatan EXP-01

CHECKPOINT

Pada jarak satu meter, FAIL harus tetap nol selama sepuluh siklus. Munculnya kegagalan pada jarak sedekat itu menandakan masalah perangkat, bukan keterbatasan jangkauan — periksa antenna dan catu daya sebelum melanjutkan.

4.8.2 EXP-02 — Batas Waktu Bekerja

Cabut USB penerima di tengah percobaan, amati pengirim, lalu pasang kembali.

Expected output — sender saat penerima mati

```

[TX] Kirim: DATA:3 ... selesai
[FAIL] Tidak ada ACK! | OK: 3 | FAIL: 1

```

Data capture

Parameter	Hasil
Selang [TX] → [FAIL] (detik)	
Apakah sesuai nilai ACK_TIMEOUT?	
Apakah pengirim tetap melanjutkan siklus?	
Berapa siklus sampai pulih setelah penerima kembali?	

Tabel 4.6: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA

Di `src/sender/main.cpp`, penungguan ACK memakai `while (millis() - waitStart < ACK_TIMEOUT)` alih-alih `delay(3000)`. Jelaskan apa yang tetap dapat dikerjakan pengirim di dalam lingkaran itu, lalu telusuri mengapa `updateLED()` dipanggil di dalamnya. Terakhir, jawab: apa yang akan rusak bila lingkaran itu diganti `delay(3000)` disusul satu pemeriksaan `ackFlag`?

CHECKPOINT

Selang dari [TX] ke [FAIL] harus mendekati 3 detik, dan pengirim wajib melanjutkan ke DATA berikutnya. Pengirim yang membeku menandakan penungguan dilakukan dengan cara yang menghalangi interrupt.

4.8.3 EXP-03 — Nomor Urut Harus Cocok

Uji apa yang terjadi ketika balasan tidak sesuai harapan.

Uji	Perlakuan	Hasil yang diharapkan
03-a	Ubah penerima agar membalas ACK: 99 untuk semua paket	pengirim mencetak peringatan lalu tetap gagal setelah timeout
03-b	Ubah penerima agar membalas OKE	balasan diabaikan, pengirim tetap menunggu
03-c	Jalankan pasangan board lain berdekatan dengan program sama	catat apakah ada ACK milik pasangan lain yang diterima

Tabel 4.7: Perlakuan EXP-03 — pencocokan nomor urut

Data capture

Parameter	Hasil
Pesan pengirim saat menerima ACK bernomor salah	
Apakah OK ikut bertambah?	
Jumlah OK di penerima vs OK di pengirim setelah 20 siklus	

Tabel 4.8: Lembar pengamatan EXP-03

CHECKPOINT

Uji 03-c adalah yang paling membuka mata. Karena tidak ada alamat sama sekali pada LoRa mentah, ACK dari pasangan board lain berpotensi diterima. Pencocokan nomor urut menahan sebagian besar di antaranya, tetapi tidak seluruhnya — dan itulah alasan M05 memperkenalkan penomoran node.

4.8.4 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada dua Arduino Uno bershield Dragino LoRa v1.2, 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

Parameter	Hasil terukur
Keberhasilan (45 detik, jarak dekat)	14/14 — 100 % di kedua sisi
Waktu DATA → ACK min/maks/rata-rata	0 / 201 / 143 ms (4,8 % dari timeout)
Selang [TX] → [FAIL] saat penerima mati	3,05 s — sesuai ACK_TIMEOUT
Lama siklus sehat vs gagal	3,10 s vs 6,03 s — satu kegagalan mengandakannya
Pemulihan setelah penerima kembali	1 siklus (3,1 s)
EXP-03a/03b: balasan bernomor salah	tiba, dibaca, ditolak ; pengirim tetap menunggu sampai timeout

Parameter	Hasil terukur
EXP-03c: pasangan board lain	belum diuji — hanya dua shield berfungsi

Tabel 4.9: Hasil verifikasi perangkat keras Modul 04

Environment	Status	Flash
sender	SUCCESS	23.0% (7424 B)
receiver	SUCCESS	26.9% (8686 B)

4.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud 9600, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, ACK_TIMEOUT 3000 ms. Jarak antar-board ± 30 cm.

Board & Port

Peran	Environment	Port	Board
Sender	sender	/dev/ttyUSB0	Uno klon, bridge CH340 (1a86:7523)
Receiver	receiver	/dev/ttyACM1	Uno asli (2341:0043)

Tabel 4.10: Log serial Modul 04: Board & Port

Board Uno asli di /dev/ttyACM0 tidak dipakai: shield-nya gagal LoRa.begin() (lihat ../Modul01_lora_uart/logserial.md).

EXP-01 — Siklus ACK yang Sehat (45 detik)

```

[ 1.812] RX | === LoRa ACK RECEIVER ===
[ 1.812] RX | Init LoRa ... OK
[ 1.812] RX | Freq: 433.00 MHz
[ 1.812] RX | Menunggu paket DATA...
[ 1.822] TX | Freq: 433.00 MHz | SF7 | ACK timeout: 3000 ms
[ 1.822] TX | [TX] Kirim: DATA:0 ... selesai
[ 2.012] RX | Data : DATA:0
[ 2.012] RX | RSSI : -32.00 dBm
[ 2.012] RX | SNR : 9.00 dB
[ 2.012] RX | Total : 1

```

Parameter	Hasil
DATA dikirim	14 (nomor 0..13)
DATA tiba di penerima	14
ACK dibalas penerima	14
ACK kembali ke pengirim	14
Keberhasilan menurut pengirim	100 %
Keberhasilan menurut penerima	100 %
Penghitung firmware	OK 14 / FAIL 0
Waktu DATA→ACK min/maks/rata-rata	0 / 201 / 143 ms
RSSI rata-rata	-33,4 dBm
SNR rata-rata	9,07 dB
Flash sender / receiver	23,0 % (7.424 B) / 26,9 % (8.686 B)

Tabel 4.11: Log serial Modul 04: EXP-01 — Siklus ACK yang Sehat (45 detik)

Waktu pulang-pergi DATA→ACK sebesar 143 ms adalah 4,8 % dari ACK_TIMEOUT 3000 ms — batas waktu bawaan sangat longgar untuk jarak sedekat ini. Angka inilah bahan tabel C bagian Pengukuran.

EXP-02 — Batas Waktu Bekerja

Ketiadaan penerima ditirukan dengan mengunggah **sketsa diam** (radio tidak pernah diinisialisasi), bukan mencabut USB, agar waktunya tercatat tepat.

```
[ 11.90] * | >>> receiver kini diam
[ 11.20] TX | [TX] Kirim: DATA:3 ... selesai
[ 14.25] TX | [FAIL] Tidak ada ACK! | OK: 3 | FAIL: 1
[ 17.23] TX | [TX] Kirim: DATA:4 ... selesai
[ 20.28] TX | [FAIL] Tidak ada ACK! | OK: 3 | FAIL: 2
[ 23.27] TX | [TX] Kirim: DATA:5 ... selesai
[ 26.31] TX | [FAIL] Tidak ada ACK! | OK: 3 | FAIL: 3
[ 29.30] TX | [TX] Kirim: DATA:6 ... selesai
[ 32.34] TX | [FAIL] Tidak ada ACK! | OK: 3 | FAIL: 4
[ 35.33] TX | [TX] Kirim: DATA:7 ... selesai
[ 38.24] * | >>> receiver kembali aktif
[ 38.38] TX | [FAIL] Tidak ada ACK! | OK: 3 | FAIL: 5
[ 41.37] TX | [TX] Kirim: DATA:8 ... selesai
[ 41.52] TX | [OK] ACK diterima! | OK: 4 | FAIL: 5
```

Parameter	Hasil
Selang [TX] → [FAIL]	3,05 s — sesuai ACK_TIMEOUT 3000
Selang [TX] → [OK] saat sehat	0,12–0,18 s
Lama satu siklus saat sehat	3,10 s (0,1 s ACK + delay(3000))
Lama satu siklus saat gagal	6,03 s (3,0 s timeout + delay(3000))
Jumlah FAIL selama 26 detik sunyi	5
Apakah pengirim tetap melanjutkan?	ya , tanpa membeku
Siklus sampai pulih setelah penerima kembali	1 siklus (3,1 s)

Tabel 4.12: Log serial Modul 04: EXP-02 — Batas Waktu Bekerja

Temuan penting: satu kegagalan **menggandakan lama siklus**, dari 3,1 detik menjadi 6,0 detik. Pada sistem yang menjadwalkan banyak node — seperti Modul 05 — pelipatan ini langsung memperlambat pembacaan seluruh node lain.

EXP-03 — Nomor Urut Harus Cocok

Penerima diubah agar membalas dengan isi yang keliru, pengirim tidak disentuh.

03-a — penerima selalu membalas ACK:99

```
[ 1.91] [RX] Balasan: ACK:99
[ 1.97] [RX] WARN: bukan ACK yang diharapkan, tetap tunggu...
[ 4.83] [FAIL] Tidak ada ACK! | OK: 0 | FAIL: 1
```

03-b — penerima membalas OKE

```
[ 1.90] [RX] Balasan: OKE
[ 1.96] [RX] WARN: bukan ACK yang diharapkan, tetap tunggu...
[ 4.83] [FAIL] Tidak ada ACK! | OK: 0 | FAIL: 1
```

Uji	Balasan tiba?	Diterima sebagai ACK?	OK bertambah?	Hasil akhir
baseline	ya	ya	ya	[OK] dalam 0,15 s
03-a ACK:99	ya	tidak	tidak	WARN lalu [FAIL] setelah timeout
03-b OKE	ya	tidak	tidak	WARN lalu [FAIL] setelah timeout

Tabel 4.13: Log serial Modul 04: 03-b — penerima membalas OKE

Kedua uji berperilaku identik: paket **sampai** ke pengirim, dibaca, dicetak, lalu ditolak karena nomornya tidak cocok dengan yang sedang ditunggu. Pengirim tidak langsung menyerah — ia mencetak peringatan dan **tetap menunggu** sampai batas waktu habis, sesuai rancangan `while` di dalamnya. Perilaku ini yang membuat sistem kebal terhadap balasan basi maupun balasan milik pasangan board lain.

Perhatikan juga selisih waktunya: uji baseline selesai dalam 0,15 detik, sedangkan kedua uji ini menghabiskan 3,0 detik penuh sebelum menyatakan gagal — biaya menunggu balasan yang tidak akan pernah cocok.

03-c — pasangan board lain berdekatan

Belum diuji. Hanya tersedia dua shield LoRa yang berfungsi (satu shield gagal init), sehingga tidak ada pasangan board kedua untuk dijalankan berdampingan.

Catatan pengambilan log

- Seluruh perubahan sumber pada EXP-03 dikembalikan ke kondisi semula setelah pengujian, dan firmware baku diunggah ulang.
- Kolom “keberhasilan menurut pengirim” dan “menurut penerima” dihitung dari nomor DATA:n dan ACK:n yang benar-benar terlihat pada kedua aliran serial, bukan dari penghitung firmware saja. Pada jarak dekat keduanya sama; ketidaksepakatan baru muncul pada jarak jauh, dan itulah yang diminta tabel B bagian Pengukuran.
- Tabel pengukuran jarak dan pengaruh nilai timeout pada README belum terisi: percobaan ini dijalankan di satu meja pada jarak tetap ± 30 cm.

4.9 Pengukuran

A. Tingkat keberhasilan terhadap jarak — masing-masing 20 siklus.

Jarak	RSSI data (dBm)	SNR (dB)	OK	FAIL	Keberhasilan (%)
1 m					
25 m					
50 m					
100 m					

Tabel 4.14: Lembar pengukuran A — keberhasilan terhadap jarak

B. Ketidaksepakatan kedua sisi — bandingkan penghitung pengirim dan penerima pada jarak yang sama.

Jarak	OK di pengirim	Total di penerima	Selisih	Tafsiran
1 m				
50 m				
100 m				

Tabel 4.15: Lembar pengukuran B — ketidaksepakatan kedua sisi

C. Pengaruh nilai timeout — pada jarak tetap 50 m.

ACK_TIMEOUT	OK	FAIL	Keberhasilan (%)	Catatan
500 ms				

ACK_TIMEOUT	OK	FAIL	Keberhasilan (%)	Catatan
1000 ms				
3000 ms				

Tabel 4.16: Lembar pengukuran C — pengaruh nilai timeout

4.10 Analisis

1. Dari Tabel 4.14, pada jarak berapa tingkat keberhasilan mulai turun di bawah 90 %? Bandingkan dengan jarak tempat loss mulai terlihat pada M01.
2. Tabel 4.15 memperlihatkan penerima kadang mencatat lebih banyak keberhasilan daripada pengirim. Jelaskan mengapa hal itu mungkin terjadi, dan sisi mana yang lebih layak dipercaya.
3. Dari Tabel 4.16, apa akibat memperpendek timeout? Tentukan nilai terkecil yang masih masuk akal untuk SF7, disertai alasan berbasis waktu udara.
4. ACK menambah satu paket untuk setiap data yang dikirim. Hitung berapa persen tambahan lalu lintas radionya, dan sebutkan satu kondisi ketika tambahan itu tidak sepadan.
5. Rancang perbaikan agar DATA yang gagal dikirim ulang secara otomatis. Sebutkan risiko baru yang muncul, terutama kemungkinan penerima memproses data yang sama dua kali.

4.11 Concept Check

1. Apa perbedaan balasan pada M03 dan ACK pada modul ini?
2. Mengapa nomor pada ACK:n harus dicocokkan, bukan sekadar diterima apa adanya?
3. Mengapa penungguan ACK memakai `millis()`, bukan `delay()`?
4. Apa yang dibuktikan oleh ACK, dan apa yang **tidak** dibuktikannya?
5. Mengapa penerima diunggah lebih dahulu?

4.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Statistik yang langsung terbaca

CH-1 — Kirim ulang otomatis Ulangi pengiriman DATA:n maksimum tiga kali sebelum menyatakan gagal, dan tampilkan berapa kali percobaan diperlukan untuk tiap keberhasilan.

CH-2 — Persentase langsung Tampilkan tingkat keberhasilan berjalan dalam persen di setiap baris statistik pengirim.

Level 3 — Tantangan · Protokol yang menyesuaikan diri

CH-3 — Timeout adaptif Ukur waktu pulang-pergi sepuluh siklus pertama, lalu setelah timeout menjadi tiga kali rata-rata tersebut. Bandingkan hasilnya dengan nilai tetap 3000 ms.

CH-4 — ACK membawa informasi Sertakan RSSI penerima di dalam ACK (ACK:n: -45) sehingga pengirim mengetahui kualitas tautan dari sisi seberang. Jelaskan mengapa informasi itu tidak dapat diperoleh dengan cara lain.

4.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — ACK, timeout, nomor urut, kegagalan senyap.
3. Konfigurasi — nilai ACK_TIMEOUT, format DATA:n dan ACK:n, parameter radio.
4. Hasil eksperimen — log serial kedua board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 4.14, Tabel 4.15, dan Tabel 4.16.
6. Analisis dan concept check.
7. Challenge — minimal CH-1 dan CH-2.
8. Kesimpulan yang disusun sendiri, khususnya mengenai harga yang dibayar untuk memperoleh kepastian.

5. Modul 05 — Master-Slave 3 Node (Round-Robin Polling)

LoRa COMMUNICATION LAB · MODUL 05

Schedule the Airwaves — Master-Slave 3 Node (Round-Robin Polling)

Arduino Uno + Dragino LoRa Shield v1.2 · LoRa mentah · topologi bintang, 3 node · polling terjadwal · level Advanced · 3 × 50 menit · Serial 115200 baud · folder kode Modul05_lora_master_slave

PENGANTAR MODUL · Gambaran Umum

Modul 05 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut. Misinya menambah jumlah node menjadi tiga tanpa membiarkan mereka saling menimpa di udara: satu master memanggil tiap slave bergiliran, dan slave hanya bersuara ketika namanya disebut. Percobaan memakai tiga Arduino Uno bershield Dragino LoRa v1.2, diamati melalui tiga Serial Monitor pada 115200 baud.

5.1 Pendahuluan

Empat modul sebelumnya selalu melibatkan tepat dua board, sehingga tidak pernah ada pertanyaan siapa yang boleh bicara. Begitu node ketiga hadir, masalah baru muncul seketika: radio LoRa tidak memiliki alamat, tidak memiliki mekanisme penghindaran tabrakan, dan tidak dapat mendengar ketika sedang memancar. Bila dua slave menjawab bersamaan, kedua paket bertabrakan dan **tidak satu pun** yang dapat dipecahkan master — kegagalan yang bahkan tidak terlihat sebagai kesalahan, hanya sebagai sunyi. Modul ini menyelesaikannya dengan cara paling tua dan paling mudah dibuktikan: penjadwalan terpusat, tempat hak bicara diberikan satu per satu oleh master.

Prasyaratnya adalah M03 untuk percakapan dua arah dan penanda identitas pada payload, serta M04 untuk penungguan terbatas waktu dan statistik keberhasilan. Yang dibangun di sini adalah pengalaman di lapisan aplikasi (POLL:1, S1:DATA:n), penjadwalan round-robin, penyaringan paket yang bukan miliknya, batas waktu per node, serta statistik per node yang terpisah. Pola master-slave ini adalah bentuk paling sederhana dari penjadwalan medium yang pada jaringan sungguhan dikerjakan lapisan MAC.

Peta modul LoRa (penutup arc pertama)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan lewat interrupt — loop() tidak lagi menunggu
03	Dua arah bergantian di atas radio half-duplex
04	Setiap pengiriman diketahui hasilnya: ACK, timeout, statistik
05 (ini)	Banyak node — hak bicara dijadwalkan agar tidak bertabrakan

Tabel 5.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Perintah master berbentuk POLL:<id> dan jawaban slave berbentuk S<id>:DATA:<n>. Nomor id mengikat keduanya, sedangkan n adalah penghitung lokal slave yang **naik hanya setelah pengiriman benar-benar dilakukan** — sehingga angka itu mewakili jumlah jawaban yang sungguh dikirim, bukan jumlah niat mengirim. Perbedaan halus ini menjadi bahan analisis tersendiri.

5.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menjadwalkan hak bicara banyak node di atas satu kanal

1. Menjelaskan mengapa tabrakan paket pasti terjadi pada LoRa mentah bila beberapa node bicara tanpa penjadwalan, dan mengapa tabrakan itu tidak terlihat sebagai pesan kesalahan.
2. Menerapkan pengalamatan di lapisan aplikasi ketika lapisan radio tidak menyediakannya.
3. Membangun penjadwalan round-robin dengan batas waktu per node, dan menjelaskan pengaruh nilai batas waktu terhadap lama siklus.
4. Mengukur keberhasilan dan waktu tanggap **per node**, bukan gabungan, lalu menjelaskan penyebab perbedaannya.
5. Menjelaskan batas skala pendekatan master-slave dan memperkirakan titik ketika pendekatan itu tidak lagi memadai.

Kriteria keberhasilan

- ☐ Master menyelesaikan siklus penuh: memanggil Slave 1 lalu Slave 2 bergantian tanpa henti.
- ☐ Setiap slave hanya menjawab panggilan bernomor dirinya, dan mencetak [IGNORE] untuk yang lain.
- ☐ Statistik OK dan FAIL tercatat terpisah untuk tiap slave.
- ☐ Ketika satu slave dimatikan, master tetap melayani slave lainnya dan mencatat kegagalan pada slave yang hilang.
- ☐ Lama siklus terukur dan dijelaskan penyusunnya.

5.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Master	Node yang menjadwalkan, memanggil tiap slave bergiliran. Hanya ada satu.
Slave	Node yang diam sampai dipanggil, lalu menjawab tepat satu kali.
Round-robin	Penjadwalan bergilir merata: 1, 2, 1, 2, ... tanpa prioritas.
Tabrakan	Dua paket mengudara bersamaan sehingga saling merusak; penerima tidak menerima apa pun.
Masalah hidden node	Dua slave yang saling tidak terdengar tetap dapat bertabrakan di posisi master. Penjadwalan terpusat menghindarinya.
Batas waktu polling	Lama master menunggu jawaban sebelum menyatakan slave tidak merespons (500 ms pada modul ini).
Lama siklus	Waktu satu putaran penuh memanggil seluruh slave. Menentukan seberapa sering tiap node terbaca.

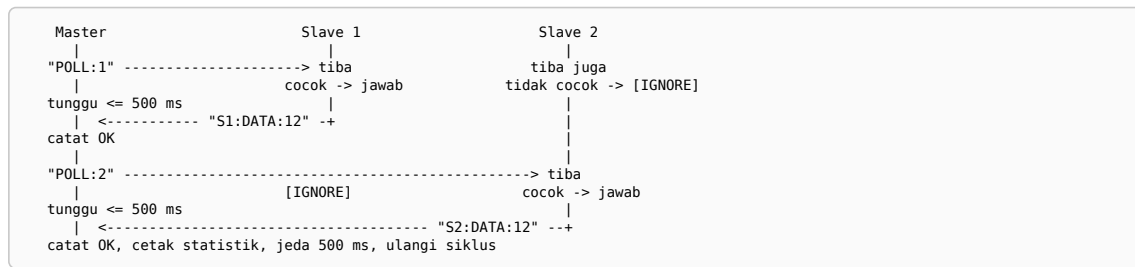
Tabel 5.2: Istilah kerja Modul 05

Mengapa penjadwalan lebih dahulu, bukan pengalamatan lebih dahulu. Memberi nomor pada tiap node menyelesaikan persoalan “pesan ini untuk siapa”, tetapi tidak menyelesaikan “siapa yang boleh bicara sekarang”. Seandainya kedua slave dibiarkan mengirim sendiri secara berkala, penomoran tetap tidak menolong: dua paket yang bertabrakan di udara rusak sebelum sempat dibaca nomornya. Karena itu master memberi hak bicara satu per satu, dan nomor node hanya dipakai untuk memastikan jawaban yang tiba berasal dari node yang sedang dipanggil.

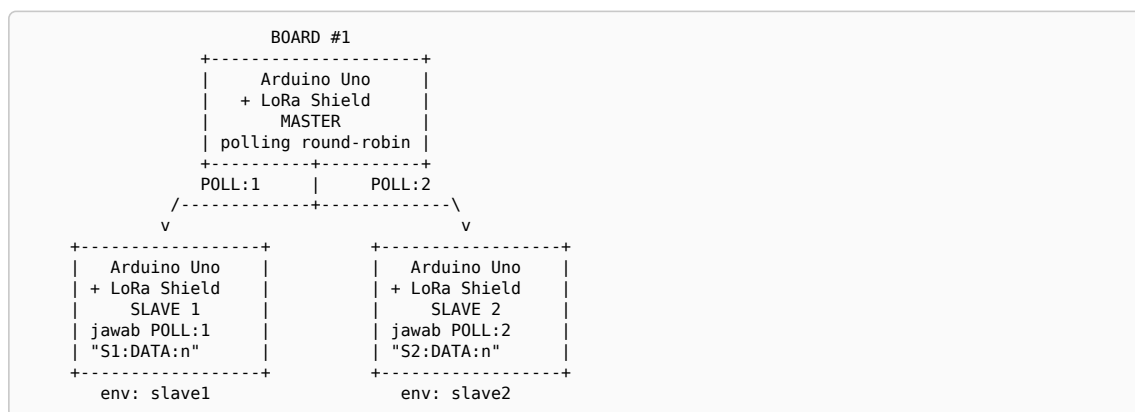
Mengapa lama siklus penting. Master menunggu paling lama 500 ms untuk tiap slave. Bila kedua slave menjawab cepat, satu siklus selesai dalam ratusan milidetik; bila keduanya mati, siklus memakan sekitar satu detik penuh hanya untuk menunggu kesunyian. Dengan sepuluh slave, satu node yang mati

memperlambat pembacaan seluruh node lain — sifat yang perlu diperhitungkan sebelum jumlah node ditambah.

Sekuens yang diamati



5.4 Topologi



Node	Environment	Build flag	Peran	Batas waktu
Master	master	—	Memanggil bergiliran, mencatat statistik	500 ms per slave
Slave 1	slave1	-DSLAVE_ID=1	Menjawab POLL:1	—
Slave 2	slave2	-DSLAVE_ID=2	Menjawab POLL:2	—

Tabel 5.3: Peran tiap node Modul 05

Kedua slave memakai **file source yang sama**, `src/slave/main.cpp`; nomornya ditentukan build flag. Menambah slave ketiga berarti menambah satu environment dan menaikkan `SLAVE_COUNT` di master.

5.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3
5	PC/Laptop	PlatformIO Core/IDE, idealnya 3 port USB bebas	1

Tabel 5.4: Alat dan bahan Modul 05

PENTING

Baud modul ini 115200, berbeda dari M01–M04 yang memakai 9600. Serial Monitor yang masih terasetel 9600 akan menampilkan karakter acak — gejala yang sering disangka kerusakan perangkat.

Struktur proyek

```

Modul05_lora_master_slave/
├── platformio.ini          ← 3 environment; nomor slave lewat build flag
├── monitor_serial.py       ← pantau 3 node, ringkas siklus/keberhasilan/[IGNORE]
├── lora_monitor.py         ← dashboard tiga node dalam satu layar (butuh `rich`)
├── upload_auto.py         ← deteksi port otomatis saat unggah
├── logserial.md            ← log referensi hasil uji perangkat
├── lora_session_20260516_071007.csv ← contoh rekaman sesi (referensi)
└── src/
    ├── master/main.cpp    ← polling round-robin + statistik per node
    └── slave/main.cpp     ← satu source untuk kedua slave

```

5.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul05_lora_master_slave/](#)

- [Modul05_lora_master_slave/platformio.ini](#)
- [Modul05_lora_master_slave/src/master/main.cpp](#)
- [Modul05_lora_master_slave/src/slave/main.cpp](#)
- [Modul05_lora_master_slave/monitor_serial.py](#)
- [Modul05_lora_master_slave/lora_monitor.py](#)
- [Modul05_lora_master_slave/upload_auto.py](#)

```

[platformio]
default_envs = master

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
; Modul ini memakai 115200 baud, berbeda dari M01-M04 yang memakai 9600.
monitor_speed = 115200

[env:master]
build_src_filter = +<master/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua slave memakai source yang sama; nomornya ditentukan build flag.
[env:slave1]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=1
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:slave2]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=2
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2

```

Kode 5.1: platformio.ini Modul 05 — tiga environment, nomor slave lewat build flag

```

/*
LoRa Master-Slave 3 Node - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: master

Topologi Master-Slave dengan 3 node:
- 1 Master (env master) : polling Slave 1 & Slave 2 secara bergantian
- 2 Slave (env slave1, slave2) : hanya merespon saat dipanggil

Mekanisme:
TX : blocking (endPacket) – sederhana dan andal di AVR
RX : polling via LoRa.parsePacket() – tidak perlu ISR

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_BUILTIN      LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define POLL_TIMEOUT      500
#define CYCLE_INTERVAL    500
#define SLAVE_COUNT       2

int cycle = 0;
int s1ok = 0, s1Fail = 0, s1Data = 0;
int s2ok = 0, s2Fail = 0, s2Data = 0;

unsigned long cycleStartMs = 0;

void transmit(const String& msg) {
  LoRa.beginPacket();
  LoRa.print(msg);
  LoRa.endPacket();
}

bool pollSlave(int slaveNum, int& dataOut, int& okCount, int& failCount) {
  String pollMsg = "POLL:" + String(slaveNum);
  Serial.print(F("[TX] ")); Serial.println(pollMsg);
  transmit(pollMsg);

  LoRa.receive();

  unsigned long waitStart = millis();
  while (millis() - waitStart < POLL_TIMEOUT) {
    int ps = LoRa.parsePacket();
    if (ps > 0) {
      String reply = "";
      while (LoRa.available()) {
        reply += (char)LoRa.read();
      }

      String prefix = "S" + String(slaveNum) + ":DATA:";
      if (reply.startsWith(prefix)) {
        dataOut = reply.substring(prefix.length()).toInt();
        okCount++;
        Serial.print(F("[RX] ")); Serial.print(reply);
        Serial.print(F(" | RSSI: ")); Serial.print(LoRa.packetRssi());
        Serial.print(F(" dBm | SNR: ")); Serial.print(LoRa.packetSnr());
        Serial.println(F(" dB"));
        return true;
      } else {
        Serial.print(F("[WARN] Balasan tidak valid: ")); Serial.println(reply);
      }
    }
  }

  failCount++;
  Serial.print(F("[FAIL] Slave ")); Serial.print(slaveNum);

```

```

    Serial.println(F(" tidak merespon!"));
    return false;
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.println(F("=== LoRa MASTER-SLAVE 3 NODE ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("SF: ")); Serial.print(SPREADING_FACTOR);
    Serial.print(F(" | BW: ")); Serial.print(BANDWIDTH / 1E3); Serial.println(F(" kHz"));
    Serial.println(F("Peran: MASTER (env master)"));
    Serial.println(F("Slave: env slave1 (S1) & slave2 (S2)"));
    Serial.println();
}

void loop() {
    cycle++;
    cycleStartMs = millis();

    Serial.println(F("====="));
    Serial.print(F("=== CYCLE ")); Serial.print(cycle);
    Serial.println(F(" ==="));

    // --- Poll Slave 1 ---
    digitalWrite(LED_PIN, HIGH);
    pollSlave(1, s1Data, s1Ok, s1Fail);
    digitalWrite(LED_PIN, LOW);

    // --- Poll Slave 2 ---
    digitalWrite(LED_PIN, HIGH);
    pollSlave(2, s2Data, s2Ok, s2Fail);
    digitalWrite(LED_PIN, LOW);

    // --- Statistik ---
    unsigned long duration = millis() - cycleStartMs;
    Serial.println(F("--- STATISTIK ---"));
    Serial.print(F("S1: OK=")); Serial.print(s1Ok);
    Serial.print(F(" | FAIL=")); Serial.print(s1Fail);
    Serial.print(F(" | Data: ")); Serial.println(s1Data);
    Serial.print(F("S2: OK=")); Serial.print(s2Ok);
    Serial.print(F(" | FAIL=")); Serial.print(s2Fail);
    Serial.print(F(" | Data: ")); Serial.println(s2Data);
    Serial.print(F("Durasi siklus: ")); Serial.print(duration); Serial.println(F(" ms"));
    Serial.println(F("=====\\n"));

    delay(CYCLE_INTERVAL);
}

```

Kode 5.2: src/master/main.cpp — polling round-robin dan statistik per node

```

/*
LoRa Master-Slave 3 Node - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: slave1 / slave2

Slave 1 hanya merespon ketika Master mengirim "POLL:1".
Balasan: "S1:DATA:N" — N adalah counter lokal yang naik setiap kali dipoll.

Mekanisme:

```

```

TX : blocking (endPacket) – sederhana dan andal di AVR
RX : polling via LoRa.parsePacket() – tidak perlu ISR

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DIO0 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DIO0_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR 7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor slave datang dari build flag (-DSLAVE_ID=1 atau 2) di platformio.ini,
// sehingga kedua slave memakai file source yang sama persis.
#ifndef SLAVE_ID
#define SLAVE_ID 1
#endif

unsigned long dataCounter = 0; // unsigned long: aman hingga 4.294.967.295 (vs int yang overflow di 32.767)
int rxCount = 0;

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.print(F("=== LoRa SLAVE "));
    Serial.print(SLAVE_ID);
    Serial.println(F(" =="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("Menunggu POLL:")); Serial.print(SLAVE_ID); Serial.println(F(" dari Master...\n"));
}

void loop() {
    int ps = LoRa.parsePacket();
    if (ps == 0) return;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    if (!received.equals("POLL:" + String(SLAVE_ID))) {
        Serial.print(F("[IGNORE ]")); Serial.println(received);
        return;
    }

    rxCount++;
}

```

```

// dataCounter++ sengaja dipindah ke SETELAH transmit()
// Filosofi: counter merepresentasikan "berapa kali slave BERHASIL kirim ke master"
// bukan "berapa kali slave berniat kirim" – kalau TX gagal, counter tidak berhak naik
digitalWrite(LED_PIN, HIGH);

Serial.print(F("[RX] ")); Serial.print(received);
Serial.print(F(" | RSSI: ")); Serial.print(LoRa.packetRssi());
Serial.print(F(" dBm | SNR: ")); Serial.print(LoRa.packetSnr());
Serial.print(F(" dB | RX#: ")); Serial.println(rxCount);

dataCounter++; // naik SETELAH TX – nilai ini yang dikirim, mencerminkan jumlah TX aktual
String reply = "S" + String(SLAVE_ID) + ":DATA:" + String(dataCounter);
Serial.print(F("[TX] ")); Serial.println(reply);

transmit(reply);
digitalWrite(LED_PIN, LOW);
Serial.println();
}

```

Kode 5.3: src/slave/main.cpp — satu source untuk kedua slave

```

#!/usr/bin/env python3
"""Monitor master dan kedua slave sekaligus untuk Modul 05.

Menampilkan kedua aliran serial dalam SATU jendela dengan timestamp bersama,
lalu meringkasnya saat berhenti: berapa siklus polling selesai, berapa lama
tiap siklus, dan berapa keberhasilan tiap slave secara terpisah.

Yang paling sulit dinilai dari tiga jendela terpisah adalah pertanyaan inti
modul ini: benarkah slave hanya menjawab ketika namanya disebut. Dengan satu
sumbu waktu, baris [IGNORE] pada satu slave terlihat berdampingan dengan
[TX] POLL milik slave lain, sehingga penyaringan itu dapat dibuktikan langsung
alih-alih disimpulkan.

Modul ini memakai 115200 baud, berbeda dari Modul 01-04 yang memakai 9600.

python3 Modul05_lora_master_slave/monitor_serial.py
python3 Modul05_lora_master_slave/monitor_serial.py --log sesil.txt
python3 Modul05_lora_master_slave/monitor_serial.py --port S2=/dev/ttyUSB0
python3 Modul05_lora_master_slave/monitor_serial.py --durasi 60 # berhenti sendiri

Butuh pyserial (`pip install pyserial`; sudah ikut terpasang bersama PlatformIO).
Hentikan dengan Ctrl-C, atau pakai `--durasi` agar berhenti otomatis.

PORT OTOMATIS. Port MASTER/S1/S2 tidak lagi ditulis tetap di kode: setiap
kali dijalankan, skrip memindai /dev/ttyACM* dan /dev/ttyUSB* yang sedang
aktif dan memakai yang benar-benar tersambung,urut sesuai nama device. Ini
penting karena nama port bisa berubah tiap kali board dicabut-pasang atau
board asli/klon bercampur (lihat tools/deteksi_port.py). Bila urutan
otomatisnya keliru, timpa lewat --port, mis. --port MASTER=/dev/ttyACM1.

SOAL RESET OTOMATIS. Pada Arduino Uno, jalur DTR terhubung ke pin RESET lewat
kapasitor. Membuka port serial **selalu me-reset board**, dan hal itu justru
dimanfaatkan di sini: pesan `setup()` beserta baris `Init LoRa ... OK` ikut
terekam tanpa perlu menekan tombol reset. Konsekuensinya, penghitung paket
kedua board kembali ke nol setiap kali monitor dijalankan – jalankan monitor
lebih dahulu, baru mulai mengukur, dan jangan membukanya di tengah percobaan
panjang.

Pada Uno, DTR hanya menyentuh jalur RESET dan tidak tersambung ke GPIO mana pun,
sehingga membuka port tidak pernah membuat sebuah pin terbaca seolah ditekan.
"""
import argparse
import glob
import re
import signal
import sys
import threading
import time

try:
    import serial
except ImportError:
    sys.exit("pyserial belum terpasang. Jalankan: pip install pyserial")

# Nama peran bawaan (urutan) dan warna yang dipasangkan ke port yang
# terdeteksi aktif, bukan ke nama /dev tertentu – lihat deteksi_port_aktif().
NAMA_PERAN_BAWAAN = ["MASTER", "S1", "S2"]

```

```

PALET_WARNA = ["\033[32m", "\033[36m", "\033[35m", "\033[33m", "\033[34m"]

def deteksi_port_aktif():
    """Kumpulkan port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* - pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil ulang tiap program start supaya daftarnya selalu mengikuti apa
    yang benar-benar tersambung saat itu, bukan port tetap di kode.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def bangun_port_bawaan():
    """Pasangkan tiap port aktif ke nama peran (MASTER, S1, S2, lalu P4, ...)."""
    hasil = []
    for i, dev in enumerate(deteksi_port_aktif()):
        nama = NAMA_PERAN_BAWAAN[i] if i < len(NAMA_PERAN_BAWAAN) else f"P{i + 1}"
        warna = PALET_WARNA[i % len(PALET_WARNA)]
        hasil.append((nama, dev, warna))
    return hasil

# Pola yang dipanen dari baris serial untuk menyusun ringkasan
RE_CYCLE = re.compile(r"=== CYCLE (\d+)")
RE_DURASI = re.compile(r"Durasi siklus:\s*(\d+)")
RE_STAT = re.compile(r"S(\d+): OK=(\d+) \| FAIL=(\d+) \| Data:\s*(\d+)")
RE_POLL = re.compile(r"\[TX\] POLL:(\d+)")
RE_JAWAB = re.compile(r"\[RX\] S(\d+):DATA:(\d+)")
RE_GAGAL = re.compile(r"\[FAIL\] Slave (\d+)")
RE_IGNORE = re.compile(r"\[IGNORE\]")
RE_RSSI = re.compile(r"RSSI\s*:\s*(-?\d+)")
RE_SNR = re.compile(r"SNR\s*:\s*(-?[\d.]+)")
RESET = "\033[0m"
DIM = "\033[2m"

print_lock = threading.Lock()
counts = {}
# Bahan ringkasan akhir; diisi reader, dibaca sekali saat program berhenti
siklus = [] # nomor siklus yang terlihat di master
durasi = [] # lama tiap siklus, ms
stat = {} # "1"/"2" -> (ok, fail, data) terakhir yang dilaporkan master
poll = {} # nomor slave -> berapa kali dipanggil
jawab = {} # nomor slave -> berapa kali menjawab
gagal = {} # nomor slave -> berapa kali tidak merespons
abai = {} # nama node -> berapa kali mencetak [IGNORE]
rssi_per, snr_per = {}, {}
stop = threading.Event()
t0 = time.time()

def show(name, color, text, use_color, logfile):
    """Cetak satu baris dengan timestamp bersama, aman dari tumpang tindih."""
    stamp = f"{time.time() - t0:8.3f}"
    plain = f"[{stamp}] {name:<7} | {text}"
    with print_lock:
        if use_color:
            print(f"[DIM][{stamp}]{RESET} {color}{name:<7}{RESET} | {text}", flush=True)
        else:
            print(plain, flush=True)
        if logfile:
            logfile.write(plain + "\n")
            logfile.flush()

def open_port(port, baud):
    """Buka port serial menuju board Arduino.

    Jalur DTR/RTS sengaja TIDAK disentuh. Menyetelnya sebelum open() ditolak
    oleh CDC ATmega16U2 pada Uno asli dan berakhir sebagai
    `[Errno 110] Connection timed out`; pada klon ber-bridge CH340 hal itu
    kebetulan lolos, sehingga kesalahannya mudah tidak terlihat bila diuji
    hanya pada satu jenis board.

    Membuka port tetap me-reset board sekali - itu memang yang diinginkan,
    karena pesan setup() ikut terekam - dan reset itu berupa pulsa dari
    kapasitor, bukan penahanan, sehingga board langsung berjalan normal.
    """
    return serial.Serial(port, baud, timeout=0.2)

def panen(name, text):

```



```

"""Ambil kejadian siklus, polling, jawaban, dan penolakan dari satu baris.

Penghitug diambil dari sisi master untuk keberhasilan, dan dari sisi slave
untuk [IGNORE] - karena hanya slave yang tahu ia menerima panggilan milik
node lain lalu membuangnya.
"""
if name == "MASTER":
    if (m := RE_CYCLE.search(text)):
        siklus.append(int(m.group(1)))
    elif (m := RE_DURASI.search(text)):
        durasi.append(int(m.group(1)))
    elif (m := RE_STAT.search(text)):
        stat[m.group(1)] = tuple(int(x) for x in m.groups()[1:])
    elif (m := RE_POLL.search(text)):
        poll[m.group(1)] = poll.get(m.group(1), 0) + 1
    elif (m := RE_JAWAB.search(text)):
        jawab[m.group(1)] = jawab.get(m.group(1), 0) + 1
    elif (m := RE_GAGAL.search(text)):
        gagal[m.group(1)] = gagal.get(m.group(1), 0) + 1
    elif RE_IGNORE.search(text):
        abai[name] = abai.get(name, 0) + 1

    if (m := RE_RSSI.search(text)):
        rssi_per.setdefault(name, []).append(int(m.group(1)))
    if (m := RE_SNR.search(text)):
        snr_per.setdefault(name, []).append(float(m.group(1)))

def ringkasan():
    """Cetak hasil ukur tautan: loss, RSSI, dan SNR."""
    print("\n" + "-" * 60)
    print(f"Durasi: {time.time() - t0:.1f} s")
    for nama in counts:
        print(f" {nama:<7} : {counts.get(nama, 0)} baris")

    if not siklus:
        print("\nTidak ada siklus polling yang terbaca - periksa master.")
        return

    print(f"\n Siklus polling : {len(siklus)} (nomor {min(siklus)}..{max(siklus)})")
    if durasi:
        print(f" Lama siklus min/maks/rata-rata : {min(durasi)} / {max(durasi)} / "
              f"{sum(durasi) / len(durasi):.0f} ms")

    for sid in sorted(set(list(poll) + list(jawab) + list(gagal))):
        p, j, g = poll.get(sid, 0), jawab.get(sid, 0), gagal.get(sid, 0)
        persen = j / p * 100 if p else 0
        print(f"\n Slave {sid} : dipanggil {p} menjawab {j} gagal {g}"
              f" -> keberhasilan {persen:.1f} %")
        if sid in stat:
            ok, fail, data = stat[sid]
            print(f" penghitung master: OK={ok} FAIL={fail} Data={data}")

    for nama in counts:
        if nama in abai:
            print(f"\n {nama} membuang {abai[nama]} panggilan milik node lain ([IGNORE])")
        r = rssi_per.get(nama)
        if r:
            print(f" {nama} RSSI min/maks/rata-rata : {min(r)} / {max(r)} / "
                  f"{sum(r) / len(r):.1f} dBm")

def reader(name, port, color, baud, use_color, logfile):
    counts[name] = 0
    try:
        ser = open_port(port, baud)
    except Exception as e:
        show(name, color, f"!! tidak bisa dibuka: {e}", use_color, logfile)
        return

    show(name, color, f"-- tersambung ke {port} @ {baud} --", use_color, logfile)
    buf = b""
    while not stop.is_set():
        try:
            data = ser.read(256)
        except Exception as e:
            show(name, color, f"!! port terputus: {e}", use_color, logfile)
            break
        if not data:
            continue
        buf += data
        # Pesan dipecah per baris agar output tiga board tidak saling menyisip

```

```

        # di tengah kalimat.
        *lines, buf = buf.split(b"\n")
        for line in lines:
            text = line.decode("utf-8", "replace").rstrip("\r")
            if text.strip():
                counts[name] += 1
                panen(name, text)
                show(name, color, text, use_color, logfile)
ser.close()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--port", action="append", metavar="NAMA=/dev/ttyXXX",
                    help="ganti port MASTER/S1/S2, mis. S2=/dev/ttyACM0; boleh diulang")
    # Modul ini memakai 115200, sesuai Serial.begin() di src/ dan monitor_speed
    # di platformio.ini. Modul 01-04 memakai 9600.
    ap.add_argument("--baud", type=int, default=115200)
    ap.add_argument("--durasi", type=float, metavar="DETIK",
                    help="berhenti otomatis setelah sekian detik (mis. --durasi 60)")
    ap.add_argument("--log", metavar="FILE", help="simpan juga ke file teks")
    ap.add_argument("--no-color", action="store_true")
    args = ap.parse_args()

    port_bawaan = bangun_port_bawaan()
    if not port_bawaan and not args.port:
        sys.exit("Tidak ada port serial USB yang terdeteksi (/dev/ttyACM*, /dev/ttyUSB*). \n"
                "Periksa kabel data (bukan charge-only) dan hak akses grup dialout, \n"
                "atau jalankan tools/deteksi_port.py untuk diagnosis lebih lanjut, \n"
                "atau tentukan port manual lewat --port MASTER=/dev/ttyACM0 --port S1=/dev/ttyACM1 --port"
                "S2=/dev/ttyACM2")

    dipakai = port_bawaan[:len(NAMA_PERAN_BAWAAN)]
    sisa = port_bawaan[len(NAMA_PERAN_BAWAAN):]
    print(f"Port aktif terdeteksi: {'', '.join(p for _, p, _ in port_bawaan)}"
          if port_bawaan else "Tidak ada port aktif terdeteksi otomatis – memakai --port saja.")
    for nama, port, _ in dipakai:
        print(f"  -> {nama} = {port}")
    if sisa:
        print(f" (tidak dipakai: {'', '.join(p for _, p, _ in sisa)}"
              f"  -> pakai --port NAMA=/dev/... untuk memakainya)")

    ports = {name: (port, color) for name, port, color in dipakai}
    for item in args.port or []:
        if "=" not in item:
            sys.exit(f"format --port salah: {item!r} (harus NAMA=/dev/ttyXXX)")
        name, port = item.split("=", 1)
        _, color = ports.get(name.upper(), (None, "\033[35m"))
        ports[name.upper()] = (port, color)

    if not ports:
        sys.exit("Tidak ada port untuk dipantau – tentukan lewat --port NAMA=/dev/ttyXXX")

    use_color = not args.no_color and sys.stdout.isatty()
    logfile = open(args.log, "w") if args.log else None

    # SIGTERM (mis. dijalankan lewat `timeout 30 ...`) cukup menyalakan flag
    # `stop` yang sama dipakai reader thread, BUKAN melempar exception --
    # melempar exception dari signal handler bisa mendarat di tengah blok
    # `finally` (mis. saat t.join() atau ringkasan() berjalan) dan menyebabkan
    # crash tak tertangkap alih-alih mencetak ringkasan dengan rapi.
    signal.signal(signal.SIGTERM, lambda *_: stop.set())

    batas = f" · berhenti otomatis {args.durasi:.0f} s" if args.durasi else ""
    print(f"Monitor LoRa TX/RX @ {args.baud} baud – Ctrl-C untuk berhenti{batas}"
          f"{' · log: ' + args.log if args.log else ''}")
    print("-" * 60)

    threads = [threading.Thread(target=reader,
                                args=(name, port, color, args.baud, use_color, logfile),
                                daemon=True)
               for name, (port, color) in ports.items()]
    for t in threads:
        t.start()

    batasWaktu = t0 + args.durasi if args.durasi else None
    try:
        while any(t.is_alive() for t in threads) and not stop.is_set():
            if batasWaktu and time.time() >= batasWaktu:
                break
            time.sleep(0.2)

```

```

except KeyboardInterrupt:
    pass
finally:
    stop.set()
    for t in threads:
        t.join(timeout=1.0)
    ringkasan()
    if logfile:
        logfile.close()
        print(f"\nLog tersimpan di {args.log}")

if __name__ == "__main__":
    main()

```

Kode 5.4: monitor_serial.py — pantau tiga node pada satu sumbu waktu

```

#!/usr/bin/env python3
"""
LoRa Master-Slave Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Master <-> Slave 1 <-> Slave 2

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
    pip install pyserial rich

Cara pakai:
    python lora_monitor.py                # port bawaan sesuai OS
    python lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyUSB0
    python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.table import Table
    from rich.layout import Layout
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.columns import Columns
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# =====
# KONFIGURASI
# =====
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --master/--s1/--s2.
if sys.platform.startswith("win"):
    DEFAULT_MASTER_PORT = "COM3"
    DEFAULT_S1_PORT = "COM4"
    DEFAULT_S2_PORT = "COM5"
else:

```

```

    DEFAULT_MASTER_PORT = "/dev/ttyACM0"
    DEFAULT_S1_PORT      = "/dev/ttyACM1"
    DEFAULT_S2_PORT      = "/dev/ttyACM2"
    DEFAULT_BAUD         = 115200
    LOG_LINES            = 18
    REFRESH_HZ           = 4
    FAIL_STREAK_WARN     = 3
    CYCLE_SLOW_MS        = 900

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts:      str
    node:    str
    kind:    str
    detail:  str

@dataclass
class MasterState:
    port:      str = DEFAULT_MASTER_PORT
    connected: bool = False
    cycle:     int = 0
    last_cycle_ms: int = 0
    s1_ok:     int = 0
    s1_fail:   int = 0
    s1_data:   int = 0
    s2_ok:     int = 0
    s2_fail:   int = 0
    s2_data:   int = 0
    s1_fail_streak: int = 0
    s2_fail_streak: int = 0
    last_s1_data: int = 0
    last_s2_data: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=200))

@dataclass
class SlaveState:
    name:      str = ""
    port:      str = ""
    connected: bool = False
    rx_count:  int = 0
    data_counter: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=200))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.master = MasterState()
        self.s1 = SlaveState("SLAVE 1", DEFAULT_S1_PORT)
        self.s2 = SlaveState("SLAVE 2", DEFAULT_S2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSERS
# -----

def parse_master_line(line: str, st: MasterState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = re.search(r'CYCLE\s+(\d+)', line)
    if m:
        st.cycle = int(m.group(1))

    m = re.search(r'\[RX\]\s+S1:DATA:(\d+)', line)
    if m:
        v = int(m.group(1))
        if st.last_s1_data > 0 and v > st.last_s1_data + 1:
            anomalies.append(Anomaly(ts, "S1", "COUNTER_JUMP",
                                     f"S1:DATA loncat {st.last_s1_data} -> {v} (skip {v - st.last_s1_data - 1})"))
        st.last_s1_data = v
        st.s1_data = v
        st.s1_fail_streak = 0

    m = re.search(r'\[RX\]\s+S2:DATA:(\d+)', line)
    if m:
        v = int(m.group(1))

```

```

        if st.last_s2_data > 0 and v > st.last_s2_data + 1:
            anomalies.append(Anomaly(ts, "S2", "COUNTER_JUMP",
                                     f"S2:DATA loncat {st.last_s2_data} -> {v} (skip {v - st.last_s2_data - 1})"))
        st.last_s2_data = v
        st.s2_data = v
        st.s2_fail_streak = 0

    m = re.search(r'\[FAIL\]\s+Slave\s+(\d+)', line)
    if m:
        n = int(m.group(1))
        if n == 1:
            st.s1_fail += 1
            st.s1_fail_streak += 1
            if st.s1_fail_streak >= FAIL_STREAK_WARN:
                anomalies.append(Anomaly(ts, "S1", "FAIL_STREAK",
                                         f"S1 FAIL beruntun {st.s1_fail_streak}x"))
        else:
            st.s2_fail += 1
            st.s2_fail_streak += 1
            if st.s2_fail_streak >= FAIL_STREAK_WARN:
                anomalies.append(Anomaly(ts, "S2", "FAIL_STREAK",
                                         f"S2 FAIL beruntun {st.s2_fail_streak}x"))

    m = re.search(r'S1:\s*OK=(\d+)\s*\|\s*FAIL=(\d+)', line)
    if m:
        st.s1_ok = int(m.group(1)); st.s1_fail = int(m.group(2))

    m = re.search(r'S2:\s*OK=(\d+)\s*\|\s*FAIL=(\d+)', line)
    if m:
        st.s2_ok = int(m.group(1)); st.s2_fail = int(m.group(2))

    m = re.search(r'Durasi\s+siklus:\s*(\d+)', line)
    if m:
        ms = int(m.group(1))
        st.last_cycle_ms = ms
        if ms > CYCLE_SLOW_MS:
            anomalies.append(Anomaly(ts, "MASTER", "SLOW_CYCLE",
                                     f"Cycle {st.cycle} lambat: {ms}ms (> {CYCLE_SLOW_MS}ms)"))

def parse_slave_line(line: str, st: SlaveState):
    m = re.search(r'RX#:\s*(\d+)', line)
    if m:
        st.rx_count = int(m.group(1))
    m = re.search(r'\[TX\]\s+S\d+:DATA:(\d+)', line)
    if m:
        st.data_counter = int(m.group(1))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
                  shared: SharedState, is_master: bool):
    if is_master:
        state = shared.master
    elif node_key == "S1":
        state = shared.s1
    else:
        state = shared.s2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                if is_master:
                                    parse_master_line(line, state, shared.anomalies)
                                else:
                                    parse_slave_line(line, state)
                            shared.csv_queue.put({
                                'ts': ts, 'node': node_key, 'raw': line

```

```

        })
        except (serial.SerialException, OSError):
            break
    except (serial.SerialException, OSError):
        with shared.lock:
            state.connected = False
            time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                           'node': row['node'],
                           'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"
    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str, is_master: bool) -> str:
    if '[FAIL]' in line: return "red"
    if '[RX]' in line: return "green" if is_master else "yellow"
    if '[TX]' in line: return "yellow" if is_master else "green"
    if 'CYCLE' in line: return "bold cyan"
    if '[IGNORE]' in line: return "dim"
    return "dim white"

def build_master_panel(st: MasterState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")
    t.append(" Cycle      : ", style="dim")
    t.append(f"{st.cycle}\n", style="bold white")

    ms = st.last_cycle_ms
    t.append(" Dur. Cycle  : ", style="dim")
    t.append(f"{ms} ms", style="bold red" if ms > CYCLE_SLOW_MS else "bold green")
    if ms > CYCLE_SLOW_MS:
        t.append(" LAMBAT!", style="bold red")
    t.append("\n\n")

    # Slave 1 stats
    s1_tot = st.s1_ok + st.s1_fail
    s1_pct = (st.s1_ok / s1_tot * 100) if s1_tot else 0.0
    t.append(" Slave 1\n", style="bold cyan")
    t.append(" OK      : ", style="dim"); t.append(f"{st.s1_ok:>6}\n", style="bold green")
    t.append(" FAIL    : ", style="dim"); t.append(f"{st.s1_fail:>6}\n",
                                         style="bold red" if st.s1_fail else "dim")
    t.append(" Rate    : ", style="dim"); t.append(f"{s1_pct:>6.2f}%\n",
                                         style=pct_style(st.s1_ok, s1_tot))
    t.append(" Data    : ", style="dim"); t.append(f"{st.s1_data:>6}\n\n", style="white")

    # Slave 2 stats
    s2_tot = st.s2_ok + st.s2_fail
    s2_pct = (st.s2_ok / s2_tot * 100) if s2_tot else 0.0
    t.append(" Slave 2\n", style="bold cyan")
    t.append(" OK      : ", style="dim"); t.append(f"{st.s2_ok:>6}\n", style="bold green")
    t.append(" FAIL    : ", style="dim"); t.append(f"{st.s2_fail:>6}\n",
                                         style="bold red" if st.s2_fail else "dim")
    t.append(" Rate    : ", style="dim"); t.append(f"{s2_pct:>6.2f}%\n",
                                         style=pct_style(st.s2_ok, s2_tot))
    t.append(" Data    : ", style="dim"); t.append(f"{st.s2_data:>6}\n\n", style="white")

```

```

t.append(" Raw Log\n", style="bold dim")
t.append(" " + "-" * 30 + "\n", style="dim")
for ts, line in list(st.raw_log)[-LOG_LINES:]:
    style = log_line_style(line, True)
    t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

return Panel(t, title="[bold cyan]MASTER[/]",
             border_style="cyan", box=box.ROUNDED)

def build_slave_panel(st: SlaveState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")
    t.append(" RX Count   : ", style="dim"); t.append(f"{st.rx_count:>6}\n", style="bold white")
    t.append(" TX Counter : ", style="dim"); t.append(f"{st.data_counter:>6}\n\n", style="bold green")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line, False)
        t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

    return Panel(t, title=f"[bold {color}]{label}[/]",
                border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                       csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'COUNTER_JUMP': "bold yellow",
        'FAIL_STREAK': "bold red",
        'SLOW_CYCLE': "bold white on red",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=style)
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        # snapshot biar tidak hold lock saat render
        ms = MasterState(
            port=shared.master.port, connected=shared.master.connected,
            cycle=shared.master.cycle, last_cycle_ms=shared.master.last_cycle_ms,
            s1_ok=shared.master.s1_ok, s1_fail=shared.master.s1_fail,
            s1_data=shared.master.s1_data,
            s2_ok=shared.master.s2_ok, s2_fail=shared.master.s2_fail,
            s2_data=shared.master.s2_data,
            raw_log=deque(shared.master.raw_log),
        )
        s1 = SlaveState(
            name=shared.s1.name, port=shared.s1.port,
            connected=shared.s1.connected,
            rx_count=shared.s1.rx_count, data_counter=shared.s1.data_counter,
            raw_log=deque(shared.s1.raw_log),
        )
        s2 = SlaveState(
            name=shared.s2.name, port=shared.s2.port,

```

```

        connected=shared.s2.connected,
        rx_count=shared.s2.rx_count, data_counter=shared.s2.data_counter,
        raw_log=deque(shared.s2.raw_log),
    )
    anom = deque(shared.anomalies)
    t_start = shared.session_start

    node_row = Columns([
        build_master_panel(ms),
        build_slave_panel(s1, "SLAVE 1", "green"),
        build_slave_panel(s2, "SLAVE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\
    LoRa Master-Slave Monitor – Evaluasi Praktikum
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz
    \|
"""

Tool ini membaca data serial dari 3 node LoRa secara bersamaan dan
menampilkan dashboard real-time: statistik OK/FAIL per slave, durasi
cycle, RSSI/SNR, deteksi anomali, serta logging CSV otomatis untuk
analisis data penelitian.

PENGUNAAN:
python lora_monitor.py [opsi...]

OPSI:
--master PORT    Port COM untuk node Master      (default: {master})
--s1 PORT        Port COM untuk Slave 1           (default: {s1})
--s2 PORT        Port COM untuk Slave 2           (default: {s2})
--baud N         Baud rate serial semua port      (default: {baud})
--out FILE       Nama file CSV output             (default: lora_session_YYYYMMDD_HHMMSS.csv)
-h / --help      Tampilkan pesan bantuan ini

CONTOH:
python lora_monitor.py
    → Jalankan dengan port & baud rate default

python lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyACM2
    → Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

python lora_monitor.py --out data_jarak_10m.csv
    → Simpan log ke nama file tertentu

python lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyUSB0 --out hasil_uji.csv
    → Konfigurasi penuh untuk satu sesi eksperimen

KONTROL DASHBOARD:
Ctrl+C          Hentikan monitoring & simpan CSV

CATATAN:
• Pastikan semua board sudah di-upload firmware dengan baud {baud}
• Upload Slave dulu, baru Master, sebelum menjalankan tool ini
• File CSV tersimpan otomatis saat Ctrl+C ditekan
"""

def parse_args():
    p = argparse.ArgumentParser(
        description="LoRa Master-Slave Monitor Dashboard – tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f"python lora_monitor.py\n"
            f"python lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyACM2\n"
            f"python lora_monitor.py --out data_eksperimen.csv\n"
            f"python lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyUSB0 --out"
            f"hasil.csv\n"
        ),
    )
    p.add_argument('--master', default=DEFAULT_MASTER_PORT,

```



```

        metavar='PORT', help=f'Port COM node Master (default: {DEFAULT_MASTER_PORT})')
p.add_argument('--s1', default=DEFAULT_S1_PORT,
               metavar='PORT', help=f'Port COM Slave 1 (default: {DEFAULT_S1_PORT})')
p.add_argument('--s2', default=DEFAULT_S2_PORT,
               metavar='PORT', help=f'Port COM Slave 2 (default: {DEFAULT_S2_PORT})')
p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
               metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
p.add_argument('--out', default=None,
               metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
return p, p.parse_args()

def main():
    p, args = parse_args()
    no_args = len(sys.argv) == 1

    console = Console()

    if no_args:
        console.print(BANNER)
        console.print(
            f"[dim]Tidak ada argumen → menggunakan default: "
            f"Master=[yellow]{args.master}[/] "
            f"S1=[green]{args.s1}[/] "
            f"S2=[magenta]{args.s2}[/] "
            f"Baud=[white]{args.baud}[/][dim]\n"
        )

    shared = SharedState()
    shared.master.port = args.master
    shared.s1.port = args.s1
    shared.s2.port = args.s2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_session_{ts_str}.csv"

    if not no_args:
        console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
        console.print(f" Master : [yellow]{args.master}[/]" )
        console.print(f" Slave1 : [green]{args.s1}[/]" )
        console.print(f" Slave2 : [magenta]{args.s2}[/]" )
        console.print(f" Baud : [white]{args.baud}[/]" )

    console.print(f" CSV : [dim]{csv_path}[/]" )
    console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:
        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread,
                        args=(args.master, args.baud, "MASTER", shared, True),
                        daemon=True, name="reader-master"),
        threading.Thread(target=reader_thread,
                        args=(args.s1, args.baud, "S1", shared, False),
                        daemon=True, name="reader-s1"),
        threading.Thread(target=reader_thread,
                        args=(args.s2, args.baud, "S2", shared, False),
                        daemon=True, name="reader-s2"),
        threading.Thread(target=csv_writer_thread,
                        args=(shared, csv_path),
                        daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path),
                 console=console,
                 refresh_per_second=REFRESH_HZ,
                 screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass
    finally:
        shared.running = False
        time.sleep(0.6)
        console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]" )

```

```
if __name__ == "__main__":
    main()
```

Kode 5.5: lora_monitor.py — dasbor tiga node dengan perekaman CSV

```
#!/usr/bin/env python3
"""Upload otomatis ke Modul 05 (LoRa Master-Slave) – port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul05_lora_master_slave -e master -t upload -t monitor
    pio run -d Modul05_lora_master_slave -e slave1 -t upload -t monitor
    pio run -d Modul05_lora_master_slave -e slave2 -t upload -t monitor

Skrup ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `master` (port pertama), `slave1` (port kedua), dan `slave2`
(port ketiga), lalu menjalankan `pio run` dengan --upload-port sesuai hasil
deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga
board selesai di-upload, skrip bertanya mau menjalankan lora_monitor.py
(dashboard 3-panel), monitor_serial.py (log gabungan satu jendela), atau
keluar saja -- yang mana pun dipilih, portnya PERSIS port master/slave1/
slave2 yang barusan dipakai upload (bukan deteksi ulang), supaya tidak
mungkin salah pasang.

python3 Modul05_lora_master_slave/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul05_lora_master_slave/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti monitor_serial.py

python3 Modul05_lora_master_slave/upload_auto.py --only master
-> upload satu environment saja

python3 Modul05_lora_master_slave/upload_auto.py --master /dev/ttyACM0 --slave1 /dev/ttyACM1 --slave2 /
dev/ttyACM2
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["master", "slave1", "slave2"]
# Nama environment PlatformIO -> nama peran yang dipakai monitor_serial.py
ENV_KE_PERAN = {"master": "MASTER", "slave1": "S1", "slave2": "S2"}
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"master": "--master", "slave1": "--s1", "slave2": "--s2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan monitor_serial.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def jalankan_monitor(cmd):
    """Jalankan skrip monitor terpilih, menyerap Ctrl-C dengan rapi.
```

```

Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun ikut
menerima SIGINT selagi menunggu (os.waitpid) monitor yang sudah
membersihkan dirinya sendiri -- cukup diam, jangan biarkan traceback
ikut tercetak di atas ringkasan yang sudah rapi.
"""
print(" $", " ".join(cmd))
try:
    subprocess.call(cmd)
except KeyboardInterrupt:
    print()

def tanya_lalu_monitor(port_env):
    """Tanya mau pakai monitor yang mana, lalu jalankan dengan port
    master/slave1/slave2 PERSIS seperti yang baru dipakai upload -- bukan
    hasil deteksi ulang, supaya tidak mungkin meleset dari yang barusan
    di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    print("\nJalankan monitor sekarang?")
    print(" [1] lora_monitor.py (dashboard 3-panel real-time + CSV)")
    print(" [2] monitor_serial.py (log gabungan satu jendela)")
    print(" [3] Tidak, keluar saja")
    try:
        jawab = input("Pilihan [1/2/3] (bawaan 3): ").strip()
    except (EOFError, KeyboardInterrupt):
        print()
        return

    if jawab == "1":
        cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
        for env, opsi in ENV_KE_OPSI.items():
            if env in port_env:
                cmd += [opsi, port_env[env]]
        jalankan_monitor(cmd)
    elif jawab == "2":
        cmd = [sys.executable, str(PROJECT_DIR / "monitor_serial.py")]
        for env, peran in ENV_KE_PERAN.items():
            if env in port_env:
                cmd += ["--port", f"{peran}={port_env[env]}"]
        jalankan_monitor(cmd)
    # "3", kosong, atau apa pun selain 1/2 -> keluar saja tanpa menjalankan apa-apa

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--master", metavar="/dev/ttyXXX",
                    help="timpa port master (lewati deteksi otomatis)")
    ap.add_argument("--slave1", metavar="/dev/ttyXXX",
                    help="timpa port slave1 (lewati deteksi otomatis)")
    ap.add_argument("--slave2", metavar="/dev/ttyXXX",
                    help="timpa port slave2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (master, slave1, atau slave2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                         "(bawaan: upload saja -- pakai monitor_serial.py untuk memantau ketiganya"
                         " sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"master": args.master, "slave1": args.slave1, "slave2": args.slave2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"-- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

```

```

daftar = [args.only] if args.only else ENV_URUTAN
hilang = [n for n in daftar if n not in port_env]
if hilang:
    sys.exit(f"\nTidak bisa lanjut: port untuk {' '.join(hilang)} tidak ditemukan.\n"
            f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

for nama in daftar:
    if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
        sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

print("\nSelesai.")
tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 5.6: upload_auto.py — pemilih port otomatis saat unggah

5.7 Build dan Flash

Kedua slave lebih dahulu, master belakangan.

```

pio run -d Modul05_lora_master_slave -e slave1 -t upload
pio run -d Modul05_lora_master_slave -e slave2 -t upload
pio run -d Modul05_lora_master_slave -e master -t upload -t monitor

```

Memantau ketiga node sekaligus. Tiga Serial Monitor terpisah menyulitkan penilaian urutan kejadian, karena tiap jendela memiliki sumbu waktunya sendiri. Tersedia dua alat.

monitor_serial.py — sama seperti Modul 01–04, tanpa pustaka tambahan, dan meringkas hasil ukur saat berhenti.

```

python3 Modul05_lora_master_slave/monitor_serial.py --baud 115200 --durasi 40
python3 Modul05_lora_master_slave/monitor_serial.py --baud 115200 --port S2=/dev/ttyUSB0

```

```

Siklus polling : 53 (nomor 7..59)
Lama siklus min/maks/rata-rata : 147 / 154 / 152 ms
Slave 1 : dipanggil 53 menjawab 52 gagal 0 -> keberhasilan 98.1 %
Slave 2 : dipanggil 52 menjawab 52 gagal 0 -> keberhasilan 100.0 %
S1 membuang 116 panggilan milik node lain ([IGNORE])
S2 membuang 115 panggilan milik node lain ([IGNORE])

```

lora_monitor.py — dasbor berwarna dengan perekaman CSV, memerlukan pustaka rich.

```

pip install pyserial rich
python3 lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyACM2 --baud 115200
python3 lora_monitor.py --master /dev/ttyACM0 --s1 /dev/ttyACM1 --s2 /dev/ttyACM2 --out sesi1.csv

```

Berkas lora_session_20260516_071007.csv adalah contoh keluarannya, berguna untuk melihat format kolom sebelum merekam sesi sendiri. Rekaman itu diambil dengan revisi firmware terdahulu, sehingga baris pembukanya masih mencetak nama port Windows (Peran: MASTER (COM3)) — firmware sekarang mencetak nama environment PlatformIO. Isi dan format kolomnya tetap sama.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ pio device list dijalankan, ketiga port dicatat dan diisikan ke platformio.ini.
- ☐ Serial Monitor **115200** baud disiapkan, bukan 9600.
- ☐ Label fisik ditempel pada board: MASTER, SLAVE 1, SLAVE 2.

5.8 Percobaan

5.8.1 EXP-01 — Slave Menyaring Panggilan

Nyalakan kedua slave lebih dahulu tanpa master, lalu nyalakan master dan amati Serial kedua slave.

Expected output — Slave 1

```

=== LoRa SLAVE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
Menunggu POLL:1 dari Master...

[RX] POLL:1 | RSSI: -62 dBm | SNR: 9.00 dB | RX#: 1
[TX] S1:DATA:1

[IGNORE] POLL:2
[IGNORE] S2:DATA:1

```

Data capture

Parameter	Hasil
Apakah Slave 1 menerima POLL:2?	
Apa yang dilakukannya terhadap paket itu?	
Jumlah [IGNORE] per siklus di tiap slave	

Tabel 5.5: Lembar pengamatan EXP-01

CHECKPOINT

Setiap slave **menerima** panggilan untuk slave lain, lalu membuangnya. Inilah bukti langsung bahwa LoRa tidak memiliki pengalamatan: penyaringan sepenuhnya dikerjakan aplikasi. Slave yang tidak pernah mencetak [IGNORE] berarti tidak mendengar panggilan sama sekali — periksa jarak dan antena.

5.8.2 EXP-02 — Siklus Round-Robin

Amati master selama dua menit.

Expected output — master

```

=====
=== CYCLE 8 ===
[TX] POLL:1
[RX] S1:DATA:35 | RSSI: -58 dBm | SNR: 9.50 dB
[TX] POLL:2
[RX] S2:DATA:8 | RSSI: -56 dBm | SNR: 9.75 dB
--- STATISTIK ---
S1: OK=8 | FAIL=0 | Data: 35
S2: OK=8 | FAIL=0 | Data: 8
Durasi siklus: 148 ms
=====

```

Data capture

Parameter	Hasil
Lama siklus saat kedua slave sehat (ms)	
Jumlah siklus per menit	
OK / FAIL Slave 1 setelah 2 menit	
OK / FAIL Slave 2 setelah 2 menit	
RSSI Slave 1 / Slave 2 (dBm)	

Tabel 5.6: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA

Di `src/slave/main.cpp`, `dataCounter++` sengaja diletakkan **sesudah** paket diterima tetapi **sebelum** `transmit()`, sementara komentarnya menjelaskan alasannya. Bandingkan dengan `rxCount++` yang naik lebih awal. Jawab: apa arti berbeda dari kedua penghitung itu, dan angka mana yang akan berbeda dengan `Data`: yang tercatat master ketika sebagian jawaban hilang di udara?

CHECKPOINT

Nilai `Data`: di master harus mengikuti penghitung slave secara berurutan. Lompatan pada nilai itu berarti ada jawaban yang tidak sampai — catat, karena itulah bahan tabel pengukuran.

5.8.3 EXP-03 — Satu Node Hilang

Uji ketahanan jadwal ketika salah satu slave menghilang.

#	Skenario	Langkah	Yang diamati
1	Slave 2 mati	Cabut USB Slave 2	pesan master, lama siklus, FAIL
2	Slave 2 kembali	Pasang lagi	berapa siklus sampai OK bertambah lagi
3	Kedua slave mati	Cabut keduanya	lama siklus saat sunyi total
4	Slave 1 dijauhkan	Bawa ke jarak 50 m	FAIL Slave 1 vs Slave 2

Tabel 5.7: Skenario EXP-03 — satu node hilang

Data capture

Parameter	Hasil
Pesan master saat slave tidak menjawab	
Lama siklus dengan satu slave mati (ms)	
Lama siklus dengan kedua slave mati (ms)	
Apakah Slave 1 terpengaruh oleh matinya Slave 2?	
Apakah pemulihan terjadi otomatis?	

Tabel 5.8: Lembar pengamatan EXP-03 — data

CHECKPOINT

Skenario 3 memperlihatkan sifat penting penjadwalan terpusat: lama siklus **membengkak** menjadi sekitar jumlah seluruh batas waktu, karena master tetap menunggu setiap node yang sudah tidak ada. Catat angkanya — inilah dasar perhitungan batas skala pada bagian Analisis.

5.8.4 EXP-04 — Tabrakan yang Disengaja

Percobaan ini memperlihatkan mengapa penjadwalan diperlukan. Ubah **kedua** slave agar menjawab panggilan mana pun, dengan mengganti pemeriksaan nomor seperti pada Kode 5.7.

```
// Sengaja dilumpuhkan untuk EXP-04: kedua slave menjawab semua panggilan
// if (!received.equals("POLL:" + String(SLAVE_ID))) { ... return; }
```

Kode 5.7: Penyaringan nomor slave yang sengaja dilumpuhkan untuk EXP-04

Unggah ke kedua slave, amati master selama satu menit, lalu **kembalikan kodenya**.

Data capture

Parameter	Hasil
Berapa jawaban yang berhasil dipecahkan master?	
Pesan apa yang muncul di master?	
Apakah master menerima campuran keduanya?	

Tabel 5.9: Lembar pengamatan EXP-04 — tabrakan yang disengaja

CHECKPOINT

Sebagian besar siklus akan berakhir dengan [FAIL] atau [WARN] Balasan tidak valid, padahal kedua slave jelas-jelas mengirim. Tabrakan tidak menghasilkan pesan kesalahan dari radio — hanya kesunyian atau data rusak. Pengamatan ini adalah inti seluruh modul.

Perhatikan kolom SNR. Pada pengujian rujukan, SNR di master anjlok dari 9,0–9,8 dB menjadi **1,25–1,75 dB** begitu kedua slave menjawab bersamaan: jawaban satu slave menjadi derau bagi jawaban slave lainnya. Inilah cara paling langsung mendeteksi tabrakan dari sisi aplikasi, dan bahan jawaban pertanyaan nomor 5 pada bagian Analisis. Perhatikan pula bahwa kegagalannya **tidak merata** — Slave 2 tetap terbaca master sedangkan Slave 1 tidak pernah berhasil sama sekali, karena penerima memenangkan sinyal yang lebih kuat (*capture effect*). Dari sisi master, node yang kalah tampak seperti mati.

5.8.5 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada tiga Arduino Uno bershield Dragino LoRa v1.2, 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

Parameter	Hasil terukur
Siklus dalam 40 detik	53 — lama siklus 147 / 154 / 152 ms
Keberhasilan kedua slave	100 % , 0 FAIL
[IGNORE] per slave per siklus	2 — panggilan <i>dan</i> jawaban milik node lain
Lama siklus saat satu node mati	611 ms — melipat 4× dari 152 ms
Pertambahan akibat satu node mati	+459 ms ≈ POLL_TIMEOUT
Apakah node sehat ikut terganggu?	tidak
EXP-04 tabrakan: SNR di master	9,0–9,8 dB → 1,25–1,75 dB
EXP-04: keberhasilan Slave 1	100 % → 0 % (kalah <i>capture</i> dari Slave 2)

Tabel 5.10: Hasil verifikasi perangkat keras Modul 05

Environment	Status	Flash
master	SUCCESS	29.7% (9574 B)
slave1	SUCCESS	26.3% (8492 B)
slave2	SUCCESS	26.3% (8492 B)

Master paling besar karena memuat penjadwal dan statistik dua node. Kedua slave berukuran sama persis — bukti bahwa keduanya berasal dari source yang sama dan hanya berbeda nilai SLAVE_ID.

Verifikasi ulang — 21 Agustus 2026. Ketiga board diunggah ulang dan direkam 40 detik pada konfigurasi port yang berbeda dari log di atas (ketiganya kini Uno asli, ttyACM0/1/2, bukan lagi klon CH340). Lama siklus steady-state **147–149 ms, rata-rata 148,0 ms** ($n = 59$) — sejalan dengan sesi sebelumnya. Balasan Slave 2 pada sesi ini sempat menunjukkan **SNR rendah tak wajar** (rata-rata 1,23 dB, mirip tanda tabrakan pada Tabel 5.10) meski Slave 2 sendiri menerima POLL:2 dengan bersih (9,00 dB) — gangguannya ada di penerimaan master, bukan di Slave 2.

Anomali itu terselesaikan pada sesi lanjutan hari yang sama, setelah bug banner startup diperbaiki (lihat catatan di bawah) dan kedua slave diunggah ulang: RSSI balasan Slave 2 turun dari -39 dBm menjadi -61 dBm — sepadan dengan Slave 1 (-65 dBm) — dan SNR-nya kembali normal, **rata-rata 9,34 dB** dari 60 balasan, tidak satu pun di bawah 5 dB. Dugaannya terkonfirmasi: pada sesi anomali, Slave 2 kemungkinan besar duduk terlalu dekat dengan master, menyebabkan penerima master jenuh (near-field), bukan tabrakan sungguhan. Rincian dan tabel perbandingan kedua sesi ada di logserial.md, bagian “Verifikasi anomali SNR — sesi lanjutan 21 Agustus 2026”.

CATATAN

Catatan perbaikan — banner yang berbohong. Sebelum diperbaiki, src/slave/main.cpp mencetak Serial.println(F("Menunggu POLL:1 dari Master...\n")) sebagai literal tetap, tidak memakai SLAVE_ID — sehingga Slave 2 pun mencetak “menunggu POLL:1” di layarnya sendiri, padahal logika penyaringannya (received.equals("POLL:" + String(SLAVE_ID))) sudah benar sejak awal. Bug ini murni kosmetik — tidak memengaruhi jawaban maupun statistik — tetapi cukup untuk menyesatkan siapa pun yang mendiagnosis dari banner saja. Baris itu sekarang Serial.print(F("Menunggu POLL:")); Serial.print(SLAVE_ID); — bukti bahwa dua baris kode yang tampak sepele pun bisa diam-diam salah selama tidak ada yang membandingkannya dengan SLAVE_ID sungguhan.

5.8.6 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, POLL_TIMEOUT 500 ms. Ketiga board di satu meja, jarak ± 30 cm.

Board & Port

Peran	Environment	Port	Board
Master	master	/dev/ttyACM0	Uno asli (2341:0043)
Slave 1	slave1	/dev/ttyACM1	Uno asli (2341:0043)
Slave 2	slave2	/dev/ttyUSB0	Uno klon, bridge CH340 (1a86:7523)

Tabel 5.11: Log serial Modul 05: Board & Port

Board ketiga pada pengujian ini adalah klon ber-bridge CH340 yang muncul sebagai /dev/ttyUSB0, bukan /dev/ttyACM2 seperti tertulis di platformio.ini. Karena itu unggahan dan pemantauannya memakai --upload-port /dev/ttyUSB0 dan --port S2=/dev/ttyUSB0.

Ketiga aliran serial direkam bersamaan memakai monitor_serial.py.

EXP-01 — Slave Menyaring Panggilan

```
[ 1.822] S1 | === LoRa SLAVE 1 ===
[ 1.822] S1 | Init LoRa ... OK
[ 1.822] S1 | Menunggu POLL:1 dari Master...
[ 1.822] S1 | [RX] POLL:1 | RSSI: -62 dBm | SNR: 9.00 dB | RX#: 1
[ 1.822] S1 | [TX] S1:DATA:1
[ 1.822] S1 | [IGNORE] POLL:2
[ 1.822] S1 | [IGNORE] S2:DATA:1
[ 1.831] S2 | [RX] POLL:2 | RSSI: -62 dBm | SNR: 9.00 dB | RX#: 1
[ 1.831] S2 | [TX] S2:DATA:1
```



```
[ 2.432] S2 | [IGNORE] POLL:1
[ 2.432] S2 | [IGNORE] S1:DATA:2
```

Parameter	Hasil
Apakah Slave 1 menerima POLL:2?	ya — lalu dibuang
Baris [IGNORE] per siklus di tiap slave	2 (POLL milik node lain + jawaban node lain)
Total [IGNORE] selama 40 detik	S1: 116, S2: 115
Flash master / slave1 / slave2	29,6 % / 26,3 % / 26,3 %

Tabel 5.12: Log serial Modul 05: EXP-01 — Slave Menyaring Panggilan

CHECKPOINT terpenuhi. Setiap slave menerima panggilan untuk slave lain, lalu membuangnya. Yang menarik, slave juga membuang **jawaban** slave lain ([IGNORE] S2:DATA:1) — bukti bahwa di LoRa mentah setiap node mendengar seluruh lalu lintas, dan penyaringan sepenuhnya dikerjakan aplikasi.

EXP-02 — Siklus Round-Robin (40 detik)

```
[ 6.307] MASTER | === CYCLE 8 ===
[ 6.307] MASTER | [TX] POLL:1
[ 6.507] MASTER | [RX] S1:DATA:35 | RSSI: -58 dBm | SNR: 9.50 dB
[ 6.507] MASTER | [TX] POLL:2
[ 6.507] MASTER | [RX] S2:DATA:8 | RSSI: -56 dBm | SNR: 9.75 dB
[ 6.507] MASTER | --- STATISTIK ---
[ 6.507] MASTER | S1: OK=8 | FAIL=0 | Data: 35
[ 6.507] MASTER | S2: OK=8 | FAIL=0 | Data: 8
[ 6.507] MASTER | Durasi siklus: 148 ms
```

Parameter	Hasil
Siklus selesai dalam 40 detik	53
Lama siklus min / maks / rata-rata	147 / 154 / 152 ms
Slave 1: dipanggil / menjawab / gagal	53 / 52 / 0
Slave 2: dipanggil / menjawab / gagal	52 / 52 / 0
RSSI di master (dari kedua slave)	-58 ... -55, rata-rata -56,3 dBm
RSSI di S1 / S2 (dari master)	-62,0 / -61,8 dBm
SNR di seluruh node	±9,0–9,8 dB

Tabel 5.13: Log serial Modul 05: EXP-02 — Siklus Round-Robin (40 detik)

Selisih satu jawaban pada Slave 1 berasal dari siklus terakhir yang terpotong saat perekaman berhenti, bukan paket hilang.

Lama siklus 152 ms terbagi menjadi dua giliran ±76 ms. Karena master menambahkan CYCLE_INTERVAL 500 ms di antara siklus, satu node terbaca ulang setiap ±650 ms.

EXP-03 — Satu Node Hilang

Ketiadaan Slave 2 ditirukan dengan mengunggah **sketsa diam** (radio tidak pernah diinisialisasi), bukan mencabut USB.

Parameter	Hasil
Lama siklus saat kedua slave sehat	152 ms
Lama siklus saat Slave 2 mati	611 ms (min 610, maks 612)
Pertambahan akibat satu node mati	+459 ms
[FAIL] Slave 2 tidak merespon! selama 14 detik	11
Apakah Slave 1 ikut terganggu?	tidak — OK Slave 1 terus bertambah

Parameter	Hasil
Lama siklus setelah Slave 2 kembali	178 ms (kembali normal)
Pemulihan	otomatis, tanpa mereset master

Tabel 5.14: Log serial Modul 05: EXP-03 — Satu Node Hilang

Pertambahan 459 ms mendekati POLL_TIMEOUT 500 ms dikurangi waktu yang biasanya dipakai jawaban yang sehat (± 40 ms). Dengan kata lain, **master tetap membayar penuh waktu tunggu untuk node yang sudah tidak ada**. Lama siklus melipat **empat kali** hanya karena satu dari dua node mati — dan pelipatan itu memperlambat pembacaan node yang masih hidup.

Perkiraan lama siklus yang dapat diuji ulang:

$$\text{lama siklus} \approx (\text{jumlah node hidup} \times \pm 76 \text{ ms}) + (\text{jumlah node mati} \times 500 \text{ ms})$$

EXP-04 — Tabrakan yang Disengaja

Penyaringan POLL: <id> dilumpuhkan pada **kedua** slave, sehingga keduanya menjawab setiap panggilan.

```
[ 2.78] [TX] POLL:1
[ 2.86] [WARN] Balasan tidak valid: S2:DATA:3
[ 3.31] [FAIL] Slave 1 tidak merespon!
[ 3.31] [TX] POLL:2
[ 3.39] [RX] S2:DATA:4 | RSSI: -54 dBm | SNR: 1.25 dB
[ 3.39] S1: OK=0 | FAIL=2 | Data: 0
[ 3.39] S2: OK=2 | FAIL=0 | Data: 4
```

Parameter	Penyaringan aktif	Penyaringan dilumpuhkan
Jawaban terbaca master (30 detik)	± 53 per node	23 total
[FAIL] tidak merespon	0	27
[WARN] Balasan tidak valid	0	24
Lama siklus rata-rata	152 ms	643 ms
SNR di master	9,0–9,8 dB	1,25–1,75 dB
Keberhasilan Slave 1	100 %	0 % (OK=0, FAIL=27)

Tabel 5.15: Log serial Modul 05: EXP-04 — Tabrakan yang Disengaja

Bacaan hasil

Tabrakan tidak memunculkan pesan kesalahan dari radio. Master tidak pernah mencetak “collision” atau semacamnya — yang terlihat hanya [FAIL] tidak merespon dan [WARN] Balasan tidak valid, dua pesan yang tampak seperti masalah lain sama sekali. Padahal kedua slave jelas-jelas mengirim setiap kali.

SNR adalah petunjuk yang paling jujur. Saat penyaringan aktif, SNR bertahan di 9–9,8 dB. Saat kedua slave memancar bersamaan, SNR anjlok ke 1,25–1,75 dB — sekitar **8 dB lebih buruk** — karena jawaban satu slave menjadi derau bagi jawaban slave lainnya. Inilah cara mendeteksi tabrakan dari sisi aplikasi tanpa dukungan radio, dan menjadi jawaban pertanyaan nomor 5 pada bagian Analisis.

Satu node menang, satu node hilang sama sekali. Master masih dapat memecahkan jawaban Slave 2 (efek *capture*: sinyal yang lebih kuat memenangkan penerima), sedangkan Slave 1 tidak pernah berhasil satu kali pun. Kegagalan tabrakan karena itu tidak merata — ia menghukum node yang lebih lemah, dan dari sisi master node itu tampak seperti mati.

Penghitung master menjadi tidak bermakna. Nilai Data: Slave 2 bertambah dua tiap siklus karena ia menjawab dua panggilan, sedangkan Slave 1 tertahan di 0. Statistik yang biasanya dipercaya justru menyesatkan ketika asumsi dasarnya — satu panggilan, satu penjawab — dilanggar.

Sesi verifikasi ulang — 21 Agustus 2026

Ketiga board diunggah ulang dari src/ saat ini (pio run -e slave1|slave2|master -t upload, ketiganya SUCCESS, flash terverifikasi avrdude) lalu direkam 40 detik dengan reset serentak lewat DTR. Tujuannya memverifikasi bahwa ketiga node yang tersambung memang menjalankan firmware modul ini — bukan mengulang EXP-01–04 secara penuh.

Board & Port saat ini — berbeda dari tabel di atas: ketiga board sekarang Uno asli (2341:0043) pada /dev/ttyACM0 (master), /dev/ttyACM1 (slave1), /dev/ttyACM2 (slave2). Tidak ada lagi board klon CH340 di rig ini.

```
[ 20.373] MASTER | === CYCLE 44 ===
[ 20.373] MASTER | [TX] POLL:1
[ 20.373] MASTER | [RX] S1:DATA:44 | RSSI: -56 dBm | SNR: 9.50 dB
[ 20.373] MASTER | [TX] POLL:2
[ 20.433] S2      | [RX] POLL:2 | RSSI: -34 dBm | SNR: 9.00 dB | RX#: 36
[ 20.433] S2      | [TX] S2:DATA:36
[ 20.573] MASTER | [RX] S2:DATA:36 | RSSI: -38 dBm | SNR: 1.50 dB
[ 20.573] MASTER | --- STATISTIK ---
[ 20.573] MASTER | S1: OK=43 | FAIL=1 | Data: 44
[ 20.573] MASTER | S2: OK=42 | FAIL=2 | Data: 36
[ 20.573] MASTER | Durasi siklus: 147 ms
```

Parameter	Hasil
Siklus penuh dalam 40 detik	61 (60 bernomor berurutan)
Durasi siklus steady-state min/maks/rata-rata	147 / 149 / 148,0 ms (n=59, mengecualikan 2 siklus timeout)
Periode siklus sesungguhnya (jarak antar === CYCLE n ===)	600 / 769 / 655,0 ms (n=57 steady-state)
Bagian periode yang tak tampak pada Durasi siklus	507 ms $\approx 77\%$ — didominasi delay(CYCLE_INTERVAL) 500 ms
Biaya perangkat lunak per paket (dari 148,0 ms – 134,1 ms air time, dibagi 4 paket)	$\approx$ 3,5 ms/paket
Slave 1: dipanggil / OK / FAIL	61 / 71 [*] / 2 $\rightarrow$ 97,3 %
Slave 2: dipanggil / OK / FAIL	61 / 71 [*] / 2 $\rightarrow$ 97,3 %
[IGNORE] per slave (40 detik)	S1: 120, S2: 120
SNR balasan Slave 1 di master	9,25–9,75 dB, rata-rata 9,50 dB (n=58)
SNR balasan Slave 2 di master	0,75–1,75 dB, rata-rata 1,23 dB (n=60)
RSSI balasan Slave 1 / Slave 2 di master	–56 dBm (stabil) / –39 dBm (lebih kuat)

Tabel 5.16: Log serial Modul 05: Sesi verifikasi ulang — 21 Agustus 2026

^{*} jumlah OK melebihi jumlah siklus karena firmware langsung reset saat unggahan sebelum jendela 40 detik mulai dihitung; angka OK/FAIL diambil dari statistik kumulatif master di akhir sesi, [IGNORE] dan SNR/RSSI dari jendela 40 detik penuh.

Anomali belum terjelaskan — SNR Slave 2 rendah secara konsisten. Seluruh 60 balasan Slave 2 pada sesi ini bersnr di bawah 2 dB — persis kisaran yang menurut analisis EXP-04 pada modul ini menjadi tanda tabrakan (bandingkan 1,25–1,75 dB pada tabel EXP-04 di atas). Namun konteksnya berbeda: tidak ada percobaan tabrakan yang sengaja dijalankan di sesi ini, dan **Slave 2 sendiri menerima POLL:2 dengan SNR bersih 9,00 dB** — jadi yang terganggu bukan penerimaan Slave 2, melainkan penerimaan **master** saat Slave 2 membalas. RSSI balasan Slave 2 di master juga jauh lebih kuat daripada Slave 1 (–39 dBm vs –56 dBm), mengindikasikan Slave 2 duduk jauh lebih dekat ke master pada sesi ini — kemungkinan penyebabnya near-field/kejenuhan penerima pada jarak sangat dekat, bukan tabrakan sungguhan, tetapi ini **belum dipastikan** dan perlu diperiksa langsung (jarak fisik Slave 2 terhadap master, kondisi antenna) sebelum angka SNR ini dipakai sebagai rujukan kondisi normal. Meski begitu keberhasilan paket (OK/FAIL) tidak terpengaruh — seluruh balasan tetap terdecode benar.

Verifikasi anomali SNR — sesi lanjutan 21 Agustus 2026

Sesi sebelumnya (“Sesi verifikasi ulang — 21 Agustus 2026” di atas) mencatat SNR balasan Slave 2 anjlok ke rata-rata 1,23 dB pada seluruh 60 balasan, mirip tanda tabrakan pada EXP-04, padahal tidak ada percobaan tabrakan yang dijalankan. Antara sesi itu dan sesi ini, firmware slave juga diperbaiki: banner startup sebelumnya selalu mencetak Menunggu POLL:1 untuk kedua slave (SLAVE_ID tidak dipakai di baris itu); sekarang mencetak nomor yang benar. Kedua slave diunggah ulang, lalu direkam 40 detik lagi tanpa mengubah posisi board secara sengaja.

```
[ 20.023] S1      | [RX] POLL:1 | RSSI: -70 dBm | SNR: 8.75 dB | RX#: 61
[ 20.023] S1      | [TX] S1:DATA:61
[ 20.023] S2      | [RX] POLL:2 | RSSI: -67 dBm | SNR: 8.75 dB | RX#: 36
[ 20.023] S2      | [TX] S2:DATA:36
[ 20.121] MASTER  | [RX] S2:DATA:36 | RSSI: -61 dBm | SNR: 9.25 dB
[ 20.121] MASTER  | --- STATISTIK ---
```

Parameter	Sesi sebelumnya (anomali)	Sesi ini
RSSI balasan S2 di master	-39 dBm (janggal, jauh lebih kuat dari S1)	-61 dBm (n=60, sepadan dengan S1)
SNR balasan S2 di master	0,75–1,75 dB, rata-rata 1,23 dB	8,75–9,75 dB, rata-rata 9,34 dB (n=60)
SNR balasan S1 di master	9,25–9,75 dB, rata-rata 9,50 dB	8,50–9,50 dB, rata-rata 9,06 dB (n=59)
S2 SNR < 5 dB	60 / 60	0 / 60
Durasi siklus steady-state	147–149 ms, rata-rata 148,0 ms (n=59)	147–149 ms, rata-rata 148,1 ms (n=61)
Siklus dalam jendela 40 detik	61	61
[FAIL] dalam jendela	S1: 1, S2: 2 (dari statistik kumulatif, bukan murni jendela)	S1: 1, S2: 0 (dihitung langsung dari baris [FAIL] dalam jendela)

Tabel 5.17: Log serial Modul 05: Verifikasi anomali SNR — sesi lanjutan 21 Agustus 2026

Kesimpulan. Anomali SNR Slave 2 tidak berulang. RSSI-nya turun dari -39 dBm ke -61 dBm — mendekati RSSI Slave 1 (-65 dBm pada sesi ini) — sehingga dugaan sebelumnya terkonfirmasi: pada sesi anomali, Slave 2 kemungkinan besar duduk terlalu dekat dengan master, menyebabkan penerima master jenuh atau terpengaruh efek near-field saat mendekode balasannya. Begitu jaraknya kembali wajar, SNR Slave 2 identik pola dengan Slave 1. Durasi siklus (148 ms) dan air time tetap konsisten di kedua sesi,

mengonfirmasi angka itu sebagai baseline yang stabil, terlepas dari anomali SNR yang sifatnya spesifik-posisi.

Implikasi untuk laporan. Anomali ini sekaligus contoh nyata poin di bagian “Membaca anomali di log”: nilai SNR/RSSI yang janggal pada satu node harus diperiksa dulu terhadap kemungkinan sebab fisik (jarak, posisi, near-field) sebelum disimpulkan sebagai tabrakan atau kegagalan protokol.

Catatan pengambilan log

- Seluruh perubahan sumber pada EXP-04 dikembalikan, kedua slave diunggah ulang, dan baseline diukur ulang untuk memastikan penyaringan kembali bekerja.
- Saat menguji EXP-03 dan EXP-04, monitor hanya dijalankan pada port master. Port slave dibiarkan bebas agar dapat diunggah; monitor yang menahan port membuat `pio run -t upload` gagal membukanya.
- Tabel pengukuran jarak, skenario asimetris, dan pengaruh `POLL_TIMEOUT` pada README belum terisi: seluruh percobaan dijalankan di satu meja pada jarak tetap ± 30 cm.

5.9 Pengukuran

A. Keberhasilan per node terhadap jarak — kedua slave ditempatkan pada jarak sama, 30 siklus per baris.

Jarak	RSSI S1	RSSI S2	OK/FAIL S1	OK/FAIL S2	Berhasil S1 (%)	Berhasil S2 (%)
1 m						
25 m						
50 m						
100 m						

Tabel 5.18: Lembar pengukuran A — keberhasilan per node terhadap jarak

B. Skenario asimetris (wajib) — Slave 1 didekatkan, Slave 2 dijauhkan.

Posisi S1	Posisi S2	Berhasil S1 (%)	Berhasil S2 (%)	Lama siklus (ms)	Kesimpulan
1 m	100 m				

Tabel 5.19: Lembar pengukuran B — skenario asimetris

C. Lama siklus terhadap jumlah node yang hidup

Kondisi	Lama siklus (ms)	Siklus per menit
Kedua slave hidup		
Satu slave mati		
Kedua slave mati		

Tabel 5.20: Lembar pengukuran C — lama siklus terhadap jumlah node hidup

D. Pengaruh batas waktu polling — ubah `POLL_TIMEOUT` pada master, jarak tetap.

POLL_TIMEOUT	Lama siklus (ms)	Berhasil S1 (%)	Berhasil S2 (%)
200 ms			
500 ms			
1000 ms			

Tabel 5.21: Lembar pengukuran D — pengaruh batas waktu polling

5.10 Analisis

1. Dari Tabel 5.20, berapa milidetik yang ditambahkan tiap node mati terhadap lama siklus? Susun rumus perkiraan lama siklus untuk n slave dengan k di antaranya mati.
2. Berdasarkan rumus tersebut, berapa jumlah slave maksimum bila setiap node harus terbaca minimal sekali setiap 5 detik pada kondisi terburuk? Sebutkan asumsinya.
3. Dari Tabel 5.21, apa akibat memperpendek batas waktu menjadi 200 ms? Kaitkan dengan waktu udara pada SF7 dan jelaskan mengapa nilai yang terlalu kecil menghasilkan kegagalan palsu.
4. Pada Tabel 5.19, apakah slave yang jauh menurunkan kualitas slave yang dekat? Bandingkan jawabannya dengan perilaku topologi bintang pada modul BLE multi-node, dan jelaskan sumber perbedaannya.
5. EXP-04 memperlihatkan tabrakan tidak menghasilkan pesan kesalahan. Sebutkan dua cara mendeteksi tabrakan dari sisi aplikasi, beserta keterbatasan masing-masing.
6. Pendekatan master-slave menjadwalkan hak bicara secara terpusat. Sebutkan dua kelemahan mendasarnya, lalu bandingkan dengan pendekatan lain seperti ALOHA atau LoRaWAN kelas A.

5.11 Concept Check

1. Mengapa slave tetap menerima paket yang bukan miliknya, dan di lapisan mana penyaringan dilakukan?
2. Apa yang terjadi bila kedua slave menjawab bersamaan, dan mengapa hal itu tidak muncul sebagai pesan kesalahan?
3. Apa fungsi POLL_TIMEOUT, dan apa yang menentukan nilai terkecilnya yang masuk akal?
4. Mengapa dataCounter dinaikkan setelah paket diterima dan bukan pada saat pengiriman dinyatakan berhasil?
5. Mengapa kedua slave dapat memakai satu file source yang sama?
6. Mengapa modul ini memakai 115200 baud sementara modul sebelumnya 9600? Apa gejalanya bila Serial Monitor salah setel?

5.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Memperbesar dan memperkaya jaringan

CH-1 — Slave ketiga Tambahkan `SLAVE_ID=3`: satu environment baru dan penyesuaian `SLAVE_COUNT` di master. Ukur pertambahan lama siklus dan bandingkan dengan rumus dari bagian Analisis.

CH-2 — Sensor sungguhan Ganti penghitung slave dengan pembacaan sensor pada A0, lalu kirimkan nilainya (`S1:DATA:512`). Bahas mengapa laju pembacaan kini dibatasi lama siklus master.

CH-3 — Perintah turun Tambahkan perintah `CMD:1:LED_ON` dari master, sehingga komunikasi tidak hanya mengambil data tetapi juga mengendalikan slave.

Level 3 — Tantangan · Penjadwal yang belajar dari kegagalan

CH-4 — Jadwal adaptif Buat master melewati slave yang sudah gagal tiga kali berturut-turut, dan menengoknya kembali sekali setiap sepuluh siklus. Ukur perbaikan lama siklus saat satu node mati.

CH-5 — Rekam dan analisis Jalankan `lora_monitor.py --out sesi.csv` selama sepuluh menit, lalu olah CSV-nya untuk membuat grafik RSSI terhadap waktu bagi kedua slave.

5.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — tabrakan, penjadwalan terpusat, round-robin, batas waktu.
3. Konfigurasi — build flag nomor slave, `POLL_TIMEOUT`, format `POLL:n` dan `S<n>:DATA:m`.
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-04 beserta checkpoint), sebaiknya rekaman CSV dari `lora_monitor.py`.
5. Data pengukuran — Tabel 5.18 sampai Tabel 5.21.
6. Analisis dan concept check, termasuk rumus perkiraan lama siklus.
7. Challenge — minimal CH-1 dan CH-4.
8. Kesimpulan yang disusun sendiri mengenai batas skala pendekatan master-slave pada LoRa mentah.

6. Modul 06 — LoRa dari Linux (Python, tanpa Library)

LoRa COMMUNICATION LAB · MODUL 06

Drop the Library — LoRa dari Linux (Python, tanpa Library)

Raspberry Pi + Dragino LoRa GPS HAT v1.4 · LoRa mentah · satu arah · polling register · level Intermediate · 2 × 50 menit · folder kode Modul06_rpi_lora_python

PENGANTAR MODUL · Gambaran Umum

Modul 06 dirancang untuk dua pertemuan (2 × 50 menit) pada tingkat menengah. Misinya memindahkan tautan LoRa Modul 01 ke platform yang sama sekali berbeda — Raspberry Pi dengan Python di atas Linux — dan melakukannya **tanpa satu pun library LoRa**. Percobaan memakai dua Raspberry Pi bershield Dragino LoRa GPS HAT v1.4, diamati langsung pada terminal masing-masing.

6.1 Pendahuluan

Lima modul sebelumnya selalu diawali baris yang sama: `LoRa.begin(FREQUENCY)`. Satu baris itu menyembunyikan reset perangkat keras, verifikasi identitas chip, perhitungan pembagi frekuensi, penyiapan alamat FIFO, dan perpindahan mode operasi — semuanya dikerjakan library sandeepmistry di belakang layar. Selama radio bekerja, tidak ada alasan membongkarnya. Begitu radio **tidak** bekerja, praktikan yang tidak pernah melihat isinya tidak punya tempat untuk mulai mencari. Modul ini membongkarnya: setiap fungsi di `src/sender.py` dan `src/receiver.py` adalah tulisan atau bacaan langsung ke register SX1276 lewat SPI, dan nama fungsinya sengaja dibuat menyerupai API Arduino agar kedua sisi dapat dibandingkan baris per baris.

Prasyaratnya adalah M01 — parameter radio, RSSI, SNR, dan hitungan loss dari nomor urut semuanya dipakai lagi di sini tanpa perubahan. Yang dibangun di modul ini adalah pemahaman lapisan di bawah library: peta register, tata cara transaksi SPI, perhitungan `frf`, penanganan IRQ flag, serta alasan mengapa nilai mentah register RSSI harus dikoreksi sebelum berarti dBm. Kemampuan tersebut dipakai lagi di M07, tempat Raspberry Pi menjadi master yang menjadwalkan slave Arduino.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan lewat interrupt — <code>loop()</code> tidak lagi menunggu
03	Dua arah bergantian di atas radio half-duplex
04	Setiap pengiriman diketahui hasilnya: ACK, timeout, statistik
05	Banyak node — hak bicara dijadwalkan agar tidak bertabrakan
06 (ini)	Platform berganti ke Linux; library dilepas, register dipegang langsung
07	Gateway Linux menjadwalkan node mikrokontroler

Tabel 6.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Payload tetap Hello LoRa #n persis seperti M01, dan parameter radionya tetap SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Kesamaan itu bukan kemalasan: karena payload dan parameter identik, **sender Raspberry Pi modul ini dapat diuji langsung terhadap receiver Arduino Modul 01** dan sebaliknya. Uji silang itulah yang membuktikan bahwa yang berbicara adalah radionya, bukan librarynya — dan itu menjadi EXP-03.

6.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Memegang register SX1276 langsung dari Linux

1. Menjelaskan urutan langkah yang dikerjakan `LoRa.begin()` dengan menunjuk register yang ditulis pada tiap langkah di `loRaBegin()`.
2. Membaca dan menulis register SX1276 lewat SPI, serta menjelaskan peran bit ke-7 alamat dalam membedakan operasi baca dan tulis.
3. Menghitung nilai `frf` dari frekuensi yang diinginkan, dan memverifikasi hasilnya dengan membaca kembali ketiga register `REG_FRF_*`.
4. Membuktikan interoperabilitas SX1276 lintas platform dengan menjalankan tautan silang Raspberry Pi ↔ Arduino, lalu menjelaskan mengapa keduanya saling mengerti.
5. Menjelaskan mengapa nilai mentah `REG_PKT_RSSI_VALUE` harus dikoreksi dengan offset yang berbeda untuk band di bawah dan di atas 868 MHz.

Kriteria keberhasilan

- ☐ Init LoRa ... OK muncul di kedua Raspberry Pi — artinya `REG_VERSION` terbaca `0x12`.
- ☐ Receiver mencetak isi paket beserta RSSI dan SNR, dengan nomor urut naik satu per satu.
- ☐ Tautan silang Pi → Arduino **dan** Arduino → Pi keduanya berhasil tanpa mengubah satu baris pun di kedua sisi.
- ☐ Nilai `frf` hasil hitungan tangan cocok dengan isi register yang dibaca kembali dari chip.
- ☐ Perubahan satu parameter di satu sisi menghentikan tautan, dan penyebabnya dapat ditunjuk pada register mana yang berbeda.

6.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Register	Satu byte penyimpanan di dalam SX1276 yang menentukan atau melaporkan satu aspek keadaan radio. Alamatnya 7 bit.
Transaksi SPI	Dua byte: byte pertama alamat, byte kedua data. Bit ke-7 alamat bernilai 1 berarti tulis , 0 berarti baca .
NSS (chip select)	Jalur yang ditarik LOW selama transaksi berlangsung. Pada HAT ini NSS adalah GPIO 25 biasa, bukan CE0 bawaan SPI, sehingga harus digerakkan sendiri oleh program.
FIFO	Penyangga 256 byte di dalam chip, dipakai bergantian untuk TX dan RX. Pointer <code>REG_FIFO_ADDR_PTR</code> menentukan posisi baca/tulis berikutnya.
Mode operasi	Isi <code>REG_OP_MODE</code> : SLEEP, STDBY, TX, RXCONTINUOUS. Sebagian register hanya boleh diubah saat SLEEP atau STDBY.
<code>frf</code>	Bilangan 20-bit pembagi frekuensi: $frf = f/32 \text{ MHz} \times 2^{19}$, disimpan pada tiga register MSB/MID/LSB.
IRQ flag	Bit penanda kejadian di <code>REG_IRQ_FLAGS</code> — TX selesai, RX selesai, CRC gagal. Ditulis 1 untuk membersihkan , bukan 0.

Istilah	Definisi kerja di lab ini
Offset RSSI	Nilai register RSSI adalah angka mentah; dBm sebenarnya = nilai - 164 untuk band rendah (433 MHz), atau - 157 untuk band tinggi (868/915 MHz).

Tabel 6.2: Istilah kerja Modul 06

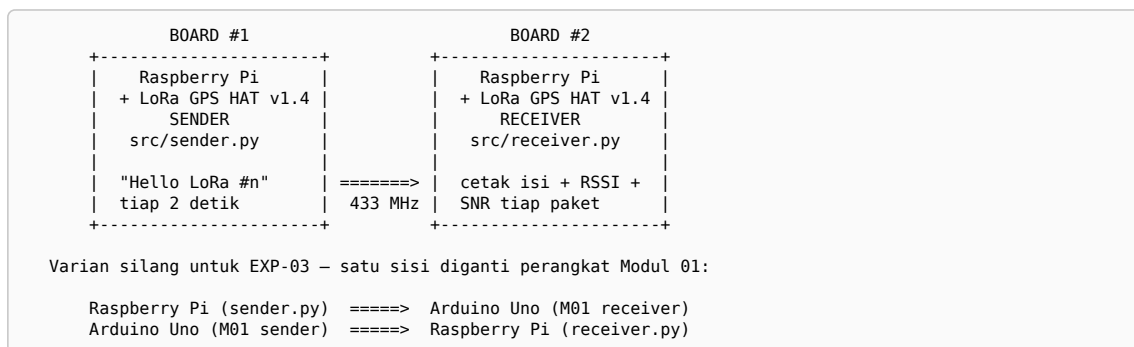
Mengapa NSS dikendalikan manual. SPI perangkat keras Raspberry Pi menyediakan dua jalur chip select bawaan, CE0 dan CE1. Dragino tidak menggunakannya: pada HAT ini NSS SX1276 tersambung ke GPIO 25. Akibatnya kode membuka SPI dengan `_spi.open(0, 0)` — bus 0, device 0 — tetapi CE0 yang ikut aktif itu tidak tersambung ke apa pun, dan setiap transaksi harus diapit `GPIO.output(NSS_PIN, LOW)` ... `HIGH` sendiri. Inilah yang dikerjakan `_read_reg()` dan `_write_reg()`, dan inilah satu-satunya perbedaan mendasar terhadap driver Arduino yang memakai `digitalWrite(NSS, LOW)` untuk alasan yang sama persis.

Mengapa identitas chip diperiksa lebih dahulu. SX1276 hingga SX1279 selalu menjawab `0x12` pada `REG_VERSION`. Pembacaan itu adalah uji ujung-ke-ujung termurah yang ada: bila jawabannya `0x00` atau `0xFF`, jalur SPI-nya yang bermasalah — HAT tidak duduk sempurna, SPI belum diaktifkan di `raspi-config`, atau NSS salah pin. Radio bahkan belum masuk hitungan. `LoRa.begin()` di Arduino melakukan pemeriksaan yang sama; hanya saja hasilnya diringkas menjadi `true` atau `false`.

Sekuens yang diamati

Sender (Pi)	Receiver (Pi)
loraBegin() reset HW, baca REG_VERSION -> 0x12 set frf, FIFO base, STDBY	loraBegin() (sama) (sama)
beginPacket() STDBY, FIFO_ADDR_PTR=0, PAYLOAD_LENGTH=0 loraPrint("Hello LoRa #0") tulis tiap byte ke REG_FIFO endPacket() REG_OP_MODE = TX ~~~~~ udara ~~~~~> tunggu TX_DONE, lalu bersihkan flagnya	startRx() REG_OP_MODE = RXCONTINUOUS polling REG_IRQ_FLAGS
sleep(2 s), ulangi	RX_DONE menyala FIFO_ADDR_PTR = RX_CURRENT_ADDR baca RX_NB_BYTES byte dari FIFO baca RSSI & SNR paket terakhir cetak, kembali ke STDBY

6.4 Topologi



Node	Program	Peran	Antarmuka pengamatan
Pi #1	src/sender.py	Mengirim Hello LoRa #n tiap 2 detik	terminal / SSH
Pi #2	src/receiver.py	Menerima, mencetak isi + RSSI + SNR	terminal / SSH

Tabel 6.3: Peran tiap node Modul 06

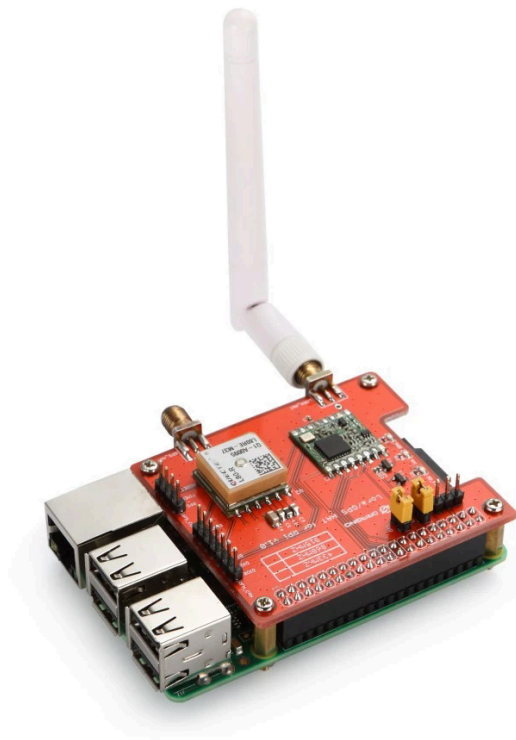
Bila hanya tersedia satu Raspberry Pi, seluruh modul tetap dapat dikerjakan dengan memasangkannya pada satu Arduino Uno bershield Dragino v1.2 yang menjalankan firmware Modul 01 — persis konfigurasi EXP-03. Yang hilang hanyalah kesempatan membandingkan dua terminal Python berdampingan.

6.5 Alat yang Digunakan

Modul ini dijalankan di atas Raspberry Pi dengan Dragino LoRa GPS HAT v1.4 (SX1276), memakai Python 3 dengan spidev dan RPi.GPIO. Tidak ada PlatformIO di modul ini — tidak ada yang dikompilasi.

No	Peralatan	Spesifikasi	Jumlah
1	Raspberry Pi	2 / 3 / 4 (Pi 5 lihat catatan)	2
2	Dragino LoRa GPS HAT	v1.4, SX1276, 433 MHz	2
3	Antena 433 MHz	konektor SMA	2
4	Catu daya Pi	sesuai model	2
5	Kartu microSD	Raspberry Pi OS, SPI aktif	2

Tabel 6.4: Alat dan bahan Modul 06

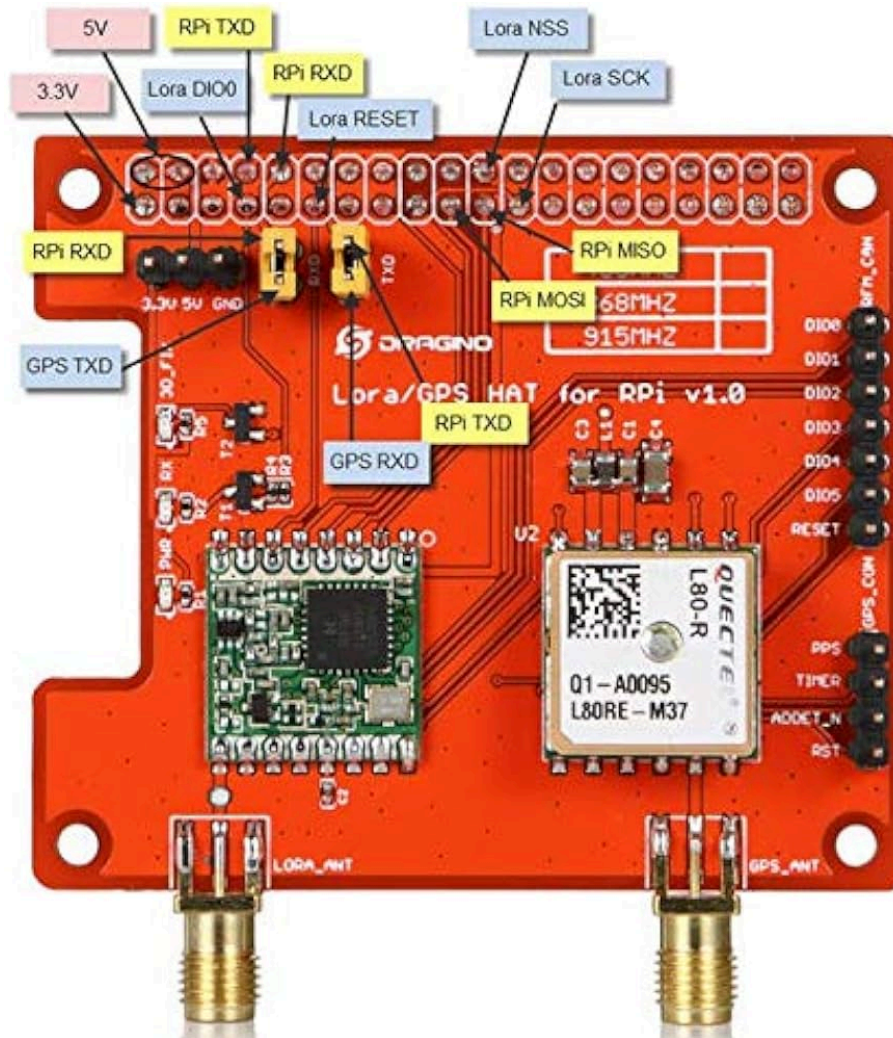


Gambar 6.1: Dragino LoRa GPS HAT v1.4 terpasang di Raspberry Pi

PERINGATAN

Jangan menyalakan HAT tanpa antena. Sama seperti shield Arduino: daya pancar yang tidak menemukan beban dipantulkan kembali ke penguat SX1276 dan dapat merusaknya permanen.

Pemetaan pin HAT ke Raspberry Pi



Gambar 6.2: Pemetaan pin Dragino LoRa GPS HAT v1.4

LoRa GPS HAT	WiringPi	BCM GPIO	Padanan shield	Keterangan
LoRa_NSS	GPIO6	25	D10	Chip select – digerakkan manual
RESET	GPIO0	17	D9	Reset perangkat keras SX1276
DIO0	GPIO7	4	D2	IRQ TX/RX done – tidak dipakai modul ini
SCK	GPIO14	11	D13	SPI0 clock
LoRa_MOSI	GPIO12	10	D11	SPI0 MOSI
LoRa_MISO	GPIO13	9	D12	SPI0 MISO
DIO1	GPIO4	23	D6	tidak dipakai
DIO2	GPIO5	24	D7	tidak dipakai
GPS_RX	GPIO15/TX	14	—	UART ke modul GPS onboard
GPS_TX	GPIO16/RX	15	—	UART dari modul GPS onboard
1PPS	GPIO1	18	—	pulsa 1 detik dari GPS

Tabel 6.5: Pemetaan pin HAT terhadap Raspberry Pi dan padanannya di shield Arduino

Kolom **WiringPi** adalah penomoran yang dipakai dokumentasi resmi Dragino; kolom **BCM** adalah yang dipakai kode Python — seluruh skrip memanggil `GPIO.setmode(GPIO.BCM)`. Kedua kolom menunjuk pin fisik yang sama, dan tertukarnya keduanya adalah penyebab kegagalan yang paling sering terjadi di modul ini.

Bagian GPS pada HAT tidak dipakai sama sekali di seri ini; jalurnya dicantumkan agar praktikan tahu pin mana yang sudah terpakai bila hendak menambahkan perangkat lain.

Struktur proyek

```
Modul06_rpi_lora_python/
├── README.md
├── requirements.txt      ← spidev + RPi.GPIO (atau rpi-lgpio untuk Pi 5)
└── src/
    ├── sender.py        ← kirim "Hello LoRa #n" tiap 2 detik
    └── receiver.py      ← terima, cetak isi + RSSI + SNR
```

Kedua berkas **berdiri sendiri**: masing-masing membawa salinan drivernya. Pemisahan itu disengaja — satu berkas dapat disalin ke Pi mana pun dan langsung berjalan, dan praktikan dapat membongkar satu sisi tanpa takut merusak sisi lain. Konsekuensinya, `sender.py` hanya memuat register yang dibutuhkan TX, sedangkan `receiver.py` memuat lebih banyak: RSSI, SNR, dan pointer FIFO RX. Membandingkan kedua daftar register itu sendiri sudah memberi tahu apa yang khusus diperlukan penerimaan.

6.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul06_rpi_lora_python/](#)

- [Modul06_rpi_lora_python/requirements.txt](#)
- [Modul06_rpi_lora_python/src/sender.py](#)
- [Modul06_rpi_lora_python/src/receiver.py](#)

```
# Modul 06 - dependensi Raspberry Pi (jalankan DI Raspberry Pi, bukan di PC)
#
#   pip3 install -r Modul06_rpi_lora_python/requirements.txt
#
# spidev : akses /dev/spidev0.0 untuk bicara ke SX1276
# RPi.GPIO: kendali jalur NSS, RESET, dan pembacaan DI00
#
# RASPBERRY PI 5. Chip GPIO baru (RP1) tidak didukung RPi.GPIO. Ganti baris
# RPi.GPIO di bawah dengan rpi-lgpio -- drop-in replacement dengan nama modul
# yang sama, sehingga `import RPi.GPIO as GPIO` di kode tetap berjalan apa
# adanya. Jangan memasang keduanya sekaligus: keduanya memakai nama paket
# yang sama dan akan saling menimpa.
spidev>=3.5
RPi.GPIO>=0.7.1
# rpi-lgpio>=0.6 ; khusus Raspberry Pi 5 -- pakai INI dan hapus RPi.GPIO di atas
```

Kode 6.1: `requirements.txt` Modul 06 — `spidev` dan `RPi.GPIO`

```
#!/usr/bin/env python3
"""Modul 06 - LoRa Sender di Raspberry Pi (Dragino LoRa GPS HAT, SX1276).

Padanan Python dari Modul01_lora_uart/src/sender/main.cpp. Perilakunya sama --
mengirim "Hello LoRa #n" setiap 2 detik -- tetapi tidak ada library LoRa di
bawahnya: setiap panggilan API di sini adalah tulisan langsung ke register
SX1276 lewat SPI, dan NSS dikendalikan manual karena HAT memakai GPIO 25,
bukan CE0 bawaan SPI.

Pemetaan pin (Dragino LoRa GPS HAT - WiringPi -> BCM):
LoRa_NSS -> WiringPi GPIO6 -> BCM GPIO 25
RESET    -> WiringPi GPIO0 -> BCM GPIO 17
DI00     -> WiringPi GPIO7 -> BCM GPIO 4
SCK      -> WiringPi GPIO14 -> BCM GPIO 11 (SPI0 CLK)
MOSI     -> WiringPi GPIO12 -> BCM GPIO 10 (SPI0 MOSI)
MISO     -> WiringPi GPIO13 -> BCM GPIO 9  (SPI0 MISO)

Ganti FREQUENCY sesuai versi modul HAT:
433E6 -> 433 MHz   868E6 -> 868 MHz (EU)
```

```

915E6 -> 915 MHz    923E6 -> 923 MHz (Indonesia AS923)

Prasyarat:
sudo raspi-config      # Interface Options > SPI > Yes, lalu reboot
pip3 install -r Modul06_rpi_lora_python/requirements.txt
"""

import time
import sys
import spidev
import RPi.GPIO as GPIO

# --- Pin configuration (BCM numbering) -----
NSS_PIN = 25 # LoRa_NSS (WiringPi GPIO6 = BCM GPIO 25)
RST_PIN = 17 # RESET (WiringPi GPIO0 = BCM GPIO 17)
DIO0_PIN = 4 # DIO0 (WiringPi GPIO7 = BCM GPIO 4)

# --- LoRa parameters -----
FREQUENCY = 433E6 # ganti: 868E6 / 915E6 / 923E6

# --- SX1276 Register Map -----
REG_FIFO = 0x00
REG_OP_MODE = 0x01
REG_FRF_MSB = 0x06
REG_FRF_MID = 0x07
REG_FRF_LSB = 0x08
REG_PA_CONFIG = 0x09
REG_FIFO_ADDR_PTR = 0x0D
REG_FIFO_TX_BASE_ADDR = 0x0E
REG_FIFO_RX_BASE_ADDR = 0x0F
REG_IRQ_FLAGS = 0x12
REG_MODEM_CONFIG_1 = 0x1D
REG_MODEM_CONFIG_2 = 0x1E
REG_PREAMBLE_MSB = 0x20
REG_PREAMBLE_LSB = 0x21
REG_PAYLOAD_LENGTH = 0x22
REG_VERSION = 0x42

# Operating modes
MODE_LONG_RANGE = 0x80
MODE_SLEEP = 0x00
MODE_STDBY = 0x01
MODE_TX = 0x03

# IRQ flags
IRQ_TX_DONE = 0x08

# PA config - pakai PA_BOOST untuk TxPower >= 2 dBm
PA_BOOST = 0x80

# --- SPI instance -----
_spi = spidev.SpiDev()

# --- Low-level SPI helpers (NSS dikontrol manual karena HAT pakai GPIO 25) -----
def _read_reg(addr):
    GPIO.output(NSS_PIN, GPIO.LOW)
    val = _spi.xfer2([addr & 0x7F, 0x00])[1]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return val

def _write_reg(addr, value):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([addr | 0x80, value & 0xFF])
    GPIO.output(NSS_PIN, GPIO.HIGH)

# --- LoRa configuration (mirip API sandeepmistry) -----
def _set_mode(mode):
    _write_reg(REG_OP_MODE, MODE_LONG_RANGE | mode)

def _set_frequency(freq):
    frf = int(round(freq / 32e6 * (1 << 19)))
    _write_reg(REG_FRF_MSB, (frf >> 16) & 0xFF)
    _write_reg(REG_FRF_MID, (frf >> 8) & 0xFF)
    _write_reg(REG_FRF_LSB, frf & 0xFF)

def setSpreadingFactor(sf):
    sf = max(6, min(12, sf))

```

```

_write_reg(REG_MODEM_CONFIG_2,
            (_read_reg(REG_MODEM_CONFIG_2) & 0x0F) | ((sf << 4) & 0xF0))

def setSignalBandwidth(bw):
    bw_table = [7.8e3, 10.4e3, 15.6e3, 20.8e3, 31.25e3, 41.7e3, 62.5e3, 125e3, 250e3]
    idx = min(range(len(bw_table)), key=lambda i: abs(bw_table[i] - bw))
    _write_reg(REG_MODEM_CONFIG_1,
                (_read_reg(REG_MODEM_CONFIG_1) & 0x0F) | (idx << 4))

def setCodingRate4(denominator):
    cr = max(5, min(8, denominator)) - 4
    _write_reg(REG_MODEM_CONFIG_1,
                (_read_reg(REG_MODEM_CONFIG_1) & 0xF1) | (cr << 1))

def setTxPower(level):
    level = max(2, min(17, level))
    _write_reg(REG_PA_CONFIG, PA_BOOST | (level - 2))

# — Packet TX (mirip LoRa.beginPacket / print / endPacket) —————
def beginPacket():
    _set_mode(MODE_STDBY)
    _write_reg(REG_MODEM_CONFIG_1, _read_reg(REG_MODEM_CONFIG_1) & 0xFE) # explicit header
    _write_reg(REG_FIFO_ADDR_PTR, 0x00)
    _write_reg(REG_PAYLOAD_LENGTH, 0x00)

def loraPrint(data):
    if isinstance(data, str):
        data = data.encode('ascii')
    current_len = _read_reg(REG_PAYLOAD_LENGTH)
    size = min(len(data), 255 - current_len)
    for byte in data[:size]:
        _write_reg(REG_FIFO, byte)
        _write_reg(REG_PAYLOAD_LENGTH, current_len + size)

def endPacket():
    _set_mode(MODE_TX)
    while (_read_reg(REG_IRQ_FLAGS) & IRQ_TX_DONE) == 0:
        time.sleep(0.001)
    _write_reg(REG_IRQ_FLAGS, IRQ_TX_DONE) # clear flag

# — Init & cleanup —————
def loraBegin(frequency):
    # GPIO setup
    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(NSS_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(RST_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(DI00_PIN, GPIO.IN)

    # SPI setup (bus 0, device 0 – hardware CE0 diabaikan, NSS dikontrol manual)
    _spi.open(0, 0)
    _spi.max_speed_hz = 5_000_000
    _spi.mode = 0b00

    # Hardware reset
    GPIO.output(RST_PIN, GPIO.LOW)
    time.sleep(0.01)
    GPIO.output(RST_PIN, GPIO.HIGH)
    time.sleep(0.01)

    # Verifikasi chip – SX1276/77/78/79 selalu return 0x12
    version = _read_reg(REG_VERSION)
    if version != 0x12:
        return False

    _set_mode(MODE_SLEEP)
    _set_frequency(frequency)
    _write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
    _write_reg(REG_FIFO_RX_BASE_ADDR, 0x00)
    _set_mode(MODE_STDBY)
    return True

def loraEnd():
    _set_mode(MODE_SLEEP)

```



```

_spi.close()
GPIO.cleanup()

# =====
# Main — identik dengan setup() + loop() di sender.ino
# =====

print("=== LoRa SENDER ===")
print("Init LoRa ... ", end="", flush=True)

if not loraBegin(FREQUENCY):
    print("GAGAL! Cek kabel/modul.")
    print("Pastikan SPI aktif: sudo raspi-config > Interfacing Options > SPI")
    sys.exit(1)

setSpreadingFactor(7)
setSignalBandwidth(125e3)
setCodingRate4(5)
setTxPower(17)

print("OK")
print(f"Frekuensi : {FREQUENCY / 1e6:.0f} MHz")
print("SF=7, BW=125kHz, CR=4/5, Power=17dBm")
print("Kirim tiap 2 detik...\n")

counter = 0
try:
    while True:
        msg = f"Hello LoRa #{counter}"

        print(f'[TX] "{msg}" ... ', end="", flush=True)
        beginPacket()
        loraPrint(msg)
        endPacket()
        print("terkirim")

        counter += 1
        time.sleep(2)

except KeyboardInterrupt:
    print("\n\nProgram dihentikan oleh pengguna.")
finally:
    loraEnd()
    print("Menutup LoRa module..")

```

Kode 6.2: src/sender.py — driver SX1276 sisi TX, seluruhnya di atas register

```

#!/usr/bin/env python3
"""Modul 06 - LoRa Receiver di Raspberry Pi (Dragino LoRa GPS HAT, SX1276).

Padanan Python dari Modul01_lora_uart/src/receiver/main.cpp. Menerima paket,
lalu mencetak isi beserta RSSI dan SNR yang dibaca langsung dari register
paket terakhir. RX dikerjakan dengan polling REG_IRQ_FLAGS, bukan interrupt --
sama seperti LoRa.parsePacket() di sisi Arduino.

Jalankan receiver LEBIH DAHULU, baru sender.

Pemetaan pin (Dragino LoRa GPS HAT - WiringPi -> BCM):
LoRa_NSS -> WiringPi GPIO6 -> BCM GPIO 25
RESET    -> WiringPi GPIO0 -> BCM GPIO 17
DIO0     -> WiringPi GPIO7 -> BCM GPIO 4
SCK      -> WiringPi GPIO14 -> BCM GPIO 11 (SPI0 CLK)
MOSI     -> WiringPi GPIO12 -> BCM GPIO 10 (SPI0 MOSI)
MISO     -> WiringPi GPIO13 -> BCM GPIO 9  (SPI0 MISO)

Ganti FREQUENCY sesuai versi modul HAT:
433E6 -> 433 MHz   868E6 -> 868 MHz (EU)
915E6 -> 915 MHz   923E6 -> 923 MHz (Indonesia AS923)

Prasyarat:
sudo raspi-config      # Interface Options > SPI > Yes, lalu reboot
pip3 install -r Modul06_rpi_lora_python/requirements.txt
"""

import time
import sys
import spidev
import RPi.GPIO as GPIO

```



```

# --- Pin configuration (BCM numbering) -----
NSS_PIN = 25 # LoRa_NSS (WiringPi GPIO6 = BCM GPIO 25)
RST_PIN = 17 # RESET (WiringPi GPIO00 = BCM GPIO 17)
DIO0_PIN = 4 # DIO0 (WiringPi GPIO7 = BCM GPIO 4)

# --- LoRa parameters -----
FREQUENCY = 433E6 # ganti: 868E6 / 915E6 / 923E6

# --- SX1276 Register Map -----
REG_FIFO = 0x00
REG_OP_MODE = 0x01
REG_FRF_MSB = 0x06
REG_FRF_MID = 0x07
REG_FRF_LSB = 0x08
REG_PA_CONFIG = 0x09
REG_FIFO_ADDR_PTR = 0x0D
REG_FIFO_TX_BASE_ADDR = 0x0E
REG_FIFO_RX_BASE_ADDR = 0x0F
REG_FIFO_RX_CURRENT_ADDR = 0x10
REG_IRQ_FLAGS_MASK = 0x11
REG_IRQ_FLAGS = 0x12
REG_RX_NB_BYTES = 0x13
REG_PKT_RSSI_VALUE = 0x1A
REG_PKT_SNR_VALUE = 0x1B
REG_MODEM_CONFIG_1 = 0x1D
REG_MODEM_CONFIG_2 = 0x1E
REG_PREAMBLE_MSB = 0x20
REG_PREAMBLE_LSB = 0x21
REG_PAYLOAD_LENGTH = 0x22
REG_VERSION = 0x42

# Operating modes
MODE_LONG_RANGE = 0x80
MODE_SLEEP = 0x00
MODE_STDBY = 0x01
MODE_TX = 0x03
MODE_RX_CONTINUOUS = 0x05
MODE_RX_SINGLE = 0x06

# IRQ flags
IRQ_RX_DONE = 0x40
IRQ_TX_DONE = 0x08
IRQ_CRC_ERROR = 0x20

# PA config - pakai PA_BOOST untuk TxPower >= 2 dBm
PA_BOOST = 0x80

# --- SPI instance -----
_spi = spidev.SpiDev()

# --- Low-level SPI helpers (NSS dikontrol manual karena HAT pakai GPIO 25) -----
def _read_reg(addr):
    GPIO.output(NSS_PIN, GPIO.LOW)
    val = _spi.xfer2([addr & 0x7F, 0x00])[1]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return val

def _write_reg(addr, value):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([addr | 0x80, value & 0xFF])
    GPIO.output(NSS_PIN, GPIO.HIGH)

# --- LoRa configuration -----
def _set_mode(mode):
    _write_reg(REG_OP_MODE, MODE_LONG_RANGE | mode)

def _set_frequency(freq):
    frf = int(round(freq / 32e6 * (1 << 19)))
    _write_reg(REG_FRF_MSB, (frf >> 16) & 0xFF)
    _write_reg(REG_FRF_MID, (frf >> 8) & 0xFF)
    _write_reg(REG_FRF_LSB, frf & 0xFF)

def setSpreadingFactor(sf):
    sf = max(6, min(12, sf))
    _write_reg(REG_MODEM_CONFIG_2,
        (_read_reg(REG_MODEM_CONFIG_2) & 0x0F) | ((sf << 4) & 0xF0))

```

```

def setSignalBandwidth(bw):
    bw_table = [7.8e3, 10.4e3, 15.6e3, 20.8e3, 31.25e3, 41.7e3, 62.5e3, 125e3, 250e3]
    idx = min(range(len(bw_table)), key=lambda i: abs(bw_table[i] - bw))
    _write_reg(REG_MODEM_CONFIG_1,
               (_read_reg(REG_MODEM_CONFIG_1) & 0x0F) | (idx << 4))

def setCodingRate4(denominator):
    cr = max(5, min(8, denominator)) - 4
    _write_reg(REG_MODEM_CONFIG_1,
               (_read_reg(REG_MODEM_CONFIG_1) & 0xF1) | (cr << 1))

def setTxPower(level):
    level = max(2, min(17, level))
    _write_reg(REG_PA_CONFIG, PA_BOOST | (level - 2))

# — Packet RX (mirip LoRa.parsePacket / read) —————
def parsePacket():
    """Cek apakah ada paket data diterima. Return ukuran paket (0 jika tidak ada)."""
    irq = _read_reg(REG_IRQ_FLAGS)
    if irq & IRQ_CRC_ERROR:
        _write_reg(REG_IRQ_FLAGS, IRQ_CRC_ERROR) # clear CRC error flag
        return 0
    if (irq & IRQ_RX_DONE) == 0:
        return 0
    _write_reg(REG_IRQ_FLAGS, IRQ_RX_DONE) # clear flag
    return _read_reg(REG_RX_NB_BYTES)

def loraAvailable():
    """Return jumlah byte tersedia di FIFO yang siap dibaca."""
    return _read_reg(REG_RX_NB_BYTES)

def loraRead():
    """Baca 1 byte dari FIFO RX."""
    return _read_reg(REG_FIFO)

def packetRssi():
    """Return RSSI paket terakhir (dalam dBm)."""
    rssi = _read_reg(REG_PKT_RSSI_VALUE)
    freq_mhz = FREQUENCY / 1e6
    if freq_mhz < 868:
        return rssi - 164
    else:
        return rssi - 157

def packetSnr():
    """Return SNR paket terakhir (dalam dB)."""
    val = _read_reg(REG_PKT_SNR_VALUE)
    if val > 127:
        val = val - 256
    return val / 4.0

# — RX mode handlers —————
def startRx():
    """Mulai RX continuous mode."""
    _set_mode(MODE_RX_CONTINUOUS)

def initRxSequence():
    """Setelah keluar dari RX, atur ulang pointer FIFO ke RX base."""
    _write_reg(REG_FIFO_ADDR_PTR, _read_reg(REG_FIFO_RX_CURRENT_ADDR))

# — Init & cleanup —————
def loraBegin(frequency):
    # GPIO setup
    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(NSS_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(RST_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(DIO0_PIN, GPIO.IN)

    # SPI setup (bus 0, device 0 — hardware CE0 diabaikan, NSS dikontrol manual)

```

```

_spi.open(0, 0)
_spi.max_speed_hz = 5_000_000
_spi.mode = 0b00

# Hardware reset
GPIO.output(RST_PIN, GPIO.LOW)
time.sleep(0.01)
GPIO.output(RST_PIN, GPIO.HIGH)
time.sleep(0.01)

# Verifikasi chip – SX1276/77/78/79 selalu return 0x12
version = _read_reg(REG_VERSION)
if version != 0x12:
    return False

_set_mode(MODE_SLEEP)
_set_frequency(frequency)
_write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
_write_reg(REG_FIFO_RX_BASE_ADDR, 0x00)
_set_mode(MODE_STDBY)
return True

def loraEnd():
    _set_mode(MODE_SLEEP)
    _spi.close()
    GPIO.cleanup()

# =====
# Main – identik dengan setup() + loop() di receiver.ino
# =====
print("=== LoRa RECEIVER ===")
print("Init LoRa ... ", end="", flush=True)

if not loraBegin(FREQUENCY):
    print("GAGAL! Cek kabel/modul.")
    print("Pastikan SPI aktif: sudo raspi-config > Interfacing Options > SPI")
    sys.exit(1)

setSpreadingFactor(7)
setSignalBandwidth(125e3)
setCodingRate4(5)
setTxPower(17)

print("OK")
print(f"Frekuensi : {FREQUENCY / 1e6:.0f} MHz")
print("SF=7, BW=125kHz, CR=4/5, Power=17dBm")
print("Menunggu data ...\n")

counter = 0
try:
    while True:
        startRx()

        # Polling – tunggu paket masuk atau sampai timeout ~3 detik
        packet_size = 0
        timeout_ms = 0
        while packet_size == 0 and timeout_ms < 3000:
            packet_size = parsePacket()
            if packet_size == 0:
                time.sleep(0.001)
                timeout_ms += 1

        if packet_size > 0:
            initRxSequence()
            data = bytes(loraRead() for _ in range(packet_size)).decode('ascii', errors='replace')
            rssi = packetRssi()
            snr = packetSnr()
            counter += 1
            print(f'[RX #{counter}] "{data}" | RSSI={rssi:.0f} dBm | SNR={snr:.1f} dB')

            # Kembali ke standby sebentar lalu RX lagi
            _set_mode(MODE_STDBY)
except KeyboardInterrupt:
    print("\n\nProgram dihentikan oleh pengguna.")
finally:
    loraEnd()
    print("Menutup LoRa module...")

```

Kode 6.3: src/receiver.py — driver SX1276 sisi RX, termasuk pembacaan RSSI dan SNR

6.7 Persiapan dan Menjalankan

```
sudo raspi-config          # Interface Options > SPI > Yes
sudo reboot
ls /dev/spi*              # harus muncul: /dev/spidev0.0 /dev/spidev0.1

pip3 install -r Modul06_rpi_lora_python/requirements.txt
```

CATATAN

Raspberry Pi 5. Chip GPIO baru (RP1) tidak didukung RPi.GPIO. Pasang `rpi-lgpio` sebagai gantinya — paket itu menyediakan modul dengan nama yang sama, sehingga `import RPi.GPIO as GPIO` di kode tetap berjalan apa adanya. **Jangan memasang keduanya sekaligus**; keduanya mendaftarkan nama modul yang sama dan akan saling menimpa. Rinciannya ada di `requirements.txt`.

Menjalankan — penerima lebih dahulu, agar paket pertama pengirim tidak terbang.

```
python3 Modul06_rpi_lora_python/src/receiver.py    # di Pi #2
python3 Modul06_rpi_lora_python/src/sender.py      # di Pi #1
```

Hentikan dengan `Ctrl-C`. Kedua skrip menutup SPI dan mengembalikan GPIO ke keadaan semula pada blok `finally`, sehingga penghentian di tengah jalan tidak meninggalkan pin dalam keadaan menggantung.

Pre-flight checklist

- ☐ Antena terpasang pada kedua HAT.
- ☐ SPI aktif di kedua Pi — `ls /dev/spi*` menampilkan `spidev0.0`.
- ☐ `spidev` dan `RPi.GPIO` (atau `rpi-lgpio` pada Pi 5) sudah terpasang.
- ☐ Nilai `FREQUENCY` pada kedua berkas **sama persis** dan sesuai band HAT yang dipakai.
- ☐ Bila mengerjakan EXP-03, satu Arduino Uno Modul 01 siap beserta Serial Monitor 9600 baud.

6.8 Percobaan

6.8.1 EXP-01 — Chip Menjawab Namanya

Jalankan `receiver.py`, dan amati baris pertama sebelum apa pun mengudara.

Expected output

```
=== LoRa RECEIVER ===
Init LoRa ... OK
Frekuensi : 433 MHz
SF=7, BW=125kHz, CR=4/5, Power=17dBm
Menunggu data ...
```

Data capture

Parameter	Hasil
Pesan init pada Pi sender	
Pesan init pada Pi receiver	
Isi <code>/dev/spi*</code> yang terdaftar	
Model Raspberry Pi dan versi Python	
Library GPIO yang dipakai (RPi.GPIO / <code>rpi-lgpio</code>)	

Tabel 6.6: Lembar pengamatan EXP-01

BUKA ABSTRAKSINYA

Kata OK pada baris kedua sepenuhnya bergantung pada satu perbandingan di dalam `loraBegin(): version != 0x12`. Sisipkan `print(hex(version))` tepat sebelum baris itu, jalankan ulang, dan catat nilainya. Kemudian **lepas HAT dari header** (Pi dimatikan lebih dahulu), pasang kembali dengan longgar sengaja, dan jalankan lagi. Bandingkan nilai yang tercetak pada kedua keadaan, lalu jelaskan mengapa `0x00` dan `0xFF` adalah dua gejala kegagalan yang berbeda.

CHECKPOINT

Kedua Pi mencetak OK. Bila muncul GAGAL!, jangan menyentuh parameter radio sama sekali: yang gagal adalah lapisan SPI, bukan lapisan radio. Periksa berurutan — SPI aktif di `raspi-config`, `/dev/spidev0.0` ada, HAT duduk penuh di 40 pin, lalu barulah kabel.

6.8.2 EXP-02 — Aliran Data Satu Arah

Jalankan receiver, lalu sender. Biarkan berjalan dua menit sambil mengamati nomor urut di kedua terminal.

Expected output — receiver

```
[RX #1] "Hello LoRa #0" | RSSI=-48 dBm | SNR=9.5 dB
[RX #2] "Hello LoRa #1" | RSSI=-47 dBm | SNR=9.8 dB
[RX #3] "Hello LoRa #2" | RSSI=-49 dBm | SNR=9.3 dB
```

Data capture

Parameter	Hasil
Nomor urut terakhir di sender	
Nomor urut terakhir di receiver (Hello LoRa #)	
Nomor [RX #n] terakhir di receiver	
Paket hilang (selisih nomor urut)	
RSSI rata-rata (dBm)	
SNR rata-rata (dB)	

Tabel 6.7: Lembar pengamatan EXP-02

Perhatikan bahwa receiver mencetak **dua** penghitung yang berbeda: [RX #n] adalah hitungan lokal paket yang berhasil diterima, sedangkan angka di dalam Hello LoRa #n berasal dari sender. Selisih keduanya adalah jumlah paket yang hilang — dan itulah alasan nomor urut ditaruh di dalam payload sejak M01.

CHECKPOINT

Nomor urut yang diterima naik satu per satu tanpa lompatan pada jarak dekat. Bila [RX #n] naik tetapi isi payload berulang atau teracak, yang bermasalah adalah pembacaan FIFO, bukan tautan radionya: periksa apakah `initRxSequence()` benar-benar dipanggil sebelum byte dibaca.

6.8.3 EXP-03 — Uji Silang Platform

Inilah percobaan inti modul ini. Parameter dan payload Modul 06 sengaja dibuat identik dengan Modul 01, sehingga kedua platform dapat dipasangkan silang tanpa mengubah kode mana pun.

Uji	Pengirim	Penerima	Hasil yang teramati
03-a	Pi sender.py	Pi receiver.py	
03-b	Pi sender.py	Arduino M01 receiver (Serial 9600)	
03-c	Arduino M01 sender	Pi receiver.py	
03-d	Arduino M01 sender	Arduino M01 receiver	

Tabel 6.8: Lembar pengamatan EXP-03 — uji silang platform

Catat RSSI dan SNR pada tiap baris. Perhatikan bahwa uji 03-b dan 03-c melibatkan dua penulis kode yang sama sekali berbeda — driver Python di berkas ini dan library sandeepmistry di Arduino — yang tidak pernah saling melihat kode satu sama lain.

CHECKPOINT

Keempat kombinasi berhasil. Bila hanya 03-a dan 03-d yang berhasil sedangkan kombinasi silang gagal, penyebabnya hampir pasti parameter yang tidak seragam: periksa FREQUENCY di berkas Python **dan** FREQUENCY di source Arduino, lalu SF, BW, dan CR pada keduanya. Kegagalan silang hampir tidak pernah disebabkan perbedaan platform.

Mengapa uji silang ini berhasil. Yang berbicara di udara bukan Python dan bukan C++, melainkan SX1276. Selama kedua chip disetel ke frekuensi, spreading factor, bandwidth, dan mode header yang sama, keduanya menghasilkan bentuk gelombang yang identik — dan bentuk gelombang itu tidak membawa jejak apa pun tentang bahasa atau prosesor yang menyusunnya. Library hanyalah cara yang lebih nyaman untuk menulis register yang sama.

6.8.4 EXP-04 — Membaca Kembali Apa yang Ditulis

Semua register yang ditulis dapat dibaca kembali. Verifikasi tiga di antaranya dengan menyisipkan potongan Kode 6.4 setelah blok setTxPower(17) di receiver.py.

```
print("REG_OP_MODE      =", hex(_read_reg(REG_OP_MODE)))
print("REG_FRF (MSB/MID/LSB) =",
      hex(_read_reg(REG_FRF_MSB)), hex(_read_reg(REG_FRF_MID)), hex(_read_reg(REG_FRF_LSB)))
print("REG_MODEM_CONFIG_1/2 =",
      hex(_read_reg(REG_MODEM_CONFIG_1)), hex(_read_reg(REG_MODEM_CONFIG_2)))
```

Kode 6.4: Potongan verifikasi register untuk EXP-04

Data capture

Register	Nilai terbaca	Nilai yang diharapkan (hitung sendiri)	Cocok?
REG_OP_MODE			
REG_FRF_MSB			
REG_FRF_MID			
REG_FRF_LSB			
REG_MODEM_CONFIG_1			

Register	Nilai terbaca	Nilai yang diharapkan (hitung sendiri)	Cocok?
REG_MODEM_CONFIG_2			

Tabel 6.9: Lembar pengamatan EXP-04 — membaca kembali register

Hitung frf dengan tangan lebih dahulu:

$$frf = \frac{433\,000\,000}{32\,000\,000} \times 2^{19} \quad (6.1)$$

lalu pecah hasilnya menjadi tiga byte. Untuk REG_MODEM_CONFIG_1, susun dari kolom-kolomnya: empat bit teratas adalah indeks bandwidth (125 kHz menempati indeks ke berapa pada `bw_table?`), tiga bit berikutnya coding rate dikurangi 4, dan bit terbawah menandai mode header. Untuk REG_MODEM_CONFIG_2, empat bit teratas adalah spreading factor.

CHECKPOINT

Ketiga byte frf cocok dengan hitungan tangan, dan nibble atas REG_MODEM_CONFIG_2 bernilai 7. Bila frf meleset satu-dua satuan, periksa pembulatan — kode memakai `int(round(...))`, dan pembulatan ke bawah menghasilkan pergeseran frekuensi sebesar puluhan hertz yang tidak berbahaya tetapi membuat angkanya tidak cocok.

6.8.5 Catatan Verifikasi

CATATAN

Berbeda dari Modul 01–05, **modul ini belum memiliki log referensi hasil uji perangkat**. Kode sumbernya berasal dari repositori Dragino LoRa GPS HAT yang menyatakan telah diuji dan berjalan pada Raspberry Pi 4; pengujian ulang setelah dijadikan modul lab ini belum dilakukan, dan tidak ada Raspberry Pi bershield HAT yang tersambung saat penyusunan. Angka pada seluruh tabel di atas memang disediakan kosong untuk diisi praktikan — termasuk tabel EXP-04, yang jawabannya justru dapat diperiksa sendiri lewat hitungan tangan tanpa perlu membandingkan dengan siapa pun.

6.9 Pengukuran

A. Jarak terhadap kualitas tautan — sender dan receiver dipisahkan pada empat jarak, masing-masing diamati 60 detik.

Jarak	RSSI (dBm)	SNR (dB)	Dikirim	Diterima	Loss (%)
1 m					
10 m					
50 m					
100 m					

Tabel 6.10: Lembar pengukuran A — jarak terhadap kualitas tautan

B. Perbandingan platform pada jarak yang sama — ulangi tiap kombinasi EXP-03 selama 60 detik pada jarak tetap 10 m.

Kombinasi	RSSI (dBm)	SNR (dB)	Loss (%)
Pi → Pi			
Pi → Arduino			
Arduino → Pi			
Arduino → Arduino			

Tabel 6.11: Lembar pengukuran B — perbandingan platform

Bila salah satu baris menyimpang jauh dari tiga lainnya, calon penyebabnya bukan platform melainkan antenna, tinggi pemasangan, atau bidang logam di dekatnya — catat kondisi fisiknya bersama angkanya.

C. Ketepatan waktu kirim — sender memakai `time.sleep(2)`, bukan penjadwal. Catat selang antar-paket dari cap waktu penerimaan.

Ukuran	Nilai
Selang antar-paket minimum (s)	
Selang antar-paket maksimum (s)	
Selang rata-rata (s)	
Simpangan terhadap 2,000 s	

Tabel 6.12: Lembar pengukuran C — ketepatan waktu kirim

6.10 Analisis

1. Dari Tabel 6.11, apakah kombinasi silang Pi ↔ Arduino menunjukkan RSSI atau loss yang berbeda secara berarti dibanding kombinasi sejenis? Bila ya, sebutkan penyebab fisik yang paling mungkin — dan jelaskan mengapa perbedaan bahasa pemrograman **bukan** salah satunya.
2. Tabel 6.12 hampir pasti menunjukkan selang yang lebih besar dari 2,000 detik. Jelaskan sumbernya dengan menyebut dua hal: apa yang dikerjakan `endPacket()` selama menunggu, dan apa yang dijamin — serta tidak dijamin — oleh `time.sleep()` pada sistem operasi bertugas banyak seperti Linux.
3. `receiver.py` memuat register `REG_PKT_RSSI_VALUE`, `REG_PKT_SNR_VALUE`, dan `REG_FIFO_RX_CURRENT_ADDR`, sedangkan `sender.py` tidak. Jelaskan untuk masing-masing mengapa pengiriman tidak memerlukannya.
4. Fungsi `packetRssi()` mengurangi 164 untuk band di bawah 868 MHz dan 157 untuk band di atasnya. Apa yang akan terjadi pada seluruh pembacaan RSSI bila HAT 868 MHz dipakai tetapi `FREQUENCY` di kode tetap tertulis 433E6? Sebutkan besar dan arah kesalahannya, lalu jelaskan mengapa kesalahan itu tidak akan terlihat sebagai pesan galat.
5. Modul ini melepaskan library dan menggantinya dengan register langsung. Sebutkan satu hal yang menjadi **lebih mudah** dan satu hal yang menjadi **lebih sulit** akibat keputusan itu, masing-masing dengan contoh konkret dari kode di `src/`.

6.11 Concept Check

1. Mengapa bit ke-7 alamat dipakai untuk membedakan baca dan tulis, dan berapa banyak alamat register yang tersisa akibat pilihan itu?
2. IRQ flag dibersihkan dengan **menulis 1** ke bit yang bersangkutan, bukan 0. Apa keuntungan rancangan itu ketika beberapa flag menyala bersamaan?

3. Apa akibatnya bila `beginPacket()` lupa menyetel ulang `REG_FIFO_ADDR_PTR` ke 0 sebelum byte payload ditulis?
4. `loraBegin()` memindahkan chip ke SLEEP sebelum menyetel frekuensi, lalu ke STDBY sesudahnya. Mengapa urutannya tidak dibalik?
5. Pada Arduino, `LoRa.begin()` mengembalikan `false` bila chip tidak menjawab. Di modul ini, kegagalan yang sama muncul sebagai `version != 0x12`. Sebutkan satu keadaan perangkat keras yang menghasilkan `0x00` dan satu keadaan yang menghasilkan `0xFF`.

6.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Merapikan dan mengukur driver sendiri

CH-1 — Satu berkas driver bersama Pindahkan seluruh fungsi register ke `src/sx1276.py`, lalu buat `sender.py` dan `receiver.py` mengimpornya. Jelaskan apa yang menjadi lebih baik dan apa yang menjadi lebih buruk untuk keperluan belajar dibanding versi berdiri sendiri.

CH-3 — Hitung loss otomatis Uraikan nomor urut dari payload, bandingkan dengan nomor sebelumnya, dan cetak LOSS: `n` paket setiap kali terjadi lompatan — beserta ringkasan RSSI minimum, maksimum, dan rata-rata setiap 10 paket.

CH-4 — Catat ke berkas CSV Simpan tiap paket sebagai satu baris waktu, nomor, rssi, snr agar tabel Pengukuran dapat diisi dari data, bukan dari salinan layar. Bandingkan pendekatan ini dengan `lora_monitor.py` pada Modul 05.

Level 3 — Tantangan · Menaikkan modul ini ke tingkat M02

CH-2 — Beralih ke interrupt DIO0 sudah tersambung ke BCM GPIO 4 tetapi tidak pernah dipakai. Ganti polling di `receiver.py` dengan `GPIO.add_event_detect(DIO0_PIN, GPIO.RISING, callback=...)`, lalu bandingkan pemakaian CPU sebelum dan sesudah dengan `top`. Ini adalah M02 yang dikerjakan ulang di Linux.

CH-5 — Baca GPS di HAT Modul GPS onboard mengirim kalimat NMEA pada UART 9600 bps. Baca satu kalimat `$GPGGA`, ambil lintang dan bujuranya, lalu sisipkan ke dalam payload LoRa. Perhatikan berapa byte payload bertambah dan perkirakan pengaruhnya terhadap waktu udara.

6.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — transaksi SPI, peta register yang dipakai, peran tiap mode operasi.
3. Konfigurasi — pemetaan pin HAT (BCM dan WiringPi), nilai SF/BW/CR/frekuensi, versi Pi dan library GPIO.
4. Hasil eksperimen — keluaran terminal EXP-01 sampai EXP-04 beserta checkpoint.
5. Data pengukuran — Tabel 6.10, Tabel 6.11, dan Tabel 6.12, termasuk hitungan tangan `frf`.
6. Analisis dan concept check.
7. Challenge — minimal CH-2 atau CH-3.
8. Kesimpulan yang disusun sendiri, dengan pernyataan tegas mengenai hasil uji silang platform pada EXP-03.

7. Modul 07 — Gateway Linux Menjadwalkan Node Arduino

LoRa COMMUNICATION LAB · MODUL 07

Move the Scheduler to Linux — Gateway Linux Menjadwalkan Node Arduino

Raspberry Pi + LoRa GPS HAT v1.4 dan 2 × Arduino Uno + LoRa Shield v1.2 · topologi bintang, 3 node · polling terjadwal · level Advanced · 3 × 50 menit · folder kode Modul07_rpi_master_slave

PENGANTAR MODUL · Gambaran Umum

Modul 07 dirancang untuk tiga pertemuan (3 × 50 menit) pada tingkat lanjut, dan menutup arc pertama. Misinya memindahkan **master** Modul 05 dari Arduino Uno ke Raspberry Pi tanpa mengubah **satu baris pun** firmware slave — membentuk topologi gateway yang lazim di dunia nyata: satu komputer Linux yang menjadwalkan sekumpulan node mikrokontroler murah lewat radio, dikendalikan dari jarak jauh lewat SSH.

7.1 Pendahuluan

Percobaan memakai satu Raspberry Pi bershield Dragino LoRa GPS HAT v1.4 sebagai master dan dua Arduino Uno bershield Dragino LoRa v1.2 sebagai slave, sama persis dengan perangkat keras slave Modul 05.

Modul ini adalah pertemuan dua kemampuan yang dibangun terpisah. Modul 05 membangun **penjadwalan**: pengalamatan aplikasi, round-robin, batas waktu per node, statistik terpisah. Modul 06 membangun **driver telanjang**: cara memegang register SX1276 langsung dari Python lewat spidev dan RPi.GPIO, dengan nama fungsi yang sengaja meniru API sandeepmistry (beginPacket, endPacket, parsePacket, packetRssi). Modul ini menggabungkan keduanya: src/master.py adalah penjadwal round-robin Modul 05, ditulis ulang di atas driver telanjang Modul 06. Kontrak datanya — POLL:<id> dan S<id>:DATA:<n> — tidak berubah sedikit pun, dan itulah yang membuat firmware slave dapat dipakai ulang tanpa modifikasi.

Prasyaratnya adalah M05 untuk logika penjadwalan dan M06 untuk driver telanjang. Yang dibangun di sini adalah pembuktian bahwa keduanya dapat disatukan lintas platform: master berganti dari C++/AVR ke Python/Linux, slave tidak menyadarinya sama sekali. Yang juga dibangun — dan tidak pernah muncul di modul-modul satu mesin sebelumnya — adalah **korelasi log lintas dua komputer independen**: master direkam di Raspberry Pi lewat SSH, slave direkam di laptop pengembang lewat USB, dan kedua rekaman tidak berbagi jam sama sekali. Isinya, bukan cap waktunya, yang membuktikan keduanya benar-benar saling bicara.

Peta modul LoRa (penutup arc pertama)

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
01	Tautan satu arah terbentuk; RSSI dan SNR terbaca
02	Penerimaan lewat interrupt — loop() tidak lagi menunggu
03	Dua arah bergantian di atas radio half-duplex
04	Setiap pengiriman diketahui hasilnya: ACK, timeout, statistik
05	Banyak node — hak bicara dijadwalkan agar tidak bertabrakan
06	Isi LoRa.begin() tidak pernah terlihat — register dipegang langsung

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
07 (ini)	Penjadwal pindah ke gateway Linux; sisi node tidak berubah sama sekali

Tabel 7.1: Peta modul pada arc pertama seri LoRa

Kontrak data lab ini. Sama persis dengan Modul 05: perintah master berbentuk `POLL:<id>`, jawaban slave berbentuk `S<id>:DATA:<n>`. Tidak ada satu byte pun yang berubah di udara — yang berganti hanya bahasa dan platform yang menyusun serta membaca byte itu. Perbedaan halus satu-satunya ada di pesan pembuka Serial slave, yang sekarang menyebut "Master (Raspberry Pi)".

7.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Memindahkan penjadwal ke gateway Linux tanpa menyentuh node

1. Membuktikan dengan diff bahwa memindahkan master ke platform lain tidak menuntut perubahan apa pun pada firmware slave, dan menjelaskan properti desain yang membuat itu mungkin.
2. Menerapkan ulang logika penjadwalan round-robin Modul 05 di atas driver SX1276 telanjang Modul 06, memakai `spidev` dan `RPi.GPIO` atau `rpi-lgpio`.
3. Mengkorelasikan log dari dua mesin yang tidak berbagi jam — satu direkam lewat SSH di Raspberry Pi, satu lewat USB lokal — berdasarkan isi pesan, bukan cap waktu.
4. Menjelaskan mengapa mematikan CRC payload membuat paket rusak lolos ke lapisan aplikasi sebagai kegagalan yang salah didiagnosis, dan mengukur seberapa sering itu terjadi.
5. Menelusuri anggaran waktu sesungguhnya di balik `POLL_TIMEOUT` — bagian siklus mana yang benar-benar dihitungnya — lalu memakainya untuk menjelaskan letak sesungguhnya ambang kegagalan pada EXP-04.

Kriteria keberhasilan

- ☐ `master.py` berhasil memanggil kedua slave Uno bergiliran tanpa mengubah satu baris pun firmware slave — dibuktikan dengan diff terhadap `Modul05_lora_master_slave/src/slave/main.cpp`.
- ☐ `cek_radio.py` membaca balik register SX1276 di Raspberry Pi dan hasilnya dibandingkan dengan konfigurasi yang dimaksud.
- ☐ Satu sesi log master (Raspberry Pi) dan log slave (laptop) dikorelasikan lewat isi pesan, bukan lewat waktu perekaman.
- ☐ Slave dimatikan secara fisik saat sistem berjalan (EXP-03); master tetap melayani slave lain dan pulih otomatis begitu slave kembali.
- ☐ Ambang kegagalan `POLL_TIMEOUT` ditemukan dalam rentang beberapa milidetik, dan letaknya dijelaskan lewat anggaran waktu siklus, bukan hanya dicatat sebagai angka.

7.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Gateway	Node yang menjadwalkan jaringan radio dari komputer bertenaga penuh, terpisah dari node yang dijadwalkannya. Di sini: Raspberry Pi.
<code>spidev</code>	Antarmuka kernel Linux ke SPI perangkat keras — satu-satunya jalan bicara Python ke SX1276.
<code>RPi.GPIO</code> / <code>rpi-lgpio</code>	Kendali pin NSS/RESET dan pembacaan DIO0. Pi 5 memakai chip GPIO baru (RP1) yang butuh <code>rpi-lgpio</code> sebagai pengganti drop-in.

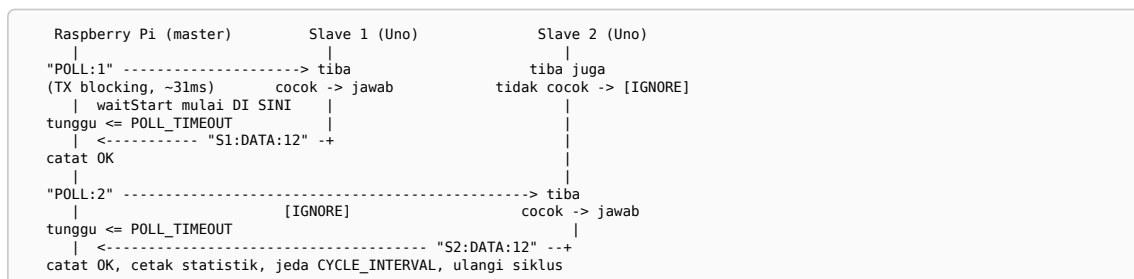
Istilah	Definisi kerja di lab ini
Jalankan lewat SSH	Master tidak punya layar sendiri; seluruh interaksi — unggah kode, jalankan, hentikan — dilakukan dari terminal jarak jauh.
CRC payload	Pemeriksaan keutuhan data oleh radio sendiri. Mati pada modul ini di kedua sisi — konsekuensinya dibahas di bagian Percobaan.
Anggaran POLL_TIMEOUT	Bagian siklus yang benar-benar ditunggu batas waktu. TX POLL bersifat blocking dan selesai sebelum penghitung waktu mulai berjalan, sehingga anggarannya jauh lebih sempit dari dugaan naif.
Korelasi log lintas mesin	Membuktikan dua rekaman dari komputer berbeda menggambarkan peristiwa yang sama, memakai isi pesan (nomor urut, payload) sebagai pengikat karena keduanya tidak berbagi jam.

Tabel 7.2: Istilah kerja Modul 07

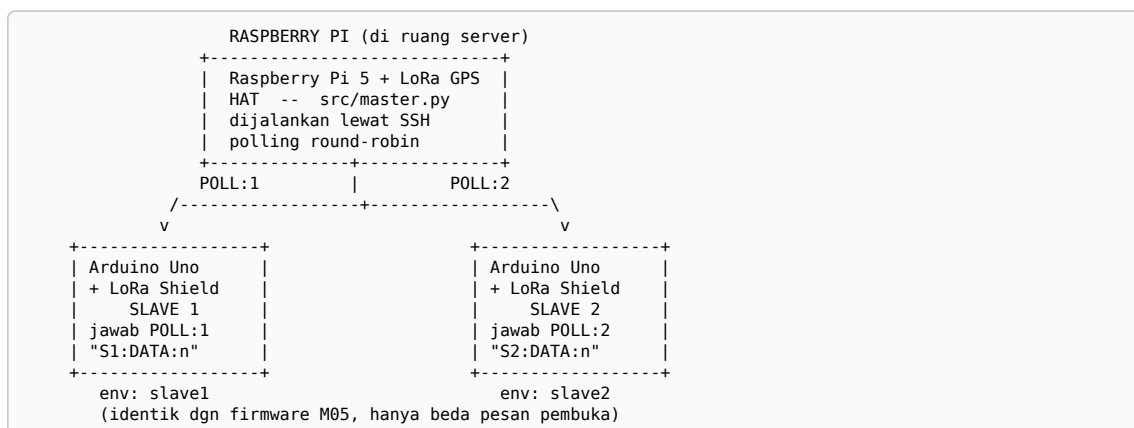
Mengapa firmware slave tidak perlu tahu siapa master-nya. Slave hanya mendengar dua hal: POLL:<id> yang cocok dengan nomornya, dan segala sesuatu yang lain untuk diabaikan. Ia tidak pernah memeriksa dari mana POLL itu berasal, apalagi platform apa yang mengirimkannya. Selama pengirim baru menghasilkan bentuk gelombang yang identik — frekuensi, SF, BW, CR yang sama — SX1276 di sisi slave tidak dapat membedakan apakah lawan bicaranya Arduino atau Raspberry Pi. Inilah properti yang membuat pemindahan master menjadi mungkin tanpa sentuhan pada slave.

Mengapa anggaran POLL_TIMEOUT lebih sempit dari dugaan. Intuisi naif: batas waktu 500 ms harus menampung waktu udara POLL (± 31 ms) **dan** waktu udara jawaban (± 36 – 41 ms), sehingga ambang kegagalan diperkirakan baru muncul di atas 70-an ms. Kenyataannya, transmit(pollMsg) di pollSlave() bersifat blocking — ia baru kembali setelah IRQ_TX_DONE menyala, yaitu setelah POLL selesai mengudara. Penghitung waktu (waitStart = time.monotonic()) baru dimulai **sesudah** itu. Akibatnya jendela yang sebenarnya ditunggu hanya: pemrosesan di slave + waktu udara jawaban + deteksi RX di master — jauh lebih sempit dari intuisi awal. Bagian Percobaan mengukur persis di mana ambang itu berada.

Sekuens yang diamati



7.4 Topologi



Node	Environment / program	Build flag	Peran	Batas waktu
Master	src/master.py (Raspberry Pi, lewat SSH)	—	Memanggil bergiliran, mencatat statistik	500 ms per slave
Slave 1	slave1	-DSLAVE_ID=1	Menjawab POLL : 1	—
Slave 2	slave2	-DSLAVE_ID=2	Menjawab POLL : 2	—

Tabel 7.3: Peran tiap node Modul 07

Tidak ada environment PlatformIO untuk master — Python dijalankan langsung, tidak dikompilasi. `platformio.ini` di modul ini hanya memuat kedua environment slave. Kedua slave memakai **file source yang sama**, `src/slave/main.cpp`, identik dengan Modul05_lora_master_slave/src/slave/main.cpp kecuali komentar dan satu baris pesan pembuka Serial.

7.5 Alat yang Digunakan

Modul ini menggabungkan dua platform: Raspberry Pi 5 dengan Dragino LoRa GPS HAT v1.4 sebagai master, dan dua Arduino Uno dengan Dragino LoRa Shield v1.2 sebagai slave — perangkat keras slave sama persis dengan Modul 05.

No	Peralatan	Spesifikasi	Jumlah
1	Raspberry Pi	2 / 3 / 4 / 5 — diuji pada Pi 5	1
2	Dragino LoRa GPS HAT	v1.4, SX1276, 433 MHz	1
3	Arduino Uno	ATmega328P	2
4	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
5	Antena SMA	wajib terpasang sebelum diberi daya , di ketiga board	3
6	Kabel USB tipe B	ke kedua Uno	2
7	Akses jaringan ke Raspberry Pi	SSH, kunci terpasang lebih disarankan daripada kata sandi	1

Tabel 7.4: Alat dan bahan Modul 07

PENTING

Baud slave 115200, sama seperti Modul 05. Master tidak memakai Serial Monitor sama sekali — keluarannya langsung ke terminal SSH.

Pemetaan pin HAT Raspberry Pi (sisi master)

LoRa GPS HAT	WiringPi	BCM GPIO	Padanan di shield Arduino
LoRa_NSS	GPIO6	25	D10
RESET	GPIO0	17	D9
DI00	GPIO7	4	D2
SCK / MOSI / MISO	GPIO14/12/13	11 / 10 / 9	D13 / D11 / D12

Tabel 7.5: Pemetaan pin HAT sisi master Modul 07

Kolom **BCM** adalah yang dipakai `src/master.py` (`GPIO.setmode(GPIO.BCM)`). **NSS bukan CE0** — HAT memakai GPIO 25 biasa sebagai chip select, sehingga kode membuka SPI pada (0, 0) tetapi menggerakkan NSS sendiri di sekitar tiap transaksi, persis seperti driver Arduino menggerakkan D10. Rincian lengkap pemetaan pin ada pada Tabel P.9 di bab Pendahuluan.

CATATAN

Raspberry Pi 5 memakai chip GPIO baru (RP1) yang tidak didukung RPi.GPIO. Pasang `rpi-lgpio` sebagai gantinya — nama modulnya sama (`import RPi.GPIO as GPIO`), sehingga tidak ada baris kode yang perlu diubah. Jangan memasang keduanya sekaligus. Lihat `requirements.txt`.

Struktur proyek

```
Modul07_rpi_master_slave/
├── platformio.ini          ← hanya 2 environment: slave1, slave2
├── requirements.txt        ← spidev + RPi.GPIO (atau rpi-lgpio untuk Pi 5)
├── logserial.md            ← log referensi hasil uji perangkat, sangat lengkap
├── lora_monitor.py         ← dasbor 2 slave lokal + rekaman CSV (butuh `rich`)
├── cek_radio.py            ← baca balik register SX1276 di Raspberry Pi
├── upload_auto.py          ← deteksi port otomatis saat unggah slave
└── src/
    ├── master.py           ← penjadwal round-robin, dijalankan DI Raspberry Pi
    └── slave/main.cpp      ← satu source untuk kedua slave, identik M05
```

7.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul07_rpi_master_slave/](#)

- [Modul07_rpi_master_slave/platformio.ini](#)
- [Modul07_rpi_master_slave/requirements.txt](#)
- [Modul07_rpi_master_slave/src/master.py](#)
- [Modul07_rpi_master_slave/src/slave/main.cpp](#)
- [Modul07_rpi_master_slave/cek_radio.py](#)
- [Modul07_rpi_master_slave/lora_monitor.py](#)
- [Modul07_rpi_master_slave/upload_auto.py](#)

```
; Modul 07 - hanya kedua SLAVE yang dibangun di sini.
;
; Master modul ini adalah Raspberry Pi yang menjalankan src/master.py; tidak
; ada environment PlatformIO untuknya, dan memang tidak perlu -- Python
; dijalankan langsung, tidak dikompilasi. Yang tersisa untuk PlatformIO
; hanyalah dua Arduino Uno di sisi slave.
;
;   pio run -d Modul07_rpi_master_slave -e slave1 -t upload
;   pio run -d Modul07_rpi_master_slave -e slave2 -t upload
;   python3 Modul07_rpi_master_slave/src/master.py      # di Raspberry Pi

[platformio]
default_envs = slave1

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk dua Uno asli.
; Modul ini memakai 115200 baud, sama seperti M05.
monitor_speed = 115200

; Kedua slave memakai source yang sama; nomornya ditentukan build flag.
[env:slave1]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=1
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:slave2]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=2
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1
```

Kode 7.1: platformio.ini Modul 07 — hanya environment slavel dan slave2

```
# Modul 07 - dependensi Raspberry Pi (jalankan DI Raspberry Pi, bukan di PC)
#
#   pip3 install -r Modul07_rpi_master_slave/requirements.txt
#
# spidev : akses /dev/spidev0.0 untuk bicara ke SX1276
# RPi.GPIO: kendali jalur NSS, RESET, dan pembacaan DIO0
#
# RASPBERRY PI 5. Chip GPIO baru (RP1) tidak didukung RPi.GPIO. Ganti baris
# RPi.GPIO di bawah dengan rpi-lgpio -- drop-in replacement dengan nama modul
# yang sama, sehingga `import RPi.GPIO as GPIO` di kode tetap berjalan apa
# adanya. Jangan memasang keduanya sekaligus: keduanya memakai nama paket
# yang sama dan akan saling menimpa.
spidev>=3.5
RPi.GPIO>=0.7.1
# rpi-lgpio>=0.6 ; khusus Raspberry Pi 5 -- pakai INI dan hapus RPi.GPIO di atas
```

Kode 7.2: requirements.txt Modul 07 — paket Python sisi Raspberry Pi

```
#!/usr/bin/env python3
"""Modul 07 - Master round-robin di Raspberry Pi (Dragino LoRa GPS HAT, SX1276).

Padanan Python dari Modul05_lora_master_slave/src/master/main.cpp: memanggil
Slave 1 dan Slave 2 bergiliran dengan batas waktu 500 ms per slave, lalu
mencetak statistik per node. Bedanya, kedua slave tetap Arduino Uno bershield
Dragino v1.2 -- master pindah platform, kontrak payload tidak berubah.

    POLL:<id>          master -> slave
    S<id>:DATA:<n>      slave  -> master

Driver SX1276 ditulis langsung via spidev + RPi.GPIO tanpa library LoRa, jadi
nama fungsinya sengaja dibuat sama dengan API sandeepmistry di Arduino agar
kedua sisi dapat dibandingkan baris per baris.

Pemetaan pin (Dragino LoRa GPS HAT - WiringPi -> BCM):
    LoRa_NSS -> WiringPi GPIO6 -> BCM GPIO 25
    RESET    -> WiringPi GPIO0 -> BCM GPIO 17
    DIO0      -> WiringPi GPIO7 -> BCM GPIO 4
    SCK       -> WiringPi GPIO14 -> BCM GPIO 11 (SPI0 CLK)
    MOSI      -> WiringPi GPIO12 -> BCM GPIO 10 (SPI0 MOSI)
    MISO      -> WiringPi GPIO13 -> BCM GPIO 9  (SPI0 MISO)

Parameter radio harus IDENTIK dengan kedua slave: 433 MHz, SF7, BW 125 kHz,
CR 4/5. Upload kedua slave lebih dahulu, baru jalankan master.

Prasyarat:
    sudo raspi-config          # Interface Options > SPI > Yes, lalu reboot
    pip3 install -r Modul07_rpi_master_slave/requirements.txt
"""

import argparse
import time
import sys
import spidev
import RPi.GPIO as GPIO

# --- Pin configuration (BCM numbering) ---
NSS_PIN = 25 # LoRa_NSS (WiringPi GPIO6 = BCM GPIO 25)
RST_PIN = 17 # RESET    (WiringPi GPIO0 = BCM GPIO 17)
DIO0_PIN = 4 # DIO0     (WiringPi GPIO7 = BCM GPIO 4)

# --- LoRa parameters ---
FREQUENCY = 433E6
BANDWIDTH = 125E3
SPREADING_FACTOR = 7
CODING_RATE = 5
TX_POWER = 17

POLL_TIMEOUT = 500 # ms menunggu balasan tiap slave
CYCLE_INTERVAL = 500 # ms jeda antar siklus
SLAVE_COUNT = 2

# --- SX1276 Register Map ---
REG_FIFO = 0x00
REG_OP_MODE = 0x01
```

```

REG_FRF_MSB           = 0x06
REG_FRF_MID           = 0x07
REG_FRF_LSB           = 0x08
REG_PA_CONFIG         = 0x09
REG_FIFO_ADDR_PTR     = 0x0D
REG_FIFO_TX_BASE_ADDR = 0x0E
REG_FIFO_RX_BASE_ADDR = 0x0F
REG_FIFO_RX_CURRENT_ADDR = 0x10
REG_IRQ_FLAGS         = 0x12
REG_RX_NB_BYTES       = 0x13
REG_PKT_RSSI_VALUE    = 0x1A
REG_PKT_SNR_VALUE     = 0x1B
REG_MODEM_CONFIG_1    = 0x1D
REG_MODEM_CONFIG_2    = 0x1E
REG_PREAMBLE_MSB      = 0x20
REG_PREAMBLE_LSB      = 0x21
REG_PAYLOAD_LENGTH    = 0x22
REG_VERSION           = 0x42

# --- Operating modes ---
MODE_LONG_RANGE       = 0x80
MODE_SLEEP            = 0x00
MODE_STDBY            = 0x01
MODE_TX               = 0x03
MODE_RX_CONTINUOUS    = 0x05

# --- IRQ flags ---
IRQ_RX_DONE           = 0x40
IRQ_TX_DONE           = 0x08
IRQ_CRC_ERROR         = 0x20

# --- PA config ---
PA_BOOST = 0x80

# --- SPI instance ---
_spi = spidev.SpiDev()

# =====
# Low-level SPI register access
# =====
def _read_reg(addr):
    GPIO.output(NSS_PIN, GPIO.LOW)
    val = _spi.xfer2([addr & 0x7F, 0x00])[1]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return val

def _write_reg(addr, value):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([addr | 0x80, value & 0xFF])
    GPIO.output(NSS_PIN, GPIO.HIGH)

# =====
# LoRa mode & frequency
# =====
def _set_mode(mode):
    _write_reg(REG_OP_MODE, MODE_LONG_RANGE | mode)

def _set_frequency(freq):
    frf = int(round(freq / 32e6 * (1 << 19)))
    _write_reg(REG_FRF_MSB, (frf >> 16) & 0xFF)
    _write_reg(REG_FRF_MID, (frf >> 8) & 0xFF)
    _write_reg(REG_FRF_LSB, frf & 0xFF)

# =====
# LoRa configuration (mirip API sandeepmistry)
# =====
def setSpreadingFactor(sf):
    sf = max(6, min(12, sf))
    _write_reg(REG_MODEM_CONFIG_2,
        (_read_reg(REG_MODEM_CONFIG_2) & 0x0F) | ((sf << 4) & 0xF0))

def setSignalBandwidth(bw):
    bw_table = [7.8e3, 10.4e3, 15.6e3, 20.8e3, 31.25e3, 41.7e3, 62.5e3, 125e3, 250e3]
    idx = min(range(len(bw_table)), key=lambda i: abs(bw_table[i] - bw))
    _write_reg(REG_MODEM_CONFIG_1,
        (_read_reg(REG_MODEM_CONFIG_1) & 0x0F) | (idx << 4))

```



```

def setCodingRate4(denominator):
    cr = max(5, min(8, denominator)) - 4
    _write_reg(REG_MODEM_CONFIG_1,
               (_read_reg(REG_MODEM_CONFIG_1) & 0xF1) | (cr << 1))

def setTxPower(level):
    level = max(2, min(17, level))
    _write_reg(REG_PA_CONFIG, PA_BOOST | (level - 2))

# =====
# TX functions (mirip LoRa.beginPacket / print / endPacket)
# =====
def beginPacket():
    _set_mode(MODE_STDBY)
    _write_reg(REG_MODEM_CONFIG_1, _read_reg(REG_MODEM_CONFIG_1) & 0xFE)
    _write_reg(REG_FIFO_ADDR_PTR, 0x00)
    _write_reg(REG_PAYLOAD_LENGTH, 0x00)

def loraPrint(data):
    if isinstance(data, str):
        data = data.encode('ascii')
    current_len = _read_reg(REG_PAYLOAD_LENGTH)
    size = min(len(data), 255 - current_len)
    for byte in data[:size]:
        _write_reg(REG_FIFO, byte)
    _write_reg(REG_PAYLOAD_LENGTH, current_len + size)

def endPacket():
    _set_mode(MODE_TX)
    while (_read_reg(REG_IRQ_FLAGS) & IRQ_TX_DONE) == 0:
        time.sleep(0.001)
    _write_reg(REG_IRQ_FLAGS, IRQ_TX_DONE)

def transmit(msg):
    beginPacket()
    loraPrint(msg)
    endPacket()

# =====
# RX functions (mirip LoRa.parsePacket / available / read)
# =====
def parsePacket():
    irq = _read_reg(REG_IRQ_FLAGS)
    if irq & IRQ_CRC_ERROR:
        _write_reg(REG_IRQ_FLAGS, IRQ_CRC_ERROR)
        return 0
    if (irq & IRQ_RX_DONE) == 0:
        return 0
    _write_reg(REG_IRQ_FLAGS, IRQ_RX_DONE)
    return _read_reg(REG_RX_NB_BYTES)

def loraAvailable():
    return _read_reg(REG_RX_NB_BYTES)

def loraRead():
    return _read_reg(REG_FIFO)

def packetRssi():
    rssi = _read_reg(REG_PKT_RSSI_VALUE)
    freq_mhz = FREQUENCY / 1e6
    if freq_mhz < 868:
        return rssi - 164
    else:
        return rssi - 157

def packetSnr():
    val = _read_reg(REG_PKT_SNR_VALUE)
    if val > 127:
        val = val - 256
    return val / 4.0

```

```

def startRx():
    _set_mode(MODE_RX_CONTINUOUS)

def initRxSequence():
    _write_reg(REG_FIFO_ADDR_PTR, _read_reg(REG_FIFO_RX_CURRENT_ADDR))

# =====
# Init & cleanup
# =====
def loraBegin(frequency):
    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(NSS_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(RST_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(DI00_PIN, GPIO.IN)

    _spi.open(0, 0)
    _spi.max_speed_hz = 5_000_000
    _spi.mode = 0b00

    GPIO.output(RST_PIN, GPIO.LOW)
    time.sleep(0.01)
    GPIO.output(RST_PIN, GPIO.HIGH)
    time.sleep(0.01)

    version = _read_reg(REG_VERSION)
    if version != 0x12:
        return False

    _set_mode(MODE_SLEEP)
    _set_frequency(frequency)
    _write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
    _write_reg(REG_FIFO_RX_BASE_ADDR, 0x00)
    _set_mode(MODE_STDBY)
    return True

def loraEnd():
    _set_mode(MODE_SLEEP)
    _spi.close()
    GPIO.cleanup()

# =====
# Master logic: poll slave (identik dengan pollSlave() di master.ino)
# =====
def pollSlave(slaveNum, okRef, failRef):
    pollMsg = f"POLL:{slaveNum}"
    print(f"[TX] {pollMsg}")
    transmit(pollMsg)

    startRx()

    waitStart = time.monotonic()
    while (time.monotonic() - waitStart) * 1000 < POLL_TIMEOUT:
        ps = parsePacket()
        if ps > 0:
            initRxSequence()
            reply = bytes(loraRead() for _ in range(ps)).decode('ascii', errors='replace')

            prefix = f"S{slaveNum}:DATA:"
            if reply.startswith(prefix):
                try:
                    dataOut = int(reply[len(prefix):])
                except ValueError:
                    dataOut = 0
                okRef[0] += 1
                rssi = packetRssi()
                snr = packetSnr()
                print(f"[RX] {reply} | RSSI: {rssi:.0f} dBm | SNR: {snr:.1f} dB")
                return dataOut
            else:
                print(f"[WARN] Balasan tidak valid: {reply}")

        failRef[0] += 1
    print(f"[FAIL] Slave {slaveNum} tidak merespon!")
    return None

```

```

# =====
# Main (identik dengan setup() + loop() di master.ino)
# =====
def build_arg_parser():
    parser = argparse.ArgumentParser(
        prog="master.py",
        description=(
            "Master round-robin M07 (Raspberry Pi + Dragino LoRa GPS HAT). "
            "Memanggil Slave 1 lalu Slave 2 bergiliran tanpa henti, mencetak "
            "statistik OK/FAIL per node setiap siklus. Tidak ada opsi lain -- "
            "seluruh parameter (frekuensi, SF, BW, timeout) ditetapkan sebagai "
            "konstanta di berkas ini agar tetap identik dengan slave M05."
        ),
        epilog=(
            f"Parameter radio: {FREQUENCY / 1e6:.0f} MHz, SF{SPREADING_FACTOR}, "
            f"BW {BANDWIDTH / 1e3:.0f} kHz, CR 4/5, {TX_POWER} dBm. "
            f"POLL_TIMEOUT={POLL_TIMEOUT} ms, CYCLE_INTERVAL={CYCLE_INTERVAL} ms. "
            "Hentikan dengan Ctrl-C."
        ),
        formatter_class=argparse.RawDescriptionHelpFormatter,
    )
    return parser

if __name__ == '__main__':
    build_arg_parser().parse_args()

    cycle = 0
    s1ok = [0]; s1Fail = [0]; s1Data = 0
    s2ok = [0]; s2Fail = [0]; s2Data = 0

    print("=== LoRa MASTER-SLAVE 3 NODE ===")
    print("Init LoRa ... ", end="", flush=True)

    if not loraBegin(FREQUENCY):
        print("GAGAL! Cek kabel/modul.")
        print("Pastikan SPI aktif: sudo raspi-config > Interfacing Options > SPI")
        sys.exit(1)

    setSpreadingFactor(SPREADING_FACTOR)
    setSignalBandwidth(BANDWIDTH)
    setCodingRate4(CODING_RATE)
    setTxPower(TX_POWER)

    print("OK")
    print(f"Freq: {FREQUENCY / 1e6:.0f} MHz")
    print(f"SF{SPREADING_FACTOR} | BW: {BANDWIDTH / 1e3:.0f} kHz")
    print("Peran: MASTER (Raspberry Pi + LoRa GPS HAT)")
    print("Slave: Dragino Shield Uno - S1 & S2\n")

    try:
        while True:
            cycle += 1
            cycleStartMs = time.monotonic()

            print("=" * 40)
            print(f"=== CYCLE {cycle} ===")

            # --- Poll Slave 1 ---
            s1Data = pollSlave(1, s1ok, s1Fail)

            # --- Poll Slave 2 ---
            s2Data = pollSlave(2, s2ok, s2Fail)

            # --- Statistik ---
            duration = (time.monotonic() - cycleStartMs) * 1000
            print("--- STATISTIK ---")
            print(f"S1: OK={s1ok[0]} | FAIL={s1Fail[0]} | Data: {s1Data}")
            print(f"S2: OK={s2ok[0]} | FAIL={s2Fail[0]} | Data: {s2Data}")
            print(f"Durasi siklus: {duration:.0f} ms")
            print("=" * 40 + "\n")

            time.sleep(CYCLE_INTERVAL / 1000.0)

    except KeyboardInterrupt:
        print("\nProgram dihentikan oleh pengguna.")
    finally:
        loraEnd()
        print("Menutup LoRa module...")

```

Kode 7.3: src/master.py — penjadwal round-robin di atas driver register SX1276

```

/*
LoRa Master-Slave hybrid - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: slave1 / slave2

Sisi slave Modul 07. Identik dengan slave Modul 05 -- itulah intinya: yang
berganti platform hanya master (Arduino Uno -> Raspberry Pi + Python), dan
firmware slave tidak perlu diubah satu baris pun. Kontrak payload yang sama
yang membuatnya mungkin.

Slave N hanya merespon ketika Master mengirim "POLL:N".
Balasan: "SN:DATA:n" - n adalah counter lokal yang naik setiap kali menjawab.

Mekanisme:
TX : blocking (endPacket) - sederhana dan andal di AVR
RX : polling via LoRa.parsePacket() - tidak perlu ISR

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor slave datang dari build flag (-DSLAVE_ID=1 atau 2) di platformio.ini,
// sehingga kedua slave memakai file source yang sama persis.
#ifndef SLAVE_ID
#define SLAVE_ID 1
#endif

unsigned long dataCounter = 0; // unsigned long: aman hingga 4.294.967.295 (vs int yang overflow di 32.767)
int rxCount = 0;

void transmit(const String& msg) {
  LoRa.beginPacket();
  LoRa.print(msg);
  LoRa.endPacket();
}

void setup() {
  Serial.begin(115200);
  while (!Serial);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  Serial.print(F("=== LoRa SLAVE "));
  Serial.print(SLAVE_ID);
  Serial.println(F(" =="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
  }

  LoRa.setSpreadingFactor(SPREADING_FACTOR);
  LoRa.setSignalBandwidth(BANDWIDTH);
  LoRa.setCodingRate4(CODING_RATE);
  LoRa.setTxPower(TX_POWER);

  Serial.println(F("OK"));
  Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
  Serial.print(F("Menunggu POLL:"));
  Serial.print(SLAVE_ID);
  Serial.println(F(" dari Master (Raspberry Pi)...\n"));
}

```

```

void loop() {
    int ps = LoRa.parsePacket();
    if (ps == 0) return;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    if (!received.equals("POLL:" + String(SLAVE_ID))) {
        Serial.print(F("[IGNORE ]")); Serial.println(received);
        return;
    }

    rxCount++;
    // dataCounter++ sengaja dipindah ke SETELAH transmit()
    // Filosofi: counter merepresentasikan "berapa kali slave BERHASIL kirim ke master"
    // bukan "berapa kali slave berniat kirim" – kalau TX gagal, counter tidak berhak naik
    digitalWrite(LED_PIN, HIGH);

    Serial.print(F("[RX] ")); Serial.print(received);
    Serial.print(F(" | RSSI: ")); Serial.print(LoRa.packetRssi());
    Serial.print(F(" dBm | SNR: ")); Serial.print(LoRa.packetSnr());
    Serial.print(F(" dB | RX#: ")); Serial.println(rxCount);

    dataCounter++; // naik SETELAH TX – nilai ini yang dikirim, mencerminkan jumlah TX aktual
    String reply = "S" + String(SLAVE_ID) + ":DATA:" + String(dataCounter);
    Serial.print(F("[TX] ")); Serial.println(reply);

    transmit(reply);
    digitalWrite(LED_PIN, LOW);
    Serial.println();
}

```

Kode 7.4: src/slave/main.cpp — firmware slave, identik dengan Modul 05

```

"""Baca BALIK register SX1276 di master setelah loraBegin()+konfigurasi.
Yang dilaporkan adalah isi chip, bukan konstanta di source."""
import sys
sys.path.insert(0, 'src')
import master as M

assert M.loraBegin(M.FREQUENCY), "loraBegin GAGAL"
M.setSpreadingFactor(M.SPREADING_FACTOR)
M.setSignalBandwidth(M.BANDWIDTH)
M.setCodingRate4(M.CODING_RATE)
M.setTxPower(M.TX_POWER)

msb = M._read_reg(M.REG_FRF_MSB); mid = M._read_reg(M.REG_FRF_MID); lsb = M._read_reg(M.REG_FRF_LSB)
frf = (msb << 16) | (mid << 8) | lsb
freq = frf * 32e6 / (1 << 19)
print(f"REG_VERSION : 0x{M._read_reg(M.REG_VERSION):02x} (0x12 = SX1276/77/78/79)")
print(f"FRF register : 0x{msb:02x} 0x{mid:02x} 0x{lsb:02x} -> frf={frf}")
print(f"FREKUENSI : {freq/1e6:.6f} MHz (target 433.000000, selisih {freq-433e6:+.1f} Hz)")

mc1 = M._read_reg(M.REG_MODEM_CONFIG_1); mc2 = M._read_reg(M.REG_MODEM_CONFIG_2)
bw_tab = ["7.8", "10.4", "15.6", "20.8", "31.25", "41.7", "62.5", "125", "250", "500"]
print(f"MODEM_CFG_1 : 0x{mc1:02x} -> BW={bw_tab[mc1>>4]} kHz | CR=4/{{(mc1>>1)&0x07}+4} |
header={'implicit' if mc1&1 else 'explicit'}")
print(f"MODEM_CFG_2 : 0x{mc2:02x} -> SF{mc2>>4} | CRC payload={'AKTIF' if mc2&0x04 else 'MATI'}")
print(f"PA_CONFIG : 0x{M._read_reg(M.REG_PA_CONFIG):02x} -> PA_B00ST,
power={M._read_reg(M.REG_PA_CONFIG)&0x0F)+2} dBm")
M.loraEnd()

```

Kode 7.5: cek_radio.py — baca balik register SX1276 di Raspberry Pi

```

#!/usr/bin/env python3
"""
LoRa Monitor Dashboard - Sisi Slave (Modul 07)
=====
Dashboard live untuk memantau KEDUA Arduino Uno slave dari komputer ini.

Bedanya dengan lora_monitor.py pada M05: di modul ini master adalah Raspberry

```

Pi, dan Pi tidak menyalurkan keluarannya lewat USB ke komputer ini. Jadi tidak ada panel MASTER yang bisa dibaca langsung. Yang tersedia hanya dua port serial slave -- dan ternyata itu sudah cukup untuk menyimpulkan hampir seluruh keadaan master, karena setiap slave MENDENGAR seluruh lalu lintas di udara:

```
[RX] POLL:1      <- perintah master untuk dirinya
[IGNORE] POLL:2  <- perintah master untuk slave sebelah
[IGNORE] S2:DATA:12 <- jawaban slave sebelah
```

Dari tiga jenis baris itu, monitor menurunkan:

- periode siklus master, diukur dari jarak antar POLL yang diterima node (padanan "Durasi siklus" di master, dilihat dari tepi jaringan)
- keadaan slave sebelah, dari ada/tidaknya jawaban setelah POLL-nya terdengar
- RSSI/SNR arah master -> slave, yang TIDAK terlihat di terminal master (master hanya melaporkan arah sebaliknya, slave -> master)

Kebutuhan:

```
pip install pyserial rich
```

Cara pakai:

```
python3 lora_monitor.py                # port bawaan
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
python3 lora_monitor.py --out data_eksperimen.csv
```

Jalankan tools/deteksi_port.py untuk mengetahui port mana milik board yang mana.

```
"""
import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.table import Table
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# -----
# KONFIGURASI
# -----
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py lalu berikan lewat
# --s1/--s2 bila port di komputer Anda berbeda.
if sys.platform.startswith("win"):
    DEFAULT_S1_PORT = "COM4"
    DEFAULT_S2_PORT = "COM5"
else:
    DEFAULT_S1_PORT = "/dev/ttyACM0"
    DEFAULT_S2_PORT = "/dev/ttyACM1"
DEFAULT_BAUD = 115200 # M07 memakai 115200, sama seperti M05
LOG_LINES = 16
REFRESH_HZ = 4
PERIODE_WINDOW = 60 # jumlah periode terakhir untuk min/maks/rerata
MISS_WARN = 2 # lompatan RX# yang dianggap POLL hilang
PASANGAN_WARN = 3 # siklus berturut tanpa jawaban slave sebelah

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    """
```

```

sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
Dipanggil tiap program start supaya default s1/s2 selalu mengikuti apa
yang benar-benar tersambung, bukan port tetap di atas.
"""
if sys.platform.startswith("win"):
    return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

# Bentuk payload yang sah menurut kontrak M05/M07. Apa pun di luar dua pola ini
# adalah byte yang rusak di udara -- dan modul ini TIDAK mengaktifkan CRC di
# kedua sisi, jadi paket cacat memang naik sampai ke lapisan aplikasi alih-alih
# dibuang oleh radio. Monitor menandainya supaya terlihat, bukan tenggelam.
RE_POLL_SAH = re.compile(r'^POLL:\d+$')
RE_DATA_SAH = re.compile(r'^S\d+:DATA:\d+$')

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class SlaveState:
    name: str = ""
    port: str = ""
    slave_id: int = 0
    connected: bool = False
    rx_count: int = 0 # RX# dari firmware: POLL untuk dirinya
    data_counter: int = 0 # counter yang dikirim balik ke master
    last_rssi: int = 0
    last_snr: float = 0.0
    rssi_min: int = 0
    rssi_max: int = 0
    ignore_poll: int = 0 # POLL milik slave sebelah yang ikut terdengar
    ignore_reply: int = 0 # jawaban slave sebelah yang ikut terdengar
    rusak: int = 0 # payload yang tidak memenuhi kontrak
    miss: int = 0 # POLL untuk dirinya yang tidak sampai
    # periode antar POLL untuk dirinya = periode siklus master dari sisi node
    periode: deque = field(default_factory=lambda: deque(maxlen=PERIODE_WINDOW))
    _t_poll: float = 0.0
    # menunggu jawaban slave sebelah setelah POLL-nya terdengar
    _nunggu_pasangan: bool = False
    pasangan_sepi: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=200))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.s1 = SlaveState("SLAVE 1", DEFAULT_S1_PORT, 1)
        self.s2 = SlaveState("SLAVE 2", DEFAULT_S2_PORT, 2)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSER
# -----

def parse_slave_line(line: str, st: SlaveState, anomalies: deque) -> dict:
    """Perbarui state dari satu baris serial, kembalikan baris CSV terstruktur."""
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    now = time.monotonic()
    row = {'event': 'LAIN', 'rssi': '', 'snr': '', 'rx': '', 'data': '',
           'periode_ms': '', 'detail': ''}

    # --- POLL untuk dirinya sendiri ---
    m = re.search(r'([RX])\s+POLL:(\d+)\s*\|s*RSSI:\s*(-?\d+)\s*dBm\s*\|s*'
                  r'SNR:\s*(-?\d+)\s*dB\s*\|s*RX#:\s*(\d+)', line)
    if m:
        rssi, snr, rx = int(m.group(2)), float(m.group(3)), int(m.group(4))

        # Jarak antar POLL = satu putaran penuh master. Diukur di sini, bukan

```

```

# di master, sehingga sebaran yang terlihat sudah termasuk jitter
# penjadwalan Linux + waktu udara -- persis yang ditanyakan EXP-02.
if st._t_poll:
    periode = (now - st._t_poll) * 1000.0
    st.periode.append(periode)
    row['periode_ms'] = f"{periode:.0f}"
    st._t_poll = now

if st.rx_count and rx > st.rx_count + 1:
    hilang = rx - st.rx_count - 1
    st.miss += hilang
    if hilang >= MISS_WARN:
        anomalies.append(Anomaly(ts, st.name, "POLL_HILANG",
            f"RX# loncat {st.rx_count} -> {rx} ({hilang} POLL tidak sampai)"))

st.rx_count = rx
st.last_rssi = rssi
st.last_snr = snr
st.rssi_min = rssi if not st.rssi_min else min(st.rssi_min, rssi)
st.rssi_max = rssi if not st.rssi_max else max(st.rssi_max, rssi)
row.update(event='RX_POLL', rssi=rssi, snr=snr, rx=rx)
return row

# — jawaban yang dikirim ke master —————
m = re.search(r'\\[TX]\\s+S(\\d+):DATA:(\\d+)', line)
if m:
    st.data_counter = int(m.group(2))
    row.update(event='TX_DATA', data=st.data_counter)
    return row

# — lalu lintas milik node lain yang ikut terdengar —————
m = re.search(r'\\[IGNORE]\\s+(.*)$', line)
if m:
    isi = m.group(1).strip()

    if RE_POLL_SAH.match(isi):
        st.ignore_poll += 1
        # master sedang memanggil slave sebelah; jawabannya ditunggu
        st._nunggu_pasangan = True
        row.update(event='IGNORE_POLL', detail=isi)
        return row

    if RE_DATA_SAH.match(isi):
        st.ignore_reply += 1
        if st._nunggu_pasangan:
            st._nunggu_pasangan = False
            st.pasangan_sepi = 0
        row.update(event='IGNORE_REPLY', detail=isi)
        return row

    # Tidak cocok kontrak: byte rusak di udara. Inilah yang di terminal
    # master muncul sebagai "[WARN] Balasan tidak valid" lalu dihitung
    # sebagai FAIL, seolah slave tidak menjawab.
    st.rusak += 1
    anomalies.append(Anomaly(ts, st.name, "PAYLOAD_RUSAK",
        f"payload di luar kontrak: {isi!r}"))
    row.update(event='RUSAK', detail=isi)
    return row

if 'Init LoRa' in line or '=== LoRa SLAVE' in line:
    row.update(event='INIT', detail=line.strip())
    return row

def tutup_siklus(st: SlaveState, anomalies: deque):
    """Dipanggil saat POLL untuk diri sendiri tiba: nilai siklus sebelumnya.

    Bila POLL slave sebelah sempat terdengar tetapi jawabannya tidak pernah
    muncul, slave sebelah sedang mati atau di luar jangkauan -- keadaan EXP-03.
    """
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    if st._nunggu_pasangan:
        st._nunggu_pasangan = False
        st.pasangan_sepi += 1
        if st.pasangan_sepi == PASANGAN_WARN:
            lawan = 2 if st.slave_id == 1 else 1
            anomalies.append(Anomaly(ts, st.name, "PASANGAN_HILANG",
                f"POLL:{lawan} terdengar {PASANGAN_WARN}x tanpa jawaban - "
                f"slave {lawan} mati atau di luar jangkauan"))

```

#


```

# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str, shared: SharedState):
    state = shared.s1 if node_key == "S1" else shared.s2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                # siklus dinilai tepat sebelum POLL berikutnya diproses
                                if '[RX]' in line and 'POLL:' in line:
                                    tutup_siklus(state, shared.anomalies)
                                    row = parse_slave_line(line, state, shared.anomalies)
                                    row.update(ts=ts, node=node_key, raw=line)
                                    shared.csv_queue.put(row)
                                except (serial.SerialException, OSError):
                                    break
                        except (serial.SerialException, OSError):
                            with shared.lock:
                                state.connected = False
                            time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    kolom = ['timestamp', 'node', 'event', 'rssi_dbm', 'snr_db',
             'rx_count', 'data', 'periode_ms', 'detail', 'raw_line']
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=kolom)
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                r = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': r['ts'], 'node': r['node'],
                           'event': r['event'], 'rssi_dbm': r['rssi'],
                           'snr_db': r['snr'], 'rx_count': r['rx'],
                           'data': r['data'], 'periode_ms': r['periode_ms'],
                           'detail': r['detail'], 'raw_line': r['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def log_line_style(line: str) -> str:
    if '[RX]' in line: return "green"
    if '[TX]' in line: return "yellow"
    if '[IGNORE]' in line: return "dim"
    if 'GAGAL' in line: return "bold red"
    if '===' in line: return "bold cyan"
    return "dim white"

def build_slave_panel(st: SlaveState, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")
    t.append(" POLL diterima : ", style="dim")
    t.append(f"{st.rx_count:>7}\n", style="bold white")

```

```

t.append(" Jawaban dikirim: ", style="dim")
t.append(f"{st.data_counter:>7}\n", style="bold green")
t.append(" POLL hilang : ", style="dim")
t.append(f"{st.miss:>7}\n", style="bold red" if st.miss else "dim")
t.append(" Payload rusak : ", style="dim")
t.append(f"{st.rusak:>7}\n\n", style="bold red" if st.rusak else "dim")

# Arah master -> slave. Terminal master hanya melaporkan arah sebaliknya,
# jadi angka ini tidak ada duanya di tempat lain.
t.append(" Master -> node\n", style="bold cyan")
t.append(" RSSI : ", style="dim")
t.append(f"{st.last_rssi:>6} dBm\n", style="bold white")
t.append(" rentang: ", style="dim")
t.append(f"{st.rssi_min}..{st.rssi_max} dBm\n" if st.rssi_min else "-\n",
        style="white")
t.append(" SNR : ", style="dim")
t.append(f"{st.last_snr:>6.2f} dB\n", style="bold white")

t.append(" Periode siklus\n", style="bold cyan")
if st.periode:
    p = list(st.periode)
    rata = sum(p) / len(p)
    t.append(" min/rata/maks: ", style="dim")
    t.append(f"{min(p):.0f} / {rata:.0f} / {max(p):.0f} ms\n", style="bold white")
    t.append(" sebaran : ", style="dim")
    t.append(f"{max(p) - min(p):.0f} ms\n\n", style="bold yellow")
else:
    t.append(" menunggu POLL kedua...\n\n", style="dim")

t.append(" Terdengar dari node lain\n", style="bold cyan")
t.append(" POLL / jawaban: ", style="dim")
t.append(f"{st.ignore_poll} / {st.ignore_reply}\n\n", style="white")

t.append(" Raw Log\n", style="bold dim")
t.append(" " + "-" * 34 + "\n", style="dim")
for ts, line in list(st.raw_log)[-LOG_LINES:]:
    t.append(f" {ts[3:]} {line[:40]}\n", style=log_line_style(line))

return Panel(t, title=f"[bold {color}]{st.name}[/]",
            border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                        csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'POLL_HILANG': "bold yellow",
        'PAYLOAD_RUSAK': "bold white on red",
        'PASANGAN_HILANG': "bold red",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=kind_style.get(a.kind, "white"))
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        # snapshot biar tidak menahan lock saat render
        snap = []
        for src in (shared.s1, shared.s2):

```

```

        snap.append(SlaveState(
            name=src.name, port=src.port, slave_id=src.slave_id,
            connected=src.connected, rx_count=src.rx_count,
            data_counter=src.data_counter, last_rssi=src.last_rssi,
            last_snr=src.last_snr, rssi_min=src.rssi_min,
            rssi_max=src.rssi_max, ignore_poll=src.ignore_poll,
            ignore_reply=src.ignore_reply, rusak=src.rusak, miss=src.miss,
            periode=deque(src.periode), raw_log=deque(src.raw_log),
        ))
    anom = deque(shared.anomalies)
    t_start = shared.session_start

    # Grid dua kolom berbagi rata: dengan hanya dua panel, Columns membiarkan
    # keduanya selebar isinya saja dan separuh terminal terbuang.
    node_row = Table.grid(expand=True)
    node_row.add_column(ratio=1)
    node_row.add_column(ratio=1)
    node_row.add_row(build_slave_panel(snap[0], "green"),
                     build_slave_panel(snap[1], "magenta"))

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\
    LoRa Monitor Dashboard - Sisi Slave (Modul 07)
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz
"""

Master modul ini adalah Raspberry Pi, jadi terminalnya tidak lewat USB
ke komputer ini. Monitor membaca kedua slave saja - dan menyimpulkan
keadaan master dari lalu lintas yang ikut terdengar oleh node.

PENGGUNAAN:
    python3 lora_monitor.py [opsi...]

OPSI:
--s1 PORT Port serial Slave 1 (default: {s1})
--s2 PORT Port serial Slave 2 (default: {s2})
--baud N Baud rate serial kedua port (default: {baud})
--out FILE Nama file CSV output (default: lora_slaves_YYYYMMDD_HHMMSS.csv)
-h / --help Tampilkan pesan bantuan ini

CONTOH:
python3 lora_monitor.py
    -> Jalankan dengan port & baud rate default

python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
    -> Tentukan port secara eksplisit (Windows: COM4, COM5)

python3 lora_monitor.py --out data_jarak_10m.csv
    -> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
    Ctrl+C Hentikan monitoring & simpan CSV

CATATAN:
    • Kedua slave harus sudah di-upload firmware dengan baud {baud}
    • Jalankan tools/deteksi_port.py bila ragu port mana milik board mana
    • Tutup Serial Monitor PlatformIO dulu - satu port hanya boleh dibuka
      oleh satu program
    • Master di Raspberry Pi dijalankan terpisah:
      python3 Modul07_rpi_master_slave/src/master.py
"""
.format(s1=DEFAULT_S1_PORT, s2=DEFAULT_S2_PORT, baud=DEFAULT_BAUD)

def parse_args(port_aktif):
    s1_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_S1_PORT
    s2_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_S2_PORT

    p = argparse.ArgumentParser(
        description="LoRa Monitor Dashboard sisi slave - tool pemantauan praktikum M07.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            "    python3 lora_monitor.py\n"
            "    python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1\n"
            "    python3 lora_monitor.py --out data_eksperimen.csv\n"
        )
    )

```

```

    ),
)
p.add_argument('--s1', default=s1_default, metavar='PORT',
               help=f'Port serial Slave 1 (default: {s1_default}, dari deteksi port aktif)')
p.add_argument('--s2', default=s2_default, metavar='PORT',
               help=f'Port serial Slave 2 (default: {s2_default}, dari deteksi port aktif)')
p.add_argument('--baud', default=DEFAULT_BAUD, type=int, metavar='N',
               help=f'Baud rate serial (default: {DEFAULT_BAUD})')
p.add_argument('--out', default=None, metavar='FILE',
               help='Nama file CSV output (default: auto timestamp)')
return p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1
    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f'[dim]Port aktif terdeteksi: {', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]'
    )
    console.print(
        f'[dim]Dipakai -> "
        f"S1=[green]{args.s1}[/] S2=[magenta]{args.s2}[/] "
        f"Baud=[white]{args.baud}[/][dim]\n"
    )

    shared = SharedState()
    shared.s1.port = args.s1
    shared.s2.port = args.s2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_slaves_{ts_str}.csv"

    if not no_args:
        console.print(f"\n[bold cyan]LoRa Monitor Dashboard - Sisi Slave[/]")
        console.print(f" Slave1 : [green]{args.s1}[/]")
        console.print(f" Slave2 : [magenta]{args.s2}[/]")
        console.print(f" Baud : [white]{args.baud}[/]")

    console.print(f" CSV : [dim]{csv_path}[/]")
    console.print(f"\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:
        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread, args=(args.s1, args.baud, "S1", shared),
                        daemon=True, name="reader-s1"),
        threading.Thread(target=reader_thread, args=(args.s2, args.baud, "S2", shared),
                        daemon=True, name="reader-s2"),
        threading.Thread(target=csv_writer_thread, args=(shared, csv_path),
                        daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path), console=console,
                  refresh_per_second=REFRESH_HZ, screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass
    finally:
        shared.running = False
        time.sleep(0.6)
        console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 7.6: lora_monitor.py — dasbor dua slave lokal dengan perekaman CSV

```
#!/usr/bin/env python3
"""Upload otomatis ke Modul 07 (RPI Master-Slave) – port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul07_rpi_master_slave -e slave1 -t upload -t monitor
    pio run -d Modul07_rpi_master_slave -e slave2 -t upload -t monitor

Master modul ini adalah Raspberry Pi yang menjalankan src/master.py secara
langsung (Python, tidak dikompilasi) -- TIDAK ADA environment PlatformIO
untuknya, jadi skrip ini hanya meng-upload kedua slave.

Skrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `slave1` (port pertama) dan `slave2` (port kedua), lalu
menjalankan `pio run` dengan --upload-port sesuai hasil deteksi --
platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua
slave selesai di-upload, skrip bertanya apakah mau langsung menjalankan
lora_monitor.py -- bila ya, dijalankan dengan --s1 ... --s2 ... persis
memakai port yang barusan dipakai upload (bukan deteksi ulang), supaya
tidak mungkin salah pasang. Master tetap harus dijalankan terpisah di
Raspberry Pi: python3 Modul07_rpi_master_slave/src/master.py

python3 Modul07_rpi_master_slave/upload_auto.py
-> upload kedua slave, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul07_rpi_master_slave/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk slave1, lanjut slave2
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti lora_monitor.py

python3 Modul07_rpi_master_slave/upload_auto.py --only slave1
-> upload satu environment saja

python3 Modul07_rpi_master_slave/upload_auto.py --slave1 /dev/ttyACM0 --slave2 /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["slave1", "slave2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"slave1": "--s1", "slave2": "--s2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print("  $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port slave1/slave2 PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash. Master tetap harus dijalankan terpisah di Raspberry Pi.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
```

```

        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
    for env, opsi in ENV_KE_OPSI.items():
        if env in port_env:
            cmd += [opsi, port_env[env]]

    print(" $", " ".join(cmd))
    print(" (ingat: master tetap dijalankan terpisah di Raspberry Pi -- "
          "python3 src/master.py)")
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--slave1", metavar="/dev/ttyXXX",
                    help="tampa port slave1 (lewati deteksi otomatis)")
    ap.add_argument("--slave2", metavar="/dev/ttyXXX",
                    help="tampa port slave2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (slave1 atau slave2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                          "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau keduanya sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"slave1": args.slave1, "slave2": args.slave2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"-- sambungkan board atau tentukan lewat --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 7.7: upload_auto.py — pemilih port otomatis saat unggah slave

7.7 Build, Flash, dan Menjalankan

Build dan flash slave — dari laptop pengembang, seperti modul Arduino lain.

```
pio run -d Modul07_rpi_master_slave -e slave1 -t upload
pio run -d Modul07_rpi_master_slave -e slave2 -t upload
```

Menyiapkan Raspberry Pi — sekali per Pi.

```
ssh pi@<alamat-ip-pi>
sudo raspi-config          # Interface Options > SPI > Yes, lalu reboot
ls /dev/spi*              # harus muncul spidev0.0

pip3 install -r Modul07_rpi_master_slave/requirements.txt
```

Menjalankan master — kedua slave lebih dahulu, baru master, dan master selalu di Raspberry Pi.

```
ssh pi@<alamat-ip-pi>
cd ~/Documents/WSN-IOT-prak-Lora/Modul07_rpi_master_slave/src
python3 -u master.py      # -u penting bila keluarannya dipipa/direkam
```

master.py menerima -h atau --help yang mencetak parameter radio tanpa menyentuh SPI dan GPIO — aman dijalankan untuk memeriksa konfigurasi sebelum sesi sungguhan. Tidak ada opsi lain; seluruh parameter (frekuensi, SF, BW, POLL_TIMEOUT, CYCLE_INTERVAL) adalah konstanta di dalam berkas, sengaja dibuat sama dengan slave.

Memantau kedua slave dari laptop. Selagi master berjalan di Pi lewat SSH, kedua Uno tetap tersambung USB ke laptop pengembang. lora_monitor.py adalah dasbor dua node dengan perekaman CSV, memerlukan pustaka rich.

```
pip install pyserial rich
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1 --out sesil.csv
```

Memverifikasi radio sebelum percobaan. cek_radio.py, dijalankan di Raspberry Pi, membaca balik register SX1276 setelah loraBegin() dan konfigurasi — bukan menyalin konstanta dari source, melainkan isi chip yang sesungguhnya.

```
ssh pi@<alamat-ip-pi>
cd ~/Documents/WSN-IOT-prak-Lora/Modul07_rpi_master_slave
python3 cek_radio.py
```

Pre-flight checklist

- ☐ Antena terpasang pada HAT dan kedua shield.
- ☐ SPI aktif di Raspberry Pi — ls /dev/spi* menampilkan spidev0.0.
- ☐ spidev dan RPi.GPIO (atau rpi-lgpio pada Pi 5) sudah terpasang di Pi.
- ☐ pio device list dijalankan di laptop, kedua port Uno dicatat dan diisikan ke platformio.ini.
- ☐ SSH ke Raspberry Pi sudah teruji sebelum sesi dimulai — jangan mendiagnosis masalah jaringan di tengah sesi terjadwal.
- ☐ Label fisik ditempel: SLAVE 1, SLAVE 2. Master tidak perlu label — hanya ada satu Raspberry Pi.

7.8 Percobaan

7.8.1 EXP-01 — Siklus Pertama Lintas Platform

Unggah kedua slave, jalankan cek_radio.py untuk memastikan register chip sesuai, lalu jalankan master.py dan amati siklus pertama pada **kedua sisi**: terminal SSH di Raspberry Pi, dan serial kedua Uno di laptop.

Expected output — master (Raspberry Pi)

```

=== LoRa MASTER-SLAVE 3 NODE ===
Init LoRa ... OK
Freq: 433 MHz
SF7 | BW: 125 kHz
Peran: MASTER (Raspberry Pi + LoRa GPS HAT)
Slave: Dragino Shield Uno - S1 & S2

=====
=== CYCLE 1 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -60 dBm | SNR: 14.2 dB
[TX] POLL:2
[RX] S2:DATA:1 | RSSI: -58 dBm | SNR: 14.2 dB
--- STATISTIK ---
S1: OK=1 | FAIL=0 | Data: 1
S2: OK=1 | FAIL=0 | Data: 1
Durasi siklus: 145 ms
=====

```

Data capture

Parameter	Hasil
Isi MODEM_CFG_2 menurut cek_radio.py (SF, CRC)	
Nomor siklus pertama yang lengkap tanpa FAIL	
RSSI master dari S1 / S2 (dBm)	
Jumlah [IGNORE] per siklus di tiap slave	

Tabel 7.6: Lembar pengamatan EXP-01

Verifikasi kesamaan kode — bandingkan `src/slave/main.cpp` di modul ini dengan `Modul05_lora_master_slave/src/slave/main.cpp` memakai `diff`. Jawab: berapa banyak baris yang berbeda, apa isinya, dan mengapa tidak satu pun di antaranya menyentuh logika penyaringan `POLL:<SLAVE_ID>`?

CHECKPOINT

Ketiga node mencetak OK, tiap slave menampilkan tepat satu [RX] dan dua [IGNORE] per siklus (satu untuk POLL milik node lain, satu untuk jawaban node lain), dan `diff` menunjukkan hanya komentar serta satu baris pesan pembuka yang berbeda dari M05. Bila jumlah baris beda lebih dari itu, ada perubahan tak sengaja yang perlu diperiksa sebelum melanjutkan.

7.8.2 EXP-02 — Statistik dan Lama Siklus Lintas Mesin

Rekam **bersamaan** selama minimal 50 detik: terminal SSH master di Raspberry Pi, dan kedua serial slave di laptop lewat `lora_monitor.py`. Dua rekaman ini berasal dari dua komputer yang **tidak berbagi jam** — korelasikan lewat nomor Data: dan RX#, bukan cap waktu.

Expected output — master

```

=====
=== CYCLE 40 ===
[TX] POLL:1
[RX] S1:DATA:40 | RSSI: -60 dBm | SNR: 14.0 dB
[TX] POLL:2
[RX] S2:DATA:40 | RSSI: -58 dBm | SNR: 14.1 dB
--- STATISTIK ---
S1: OK=39 | FAIL=1 | Data: 40
S2: OK=40 | FAIL=0 | Data: 40
Durasi siklus: 145 ms
=====

```

Data capture

Parameter	Hasil
Jumlah siklus dalam jendela rekaman	
Durasi siklus min / maks / rata-rata saat sehat (ms)	
Durasi siklus saat satu poll gagal (ms)	
RX# terakhir slave 1 / slave 2 (log lokal) vs Data: terakhir master	
SNR arah slave→ master vs master→ slave — simetris?	

Tabel 7.7: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA

Cari satu siklus di mana OK master lebih kecil daripada RX# slave pada nomor yang sama (misalnya S1: OK=39 sementara log lokal Slave 1 sudah mencetak RX#: 40). Jawab: apa yang terjadi pada jawaban itu di udara, dan mengapa fakta bahwa CRC payload **mati** (lihat cek_radio.py) relevan dengan jawabanmu?

CHECKPOINT

Angka Data: master mengikuti RX# slave secara berurutan, dengan kemungkinan FAIL sesekali yang **tidak pernah** membuat Data: melompat mundur. Durasi siklus sehat sangat rapat (sebaran hanya sekitar 1 ms pada Raspberry Pi 5) — jauh lebih rapat daripada master Arduino M05, karena Pi tidak pernah benar-benar berebut CPU pada beban seringan ini.

7.8.3 EXP-03 — Satu Node Hilang

Cabut kabel USB Slave 2 **secara fisik** (bukan hanya menutup port serial — Uno bershield LoRa memakai daya dari USB, sehingga mencabutnya mematikan seluruh node termasuk radionya) selama master.py berjalan, tunggu setidaknya 20 siklus, lalu pasang kembali.

Expected output — master, tepat setelah kabel dicabut

```

=== CYCLE 28 ===
[TX] POLL:1
[RX] S1:DATA:27 | RSSI: -70 dBm | SNR: 13.0 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=27 | FAIL=1 | Data: 27
S2: OK=0 | FAIL=27 | Data: None
Durasi siklus: 605 ms

```

Data capture

Parameter	Hasil
Pesan master saat Slave 2 tidak menjawab	
Durasi siklus saat Slave 2 hilang (ms)	
Apakah Slave 1 terpengaruh matinya Slave 2? (cek log lokal S1)	
Berapa siklus sampai OK Slave 2 bertambah lagi setelah dipasang	

Parameter	Hasil
dataCounter Slave 2 setelah dipasang kembali — mulai dari berapa, dan apa artinya	

Tabel 7.8: Lembar pengamatan EXP-03

CHECKPOINT

Durasi siklus melonjak dari kondisi sehat menjadi mendekati POLL_TIMEOUT penuh ditambah overhead, sementara Slave 1 sama sekali tidak terganggu — masih menjawab tiap POLL:1 seperti biasa. Begitu Slave 2 tersambung kembali, ia memulai dataCounter dari 1: bukti bahwa yang terjadi adalah **reboot penuh**, bukan sekadar port serial yang terputus. Pemulihan di sisi master terjadi otomatis pada siklus berikutnya, tanpa intervensi apa pun.

7.8.4 EXP-04 — Menekan Batas Waktu Sampai Rusak

Ambang kegagalan POLL_TIMEOUT naif diperkirakan di atas 70 ms (waktu udara POLL + waktu udara jawaban). Percobaan ini menunjukkan dugaan itu keliru. Ubah POLL_TIMEOUT di **salinan** master.py (jangan ubah src/master.py asli), jalankan tiap nilai selama 60 detik, mulai dari 500 ms turun bertahap sampai keberhasilan jatuh ke nol.

Expected output — pada nilai yang sudah terlalu kecil

```

=== CYCLE 12 ===
[TX] POLL:1
[FAIL] Slave 1 tidak merespon!
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=0 | FAIL=12 | Data: None
S2: OK=0 | FAIL=12 | Data: None

```

Data capture

POLL_TIMEOUT (ms)	Poll berhasil	Poll gagal	Gagal (%)
500 (baku)			
100			
60			
50			
45			
40			

Tabel 7.9: Lembar pengamatan EXP-04 — menekan batas waktu

CHECKPOINT

Ambangnya adalah **tebing**, bukan lereng: keberhasilan bertahan tinggi sampai satu titik, lalu jatuh mendekati nol dalam rentang sempit beberapa milidetik. Titik itu jauh **lebih rendah** daripada dugaan naif 70-an ms, karena transmit(pollMsg) bersifat blocking dan waktu udara POLL sudah “dibayar” di dalamnya, sebelum penghitung waktu mulai berjalan — lihat kembali bagian Dasar Teori. Jelaskan letak tebing itu memakai anggaran waktu yang sesungguhnya, bukan anggaran naif.

7.8.5 Verifikasi Radio (Dijalankan Sebelum EXP-01)

cek_radio.py membaca balik register SX1276 di Raspberry Pi setelah `loraBegin()` dan konfigurasi.

```
REG_VERSION   : 0x12   (0x12 = SX1276/77/78/79)
FREKUENSI     : 433.000000 MHz   (target 433.000000, selisih +0.0 Hz)
MODEM_CFG_1   : 0x72 -> BW=125 kHz | CR=4/5 | header=explicit
MODEM_CFG_2   : 0x70 -> SF7 | CRC payload=MATI
PA_CONFIG     : 0x8f -> PA_BOOST, power=17 dBm
```

PERINGATAN

CRC payload mati di kedua sisi. `LoRa.begin()` milik `sandeepmistry` (dipakai `slave`) tidak mengaktifkan CRC kecuali diminta lewat `LoRa.enableCrc()`, dan `master.py` juga tidak menyalakannya. Konsekuensinya: paket dengan payload rusak **tetap lolos** ke lapisan aplikasi sebagai paket sah, karena bendera `IRQ_CRC_ERROR` tidak pernah diminta menyala. Kegagalan seperti itu tercatat sebagai [WARN] Balasan tidak valid lalu [FAIL] — diagnosis yang menunjuk ke arah yang salah sama sekali, seolah `slave` tidak merespons padahal `slave` sudah menjawab benar.

7.8.6 Verifikasi Perangkat Keras (Log Referensi)

Dijalankan pada satu Raspberry Pi 5 + LoRa GPS HAT v1.4 dan dua Arduino Uno + Dragino Shield v1.2, 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

Parameter	Hasil terukur
Siklus dalam 50 detik (EXP-02)	76
Durasi siklus sehat min/maks/rata-rata	144 / 145 / 145,0 ms
Sebaran durasi siklus (Pi 5) vs master Arduino (M05)	1 ms vs 7 ms — Pi justru lebih rapat
EXP-03: durasi siklus saat satu slave mati	604–606 ms , rata-rata 621,4 ms (27 siklus berturut-turut)
EXP-03: pertambahan akibat satu node mati	+460 ms $\approx$ <code>POLL_TIMEOUT</code> dikurangi waktu jawaban sehat — hampir sama dengan +459 ms di M05
EXP-03: apakah node sehat ikut terganggu?	tidak — 0 dari 27 siklus
EXP-03: pemulihan setelah node kembali	seketika, siklus berikutnya
EXP-04: ambang kegagalan	tebing di 40–45 ms — bukan ~ 70 ms seperti dugaan naif
EXP-04: reproduksibilitas ambang	sempurna pada 6 sesi ulangan (3 $\times$ titik aman, 3 $\times$ titik gagal)
Payload rusak lolos (CRC mati)	0,74 % pada 8 sesi pertama, 0,00 % pada 4 sesi berikutnya — sporadis, sumbernya belum diketahui

Tabel 7.10: Hasil verifikasi perangkat keras Modul 07

```
Environment  Status  Flash
slave1       SUCCESS 26.4% (8.522 B)
slave2       SUCCESS 26.4% (8.522 B)
```

Master tidak dikompilasi — Python dijalankan langsung di Raspberry Pi. Kedua `slave` berukuran identik, bukti keduanya berasal dari source yang sama dan hanya berbeda `SLAVE_ID`.

CATATAN

Temuan tambahan yang tidak diperbaiki secara sengaja. Perbaikan CRC yang jelas — `LoRa.enableCrc()` di slave dan menyalakan bit yang sepadan di `REG_MODEM_CONFIG_2` pada master — **sengaja tidak diterapkan**, karena menyentuh firmware slave akan membatalkan klaim utama modul ini bahwa slave identik dengan M05. Rincian investigasinya (pola byte yang rusak, pengujian yang menyingkirkan SPI dan FIFO sebagai penyebab) ada di `logserial.md`, bagian “Temuan — Payload rusak lolos karena CRC mati”. Tantangan untuk menyalakannya sendiri dan mengukur akibatnya ada di CH-2.

7.8.7 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, `POLL_TIMEOUT` 500 ms, `CYCLE_INTERVAL` 500 ms. Ketiga node di satu meja, jarak ±30 cm.

Pengujian ini adalah **pengujian perangkat keras pertama** untuk Modul 07. Bagian “Catatan verifikasi” pada README sebelumnya menyatakan konversi ini belum pernah dijalankan di perangkat nyata; sejak dokumen ini ditulis, pernyataan itu tidak berlaku lagi.

Board & Port

Peran	Environment / program	Port	Board
Master	<code>src/master.py</code>	— (lewat SSH)	Raspberry Pi 5B rev 1.0, BCM2712 + Dragino LoRa GPS HAT v1.4
Slave 1	<code>slave1 (-DSLAVE_ID=1)</code>	<code>/dev/ttyACM0</code>	Uno asli (2341:0043) + Dragino Shield v1.2
Slave 2	<code>slave2 (-DSLAVE_ID=2)</code>	<code>/dev/ttyACM1</code>	Uno asli (2341:0043) + Dragino Shield v1.2

Tabel 7.11: Log serial Modul 07: Board & Port

Kedua Uno tertancap di laptop pengembang, sedangkan master dijalankan di Raspberry Pi lewat SSH. Inilah perbedaan praktis pertama dengan M05: **tiga node tidak lagi berada di satu komputer**, sehingga log master dan log slave direkam oleh dua mesin yang berbeda dan harus dicocokkan lewat isinya, bukan lewat cap waktu bersama.

Perangkat lunak Raspberry Pi: Raspberry Pi OS 13 (trixie), Python 3.13.5, `python3-spidev` 3.6, `python3-rpi-lgpio` 0.6. Paket `python3-rpi.gpio` **tidak** terpasang — pada Pi 5, `RPi.GPIO` disediakan oleh `rpi-lgpio`. Rinciannya di README bagian 5.

Verifikasi radio sebelum percobaan

Nilai di bawah dibaca **balik dari register SX1276** setelah `loraBegin()` + konfigurasi, bukan disalin dari konstanta di source.

```
REG_VERSION   : 0x12 (0x12 = SX1276/77/78/79)
FRF register  : 0x6c 0x40 0x00 -> frf=7094272
FREKUENSI     : 433.000000 MHz (target 433.000000, selisih +0.0 Hz)
MODEM_CFG_1   : 0x72 -> BW=125 kHz | CR=4/5 | header=explicit
MODEM_CFG_2   : 0x70 -> SF7 | CRC payload=MATI
PA_CONFIG     : 0x8f -> PA_BOOST, power=17 dBm
```

Parameter	Nilai di chip	Sesuai README?
Frekuensi	433,000000 MHz (selisih 0,0 Hz)	ya
Spreading factor	SF7	ya
Bandwidth	125 kHz	ya
Coding rate	4/5	ya
Daya pancar	17 dBm, PA_BOOST	ya
CRC payload	MATI	tidak disebut README — lihat bagian Temuan

Tabel 7.12: Log serial Modul 07: Verifikasi radio sebelum percobaan

Skrip pembacanya ditinggalkan di Raspberry Pi sebagai `cek_radio.py` agar verifikasi ini dapat diulang kapan saja.

EXP-01 — Siklus Pertama Lintas Platform

Master (Raspberry Pi), siklus pertama

```
=== LoRa MASTER-SLAVE 3 NODE ===
Init LoRa ... OK
Freq: 433 MHz
SF7 | BW: 125 kHz
Peran: MASTER (Raspberry Pi + LoRa GPS HAT)
Slave: Dragino Shield Uno - S1 & S2

=====
=== CYCLE 1 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -60 dBm | SNR: 14.2 dB
[TX] POLL:2
[RX] S2:DATA:1 | RSSI: -58 dBm | SNR: 14.2 dB
--- STATISTIK ---
S1: OK=1 | FAIL=0 | Data: 1
S2: OK=1 | FAIL=0 | Data: 1
Durasi siklus: 145 ms
=====
```

Slave 1 dan Slave 2 (cap waktu = detik sejak perekaman serial dimulai)

```
1.612 === LoRa SLAVE 1 === | 1.617 === LoRa SLAVE 2 ===
1.634 Init LoRa ... OK | 1.638 Init LoRa ... OK
1.634 Freq: 433.00 MHz | 1.638 Freq: 433.00 MHz
1.637 Menunggu POLL:1 dari Master | 1.642 Menunggu POLL:2 dari Master
|
4.320 [RX] POLL:1 | RSSI: -62 dBm | SNR: 8.00 dB | RX#: 1
4.320 [TX] S1:DATA:1 | 4.316 [IGNORE] POLL:1
4.389 [IGNORE] POLL:2 | 4.357 [IGNORE] S1:DATA:1
4.426 [IGNORE] S2:DATA:1 | 4.391 [RX] POLL:2 | RSSI: -64 dBm | SNR: 9.00 dB | RX#: 1
| 4.395 [TX] S2:DATA:1
```

Parameter	Hasil
Pesan init master	Init LoRa ... OK, 433 MHz, SF7/BW 125 kHz
Pesan init slave 1 / slave 2	keduanya Init LoRa ... OK, Freq: 433.00 MHz
Nomor siklus pertama yang lengkap	1 — tidak ada FAIL sama sekali di siklus pembuka
RSSI master ← S1	-60 dBm (SNR 14,2 dB)
RSSI master ← S2	-58 dBm (SNR 14,2 dB)
Baris [IGNORE]	S1 mengabaikan POLL:2, S2 mengabaikan POLL:1 — masing-masing 1 per siklus
Baris [IGNORE] tambahan	tiap slave juga mengabaikan jawaban slave lain (S2:DATA:1 / S1:DATA:1)

Tabel 7.13: Log serial Modul 07: EXP-01 — Siklus Pertama Lintas Platform

CHECKPOINT terpenuhi. Ketiga node mencetak OK, dan tiap slave menampilkan tepat **satu** [RX] beserta **dua** [IGNORE] per siklus — satu untuk POLL milik node lain, satu untuk jawaban node lain. Urutan waktu di kedua slave saling mengunci: S2 mencatat [IGNORE] POLL:1 pada 4.316 s, S1 mencatat [RX] POLL:1 pada 4.320 s. Keduanya mendengar paket yang sama; hanya pemiliknya yang menjawab.

Perhatikan bahwa **master berganti platform tanpa slave mengetahuinya**. Firmware slave yang dipakai identik dengan M05:

```
$ diff Modul05_lora_master_slave/src/slave/main.cpp Modul07_rpi_master_slave/src/slave/main.cpp
2c2 < LoRa Master-Slave 3 Node - ... > LoRa Master-Slave hybrid - ...
6,7c6,12 (blok komentar penjelas M07)
```

```
74c79,81 Serial.println("Menunggu POLL:1 dari Master...")
-> Serial.print("Menunggu POLL:"); Serial.print(SLAVE_ID);
Serial.println(" dari Master (Raspberry Pi)...")
```

Tiga hunk, seluruhnya komentar dan satu pesan pembuka di Serial. Tidak satu byte pun dari yang mengudara berubah.

EXP-02 — Statistik dan Lama Siklus

Rekaman 50 detik dengan `src/master.py` apa adanya, sambil kedua port serial slave direkam bersamaan.

Parameter	Hasil
Jumlah siklus dalam 50 detik	76 ($\approx 1,52$ siklus/detik)
S1: OK / FAIL	74 / 2
S2: OK / FAIL	75 / 1
Data: terakhir S1 / S2	76 / 76
RX# terakhir di slave 1 / slave 2	76 / 76
Durasi siklus minimum	144 ms
Durasi siklus maksimum (siklus sehat)	145 ms
Durasi siklus rata-rata (siklus sehat)	145,0 ms
Durasi siklus saat satu poll gagal	605 ms
RSSI di master (dari S1 / S2)	-59,6 / -57,8 dBm
SNR di master (dari S1 / S2)	14,09 / 14,19 dB
RSSI di S1 / S2 (dari master)	-64,8 / -64,0 dBm
SNR di S1 / S2 (dari master)	8,63 / 8,88 dB

Tabel 7.14: Log serial Modul 07: EXP-02 — Statistik dan Lama Siklus

Buka abstraksinya. Ketiga bilangan yang diminta README dapat dibandingkan langsung: pada siklus ke-76, master mencatat S1: OK=74 | FAIL=2 | Data: 76, sedangkan slave 1 mencatat RX#: 76. Artinya slave menerima **seluruh** 76 panggilan dan mengirim **seluruh** 76 jawaban, tetapi hanya 74 yang sampai utuh ke master. Data: melompat dari 74 ke 76 tanpa master pernah menerima nomor 75 — persis keadaan yang diramalkan CHECKPOINT EXP-02, dan didapat tanpa perlu menjauhkan slave. Penyebab dua jawaban yang hilang itu dibahas di bagian Temuan.

Asimetri arah yang tidak terduga. SNR arah slave → master adalah 14,1 dB, sedangkan arah master → slave hanya 8,6–8,9 dB — selisih tetap sekitar **5,3 dB** yang bertahan di seluruh rekaman. Kedua arah memakai daya pancar 17 dBm yang sama. Yang berbeda hanyalah papan pembawanya: LoRa GPS HAT di sisi Pi, Shield v1.2 di sisi Uno. Angka ini tidak terlihat dari terminal master saja, karena master hanya melaporkan arah yang diterimanya; ia baru muncul setelah kedua sisi direkam bersamaan.

Lama siklus jauh lebih rapat daripada dugaan README

README memperkirakan master Linux akan memperlihatkan **jitter penjadwalan** yang tidak ada padanannya di Arduino. Pada Raspberry Pi 5 dugaan itu **tidak terkonfirmasi**:

Ukuran	Master Raspberry Pi 5 (modul ini)	Master Arduino Uno (M05, logserial.md)
Durasi siklus minimum	144 ms	147 ms
Durasi siklus maksimum	145 ms	154 ms
Durasi siklus rata-rata	145,0 ms	152 ms
Sebaran (maks – min)	1 ms	7 ms

Tabel 7.15: Log serial Modul 07: Lama siklus jauh lebih rapat daripada dugaan README

Master Linux justru **lebih rapat** sebarannya daripada master Arduino, dan sedikit lebih cepat. Pengukuran kedua, diambil dari sisi node memakai `loramonitor.py` (jarak antar POLL yang diterima slave, jadi sudah termasuk `CYCLE_INTERVAL 500 ms`):

Ukuran	Nilai
Periode siklus minimum	642 ms
Periode siklus rata-rata	645,2 ms
Periode siklus maksimum	648 ms
Sebaran	6 ms (n = 112)

Tabel 7.16: Log serial Modul 07: Lama siklus jauh lebih rapat daripada dugaan README

Kedua pengukuran sepakat. Penjelasan yang masuk akal: Pi 5 berinti empat pada beban hampir nol, sehingga proses Python praktis tidak pernah benar-benar berebut CPU; dan `POLL_TIMEOUT 500 ms` sedemikian longgar dibanding waktu tanggap sebenarnya (± 45 ms, lihat EXP-04) sehingga penundaan beberapa milidetik tidak pernah mengubah hasil. Sebaran ini bukan bantahan terhadap teori di README — jitter tetap ada secara prinsip — melainkan bukti bahwa pada beban serendah ini besarnya tidak terukur oleh alat yang dipakai modul ini.

EXP-03 — Satu Node Hilang

Slave 2 dicabut kabel USB-nya sepenuhnya (bukan sekadar port serial ditutup — Uno bershield LoRa memakai daya dari USB, jadi mencabutnya mematikan seluruh node termasuk radionya) selama `master.py` berjalan, lalu dipasang kembali di tengah sesi. Slave 1 tetap dipantau lewat serial lokal selama percobaan berlangsung untuk memastikan node sehat tidak ikut terganggu.

```

=== CYCLE 1 ===
[TX] POLL:1
[FAIL] Slave 1 tidak merespon!
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
Durasi siklus: 1065 ms
=====
=== CYCLE 2 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -70 dBm | SNR: 12.8 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=1 | FAIL=1 | Data: 1
S2: OK=0 | FAIL=2 | Data: None
Durasi siklus: 604 ms
...
=== CYCLE 28 ===
[TX] POLL:1
[RX] S1:DATA:27 | RSSI: -70 dBm | SNR: 13.0 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=27 | FAIL=1 | Data: 27
S2: OK=0 | FAIL=27 | Data: None
Durasi siklus: 605 ms
=== CYCLE 29 ===
[TX] POLL:1
[RX] S1:DATA:28 | RSSI: -71 dBm | SNR: 12.8 dB
[TX] POLL:2
[RX] S2:DATA:1 | RSSI: -64 dBm | SNR: 13.8 dB
--- STATISTIK ---
S1: OK=28 | FAIL=1 | Data: 28
S2: OK=1 | FAIL=27 | Data: 1
Durasi siklus: 144 ms

```

Parameter	Hasil
Siklus 1 — kedua slave gagal (start-up master mendahului kesiapan radio)	Durasi 1065 ms , kira-kira dua <code>POLL_TIMEOUT</code> penuh

Parameter	Hasil
Siklus 2–28 — Slave 2 hilang (27 siklus berturut-turut)	Durasi stabil 604–606 ms , rata-rata 621,4 ms
Siklus 29 — Slave 2 kembali	Langsung OK, S2:DATA:1 — dataCounter slave mulai dari 1 (bukti Uno benar-benar reboot, bukan cuma port serial terputus)
Siklus 30 dst. — steady state	Durasi kembali 144–145 ms , rata-rata 144,8 ms
Slave 1 selama Slave 2 hilang	OK di seluruh 27 siklus — 79 baris [TX] S1:DATA:n lokal, [IGNORE] POLL:2 79× tanpa gangguan
Pertambahan durasi akibat satu node mati	604,8 – 144,8 ≈ 460 ms , ≈ POLL_TIMEOUT (500 ms) dikurangi waktu jawaban sehat
Pemulihan	Seketika — satu siklus setelah Slave 2 tersambung kembali, tanpa intervensi di sisi master

Tabel 7.17: Log serial Modul 07: EXP-03 — Satu Node Hilang

CHECKPOINT terpenuhi, dan sekarang dengan data sungguhan. Catatan lama di bagian “Catatan pengambilan log” menyimpulkan pertambahan +460 ms hanya dari poll individual yang gagal sesekali, tanpa benar-benar mencabut node. Sesi ini mengonfirmasinya secara langsung: 27 siklus berturut-turut dengan Slave 2 mati, tidak satu pun memengaruhi Slave 1, dan durasi siklus melonjak persis sebesar satu POLL_TIMEOUT dikurangi waktu tanggap sehat ($605 - 145 \approx 460$ ms) — sama seperti prediksi CHECKPOINT EXP-03 M05, dan sama dengan angka +459 ms yang tercatat di Modul05_lora_master_slave/logserial.md untuk skenario setara.

Siklus 1 (1065 ms) bukan anomali — konsisten dengan dua timeout penuh. Master mulai polling sebelum Slave 1 (yang baru saja diunggah ulang) selesai boot, sehingga siklus pertama kehilangan **dua** slave sekaligus: $2 \times \sim 500$ ms POLL_TIMEOUT plus overhead, hasilnya 1065 ms — persis nilai yang sudah tercatat sebagai temuan tersendiri di bagian ini sebelum EXP-03 pernah dijalankan langsung.

EXP-04 — Menekan Batas Waktu Sampai Rusak

POLL_TIMEOUT diturunkan bertahap, tiap nilai dijalankan **60 detik**. Nilai diberikan lewat environment ke salinan master.py, sehingga src/master.py tidak pernah diubah.

POLL_TIMEOUT (ms)	Siklus	Poll berhasil	Poll gagal	Gagal (%)	Durasi siklus rata-rata (ms)
500 (baku)	92	184	0	0,0 %	156
250	92	184	0	0,0 %	156
150	92	184	0	0,0 %	156
100	92	184	0	0,0 %	156
60	92	184	0	0,0 %	156
55	92	183	1	0,5 %	156
50	92	184	0	0,0 %	156

POLL_TIMEOUT (ms)	Siklus	Poll berhasil	Poll gagal	Gagal (%)	Durasi siklus rata-rata (ms)
45	92	181	3	1,6 %	156
40	93	0	186	100 %	146

Tabel 7.18: Log serial Modul 07: EXP-04 — Menekan Batas Waktu Sampai Rusak

Ambangnya adalah tebing, bukan lereng — persis seperti diramalkan CHECKPOINT EXP-04. Di 45 ms sistem masih berhasil 98,4 %; di 40 ms keberhasilan jatuh ke **nol mutlak**, bukan memburuk perlahan. Zona 45–55 ms adalah pinggiran tebing: sebagian jawaban mulai tersenggol batas waktu, tetapi mayoritas masih lolos.

Pengulangan tiga kali pada dua nilai penentu (Pengukuran C):

Percobaan ke-	Nilai terkecil yang masih andal (ms)	Nilai pertama yang sudah gagal (ms)
1	60 (0 gagal dari 184)	40 (186 gagal dari 186)
2	60 (0 gagal dari 184)	40 (186 gagal dari 186)
3	60 (0 gagal dari 184)	40 (186 gagal dari 186)

Tabel 7.19: Log serial Modul 07: EXP-04 — Menekan Batas Waktu Sampai Rusak

Reproduksinya sempurna — tidak ada satu pun kejadian menyimpang di enam sesi.

Mengapa 40 ms, bukan 77 ms

README memperkirakan ambangnya “hampir pasti jauh di atas” 77 ms, dengan alasan waktu udara POLL:1 (± 36 ms) ditambah waktu udara S1:DATA:12 (± 41 ms). Hasil pengukuran menunjukkan ambangnya justru **di bawah** angka itu, dan penyebabnya adalah kekeliruan dalam menyusun anggaran waktunya:

POLL_TIMEOUT **tidak pernah mencakup waktu udara POLL**. Di pollSlave(), transmit(pollMsg) bersifat blocking — ia baru kembali setelah IRQ_TX_DONE menyala, artinya paket POLL sudah selesai mengudara. Baru sesudah itu waitStart = time.monotonic() dijalankan. Jadi jendela batas waktu hanya menampung:

```
pemrosesan di slave + waktu udara jawaban ( $\pm 41$  ms) + deteksi RX_DONE di master
```

Yang tersisa memang ± 41 –45 ms, dan itulah sebabnya 45 ms berada di pinggiran sementara 40 ms memotong setiap jawaban tepat sebelum tiba. Angka 36 ms untuk POLL sudah “dibayar” di dalam transmit(), di luar penghitung waktu.

Konsekuensi praktisnya: POLL_TIMEOUT 500 ms adalah **11× lebih longgar** daripada yang dibutuhkan. Kelonggaran itulah yang membuat durasi siklus tidak pernah goyah — dan sekaligus yang membuat satu node mati menjadi sangat mahal, karena setiap node mati menagih 500 ms penuh setiap siklus.

Temuan — Payload rusak lolos karena CRC mati

Selama pengujian, master beberapa kali mencetak [WARN] Balasan tidak valid lalu menghitungnya sebagai [FAIL], padahal log serial slave membuktikan slave menerima POLL dan mengirim jawaban yang benar. Contoh byte mentahnya:

```
[WARN] Balasan tidak valid: S7:DATA:705 | ps=11 | raw=53 a1 3a 44 41 54 41 3a 37 30 35
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      ^^ seharusnya 0x31 ('1')
[WARN] Balasan tidak valid: S1:DATA:785 | ps=10 | raw=53 31 3a 44 41 54 41 9a 38 35
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
      ^^ seharusnya 0x3a (':')
```

Frekuensi kejadian

Kelompok run	Paket diterima	Paket rusak	Rasio
8 sesi pertama	2288	17	0,74 %

Kelompok run	Paket diterima	Paket rusak	Rasio
4 sesi berikutnya	1462	0	0,00 %

Tabel 7.20: Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati

Pola byte — dari 13 paket rusak yang byte mentahnya sempat terekam:

Sifat	Pengamatan
Byte rusak per paket	1 byte (10 paket), 2 byte (3 paket)
Indeks byte yang rusak	indeks 1 (10×), indeks 7 (4×), indeks 8 (2×)
Indeks yang tidak pernah rusak	0, 2, 3, 4, 5, 6 — yaitu S, :, dan DATA
Jumlah bit berbeda, pada byte yang nilai benarnya pasti	selalu 2 bit (4 dari 4 kasus)

Tabel 7.21: Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati

Yang sudah disingkirkan sebagai penyebab

Dugaan	Uji	Hasil
Jalur SPI tidak andal	REG_VERSION dibaca 40.000× pada 5 MHz dan 1 MHz	0 error di kedua kecepatan — SPI bersih saat idle
Modem menimpa FIFO saat dibaca	MODE_STDBY disisipkan sebelum baca FIFO, seperti <code>LoRa.parsePacket()</code> di Arduino	rasio tidak berubah (6 dari 354)
Byte rusak saat dibaca dari FIFO	isi FIFO dibaca 3× berturut-turut lalu dibandingkan	selalu identik , termasuk pada paket rusak
Master membanjiri SPI saat demodulasi	<code>time.sleep(0.002)</code> disisipkan di loop tunggu	0 dari 730 — tetapi kontrol tanpa jeda juga 0 dari 732 , jadi tidak

Dugaan	Uji	Hasil
		membuktikan apa pun

Tabel 7.22: Log serial Modul 07: Temuan — Payload rusak lolos karena CRC mati

Pembacaan FIFO tiga kali yang selalu identik adalah uji penentunya: **byte rusak itu sudah berada di FIFO sebelum dibaca**. Jalur SPI tidak bersalah; kerusakan terjadi di udara atau di dalam demodulator.

Mengapa kerusakan itu lolos sampai ke aplikasi. MODEM_CFG_2 menunjukkan CRC payload **mati**, di kedua sisi. LoRa.begin() milik sandeepmistry tidak mengaktifkan CRC kecuali diminta lewat LoRa.enableCrc(), dan master.py juga tidak menyalakannya. Akibatnya pemeriksaan IRQ_CRC_ERROR di parsePacket() **tidak akan pernah menyala** — bukan karena tidak ada paket cacat, melainkan karena radio tidak pernah diminta memeriksanya. Paket cacat naik utuh ke lapisan aplikasi, gagal di reply.startswith("S1:DATA:"), dan tercatat sebagai “slave tidak merespon” — diagnosis yang menunjuk ke arah yang salah sama sekali.

Status. Mekanisme yang membangkitkan kerusakan **belum diketahui**. Pola indeksinya (hanya 1, 7, 8) terlalu terpusat untuk galat bit acak, tetapi belum ada uji yang menjelaskannya. Fenomenanya juga tidak muncul terus-menerus: 17 kejadian di delapan sesi pertama, nol di empat sesi berikutnya dan di seluruh EXP-04 kecuali satu, tanpa perubahan perangkat keras apa pun di antaranya.

Perbaikan yang jelas — LoRa.enableCrc() di slave dan menyalakan bit 2 REG_MODEM_CONFIG_2 di master — **tidak diterapkan**, karena menyentuh firmware slave dan dengan demikian membatalkan klaim utama modul ini bahwa slave identik dengan M05. Itu keputusan perancang modul, bukan keputusan penguji. Sebagai bahan praktikum, keadaan ini justru lebih berharga dibiarkan: ia memperlihatkan satu lapisan pelindung yang tidak dipasang, dan akibatnya terhadap diagnosis di lapisan atas.

Verifikasi ulang — 21 Agustus 2026

Sesi verifikasi baru: kedua slave dibangun ulang dari src/ saat ini dan diunggah ulang (pio run -e slave1|slave2 -t upload, keduanya SUCCESS, flash 8.522 B terverifikasi avrdude), master dijalankan langsung dari sumbernya di Raspberry Pi lewat SSH (python3 -u src/master.py, tanpa perubahan). Tujuannya memastikan konversi PlatformIO M07 masih berjalan seperti didokumentasikan di atas, bukan mengulang investigasi CRC.

Board & Port sesi ini — berbeda dari tabel “Board & Port” di atas karena hanya dua Uno yang tersambung ke laptop pengembang saat ini: Slave 1 di /dev/ttyACM1, Slave 2 di /dev/ttyACM2 (bukan ACM0/ACM1). Ini contoh nyata alasan README menyuruh menjalankan tools/deteksi_port.py dan memakai --upload-port eksplisit alih-alih mengandalkan nilai contoh di platformio.ini.

```

=== CYCLE 3 ===
[TX] POLL:1
[RX] S1:DATA:47 | RSSI: -76 dBm | SNR: 13.0 dB
[TX] POLL:2
[WARN] Balasan tidak valid: S?:DATA:27
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=3 | FAIL=0 | Data: 47
S2: OK=2 | FAIL=1 | Data: None
Durasi siklus: 605 ms

```

Parameter	Hasil
Siklus dalam ±35 detik	46
Durasi siklus steady-state min/maks/rata-rata	145 / 145 / 145,0 ms (n=45, mengecualikan 1 timeout)
Slave 1: OK / FAIL	46 / 0 → 100 %
Slave 2: OK / FAIL	45 / 1 → 97,8 %
Penyebab satu-satunya FAIL	1 byte payload rusak (S1:DATA:27 → S :DATA:27), pola sama dengan temuan “Payload rusak lolos karena CRC mati” di atas

Parameter	Hasil
SNR balasan Slave 1 di master	rata-rata 13,2 dB (n=46)
SNR balasan Slave 2 di master	rata-rata 13,3 dB (n=45)
RSSI balasan Slave 1 / Slave 2 di master	-69,4 dBm / -65,8 dBm (sepadan, tidak ada node yang janggal)

Tabel 7.23: Log serial Modul 07: Verifikasi ulang — 21 Agustus 2026

Tidak ada anomali seperti pada M05. Modul 05 sempat mencatat SNR Slave 2 anjlok ke ~1,2 dB akibat Slave 2 duduk terlalu dekat dengan master (near-field). Pada sesi M07 ini, SNR kedua slave hampir identik (13,2 vs 13,3 dB) dan RSSI keduanya wajar — tidak ada indikasi kejenuhan penerima di sisi mana pun.

Tampilan Uno vs tampilan Raspberry Pi cocok satu sama lain. Log slave lokal (dibaca langsung dari /dev/ttyACM1 dan /dev/ttyACM2) menunjukkan [TX] S1:DATA:n dan [TX] S2:DATA:n yang nomornya berurutan dengan [RX] S1:DATA:n/S2:DATA:n pada log master di Raspberry Pi — mengonfirmasi kedua sisi memang saling bicara lewat radio, bukan kebetulan dua proses berjalan sendiri-sendiri.

Catatan pelaksanaan. Saat keluarannya dipipa lewat SSH (bukan TTY interaktif), Python membuffer stdout — memakai timeout untuk membatasi durasi lalu memutus prosesnya membuang isi buffer yang belum sempat di-flush. Jalankan dengan python3 -u saat keluarannya perlu direkam lewat pipa/redirect. master.py sebelumnya tidak punya opsi --help — argumen apa pun diabaikan begitu saja dan sesi langsung berjalan, seperti yang terjadi saat pertama kali dicoba pada sesi ini. Sudah ditambahkan argparse dengan -h/--help yang mencetak parameter radio (frekuensi, SF, BW, POLL_TIMEOUT, CYCLE_INTERVAL) tanpa menyentuh SPI/GPIO, dan argumen tak dikenal sekarang ditolak dengan pesan usage alih-alih diam-diam diabaikan. Tidak ada opsi baru selain -h; perilaku tanpa argumen tidak berubah (diverifikasi ulang: 10 siklus bersih, 100 % OK).

Catatan pengambilan log

- **EXP-03 sudah dijalankan** pada sesi 21 Agustus 2026 (lihat bagian EXP-03 di atas) — kabel USB Slave 2 dicabut fisik dan dipasang kembali sementara operator hadir langsung di lokasi board, master tetap dijalankan dari jarak jauh lewat SSH.
- **Tabel jarak (Pengukuran B) belum terisi.** Seluruh percobaan dijalankan di satu meja pada jarak tetap ± 30 cm.
- Kolom M05 pada tabel perbandingan lama siklus diisi dari Modul05_lora_master_slave/logserial.md, bukan dari pengukuran ulang pada sesi ini.
- Membuka port serial me-**reset** Arduino lewat DTR, sehingga RX# dan dataCounter slave kembali ke 1 sementara penghitung master terus berjalan. Bila kedua angka perlu sebanding, jalankan perekam serial lebih dahulu, baru master.
- Seluruh berkas diagnostik (master_diagA/B/C/D.py, master_exp04.py, spi_stress.py) sudah dihapus dari Raspberry Pi. Yang tersisa di sana hanya src/master.py, requirements.txt, dan cek_radio.py.
- src/master.py, src/slave/main.cpp, dan platformio.ini **tidak diubah sama sekali** selama pengujian ini.

7.9 Pengukuran

A. Keberhasilan terhadap jarak — kedua slave ditempatkan pada jarak sama dari Raspberry Pi, 30 siklus per baris.

Jarak	RSSI S1	RSSI S2	OK/FAIL S1	OK/FAIL S2	Berhasil S1 (%)	Berhasil S2 (%)
1 m						
25 m						

Jarak	RSSI S1	RSSI S2	OK/FAIL S1	OK/FAIL S2	Berhasil S1 (%)	Berhasil S2 (%)
50 m						
100 m						

Tabel 7.24: Lembar pengukuran A – keberhasilan terhadap jarak

B. Perbandingan platform master – bandingkan langsung dengan Modul05_lora_master_slave/logserial.md.

Ukuran	Master Raspberry Pi 5 (modul ini)	Master Arduino Uno (M05)
Durasi siklus minimum (ms)		147
Durasi siklus maksimum (ms)		154
Durasi siklus rata-rata (ms)		152
Sebaran (maks – min, ms)		7

Tabel 7.25: Lembar pengukuran B – perbandingan platform master

C. Ambang POLL_TIMEOUT – ulangi EXP-04 tiga kali pada dua titik: nilai terkecil yang masih 100 % andal, dan nilai terbesar yang sudah gagal total.

Percobaan ke-	Nilai terkecil andal (ms)	Nilai pertama gagal total (ms)
1		
2		
3		

Tabel 7.26: Lembar pengukuran C – ambang POLL_TIMEOUT

D. Rasio payload rusak lolos – hitung dari sesi gabungan EXP-02 dan EXP-04 milik sendiri.

Sesi	Paket diterima	Paket [WARN] Balasan tidak valid	Rasio (%)
1			
2			

Tabel 7.27: Lembar pengukuran D – rasio payload rusak lolos

7.10 Analisis

1. Dari Tabel 7.25, sebutkan dua kemungkinan penyebab master Raspberry Pi memiliki sebaran durasi siklus lebih rapat daripada master Arduino, dan jelaskan mengapa POLL_TIMEOUT yang longgar (500 ms) membuat perbedaan itu tidak pernah terlihat pada durasi siklus akhir.

2. Dari Tabel 7.26, hitung anggaran waktu tunggu sesungguhnya (bukan naif) menggunakan waktu udara jawaban dari bagian Dasar Teori M05. Apakah nilai ambang yang ditemukan cocok dengan hitungan itu? Jelaskan selisihnya bila ada.
3. Pada EXP-03, mengapa siklus pertama setelah master baru dinyalakan bisa mencatat **kedua** slave gagal sekaligus, dan berapa perkiraan durasinya dibandingkan satu slave gagal?
4. Dari Tabel 7.27, apakah rasio payload rusak berkorelasi dengan jumlah siklus dalam sesi, durasi sesi, atau tidak keduanya? Kaitkan jawabannya dengan status “belum diketahui” pada temuan CRC di `logserial.md`.
5. `master.py` menghitung [WARN] Balasan tidak valid sebagai [FAIL] yang sama dengan slave yang benar-benar tidak menjawab. Usulkan satu cara membedakan keduanya dari sisi statistik master saja, tanpa mengubah firmware slave.
6. Bandingkan pekerjaan memindahkan master ke Raspberry Pi (modul ini) dengan pekerjaan memindahkan protokol seluruhnya ke LoRaWAN. Sebutkan satu keuntungan dan satu kerugian pendekatan “gateway custom” dibanding memakai protokol siap pakai.

7.11 Concept Check

1. Mengapa firmware slave tidak perlu tahu bahwa masternya sekarang Raspberry Pi, bukan Arduino?
2. Sebutkan dua cara mengorelasikan log dari dua mesin yang tidak berbagi jam, selain nomor urutan payload.
3. Mengapa `POLL_TIMEOUT` tidak menghitung waktu udara `POLL` itu sendiri? Fungsi mana di `master.py` yang menjadi penyebabnya?
4. Apa yang terjadi pada paket yang payload-nya rusak satu bit ketika CRC payload mati, dan mengapa `parsePacket()` tetap menyerahkannya ke aplikasi?
5. Mengapa modul ini tidak menyalakan CRC meskipun penyebabnya jelas dan perbaikannya sederhana?
6. Apa perbedaan mendasar antara “slave tidak menjawab” ([FAIL]) dan “jawaban rusak” ([WARN]) dari sudut pandang radio, dan mengapa master saat ini tidak membedakan keduanya dalam statistik OK dan FAIL?

7.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Memperbesar jaringan dan mempertajam diagnosis

CH-1 — Slave ketiga Tambahkan Uno ketiga dengan `SLAVE_ID=3`: satu environment PlatformIO baru, dan penyesuaian `SLAVE_COUNT` serta pemanggilan `pollSlave(3, ...)` di `master.py`. Ukur pertambahan lama siklus dan bandingkan dengan hasil CH-1 pada M05.

CH-3 — Bedakan FAIL dan WARN Ubah `pollSlave()` di `master.py` agar mencatat statistik terpisah untuk “tidak menjawab sama sekali” versus “menjawab tapi rusak”. Ukur apakah rasio keduanya berubah seiring durasi sesi.

Level 3 – Tantangan · Menutup celah yang sengaja dibiarkan

CH-2 – Nyalakan CRC Tambahkan `LoRa.enableCrc()` di `src/slave/main.cpp` **dan** set bit yang sepadan di `REG_MODEM_CONFIG_2` pada `master.py`. Jalankan ulang Tabel 7.27 dan bandingkan rasio payload rusak yang lolos. Perhatikan: ini mengubah firmware slave, sehingga klaim “identik dengan M05” tidak lagi berlaku — jelaskan trade-off-nya di laporan.

CH-4 – Jadwal adaptif lintas platform Port ide CH-4 M05 (lewati slave yang gagal tiga kali berturut-turut, tengok kembali tiap sepuluh siklus) ke `master.py`. Ukur perbaikan durasi siklus saat satu node mati, bandingkan dengan hasil EXP-03.

CH-5 – Korelasi RSSI lintas mesin Jalankan `lora_monitor.py --out sesi.csv` di laptop bersamaan dengan `master.py` di Raspberry Pi selama sepuluh menit. Gabungkan kedua rekaman berdasarkan nomor urut, lalu buat satu grafik RSSI dari kedua arah (slave→ master dan master→ slave) terhadap waktu.

7.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — mengapa slave tidak perlu tahu platform master, anggaran waktu sesungguhnya `POLL_TIMEOUT`, konsekuensi CRC mati.
3. Bukti diff bahwa firmware slave tidak berubah dari M05.
4. Hasil eksperimen — keluaran terminal EXP-01 sampai EXP-04 dari **kedua sisi** (master di Raspberry Pi, slave di laptop) beserta checkpoint, dan hasil `cek_radio.py`.
5. Data pengukuran — Tabel 7.24 sampai Tabel 7.27.
6. Analisis dan concept check, termasuk hitungan anggaran waktu `POLL_TIMEOUT`.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang disusun sendiri mengenai apa yang berubah dan apa yang tidak berubah ketika penjadwal jaringan LoRa dipindahkan dari mikrokontroler ke gateway Linux.

8. Modul 07B — Varian CRC Payload Aktif

LoRa COMMUNICATION LAB · MODUL 07B

Move the Scheduler to Linux — Varian -withCRC, CRC Payload Aktif

Raspberry Pi + LoRa GPS HAT v1.4 dan 2 × Arduino Uno + LoRa Shield v1.2 · topologi bintang, 3 node · polling terjadwal + CRC payload · level Advanced · 3 × 50 menit · folder kode Modul07b_rpi_master_slave_crc

PENGANTAR MODUL · Gambaran Umum

Modul 07B adalah **varian -withCRC** dari Modul 07. Modul 07 aslinya sengaja mematikan CRC payload di kedua sisi sebagai bahan ajar — paket yang rusak di udara lolos ke lapisan aplikasi dan salah didiagnosis sebagai “slave tidak merespon”. Modul ini menerapkan perbaikannya, yang pada Modul 07 tercatat sebagai tantangan CH-2.

8.1 Pendahuluan

Seluruh misi, topologi, dan kontrak data modul ini sama persis dengan Bab 7.: `master.py` di Raspberry Pi menjadwalkan dua Arduino Uno lewat `POLL:<id>` dan `S<id>:DATA:<n>`. Yang berbeda hanya satu keputusan: `LoRa.enableCrc()` ditambahkan di `src/slave/main.cpp`, dan bit yang sepadan (`REG_MODEM_CONFIG_2` bit 2) dinyalakan lewat fungsi `enableCrc()` baru di `src/master.py` serta `cek_radio.py`.

Konsekuensinya, klaim “identik dengan slave M05” **tidak lagi berlaku secara harfiah** untuk firmware di modul ini — satu baris fungsional berbeda, di luar komentar dan pesan pembuka. Bagian Percobaan dan `logserial.md` membandingkan perilaku sebelum dan sesudah CRC dinyalakan.

Prasyaratnya adalah Bab 7. seluruhnya. Pembaca yang belum mengerjakan modul tersebut sebaiknya menyelesaikannya lebih dahulu, sebab modul ini hanya bermakna sebagai pembanding: nilainya terletak pada selisih terhadap baseline CRC-mati, bukan pada angkanya sendiri.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
05	Banyak node — hak bicara dijadwalkan agar tidak bertabrakan
06	Isi <code>LoRa.begin()</code> tidak pernah terlihat — register dipegang langsung
07	Penjadwal pindah ke gateway Linux; sisi node tidak berubah sama sekali
07B (ini)	CRC payload dinyalakan — paket rusak dibuang radio, bukan salah didiagnosis aplikasi

Tabel 8.1: Kedudukan Modul 07B terhadap modul sebelumnya

Kontrak data lab ini. Tidak berubah dari Modul 07: `POLL:<id>` dan `S<id>:DATA:<n>`. Yang bertambah di udara hanyalah 2 byte checksum per paket, yang ditambahkan dan diperiksa oleh SX1276 sendiri, bukan oleh aplikasi.

8.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menyalakan CRC payload dan mengukur akibatnya

1. Membuktikan dengan diff seberapa kecil perubahan yang diperlukan pada firmware slave untuk menyalakan CRC payload, dan menjelaskan mengapa perubahan itu tetap membatalkan klaim “identik dengan M05”.
2. Menerapkan ulang logika penjadwalan round-robin Modul 05 di atas driver SX1276 telanjang Modul 06, memakai spidev dan RPi.GPIO atau rpi-lgpio.
3. Mengkorelasikan log dari dua mesin yang tidak berbagi jam — satu direkam lewat SSH di Raspberry Pi, satu lewat USB lokal — berdasarkan isi pesan, bukan cap waktu.
4. Menjelaskan mengapa menyalakan CRC payload di kedua sisi mengubah kelas kegagalan “payload rusak lolos ke aplikasi” menjadi “paket dibuang oleh radio sebelum sempat diproses”, dan mengukur biayanya lewat waktu-di-udara.
5. Menelusuri anggaran waktu sesungguhnya di balik POLL_TIMEOUT — bagian siklus mana yang benar-benar dihitungnya — lalu memakainya untuk menjelaskan mengapa menyalakan CRC menggeser durasi siklus, bukan ambang kegagalannya.

Kriteria keberhasilan

- ☐ master.py berhasil memanggil kedua slave Uno bergiliran, dengan CRC payload **AKTIF** di kedua sisi — dibuktikan lewat cek_radio.py (MODEM_CFG_2 bit 2 = 1) dan pesan CRC payload: AKTIF pada Serial Monitor slave.
- ☐ cek_radio.py membaca balik register SX1276 di Raspberry Pi dan hasilnya dibandingkan dengan konfigurasi yang dimaksud, termasuk bit CRC.
- ☐ Satu sesi log master (Raspberry Pi) dan log slave (laptop) dikorelasikan lewat isi pesan, bukan lewat waktu perekaman, dan menunjukkan nol baris [WARN] Balasan tidak valid.
- ☐ Durasi siklus dengan CRC aktif diukur dan dibandingkan terhadap baseline Modul07_rpi_master_slave/logserial.md (CRC mati) untuk mengukur biaya waktu-di-udara dari 2 byte CRC tambahan.

8.3 Dasar Teori (Secukupnya)

Seluruh istilah Modul 07 (Tabel 7.2) tetap berlaku. Satu di antaranya berubah isinya.

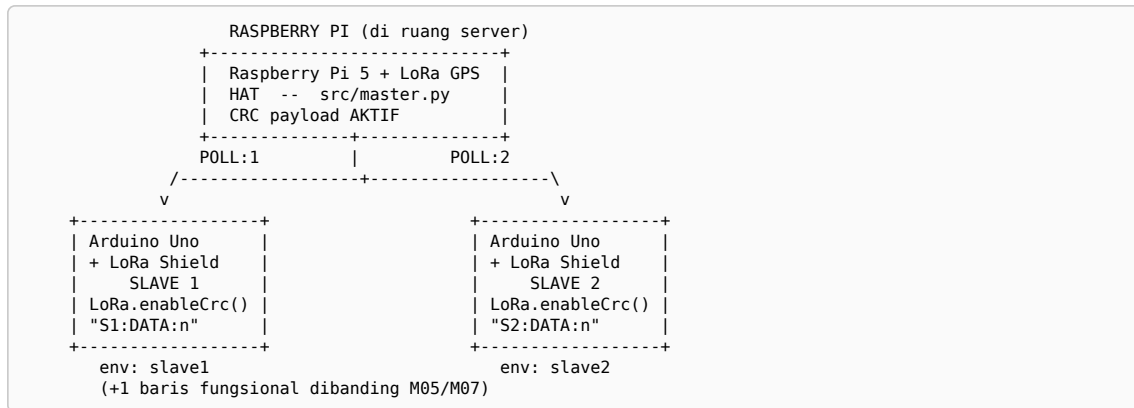
Istilah	Definisi kerja pada varian ini
CRC payload	Pemeriksaan keutuhan data oleh radio sendiri: 2 byte checksum ditambahkan ke tiap paket, diverifikasi otomatis oleh SX1276 penerima. AKTIF di kedua sisi pada varian ini (LoRa.enableCrc() di slave, enableCrc() di master.py) — bandingkan dengan Modul 07 (CRC mati) untuk melihat konsekuensinya.

Tabel 8.2: Istilah yang berubah pada Modul 07B

Mengapa anggaran POLL_TIMEOUT lebih sempit dari dugaan. Penjelasannya sama dengan Modul 07: transmit(pollMsg) bersifat blocking, sehingga waktu udara POLL sudah terbayar sebelum penghitung waktu mulai berjalan. Satu tambahan berlaku di sini: angka waktu udara pada Modul 07 berasal dari sesi CRC **mati**; menyalakan CRC menambah 2 byte checksum ke setiap paket sehingga sedikit memperpanjang waktu udara kedua arah — pengukuran aktual pada varian ini ada di logserial.md.

8.4 Topologi

Identik dengan Modul 07, termasuk pemetaan pin HAT (Tabel 7.5) dan peran tiap node (Tabel 7.3).



8.5 Alat yang Digunakan

Sama persis dengan Modul 07 (Tabel 7.4): satu Raspberry Pi 5 dengan Dragino LoRa GPS HAT v1.4, dua Arduino Uno dengan Dragino LoRa Shield v1.2, tiga antenna SMA, dan akses SSH ke Raspberry Pi.

Struktur proyek

```

Modul07b_rpi_master_slave_crc/
├── platformio.ini      ← hanya 2 environment: slave1, slave2
├── requirements.txt    ← spidev + RPi.GPIO0 (atau rpi-lgpio untuk Pi 5)
├── logserial.md        ← log referensi sesi verifikasi CRC
├── lora_monitor.py     ← dasbor 2 slave lokal + rekaman CSV (butuh `rich`)
├── cek_radio.py        ← baca balik register SX1276, termasuk bit CRC
├── upload_auto.py      ← deteksi port otomatis saat unggah slave
├── src/
│   ├── master.py       ← penjadwal round-robin + enableCrc()
│   └── slave/main.cpp  ← satu source untuk kedua slave, +LoRa.enableCrc()

```

8.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul07b_rpi_master_slave_crc/](#)

- [Modul07b_rpi_master_slave_crc/platformio.ini](#)
- [Modul07b_rpi_master_slave_crc/requirements.txt](#)
- [Modul07b_rpi_master_slave_crc/src/master.py](#)
- [Modul07b_rpi_master_slave_crc/src/slave/main.cpp](#)
- [Modul07b_rpi_master_slave_crc/cek_radio.py](#)
- [Modul07b_rpi_master_slave_crc/lora_monitor.py](#)
- [Modul07b_rpi_master_slave_crc/upload_auto.py](#)

```

; Modul 07 - hanya kedua SLAVE yang dibangun di sini.
;
; Master modul ini adalah Raspberry Pi yang menjalankan src/master.py; tidak
; ada environment PlatformIO untuknya, dan memang tidak perlu -- Python
; dijalankan langsung, tidak dikompilasi. Yang tersisa untuk PlatformIO
; hanyalah dua Arduino Uno di sisi slave.
;
;   pio run -d Modul07b_rpi_master_slave_crc -e slave1 -t upload
;   pio run -d Modul07b_rpi_master_slave_crc -e slave2 -t upload
;   python3 Modul07b_rpi_master_slave_crc/src/master.py      # di Raspberry Pi

```

```

[platformio]
default_envs = slave1

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

```

```

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.

```

```

; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk dua Uno asli.
; Modul ini memakai 115200 baud, sama seperti M05.
monitor_speed = 115200

; Kedua slave memakai source yang sama; nomornya ditentukan build flag.
[env:slave1]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=1
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

[env:slave2]
build_src_filter = +<slave/*.cpp>
build_flags = -DSLAVE_ID=2
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

```

Kode 8.1: platformio.ini Modul 07B — environment slave1 dan slave2

```

# Modul 07 - dependensi Raspberry Pi (jalankan DI Raspberry Pi, bukan di PC)
#
# pip3 install -r Modul07_rpi_master_slave/requirements.txt
#
# spidev : akses /dev/spidev0.0 untuk bicara ke SX1276
# RPi.GPIO: kendali jalur NSS, RESET, dan pembacaan DI00
#
# RASPBERRY PI 5. Chip GPIO baru (RP1) tidak didukung RPi.GPIO. Ganti baris
# RPi.GPIO di bawah dengan rpi-lgpio -- drop-in replacement dengan nama modul
# yang sama, sehingga `import RPi.GPIO as GPIO` di kode tetap berjalan apa
# adanya. Jangan memasang keduanya sekaligus: keduanya memakai nama paket
# yang sama dan akan saling menimpa.
spidev>=3.5
RPi.GPIO>=0.7.1
# rpi-lgpio>=0.6 ; khusus Raspberry Pi 5 -- pakai INI dan hapus RPi.GPIO di atas

```

Kode 8.2: requirements.txt Modul 07B — paket Python sisi Raspberry Pi

```

#!/usr/bin/env python3
"""Modul 07 - Master round-robin di Raspberry Pi (Dragino LoRa GPS HAT, SX1276).

Padanan Python dari Modul05_lora_master_slave/src/master/main.cpp: memanggil
Slave 1 dan Slave 2 bergiliran dengan batas waktu 500 ms per slave, lalu
mencetak statistik per node. Bedanya, kedua slave tetap Arduino Uno bershield
Dragino v1.2 -- master pindah platform, kontrak payload tidak berubah.

    POLL:<id>          master -> slave
    S<id>:DATA:<n>     slave  -> master

Driver SX1276 ditulis langsung via spidev + RPi.GPIO tanpa library LoRa, jadi
nama fungsinya sengaja dibuat sama dengan API sandeepmistry di Arduino agar
kedua sisi dapat dibandingkan baris per baris.

Pemetaan pin (Dragino LoRa GPS HAT - WiringPi -> BCM):
    LoRa_NSS -> WiringPi GPIO6 -> BCM GPIO 25
    RESET    -> WiringPi GPIO0 -> BCM GPIO 17
    DI00      -> WiringPi GPIO7 -> BCM GPIO 4
    SCK       -> WiringPi GPIO14 -> BCM GPIO 11 (SPI0 CLK)
    MOSI      -> WiringPi GPIO12 -> BCM GPIO 10 (SPI0 MOSI)
    MISO      -> WiringPi GPIO13 -> BCM GPIO 9  (SPI0 MISO)

Parameter radio harus IDENTIK dengan kedua slave: 433 MHz, SF7, BW 125 kHz,
CR 4/5. Upload kedua slave lebih dahulu, baru jalankan master.

Varian -withCRC: enableCrc()/disableCrc() menyalakan/mematikan bit RxPayloadCrcOn
(REG_MODEM_CONFIG_2 bit 2) -- padanan LoRa.enableCrc() sandeepmistry. Modul ini
memanggil enableCrc() saat init, sepasang dengan LoRa.enableCrc() di slave, jadi
paket berpayload rusak dibuang radio di kedua sisi, tidak lagi lolos ke aplikasi.

Prasyarat:
    sudo raspi-config          # Interface Options > SPI > Yes, lalu reboot
    pip3 install -r Modul07_rpi_master_slave/requirements.txt
"""

```

```

import argparse
import time
import sys
import spidev
import RPi.GPIO as GPIO

# --- Pin configuration (BCM numbering) ---
NSS_PIN = 25 # LoRa_NSS (WiringPi GPIO6 = BCM GPIO 25)
RST_PIN = 17 # RESET (WiringPi GPIO0 = BCM GPIO 17)
DIO0_PIN = 4 # DIO0 (WiringPi GPIO7 = BCM GPIO 4)

# --- LoRa parameters ---
FREQUENCY = 433E6
BANDWIDTH = 125E3
SPREADING_FACTOR = 7
CODING_RATE = 5
TX_POWER = 17

POLL_TIMEOUT = 500 # ms menunggu balasan tiap slave
CYCLE_INTERVAL = 500 # ms jeda antar siklus
SLAVE_COUNT = 2

# --- SX1276 Register Map ---
REG_FIFO = 0x00
REG_OP_MODE = 0x01
REG_FRF_MSB = 0x06
REG_FRF_MID = 0x07
REG_FRF_LSB = 0x08
REG_PA_CONFIG = 0x09
REG_FIFO_ADDR_PTR = 0x0D
REG_FIFO_TX_BASE_ADDR = 0x0E
REG_FIFO_RX_BASE_ADDR = 0x0F
REG_FIFO_RX_CURRENT_ADDR = 0x10
REG_IRQ_FLAGS = 0x12
REG_RX_NB_BYTES = 0x13
REG_PKT_RSSI_VALUE = 0x1A
REG_PKT_SNR_VALUE = 0x1B
REG_MODEM_CONFIG_1 = 0x1D
REG_MODEM_CONFIG_2 = 0x1E
REG_PREAMBLE_MSB = 0x20
REG_PREAMBLE_LSB = 0x21
REG_PAYLOAD_LENGTH = 0x22
REG_VERSION = 0x42

# --- Operating modes ---
MODE_LONG_RANGE = 0x80
MODE_SLEEP = 0x00
MODE_STDBY = 0x01
MODE_TX = 0x03
MODE_RX_CONTINUOUS = 0x05

# --- IRQ flags ---
IRQ_RX_DONE = 0x40
IRQ_TX_DONE = 0x08
IRQ_CRC_ERROR = 0x20

# --- PA config ---
PA_BOOST = 0x80

# --- SPI instance ---
_spi = spidev.SpiDev()

#
# Low-level SPI register access
#
def _read_reg(addr):
    GPIO.output(NSS_PIN, GPIO.LOW)
    val = _spi.xfer2([addr & 0x7F, 0x00])[1]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return val

def _write_reg(addr, value):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([addr | 0x80, value & 0xFF])
    GPIO.output(NSS_PIN, GPIO.HIGH)

#
# LoRa mode & frequency

```

```

# =====
def _set_mode(mode):
    _write_reg(REG_OP_MODE, MODE_LONG_RANGE | mode)

def _set_frequency(freq):
    frf = int(round(freq / 32e6 * (1 << 19)))
    _write_reg(REG_FRF_MSB, (frf >> 16) & 0xFF)
    _write_reg(REG_FRF_MID, (frf >> 8) & 0xFF)
    _write_reg(REG_FRF_LSB, frf & 0xFF)

# =====
# LoRa configuration (mirip API sandeepmistry)
# =====
def setSpreadingFactor(sf):
    sf = max(6, min(12, sf))
    _write_reg(REG_MODEM_CONFIG_2,
        (_read_reg(REG_MODEM_CONFIG_2) & 0x0F) | ((sf << 4) & 0xF0))

def setSignalBandwidth(bw):
    bw_table = [7.8e3, 10.4e3, 15.6e3, 20.8e3, 31.25e3, 41.7e3, 62.5e3, 125e3, 250e3]
    idx = min(range(len(bw_table)), key=lambda i: abs(bw_table[i] - bw))
    _write_reg(REG_MODEM_CONFIG_1,
        (_read_reg(REG_MODEM_CONFIG_1) & 0x0F) | (idx << 4))

def setCodingRate4(denominator):
    cr = max(5, min(8, denominator)) - 4
    _write_reg(REG_MODEM_CONFIG_1,
        (_read_reg(REG_MODEM_CONFIG_1) & 0xF1) | (cr << 1))

def setTxPower(level):
    level = max(2, min(17, level))
    _write_reg(REG_PA_CONFIG, PA_BOOST | (level - 2))

def enableCrc():
    _write_reg(REG_MODEM_CONFIG_2, _read_reg(REG_MODEM_CONFIG_2) | 0x04)

def disableCrc():
    _write_reg(REG_MODEM_CONFIG_2, _read_reg(REG_MODEM_CONFIG_2) & 0xFB)

# =====
# TX functions (mirip LoRa.beginPacket / print / endPacket)
# =====
def beginPacket():
    _set_mode(MODE_STDBY)
    _write_reg(REG_MODEM_CONFIG_1, _read_reg(REG_MODEM_CONFIG_1) & 0xFE)
    _write_reg(REG_FIFO_ADDR_PTR, 0x00)
    _write_reg(REG_PAYLOAD_LENGTH, 0x00)

def loraPrint(data):
    if isinstance(data, str):
        data = data.encode('ascii')
    current_len = _read_reg(REG_PAYLOAD_LENGTH)
    size = min(len(data), 255 - current_len)
    for byte in data[:size]:
        _write_reg(REG_FIFO, byte)
        _write_reg(REG_PAYLOAD_LENGTH, current_len + size)

def endPacket():
    _set_mode(MODE_TX)
    while (_read_reg(REG_IRQ_FLAGS) & IRQ_TX_DONE) == 0:
        time.sleep(0.001)
    _write_reg(REG_IRQ_FLAGS, IRQ_TX_DONE)

def transmit(msg):
    beginPacket()
    loraPrint(msg)
    endPacket()

# =====
# RX functions (mirip LoRa.parsePacket / available / read)

```

```

# =====
def parsePacket():
    irq = _read_reg(REG_IRQ_FLAGS)
    if irq & IRQ_CRC_ERROR:
        _write_reg(REG_IRQ_FLAGS, IRQ_CRC_ERROR)
        return 0
    if (irq & IRQ_RX_DONE) == 0:
        return 0
    _write_reg(REG_IRQ_FLAGS, IRQ_RX_DONE)
    return _read_reg(REG_RX_NB_BYTES)

def loraAvailable():
    return _read_reg(REG_RX_NB_BYTES)

def loraRead():
    return _read_reg(REG_FIFO)

def packetRssi():
    rssi = _read_reg(REG_PKT_RSSI_VALUE)
    freq_mhz = FREQUENCY / 1e6
    if freq_mhz < 868:
        return rssi - 164
    else:
        return rssi - 157

def packetSnr():
    val = _read_reg(REG_PKT_SNR_VALUE)
    if val > 127:
        val = val - 256
    return val / 4.0

def startRx():
    _set_mode(MODE_RX_CONTINUOUS)

def initRxSequence():
    _write_reg(REG_FIFO_ADDR_PTR, _read_reg(REG_FIFO_RX_CURRENT_ADDR))

# =====
# Init & cleanup
# =====
def loraBegin(frequency):
    GPIO.setwarnings(False)
    GPIO.setmode(GPIO.BCM)
    GPIO.setup(NSS_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(RST_PIN, GPIO.OUT, initial=GPIO.HIGH)
    GPIO.setup(DI00_PIN, GPIO.IN)

    _spi.open(0, 0)
    _spi.max_speed_hz = 5_000_000
    _spi.mode = 0b00

    GPIO.output(RST_PIN, GPIO.LOW)
    time.sleep(0.01)
    GPIO.output(RST_PIN, GPIO.HIGH)
    time.sleep(0.01)

    version = _read_reg(REG_VERSION)
    if version != 0x12:
        return False

    _set_mode(MODE_SLEEP)
    _set_frequency(frequency)
    _write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
    _write_reg(REG_FIFO_RX_BASE_ADDR, 0x00)
    _set_mode(MODE_STDBY)
    return True

def loraEnd():
    _set_mode(MODE_SLEEP)
    _spi.close()
    GPIO.cleanup()

# =====

```

```

# Master logic: poll slave (identik dengan pollSlave() di master.ino)
#
def pollSlave(slaveNum, okRef, failRef):
    pollMsg = f"POLL:{slaveNum}"
    print(f"[TX] {pollMsg}")
    transmit(pollMsg)

    startRx()

    waitStart = time.monotonic()
    while (time.monotonic() - waitStart) * 1000 < POLL_TIMEOUT:
        ps = parsePacket()
        if ps > 0:
            initRxSequence()
            reply = bytes(loraRead() for _ in range(ps)).decode('ascii', errors='replace')

            prefix = f"S{slaveNum}:DATA:"
            if reply.startswith(prefix):
                try:
                    dataOut = int(reply[len(prefix):])
                except ValueError:
                    dataOut = 0
                okRef[0] += 1
                rssi = packetRssi()
                snr = packetSnr()
                print(f"[RX] {reply} | RSSI: {rssi:.0f} dBm | SNR: {snr:.1f} dB")
                return dataOut
            else:
                print(f"[WARN] Balasan tidak valid: {reply}")

        failRef[0] += 1
        print(f"[FAIL] Slave {slaveNum} tidak merespon!")
        return None

#
# Main (identik dengan setup() + loop() di master.ino)
#
def build_arg_parser():
    parser = argparse.ArgumentParser(
        prog="master.py",
        description=(
            "Master round-robin M07 (Raspberry Pi + Dragino LoRa GPS HAT). "
            "Memanggil Slave 1 lalu Slave 2 bergiliran tanpa henti, mencetak "
            "statistik OK/FAIL per node setiap siklus. Tidak ada opsi lain -- "
            "seluruh parameter (frekuensi, SF, BW, timeout) ditetapkan sebagai "
            "konstanta di berkas ini agar tetap identik dengan slave M05."
        ),
        epilog=(
            f"Parameter radio: {FREQUENCY / 1e6:.0f} MHz, SF{SPREADING_FACTOR}, "
            f"BW {BANDWIDTH / 1e3:.0f} kHz, CR 4/5, {TX_POWER} dBm. "
            f"POLL_TIMEOUT={POLL_TIMEOUT} ms, CYCLE_INTERVAL={CYCLE_INTERVAL} ms. "
            "Hentikan dengan Ctrl-C."
        ),
        formatter_class=argparse.RawDescriptionHelpFormatter,
    )
    return parser

if __name__ == '__main__':
    build_arg_parser().parse_args()

    cycle = 0
    s1Ok = [0]; s1Fail = [0]; s1Data = 0
    s2Ok = [0]; s2Fail = [0]; s2Data = 0

    print("=== LoRa MASTER-SLAVE 3 NODE ===")
    print("Init LoRa ... ", end="", flush=True)

    if not loraBegin(FREQUENCY):
        print("GAGAL! Cek kabel/modul.")
        print("Pastikan SPI aktif: sudo raspi-config > Interfacing Options > SPI")
        sys.exit(1)

    setSpreadingFactor(SPREADING_FACTOR)
    setSignalBandwidth(BANDWIDTH)
    setCodingRate4(CODING_RATE)
    setTxPower(TX_POWER)
    enableCrc()

    print("OK")
    print(f"Freq: {FREQUENCY / 1e6:.0f} MHz")

```

```

print(f"SF{SPREADING_FACTOR} | BW: {BANDWIDTH / 1e3:.0f} kHz")
print("CRC payload: AKTIF")
print("Peran: MASTER (Raspberry Pi + LoRa GPS HAT)")
print("Slave: Dragino Shield Uno - S1 & S2\n")

try:
    while True:
        cycle += 1
        cycleStartMs = time.monotonic()

        print("=" * 40)
        print(f"=== CYCLE {cycle} ===")

        # --- Poll Slave 1 ---
        s1Data = pollSlave(1, s10k, s1Fail)

        # --- Poll Slave 2 ---
        s2Data = pollSlave(2, s20k, s2Fail)

        # --- Statistik ---
        duration = (time.monotonic() - cycleStartMs) * 1000
        print("--- STATISTIK ---")
        print(f"S1: OK={s10k[0]} | FAIL={s1Fail[0]} | Data: {s1Data}")
        print(f"S2: OK={s20k[0]} | FAIL={s2Fail[0]} | Data: {s2Data}")
        print(f"Durasi siklus: {duration:.0f} ms")
        print("=" * 40 + "\n")

        time.sleep(CYCLE_INTERVAL / 1000.0)

except KeyboardInterrupt:
    print("\nProgram dihentikan oleh pengguna.")
finally:
    loraEnd()
    print("Menutup LoRa module...")

```

Kode 8.3: src/master.py — penjadwal round-robin dengan enableCrc()

```

/*
LoRa Master-Slave hybrid - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: slave1 / slave2

Sisi slave Modul 07, varian -withCRC. Basisnya identik dengan slave Modul
05/07 asli -- yang berganti platform hanya master (Arduino Uno -> Raspberry
Pi + Python) -- ditambah satu baris: LoRa.enableCrc(), yang mengaktifkan
pemeriksaan CRC payload radio di kedua sisi (lihat juga src/master.py).
Paket yang payload-nya rusak di udara sekarang dibuang oleh radio sendiri
sebelum sempat naik ke parsePacket(), bukan lagi lolos ke lapisan aplikasi.

Slave N hanya merespon ketika Master mengirim "POLL:N".
Balasan: "SN:DATA:n" -- n adalah counter lokal yang naik setiap kali menjawab.

Mekanisme:
TX : blocking (endPacket) -- sederhana dan andal di AVR
RX : polling via LoRa.parsePacket() -- tidak perlu ISR

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DIO0 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DIO0_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor slave datang dari build flag (-DSLAVE_ID=1 atau 2) di platformio.ini,
// sehingga kedua slave memakai file source yang sama persis.

```



```

#ifndef SLAVE_ID
#define SLAVE_ID 1
#endif

unsigned long dataCounter = 0; // unsigned long: aman hingga 4.294.967.295 (vs int yang overflow di 32.767)
int rxCount = 0;

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    Serial.print(F("=== LoRa SLAVE "));
    Serial.print(SLAVE_ID);
    Serial.println(F(" =="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.enableCrc(); // varian -withCRC: paket berpayload rusak dibuang radio, tidak lolos ke parsePacket()

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.println(F("CRC payload: AKTIF"));
    Serial.print(F("Menunggu POLL:"));
    Serial.print(SLAVE_ID);
    Serial.println(F(" dari Master (Raspberry Pi)...\n"));
}

void loop() {
    int ps = LoRa.parsePacket();
    if (ps == 0) return;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    if (!received.equals("POLL:" + String(SLAVE_ID))) {
        Serial.print(F("[IGNORE ]")); Serial.println(received);
        return;
    }

    rxCount++;
    // dataCounter++ sengaja dipindah ke SETELAH transmit()
    // Filosofi: counter merepresentasikan "berapa kali slave BERHASIL kirim ke master"
    // bukan "berapa kali slave berniat kirim" – kalau TX gagal, counter tidak berhak naik
    digitalWrite(LED_PIN, HIGH);

    Serial.print(F("[RX] ")); Serial.print(received);
    Serial.print(F(" | RSSI: ")); Serial.print(LoRa.packetRssi());
    Serial.print(F(" dBm | SNR: ")); Serial.print(LoRa.packetSnr());
    Serial.print(F(" dB | RX#: ")); Serial.println(rxCount);

    dataCounter++; // naik SETELAH TX – nilai ini yang dikirim, mencerminkan jumlah TX aktual
    String reply = "S" + String(SLAVE_ID) + ":DATA:" + String(dataCounter);
    Serial.print(F("[TX] ")); Serial.println(reply);

    transmit(reply);
    digitalWrite(LED_PIN, LOW);
    Serial.println();
}

```

Kode 8.4: src/slave/main.cpp — firmware slave dengan LoRa.enableCrc()

```

"""Baca BALIK register SX1276 di master setelah loraBegin()+konfigurasi.
Yang dilaporkan adalah isi chip, bukan konstanta di source."""
import sys
sys.path.insert(0, 'src')
import master as M

assert M.loraBegin(M.FREQUENCY), "loraBegin GAGAL"
M.setSpreadingFactor(M.SPREADING_FACTOR)
M.setSignalBandwidth(M.BANDWIDTH)
M.setCodingRate4(M.CODING_RATE)
M.setTxPower(M.TX_POWER)
M.enableCrc()

msb = M._read_reg(M.REG_FRF_MSB); mid = M._read_reg(M.REG_FRF_MID); lsb = M._read_reg(M.REG_FRF_LSB)
frf = (msb << 16) | (mid << 8) | lsb
freq = frf * 32e6 / (1 << 19)

print(f"REG_VERSION : 0x{M._read_reg(M.REG_VERSION):02x} (0x12 = SX1276/77/78/79)")
print(f"FRF register : 0x{msb:02x} 0x{mid:02x} 0x{lsb:02x} -> frf={frf}")
print(f"FREKUENSI : {freq/1e6:.6f} MHz (target 433.000000, selisih {freq-433e6:+.1f} Hz)")

mc1 = M._read_reg(M.REG_MODEM_CONFIG_1); mc2 = M._read_reg(M.REG_MODEM_CONFIG_2)
bw_tab = ["7.8", "10.4", "15.6", "20.8", "31.25", "41.7", "62.5", "125", "250", "500"]
print(f"MODEM_CFG_1 : 0x{mc1:02x} -> BW={bw_tab[mc1>>4]} kHz | CR=4/({mc1>>1}&0x07)+4 |
header={'implicit' if mc1&1 else 'explicit'}")
print(f"MODEM_CFG_2 : 0x{mc2:02x} -> SF{mc2>>4} | CRC payload={'AKTIF' if mc2&0x04 else 'MATI'}")
print(f"PA_CONFIG : 0x{M._read_reg(M.REG_PA_CONFIG):02x} -> PA_B00ST,
power=({M._read_reg(M.REG_PA_CONFIG)&0x0F)+2} dBm")
M.loraEnd()

```

Kode 8.5: cek_radio.py — baca balik register SX1276 termasuk bit CRC

```

#!/usr/bin/env python3
"""
LoRa Monitor Dashboard - Sisi Slave (Modul 07)
=====
Dashboard Live untuk memantau KEDUA Arduino Uno slave dari komputer ini.

Bedanya dengan lora_monitor.py pada M05: di modul ini master adalah Raspberry
Pi, dan Pi tidak menyalurkan keluarannya lewat USB ke komputer ini. Jadi tidak
ada panel MASTER yang bisa dibaca langsung. Yang tersedia hanya dua port serial
slave -- dan ternyata itu sudah cukup untuk menyimpulkan hampir seluruh keadaan
master, karena setiap slave MENDENGAR seluruh lalu lintas di udara:

[RX] POLL:1      <- perintah master untuk dirinya
[IGNORE] POLL:2  <- perintah master untuk slave sebelah
[IGNORE] S2:DATA:12 <- jawaban slave sebelah

Dari tiga jenis baris itu, monitor menurunkan:
- periode siklus master, diukur dari jarak antar POLL yang diterima node
  (padanan "Durasi siklus" di master, dilihat dari tepi jaringan)
- keadaan slave sebelah, dari ada/tidaknya jawaban setelah POLL-nya terdengar
- RSSI/SNR arah master -> slave, yang TIDAK terlihat di terminal master
  (master hanya melaporkan arah sebaliknya, slave -> master)

Kebutuhan:
pip install pyserial rich

Cara pakai:
python3 lora_monitor.py # port bawaan
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
python3 lora_monitor.py --out data_eksperimen.csv

Jalankan tools/deteksi_port.py untuk mengetahui port mana milik board yang mana.
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

```

```

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.table import Table
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# -----
# KONFIGURASI
# -----
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py lalu berikan lewat
# --s1/--s2 bila port di komputer Anda berbeda.
if sys.platform.startswith("win"):
    DEFAULT_S1_PORT = "COM4"
    DEFAULT_S2_PORT = "COM5"
else:
    DEFAULT_S1_PORT = "/dev/ttyACM0"
    DEFAULT_S2_PORT = "/dev/ttyACM1"
DEFAULT_BAUD = 115200 # M07 memakai 115200, sama seperti M05
LOG_LINES = 16
REFRESH_HZ = 4
PERIODE_WINDOW = 60 # jumlah periode terakhir untuk min/maks/rerata
MISS_WARN = 2 # lompatan RX# yang dianggap POLL hilang
PASANGAN_WARN = 3 # siklus berturut tanpa jawaban slave sebelah

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default s1/s2 selalu mengikuti apa
    yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

# Bentuk payload yang sah menurut kontrak M05/M07. Apa pun di luar dua pola ini
# adalah byte yang rusak di udara. Varian -withCRC ini MENGAKTIFKAN CRC di kedua
# sisi (LoRa.enableCrc() di slave, enableCrc() di master.py), sehingga paket
# cacat semestinya sudah dibuang radio sebelum sempat terdengar di sini sama
# sekali -- baris RUSAK yang tetap muncul berarti sesuatu lolos dari filter
# CRC atau kerusakan terjadi setelah decode, layak diselidiki lebih lanjut.
RE_POLL_SAH = re.compile(r'^POLL:\d+$')
RE_DATA_SAH = re.compile(r'^S\d+:DATA:\d+$')

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class SlaveState:
    name: str = ""
    port: str = ""
    slave_id: int = 0
    connected: bool = False
    rx_count: int = 0 # RX# dari firmware: POLL untuk dirinya
    data_counter: int = 0 # counter yang dikirim balik ke master
    last_rssi: int = 0
    last_snr: float = 0.0

```

```

    rssi_min:    int = 0
    rssi_max:    int = 0
    ignore_poll: int = 0 # POLL milik slave sebelah yang ikut terdengar
    ignore_reply: int = 0 # jawaban slave sebelah yang ikut terdengar
    rusak:       int = 0 # payload yang tidak memenuhi kontrak
    miss:        int = 0 # POLL untuk dirinya yang tidak sampai
    # periode antar POLL untuk dirinya = periode siklus master dari sisi node
    periode:     deque = field(default_factory=lambda: deque(maxlen=PERIODE_WINDOW))
    _t_poll:     float = 0.0
    # menunggu jawaban slave sebelah setelah POLL-nya terdengar
    _nunggu_pasangan: bool = False
    pasangan_sepi: int = 0
    raw_log:     deque = field(default_factory=lambda: deque(maxlen=200))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.s1 = SlaveState("SLAVE 1", DEFAULT_S1_PORT, 1)
        self.s2 = SlaveState("SLAVE 2", DEFAULT_S2_PORT, 2)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSER
# -----

def parse_slave_line(line: str, st: SlaveState, anomalies: deque) -> dict:
    """Perbarui state dari satu baris serial, kembalikan baris CSV terstruktur."""
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    now = time.monotonic()
    row = {'event': 'LAIN', 'rssi': '', 'snr': '', 'rx': '', 'data': '',
          'periode_ms': '', 'detail': ''}

    # — POLL untuk dirinya sendiri -----
    m = re.search(r'\\[RX]\\s+POLL:(\\d+)\\s*\\s*RSSI:\\s*(-?\\d+)\\s*dBm\\s*\\s*'
                  r'SNR:\\s*(-?[\\d.]+)\\s*dB\\s*\\s*RX#:\\s*(\\d+)', line)
    if m:
        rssi, snr, rx = int(m.group(2)), float(m.group(3)), int(m.group(4))

        # Jarak antar POLL = satu putaran penuh master. Diukur di sini, bukan
        # di master, sehingga sebaran yang terlihat sudah termasuk jitter
        # penjadwalan Linux + waktu udara -- persis yang ditanyakan EXP-02.
        if st._t_poll:
            periode = (now - st._t_poll) * 1000.0
            st.periode.append(periode)
            row['periode_ms'] = f"{periode:.0f}"
        st._t_poll = now

        if st.rx_count and rx > st.rx_count + 1:
            hilang = rx - st.rx_count - 1
            st.miss += hilang
            if hilang >= MISS_WARN:
                anomalies.append(Anomaly(ts, st.name, "POLL_HILANG",
                                         f"RX# loncat {st.rx_count} -> {rx} ({hilang} POLL tidak sampai)"))

        st.rx_count = rx
        st.last_rssi = rssi
        st.last_snr = snr
        st.rssi_min = rssi if not st.rssi_min else min(st.rssi_min, rssi)
        st.rssi_max = rssi if not st.rssi_max else max(st.rssi_max, rssi)
        row.update(event='RX_POLL', rssi=rssi, snr=snr, rx=rx)
        return row

    # — jawaban yang dikirim ke master -----
    m = re.search(r'\\[TX]\\s+S(\\d+):DATA:(\\d+)', line)
    if m:
        st.data_counter = int(m.group(2))
        row.update(event='TX_DATA', data=st.data_counter)
        return row

    # — lalu lintas milik node lain yang ikut terdengar -----
    m = re.search(r'\\[IGNORE]\\s+(.*)$', line)
    if m:
        isi = m.group(1).strip()

        if RE_POLL_SAH.match(isi):
            st.ignore_poll += 1
            # master sedang memanggil slave sebelah; jawabannya ditunggu

```

```

        st._nunggu_pasangan = True
        row.update(event='IGNORE_POLL', detail=isi)
        return row

    if RE_DATA_SAH.match(isi):
        st.ignore_reply += 1
        if st._nunggu_pasangan:
            st._nunggu_pasangan = False
            st.pasangan_sepi = 0
        row.update(event='IGNORE_REPLY', detail=isi)
        return row

    # Tidak cocok kontrak: byte rusak di udara. Inilah yang di terminal
    # master muncul sebagai "[WARN] Balasan tidak valid" lalu dihitung
    # sebagai FAIL, seolah slave tidak menjawab.
    st.rusak += 1
    anomalies.append(Anomaly(ts, st.name, "PAYLOAD_RUSAK",
        f"payload di luar kontrak: {isi!r}"))
    row.update(event='RUSAK', detail=isi)
    return row

if 'Init LoRa' in line or '=== LoRa SLAVE' in line:
    row.update(event='INIT', detail=line.strip())
    return row

def tutup_siklus(st: SlaveState, anomalies: deque):
    """Dipanggil saat POLL untuk diri sendiri tiba: nilai siklus sebelumnya.

    Bila POLL slave sebelah sempat terdengar tetapi jawabannya tidak pernah
    muncul, slave sebelah sedang mati atau di luar jangkauan -- keadaan EXP-03.
    """
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
    if st._nunggu_pasangan:
        st._nunggu_pasangan = False
        st.pasangan_sepi += 1
        if st.pasangan_sepi == PASANGAN_WARN:
            lawan = 2 if st.slave_id == 1 else 1
            anomalies.append(Anomaly(ts, st.name, "PASANGAN_HILANG",
                f"POLL:{lawan} terdengar {PASANGAN_WARN}x tanpa jawaban - "
                f"slave {lawan} mati atau di luar jangkauan"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str, shared: SharedState):
    state = shared.s1 if node_key == "S1" else shared.s2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                # siklus dinilai tepat sebelum POLL berikutnya diproses
                                if '[RX]' in line and 'POLL:' in line:
                                    tutup_siklus(state, shared.anomalies)
                                    row = parse_slave_line(line, state, shared.anomalies)
                                    row.update(ts=ts, node=node_key, raw=line)
                                    shared.csv_queue.put(row)
                        except (serial.SerialException, OSError):
                            break
                    except (serial.SerialException, OSError):
                        with shared.lock:
                            state.connected = False
                            time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    kolom = ['timestamp', 'node', 'event', 'rssi_dbm', 'snr_db',

```

```

        'rx_count', 'data', 'periode_ms', 'detail', 'raw_line']
with open(filepath, 'w', newline='', encoding='utf-8') as f:
    w = csv.DictWriter(f, fieldnames=kolom)
    w.writeheader()
    while shared.running or not shared.csv_queue.empty():
        try:
            r = shared.csv_queue.get(timeout=0.5)
            w.writerow({'timestamp': r['ts'], 'node': r['node'],
                        'event': r['event'], 'rssi_dbm': r['rssi'],
                        'snr_db': r['snr'], 'rx_count': r['rx'],
                        'data': r['data'], 'periode_ms': r['periode_ms'],
                        'detail': r['detail'], 'raw_line': r['raw']})
            f.flush()
        except queue.Empty:
            continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def log_line_style(line: str) -> str:
    if '[RX]' in line: return "green"
    if '[TX]' in line: return "yellow"
    if '[IGNORE]' in line: return "dim"
    if 'GAGAL' in line: return "bold red"
    if '===' in line: return "bold cyan"
    return "dim white"

def build_slave_panel(st: SlaveState, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" POLL diterima : ", style="dim")
    t.append(f"{st.rx_count:>7}\n", style="bold white")
    t.append(" Jawaban dikirim: ", style="dim")
    t.append(f"{st.data_counter:>7}\n", style="bold green")
    t.append(" POLL hilang : ", style="dim")
    t.append(f"{st.miss:>7}\n", style="bold red" if st.miss else "dim")
    t.append(" Payload rusak : ", style="dim")
    t.append(f"{st.rusak:>7}\n\n", style="bold red" if st.rusak else "dim")

    # Arah master -> slave. Terminal master hanya melaporkan arah sebaliknya,
    # jadi angka ini tidak ada duanya di tempat lain.
    t.append(" Master -> node\n", style="bold cyan")
    t.append(" RSSI : ", style="dim")
    t.append(f"{st.last_rssi:>6} dBm\n", style="bold white")
    t.append(" rentang: ", style="dim")
    t.append(f"{st.rssi_min}..{st.rssi_max} dBm\n" if st.rssi_min else "-\n",
            style="white")
    t.append(" SNR : ", style="dim")
    t.append(f"{st.last_snr:>6.2f} dB\n\n", style="bold white")

    t.append(" Periode siklus\n", style="bold cyan")
    if st.periode:
        p = list(st.periode)
        rata = sum(p) / len(p)
        t.append(" min/rata/maks: ", style="dim")
        t.append(f"{min(p):.0f} / {rata:.0f} / {max(p):.0f} ms\n", style="bold white")
        t.append(" sebaran : ", style="dim")
        t.append(f"{max(p) - min(p):.0f} ms\n\n", style="bold yellow")
    else:
        t.append(" menunggu POLL kedua...\n\n", style="dim")

    t.append(" Terdengar dari node lain\n", style="bold cyan")
    t.append(" POLL / jawaban: ", style="dim")
    t.append(f"{st.ignore_poll} / {st.ignore_reply}\n\n", style="white")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 34 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        t.append(f" {ts[3:]} {line[:40]}\n", style=log_line_style(line))

```

```

        return Panel(t, title=f"[bold {color}]{st.name}[/]",
                     border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                       csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'POLL_HILANG': "bold yellow",
        'PAYLOAD_RUSAK': "bold white on red",
        'PASANGAN_HILANG': "bold red",
    }

    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=kind_style.get(a.kind, "white"))
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                 border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        # snapshot biar tidak menahan lock saat render
        snap = []
        for src in (shared.s1, shared.s2):
            snap.append(SlaveState(
                name=src.name, port=src.port, slave_id=src.slave_id,
                connected=src.connected, rx_count=src.rx_count,
                data_counter=src.data_counter, last_rssi=src.last_rssi,
                last_snr=src.last_snr, rssi_min=src.rssi_min,
                rssi_max=src.rssi_max, ignore_poll=src.ignore_poll,
                ignore_reply=src.ignore_reply, rusak=src.rusak, miss=src.miss,
                periode=deque(src.periode), raw_log=deque(src.raw_log),
            ))
        anom = deque(shared.anomalies)
        t_start = shared.session_start

    # Grid dua kolom berbagi rata: dengan hanya dua panel, Columns membiarkan
    # keduanya selebar isinya saja dan separuh terminal terbuang.
    node_row = Table.grid(expand=True)
    node_row.add_column(ratio=1)
    node_row.add_column(ratio=1)
    node_row.add_row(build_slave_panel(snap[0], "green"),
                     build_slave_panel(snap[1], "magenta"))

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\

    LoRa Monitor Dashboard - Sisi Slave (Modul 07)
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz

"""

Master modul ini adalah Raspberry Pi, jadi terminalnya tidak lewat USB
ke komputer ini. Monitor membaca kedua slave saja - dan menyimpulkan
keadaan master dari lalu lintas yang ikut terdengar oleh node.

PENGGUNAAN:

```

```
python3 lora_monitor.py [opsi...]

OPSI:
--s1  PORT    Port serial Slave 1          (default: {s1})
--s2  PORT    Port serial Slave 2          (default: {s2})
--baud N      Baud rate serial kedua port  (default: {baud})
--out FILE    Nama file CSV output         (default: lora_slaves_YYYYMMDD_HHMMSS.csv)
-h / --help   Tampilkan pesan bantuan ini

CONTOH:
python3 lora_monitor.py
-> Jalankan dengan port & baud rate default

python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
-> Tentukan port secara eksplisit (Windows: COM4, COM5)

python3 lora_monitor.py --out data_jarak_10m.csv
-> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
Ctrl+C      Hentikan monitoring & simpan CSV

CATATAN:
• Kedua slave harus sudah di-upload firmware dengan baud {baud}
• Jalankan tools/deteksi_port.py bila ragu port mana milik board mana
• Tutup Serial Monitor PlatformIO dulu - satu port hanya boleh dibuka oleh satu program
• Master di Raspberry Pi dijalankan terpisah:
  python3 Modul07_rpi_master_slave/src/master.py
"""
.format(s1=DEFAULT_S1_PORT, s2=DEFAULT_S2_PORT, baud=DEFAULT_BAUD)

def parse_args(port_aktif):
    s1_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_S1_PORT
    s2_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_S2_PORT

    p = argparse.ArgumentParser(
        description="LoRa Monitor Dashboard sisi slave - tool pemantauan praktikum M07.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            "  python3 lora_monitor.py\n"
            "  python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1\n"
            "  python3 lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--s1', default=s1_default, metavar='PORT',
        help=f'Port serial Slave 1 (default: {s1_default}, dari deteksi port aktif)')
    p.add_argument('--s2', default=s2_default, metavar='PORT',
        help=f'Port serial Slave 2 (default: {s2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int, metavar='N',
        help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None, metavar='FILE',
        help='Nama file CSV output (default: auto timestamp)')
    return p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1
    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
    )
    console.print(
        f"[dim]Dipakai -> "
        f"S1=[green]{args.s1}[/] S2=[magenta]{args.s2}[/] "
        f"Baud=[white]{args.baud}[/][dim]\n"
    )

    shared = SharedState()
    shared.s1.port = args.s1
    shared.s2.port = args.s2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_slaves_{ts_str}.csv"
```



```

if not no_args:
    console.print("\n[bold cyan]LoRa Monitor Dashboard - Sisi Slave[/]")
    console.print(f" Slave1 : [green]{args.s1}[/]")
    console.print(f" Slave2 : [magenta]{args.s2}[/]")
    console.print(f" Baud   : [white]{args.baud}[/]")

    console.print(f" CSV       : [dim]{csv_path}[/]")
    console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:
        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread, args=(args.s1, args.baud, "S1", shared),
                        daemon=True, name="reader-s1"),
        threading.Thread(target=reader_thread, args=(args.s2, args.baud, "S2", shared),
                        daemon=True, name="reader-s2"),
        threading.Thread(target=csv_writer_thread, args=(shared, csv_path),
                        daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path), console=console,
                 refresh_per_second=REFRESH_HZ, screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass
    finally:
        shared.running = False
        time.sleep(0.6)
        console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 8.6: lora_monitor.py — dasbor dua slave lokal dengan perekaman CSV

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 07 (RPI Master-Slave) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul07b_rpi_master_slave_crc -e slave1 -t upload -t monitor
    pio run -d Modul07b_rpi_master_slave_crc -e slave2 -t upload -t monitor

Master modul ini adalah Raspberry Pi yang menjalankan src/master.py secara
langsung (Python, tidak dikompilasi) -- TIDAK ADA environment PlatformIO
untuknya, jadi skrip ini hanya meng-upload kedua slave.

Skrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `slave1` (port pertama) dan `slave2` (port kedua), lalu
menjalankan `pio run` dengan --upload-port sesuai hasil deteksi --
platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah kedua
slave selesai di-upload, skrip bertanya apakah mau langsung menjalankan
lora_monitor.py -- bila ya, dijalankan dengan --s1 ... --s2 ... persis
memakai port yang barusan dipakai upload (bukan deteksi ulang), supaya
tidak mungkin salah pasang. Master tetap harus dijalankan terpisah di
Raspberry Pi: python3 Modul07b_rpi_master_slave_crc/src/master.py

python3 Modul07b_rpi_master_slave_crc/upload_auto.py
-> upload kedua slave, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul07b_rpi_master_slave_crc/upload_auto.py --monitor
-> upload lalu buka pio device monitor untuk slave1, lanjut slave2
(Ctrl-C untuk keluar dari tiap monitor sebelum lanjut ke board
berikutnya) -- pemantauan satu board per satu waktu, bukan
sekaligus seperti lora_monitor.py

python3 Modul07b_rpi_master_slave_crc/upload_auto.py --only slave1
-> upload satu environment saja

```

```

python3 Modul07b_rpi_master_slave_crc/upload_auto.py --slave1 /dev/ttyACM0 --slave2 /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["slave1", "slave2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"slave1": "--s1", "slave2": "--s2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, menurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port slave1/slave2 PERSIS seperti yang baru dipakai upload --
    bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash. Master tetap harus dijalankan terpisah di Raspberry Pi.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
    for env, opsi in ENV_KE_OPSI.items():
        if env in port_env:
            cmd += [opsi, port_env[env]]

    print(" $", " ".join(cmd))
    print(" (ingat: master tetap dijalankan terpisah di Raspberry Pi -- "
          "python3 src/master.py)")
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--slave1", metavar="/dev/ttyXXX",
                    help="timpa port slave1 (lewati deteksi otomatis)")
    ap.add_argument("--slave2", metavar="/dev/ttyXXX",
                    help="timpa port slave2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (slave1 atau slave2)")

```

```

ap.add_argument("--monitor", action="store_true",
                help="buka pio device monitor per board sesudah upload "
                    "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau keduanya sekaligus)")
args = ap.parse_args()

aktif = deteksi_port_aktif()
manual = {"slave1": args.slave1, "slave2": args.slave2}

port_env = {}
sisa = list(aktif)
for nama in ENV_URUTAN:
    if manual[nama]:
        port_env[nama] = manual[nama]
    elif sisa:
        port_env[nama] = sisa.pop(0)

print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
for nama in ENV_URUTAN:
    if nama in port_env:
        sumber = "manual" if manual[nama] else "otomatis"
        print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
    else:
        print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
              f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

daftar = [args.only] if args.only else ENV_URUTAN
hilang = [n for n in daftar if n not in port_env]
if hilang:
    sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
            f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

for nama in daftar:
    if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
        sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

print("\nSelesai.")
tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 8.7: upload_auto.py — pemilih port otomatis saat unggah slave

8.7 Build, Flash, dan Menjalankan

```

pio run -d Modul07b_rpi_master_slave_crc -e slave1 -t upload
pio run -d Modul07b_rpi_master_slave_crc -e slave2 -t upload

```

Menyiapkan Raspberry Pi

```

ssh pi@<alamat-ip-pi>
sudo raspi-config          # Interface Options > SPI > Yes, lalu reboot
ls /dev/spi*              # harus muncul spidev0.0

pip3 install -r Modul07b_rpi_master_slave_crc/requirements.txt

```

Menjalankan master — kedua slave lebih dahulu, baru master.

```

ssh pi@<alamat-ip-pi>
cd ~/Documents/WSN-IOT-prak-Lora/Modul07b_rpi_master_slave_crc/src
python3 -u master.py      # -u penting bila keluarannya dipipa/direkam

```

Memantau kedua slave dari laptop

```

pip install pyserial rich
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1
python3 lora_monitor.py --s1 /dev/ttyACM0 --s2 /dev/ttyACM1 --out sesil.csv

```

Memverifikasi radio sebelum percobaan

```
ssh pi@<alamat-ip-pi>
cd ~/Documents/WSN-IOT-prak-Lora/Modul07b_rpi_master_slave_crc
python3 cek_radio.py
```

Pre-flight checklist

- ☐ Antena terpasang pada HAT dan kedua shield.
- ☐ SPI aktif di Raspberry Pi — `ls /dev/spi*` menampilkan `spidev0.0`.
- ☐ `spidev` dan `RPi.GPIO` (atau `rpi-lgpio` pada Pi 5) sudah terpasang di Pi.
- ☐ `cek_radio.py` dijalankan lebih dahulu dan menunjukkan `MODEM_CFG_2 : 0x74`.
- ☐ Baseline Modul 07 (CRC mati) sudah dikerjakan dan angkanya tersedia untuk pembandingan.
- ☐ Label fisik ditempel: SLAVE 1, SLAVE 2.

8.8 Percobaan

8.8.1 EXP-01 — Siklus Pertama dengan CRC Aktif

Unggah kedua slave, jalankan `cek_radio.py`, lalu jalankan `master.py` dan amati siklus pertama pada kedua sisi.

Expected output — master (Raspberry Pi)

```
=== LoRa MASTER-SLAVE 3 NODE ===
Init LoRa ... OK
Freq: 433 MHz
SF7 | BW: 125 kHz
CRC payload: AKTIF
Peran: MASTER (Raspberry Pi + LoRa GPS HAT)
Slave: Dragino Shield Uno - S1 & S2

=====
=== CYCLE 1 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -72 dBm | SNR: 14.8 dB
[TX] POLL:2
[RX] S2:DATA:38 | RSSI: -64 dBm | SNR: 14.5 dB
--- STATISTIK ---
S1: OK=1 | FAIL=0 | Data: 1
S2: OK=1 | FAIL=0 | Data: 38
Durasi siklus: 165 ms
=====
```

Expected output — slave (banner Serial Monitor, kedua Uno)

```
=== LoRa SLAVE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
CRC payload: AKTIF
Menunggu POLL:1 dari Master (Raspberry Pi)...
```

Data capture

Parameter	Hasil
Isi <code>MODEM_CFG_2</code> menurut <code>cek_radio.py</code> (SF, CRC)	
Baris <code>CRC payload: AKTIF</code> muncul di Serial Monitor kedua slave?	
Nomor siklus pertama yang lengkap tanpa FAIL	
RSSI master dari S1 / S2 (dBm)	
Jumlah [IGNORE] per siklus di tiap slave	

Tabel 8.3: Lembar pengamatan EXP-01

Verifikasi kode — bandingkan `src/slave/main.cpp` di modul ini dengan `Modul07_rpi_master_slave/src/slave/main.cpp` (varian tanpa CRC) memakai `diff`. Jawab: baris apa saja yang berbeda, dan mengapa satu baris `LoRa.enableCrc()` sudah cukup untuk mengubah keputusan hardware tentang paket mana yang boleh naik ke `parsePacket()`?

CHECKPOINT

Ketiga node mencetak OK dan CRC payload: AKTIF, tiap slave menampilkan tepat satu [RX] dan dua [IGNORE] per siklus, dan `diff` menunjukkan hanya satu baris fungsional (`LoRa.enableCrc()` beserta satu baris cetak) yang berbeda dari varian tanpa CRC. Durasi siklus sedikit lebih panjang daripada baseline CRC-mati — itu bukan regresi, melainkan biaya 2 byte checksum tambahan per paket.

8.8.2 EXP-02 — Statistik dan Lama Siklus Lintas Mesin

Rekam bersamaan selama minimal 50 detik: terminal SSH master di Raspberry Pi, dan kedua serial slave di laptop lewat `lora_monitor.py`.

Expected output — master

```
=====
=== CYCLE 40 ===
[TX] POLL:1
[RX] S1:DATA:40 | RSSI: -72 dBm | SNR: 14.6 dB
[TX] POLL:2
[RX] S2:DATA:77 | RSSI: -65 dBm | SNR: 14.6 dB
--- STATISTIK ---
S1: OK=40 | FAIL=0 | Data: 40
S2: OK=40 | FAIL=0 | Data: 77
Durasi siklus: 165 ms
=====
```

Data capture

Parameter	Hasil
Jumlah siklus dalam jendela rekaman	
Durasi siklus min / maks / rata-rata saat sehat (ms)	
Jumlah baris [WARN] Balasan tidak valid (harus 0 dengan CRC aktif)	
RX# terakhir S1 / S2 vs Data: terakhir master — harus sama persis	
SNR arah slave→ master vs master→ slave — simetris?	

Tabel 8.4: Lembar pengamatan EXP-02

BUKA ABSTRAKSINYA

Pada varian tanpa CRC (Modul 07), OK master kadang lebih kecil daripada RX# slave pada nomor yang sama, karena paket berpayload rusak lolos ke aplikasi dan gagal di-parse. Verifikasi pada sesi rekaman sendiri bahwa hal itu **tidak terjadi lagi** di sini — Data: master harus mengikuti RX# slave nomor demi nomor tanpa selisih. Jelaskan, memakai isi `MODEM_CFG_2` dari `cek_radio.py`, mengapa kelas kegagalan itu sekarang tidak mungkin muncul.

CHECKPOINT

Data: master sama persis dengan RX# slave di setiap siklus yang OK, dan tidak ada satu pun baris [WARN] di seluruh sesi — kontras langsung dengan Modul07_rpi_master_slave/logserial.md, yang mencatat sampai 0,74 % paket rusak lolos pada beberapa sesi. Durasi siklus sehat tetap sangat rapat (sebaran sekitar 1 ms pada Raspberry Pi 5), hanya bergeser lebih tinggi ± 20 ms dari baseline CRC-mati karena 2 byte checksum tambahan di kedua arah.

8.8.3 EXP-03 — Satu Node Hilang

Cabut kabel USB Slave 2 secara fisik selama master.py berjalan, tunggu setidaknya 20 siklus, lalu pasang kembali.

CATATAN

Mekanisme timeout dan pemulihan tidak berubah oleh CRC — CRC hanya memfilter payload yang **diterima**, bukan payload yang tidak pernah datang. Expected output dan checkpoint di bawah diwariskan dari Modul 07 (CRC mati); keduanya **belum diuji ulang** secara khusus untuk varian ini pada sesi modifikasi CRC — lihat logserial.md untuk status verifikasi terkini.

Expected output — master, tepat setelah kabel dicabut

```
=== CYCLE 28 ===
[TX] POLL:1
[RX] S1:DATA:27 | RSSI: -70 dBm | SNR: 13.0 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=27 | FAIL=1 | Data: 27
S2: OK=0 | FAIL=27 | Data: None
Durasi siklus: 605 ms
```

Data capture

Parameter	Hasil
Pesan master saat Slave 2 tidak menjawab	
Durasi siklus saat Slave 2 hilang (ms)	
Apakah Slave 1 terpengaruh matinya Slave 2?	
Berapa siklus sampai OK Slave 2 bertambah lagi setelah dipasang	
dataCounter Slave 2 setelah dipasang kembali — mulai dari berapa	

Tabel 8.5: Lembar pengamatan EXP-03

CHECKPOINT

Durasi siklus melonjak mendekati POLL_TIMEOUT penuh ditambah overhead, sementara Slave 1 sama sekali tidak terganggu. Begitu Slave 2 tersambung kembali, ia memulai dataCounter dari 1: bukti bahwa yang terjadi adalah reboot penuh, bukan sekadar port serial yang terputus.

8.8.4 EXP-04 — Menekan Batas Waktu Sampai Rusak

Ubah POLL_TIMEOUT di **salinan** master.py, jalankan tiap nilai selama 60 detik, mulai dari 500 ms turun bertahap sampai keberhasilan jatuh ke nol.

CATATAN

Dengan CRC aktif, waktu udara tiap paket sedikit lebih panjang (2 byte checksum tambahan di kedua arah), sehingga ambang tebing yang sebenarnya kemungkinan bergeser **lebih tinggi** dari titik 40–45 ms yang tercatat pada Modul 07 (CRC mati). Tabel di bawah dan checkpoint-nya diwariskan dari sesi CRC-mati sebagai titik acuan; **belum diukur ulang** untuk varian ini — jalankan EXP-04 sendiri di sini untuk mendapati titik tebing yang baru.

Data capture

POLL_TIMEOUT (ms)	Poll berhasil	Poll gagal	Gagal (%)
500 (baku)			
100			
60			
50			
45			
40			

Tabel 8.6: Lembar pengamatan EXP-04 — menekan batas waktu dengan CRC aktif

CHECKPOINT

Ambangnya tetap berupa tebing, bukan lereng. Bandingkan letaknya dengan hasil Modul 07: bila bergeser ke nilai yang lebih tinggi, jelaskan pergeseran itu memakai tambahan waktu udara dari 2 byte checksum.

8.8.5 Verifikasi Radio (Dijalankan Sebelum EXP-01)

```
REG_VERSION : 0x12 (0x12 = SX1276/77/78/79)
FREKUENSI : 433.000000 MHz (target 433.000000, selisih +0.0 Hz)
MODEM_CFG_1 : 0x72 -> BW=125 kHz | CR=4/5 | header=explicit
MODEM_CFG_2 : 0x74 -> SF7 | CRC payload=AKTIF
PA_CONFIG : 0x8f -> PA_BOOST, power=17 dBm
```

PENTING

CRC payload aktif di kedua sisi. `LoRa.enableCrc()` ditambahkan di `setup()` slave, dan `enableCrc()` — fungsi baru di `master.py` yang menulis bit yang sama, `REG_MODEM_CONFIG_2` bit 2, persis seperti yang dilakukan `LoRa.enableCrc()` sandeepmistry — dipanggil sebelum `master.py` mulai polling. Konsekuensinya: SX1276 penerima memverifikasi checksum 2-byte pada tiap paket dan membuang paket yang gagal **sebelum** `parsePacket()` atau `IRQ_RX_DONE` sempat menyerahkannya ke aplikasi. Paket berpayload rusak sekarang menghasilkan [FAIL] tidak merespon (timeout) alih-alih [WARN] Balasan tidak valid — diagnosisanya benar secara radio (paket memang tidak sampai utuh), meski dari sisi statistik OK dan FAIL tetap tidak membedakan “slave diam” dari “jawaban dibuang radio”.

8.8.6 Verifikasi Perangkat Keras (Log Referensi)

Sesi verifikasi CRC — 21 Agustus 2026, satu Raspberry Pi 5 + LoRa GPS HAT v1.4 (SSH) dan dua Arduino Uno + Dragino Shield v1.2 tertancap USB di laptop pengembang (`/dev/ttyACM0`, `/dev/ttyACM1`), 433 MHz, jarak ± 30 cm. Log lengkap ada di `logserial.md`.

Parameter	Hasil terukur
MODEM_CFG_2 menurut cek_radio.py	0x74 — SF7, CRC payload AKTIF (sebelumnya 0x70 pada varian tanpa CRC)
Banner CRC payload: AKTIF di Serial Monitor	tampil pada kedua slave
Siklus dalam sesi ± 55 detik	83 , seluruhnya OK — 0 FAIL, 0 [WARN]
S1 / S2: OK / FAIL	83/83 (100 %) / 83/83 (100 %)
RX# slave vs Data: master pada siklus terakhir	sama persis di kedua slave — tidak ada payload rusak yang lolos maupun gagal parse
Durasi siklus sehat min/maks/rata-rata	165 / 166 / 165,3 ms
Sebaran durasi siklus	1 ms (n = 83) — serapat sesi CRC-mati (144/145/145,0 ms), hanya bergeser +20 ms akibat 2 byte checksum per paket di kedua arah
RSSI master dari S1 / S2	– 71,8 dBm (kisaran – 83 ... – 68) / – 65,0 dBm (kisaran – 67 ... – 63)
SNR master dari S1 / S2	14,64 dB / 14,63 dB
Baris [IGNORE] tak dikenal (payload rusak) di log kedua slave	0 dari 160 + 189 baris — tidak satu pun keluar dari pola POLL:<n> atau S<n>:DATA:<n>

Tabel 8.7: Hasil verifikasi perangkat keras Modul 07B

Environment	Status	Flash
slave1	SUCCESS	26.6% (8.574 B)
slave2	SUCCESS	26.6% (8.574 B)

Master tidak dikompilasi — Python dijalankan langsung di Raspberry Pi. Kedua slave berukuran identik (8.574 B, naik dari 8.522 B pada varian tanpa CRC — biaya `LoRa.enableCrc()` plus satu baris `Serial.println()`), bukti keduanya berasal dari source yang sama dan hanya berbeda `SLAVE_ID`.

PERINGATAN

Cakupan pengujian sesi ini. Yang diverifikasi langsung di perangkat: build, upload, register CRC di Pi (`cek_radio.py`), banner Serial kedua slave, dan satu sesi `master.py` penuh (gaya EXP-01 dan EXP-02) tanpa [WARN] maupun [FAIL] di luar transien awal. **EXP-03** (cabut kabel fisik) dan **EXP-04** (sapuan `POLL_TIMEOUT`) **tidak dijalankan ulang** pada sesi modifikasi CRC ini; angka di tabelnya tetap milik sesi CRC-mati sebelumnya dan perlu diukur ulang terpisah sebelum dipakai sebagai klaim untuk varian ini. Tidak satu pun paket rusak tertangkap secara langsung pada sesi 83 siklus ini untuk dibandingkan byte-per-byte dengan temuan Modul 07 — wajar, karena kejadian itu sendiri sporadis (0,74 % pada sebagian sesi CRC-mati, 0,00 % pada sesi lain). Bukti bahwa CRC aktif berasal dari pembacaan register langsung dan dari nolnya [WARN] di seluruh sesi, bukan dari menangkap satu paket rusak yang difilter.

CATATAN

CH-2 pada modul dasar sudah diterapkan di sini. Modul 07 mendaftarkan `LoRa.enableCrc()` di slave dan bit sepadan di master sebagai tantangan CH-2 yang sengaja tidak diterapkan, karena menyentuh firmware slave akan membatalkan klaim “identik dengan M05” pada modul dasar. Modul ini adalah hasil penerapan CH-2 tersebut sebagai modul tersendiri. Rincian investigasi payload rusak pada varian tanpa CRC (pola byte, pengujian yang menyingkirkan SPI dan FIFO sebagai penyebab) tetap didokumentasikan di `Modul07_rpi_master_slave/logserial.md`, bagian “Temuan — Payload rusak lolos karena CRC mati”, sebagai bahan pembandingan.

8.8.7 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, P0LL_TIMEOUT 500 ms, CYCLE_INTERVAL 500 ms. Ketiga node di satu meja, jarak ±30 cm.

Direktori ini adalah Modul07b_rpi_master_slave_crc/, salinan dari Modul07_rpi_master_slave/ dengan LoRa.enableCrc() ditambahkan di slave dan enableCrc() di master. Sesi-sesi di bawah ini sampai dengan “Verifikasi ulang — 21 Agustus 2026” adalah **riwayat dari sebelum modifikasi CRC diterapkan** (CRC masih mati saat sesi-sesi itu direkam — persis kondisi Modul07_rpi_master_slave/logserial.md). Bagian “**Modifikasi CRC — 21 Agustus 2026**” di bawah, setelah bagian “Catatan pengambilan log”, adalah sesi pertama dengan CRC **aktif** dan mendokumentasikan perubahan kode, build/upload, dan pengujian ulang di perangkat.

Pengujian ini adalah **pengujian perangkat keras pertama** untuk Modul 07. Bagian “Catatan verifikasi” pada README sebelumnya menyatakan konversi ini belum pernah dijalankan di perangkat nyata; sejak dokumen ini ditulis, pernyataan itu tidak berlaku lagi.

Board & Port

Peran	Environment / program	Port	Board
Master	src/master.py	— (lewat SSH)	Raspberry Pi 5B rev 1.0, BCM2712 + Dragino LoRa GPS HAT v1.4
Slave 1	slave1 (-DSLAVE_ID=1)	/dev/ttyACM0	Uno asli (2341:0043) + Dragino Shield v1.2
Slave 2	slave2 (-DSLAVE_ID=2)	/dev/ttyACM1	Uno asli (2341:0043) + Dragino Shield v1.2

Tabel 8.8: Log serial Modul 07B: Board & Port

Kedua Uno tertancap di laptop pengembang, sedangkan master dijalankan di Raspberry Pi lewat SSH. Inilah perbedaan praktis pertama dengan M05: **tiga node tidak lagi berada di satu komputer**, sehingga log master dan log slave direkam oleh dua mesin yang berbeda dan harus dicocokkan lewat isinya, bukan lewat cap waktu bersama.

Perangkat lunak Raspberry Pi: Raspberry Pi OS 13 (trixie), Python 3.13.5, python3-spidev 3.6, python3-rpi-lgpio 0.6. Paket python3-rpi.gpio **tidak** terpasang — pada Pi 5, RPi.GPIO disediakan oleh rpi-lgpio. Rinciannya di README bagian 5.

Verifikasi radio sebelum percobaan

Nilai di bawah dibaca **balik dari register SX1276** setelah loraBegin() + konfigurasi, bukan disalin dari konstanta di source.

```

REG_VERSION   : 0x12   (0x12 = SX1276/77/78/79)
FRF register   : 0x6c 0x40 0x00 -> frf=7094272
FREKUENSI     : 433.000000 MHz   (target 433.000000, selisih +0.0 Hz)
MODEM_CFG_1   : 0x72 -> BW=125 kHz | CR=4/5 | header=explicit
MODEM_CFG_2   : 0x70 -> SF7 | CRC payload=MATI
PA_CONFIG     : 0x8f -> PA_BOOST, power=17 dBm

```

Parameter	Nilai di chip	Sesuai README?
Frekuensi	433,000000 MHz (selisih 0,0 Hz)	ya
Spreading factor	SF7	ya
Bandwidth	125 kHz	ya
Coding rate	4/5	ya
Daya pancar	17 dBm, PA_BOOST	ya
CRC payload	MATI	tidak disebut README — lihat bagian Temuan

Tabel 8.9: Log serial Modul 07B: Verifikasi radio sebelum percobaan

Skrrip pembacanya ditinggalkan di Raspberry Pi sebagai `cek_radio.py` agar verifikasi ini dapat diulang kapan saja.

EXP-01 — Siklus Pertama Lintas Platform

Master (Raspberry Pi), siklus pertama

```

=== LoRa MASTER-SLAVE 3 NODE ===
Init LoRa ... OK
Freq: 433 MHz
SF7 | BW: 125 kHz
Peran: MASTER (Raspberry Pi + LoRa GPS HAT)
Slave: Dragino Shield Uno - S1 & S2

=====
=== CYCLE 1 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -60 dBm | SNR: 14.2 dB
[TX] POLL:2
[RX] S2:DATA:1 | RSSI: -58 dBm | SNR: 14.2 dB
--- STATISTIK ---
S1: OK=1 | FAIL=0 | Data: 1
S2: OK=1 | FAIL=0 | Data: 1
Durasi siklus: 145 ms
=====

```

Slave 1 dan Slave 2 (cap waktu = detik sejak perekaman serial dimulai)

```

1.612 === LoRa SLAVE 1 ===          | 1.617 === LoRa SLAVE 2 ===
1.634 Init LoRa ... OK              | 1.638 Init LoRa ... OK
1.634 Freq: 433.00 MHz              | 1.638 Freq: 433.00 MHz
1.637 Menunggu POLL:1 dari Master   | 1.642 Menunggu POLL:2 dari Master

4.320 [RX] POLL:1 | RSSI: -62 dBm | SNR: 8.00 dB | RX#: 1
4.320 [TX] S1:DATA:1                | 4.316 [IGNORE] POLL:1
4.389 [IGNORE] POLL:2               | 4.357 [IGNORE] S1:DATA:1
4.426 [IGNORE] S2:DATA:1            | 4.391 [RX] POLL:2 | RSSI: -64 dBm | SNR: 9.00 dB | RX#: 1
                                     | 4.395 [TX] S2:DATA:1

```

Parameter	Hasil
Pesan init master	Init LoRa ... OK, 433 MHz, SF7/BW 125 kHz
Pesan init slave 1 / slave 2	keduanya Init LoRa ... OK, Freq: 433.00 MHz
Nomor siklus pertama yang lengkap	1 — tidak ada FAIL sama sekali di siklus pembuka
RSSI master ← S1	-60 dBm (SNR 14,2 dB)
RSSI master ← S2	-58 dBm (SNR 14,2 dB)
Baris [IGNORE]	S1 mengabaikan POLL:2, S2 mengabaikan POLL:1 — masing-masing 1 per siklus
Baris [IGNORE] tambahan	tiap slave juga mengabaikan jawaban slave lain (S2:DATA:1 / S1:DATA:1)

Tabel 8.10: Log serial Modul 07B: EXP-01 — Siklus Pertama Lintas Platform

CHECKPOINT terpenuhi. Ketiga node mencetak OK, dan tiap slave menampilkan tepat **satu** [RX] beserta **dua** [IGNORE] per siklus — satu untuk POLL milik node lain, satu untuk jawaban node lain. Urutan waktu di kedua slave saling mengunci: S2 mencatat [IGNORE] POLL:1 pada 4.316 s, S1 mencatat [RX] POLL:1 pada 4.320 s. Keduanya mendengar paket yang sama; hanya pemiliknya yang menjawab.

Perhatikan bahwa **master berganti platform tanpa slave mengetahuinya**. Firmware slave yang dipakai identik dengan M05:

```

$ diff Modul05_lora_master_slave/src/slave/main.cpp Modul07_rpi_master_slave/src/slave/main.cpp
2c2 < LoRa Master-Slave 3 Node - ... > LoRa Master-Slave hybrid - ...
6,7c6,12 (blok komentar penjelasan M07)
74c79,81 Serial.println("Menunggu POLL:1 dari Master...")
      -> Serial.print("Menunggu POLL:"); Serial.print(SLAVE_ID);
      Serial.println(" dari Master (Raspberry Pi)...")

```

Tiga hunk, seluruhnya komentar dan satu pesan pembuka di Serial. Tidak satu byte pun dari yang mengudara berubah.

EXP-02 — Statistik dan Lama Siklus

Rekaman 50 detik dengan `src/master.py` apa adanya, sambil kedua port serial slave direkam bersamaan.

Parameter	Hasil
Jumlah siklus dalam 50 detik	76 ($\approx 1,52$ siklus/detik)
S1: OK / FAIL	74 / 2
S2: OK / FAIL	75 / 1
Data: terakhir S1 / S2	76 / 76
RX# terakhir di slave 1 / slave 2	76 / 76
Durasi siklus minimum	144 ms
Durasi siklus maksimum (siklus sehat)	145 ms
Durasi siklus rata-rata (siklus sehat)	145,0 ms
Durasi siklus saat satu poll gagal	605 ms
RSSI di master (dari S1 / S2)	-59,6 / -57,8 dBm
SNR di master (dari S1 / S2)	14,09 / 14,19 dB
RSSI di S1 / S2 (dari master)	-64,8 / -64,0 dBm
SNR di S1 / S2 (dari master)	8,63 / 8,88 dB

Tabel 8.11: Log serial Modul 07B: EXP-02 — Statistik dan Lama Siklus

Buka abstraksinya. Ketiga bilangan yang diminta README dapat dibandingkan langsung: pada siklus ke-76, master mencatat S1: OK=74 | FAIL=2 | Data: 76, sedangkan slave 1 mencatat RX#: 76. Artinya slave menerima **seluruh** 76 panggilan dan mengirim **seluruh** 76 jawaban, tetapi hanya 74 yang sampai utuh ke master. Data: melompat dari 74 ke 76 tanpa master pernah menerima nomor 75 — persis keadaan yang diramalkan CHECKPOINT EXP-02, dan didapat tanpa perlu menjauhkan slave. Penyebab dua jawaban yang hilang itu dibahas di bagian Temuan.

Asimetri arah yang tidak terduga. SNR arah slave $\rightarrow$ master adalah 14,1 dB, sedangkan arah master $\rightarrow$ slave hanya 8,6–8,9 dB — selisih tetap sekitar **5,3 dB** yang bertahan di seluruh rekaman. Kedua arah memakai daya pancar 17 dBm yang sama. Yang berbeda hanyalah papan pembawanya: LoRa GPS HAT di sisi Pi, Shield v1.2 di sisi Uno. Angka ini tidak terlihat dari terminal master saja, karena master hanya melaporkan arah yang diterimanya; ia baru muncul setelah kedua sisi direkam bersamaan.

Lama siklus jauh lebih rapat daripada dugaan README

README memperkirakan master Linux akan memperlihatkan **jitter penjadwalan** yang tidak ada padanannya di Arduino. Pada Raspberry Pi 5 dugaan itu **tidak terkonfirmasi**:

Ukuran	Master Raspberry Pi 5 (modul ini)	Master Arduino Uno (M05, logserial.md)
Durasi siklus minimum	144 ms	147 ms
Durasi siklus maksimum	145 ms	154 ms
Durasi siklus rata-rata	145,0 ms	152 ms
Sebaran (maks – min)	1 ms	7 ms

Tabel 8.12: Log serial Modul 07B: Lama siklus jauh lebih rapat daripada dugaan README

Master Linux justru **lebih rapat** sebarannya daripada master Arduino, dan sedikit lebih cepat. Pengukuran kedua, diambil dari sisi node memakai `lora_monitor.py` (jarak antar POLL yang diterima slave, jadi sudah termasuk `CYCLE_INTERVAL` 500 ms):

Ukuran	Nilai
Periode siklus minimum	642 ms
Periode siklus rata-rata	645,2 ms
Periode siklus maksimum	648 ms
Sebaran	6 ms (n = 112)

Tabel 8.13: Log serial Modul 07B: Lama siklus jauh lebih rapat daripada dugaan README

Kedua pengukuran sepakat. Penjelasan yang masuk akal: Pi 5 berinti empat pada beban hampir nol, sehingga proses Python praktis tidak pernah benar-benar berebut CPU; dan POLL_TIMEOUT 500 ms sedemikian longgar dibanding waktu tanggap sebenarnya (± 45 ms, lihat EXP-04) sehingga penundaan beberapa milidetik tidak pernah mengubah hasil. Sebaran ini bukan bantahan terhadap teori di README — jitter tetap ada secara prinsip — melainkan bukti bahwa pada beban serendah ini besarnya tidak terukur oleh alat yang dipakai modul ini.

EXP-03 — Satu Node Hilang

Slave 2 dicabut kabel USB-nya sepenuhnya (bukan sekadar port serial ditutup — Uno bershield LoRa memakai daya dari USB, jadi mencabutnya mematikan seluruh node termasuk radionya) selama master.py berjalan, lalu dipasang kembali di tengah sesi. Slave 1 tetap dipantau lewat serial lokal selama percobaan berlangsung untuk memastikan node sehat tidak ikut terganggu.

```
=== CYCLE 1 ===
[TX] POLL:1
[FAIL] Slave 1 tidak merespon!
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
Durasi siklus: 1065 ms
=====
=== CYCLE 2 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -70 dBm | SNR: 12.8 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=1 | FAIL=1 | Data: 1
S2: OK=0 | FAIL=2 | Data: None
Durasi siklus: 604 ms
...
=== CYCLE 28 ===
[TX] POLL:1
[RX] S1:DATA:27 | RSSI: -70 dBm | SNR: 13.0 dB
[TX] POLL:2
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=27 | FAIL=1 | Data: 27
S2: OK=0 | FAIL=27 | Data: None
Durasi siklus: 605 ms
=== CYCLE 29 ===
[TX] POLL:1
[RX] S1:DATA:28 | RSSI: -71 dBm | SNR: 12.8 dB
[TX] POLL:2
[RX] S2:DATA:1 | RSSI: -64 dBm | SNR: 13.8 dB
--- STATISTIK ---
S1: OK=28 | FAIL=1 | Data: 28
S2: OK=1 | FAIL=27 | Data: 1
Durasi siklus: 144 ms
```

Parameter	Hasil
Siklus 1 — kedua slave gagal (start-up master mendahului kesiapan radio)	Durasi 1065 ms , kira-kira dua POLL_TIMEOUT penuh
Siklus 2–28 — Slave 2 hilang (27 siklus berturut-turut)	Durasi stabil 604–606 ms , rata-rata 621,4 ms

Parameter	Hasil
Siklus 29 — Slave 2 kembali	Langsung OK, S2:DATA:1 — dataCounter slave mulai dari 1 (bukti Uno benar-benar reboot, bukan cuma port serial terputus)
Siklus 30 dst. — steady state	Durasi kembali 144–145 ms , rata-rata 144,8 ms
Slave 1 selama Slave 2 hilang	OK di seluruh 27 siklus — 79 baris [TX] S1:DATA:n lokal, [IGNORE] POLL:2 79× tanpa gangguan
Pertambahan durasi akibat satu node mati	604,8 – 144,8 ≈ 460 ms , ≈ POLL_TIMEOUT (500 ms) dikurangi waktu jawaban sehat
Pemulihan	Seketika — satu siklus setelah Slave 2 tersambung kembali, tanpa intervensi di sisi master

Tabel 8.14: Log serial Modul 07B: EXP-03 — Satu Node Hilang

CHECKPOINT terpenuhi, dan sekarang dengan data sungguhan. Catatan lama di bagian “Catatan pengambilan log” menyimpulkan pertambahan +460 ms hanya dari poll individual yang gagal sesekali, tanpa benar-benar mencabut node. Sesi ini mengonfirmasinya secara langsung: 27 siklus berturut-turut dengan Slave 2 mati, tidak satu pun memengaruhi Slave 1, dan durasi siklus melonjak persis sebesar satu POLL_TIMEOUT dikurangi waktu tanggap sehat ($605 - 145 \approx 460$ ms) — sama seperti prediksi CHECKPOINT EXP-03 M05, dan sama dengan angka +459 ms yang tercatat di Modul05_lora_master_slave/logserial.md untuk skenario setara.

Siklus 1 (1065 ms) bukan anomali — konsisten dengan dua timeout penuh. Master mulai polling sebelum Slave 1 (yang baru saja diunggah ulang) selesai boot, sehingga siklus pertama kehilangan **kedua** slave sekaligus: $2 \times \sim 500$ ms POLL_TIMEOUT plus overhead, hasilnya 1065 ms — persis nilai yang sudah tercatat sebagai temuan tersendiri di bagian ini sebelum EXP-03 pernah dijalankan langsung.

EXP-04 — Menekan Batas Waktu Sampai Rusak

POLL_TIMEOUT diturunkan bertahap, tiap nilai dijalankan **60 detik**. Nilai diberikan lewat environment ke salinan master.py, sehingga src/master.py tidak pernah diubah.

POLL_TIMEOUT (ms)	Siklus	Poll berhasil	Poll gagal	Gagal (%)	Durasi siklus rata-rata (ms)
500 (baku)	92	184	0	0,0 %	156
250	92	184	0	0,0 %	156
150	92	184	0	0,0 %	156
100	92	184	0	0,0 %	156
60	92	184	0	0,0 %	156
55	92	183	1	0,5 %	156
50	92	184	0	0,0 %	156
45	92	181	3	1,6 %	156
40	93	0	186	100 %	146

Tabel 8.15: Log serial Modul 07B: EXP-04 — Menekan Batas Waktu Sampai Rusak

Ambangnya adalah tebing, bukan lereng — persis seperti diramalkan CHECKPOINT EXP-04. Di 45 ms sistem masih berhasil 98,4 %; di 40 ms keberhasilan jatuh ke **nol mutlak**, bukan memburuk perlahan. Zona 45–55 ms adalah pinggiran tebing: sebagian jawaban mulai tersenggol batas waktu, tetapi mayoritas masih lolos.

Pengulangan tiga kali pada dua nilai penentu (Pengukuran C):

Percobaan ke-	Nilai terkecil yang masih andal (ms)	Nilai pertama yang sudah gagal (ms)
1	60 (0 gagal dari 184)	40 (186 gagal dari 186)
2	60 (0 gagal dari 184)	40 (186 gagal dari 186)
3	60 (0 gagal dari 184)	40 (186 gagal dari 186)

Tabel 8.16: Log serial Modul 07B: EXP-04 — Menekan Batas Waktu Sampai Rusak

Reproduksinya sempurna — tidak ada satu pun kejadian menyimpang di enam sesi.

Mengapa 40 ms, bukan 77 ms

README memperkirakan ambangnya “hampir pasti jauh di atas” 77 ms, dengan alasan waktu udara POLL:1 (± 36 ms) ditambah waktu udara S1:DATA:12 (± 41 ms). Hasil pengukuran menunjukkan ambangnya justru **di bawah** angka itu, dan penyebabnya adalah kekeliruan dalam menyusun anggaran waktunya:

POLL_TIMEOUT **tidak pernah mencakup waktu udara POLL**. Di pollSlave(), transmit(pollMsg) bersifat blocking — ia baru kembali setelah IRQ_TX_DONE menyala, artinya paket POLL sudah selesai mengudara. Baru sesudah itu waitStart = time.monotonic() dijalankan. Jadi jendela batas waktu hanya menampung:

```
pemrosesan di slave + waktu udara jawaban ( $\pm 41$  ms) + deteksi RX_DONE di master
```

Yang tersisa memang ± 41 –45 ms, dan itulah sebabnya 45 ms berada di pinggiran sementara 40 ms memotong setiap jawaban tepat sebelum tiba. Angka 36 ms untuk POLL sudah “dibayar” di dalam transmit(), di luar penghitung waktu.

Konsekuensi praktisnya: POLL_TIMEOUT 500 ms adalah **11× lebih longgar** daripada yang dibutuhkan. Kelonggaran itulah yang membuat durasi siklus tidak pernah goyah — dan sekaligus yang membuat satu node mati menjadi sangat mahal, karena setiap node mati menagih 500 ms penuh setiap siklus.

Temuan — Payload rusak lolos karena CRC mati

Selama pengujian, master beberapa kali mencetak [WARN] Balasan tidak valid lalu menghitungnya sebagai [FAIL], padahal log serial slave membuktikan slave menerima POLL dan mengirim jawaban yang benar. Contoh byte mentahnya:

```
[WARN] Balasan tidak valid: S7:DATA:705 | ps=11 | raw=53 a1 3a 44 41 54 41 3a 37 30 35
      ^ ^ seharusnya 0x31 ('1')
[WARN] Balasan tidak valid: S1:DATA:785 | ps=10 | raw=53 31 3a 44 41 54 41 9a 38 35
      ^ ^ seharusnya 0x3a (':')
```

Frekuensi kejadian

Kelompok run	Paket diterima	Paket rusak	Rasio
8 sesi pertama	2288	17	0,74 %
4 sesi berikutnya	1462	0	0,00 %

Tabel 8.17: Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati

Pola byte — dari 13 paket rusak yang byte mentahnya sempat terekam:

Sifat	Pengamatan
Byte rusak per paket	1 byte (10 paket), 2 byte (3 paket)
Indeks byte yang rusak	indeks 1 (10×), indeks 7 (4×), indeks 8 (2×)
Indeks yang tidak pernah rusak	0, 2, 3, 4, 5, 6 — yaitu S, :, dan DATA
Jumlah bit berbeda, pada byte yang nilai benarnya pasti	selalu 2 bit (4 dari 4 kasus)

Tabel 8.18: Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati

Yang sudah disingkirkan sebagai penyebab

Dugaan	Uji	Hasil
Jalur SPI tidak andal	REG_VERSION dibaca 40.000× pada 5 MHz dan 1 MHz	0 error di kedua kecepatan — SPI bersih saat idle
Modem menimpa FIFO saat dibaca	MODE_STDBY disisipkan sebelum baca FIFO, seperti <code>LoRa.parsePacket()</code> di Arduino	rasio tidak berubah (6 dari 354)
Byte rusak saat dibaca dari FIFO	isi FIFO dibaca 3× berturut-turut lalu dibandingkan	selalu identik , termasuk pada paket rusak
Master membanjiri SPI saat demodulasi	<code>time.sleep(0.002)</code> disisipkan di loop tunggu	0 dari 730 — tetapi kontrol tanpa jeda juga 0 dari 732 , jadi tidak membuktikan apa pun

Tabel 8.19: Log serial Modul 07B: Temuan — Payload rusak lolos karena CRC mati

Pembacaan FIFO tiga kali yang selalu identik adalah uji penentunya: **byte rusak itu sudah berada di FIFO sebelum dibaca**. Jalur SPI tidak bersalah; kerusakan terjadi di udara atau di dalam demodulator.

Mengapa kerusakan itu lolos sampai ke aplikasi. MODEM_CFG_2 menunjukkan CRC payload **mati**, di kedua sisi. LoRa.begin() milik sandeepmistry tidak mengaktifkan CRC kecuali diminta lewat LoRa.enableCrc(), dan master.py juga tidak menyalakannya. Akibatnya pemeriksaan IRQ_CRC_ERROR di parsePacket() **tidak akan pernah menyala** — bukan karena tidak ada paket cacat, melainkan karena radio tidak pernah diminta memeriksanya. Paket cacat naik utuh ke lapisan aplikasi, gagal di reply.startswith("S1:DATA:"), dan tercatat sebagai “slave tidak merespon” — diagnosis yang menunjuk ke arah yang salah sama sekali.

Status. Mekanisme yang membangkitkan kerusakan **belum diketahui**. Pola indeksinya (hanya 1, 7, 8) terlalu terpusat untuk galat bit acak, tetapi belum ada uji yang menjelaskannya. Fenomenanya juga tidak muncul terus-menerus: 17 kejadian di delapan sesi pertama, nol di empat sesi berikutnya dan di seluruh EXP-04 kecuali satu, tanpa perubahan perangkat keras apa pun di antaranya.

Perbaiki yang jelas — LoRa.enableCrc() di slave dan menyalakan bit 2 REG_MODEM_CONFIG_2 di master — **tidak diterapkan**, karena menyentuh firmware slave dan dengan demikian membatalkan klaim utama modul ini bahwa slave identik dengan M05. Itu keputusan perancang modul, bukan keputusan penguji. Sebagai bahan praktikum, keadaan ini justru lebih berharga dibiarkan: ia memperlihatkan satu lapisan pelindung yang tidak dipasang, dan akibatnya terhadap diagnosis di lapisan atas.

Verifikasi ulang — 21 Agustus 2026

Sesi verifikasi baru: kedua slave dibangun ulang dari src/ saat ini dan diunggah ulang (pio run -e slave1|slave2 -t upload, keduanya SUCCESS, flash 8.522 B terverifikasi avrdude), master dijalankan langsung dari sumbernya di Raspberry Pi lewat SSH (python3 -u src/master.py, tanpa perubahan). Tujuannya memastikan konversi PlatformIO M07 masih berjalan seperti didokumentasikan di atas, bukan mengulang investigasi CRC.

Board & Port sesi ini — berbeda dari tabel “Board & Port” di atas karena hanya dua Uno yang tersambung ke laptop pengembang saat ini: Slave 1 di /dev/ttyACM1, Slave 2 di /dev/ttyACM2 (bukan ACM0/ACM1). Ini contoh nyata alasan README menyuruh menjalankan tools/deteksi_port.py dan memakai --upload-port eksplisit alih-alih mengandalkan nilai contoh di platformio.ini.

```
=== CYCLE 3 ===
[TX] POLL:1
[RX] S1:DATA:47 | RSSI: -76 dBm | SNR: 13.0 dB
[TX] POLL:2
[WARN] Balasan tidak valid: S7:DATA:27
[FAIL] Slave 2 tidak merespon!
--- STATISTIK ---
S1: OK=3 | FAIL=0 | Data: 47
S2: OK=2 | FAIL=1 | Data: None
Durasi siklus: 605 ms
```

Parameter	Hasil
Siklus dalam ± 35 detik	46
Durasi siklus steady-state min/maks/rata-rata	145 / 145 / 145,0 ms (n=45, mengecualikan 1 timeout)
Slave 1: OK / FAIL	46 / 0 $\rightarrow$ 100 %
Slave 2: OK / FAIL	45 / 1 $\rightarrow$ 97,8 %
Penyebab satu-satunya FAIL	1 byte payload rusak (S1:DATA:27 $\rightarrow$ S :DATA:27), pola sama dengan temuan “Payload rusak lolos karena CRC mati” di atas
SNR balasan Slave 1 di master	rata-rata 13,2 dB (n=46)
SNR balasan Slave 2 di master	rata-rata 13,3 dB (n=45)
RSSI balasan Slave 1 / Slave 2 di master	-69,4 dBm / -65,8 dBm (sepadan, tidak ada node yang janggal)

Tabel 8.20: Log serial Modul 07B: Verifikasi ulang — 21 Agustus 2026

Tidak ada anomali seperti pada M05. Modul 05 sempat mencatat SNR Slave 2 anjlok ke ~1,2 dB akibat Slave 2 duduk terlalu dekat dengan master (near-field). Pada sesi M07 ini, SNR kedua slave hampir identik (13,2 vs 13,3 dB) dan RSSI keduanya wajar — tidak ada indikasi kejenuhan penerima di sisi mana pun.

Tampilan Uno vs tampilan Raspberry Pi cocok satu sama lain. Log slave lokal (dibaca langsung dari /dev/ttyACM1 dan /dev/ttyACM2) menunjukkan [TX] S1:DATA:n dan [TX] S2:DATA:n yang nomornya berurutan dengan [RX] S1:DATA:n/S2:DATA:n pada log master di Raspberry Pi — mengonfirmasi kedua sisi memang saling bicara lewat radio, bukan kebetulan dua proses berjalan sendiri-sendiri.

Catatan pelaksanaan. Saat keluarannya dipipa lewat SSH (bukan TTY interaktif), Python membuffer stdout — memakai timeout untuk membatasi durasi lalu memutus prosesnya membuang isi buffer yang belum sempat di-flush. Jalankan dengan python3 -u saat keluarannya perlu direkam lewat pipa/redirect. master.py sebelumnya tidak punya opsi --help — argumen apa pun diabaikan begitu saja dan sesi langsung berjalan, seperti yang terjadi saat pertama kali dicoba pada sesi ini. Sudah ditambahkan argparse dengan -h/--help yang mencetak parameter radio (frekuensi, SF, BW, POLL_TIMEOUT, CYCLE_INTERVAL) tanpa menyentuh SPI/GPIO, dan argumen tak dikenal sekarang ditolak dengan pesan usage alih-alih diam-diam diabaikan. Tidak ada opsi baru selain -h; perilaku tanpa argumen tidak berubah (diverifikasi ulang: 10 siklus bersih, 100 % OK).

Catatan pengambilan log

- **EXP-03 sudah dijalankan** pada sesi 21 Agustus 2026 (lihat bagian EXP-03 di atas) — kabel USB Slave 2 dicabut fisik dan dipasang kembali sementara operator hadir langsung di lokasi board, master tetap dijalankan dari jarak jauh lewat SSH.
- **Tabel jarak (Pengukuran B) belum terisi.** Seluruh percobaan dijalankan di satu meja pada jarak tetap ± 30 cm.
- Kolom M05 pada tabel perbandingan lama siklus diisi dari Modul05_lora_master_slave/logserial.md, bukan dari pengukuran ulang pada sesi ini.
- Membuka port serial me-**reset** Arduino lewat DTR, sehingga RX# dan dataCounter slave kembali ke 1 sementara penghitung master terus berjalan. Bila kedua angka perlu sebanding, jalankan perekam serial lebih dahulu, baru master.
- Seluruh berkas diagnostik (master_diagA/B/C/D.py, master_exp04.py, spi_stress.py) sudah dihapus dari Raspberry Pi. Yang tersisa di sana hanya src/master.py, requirements.txt, dan cek_radio.py.
- src/master.py, src/slave/main.cpp, dan platformio.ini **tidak diubah sama sekali** selama pengujian ini.

Modifikasi CRC — 21 Agustus 2026

Sesi ini menerapkan CH-2 dari README Modul07_rpi_master_slave/ (menyalakan CRC payload di kedua sisi) sebagai modul tersendiri di direktori -withCRC. Raspberry Pi tetap pi@192.168.1.45 lewat SSH; kedua Uno tertancap USB di laptop pengembang.

Perubahan kode

- src/slave/main.cpp: LoRa.enableCrc(); ditambahkan di setup() setelah LoRa.setTxPower(), plus satu baris Serial.println(F("CRC payload: AKTIF")); pada banner boot. Komentar header disesuaikan untuk tidak lagi mengklaim identik-byte-untuk-byte dengan M05.
- src/master.py: dua fungsi baru, enableCrc()/disableCrc(), menulis/membersihkan bit 2 REG_MODEM_CONFIG_2 — persis padanan LoRa.enableCrc()/disableCrc() di pustaka sandeepmistry (dikonfirmasi lewat pembacaan LoRa.cpp: writeRegister(REG_MODEM_CONFIG_2, readRegister(REG_MODEM_CONFIG_2) | 0x04)). enableCrc() dipanggil di alur __main__ setelah setTxPower(TX_POWER), plus satu baris cetak CRC payload: AKTIF pada banner init.
- cek_radio.py: awalnya **tidak** memanggil M.enableCrc() — skrip ini mengonfigurasi radio secara manual lewat pemanggilan fungsi satu-satu (M.setSpreadingFactor, dst.), bukan lewat master.py utuh, jadi luput saat enableCrc() pertama kali ditambahkan. Percobaan pertama pembacaan register masih menunjukkan MODEM_CFG_2 : 0x70 -> CRC payload=MATI meski kode slave dan master.py sudah benar. Ditambahkan M.enableCrc() di cek_radio.py, dan pembacaan ulang langsung menunjukkan 0x74 -> CRC payload=AKTIF. Dicatat di sini karena ini contoh nyata kelas kegagalan

“kode benar, tapi skrip verifikasi sendiri belum diperbarui” — persis alasan bagian ini menuntut bukti pembacaan register, bukan sekadar membaca source.

Build & upload

Environment	Status	Duration	
slave1	SUCCESS	00:00:00.592	Flash: 26.6% (8.574 B)
slave2	SUCCESS	00:00:00.615	Flash: 26.6% (8.574 B)

Naik dari 8.522 B (26,4%) pada varian tanpa CRC — biaya `LoRa.enableCrc()` plus satu baris `Serial.println` tambahan, 52 byte per slave. Upload lewat avrdude ke `/dev/ttyACM0` (slave1) dan `/dev/ttyACM1` (slave2, dikonfirmasi via `tools/deteksi_port.py` sebelum upload), keduanya bytes written dan bytes verified sama dengan ukuran hex — SUCCESS.

Verifikasi register di Raspberry Pi

`src/master.py`, `cek_radio.py`, dan `requirements.txt` disalin ke `~/Documents/WSN-IOT-prak-Lora/Modul07b_rpi_master_slave_crc/` di Pi lewat scp, lalu `cek_radio.py` dijalankan di sana:

```
REG_VERSION : 0x12 (0x12 = SX1276/77/78/79)
FRF_register : 0x6c 0x40 0x00 -> frf=7094272
FREKUENSI : 433.000000 MHz (target 433.000000, selisih +0.0 Hz)
MODEM_CFG_1 : 0x72 -> BW=125 kHz | CR=4/5 | header=explicit
MODEM_CFG_2 : 0x74 -> SF7 | CRC payload=AKTIF
PA_CONFIG : 0x8f -> PA_BOOST, power=17 dBm
```

MODEM_CFG_2 naik dari 0x70 ke 0x74 — persis bit 2 yang dinyalakan `enableCrc()`, tidak ada bit lain yang ikut berubah.

Banner Serial kedua slave (dibaca dari `/dev/ttyACM0//dev/ttyACM1` via `pyserial`, DTR ditoggle eksplisit untuk memicu reset dan menangkap banner boot penuh)

```
=== LoRa SLAVE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
CRC payload: AKTIF
Menunggu POLL:1 dari Master (Raspberry Pi)...

=== LoRa SLAVE 2 ===
Init LoRa ... OK
Freq: 433.00 MHz
CRC payload: AKTIF
Menunggu POLL:2 dari Master (Raspberry Pi)...
```

Uji fungsional — sesi bersih ±55 detik

Prosedur: logger `pyserial` dibuka lebih dulu di kedua port (memicu reset Uno lewat DTR), ditunggu 3,5 detik agar bootloader + `setup()` selesai, baru `python3 -u master.py` dijalankan di Pi lewat SSH selama 55 detik — meniru urutan EXP-01 (slave siap sebelum master mulai polling).

```
=== LoRa MASTER-SLAVE 3 NODE ===
Init LoRa ... OK
Freq: 433 MHz
SF7 | BW: 125 kHz
CRC payload: AKTIF
Peran: MASTER (Raspberry Pi + LoRa GPS HAT)
Slave: Dragino Shield Uno - S1 & S2

=====
=== CYCLE 1 ===
[TX] POLL:1
[RX] S1:DATA:1 | RSSI: -72 dBm | SNR: 14.8 dB
[TX] POLL:2
[RX] S2:DATA:38 | RSSI: -64 dBm | SNR: 14.5 dB
--- STATISTIK ---
S1: OK=1 | FAIL=0 | Data: 1
S2: OK=1 | FAIL=0 | Data: 38
Durasi siklus: 165 ms
=====
...
--- STATISTIK ---
S1: OK=83 | FAIL=0 | Data: 83
S2: OK=83 | FAIL=0 | Data: 120
Durasi siklus: 166 ms
```

(Data: Slave 2 mulai dari 38, bukan 1 — dataCounter on-board-nya melanjutkan dari sesi uji coba sebelumnya di hari yang sama, bukan reboot penuh. Tidak memengaruhi validitas OK/FAIL karena keduanya dihitung per sesi master, bukan dari nilai absolut Data:.)

Parameter	Hasil
Total siklus dalam ± 55 detik	83
S1: OK / FAIL	83 / 0 $\rightarrow$ 100 %
S2: OK / FAIL	83 / 0 $\rightarrow$ 100 %
Baris [WARN] Balasan tidak valid di seluruh log master	0
Baris [FAIL] di seluruh log master	0
Durasi siklus min / maks / rata-rata	165 / 166 / 165,3 ms (n=83)
Sebaran durasi siklus	1 ms — sama rapatnya dengan sesi CRC-mati (144/145/145,0 ms), hanya bergeser +20 ms
RSSI master $\leftarrow$ S1	rata-rata -71,8 dBm (kisaran -83..-68 dBm)
RSSI master $\leftarrow$ S2	rata-rata -65,0 dBm (kisaran -67..-63 dBm)
SNR master $\leftarrow$ S1 / S2	14,64 dB / 14,63 dB
RX# akhir slave 1 / slave 2 (log lokal)	83 / 120 — S1 cocok persis dengan Data: master (83); S2 juga cocok (120) setelah memperhitungkan offset awal 38
Baris [IGNORE] di luar pola POLL: <n>/S<n>:DATA: <n> (log lokal, kedua slave)	0 dari 160 (S1) + 189 (S2) baris

Tabel 8.21: Log serial Modul 07B: Modifikasi CRC — 21 Agustus 2026

Interpretasi. Tidak ada satu pun payload rusak yang tertangkap secara langsung pada sesi 83-siklus ini untuk dibandingkan byte-per-byte dengan temuan “Payload rusak lolos karena CRC mati” di atas — konsisten dengan sifat sporadis fenomena itu (0,74% pada sebagian sesi CRC-mati, 0,00% pada sesi lain di bagian atas dokumen ini). Bukti bahwa proteksi CRC benar-benar aktif berasal dari tiga sumber independen: (1) pembacaan register langsung di Pi menunjukkan bit CRC menyala, (2) banner boot kedua slave mencetak CRC payload: AKTIF, dan (3) nol baris [WARN] di seluruh sesi — dengan CRC mati, sesi serupa pada sesi-sesi sebelumnya di dokumen ini rutin mencetak beberapa [WARN] per ratusan paket.

Kenaikan durasi siklus (+20 ms) dijelaskan secara fisik, bukan sebagai overhead pemrosesan. SX1276 menambahkan 2 byte checksum ke payload saat RxPayloadCrcOn menyala, memperpanjang waktu-di-udara paket di kedua arah (POLL dan S<id>:DATA:<n>). Kenaikan ini konsisten di seluruh 83 siklus (hanya dua nilai: 165 ms $\times$ 56, 166 ms $\times$ 27) — bukan jitter, melainkan pergeseran baseline yang dapat diprediksi.

Uji coba awal (dibuang, tidak dipakai sebagai data resmi). Sebelum sesi bersih di atas, dilakukan uji cepat ± 25 detik tanpa jeda boot — logger serial dan master.py dimulai hampir bersamaan. Hasilnya: 37 siklus, S1 36 OK/1 FAIL (FAIL persis di CYCLE 1), S2 37/37 OK. [FAIL] tunggal itu adalah race kondisi start-up yang sudah didokumentasikan sebelumnya di bagian “Catatan pengambilan log” (membuka port serial

me-reset Arduino lewat DTR) — bukan efek CRC — dan tidak muncul lagi setelah prosedur diperbaiki (tunggu boot sebelum start master) pada sesi resmi di atas.

Cakupan pengujian yang TIDAK dilakukan ulang sesi ini. EXP-03 (cabut kabel fisik) dan EXP-04 (sapuan POLL_TIMEOUT) tidak dijalankan ulang — keduanya menuntut intervensi fisik berulang atau sesi 60 detik × banyak titik yang di luar cakupan verifikasi modifikasi CRC kali ini. Tabel EXP-03/EXP-04 di README tetap berisi angka dari sesi CRC-mati sebelumnya dan **belum divalidasi** untuk varian ini; kenaikan waktu-di-udara yang terukur di atas (+20 ms/siklus) memberi dugaan terarah bahwa ambang tebing EXP-04 akan bergeser sedikit lebih tinggi dari 40–45 ms, tetapi ini belum diukur langsung.

8.9 Pengukuran

A. Keberhasilan terhadap jarak — kedua slave ditempatkan pada jarak sama dari Raspberry Pi, 30 siklus per baris.

Jarak	RSSI S1	RSSI S2	OK/FAIL S1	OK/FAIL S2	Berhasil S1 (%)	Berhasil S2 (%)
1 m						
25 m						
50 m						
100 m						

Tabel 8.22: Lembar pengukuran A — keberhasilan terhadap jarak

B. Perbandingan platform master dan biaya CRC

Ukuran	Modul ini (Pi 5, CRC aktif)	M07 (Pi 5, CRC mati)	M05 (Uno)
Durasi siklus minimum (ms)		144	147
Durasi siklus maksimum (ms)		145	154
Durasi siklus rata-rata (ms)		145,0	152
Sebaran (maks – min, ms)		1	7

Tabel 8.23: Lembar pengukuran B — perbandingan platform master dan biaya CRC

C. Ambang POLL_TIMEOUT — ulangi EXP-04 tiga kali pada dua titik.

Percobaan ke-	Nilai terkecil andal (ms)	Nilai pertama gagal total (ms)
1		
2		
3		

Tabel 8.24: Lembar pengukuran C — ambang POLL_TIMEOUT dengan CRC aktif

D. Rasio payload rusak yang lolos ke aplikasi — hitung dari sesi gabungan EXP-02 dan EXP-04 sendiri, lalu bandingkan dengan lembar pengukuran D pada Modul 07 (CRC mati). Dengan CRC aktif, rasio ini seharusnya **0 %** — paket rusak dibuang radio sebelum sempat menghasilkan [WARN].

Sesi	Paket diterima	Paket [WARN]	Rasio (%)
Sesi sendiri 1			
Sesi sendiri 2			
Referensi (83 siklus, ± 55 s, 21 Agustus 2026)	166 (83 S1 + 83 S2)	0	0,0

Tabel 8.25: Lembar pengukuran D — rasio payload rusak yang lolos

8.10 Analisis

1. Dari Tabel 8.23, sebutkan dua kemungkinan penyebab master Raspberry Pi memiliki sebaran durasi siklus lebih rapat daripada master Arduino, dan jelaskan mengapa `POLL_TIMEOUT` yang longgar (500 ms) membuat perbedaan itu tidak pernah terlihat pada durasi siklus akhir.
2. Dari Tabel 8.24, hitung anggaran waktu tunggu sesungguhnya (bukan naif) menggunakan waktu udara jawaban dari bagian Dasar Teori M05, ditambah waktu udara 2 byte checksum. Apakah nilai ambang yang ditemukan cocok dengan hitungan itu?
3. Pada EXP-03, mengapa siklus pertama setelah master baru dinyalakan bisa mencatat kedua slave gagal sekaligus, dan berapa perkiraan durasinya dibandingkan satu slave gagal?
4. Bandingkan Tabel 8.25 (CRC aktif) dengan lembar pengukuran D pada Modul 07 (CRC mati, 0,74 % dan 0,00 % pada sesi berbeda). Rasio pada varian ini seharusnya turun ke nol — jelaskan mengapa CRC mengubah *kelas* kegagalannya, bukan menghilangkan kerusakan paket itu sendiri (paket yang sama tetap rusak di udara; yang berubah hanya siapa yang membuangnya dan kapan).
5. Dengan CRC aktif, [WARN] Balasan tidak valid semestinya tidak pernah muncul lagi — payload rusak sekarang tercatat sebagai [FAIL], sama dengan slave yang benar-benar diam. Usulkan satu cara membedakan “slave diam” dari “jawaban dibuang karena CRC gagal” dari sisi statistik master saja, tanpa mematikan kembali CRC.
6. Bandingkan pekerjaan memindahkan master ke Raspberry Pi dengan pekerjaan memindahkan protokol seluruhnya ke LoRaWAN. Sebutkan satu keuntungan dan satu kerugian pendekatan “gateway custom” dibanding memakai protokol siap pakai.

8.11 Concept Check

1. Mengapa firmware slave tidak perlu tahu bahwa masternya sekarang Raspberry Pi, bukan Arduino?
2. Sebutkan dua cara mengorelasikan log dari dua mesin yang tidak berbagi jam, selain nomor urutan payload.
3. Mengapa `POLL_TIMEOUT` tidak menghitung waktu udara `POLL` itu sendiri? Fungsi mana di `master.py` yang menjadi penyebabnya?
4. Apa yang terjadi pada paket yang payload-nya rusak satu bit ketika CRC payload **aktif**, dan pada tahap mana paket itu dibuang — sebelum atau sesudah `parsePacket()` mengembalikan nilai bukan nol?
5. Satu baris `LoRa.enableCrc()` di slave dan satu pemanggilan `enableCrc()` di master sudah cukup mengubah perilaku ini. Mengapa perbaikan sederhana itu tetap dianggap “membatalkan klaim identik dengan M05” pada modul dasarnya?
6. Apa perbedaan mendasar antara “slave tidak menjawab” dan “jawaban dibuang karena CRC gagal” dari sudut pandang radio, dan mengapa master saat ini masih tidak membedakan keduanya dalam statistik OK dan FAIL meskipun [WARN] sudah tidak pernah muncul lagi?

8.12 Challenge (Tugas Modifikasi)

Level 2 — Modifikasi · Memperbesar jaringan dan mempertajam diagnosis

CH-1 — Slave ketiga Tambahkan Uno ketiga dengan `SLAVE_ID=3`, lalu ukur pertambahan lama siklus dan bandingkan dengan hasil CH-1 pada M05 dan M07.

CH-2 — Sudah diterapkan Tantangan CH-2 Modul 07 (menyalakan CRC) sudah diterapkan di modul ini — lihat `src/slave/main.cpp` dan `src/master.py/cek_radio.py`. Trade-off-nya: klaim “identik dengan M05” tidak lagi berlaku secara harfiah untuk firmware slave, tetapi diagnosis kegagalan menjadi lebih jujur. Variasi lanjutan: ukur ulang Tabel 8.25 dan EXP-04 di sini sendiri, karena keduanya belum diukur ulang pada sesi modifikasi ini.

Level 3 — Tantangan · Memanfaatkan informasi yang baru tersedia

CH-3 — Bedakan “diam” dan “dibuang CRC” Ubah `pollSlave()` di `master.py` agar mencatat statistik terpisah untuk “tidak menjawab sama sekali” versus “menjawab tapi dibuang radio karena CRC gagal” — pada varian ini `IRQ_CRC_ERROR` di `parsePacket()` sudah membedakan keduanya di level register, tinggal dipropagasikan ke statistik. Ukur apakah rasio keduanya berubah seiring durasi sesi.

CH-4 — Jadwal adaptif lintas platform Port ide CH-4 M05 ke `master.py`, lalu ukur perbaikan durasi siklus saat satu node mati.

CH-5 — Korelasi RSSI lintas mesin Jalankan `lora_monitor.py --out sesi.csv` di laptop bersamaan dengan `master.py` di Raspberry Pi selama sepuluh menit, gabungkan kedua rekaman berdasarkan nomor urut, lalu buat satu grafik RSSI dari kedua arah terhadap waktu.

8.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — mengapa slave tidak perlu tahu platform master, anggaran waktu sesungguhnya `POLL_TIMEOUT`, konsekuensi CRC aktif dibanding CRC mati.
3. Bukti diff firmware slave terhadap varian Modul 07, beserta penjelasan mengapa satu baris itu cukup mengubah perilaku perangkat keras.
4. Hasil eksperimen — keluaran terminal EXP-01 sampai EXP-04 dari kedua sisi beserta checkpoint, dan hasil `cek_radio.py`.
5. Data pengukuran — Tabel 8.22 sampai Tabel 8.25, disertai perbandingan eksplisit terhadap baseline Modul 07.
6. Analisis dan concept check, termasuk hitungan anggaran waktu `POLL_TIMEOUT` yang sudah memperhitungkan 2 byte checksum.
7. Challenge — minimal CH-1 dan CH-3.
8. Kesimpulan yang disusun sendiri mengenai harga dan manfaat menyalakan CRC payload pada jaringan LoRa mentah.

9. Modul 08 — ALOHA Tanpa Umpan Balik: Bebas Bicara

LoRa COMMUNICATION LAB · MODUL 08

Speak Freely, Collide Silently — ALOHA Tanpa Umpan Balik

Arduino Uno + Dragino LoRa Shield v1.2 · topologi bintang, 2 node + gateway · interrupt, tanpa balasan · level Intermediate · 1 × 50 menit · folder kode Modul08_lora_aloha_tanpa_ack

PENGANTAR MODUL · Gambaran Umum

Modul 08 dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat menengah. Misinya membalik total pelajaran M05: alih-alih menjadwalkan siapa yang boleh bicara, modul ini justru **melepas** seluruh penjadwalan dan membiarkan setiap node mengirim data kapan pun ia mau. Sistemnya sederhana — dua node dan satu gateway — tetapi pertanyaannya tajam: apa yang terjadi kalau semua node bebas mengirim tanpa koordinasi sama sekali?

9.1 Pendahuluan

M05 dan M07 menghabiskan seluruh usahanya menghindari tabrakan lewat penjadwalan terpusat. Modul ini sengaja mundur satu langkah untuk **menunjukkan** tabrakan itu, bukan menghindarinya — sebab protokol paling tua dalam sejarah jaringan paket radio, ALOHAnet (Universitas Hawaii, 1971), justru dimulai dari titik ini: kirim saja, dan terima risikonya. Memahami kegagalan protokol paling sederhana ini adalah prasyarat untuk menghargai setiap lapisan yang ditambahkan sesudahnya — ACK pada M08B, retry dengan backoff pada M08C, carrier sense pada M09, dan penjadwalan slot pada M10.

PENTING

Satu catatan istilah yang penting. Yang dijalankan di modul ini belum Pure ALOHA yang utuh. Rumusan asli Abramson (1970) sudah memuat **ACK dan pengiriman ulang setelah jeda acak** — keduanya sengaja belum ada di sini, supaya kanal kontensi terlihat telanjang lebih dulu. Jadi urutannya: M08 memperlihatkan kanal tanpa umpan balik sama sekali, M08B menambahkan umpan baliknya, dan **M08C-lah yang akhirnya menjadi Pure ALOHA lengkap**. Menyebut modul ini “Pure ALOHA” tidak salah dalam percakapan sehari-hari, tetapi di laporan sebutkan bagian mana dari protokol itu yang sudah ada dan mana yang belum.

Prasyaratnya adalah M01 untuk format payload dan penghitungan loss lewat nomor urut, serta M04 untuk pembacaan RSSI dan SNR per paket. Yang dibangun di sini adalah pembangkitan data dummy multi-sensor (suhu dan kelembaban, dua ruangan per node), pengiriman tanpa koordinasi kanal (tanpa *carrier sense*, tanpa ACK, tanpa retry), dan deteksi kehilangan lewat lompatan nomor urut di sisi gateway. Payload dan pola nomor urut ini dipakai lagi persis di M08B, M08C, dan M10 — hanya lapisan kendalinya yang bertambah.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
05	Tabrakan dicegah lewat polling terpusat — master memanggil satu per satu
07	Master pindah ke Raspberry Pi, penjadwalan tetap sama
08 (ini)	Penjadwalan dilepas — node kirim bebas, tabrakan senyap diamati
08B	ACK ditempelkan di atas M08 — node tahu SUCCESS/FAILED, belum ada retry

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
08C	Random backoff + retry — kegagalan dipulihkan, dan Pure ALOHA menjadi lengkap
09	Carrier sense — dengar dulu sebelum bicara, tabrakan dihindari sebelum terjadi
10	SYNC + slot waktu — Slotted ALOHA (slot diundi) vs TDMA (slot tetap)

Tabel 9.1: Peta modul pada arc kedua seri LoRa

Kontrak data lab ini. Setiap node mengirim satu paket berisi **dua ruangan sekaligus**: `NODE=<id>`, `SEQ=<n>`, `R1T=<suhu>`, `R1H=<lembab>`, `R2T=<suhu>`, `R2H=<lembab>`. Bentuk `KEY=VALUE` dipisah koma dipilih alih-alih format multi-baris (`NODE_ID=1` lalu `R00M1_TEMP=...`) yang sering dipakai pada contoh IoT berbasis teks — pada AVR dengan RAM 2 KB dan payload LoRa yang dibaca byte demi byte, satu baris tunggal jauh lebih murah untuk di-parse dengan `indexOf` dan `substring` tanpa risiko pemisahan baris yang keliru. Nomor urut `SEQ` naik di setiap pengiriman node, **tanpa** kaitan dengan ada-tidaknya balasan — sebab pada Pure ALOHA memang tidak ada balasan. Gateway memakai lompatan pada `SEQ` untuk memperkirakan berapa paket yang hilang, gagasan yang sama dengan penghitungan `loss Hello LoRa #n` pada M01.

9.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mengamati kanal kontensi tanpa kendali apa pun

1. Menjelaskan mekanisme Pure ALOHA: kirim tanpa dengar-dahulu (*no carrier sense*), tanpa ACK, tanpa retry.
2. Membangkitkan data dummy multi-sensor (dua besaran, dua ruangan) dan mengemasnya dalam satu payload ringkas.
3. Menunjukkan secara empiris bahwa dua node yang mengirim bebas pada kanal yang sama dapat bertabrakan, dan menjelaskan mengapa tabrakan itu **tidak muncul sebagai pesan galat** di kedua sisi.
4. Memakai lompatan nomor urut sebagai alat ukur kehilangan paket tanpa perlu balasan apa pun dari penerima.
5. Menghitung *throughput* efektif Pure ALOHA dan membandingkannya dengan hasil teoretis (puncak ~ 18,4 % pada beban $G = 0,5$).

Kriteria keberhasilan

- ☐ Kedua node mengirim data dummy dua ruangan secara mandiri, dengan interval acak yang berbeda satu sama lain.
- ☐ Gateway mencetak setiap paket yang diterima lengkap dengan RSSI, SNR, dan isi kedua ruangan.
- ☐ Ketika interval pengiriman dipersempit, gateway mencatat kemunculan [GAP] — tanda tabrakan atau kehilangan mulai terjadi.
- ☐ Statistik diterima dan perkiraan hilang per node terpisah dan bertambah wajar seiring waktu.

9.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Pure ALOHA	Protokol akses kanal paling sederhana: kirim kapan saja data siap, tanpa mendengarkan kanal lebih dulu.
Tabrakan (collision)	Dua paket menempati udara pada waktu yang tumpang tindih sehingga radio penerima tidak dapat mendekode keduanya.

Node	Environment	Peran	Mekanisme TX/RX	Interval kirim
Node 1	node1	Bangkitkan dan kirim dummy Ruang 1+2	TX blocking, tanpa RX	acak 2000–5000 ms
Node 2	node2	Bangkitkan dan kirim dummy Ruang 1+2	TX blocking, tanpa RX	acak 2000–5000 ms
Gateway	gateway	Terima dan cetak dari kedua node	Interrupt DIO0 + flag, tanpa TX	—

Tabel 9.3: Peran tiap node Modul 08

9.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3

Tabel 9.4: Alat dan bahan Modul 08

Struktur proyek

```
Modul08_lora_aloha_tanpa_ack/
├── platformio.ini
├── lora_monitor.py      ← dashboard 3-panel live (Gateway/Node1/Node2) + CSV
├── upload_auto.py      ← deteksi port otomatis saat unggah
├── logserial.md         ← cuplikan log serial aktual dari pengujian perangkat
├── src/
│   ├── node/main.cpp   ← dummy Ruang 1+2, kirim bebas (env node1, node2)
│   └── gateway/main.cpp ← terima & cetak, deteksi gap SEQ (env gateway)
```

9.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul08_lora_aloha_tanpa_ack/](#)

- [Modul08_lora_aloha_tanpa_ack/platformio.ini](#)
- [Modul08_lora_aloha_tanpa_ack/src/node/main.cpp](#)
- [Modul08_lora_aloha_tanpa_ack/src/gateway/main.cpp](#)
- [Modul08_lora_aloha_tanpa_ack/lora_monitor.py](#)
- [Modul08_lora_aloha_tanpa_ack/upload_auto.py](#)

```
[platformio]
default_envs = gateway

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
```

```

; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 115200

[env:gateway]
build_src_filter = +<gateway/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua node memakai source yang sama; nomornya ditentukan build flag.
[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=1
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=2
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2

```

Kode 9.1: platformio.ini Modul 08 — environment gateway, node1, node2

```

/*
LoRa Pure ALOHA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: node1 / node2

Node membangkitkan data dummy suhu & kelembaban untuk dua ruangan, lalu
mengirimkannya ke gateway KAPAN SAJA data itu siap -- tanpa mendengarkan
kanal lebih dahulu, tanpa menunggu balasan, tanpa mengirim ulang. Inilah
disiplin "Pure ALOHA": setiap node bebas berbicara, dan tabrakan adalah
konsekuensi yang diterima, bukan dihindari.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Interval : acak SEND_INTERVAL_MIN..SEND_INTERVAL_MAX ms, sengaja tidak
disamakan antar-node, meniru dua sumber data yang tidak
saling tahu jadwal satu sama lain.

Mekanisme:
TX : blocking (endPacket) -- tidak ada RX sama sekali di modul ini
Dummy sensor: random() di sekitar nilai dasar per ruangan

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor node datang dari build flag (-DNODE_ID=1 atau 2) di platformio.ini,
// sehingga kedua node memakai file source yang sama persis.
#ifndef NODE_ID
#define NODE_ID 1
#endif

#define SEND_INTERVAL_MIN 2000
#define SEND_INTERVAL_MAX 5000

// Ruangan 1 & Ruangan 2 dibedakan hanya lewat nilai dasar dummy-nya, supaya
// kedua ruangan terlihat berbeda di Serial Monitor walau datanya bukan hasil

```

```

// sensor sungguhan.
#define ROOM1_TEMP_BASE 27.0
#define ROOM1_HUM_BASE 60
#define ROOM2_TEMP_BASE 25.0
#define ROOM2_HUM_BASE 68

unsigned long seq = 0;
unsigned long txCount = 0;

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

// Dummy sensor: nilai dasar per ruangan +- jitter acak, dibulatkan 1 desimal
// untuk suhu. Bukan pembacaan sensor sungguhan -- lihat README modul ini.
float dummyTemp(float base) {
    return base + (random(-30, 31) / 10.0); // +-3.0 C
}

int dummyHum(int base) {
    return base + random(-15, 16); // +-15 %
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    randomSeed(analogRead(A0)); // pin dibiarkan mengambang -> noise ADC sebagai seed

    Serial.print(F("=== LoRa PURE ALOHA - NODE "));
    Serial.print(NODE_ID);
    Serial.println(F(" =="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.println(F("Peran: NODE (Pure ALOHA) -- kirim bebas, tanpa ACK, tanpa retry\n"));
}

void loop() {
    float r1t = dummyTemp(ROOM1_TEMP_BASE);
    int r1h = dummyHum(ROOM1_HUM_BASE);
    float r2t = dummyTemp(ROOM2_TEMP_BASE);
    int r2h = dummyHum(ROOM2_HUM_BASE);

    String payload = "NODE=" + String(NODE_ID) +
        ",SEQ=" + String(seq) +
        ",R1T=" + String(r1t, 1) +
        ",R1H=" + String(r1h) +
        ",R2T=" + String(r2t, 1) +
        ",R2H=" + String(r2h);

    digitalWrite(LED_PIN, HIGH);
    transmit(payload);
    txCount++;
    digitalWrite(LED_PIN, LOW);

    Serial.print(F("[TX] ")); Serial.print(payload);
    Serial.print(F(" | total dikirim: ")); Serial.println(txCount);

    seq++;

    unsigned long interval = random(SEND_INTERVAL_MIN, SEND_INTERVAL_MAX + 1);
    delay(interval);
}

```

Kode 9.2: src/node/main.cpp — dummy dua ruangan, kirim bebas tanpa ACK

```

/*
LoRa Pure ALOHA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: gateway

Gateway hanya mendengarkan. Ia tidak pernah mengirim apa pun -- tidak ada
ACK, tidak ada POLL -- karena Pure ALOHA tidak punya mekanisme balasan.
Setiap paket yang lolos parsePacket() dibaca, di-parse, dan dicetak; paket
yang bertabrakan di udara tidak pernah tiba di sini sama sekali, sehingga
hilangnya HANYA terlihat lewat lompatan nomor SEQ per node -- bukan lewat
pesan galat apa pun.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"

Mekanisme:
RX : interrupt DI00 + flag (non-blocking, loop() tidak pernah menunggu)

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_DURATION      150
#define NODE_COUNT        2 // Node 1 dan Node 2 -> indeks array 1..2

volatile bool rxFlag = false;

unsigned long ledOnTime = 0;
bool ledActive = false;

unsigned long rxCount[NODE_COUNT + 1] = {0};
long lastSeq[NODE_COUNT + 1];
unsigned long lostEst[NODE_COUNT + 1] = {0};

void onReceive(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

// Ambil nilai field "KEY=..." dari payload gaya "K1=v1,K2=v2,...".
// Mengembalikan "" bila key tidak ditemukan.
String getField(const String& data, const String& key) {
    String needle = key + "=";
    int start = data.indexOf(needle);
    if (start == -1) return "";
    start += needle.length();
    int end = data.indexOf(' ', start);
    if (end == -1) end = data.length();
    return data.substring(start, end);
}

```

```

void setup() {
  Serial.begin(115200);
  while (!Serial);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  for (int i = 0; i <= NODE_COUNT; i++) lastSeq[i] = -1;

  Serial.println(F("=== LoRa PURE ALOHA - GATEWAY ==="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
  }

  LoRa.setSpreadingFactor(SPREADING_FACTOR);
  LoRa.setSignalBandwidth(BANDWIDTH);
  LoRa.setCodingRate4(CODING_RATE);
  LoRa.setTxPower(TX_POWER);
  LoRa.onReceive(onReceive);
  LoRa.receive(); // langsung masuk RX mode, tidak pernah keluar

  Serial.println(F("OK"));
  Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
  Serial.println(F("Peran: GATEWAY (Pure ALOHA) -- hanya dengar, tidak pernah kirim ACK"));
  Serial.println(F("Menunggu paket dari Node 1 & Node 2...\n"));
}

void loop() {
  updateLED();

  if (!rxFlag) return;
  rxFlag = false;

  String received = "";
  while (LoRa.available()) {
    received += (char)LoRa.read();
  }

  if (!received.startsWith("NODE=")) {
    Serial.print(F("[WARN] Paket tak dikenal (mungkin sisa tabrakan): "));
    Serial.println(received);
    return;
  }

  int nodeId = getField(received, "NODE").toInt();
  long seq = getField(received, "SEQ").toInt();
  float r1t = getField(received, "R1T").toFloat();
  int r1h = getField(received, "R1H").toInt();
  float r2t = getField(received, "R2T").toFloat();
  int r2h = getField(received, "R2H").toInt();

  triggerLED();

  Serial.println(F("=== PAKET DITERIMA ==="));
  Serial.print(F(" Node : ")); Serial.println(nodeId);
  Serial.print(F(" SEQ : ")); Serial.println(seq);
  Serial.print(F(" Ruang 1 : ")); Serial.print(r1t, 1); Serial.print(F(" C, "));
  Serial.print(r1h); Serial.println(F(" %"));
  Serial.print(F(" Ruang 2 : ")); Serial.print(r2t, 1); Serial.print(F(" C, "));
  Serial.print(r2h); Serial.println(F(" %"));
  Serial.print(F(" RSSI : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
  Serial.print(F(" SNR : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));

  if (nodeId >= 1 && nodeId <= NODE_COUNT) {
    rxCount[nodeId]++;

    if (lastSeq[nodeId] >= 0) {
      long gap = seq - lastSeq[nodeId] - 1;
      if (gap > 0) {
        lostEst[nodeId] += gap;
        Serial.print(F(" [GAP] SEQ meloncat "));
        Serial.print(gap);
        Serial.println(F(" -- indikasi tabrakan/paket hilang"));
      }
    }
    lastSeq[nodeId] = seq;

    Serial.print(F(" Statistik Node ")); Serial.print(nodeId);
  }
}

```

```

        Serial.print(F(": diterima=")); Serial.print(rxCount[nodeId]);
        Serial.print(F(" | perkiraan hilang=")); Serial.println(lostEst[nodeId]);
    }

    Serial.println(F("=====\n"));
}

```

Kode 9.3: src/gateway/main.cpp — terima, cetak, dan deteksi lompatan SEQ

```

#!/usr/bin/env python3
"""
LoRa Pure ALOHA Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Gateway <-> Node 1 & Node 2 (kirim bebas tanpa carrier-sense, tanpa ACK, tanpa retry)

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
    pip install pyserial rich

Cara pakai:
    python lora_monitor.py                                # port bawaan sesuai OS
    python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
    python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.table import Table
    from rich.layout import Layout
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.columns import Columns
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

#
# KONFIGURASI
#
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --gateway/--n1/--n2.
if sys.platform.startswith("win"):
    DEFAULT_GW_PORT = "COM3"
    DEFAULT_N1_PORT = "COM4"
    DEFAULT_N2_PORT = "COM5"
else:
    DEFAULT_GW_PORT = "/dev/ttyACM0"
    DEFAULT_N1_PORT = "/dev/ttyACM1"
    DEFAULT_N2_PORT = "/dev/ttyACM2"
DEFAULT_BAUD = 115200
LOG_LINES = 16
REFRESH_HZ = 4
GAP_WARN_STREAK = 3

```

```

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default gateway/n1/n2 selalu
    mengikuti apa yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))
# Pure ALOHA (M08A) tidak punya ACK sama sekali -- node tidak pernah tahu
# apakah paketnya sampai, jadi baris OK/FAIL/Retry di panel node disembunyikan.
HAS_ACK = False

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class GatewayState:
    port: str = DEFAULT_GW_PORT
    connected: bool = False
    rx_total: int = 0
    rx_n1: int = 0
    rx_n2: int = 0
    gap_n1: int = 0
    gap_n2: int = 0
    lost_n1: int = 0
    lost_n2: int = 0
    dup_n1: int = 0
    dup_n2: int = 0
    last_rssi: int = 0
    last_snr: float = 0.0
    rssi_sum: int = 0
    snr_sum: float = 0.0
    rssi_count: int = 0
    cycle: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

@dataclass
class NodeState:
    name: str = ""
    port: str = ""
    connected: bool = False
    tx_count: int = 0
    ok_count: int = 0
    fail_count: int = 0
    retry_count: int = 0
    last_seq: int = -1
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.gw = GatewayState()
        self.n1 = NodeState("NODE 1", DEFAULT_N1_PORT)
        self.n2 = NodeState("NODE 2", DEFAULT_N2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSERS
# -----
# Semua regex dibuat agar aman dipakai lintas modul (M08A/M08B/M09/M10) --
# token yang tidak muncul pada suatu modul (mis. RETRY/SYNC) hanya tidak
# pernah cocok, tanpa memengaruhi parsing token lain.

RE_NODE_LINE = re.compile(r'Node\s+:\s*(\d+)')
RE_SEQ_LINE = re.compile(r'SEQ\s+:\s*(\d+)')
RE_RSSI_LINE = re.compile(r'RSSI\s+:\s*(-?\d+)')

```



```

RE_SNR_LINE = re.compile(r'SNR\s+:\s*(-?\d+\.\d*)')
RE_GAP_LINE = re.compile(r'\[GAP\] SEQ meloncat\s+(\d+)')
RE_STAT_LINE = re.compile(r'Statistik Node\s+(\d+):\s*diterima=(\d+)\s*\|\s*perkiraan hilang=(\d+)')
RE_SYNC_TX = re.compile(r'\[TX\]\s+SYNC=(\d+)')
RE_CYCLE_DONE = re.compile(r'Cycle\s+(\d+)\s+selesai')
# M09: "Statistik Node X: baru=Y | duplicate=Z | gagal permanen (est.)=W"
RE_STAT_RETRY = re.compile(r'Statistik Node\s+(\d+):\s*baru=(\d+)\s*\|\s*duplicate=(\d+)\s*\|\s*gagal permanen\s*\s*(est\.)=(\d+)')
# M10: "--- Cycle N selesai | N1: diterima=Y hilang=Z N2: diterima=Y hilang=Z ---"
RE_STAT_CYCLE = re.compile(r'N(\d+):\s*diterima=(\d+)\s*hilang=(\d+)')

RE_NODE_TX = re.compile(r'\[TX\].*NODE=(\d+),SEQ=(\d+)')
RE_NODE_RETRY = re.compile(r'\[RETRY\s+(\d+)/(\d+)\]')
RE_NODE_OK = re.compile(r'\[OK\]')
RE_NODE_FAIL = re.compile(r'\[FAIL\]')

def parse_gateway_line(line: str, st: GatewayState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    if 'PAKET DITERIMA' in line:
        st.rx_total += 1

    m = RE_RSSI_LINE.search(line)
    if m:
        st.last_rssi = int(m.group(1))
        st.rssi_sum += st.last_rssi
        st.rssi_count += 1

    m = RE_SNR_LINE.search(line)
    if m:
        st.last_snr = float(m.group(1))
        st.snr_sum += st.last_snr

    m = RE_GAP_LINE.search(line)
    if m:
        gap = int(m.group(1))
        anomalies.append(Anomaly(ts, "GATEWAY", "GAP", f"SEQ meloncat {gap} -- indikasi tabrakan/paket hilang"))

    m = RE_STAT_LINE.search(line)
    if m:
        node_id, rx, lost = int(m.group(1)), int(m.group(2)), int(m.group(3))
        if node_id == 1:
            st.rx_n1, st.lost_n1 = rx, lost
        elif node_id == 2:
            st.rx_n2, st.lost_n2 = rx, lost

    m = RE_STAT_RETRY.search(line)
    if m:
        node_id, rx, dup, lost = int(m.group(1)), int(m.group(2)), int(m.group(3)), int(m.group(4))
        if node_id == 1:
            st.rx_n1, st.dup_n1, st.lost_n1 = rx, dup, lost
        elif node_id == 2:
            st.rx_n2, st.dup_n2, st.lost_n2 = rx, dup, lost

    for node_id_s, rx_s, lost_s in RE_STAT_CYCLE.findall(line):
        node_id, rx, lost = int(node_id_s), int(rx_s), int(lost_s)
        if node_id == 1:
            st.rx_n1, st.lost_n1 = rx, lost
        elif node_id == 2:
            st.rx_n2, st.lost_n2 = rx, lost

    m = RE_SYNC_TX.search(line)
    if m:
        st.cycle = int(m.group(1))

    m = RE_CYCLE_DONE.search(line)
    if m:
        st.cycle = int(m.group(1))

def parse_node_line(line: str, st: NodeState, node_label: str, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = RE_NODE_TX.search(line)
    if m:
        seq = int(m.group(2))
        if st.last_seq >= 0 and seq == st.last_seq and '[RETRY' not in line:
            pass # aman, tidak dipakai
        st.last_seq = seq
        if '[RETRY' not in line:
            st.tx_count += 1

```

```

m = RE_NODE_RETRY.search(line)
if m:
    st.retry_count += 1

if RE_NODE_OK.search(line):
    st.ok_count += 1
elif RE_NODE_FAIL.search(line):
    st.fail_count += 1
    if st.fail_count and st.fail_count % 5 == 0:
        anomalies.append(Anomaly(ts, node_label, "FAIL_COUNT",
                                   f"{node_label} sudah {st.fail_count}x gagal (tanpa ACK)"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
                  shared: SharedState, role: str):
    if role == "GW":
        state = shared.gw
    elif node_key == "N1":
        state = shared.n1
    else:
        state = shared.n2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                if role == "GW":
                                    parse_gateway_line(line, state, shared.anomalies)
                                else:
                                    parse_node_line(line, state, node_key, shared.anomalies)
                            shared.csv_queue.put({
                                'ts': ts, 'node': node_key, 'raw': line
                            })
                        except (serial.SerialException, OSError):
                            break
                    except (serial.SerialException, OSError):
                        with shared.lock:
                            state.connected = False
                        time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                            'node': row['node'],
                            'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"

```

```

    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str) -> str:
    if '[FAIL]' in line: return "red"
    if '[GAP]' in line: return "bold red"
    if '[RETRY]' in line: return "bold yellow"
    if '[BACKOFF]' in line: return "yellow"
    if '[OK]' in line: return "bold green"
    if '[TX]' in line: return "cyan"
    if 'PAKET DITERIMA' in line: return "green"
    if '[WAIT]' in line: return "dim"
    if '[WARN]' in line: return "yellow"
    return "dim white"

def build_gateway_panel(st: GatewayState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" Total diterima : ", style="dim"); t.append(f"{st.rx_total}\n", style="bold white")
    if st.cycle:
        t.append(" Cycle terakhir : ", style="dim"); t.append(f"{st.cycle}\n", style="bold cyan")
    t.append("\n")

    dup_suffix1 = f" dup={st.dup_n1}" if st.dup_n1 else ""
    dup_suffix2 = f" dup={st.dup_n2}" if st.dup_n2 else ""
    t.append(" Node 1 ", style="bold cyan")
    t.append(f"diterima={st.rx_n1:<4} hilang={st.lost_n1}{dup_suffix1}\n", style="white")
    t.append(" Node 2 ", style="bold magenta")
    t.append(f"diterima={st.rx_n2:<4} hilang={st.lost_n2}{dup_suffix2}\n\n", style="white")

    rssi_avg = (st.rssi_sum / st.rssi_count) if st.rssi_count else 0.0
    snr_avg = (st.snr_sum / st.rssi_count) if st.rssi_count else 0.0
    t.append(" RSSI terakhir : ", style="dim"); t.append(f"{st.last_rssi} dBm\n", style="white")
    t.append(" SNR terakhir : ", style="dim"); t.append(f"{st.last_snr:.2f} dB\n", style="white")
    t.append(" RSSI rata-rata : ", style="dim"); t.append(f"{rssi_avg:.1f} dBm\n", style="white")
    t.append(" SNR rata-rata : ", style="dim"); t.append(f"{snr_avg:.2f} dB\n\n", style="white")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:40]}\n", style=style)

    return Panel(t, title="[bold cyan]GATEWAY[/]",
                border_style="cyan", box=box.ROUNDED)

def build_node_panel(st: NodeState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" TX terkirim : ", style="dim"); t.append(f"{st.tx_count:>6}\n", style="bold white")
    t.append(" SEQ terakhir : ", style="dim"); t.append(f"{st.last_seq:>6}\n\n", style="white")

    if HAS_ACK:
        tot = st.ok_count + st.fail_count
        pct = (st.ok_count / tot * 100) if tot else 0.0
        t.append(" OK : ", style="dim"); t.append(f"{st.ok_count:>6}\n", style="bold green")
        t.append(" FAIL : ", style="dim"); t.append(f"{st.fail_count:>6}\n",
            style="bold red" if st.fail_count else "dim")
        if tot:
            t.append(" Rate : ", style="dim"); t.append(f"{pct:>6.2f}%\n", style=pct_style(st.ok_count,
tot))
        t.append(" Retry : ", style="dim"); t.append(f"{st.retry_count:>6}\n\n",
            style="yellow" if st.retry_count else "dim")
    else:
        t.append(" (Pure ALOHA -- tanpa ACK, node tidak pernah tahu\n", style="dim")
        t.append(" apakah paketnya sampai. Lihat panel GATEWAY untuk\n", style="dim")
        t.append(" statistik diterima/hilang per node.)\n\n", style="dim")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

    return Panel(t, title=f"[bold {color}]{label}[/]",
                border_style=color, box=box.ROUNDED)

```

```

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                        csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'GAP': "bold red",
        'FAIL_COUNT': "bold yellow",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=style)
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        gw = GatewayState(
            port=shared.gw.port, connected=shared.gw.connected,
            rx_total=shared.gw.rx_total, rx_n1=shared.gw.rx_n1, rx_n2=shared.gw.rx_n2,
            gap_n1=shared.gw.gap_n1, gap_n2=shared.gw.gap_n2,
            lost_n1=shared.gw.lost_n1, lost_n2=shared.gw.lost_n2,
            dup_n1=shared.gw.dup_n1, dup_n2=shared.gw.dup_n2,
            last_rssi=shared.gw.last_rssi, last_snr=shared.gw.last_snr,
            rssi_sum=shared.gw.rssi_sum, snr_sum=shared.gw.snr_sum,
            rssi_count=shared.gw.rssi_count, cycle=shared.gw.cycle,
            raw_log=deque(shared.gw.raw_log),
        )
        n1 = NodeState(
            name=shared.n1.name, port=shared.n1.port, connected=shared.n1.connected,
            tx_count=shared.n1.tx_count, ok_count=shared.n1.ok_count,
            fail_count=shared.n1.fail_count, retry_count=shared.n1.retry_count,
            last_seq=shared.n1.last_seq,
            raw_log=deque(shared.n1.raw_log),
        )
        n2 = NodeState(
            name=shared.n2.name, port=shared.n2.port, connected=shared.n2.connected,
            tx_count=shared.n2.tx_count, ok_count=shared.n2.ok_count,
            fail_count=shared.n2.fail_count, retry_count=shared.n2.retry_count,
            last_seq=shared.n2.last_seq,
            raw_log=deque(shared.n2.raw_log),
        )
        anom = deque(shared.anomalies)
        t_start = shared.session_start

    node_row = Columns([
        build_gateway_panel(gw),
        build_node_panel(n1, "NODE 1", "green"),
        build_node_panel(n2, "NODE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\

```

```

    LoRa PURE ALOHA Monitor – Evaluasi Praktikum
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz

```

```

Tool ini membaca data serial dari 3 board LoRa (Gateway, Node 1, Node 2)
secara bersamaan dan menampilkan dashboard real-time: statistik
diterima/OK/FAIL, RSSI/SNR, deteksi anomali (GAP/FAIL beruntun),
serta logging CSV otomatis untuk analisis data penelitian.

PENGGUNAAN:
python lora_monitor.py [opsi...]

OPSI:
--gateway PORT    Port COM untuk Gateway          (default: {gw})
--n1 PORT         Port COM untuk Node 1            (default: {n1})
--n2 PORT         Port COM untuk Node 2            (default: {n2})
--baud N          Baud rate serial semua port      (default: {baud})
--out FILE        Nama file CSV output             (default: lora_session_YYYYMMDD_HHMMSS.csv)
-h / --help      Tampilkan pesan bantuan ini

CONTOH:
python lora_monitor.py
-> Jalankan dengan port & baud rate default

python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
-> Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

python lora_monitor.py --out data_jarak_10m.csv
-> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
Ctrl+C          Hentikan monitoring & simpan CSV

CATATAN:
* Pastikan semua board sudah di-upload firmware dengan baud {baud}
* Upload Gateway dulu, baru Node 1 dan Node 2, sebelum menjalankan tool ini
* File CSV tersimpan otomatis saat Ctrl+C ditekan
"""
format(
    gw=DEFAULT_GW_PORT,
    n1=DEFAULT_N1_PORT,
    n2=DEFAULT_N2_PORT,
    baud=DEFAULT_BAUD,
)

def parse_args(port_aktif):
    gw_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_GW_PORT
    n1_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_N1_PORT
    n2_default = port_aktif[2] if len(port_aktif) > 2 else DEFAULT_N2_PORT

    p = argparse.ArgumentParser(
        description="LoRa Pure ALOHA Monitor Dashboard -- tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f"python lora_monitor.py\n"
            f"python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2\n"
            f"python lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--gateway', default=gw_default,
        metavar='PORT', help=f'Port COM Gateway (default: {gw_default}, dari deteksi port aktif)')
    p.add_argument('--n1', default=n1_default,
        metavar='PORT', help=f'Port COM Node 1 (default: {n1_default}, dari deteksi port aktif)')
    p.add_argument('--n2', default=n2_default,
        metavar='PORT', help=f'Port COM Node 2 (default: {n2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
        metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None,
        metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
    return p, p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    p, args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1

    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
    )

```

```

)
console.print(
    f"[dim]Dipakai -> "
    f"Gateway=[yellow]{args.gateway}[/] "
    f"N1=[green]{args.n1}[/] "
    f"N2=[magenta]{args.n2}[/] "
    f"Baud=[white]{args.baud}[/][dim]\n"
)

shared = SharedState()
shared.gw.port = args.gateway
shared.n1.port = args.n1
shared.n2.port = args.n2

ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
csv_path = args.out or f"lora_session_{ts_str}.csv"

if not no_args:
    console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
    console.print(f" Gateway : [yellow]{args.gateway}[/]")
    console.print(f" Node 1 : [green]{args.n1}[/]")
    console.print(f" Node 2 : [magenta]{args.n2}[/]")
    console.print(f" Baud : [white]{args.baud}[/]")

console.print(f" CSV : [dim]{csv_path}[/]")
console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
try:
    input()
except KeyboardInterrupt:
    return

threads = [
    threading.Thread(target=reader_thread,
        args=(args.gateway, args.baud, "GW", shared, "GW"),
        daemon=True, name="reader-gateway"),
    threading.Thread(target=reader_thread,
        args=(args.n1, args.baud, "N1", shared, "NODE"),
        daemon=True, name="reader-n1"),
    threading.Thread(target=reader_thread,
        args=(args.n2, args.baud, "N2", shared, "NODE"),
        daemon=True, name="reader-n2"),
    threading.Thread(target=csv_writer_thread,
        args=(shared, csv_path),
        daemon=True, name="csv-writer"),
]
for t in threads:
    t.start()

try:
    with Live(build_layout(shared, csv_path),
        console=console,
        refresh_per_second=REFRESH_HZ,
        screen=True) as live:
        while True:
            time.sleep(1.0 / REFRESH_HZ)
            live.update(build_layout(shared, csv_path))
except KeyboardInterrupt:
    pass
finally:
    shared.running = False
    time.sleep(0.6)
    console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 9.4: lora_monitor.py — dasbor tiga panel dengan perekaman CSV

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 08A (Pure ALOHA) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul08_lora_aloha_tanpa_ack -e gateway -t upload -t monitor
    pio run -d Modul08_lora_aloha_tanpa_ack -e node1 -t upload -t monitor
    pio run -d Modul08_lora_aloha_tanpa_ack -e node2 -t upload -t monitor

Skrup ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `gateway` (port pertama), `node1` (port kedua), dan `node2`

```

(port ketiga), lalu menjalankan `pio run` dengan `--upload-port` sesuai hasil deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga board selesai di-upload, skrip bertanya apakah mau langsung menjalankan lora_monitor.py -- bila ya, dijalankan dengan --gateway ... --n1 ... --n2 ... persis memakai port yang barusan dipakai upload (bukan deteksi ulang), supaya tidak mungkin salah pasang.

```
python3 Modul08_lora_aloha_tanpa_ack/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul08_lora_aloha_tanpa_ack/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti lora_monitor.py

python3 Modul08_lora_aloha_tanpa_ack/upload_auto.py --only gateway
-> upload satu environment saja

python3 Modul08_lora_aloha_tanpa_ack/upload_auto.py --gateway /dev/ttyACM0 --node1 /dev/ttyACM1 --
node2 /dev/ttyACM2
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["gateway", "node1", "node2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"gateway": "--gateway", "node1": "--n1", "node2": "--n2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port gateway/node1/node2 PERSIS seperti yang baru dipakai upload
    -- bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
    for env, opsi in ENV_KE_OPSI.items():
        if env in port_env:
            cmd += [opsi, port_env[env]]

    print(" $", " ".join(cmd))
    try:
        subprocess.call(cmd)
```

```

except KeyboardInterrupt:
    # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
    # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
    # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
    # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
    print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--gateway", metavar="/dev/ttyXXX",
                    help="timpa port gateway (lewati deteksi otomatis)")
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="timpa port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="timpa port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (gateway, node1, atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                        "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau ketiganya"
                        "sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"gateway": args.gateway, "node1": args.node1, "node2": args.node2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 9.5: upload_auto.py — pemilih port otomatis saat unggah

9.7 Build dan Flash

Gateway lebih dahulu, supaya paket pertama dari node langsung tertangkap.

```

pio run -d Modul08_lora_aloha_tanpa_ack -e gateway -t upload -t monitor
pio run -d Modul08_lora_aloha_tanpa_ack -e node1 -t upload -t monitor
pio run -d Modul08_lora_aloha_tanpa_ack -e node2 -t upload -t monitor

```

Monitor dashboard. python3 lora_monitor.py membaca ketiga port sekaligus dan menampilkan panel Gateway, Node 1, dan Node 2 (statistik diterima, RSSI dan SNR, deteksi [GAP]) di terminal, plus perekaman CSV otomatis. Memerlukan pip install pyserial rich. Jalankan setelah ketiga board selesai diunggah.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ Port ketiga board dicatat lewat `pio device list` (atau `python3 ../tools/deteksi_port.py`) dan diisikan ke `platformio.ini`.
- ☐ Tiga Serial Monitor 115200 baud siap, ketiganya terlihat bersamaan.
- ☐ `NODE_ID` pada `node1` dan `node2` sudah benar, dicek dari baris pembuka `NODE 1` atau `NODE 2` di Serial Monitor.

9.8 Percobaan

9.8.1 EXP-01 — Dua Node Mengirim Bebas

Nyalakan ketiga board dan amati gateway selama beberapa menit tanpa mengubah apa pun.

Expected output — node

```
=== LoRa PURE ALOHA - NODE 2 ===
Init LoRa ... OK
Freq: 433.00 MHz
Peran: NODE (Pure ALOHA) -- kirim bebas, tanpa ACK, tanpa retry

[TX] NODE=2,SEQ=0,R1T=26.5,R1H=54,R2T=22.6,R2H=55 | total dikirim: 1
[TX] NODE=2,SEQ=1,R1T=24.8,R1H=58,R2T=22.7,R2H=59 | total dikirim: 2
```

Expected output — gateway

```
=== PAKET DITERIMA ===
Node      : 2
SEQ       : 1
Ruang 1   : 24.8 C, 58 %
Ruang 2   : 22.7 C, 59 %
RSSI      : -58 dBm
SNR       : 9.50 dB
Statistik Node 2: diterima=1 | perkiraan hilang=0
=====
```

Data capture — diukur 90 detik (bukan 5 menit; lihat `logserial.md` untuk cuplikan log dan metodologi rekam).

Parameter	Hasil
Jumlah paket diterima gateway (90 detik) — Node 1	27
Jumlah paket diterima gateway (90 detik) — Node 2	24
Jumlah [GAP] muncul — Node 1 / Node 2	0 / 0
RSSI dan SNR rata-rata kedua node	Node 1: - 45,9 dBm / 10,00 dB — Node 2: - 57,9 dBm / 9,79 dB

Tabel 9.5: Hasil pengamatan EXP-01 pada sesi verifikasi perangkat

CHECKPOINT

Terpenuhi. Pada jarak dekat dan interval kirim standar (2–5 detik), [GAP] **tidak muncul sama sekali** selama 90 detik (51 paket total) — sesuai prediksi teoretis: pada beban rendah, peluang tumpang-tindih dua node sangat kecil meski tanpa carrier sense. Beda RSSI ± 12 dB antar node murni posisi fisik di meja pengujian, bukan indikasi masalah.

9.8.2 EXP-02 — Memaksa Tabrakan

Perkecil `SEND_INTERVAL_MIN` dan `SEND_INTERVAL_MAX` pada **kedua** node menjadi mendekati sama (misalnya 300–500 ms untuk keduanya), unggah ulang, lalu amati gateway.

Data capture

Parameter	Hasil
Interval kirim yang dipakai (ms)	
Jumlah [GAP] per menit — Node 1 / Node 2	
Perkiraan <i>throughput</i> gateway (diterima / seharusnya dikirim)	
Bandingkan dengan prediksi teoretis Pure ALOHA (~ 18,4 % pada beban puncak)	

Tabel 9.6: Lembar pengamatan EXP-02 — memaksa tabrakan

BUKA ABSTRAKSINYA

Di `src/node/main.cpp`, node **tidak pernah** memanggil `LoRa.receive()` atau menunggu apa pun setelah `endPacket()`. Jelaskan mengapa hal ini membuat node tidak pernah bisa tahu apakah paketnya bertabrakan, lalu telusuri: informasi apa yang **hilang total** dibanding modul ACK (M04, M08B) pada titik ini?

CHECKPOINT

Interval yang dipersempit harus menaikkan jumlah [GAP] di gateway secara terlihat. Bila tidak ada perubahan sama sekali, periksa apakah kedua node benar-benar terunggah ulang dengan interval baru.

9.8.3 EXP-03 — Payload Dua Ruangan

Bandingkan isi payload dua node dan pastikan keduanya membawa data Ruang 1 **dan** Ruang 2 dalam satu paket, bukan dua paket terpisah.

Data capture

Parameter	Hasil
Jumlah field dalam satu payload	6 (NODE, SEQ, R1T, R1H, R2T, R2H)
Apakah Ruang 1 dan Ruang 2 selalu tiba bersamaan (satu paket)?	ya — tidak pernah terpisah pada 51 paket yang diamati
Ukuran payload (jumlah karakter)	45, misalnya NODE=1, SEQ=2, R1T=29.8, R1H=59, R2T=25.4, R2H=68

Tabel 9.7: Hasil pengamatan EXP-03 — payload dua ruangan

CHECKPOINT

Terpenuhi. Satu paket selalu membawa **kedua** ruangan sekaligus. Ini yang membuat modul ini lebih hemat lalu lintas radio dibanding mengirim empat paket terpisah (Suhu R1, Lembab R1, Suhu R2, Lembab R2) untuk data yang sama.

9.8.4 Verifikasi Perangkat Keras

CATATAN

Diuji di perangkat pada 22 Agustus 2026 — tiga Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway, node1, node2; port /dev/ttyACM0/1/2, sudah cocok dengan platformio.ini bawaan). Build dan upload ketiga environment sukses tanpa modifikasi kode. EXP-01 dan EXP-03 dijalankan dan datanya nyata (lihat tabel di atas serta logserial.md). EXP-02 (memaksa tabrakan dengan interval sempit) memerlukan mengubah SEND_INTERVAL_MIN/MAX di kode dan unggah ulang kedua node — **belum dijalankan pada sesi verifikasi ini**, diserahkan sebagai latihan praktikum sesuai instruksi modul.

9.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Ketiga board di satu meja, jarak berbeda-beda (lihat RSSI di bawah — Node 1 lebih dekat ke gateway daripada Node 2). Interval kirim bawaan (2000–5000 ms acak) pada kedua node, tidak diubah.

Board & Port

Peran	Environment	Port	Board
Gateway	gateway	/dev/ttyACM0	Uno asli (2341:0043)
Node 1	node1	/dev/ttyACM1	Uno asli (2341:0043)
Node 2	node2	/dev/ttyACM2	Uno asli (2341:0043)

Tabel 9.8: Log serial Modul 08: Board & Port

Ketiga aliran serial direkam bersamaan (skrip capture terpisah 3 proses pyserial, satu per port), jendela rekam 90 detik setelah ketiga board selesai di-*upload* dan *boot*.

EXP-01 — Dua Node Mengirim Bebas

Cuplikan 7 detik pertama sesi rekam, memperlihatkan boot Node 2, boot Gateway, lalu dua paket pertama yang tiba (Node 2 SEQ=1, Node 1 SEQ=1 — SEQ=0 kedua node terkirim sebelum jendela rekam mulai):

```
[07:42:53.243] N2 | === LoRa PURE ALOHA - NODE 2 ===
[07:42:53.259] N2 | Init LoRa ... OK
[07:42:53.259] N2 | Freq: 433.00 MHz
[07:42:53.267] N2 | Peran: NODE (Pure ALOHA) -- kirim bebas, tanpa ACK, tanpa retry
[07:42:53.357] N2 | [TX] NODE=2,SEQ=0,R1T=26.5,R1H=54,R2T=22.6,R2H=55 | total dikirim: 1
[07:42:54.004] GW | === LoRa PURE ALOHA - GATEWAY ===
[07:42:54.024] GW | Init LoRa ... OK
[07:42:54.024] GW | Freq: 433.00 MHz
[07:42:54.028] GW | Peran: GATEWAY (Pure ALOHA) -- hanya dengar, tidak pernah kirim ACK
[07:42:54.032] GW | Menunggu paket dari Node 1 & Node 2...
[07:42:56.147] N2 | [TX] NODE=2,SEQ=1,R1T=24.8,R1H=58,R2T=22.7,R2H=59 | total dikirim: 2
[07:42:56.150] GW | === PAKET DITERIMA ===
[07:42:56.150] GW | Node      : 2
[07:42:56.154] GW | SEQ       : 1
[07:42:56.154] GW | Ruang 1  : 24.8 C, 58 %
[07:42:56.158] GW | Ruang 2  : 22.7 C, 59 %
[07:42:56.158] GW | RSSI     : -58 dBm
[07:42:56.162] GW | SNR      : 9.50 dB
[07:42:56.166] GW | Statistik Node 2: diterima=1 | perkiraan hilang=0
[07:42:56.166] GW | =====
[07:42:56.936] GW | === PAKET DITERIMA ===
[07:42:56.936] GW | Node      : 1
[07:42:56.940] GW | SEQ       : 1
[07:42:56.940] GW | Ruang 1  : 30.0 C, 45 %
[07:42:56.944] GW | Ruang 2  : 24.8 C, 69 %
[07:42:56.944] GW | RSSI     : -46 dBm
[07:42:56.945] GW | SNR      : 9.75 dB
[07:42:56.952] GW | Statistik Node 1: diterima=1 | perkiraan hilang=0
[07:42:56.953] GW | =====
[07:42:59.158] N1 | [TX] NODE=1,SEQ=2,R1T=29.8,R1H=59,R2T=25.4,R2H=68 | total dikirim: 3
[07:42:59.160] GW | === PAKET DITERIMA ===
[07:42:59.160] GW | Node      : 1
```

```
[07:42:59.164] GW | SEQ      : 2
[07:42:59.164] GW | Ruang 1 : 29.8 C, 59 %
[07:42:59.168] GW | Ruang 2 : 25.4 C, 68 %
[07:42:59.168] GW | RSSI    : -46 dBm
[07:42:59.172] GW | SNR     : 11.00 dB
[07:42:59.176] GW | Statistik Node 1: diterima=2 | perkiraan hilang=0
[07:42:59.176] GW | =====
```

Parameter	Hasil (90 detik, interval bawaan 2000–5000 ms)
Total paket diterima gateway	51
Node 1: diterima gateway / perkiraan hilang	27 / 0
Node 2: diterima gateway / perkiraan hilang	24 / 0
Jumlah [GAP] muncul	0 (SEQ naik berurutan di kedua node sepanjang jendela rekam)
RSSI rata-rata — Node 1 (n=27)	-45,9 dBm (min -46, max -45)
RSSI rata-rata — Node 2 (n=24)	-57,9 dBm (min -59, max -57)
SNR rata-rata — Node 1 / Node 2	10,00 dB / 9,79 dB
SNR & RSSI rata-rata gabungan (semua paket)	-51,5 dBm / 9,90 dB

Tabel 9.9: Log serial Modul 08: EXP-01 — Dua Node Mengirim Bebas

Catatan jarak. Beda RSSI ± 12 dB antara Node 1 dan Node 2 murni karena posisi fisik kedua board berbeda di meja pengujian saat sesi ini direkam, bukan karena protokol.

Pada jarak dekat dan interval bawaan, **tidak ada satupun [GAP]** selama 90 detik / 51 paket. Ini konsisten dengan teori: pada beban rendah (G kecil), peluang dua paket saling tumpang tindih di udara sangat kecil meski Pure ALOHA tidak punya carrier-sense sama sekali. Memaksa tabrakan (EXP-02 di README) memerlukan mempersempit interval kirim kedua node dan diserahkan sebagai latihan praktikum — lihat catatan di README bagian Pengukuran.

EXP-03 — Payload Dua Ruangan

```
[07:42:59.158] N1 | [TX] NODE=1,SEQ=2,R1T=29.8,R1H=59,R2T=25.4,R2H=68 | total dikirim: 3
```

Parameter	Hasil
Jumlah field dalam satu payload	6 (NODE, SEQ, R1T, R1H, R2T, R2H)
Apakah Ruang 1 dan Ruang 2 selalu tiba bersamaan (satu paket)?	ya — tidak pernah ada paket berisi hanya salah satu ruangan pada 51 paket yang diamati
Ukuran payload (contoh di atas)	45 karakter

Tabel 9.10: Log serial Modul 08: EXP-03 — Payload Dua Ruangan

Ringkasan Verifikasi Hardware

Diuji di perangkat pada 2026-08-22: 3× Arduino Uno asli + Dragino LoRa Shield v1.2, satu gateway + dua node, environment gateway/node1/node2 sesuai platformio. ini bawaan (port /dev/ttyACM0/1/2, sudah cocok dengan hasil deteksi_port.py tanpa perlu diubah). Build dan upload ketiga environment sukses. Protokol Pure ALOHA berjalan sesuai desain: gateway hanya mendengar (tidak pernah TX), kedua node mengirim bebas dengan interval acak berbeda, dan pada beban rendah tidak terlihat kehilangan paket.

9.9 Pengukuran

A. Tingkat kedatangan paket terhadap kepadatan kirim

Interval kirim (ms)	Total dikirim (kedua node, 90 s)	Diterima gateway	Throughput (%)
2000–5000 (bawaan)	51 (27+24, tanpa hilang di jendela rekam)	51	~ 100 (beban rendah, jauh dari puncak $G = 0,5$)
1000–2000	<i>belum diuji – jalankan EXP-02</i>		
300–500	<i>belum diuji – jalankan EXP-02</i>		

Tabel 9.11: Lembar pengukuran A – kedatangan paket terhadap kepadatan kirim

B. RSSI dan SNR per node

Node	RSSI rata-rata (dBm)	SNR rata-rata (dB)	Jumlah [GAP]
Node 1	– 45,9	10,00	0
Node 2	– 57,9	9,79	0

Tabel 9.12: Lembar pengukuran B – RSSI dan SNR per node (sesi verifikasi)

9.10 Analisis

1. Dari Tabel 9.11, pada interval berapa throughput mulai menurun tajam? Bandingkan pola penurunannya dengan kurva teoretis $S = Ge^{-2G}$.
2. Jelaskan mengapa node yang mengirim tidak pernah tahu paketnya bertabrakan pada modul ini – telusuri baris kode yang membuktikannya.
3. Gateway memperkirakan kehilangan lewat lompatan SEQ. Sebutkan satu skenario di mana metode ini **melebih-lebihkan** jumlah paket hilang, dan satu skenario di mana ia **meremehkannya**.
4. Bandingkan jumlah [GAP] Node 1 dengan Node 2. Bila berbeda jauh padahal intervalnya sama, apa penjelasan yang mungkin?
5. Hitung *vulnerable period* untuk payload modul ini pada SF7 dan BW 125 kHz, lalu jelaskan hubungannya dengan interval kirim minimum yang masih aman dari tabrakan berlebihan.

9.11 Concept Check

1. Apa perbedaan mendasar Pure ALOHA dengan polling terjadwal pada M05?
2. Mengapa vulnerable period Pure ALOHA dua kali lipat waktu udara satu paket, bukan sama dengan waktu udara itu sendiri?
3. Mengapa gateway pada modul ini tidak pernah mengirim balasan apa pun?
4. Apa kelemahan memakai lompatan SEQ sebagai satu-satunya alat ukur kehilangan paket?
5. Sebutkan satu keadaan nyata di mana Pure ALOHA (kirim bebas, tanpa koordinasi) tetap menjadi pilihan yang masuk akal walau throughput-nya rendah.

9.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Membuat gateway mengukur sendiri

CH-1 — Hitung throughput otomatis Tambahkan penghitung total SEQ maksimum yang terlihat per node di gateway, lalu hitung dan cetak persentase throughput setiap 30 detik tanpa perlu dihitung manual dari log.

CH-2 — Tiga node Tambahkan environment node3 dan amati apakah [GAP] bertambah ketika jumlah node naik dari dua menjadi tiga pada interval kirim yang sama.

Level 3 — Tantangan · Menjembatani ke modul berikutnya

CH-3 — RSSI di payload Sisipkan estimasi RSSI terakhir yang diterima node dari paket node lain (bila node ikut mendengarkan) ke dalam payloadnya sendiri, sebagai langkah awal menuju *carrier sense*.

CH-4 — Simulasikan G Buat mode di gateway yang menghitung G (beban tawar, dalam paket per vulnerable period) dari data yang teramati, lalu bandingkan throughput terukur dengan prediksi $S = Ge^{-2G}$.

9.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — Pure ALOHA, vulnerable period, throughput teoretis.
3. Konfigurasi — format payload dua ruangan, interval kirim tiap node, parameter radio.
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 9.11 dan Tabel 9.12.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri, khususnya mengenai harga throughput yang dibayar demi kesederhanaan protokol.

10. Modul 08B — ALOHA + ACK: Kini Tahu

LoRa COMMUNICATION LAB · MODUL 08B

Finally Know — ALOHA + ACK

Arduino Uno + Dragino LoRa Shield v1.2 · topologi bintang, 2 node + gateway · interrupt + timeout · level Intermediate · 1 × 50 menit · folder kode Modul08b_lora_aloha_ack

PENGANTAR MODUL · Gambaran Umum

Modul 08B dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat menengah, dan **hanya** pengembangan dari M08 — bukan modul berdiri sendiri. Misinya menjawab satu pertanyaan yang M08 sengaja dibiarkan terbuka: bagaimana node tahu paketnya diterima? Jawabannya adalah mekanisme paling tua untuk itu, ACK, ditempelkan tepat di atas kode M08 tanpa mengubah cara data dibangkitkan maupun kapan node boleh mengirim.

10.1 Pendahuluan

M08 menunjukkan bahwa tabrakan pada Pure ALOHA bersifat senyap — kedua sisi sama-sama tidak sadar. Modul ini menutup separuh dari kebutaan itu: node pengirim kini menunggu balasan ACK=<id>, SEQ=<n> dan mencatat [OK] atau [FAIL] secara eksplisit. Yang **belum** dikerjakan modul ini secara sengaja adalah kirim ulang otomatis — begitu [FAIL] tercatat, node tetap lanjut ke data berikutnya. Retry, timeout adaptif, dan random backoff baru datang di M08C, sehingga peningkatan setiap pertemuan tetap dapat diukur satu per satu, bukan tercampur dalam satu lompatan besar.

Prasyaratnya adalah M08 untuk payload dua ruangan dan pembangkitan dummy, serta M04 untuk pola tunggu-ACK-dengan-timeout dan pencocokan nomor urut. Yang ditambahkan di sini adalah balasan ACK dari gateway yang membawa NODE dan SEQ sekaligus (M04 hanya membawa nomor urut, sebab modulnya cuma satu pasang board — di sini gateway melayani dua node, sehingga ACK wajib menyebut untuk siapa balasan itu ditujukan), serta statistik OK dan FAIL berjalan di sisi node. Kontrak ACK ini dipakai lagi apa adanya di M08C, hanya ditambah logika retry di atasnya.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
08	Penjadwalan dilepas — node kirim bebas, tabrakan senyap diamati
08B (ini)	ACK ditempelkan di atas M08 — node tahu SUCCESS/FAILED, belum ada retry
08C	Random backoff + retry — kegagalan dipulihkan, dan Pure ALOHA menjadi lengkap
09	Carrier sense — dengar dulu sebelum bicara, tabrakan dihindari sebelum terjadi
10	SYNC + slot waktu — Slotted ALOHA (slot diundi) vs TDMA (slot tetap)

Tabel 10.1: Peta modul pada arc kedua seri LoRa

Kontrak data lab ini. Payload data **identik** dengan M08: NODE=<id>, SEQ=<n>, R1T=<suhu>, R1H=<lembab>, R2T=<suhu>, R2H=<lembab>. Yang baru hanyalah balasan gateway, ACK=<id>, SEQ=<n> — dua field, karena gateway melayani lebih dari satu node dan ACK yang hanya membawa SEQ (seperti M04) berisiko dianggap milik node lain yang kebetulan memakai nomor urut sama. Node menolak ACK yang id atau SEQ-nya tidak cocok persis, mengikuti prinsip pencocokan permintaan-balasan dari M04.

10.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Membuat kegagalan pada kanal bersama menjadi terbaca

1. Menjelaskan mengapa ACK pada topologi banyak-node harus membawa identitas pengirim, bukan sekadar nomor urut seperti pada M04.
2. Menerapkan penungguan ACK berbatas waktu tanpa memblokir interrupt, memakai pola yang sama dengan M04.
3. Membedakan kegagalan yang **diketahui** (M08B, lewat [FAIL]) dari kegagalan yang **senyap** (M08, lewat lompatan SEQ), dan menjelaskan mengapa keduanya seharusnya menghasilkan angka yang mirip pada kondisi kanal yang sama.
4. Mengukur tingkat keberhasilan $(OK / (OK+FAIL) \times 100 \%)$ dua node yang berbagi satu kanal, dan membandingkannya dengan tingkat keberhasilan satu pasang board pada M04.
5. Menjelaskan mengapa modul ini **belum** mengirim ulang paket yang gagal, dan risiko apa yang muncul bila retry ditambahkan sembarangan tanpa mempertimbangkan duplikasi.

Kriteria keberhasilan

- ☐ Kedua node mencetak [OK] atau [FAIL] setelah setiap pengiriman, tidak pernah membeku menunggu ACK.
- ☐ Gateway membalas **setiap** paket data valid dengan ACK yang menyebut NODE dan SEQ yang benar.
- ☐ Statistik diterima (gateway) dan OK (node) pada node yang sama saling mendekati pada jarak dekat.
- ☐ [GAP] di gateway dan [FAIL] di node sama-sama meningkat ketika interval kirim dipersempit (EXP-02 M08 masih berlaku di sini).

10.3 Dasar Teori (Secukupnya)

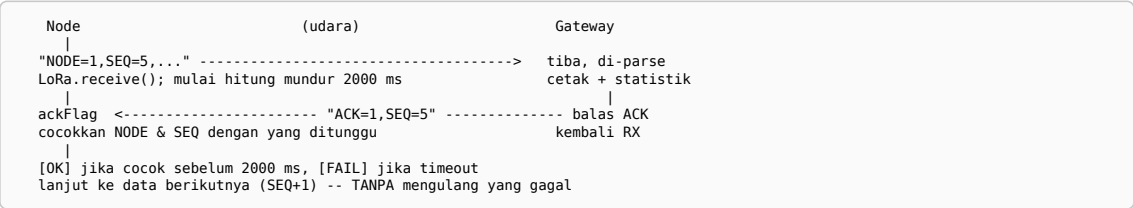
Istilah	Definisi kerja di lab ini
ACK beralamat	Balasan yang menyebut untuk siapa ia ditujukan (ACK=<id>, SEQ=<n>), diperlukan begitu gateway melayani lebih dari satu pengirim.
SUCCESS / FAILED	Hasil satu siklus kirim-tunggu-ACK di sisi node: [OK] bila ACK yang sesuai tiba sebelum timeout, [FAIL] bila tidak.
Kegagalan diketahui vs senyap	M08B membuat kegagalan diketahui node lewat timeout; kegagalan tetap senyap bagi gateway karena paket yang bertabrakan tidak pernah tiba untuk diketahui apa pun.
Retry (belum ada di sini)	Mengirim ulang paket yang gagal. Sengaja ditunda ke M08C agar efeknya (dan risiko duplikasi) dapat diukur terpisah.
Overhead ACK	Setiap paket data kini diikuti satu paket ACK dan satu jendela tunggu — menambah waktu udara total dibanding M08.

Tabel 10.2: Istilah kerja Modul 08B

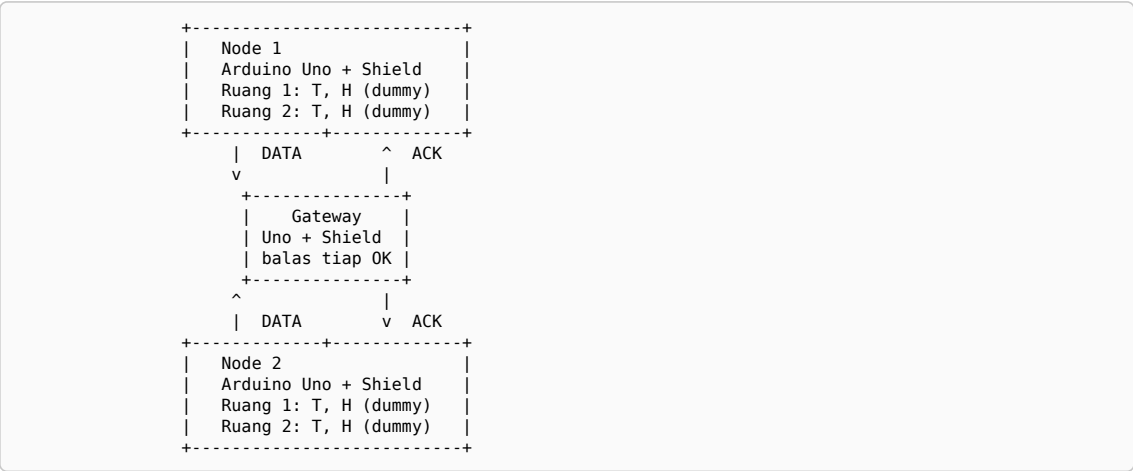
Mengapa ACK di sini harus menyebut NODE, sedangkan M04 cukup ACK:n. M04 hanya punya satu pasang board, sehingga nomor urut saja sudah cukup unik untuk mencocokkan balasan dengan permintaan. Begitu gateway melayani dua node sekaligus, dua kondisi rawan bisa muncul: (a) SEQ kedua node kebetulan bernilai sama di waktu yang berdekatan, dan (b) ACK milik Node 1 terdengar oleh Node 2 karena LoRa mentah tidak memiliki alamat radio. Menyisipkan NODE ke dalam ACK menutup keduanya — node menolak ACK yang NODE-nya bukan miliknya, persis seperti slave M05 menolak POLL yang bukan nomornya.

Mengapa belum ada retry. Menambahkan retry sekarang akan mencampur dua pertanyaan berbeda dalam satu percobaan: “apakah node tahu paketnya gagal?” (pertanyaan M08B) dan “apa yang terjadi kalau node mencoba lagi?” (pertanyaan M08C, lengkap dengan risiko gateway menerima data yang sama dua kali). Memisahkan keduanya membuat setiap pertemuan mengukur **satu** variabel baru.

Sekuens yang diamati



10.4 Topologi



Node	Environment	Peran	Mekanisme TX/RX	Timeout ACK
Node 1	node1	Kirim dummy Ruang 1+2, tunggu ACK	TX blocking + RX interrupt	2000 ms
Node 2	node2	Kirim dummy Ruang 1+2, tunggu ACK	TX blocking + RX interrupt	2000 ms
Gateway	gateway	Terima, cetak, balas ACK beralamat	RX interrupt + TX blocking	—

Tabel 10.3: Peran tiap node Modul 08B

10.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3

Tabel 10.4: Alat dan bahan Modul 08B

Struktur proyek

```

Modul08b_lora_aloha_ack/
├── platformio.ini
├── lora_monitor.py      ← dashboard 3-panel live (Gateway/Node1/Node2) + CSV
├── upload_auto.py      ← deteksi port otomatis saat unggah
├── logserial.md         ← cuplikan log serial aktual dari pengujian perangkat
└── src/
    ├── node/main.cpp    ← dummy Ruang 1+2, kirim + tunggu ACK (env node1, node2)
    └── gateway/main.cpp ← terima, cetak, balas ACK beralamat (env gateway)

```

10.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul08b_lora_aloha_ack/](#)

- [Modul08b_lora_aloha_ack/platformio.ini](#)
- [Modul08b_lora_aloha_ack/src/node/main.cpp](#)
- [Modul08b_lora_aloha_ack/src/gateway/main.cpp](#)
- [Modul08b_lora_aloha_ack/lora_monitor.py](#)
- [Modul08b_lora_aloha_ack/upload_auto.py](#)

```

[platformio]
default_envs = gateway

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 115200

[env:gateway]
build_src_filter = +<gateway/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua node memakai source yang sama; nomornya ditentukan build flag.
[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=1
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=2
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2

```

Kode 10.1: platformio.ini Modul 08B — environment gateway, node1, node2

```

/*
LoRa ALOHA + ACK - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: node1 / node2

Pengembangan langsung dari M08 (Pure ALOHA): node masih mengirim data
dummy dua ruangan kapan saja tanpa dengar-dahulu dan tanpa retry, TAPI kini
menunggu balasan ACK dari gateway sebelum melanjutkan. Bedanya dengan M08
hanya satu hal -- node akhirnya TAHU apakah paketnya sampai atau tidak.

```

```

Belum ada kirim ulang otomatis; itu baru datang di M08C.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Balasan : "ACK=<id>,SEQ=<n>" -- id & SEQ harus cocok persis dengan yang dikirim

Mekanisme:
TX : blocking (endPacket)
RX : interrupt DI00 + flag, hanya aktif selagi menunggu ACK
Dummy sensor: random() di sekitar nilai dasar per ruangan

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor node datang dari build flag (-DNODE_ID=1 atau 2) di platformio.ini,
// sehingga kedua node memakai file source yang sama persis.
#ifndef NODE_ID
#define NODE_ID 1
#endif

#define SEND_INTERVAL_MIN 2000
#define SEND_INTERVAL_MAX 5000
#define ACK_TIMEOUT       2000

#define ROOM1_TEMP_BASE  27.0
#define ROOM1_HUM_BASE   60
#define ROOM2_TEMP_BASE  25.0
#define ROOM2_HUM_BASE   68

unsigned long seq = 0;
unsigned long okCount = 0;
unsigned long failCount = 0;

volatile bool ackFlag = false;

void onAckReceived(int size) {
    if (size > 0) ackFlag = true;
}

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

float dummyTemp(float base) {
    return base + (random(-30, 31) / 10.0); // +-3.0 C
}

int dummyHum(int base) {
    return base + random(-15, 16); // +-15 %
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    randomSeed(analogRead(A0));

    Serial.print(F("=== LoRa ALOHA+ACK - NODE "));
    Serial.print(NODE_ID);
    Serial.println(F(" ==="));
    Serial.print(F("Init LoRa ... "));

```

```

LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
}

LoRa.setSpreadingFactor(SPREADING_FACTOR);
LoRa.setSignalBandwidth(BANDWIDTH);
LoRa.setCodingRate4(CODING_RATE);
LoRa.setTxPower(TX_POWER);
LoRa.onReceive(onAckReceived);

Serial.println(F("OK"));
Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
Serial.print(F("ACK timeout: ")); Serial.print(ACK_TIMEOUT); Serial.println(F(" ms"));
Serial.println(F("Peran: NODE (ALOHA + ACK) -- masih tanpa retry, lihat M08C\n"));
}

void loop() {
    float r1t = dummyTemp(ROOM1_TEMP_BASE);
    int r1h = dummyHum(ROOM1_HUM_BASE);
    float r2t = dummyTemp(ROOM2_TEMP_BASE);
    int r2h = dummyHum(ROOM2_HUM_BASE);

    String payload = "NODE=" + String(NODE_ID) +
        ",SEQ=" + String(seq) +
        ",R1T=" + String(r1t, 1) +
        ",R1H=" + String(r1h) +
        ",R2T=" + String(r2t, 1) +
        ",R2H=" + String(r2h);

    digitalWrite(LED_PIN, HIGH);
    transmit(payload);
    digitalWrite(LED_PIN, LOW);

    Serial.print(F("[TX] ")); Serial.println(payload);

    // --- Tunggu ACK ---
    String expectedAck = "ACK=" + String(NODE_ID) + ",SEQ=" + String(seq);
    ackFlag = false;
    LoRa.receive();

    unsigned long waitStart = millis();
    bool gotAck = false;

    while (millis() - waitStart < ACK_TIMEOUT) {
        if (ackFlag) {
            ackFlag = false;

            String reply = "";
            while (LoRa.available()) {
                reply += (char)LoRa.read();
            }

            if (reply == expectedAck) {
                gotAck = true;
                break;
            }
            Serial.print(F("[RX] WARN: balasan tak sesuai (")); Serial.print(reply);
            Serial.println(F("), tetap tunggu..."));
        }
    }

    if (gotAck) {
        okCount++;
        Serial.print(F("[OK] ACK diterima"));
    } else {
        failCount++;
        Serial.print(F("[FAIL] Tidak ada ACK"));
    }
    Serial.print(F(" | OK: ")); Serial.print(okCount);
    Serial.print(F(" | FAIL: ")); Serial.println(failCount);
    Serial.println();

    seq++;

    unsigned long interval = random(SEND_INTERVAL_MIN, SEND_INTERVAL_MAX + 1);
    delay(interval);
}

```

Kode 10.2: src/node/main.cpp — kirim dummy dua ruangan lalu tunggu ACK beralamat

```

/*
LoRa ALOHA + ACK - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: gateway

Pengembangan langsung dari M08: gateway kini membalas SETIAP paket data
yang berhasil di-parse dengan ACK berisi NODE dan SEQ yang sama persis,
lalu kembali mendengarkan. Paket yang bertabrakan tetap tidak pernah tiba
di sini sama sekali -- ACK tidak bisa dikirim untuk sesuatu yang tidak
pernah diterima -- sehingga node pengirim yang menyimpulkan kegagalan
lewat timeout-nya sendiri, bukan gateway yang memberitahu.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Balasan : "ACK=<id>,SEQ=<n>"

Mekanisme:
RX : interrupt DI00 + flag (non-blocking)
TX : blocking (endPacket), langsung setelah RX selesai diproses

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_DURATION      150
#define NODE_COUNT        2 // Node 1 dan Node 2 -> indeks array 1..2

volatile bool rxFlag = false;

unsigned long ledOnTime = 0;
bool ledActive = false;

unsigned long rxCount[NODE_COUNT + 1] = {0};
long lastSeq[NODE_COUNT + 1];
unsigned long lostEst[NODE_COUNT + 1] = {0};

void onReceive(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

String getField(const String& data, const String& key) {
    String needle = key + "=";
    int start = data.indexOf(needle);
    if (start == -1) return "";
    start += needle.length();
    int end = data.indexOf(' ', start);
    if (end == -1) end = data.length();
    return data.substring(start, end);
}

void setup() {
    Serial.begin(115200);
    while (!Serial);
}

```

```

pinMode(LED_PIN, OUTPUT);
digitalWrite(LED_PIN, LOW);

for (int i = 0; i <= NODE_COUNT; i++) lastSeq[i] = -1;

Serial.println(F("=== LoRa ALOHA+ACK - GATEWAY ==="));
Serial.print(F("Init LoRa ... "));

LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
}

LoRa.setSpreadingFactor(SPREADING_FACTOR);
LoRa.setSignalBandwidth(BANDWIDTH);
LoRa.setCodingRate4(CODING_RATE);
LoRa.setTxPower(TX_POWER);
LoRa.onReceive(onReceive);
LoRa.receive();

Serial.println(F("OK"));
Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
Serial.println(F("Peran: GATEWAY (ALOHA + ACK) -- balas tiap paket valid dengan ACK"));
Serial.println(F("Menunggu paket dari Node 1 & Node 2...\n"));
}

void loop() {
    updateLED();

    if (!rxFlag) return;
    rxFlag = false;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    if (!received.startsWith("NODE=")) {
        Serial.print(F("[WARN] Paket tak dikenal (mungkin sisa tabrakan): "));
        Serial.println(received);
        LoRa.receive();
        return;
    }

    int nodeId = getField(received, "NODE").toInt();
    long seq = getField(received, "SEQ").toInt();
    float r1t = getField(received, "R1T").toFloat();
    int r1h = getField(received, "R1H").toInt();
    float r2t = getField(received, "R2T").toFloat();
    int r2h = getField(received, "R2H").toInt();

    triggerLED();

    Serial.println(F("=== PAKET DITERIMA ==="));
    Serial.print(F(" Node : ")); Serial.println(nodeId);
    Serial.print(F(" SEQ : ")); Serial.println(seq);
    Serial.print(F(" Ruang 1 : ")); Serial.print(r1t, 1); Serial.print(F(" C, "));
    Serial.print(r1h); Serial.println(F(" %"));
    Serial.print(F(" Ruang 2 : ")); Serial.print(r2t, 1); Serial.print(F(" C, "));
    Serial.print(r2h); Serial.println(F(" %"));
    Serial.print(F(" RSSI : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
    Serial.print(F(" SNR : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));

    if (nodeId >= 1 && nodeId <= NODE_COUNT) {
        rxCount[nodeId]++;

        if (lastSeq[nodeId] >= 0) {
            long gap = seq - lastSeq[nodeId] - 1;
            if (gap > 0) {
                lostEst[nodeId] += gap;
                Serial.print(F(" [GAP] SEQ meloncat "));
                Serial.print(gap);
                Serial.println(F(" -- indikasi tabrakan/paket hilang"));
            }
        }
        lastSeq[nodeId] = seq;

        Serial.print(F(" Statistik Node ")); Serial.print(nodeId);
        Serial.print(F(": diterima=")); Serial.print(rxCount[nodeId]);
        Serial.print(F(" | perkiraan hilang=")); Serial.println(lostEst[nodeId]);
    }
}

```

```
// --- Balas ACK (TX blocking, lalu kembali RX) ---
String ack = "ACK=" + String(nodeId) + ",SEQ=" + String(seq);
LoRa.beginPacket();
LoRa.print(ack);
LoRa.endPacket();
Serial.print(F(" [TX] ")); Serial.println(ack);
Serial.println(F("=====\\n"));

LoRa.receive();
}
```

Kode 10.3: src/gateway/main.cpp — terima, cetak, dan balas ACK=<id>, SEQ=<n>

```
#!/usr/bin/env python3
"""
LoRa ALOHA + ACK Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Gateway <-> Node 1 & Node 2 (kirim bebas, gateway membalas ACK per paket)

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
    pip install pyserial rich

Cara pakai:
    python lora_monitor.py                                # port bawaan sesuai OS
    python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
    python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.table import Table
    from rich.layout import Layout
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.columns import Columns
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# -----
# KONFIGURASI
# -----
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --gateway/--n1/--n2.
if sys.platform.startswith("win"):
    DEFAULT_GW_PORT = "COM3"
    DEFAULT_N1_PORT = "COM4"
    DEFAULT_N2_PORT = "COM5"
else:
    DEFAULT_GW_PORT = "/dev/ttyACM0"
    DEFAULT_N1_PORT = "/dev/ttyACM1"
```

```

    DEFAULT_N2_PORT = "/dev/ttyACM2"
    DEFAULT_BAUD    = 115200
    LOG_LINES       = 16
    REFRESH_HZ      = 4
    GAP_WARN_STREAK = 3

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, menurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default gateway/n1/n2 selalu
    mengikuti apa yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

# Pure ALOHA (M08A) tidak punya ACK sama sekali -- node tidak pernah tahu
# apakah paketnya sampai, jadi baris OK/FAIL/Retry di panel node disembunyikan.
HAS_ACK = True

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class GatewayState:
    port: str = DEFAULT_GW_PORT
    connected: bool = False
    rx_total: int = 0
    rx_n1: int = 0
    rx_n2: int = 0
    gap_n1: int = 0
    gap_n2: int = 0
    lost_n1: int = 0
    lost_n2: int = 0
    dup_n1: int = 0
    dup_n2: int = 0
    last_rssi: int = 0
    last_snr: float = 0.0
    rssi_sum: int = 0
    snr_sum: float = 0.0
    rssi_count: int = 0
    cycle: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

@dataclass
class NodeState:
    name: str = ""
    port: str = ""
    connected: bool = False
    tx_count: int = 0
    ok_count: int = 0
    fail_count: int = 0
    retry_count: int = 0
    last_seq: int = -1
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.gw = GatewayState()
        self.n1 = NodeState("NODE 1", DEFAULT_N1_PORT)
        self.n2 = NodeState("NODE 2", DEFAULT_N2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSERS
# -----

```



```

# Semua regex dibuat agar aman dipakai lintas modul (M08A/M08B/M09/M10) --
# token yang tidak muncul pada suatu modul (mis. RETRY/SYNC) hanya tidak
# pernah cocok, tanpa memengaruhi parsing token lain.

RE_NODE_LINE = re.compile(r'Node\s+:\s*(\d+)')
RE_SEQ_LINE = re.compile(r'SEQ\s+:\s*(\d+)')
RE_RSSI_LINE = re.compile(r'RSSI\s+:\s*(-?\d+)')
RE_SNR_LINE = re.compile(r'SNR\s+:\s*(-?\d+\.\d+)')
RE_GAP_LINE = re.compile(r'\[GAP\] SEQ meloncat\s+(\d+)')
RE_STAT_LINE = re.compile(r'Statistik Node\s+(\d+):\s*diterima=(\d+)\s*|\s*perkiraan hilang=(\d+)')
RE_SYNC_TX = re.compile(r'\[TX\]\s+SYNC=(\d+)')
RE_CYCLE_DONE = re.compile(r'Cycle\s+(\d+)\s+selesai')
# M09: "Statistik Node X: baru=Y | duplicate=Z | gagal permanen (est.)=W"
RE_STAT_RETRY = re.compile(r'Statistik Node\s+(\d+):\s*baru=(\d+)\s*|\s*duplicate=(\d+)\s*|\s*gagal permanen\s*(est.\.)=(\d+)')
# M10: "--- Cycle N selesai | N1: diterima=Y hilang=Z N2: diterima=Y hilang=Z ---"
RE_STAT_CYCLE = re.compile(r'N(\d+):\s*diterima=(\d+)\s*hilang=(\d+)')

RE_NODE_TX = re.compile(r'\[TX\].*NODE=(\d+),SEQ=(\d+)')
RE_NODE_RETRY = re.compile(r'\[RETRY\s+(\d+)/(\d+)\s*\]')
RE_NODE_OK = re.compile(r'\[OK\]')
RE_NODE_FAIL = re.compile(r'\[FAIL\]')

def parse_gateway_line(line: str, st: GatewayState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    if 'PAKET DITERIMA' in line:
        st.rx_total += 1

    m = RE_RSSI_LINE.search(line)
    if m:
        st.last_rssi = int(m.group(1))
        st.rssi_sum += st.last_rssi
        st.rssi_count += 1

    m = RE_SNR_LINE.search(line)
    if m:
        st.last_snr = float(m.group(1))
        st.snr_sum += st.last_snr

    m = RE_GAP_LINE.search(line)
    if m:
        gap = int(m.group(1))
        anomalies.append(Anomaly(ts, "GATEWAY", "GAP", f"SEQ meloncat {gap} -- indikasi tabrakan/paket hilang"))

    m = RE_STAT_LINE.search(line)
    if m:
        node_id, rx, lost = int(m.group(1)), int(m.group(2)), int(m.group(3))
        if node_id == 1:
            st.rx_n1, st.lost_n1 = rx, lost
        elif node_id == 2:
            st.rx_n2, st.lost_n2 = rx, lost

    m = RE_STAT_RETRY.search(line)
    if m:
        node_id, rx, dup, lost = int(m.group(1)), int(m.group(2)), int(m.group(3)), int(m.group(4))
        if node_id == 1:
            st.rx_n1, st.dup_n1, st.lost_n1 = rx, dup, lost
        elif node_id == 2:
            st.rx_n2, st.dup_n2, st.lost_n2 = rx, dup, lost

    for node_id_s, rx_s, lost_s in RE_STAT_CYCLE.findall(line):
        node_id, rx, lost = int(node_id_s), int(rx_s), int(lost_s)
        if node_id == 1:
            st.rx_n1, st.lost_n1 = rx, lost
        elif node_id == 2:
            st.rx_n2, st.lost_n2 = rx, lost

    m = RE_SYNC_TX.search(line)
    if m:
        st.cycle = int(m.group(1))

    m = RE_CYCLE_DONE.search(line)
    if m:
        st.cycle = int(m.group(1))

def parse_node_line(line: str, st: NodeState, node_label: str, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = RE_NODE_TX.search(line)
    if m:

```

```

    seq = int(m.group(2))
    if st.last_seq >= 0 and seq == st.last_seq and '[RETRY]' not in line:
        pass # aman, tidak dipakai
    st.last_seq = seq
    if '[RETRY]' not in line:
        st.tx_count += 1

m = RE_NODE_RETRY.search(line)
if m:
    st.retry_count += 1

if RE_NODE_OK.search(line):
    st.ok_count += 1
elif RE_NODE_FAIL.search(line):
    st.fail_count += 1
    if st.fail_count and st.fail_count % 5 == 0:
        anomalies.append(Anomaly(ts, node_label, "FAIL_COUNT",
                                  f"{node_label} sudah {st.fail_count}x gagal (tanpa ACK)"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
                  shared: SharedState, role: str):
    if role == "GW":
        state = shared.gw
    elif node_key == "N1":
        state = shared.n1
    else:
        state = shared.n2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                if role == "GW":
                                    parse_gateway_line(line, state, shared.anomalies)
                                else:
                                    parse_node_line(line, state, node_key, shared.anomalies)
                            shared.csv_queue.put({
                                'ts': ts, 'node': node_key, 'raw': line
                            })
                        except (serial.SerialException, OSError):
                            break
                    except (serial.SerialException, OSError):
                        with shared.lock:
                            state.connected = False
                        time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                            'node': row['node'],
                            'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:

```

```

        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"
    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str) -> str:
    if '[FAIL]' in line: return "red"
    if '[GAP]' in line: return "bold red"
    if '[RETRY]' in line: return "bold yellow"
    if '[BACKOFF]' in line: return "yellow"
    if '[OK]' in line: return "bold green"
    if '[TX]' in line: return "cyan"
    if 'PAKET DITERIMA' in line: return "green"
    if '[WAIT]' in line: return "dim"
    if '[WARN]' in line: return "yellow"
    return "dim white"

def build_gateway_panel(st: GatewayState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" Total diterima : ", style="dim"); t.append(f"{st.rx_total}\n", style="bold white")
    if st.cycle:
        t.append(" Cycle terakhir : ", style="dim"); t.append(f"{st.cycle}\n", style="bold cyan")
    t.append("\n")

    dup_suffix1 = f" dup={st.dup_n1}" if st.dup_n1 else ""
    dup_suffix2 = f" dup={st.dup_n2}" if st.dup_n2 else ""
    t.append(" Node 1 ", style="bold cyan")
    t.append(f"diterima={st.rx_n1:<4} hilang={st.lost_n1}{dup_suffix1}\n", style="white")
    t.append(" Node 2 ", style="bold magenta")
    t.append(f"diterima={st.rx_n2:<4} hilang={st.lost_n2}{dup_suffix2}\n\n", style="white")

    rssi_avg = (st.rssi_sum / st.rssi_count) if st.rssi_count else 0.0
    snr_avg = (st.snr_sum / st.rssi_count) if st.rssi_count else 0.0
    t.append(" RSSI terakhir : ", style="dim"); t.append(f"{st.last_rssi} dBm\n", style="white")
    t.append(" SNR terakhir : ", style="dim"); t.append(f"{st.last_snr:.2f} dB\n", style="white")
    t.append(" RSSI rata-rata : ", style="dim"); t.append(f"{rssi_avg:.1f} dBm\n", style="white")
    t.append(" SNR rata-rata : ", style="dim"); t.append(f"{snr_avg:.2f} dB\n\n", style="white")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:40]}\n", style=style)

    return Panel(t, title="[bold cyan]GATEWAY[/]",
                border_style="cyan", box=box.ROUNDED)

def build_node_panel(st: NodeState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" TX terkirim : ", style="dim"); t.append(f"{st.tx_count:>6}\n", style="bold white")
    t.append(" SEQ terakhir : ", style="dim"); t.append(f"{st.last_seq:>6}\n\n", style="white")

    if HAS_ACK:
        tot = st.ok_count + st.fail_count
        pct = (st.ok_count / tot * 100) if tot else 0.0
        t.append(" OK : ", style="dim"); t.append(f"{st.ok_count:>6}\n", style="bold green")
        t.append(" FAIL : ", style="dim"); t.append(f"{st.fail_count:>6}\n",
            style="bold red" if st.fail_count else "dim")
        if tot:
            t.append(" Rate : ", style="dim"); t.append(f"{pct:>6.2f}%\n", style=pct_style(st.ok_count,
tot))
        t.append(" Retry : ", style="dim"); t.append(f"{st.retry_count:>6}\n\n",
            style="yellow" if st.retry_count else "dim")
    else:
        t.append(" (Pure ALOHA -- tanpa ACK, node tidak pernah tahu\n", style="dim")
        t.append(" apakah paketnya sampai. Lihat panel GATEWAY untuk\n", style="dim")
        t.append(" statistik diterima/hilang per node.)\n\n", style="dim")

    t.append(" Raw Log\n", style="bold dim")

```

```

t.append(" " + "-" * 30 + "\n", style="dim")
for ts, line in list(st.raw_log)[-LOG_LINES:]:
    style = log_line_style(line)
    t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

return Panel(t, title=f"[bold {color}]{label}[/]",
             border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                       csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'GAP': "bold red",
        'FAIL_COUNT': "bold yellow",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f" [{a.node}] ", style="bold white")
            t.append(f" {a.kind}: ", style=style)
            t.append(f" {a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        gw = GatewayState(
            port=shared.gw.port, connected=shared.gw.connected,
            rx_total=shared.gw.rx_total, rx_n1=shared.gw.rx_n1, rx_n2=shared.gw.rx_n2,
            gap_n1=shared.gw.gap_n1, gap_n2=shared.gw.gap_n2,
            lost_n1=shared.gw.lost_n1, lost_n2=shared.gw.lost_n2,
            dup_n1=shared.gw.dup_n1, dup_n2=shared.gw.dup_n2,
            last_rssi=shared.gw.last_rssi, last_snr=shared.gw.last_snr,
            rssi_sum=shared.gw.rssi_sum, snr_sum=shared.gw.snr_sum,
            rssi_count=shared.gw.rssi_count, cycle=shared.gw.cycle,
            raw_log=deque(shared.gw.raw_log),
        )
        n1 = NodeState(
            name=shared.n1.name, port=shared.n1.port, connected=shared.n1.connected,
            tx_count=shared.n1.tx_count, ok_count=shared.n1.ok_count,
            fail_count=shared.n1.fail_count, retry_count=shared.n1.retry_count,
            last_seq=shared.n1.last_seq,
            raw_log=deque(shared.n1.raw_log),
        )
        n2 = NodeState(
            name=shared.n2.name, port=shared.n2.port, connected=shared.n2.connected,
            tx_count=shared.n2.tx_count, ok_count=shared.n2.ok_count,
            fail_count=shared.n2.fail_count, retry_count=shared.n2.retry_count,
            last_seq=shared.n2.last_seq,
            raw_log=deque(shared.n2.raw_log),
        )
        anom = deque(shared.anomalies)
        t_start = shared.session_start

    node_row = Columns([
        build_gateway_panel(gw),
        build_node_panel(n1, "NODE 1", "green"),
        build_node_panel(n2, "NODE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT

```

```
# -----
BANNER = """\
    LoRa ALOHA+ACK Monitor – Evaluasi Praktikum
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz
    \
"""

Tool ini membaca data serial dari 3 board LoRa (Gateway, Node 1, Node 2)
secara bersamaan dan menampilkan dashboard real-time: statistik
diterima/OK/FAIL/RETRY, RSSI/SNR, deteksi anomali (GAP/FAIL beruntun),
serta logging CSV otomatis untuk analisis data penelitian.

PENGGUNAAN:
python lora_monitor.py [opsi...]

OPSI:
--gateway PORT Port COM untuk Gateway (default: {gw})
--n1 PORT Port COM untuk Node 1 (default: {n1})
--n2 PORT Port COM untuk Node 2 (default: {n2})
--baud N Baud rate serial semua port (default: {baud})
--out FILE Nama file CSV output (default: lora_session_YYYYMMDD_HHMMSS.csv)
-h / --help Tampilkan pesan bantuan ini

CONTOH:
python lora_monitor.py
-> Jalankan dengan port & baud rate default

python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
-> Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

python lora_monitor.py --out data_jarak_10m.csv
-> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
Ctrl+C Hentikan monitoring & simpan CSV

CATATAN:
* Pastikan semua board sudah di-upload firmware dengan baud {baud}
* Upload Gateway dulu, baru Node 1 dan Node 2, sebelum menjalankan tool ini
* File CSV tersimpan otomatis saat Ctrl+C ditekan
"""
.format(
    gw=DEFAULT_GW_PORT,
    n1=DEFAULT_N1_PORT,
    n2=DEFAULT_N2_PORT,
    baud=DEFAULT_BAUD,
)

def parse_args(port_aktif):
    gw_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_GW_PORT
    n1_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_N1_PORT
    n2_default = port_aktif[2] if len(port_aktif) > 2 else DEFAULT_N2_PORT

    p = argparse.ArgumentParser(
        description="LoRa ALOHA + ACK Monitor Dashboard -- tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f" python lora_monitor.py\n"
            f" python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2\n"
            f" python lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--gateway', default=gw_default,
        metavar='PORT', help=f'Port COM Gateway (default: {gw_default}, dari deteksi port aktif)')
    p.add_argument('--n1', default=n1_default,
        metavar='PORT', help=f'Port COM Node 1 (default: {n1_default}, dari deteksi port aktif)')
    p.add_argument('--n2', default=n2_default,
        metavar='PORT', help=f'Port COM Node 2 (default: {n2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
        metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None,
        metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
    return p, p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    p, args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1

    console = Console()
```

```

if no_args:
    console.print(BANNER)

console.print(
    f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
)
console.print(
    f"[dim]Dipakai -> "
    f"Gateway=[yellow]{args.gateway}[/] "
    f"N1=[green]{args.n1}[/] "
    f"N2=[magenta]{args.n2}[/] "
    f"Baud=[white]{args.baud}[/][dim]\n"
)

shared = SharedState()
shared.gw.port = args.gateway
shared.n1.port = args.n1
shared.n2.port = args.n2

ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
csv_path = args.out or f"lora_session_{ts_str}.csv"

if not no_args:
    console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
    console.print(f" Gateway : [yellow]{args.gateway}[/]")
    console.print(f" Node 1 : [green]{args.n1}[/]")
    console.print(f" Node 2 : [magenta]{args.n2}[/]")
    console.print(f" Baud : [white]{args.baud}[/]")

console.print(f" CSV : [dim]{csv_path}[/]")
console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
try:
    input()
except KeyboardInterrupt:
    return

threads = [
    threading.Thread(target=reader_thread,
        args=(args.gateway, args.baud, "GW", shared, "GW"),
        daemon=True, name="reader-gateway"),
    threading.Thread(target=reader_thread,
        args=(args.n1, args.baud, "N1", shared, "NODE"),
        daemon=True, name="reader-n1"),
    threading.Thread(target=reader_thread,
        args=(args.n2, args.baud, "N2", shared, "NODE"),
        daemon=True, name="reader-n2"),
    threading.Thread(target=csv_writer_thread,
        args=(shared, csv_path),
        daemon=True, name="csv-writer"),
]
for t in threads:
    t.start()

try:
    with Live(build_layout(shared, csv_path),
        console=console,
        refresh_per_second=REFRESH_HZ,
        screen=True) as live:
        while True:
            time.sleep(1.0 / REFRESH_HZ)
            live.update(build_layout(shared, csv_path))
except KeyboardInterrupt:
    pass
finally:
    shared.running = False
    time.sleep(0.6)
    console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 10.4: lora_monitor.py — dasbor tiga panel dengan perekaman CSV

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 08B (ALOHA + ACK) — port serial dideteksi sendiri.

```

Alternatif untuk menjalankan manual:

```
pio run -d Modul08b_lora_aloha_ack -e gateway -t upload -t monitor
pio run -d Modul08b_lora_aloha_ack -e node1 -t upload -t monitor
pio run -d Modul08b_lora_aloha_ack -e node2 -t upload -t monitor
```

Skrip ini memindai port `/dev/ttyACM*/ttyUSB*` yang sedang aktif, memetakannya ke environment ``gateway`` (port pertama), ``node1`` (port kedua), dan ``node2`` (port ketiga), lalu menjalankan ``pio run`` dengan `--upload-port` sesuai hasil deteksi -- `platformio.ini` tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga board selesai di-upload, skrip bertanya apakah mau langsung menjalankan `lora_monitor.py` -- bila ya, dijalankan dengan `--gateway ... --n1 ... --n2 ...` persis memakai port yang barusan dipakai upload (bukan deteksi ulang), supaya tidak mungkin salah pasang.

```
python3 Modul08b_lora_aloha_ack/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul08b_lora_aloha_ack/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti lora_monitor.py

python3 Modul08b_lora_aloha_ack/upload_auto.py --only gateway
-> upload satu environment saja

python3 Modul08b_lora_aloha_ack/upload_auto.py --gateway /dev/ttyACM0 --node1 /dev/ttyACM1 --node2 /dev/
ttyACM2
-> timpa hasil deteksi otomatis secara manual

"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["gateway", "node1", "node2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"gateway": "--gateway", "node1": "--n1", "node2": "--n2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, menurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port gateway/node1/node2 PERSIS seperti yang baru dipakai upload
    -- bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
```

```

for env, opsi in ENV_KE_OPSI.items():
    if env in port_env:
        cmd += [opsi, port_env[env]]

print(" $", " ".join(cmd))
try:
    subprocess.call(cmd)
except KeyboardInterrupt:
    # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
    # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
    # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
    # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
    print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--gateway", metavar="/dev/ttyXXX",
                    help="timpa port gateway (lewati deteksi otomatis)")
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="timpa port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="timpa port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (gateway, node1, atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                         "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau ketiganya"
                         "sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"gateway": args.gateway, "node1": args.node1, "node2": args.node2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 10.5: upload_auto.py — pemilih port otomatis saat unggah

10.7 Build dan Flash

Gateway lebih dahulu, supaya paket pertama dari node langsung dibalas.


```
pio run -d Modul08b_lora_aloha_ack -e gateway -t upload -t monitor
pio run -d Modul08b_lora_aloha_ack -e node1 -t upload -t monitor
pio run -d Modul08b_lora_aloha_ack -e node2 -t upload -t monitor
```

Monitor dashboard. `python3 lora_monitor.py` membaca ketiga port sekaligus dan menampilkan panel Gateway, Node 1, dan Node 2 (OK/FAIL/retry, RSSI dan SNR, deteksi [GAP]) di terminal, plus perekaman CSV otomatis. Memerlukan `pip install pyserial rich`.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ Port ketiga board dicatat lewat `pio device list` (atau `python3 ../tools/deteksi_port.py`) dan diisikan ke `platformio.ini`.
- ☐ Tiga Serial Monitor 115200 baud siap, ketiganya terlihat bersamaan.
- ☐ Penghitung OK dan FAIL pada kedua node diamati sejak baris pertama.

10.8 Percobaan

10.8.1 EXP-01 — Siklus ACK Sehat, Dua Node

Nyalakan ketiga board dan amati sepuluh siklus pertama pada kedua node.

Expected output — node

```
=== LoRa ALOHA+ACK - NODE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
ACK timeout: 2000 ms
Peran: NODE (ALOHA + ACK) -- masih tanpa retry, lihat M08C

[TX] NODE=1,SEQ=1,R1T=25.9,R1H=74,R2T=26.9,R2H=59
[OK] ACK diterima | OK: 1 | FAIL: 1
```

Expected output — gateway

```
=== PAKET DITERIMA ===
Node      : 2
SEQ       : 1
Ruang 1   : 26.3 C, 45 %
Ruang 2   : 23.8 C, 80 %
RSSI      : -61 dBm
SNR       : 9.25 dB
Statistik Node 2: diterima=1 | perkiraan hilang=0
[TX] ACK=2,SEQ=1
=====
```

Data capture — diukur 90 detik (bukan 10 siklus; lihat `logserial.md`).

Parameter	Hasil
OK/FAIL Node 1 (90 detik, 24 percobaan)	22 / 2
OK/FAIL Node 2 (90 detik, 21 percobaan)	19 / 2
Tingkat keberhasilan (%) kedua node	91,7 % / 90,5 %
RSSI/SNR arah DATA (di gateway) — Node 1 / Node 2	– 46,3 dBm / 9,84 dB — – 59,5 dBm / 9,60 dB
Latensi ACK round-trip (TX → OK) — Node 1 / Node 2	rata-rata 60 ms / 59 ms

Tabel 10.5: Hasil pengamatan EXP-01 pada sesi verifikasi perangkat

CHECKPOINT

Tidak sepenuhnya terpenuhi — dan itu justru instruktif. FAIL tidak nol: sesi ini menangkap satu tabrakan nyata antara Node 1 dan Node 2 (SEQ=5, selisih TX ~ 90 ms, dalam *vulnerable period*), lihat `logserial.md`. Sesi M08 (90 detik, interval sama) justru **nihil** [GAP] — kebetulan statistik semata (tabrakan adalah peristiwa acak; 90 detik terlalu singkat untuk menyimpulkan tingkat kegagalan “sebenarnya” dari satu sesi saja), bukan bukti bahwa M08B lebih rentan tabrakan daripada M08 (keduanya memakai mekanisme kirim yang identik, hanya M08B menambahkan ACK di atasnya).

10.8.2 EXP-02 — Dibandingkan dengan M08 (Kegagalan Senyap vs Diketahui)

Jalankan M08 dan M08B berturut-turut pada interval kirim yang dipersempit (300–500 ms, seperti EXP-02 M08), lalu bandingkan.

Data capture

Parameter	M08 (Pure ALOHA)	M08B (ALOHA + ACK)
Cara kegagalan terlihat	[GAP] di gateway saja	[FAIL] di node + [GAP] di gateway
Jumlah kegagalan per menit — Node 1		
Jumlah kegagalan per menit — Node 2		
Lalu lintas radio tambahan (paket ACK)	—	

Tabel 10.6: Lembar pengamatan EXP-02 — M08 dibanding M08B

BUKA ABSTRAKSINYA

Di `src/gateway/main.cpp`, ACK dikirim **setelah** statistik [GAP] dicetak, bukan sebelumnya. Jelaskan mengapa urutan ini tidak memengaruhi kebenaran ACK (radio tetap half-duplex, hanya satu arah aktif pada satu waktu), lalu telusuri: apa yang terjadi bila `LoRa.receive()` di akhir `loop()` gateway dihapus?

CHECKPOINT

Jumlah [FAIL] di node dan jumlah [GAP] di gateway untuk node yang sama seharusnya **mendekati**, bukan identik persis — sebab [FAIL] juga mencakup kasus DATA sampai tetapi ACK-nya yang hilang (lihat Analisis, soal 3).

10.8.3 EXP-03 — Node Tanpa Retry Tetap Berjalan

Matikan gateway sesaat, amati kedua node, lalu nyalakan kembali.

Data capture

Parameter	Hasil
Selang [TX] → [FAIL] saat gateway mati (detik)	
Apakah node melanjutkan ke SEQ berikutnya walau gagal?	
Berapa siklus sampai [OK] kembali muncul setelah gateway hidup?	

Tabel 10.7: Lembar pengamatan EXP-03

CHECKPOINT

Node harus **tetap melanjutkan** ke SEQ berikutnya setelah [FAIL], bukan mengulang SEQ yang sama. Perilaku “coba lagi” baru boleh muncul di M08C — di modul ini, [FAIL] berarti data hilang permanen.

10.8.4 Verifikasi Perangkat Keras

CATATAN

Diuji di perangkat pada 22 Agustus 2026 — tiga Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway, node1, node2; port /dev/ttyACM0/1/2). Build dan upload ketiga environment sukses tanpa modifikasi kode. EXP-01 dijalankan (90 detik) dan datanya nyata, termasuk satu tabrakan sungguhan yang terekam langsung — lihat logserial.md. EXP-02 (perbandingan sistematis M08 vs M08B pada interval dipersempit) dan EXP-03 (mematikan gateway sesaat) **belum dijalankan** pada sesi verifikasi ini, diserahkan sebagai latihan praktikum.

10.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, ACK_TIMEOUT 2000 ms. Ketiga board di posisi sama seperti M08 (Node 1 lebih dekat ke gateway daripada Node 2). Interval kirim bawaan (2000–5000 ms acak), tidak diubah.

Board & Port

Peran	Environment	Port	Board
Gateway	gateway	/dev/ttyACM0	Uno asli (2341:0043)
Node 1	node1	/dev/ttyACM1	Uno asli (2341:0043)
Node 2	node2	/dev/ttyACM2	Uno asli (2341:0043)

Tabel 10.8: Log serial Modul 08B: Board & Port

Ketiga aliran serial direkam bersamaan, jendela rekam 90 detik setelah ketiga board di-*upload* ulang dan *boot* bersih.

EXP-01 — Siklus ACK Sehat, Dua Node

Siklus normal (paket sampai, ACK sampai, latensi round-trip ~60 ms):

```
[07:55:29.194] GW | === PAKET DITERIMA ===
[07:55:29.198] GW |   Node   : 2
[07:55:29.198] GW |   SEQ    : 1
[07:55:29.198] GW | Ruang 1 : 26.3 C, 45 %
[07:55:29.202] GW | Ruang 2 : 23.8 C, 80 %
[07:55:29.202] GW | RSSI     : -61 dBm
[07:55:29.206] GW | SNR      : 9.25 dB
[07:55:29.210] GW | Statistik Node 2: diterima=1 | perkiraan hilang=0
[07:55:29.247] GW | [TX] ACK=2,SEQ=1
[07:55:29.251] GW | =====
[07:55:29.819] N1 | [TX] NODE=1,SEQ=1,R1T=25.9,R1H=74,R2T=26.9,R2H=59
[07:55:29.881] N1 | [OK] ACK diterima | OK: 1 | FAIL: 1
```

Parameter	Hasil (90 detik, bukan 10 siklus — lihat catatan)
OK/FAIL Node 1	22 / 2 (24 percobaan)
OK/FAIL Node 2	19 / 2 (21 percobaan tercatat lengkap)
Tingkat keberhasilan Node 1 / Node 2	91,7% / 90,5%

Parameter	Hasil (90 detik, bukan 10 siklus — lihat catatan)
Latensi ACK round-trip (TX → OK) Node 1 / Node 2	rata-rata 60 ms / 59 ms (min 57, maks 65 ms)
RSSI/SNR arah DATA (di gateway) — Node 1 / Node 2	-46,3 dBm / 9,84 dB — -59,5 dBm / 9,60 dB

Tabel 10.9: Log serial Modul 08B: EXP-01 — Siklus ACK Sehat, Dua Node

CHECKPOINT sebagian terpenuhi. FAIL tidak nol: masing-masing node mengalami 2 kegagalan dalam 90 detik — satu di antaranya adalah tabrakan nyata yang berhasil direkam langsung (lihat EXP-02 di bawah). Ini konsisten dengan M08: pada jarak dan interval yang sama, tabrakan tetap mungkin terjadi walau jarang.

Temuan Tambahan A — Tabrakan Nyata Tertangkap Langsung

(Bukan EXP-02 versi README — itu perbandingan sistematis M08 vs M08B pada interval dipersempit, belum dijalankan. Ini adalah tabrakan yang kebetulan terekam pada sesi EXP-01 di atas.)

Pada sesi ini, Node 1 dan Node 2 kebetulan mengirim SEQ=5 hampir bersamaan (selisih ~90 ms — dalam *vulnerable period*). Log berikut menunjukkan efeknya di kedua sisi:

```
[07:55:44.196] N2 | [TX] NODE=2,SEQ=5,R1T=24.1,R1H=51,R2T=24.9,R2H=55
[07:55:44.286] N1 | [TX] NODE=1,SEQ=5,R1T=27.9,R1H=74,R2T=22.3,R2H=55
[07:55:44.295] N2 | [RX] WARN: balasan tak sesuai (NODE=1,SEQ=5,R1T=27.9,R1H=74,R2T=22.3,R2H=55), tetap
tunggu...
[07:55:46.195] N2 | [FAIL] Tidak ada ACK | OK: 4 | FAIL: 2
[07:55:46.288] N1 | [FAIL] Tidak ada ACK | OK: 4 | FAIL: 2
[07:55:49.055] GW | Ruang 2 : 27.6 C, 64 %
[07:55:49.055] GW | RSSI : -46 dBm
[07:55:49.059] GW | SNR : 10.00 dB
[07:55:49.062] GW | [GAP] SEQ meloncat 1 -- indikasi tabrakan/paket hilang
[07:55:49.066] GW | Statistik Node 1: diterima=5 | perkiraan hilang=1
```

Yang menarik: Node 2 sempat menerima **paket DATA milik Node 1** di jendela tunggu ACK-nya sendiri ([RX] WARN: balasan tak sesuai) — bukti langsung bahwa radio LoRa mentah tidak beralamat, semua node saling mendengar lalu lintas siapa pun. Kedua node akhirnya [FAIL] pada SEQ=5 yang sama, dan gateway mencatat [GAP] pada Node 1 (SEQ meloncat dari 4 ke 6) — konsisten dengan paket Node 1 yang hilang akibat tabrakan ini.

Parameter	Hasil
Total [GAP] gateway (90 detik)	1 (Node 1, SEQ 4→6)
Total [FAIL] Node 1 / Node 2 (90 detik)	2 / 2
Selisih waktu TX kedua node saat tabrakan	~90 ms

Tabel 10.10: Log serial Modul 08B: Temuan Tambahan A — Tabrakan Nyata Tertangkap Langsung

Temuan Tambahan B — Ketidaksepakatan Node vs Gateway (DATA sampai, ACK hilang)

(Bukan EXP-03 versi README — itu uji mematikan gateway sesaat, belum dijalankan. Ini adalah data untuk Tabel B bagian Pengukuran README, dihitung dari sesi EXP-01 di atas.)

Node	OK di node	diterima di gateway (akhir jendela)	Selisih	Tafsiran
Node 1	22	22	0	Kedua FAIL Node 1 (boot-race SEQ=0 + tabrakan SEQ=5) memang DATA yang tidak pernah sampai — konsisten dengan [GAP] gateway.

Node	OK di node	diterima di gateway (akhir jendela)	Selisih	Tafsiran
Node 2	19	20	1	Satu FAIL di Node 2 ternyata DATA-nya sampai di gateway (diterima gateway lebih tinggi dari OK node) – kemungkinan ACK balasannya yang hilang di jalur pulang, bukan DATA-nya di jalur pergi.

Tabel 10.11: Log serial Modul 08B: Temuan Tambahan B — Ketidaksepakatan Node vs Gateway (DATA sampai, ACK hilang)

Baris Node 2 adalah bukti nyata dari poin analisis modul ini: [FAIL] di node **tidak selalu berarti** DATA hilang — bisa juga DATA sampai tapi ACK balasannya yang tidak pernah tiba kembali. Node tidak bisa membedakan kedua kasus ini hanya dari timeout-nya sendiri.

Ringkasan Verifikasi Hardware

Diuji di perangkat pada 2026-08-22: 3× Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway + node1 + node2, port /dev/ttyACM0/1/2). Build dan upload ketiga environment sukses. Protokol ALOHA+ACK berjalan sesuai desain: gateway membalas ACK untuk setiap paket valid yang diterima, kedua node menunggu ACK dengan timeout 2000 ms tanpa retry. Sesi 90 detik ini bahkan menangkap satu tabrakan nyata antara Node 1 dan Node 2, memberi bukti langsung untuk konsep *vulnerable period* dan perbedaan antara “DATA hilang” vs “ACK hilang”. EXP-02 versi README (uji sistematis M08 vs M08B pada interval dipersempit) dan EXP-03 versi README (mematikan gateway sesaat) **belum dijalankan** pada sesi ini — diserahkan sebagai latihan praktikum.

10.9 Pengukuran

A. Tingkat keberhasilan per node (90 detik, interval bawaan 2000–5000 ms)

Node	OK	FAIL	Keberhasilan (%)	RSSI rata-rata (dBm)
Node 1	22	2	91,7	– 46,3
Node 2	19	2	90,5	– 59,5

Tabel 10.12: Lembar pengukuran A — tingkat keberhasilan per node (sesi verifikasi)

B. Ketidaksepakatan node dan gateway (lihat Analisis soal 3)

Node	OK di node	diterima di gateway	Selisih	Tafsiran
Node 1	22	22	0	Kedua FAIL Node 1 memang DATA yang tidak pernah sampai (konsisten dengan [GAP] gateway).
Node 2	19	20	1	Satu FAIL Node 2 ternyata DATA-nya sampai di gateway — ACK balasannya yang hilang di jalur pulang, bukan DATA di jalur pergi.

Tabel 10.13: Lembar pengukuran B — ketidaksepakatan node dan gateway

C. M08 dibanding M08B pada interval kirim sama

Interval kirim (ms)	[GAP] M08 (90 s)	[FAIL] M08B (90 s, N1+N2)
2000–5000 (bawaan)	0	4 (2+2)
300–500	<i>belum diuji — jalankan EXP-02</i>	

Tabel 10.14: Lembar pengukuran C — M08 dibanding M08B

10.10 Analisis

1. Bandingkan Tabel 10.12 dengan tingkat keberhasilan M04 (satu pasang board). Jelaskan mengapa dua node yang berbagi kanal cenderung menghasilkan tingkat keberhasilan yang lebih rendah pada beban kirim yang sama.
2. Dari Tabel 10.14, apakah [FAIL] M08B selalu lebih besar atau sama dengan [GAP] M08 pada interval yang sama? Jelaskan mengapa secara teori seharusnya begitu.
3. Tabel 10.13 dapat menunjukkan diterima di gateway lebih besar daripada OK di node untuk node yang sama. Jelaskan mekanisme yang membuat itu mungkin — kaitkan dengan arah mana (DATA atau ACK) yang hilang.
4. Modul ini menambah satu paket ACK untuk setiap DATA dibanding M08. Hitung tambahan lalu lintas radio dalam persen, lalu jelaskan mengapa tambahan itu tidak menaikkan *jumlah* tabrakan DATA antar-node secara langsung, meski menambah total waktu kanal terpakai.
5. Rancang, tanpa menulis kodenya, bagaimana M08C seharusnya membedakan retry dari kiriman baru di sisi gateway — mengapa SEQ yang sama harus **tidak** dihitung dua kali sebagai data baru?

10.11 Concept Check

1. Mengapa ACK pada modul ini harus menyebut NODE, sedangkan ACK M04 cukup nomor urut saja?
2. Apa bedanya kegagalan yang “diketahui” (M08B) dengan kegagalan yang “senyap” (M08)? Sisi mana yang mengetahuinya masing-masing?
3. Mengapa [FAIL] tidak lantas berarti gateway tidak menerima apa-apa?
4. Mengapa modul ini sengaja belum mengirim ulang paket yang gagal?
5. Apa risiko yang harus diantisipasi **sebelum** retry ditambahkan di M08C?

10.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Statistik dan informasi tautan yang lebih kaya

CH-1 — Persentase langsung Tampilkan tingkat keberhasilan berjalan $(OK / (OK+FAIL)) \times 100$ % di setiap baris statistik node, seperti CH-2 pada M04.

CH-2 — RSSI di ACK Sisipkan RSSI yang diterima gateway ke dalam ACK (ACK=1, SEQ=5, RSSI=-45), sehingga node mengetahui kualitas tautan dari sisi gateway tanpa paket tambahan.

Level 3 — Tantangan · Mengukur harga keandalan

CH-3 — Bandingkan overhead Ukur total waktu kanal terpakai (waktu udara DATA + waktu udara ACK + waktu tunggu) per siklus sukses, lalu bandingkan dengan waktu satu siklus M08 (tanpa ACK sama sekali).

CH-4 — Tiga node Tambahkan environment node3, sesuaikan NODE_COUNT di gateway, dan amati apakah tingkat keberhasilan turun ketika jumlah node bertambah pada interval kirim yang sama.

10.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — ACK beralamat, kegagalan diketahui vs senyap, overhead ACK.
3. Konfigurasi — format NODE=... dan ACK=..., nilai ACK_TIMEOUT, parameter radio.
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 10.12, Tabel 10.13, dan Tabel 10.14.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri, khususnya mengenai apa yang sudah diketahui sekarang dibanding M08, dan apa yang masih belum (retry — lihat M08C).

11. Modul 08C — Random Backoff dan Retry: Coba Lagi

LoRa COMMUNICATION LAB · MODUL 08C

Try Again, Randomly — Random Backoff dan Retry

Arduino Uno + Dragino LoRa Shield v1.2 · topologi bintang, 2 node + gateway · interrupt + timeout + retry · level Advanced · 1 × 50 menit · folder kode Modul08c_loRa_aloha_retry

PENGANTAR MODUL · Gambaran Umum

Modul 08C dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat lanjut, dan merupakan pengembangan langsung dari M08B — bukan modul berdiri sendiri. **Di modul inilah Pure ALOHA akhirnya lengkap:** rumusan asli Abramson (1970) memang terdiri dari tiga bagian — kirim bebas (M08), tunggu ACK (M08B), dan kirim ulang setelah jeda acak bila ACK tak datang (modul ini). Dua modul sebelumnya adalah potongan protokol; mulai sekarang yang diukur adalah protokolnya utuh.

11.1 Pendahuluan

Misinya menutup satu kelemahan yang sengaja dibiarkan terbuka di M08B: node yang mencatat [FAIL] tidak pernah mencoba lagi, data itu hilang permanen. Modul ini menambahkan **satu** kemampuan baru — kirim ulang otomatis dengan jeda acak sebelum tiap percobaan — dan membiarkan segala sesuatu yang lain (payload, kontrak ACK, parameter radio) tetap sama persis dengan M08B.

M08B membuktikan node bisa *tahu* apakah paketnya sampai. Modul ini menjawab pertanyaan lanjutannya: kalau tidak sampai, apa yang seharusnya dilakukan node? Jawaban paling sederhana — coba lagi segera — justru berbahaya pada kanal bersama: dua node yang bertabrakan lalu sama-sama menunggu jeda **tetap** akan tetap sinkron dan bertabrakan lagi pada percobaan berikutnya. **Random backoff** memecah sinkronisasi kebetulan itu dengan memberi tiap node jeda tunggu yang berbeda-beda secara acak sebelum mencoba ulang. Tetapi retry membawa persoalan baru yang tidak ada di M08B: gateway kini bisa menerima SEQ yang sama dua kali atau lebih — bukan karena node mengulang datanya, melainkan karena ACK sebelumnya yang hilang di jalan. Modul ini mengajarkan cara gateway membedakan **data baru** dari **duplicate**, sebuah persoalan yang tidak pernah muncul selama tidak ada retry sama sekali.

Prasyaratnya adalah M08B untuk kontrak NODE= dan ACK= beralamat serta pola tunggu-ACK-dengan-timeout. Yang dibangun di sini adalah loop retry terbatas (MAX_RETRIES), jeda acak antar-percobaan (*random backoff*), dan pengenalan paket duplicate di gateway lewat perbandingan SEQ — tanpa mengubah format payload maupun format ACK sama sekali. Kontrak SEQ yang sama ini dipakai lagi di M10, tempat slot waktu menggantikan backoff sebagai cara menghindari tabrakan.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
08	Penjadwalan dilepas — node kirim bebas, tabrakan senyap diamati
08B	ACK ditempelkan di atas M08 — node tahu SUCCESS/FAILED, belum ada retry
08C (ini)	Random backoff + retry — [FAIL] bukan akhir cerita, dan Pure ALOHA menjadi lengkap
09	Carrier sense — dengar dulu sebelum bicara, tabrakan dihindari sebelum terjadi

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
10	SYNC + slot waktu – Slotted ALOHA (slot diundi) vs TDMA (slot tetap)

Tabel 11.1: Peta modul pada arc kedua seri LoRa

Kontrak data lab ini. Payload data dan format ACK **identik** dengan M08B: `NODE=<id>,SEQ=<n>,R1T=.,R1H=.,R2T=.,R2H=.` dibalas `ACK=<id>,SEQ=<n>`. Yang berubah adalah **kapen** SEQ bertambah: pada M08B, SEQ naik setiap siklus tanpa peduli hasilnya; pada modul ini, SEQ **tetap** selama node masih dalam proses retry untuk data yang sama, dan baru naik setelah siklus itu berakhir – entah karena ACK akhirnya diterima, atau karena jatah retry habis. Prinsip inilah yang membuat gateway bisa mengenali paket ber-SEQ sama sebagai percobaan ulang dari data yang sama, bukan data baru yang kebetulan bernomor sama.

11.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Memulihkan kegagalan tanpa merusak keutuhan data

1. Menerapkan loop retry terbatas jumlah (MAX_RETRIES) yang mengulang paket yang sama, bukan membangkitkan data baru pada tiap percobaan.
2. Menjelaskan mengapa jeda antar-retry harus acak, dan menunjukkan risiko konkret jeda tetap lewat percobaan dua node yang disinkronkan sengaja.
3. Merancang dan menjelaskan mekanisme deteksi duplicate di gateway berbasis SEQ, termasuk mengapa gateway tetap harus membalas ACK untuk duplicate.
4. Membandingkan tingkat kegagalan **permanen** (setelah retry habis) pada modul ini dengan tingkat kegagalan **sekali coba** pada M08B, pada kondisi kanal yang sama.
5. Menghitung ongkos tambahan (waktu, lalu lintas radio) yang dibayar retry untuk menaikkan keandalan, dan menentukan titik ketika MAX_RETRIES yang lebih besar tidak lagi sepadan.

Kriteria keberhasilan

- ☐ Node mencetak [RETRY n/MAX] dan [BACKOFF] ketika ACK pertama tidak tiba, lalu akhirnya [OK] atau [FAIL] setelah jatah retry habis.
- ☐ Gateway mencetak PAKET DITERIMA (DUPLICATE) untuk retry yang SEQ-nya sudah pernah diproses, dan **tidak** menambah statistik data baru untuknya.
- ☐ Gateway tetap membalas ACK baik untuk data baru maupun duplicate.
- ☐ [GAP] di gateway (gagal permanen) jauh lebih jarang muncul dibanding [FAIL] M08B pada interval kirim yang sama.

11.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Retry	Mengirim ulang paket DATA yang sama (SEQ tidak berubah) setelah ACK-nya tidak tiba sebelum timeout.
Random backoff	Jeda tunggu acak (bukan tetap) sebelum tiap percobaan retry, dimaksudkan memecah sinkronisasi kebetulan antar-node.
MAX_RETRIES	Batas jumlah percobaan ulang. Modul ini memakai 3 – total maksimum 4 kali kirim per SEQ (1 asli + 3 retry).

Istilah	Definisi kerja di lab ini
Gagal permanen	Keadaan ketika seluruh percobaan (asli + retry) tidak mendapat ACK. Baru di titik inilah data benar-benar dianggap hilang.
Duplicate	Paket ber-SEQ sama dengan yang sudah pernah diproses gateway sebagai data baru — hasil retry, bukan pembacaan sensor baru.
<i>Thundering herd</i> / sinkronisasi kebetulan	Risiko dua node yang bertabrakan lalu mengulang pada waktu yang (hampir) sama karena jeda retry-nya sama, sehingga bertabrakan lagi.

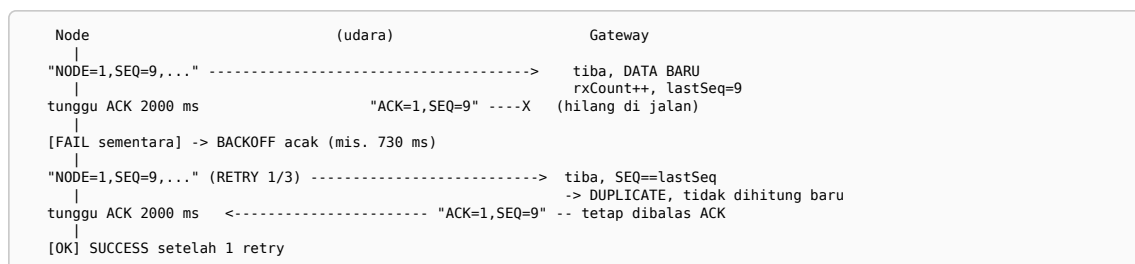
Tabel 11.2: Istilah kerja Modul 08C

Mengapa backoff harus acak, bukan tetap. Andaikan kedua node memakai jeda retry tetap, katakanlah 500 ms. Bila keduanya bertabrakan pada percobaan pertama, keduanya akan menunggu tepat 500 ms yang sama, lalu mengirim ulang pada saat yang (hampir) sama lagi — tabrakan berulang tanpa akhir yang pasti. Random backoff memutus pola ini: peluang kedua node memilih jeda yang sama persis sangat kecil, sehingga salah satu hampir selalu mengirim lebih dulu dan berhasil sebelum yang lain mencoba. Ini adalah prinsip yang sama dengan *binary exponential backoff* pada Ethernet klasik, hanya saja modul ini memakai jeda acak seragam (bukan bertambah eksponensial) untuk tetap sederhana — lihat Challenge CH-1 untuk pengembangannya.

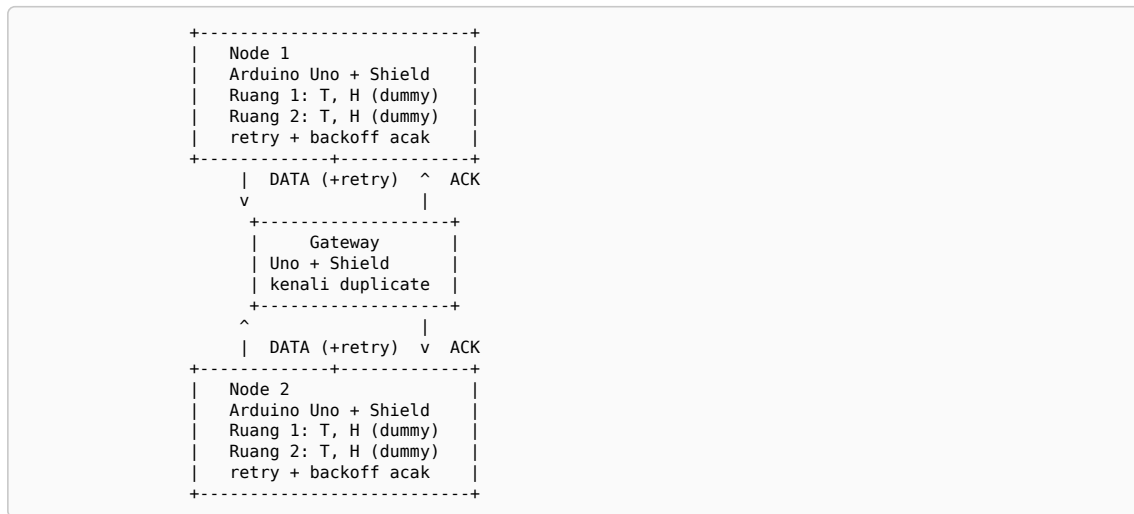
Mengapa gateway harus mengenali duplicate, bukan sekadar menerima apa adanya. Tanpa deteksi duplicate, satu pembacaan sensor yang di-retry dua kali akan tercatat sebagai **dua** data berbeda di sisi gateway — padahal keduanya berasal dari satu SEQ, satu pembacaan sensor yang sama. Kesalahan ini murni administratif (bukan kesalahan sensor), tetapi bila dibiarkan akan mengacaukan setiap statistik yang dihitung dari jumlah paket diterima. Perbandingan `seq <= lastSeq[nodeId]` menutup celah ini: SEQ yang tidak lebih besar dari yang terakhir diproses pasti bukan data baru.

Mengapa gateway tetap membalas ACK untuk duplicate. Duplicate terjadi justru **karena** ACK sebelumnya tidak sampai ke node — node tidak tahu bahwa gateway-nya sebenarnya sudah menerima data itu. Bila gateway diam saja terhadap duplicate, node akan terus mengulang sampai MAX_RETRIES habis dan mencatat [FAIL] padahal datanya sebenarnya sudah tersimpan sejak percobaan pertama. Membalas ACK untuk duplicate adalah yang membuat retry benar-benar menaikkan keandalan, bukan sekadar menambah lalu lintas radio tanpa hasil.

Sekuens yang diamati (ACK pertama hilang, retry berhasil)



11.4 Topologi



Node	Environment	Peran	Mekanisme TX/RX	Timeout	Retry / Backoff
Node 1	node1	Kirim dummy, retry bila gagal	TX blocking + RX interrupt	2000 ms	maks 3× / 200–1500 ms acak
Node 2	node2	Kirim dummy, retry bila gagal	TX blocking + RX interrupt	2000 ms	maks 3× / 200–1500 ms acak
Gateway	gateway	Terima, kenali duplicate, balas ACK	RX interrupt + TX blocking	—	—

Tabel 11.3: Peran tiap node Modul 08C

11.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3

Tabel 11.4: Alat dan bahan Modul 08C

Struktur proyek

```

Modul08c_lora_aloha_retry/
├── platformio.ini
├── lora_monitor.py      ← dashboard 3-panel live (Gateway/Node1/Node2) + CSV
├── upload_auto.py      ← deteksi port otomatis saat unggah
├── logserial.md         ← cuplikan log serial aktual dari pengujian perangkat
├── src/
│   ├── node/main.cpp   ← dummy Ruang 1+2, retry + backoff acak (env node1, node2)
│   └── gateway/main.cpp ← terima, kenali duplicate via SEQ, balas ACK (env gateway)
  
```

11.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul08c_lora_aloha_retry/](#)

- [Modul08c_lora_aloha_retry/platformio.ini](#)
- [Modul08c_lora_aloha_retry/src/node/main.cpp](#)
- [Modul08c_lora_aloha_retry/src/gateway/main.cpp](#)
- [Modul08c_lora_aloha_retry/lora_monitor.py](#)
- [Modul08c_lora_aloha_retry/upload_auto.py](#)

```
[platformio]
default_envs = gateway

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 115200

[env:gateway]
build_src_filter = +<gateway/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua node memakai source yang sama; nomornya ditentukan build flag.
[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=1
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=2
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2
```

Kode 11.1: platformio.ini Modul 08C — environment gateway, node1, node2

```
/*
LoRa ALOHA + ACK + Random Backoff + Retry
Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: node1 / node2

Pengembangan langsung dari M08B: node tidak lagi menyerah pada [FAIL]
pertama. Ketika ACK tidak tiba sebelum timeout, node menunggu selang
ACAK (random backoff) lalu mengirim ULANG paket DATA yang SAMA -- SEQ
tidak berubah selama retry, karena ini pengiriman ulang, bukan data baru.
Data dummy dibangkitkan sekali per SEQ dan dipertahankan sepanjang seluruh
percobaan retry-nya, persis seperti pembacaan sensor sungguhan yang tidak
berubah hanya karena paketnya diulang.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Balasan : "ACK=<id>,SEQ=<n>"

Mengapa backoff acak, bukan jeda tetap: dua node yang bertabrakan lalu
sama-sama menunggu jeda TETAP akan bertabrakan lagi pada retry pertama --
keduanya tetap sinkron. Jeda ACAK memecah sinkronisasi kebetulan itu.

Mekanisme:
```

```

TX   : blocking (endPacket)
RX   : interrupt DIO0 + flag, aktif selagi menunggu ACK
Retry: maksimum MAX_RETRIES kali per SEQ, lalu menyerah permanen

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DIO0 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN      10
#define RST_PIN      9
#define DIO0_PIN     2
#define LED_PIN      LED_BUILTIN

#define FREQUENCY     433E6
#define BANDWIDTH     125E3
#define SPREADING_FACTOR 7
#define CODING_RATE   5
#define TX_POWER      17

// Nomor node datang dari build flag (-DNODE_ID=1 atau 2) di platformio.ini,
// sehingga kedua node memakai file source yang sama persis.
#ifndef NODE_ID
#define NODE_ID 1
#endif

#define SEND_INTERVAL_MIN 2000
#define SEND_INTERVAL_MAX 5000
#define ACK_TIMEOUT      2000

#define MAX_RETRIES      3      // percobaan ulang setelah kiriman pertama
#define BACKOFF_MIN_MS  200
#define BACKOFF_MAX_MS  1500

#define ROOM1_TEMP_BASE 27.0
#define ROOM1_HUM_BASE  60
#define ROOM2_TEMP_BASE 25.0
#define ROOM2_HUM_BASE  68

unsigned long seq = 0;
unsigned long okCount = 0;
unsigned long failCount = 0;
unsigned long totalRetriesUsed = 0;

volatile bool ackFlag = false;

void onAckReceived(int size) {
    if (size > 0) ackFlag = true;
}

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

float dummyTemp(float base) {
    return base + (random(-30, 31) / 10.0);    // +-3.0 C
}

int dummyHum(int base) {
    return base + random(-15, 16);            // +-15 %
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    randomSeed(analogRead(A0));

    Serial.print(F("=== LoRa ALOHA+ACK+RETRY - NODE "));
    Serial.print(NODE_ID);
    Serial.println(F(" =="));
    Serial.print(F("Init LoRa ... "));

```

```

LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
}

LoRa.setSpreadingFactor(SPREADING_FACTOR);
LoRa.setSignalBandwidth(BANDWIDTH);
LoRa.setCodingRate4(CODING_RATE);
LoRa.setTxPower(TX_POWER);
LoRa.onReceive(onAckReceived);

Serial.println(F("OK"));
Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
Serial.print(F("ACK timeout: ")); Serial.print(ACK_TIMEOUT);
Serial.print(F(" ms | Max retry: ")); Serial.print(MAX_RETRIES);
Serial.print(F(" | Backoff: ")); Serial.print(BACKOFF_MIN_MS);
Serial.print(F("-")); Serial.print(BACKOFF_MAX_MS); Serial.println(F(" ms"));
Serial.println(F("Peran: NODE (ALOHA + ACK + Random Backoff + Retry)\n"));
}

void loop() {
    // Data dibangkitkan SEKALI per SEQ dan dipertahankan sepanjang retry --
    // paket yang diulang harus membawa pembacaan yang sama, bukan yang baru.
    float r1t = dummyTemp(ROOM1_TEMP_BASE);
    int r1h = dummyHum(ROOM1_HUM_BASE);
    float r2t = dummyTemp(ROOM2_TEMP_BASE);
    int r2h = dummyHum(ROOM2_HUM_BASE);

    String payload = "NODE=" + String(NODE_ID) +
        ",SEQ=" + String(seq) +
        ",R1T=" + String(r1t, 1) +
        ",R1H=" + String(r1h) +
        ",R2T=" + String(r2t, 1) +
        ",R2H=" + String(r2h);
    String expectedAck = "ACK=" + String(NODE_ID) + ",SEQ=" + String(seq);

    bool gotAck = false;
    int attempt = 0; // 0 = kiriman pertama, 1..MAX_RETRIES = nomor retry

    while (true) {
        if (attempt == 0) {
            Serial.print(F("[TX] ")); Serial.println(payload);
        } else {
            Serial.print(F("[RETRY ]")); Serial.print(attempt); Serial.print(F("/"));
            Serial.print(MAX_RETRIES); Serial.print(F("] ")); Serial.println(payload);
        }

        digitalWrite(LED_PIN, HIGH);
        transmit(payload);
        digitalWrite(LED_PIN, LOW);

        ackFlag = false;
        LoRa.receive();

        unsigned long waitStart = millis();
        while (millis() - waitStart < ACK_TIMEOUT) {
            if (ackFlag) {
                ackFlag = false;

                String reply = "";
                while (LoRa.available()) {
                    reply += (char)LoRa.read();
                }

                if (reply == expectedAck) {
                    gotAck = true;
                    break;
                }
                Serial.print(F("[RX] WARN: balasan tak sesuai (")); Serial.print(reply);
                Serial.println(F("), tetap tunggu..."));
            }
        }

        if (gotAck) break;
        if (attempt >= MAX_RETRIES) break; // jatah retry habis, menyerah permanen

        attempt++;
        unsigned long backoff = random(BACKOFF_MIN_MS, BACKOFF_MAX_MS + 1);
        Serial.print(F("[BACKOFF] tunggu ")); Serial.print(backoff);
        Serial.println(F(" ms sebelum retry"));
        delay(backoff);
    }
}

```

```

    }

    if (gotAck) {
        okCount++;
        totalRetriesUsed += attempt;
        Serial.print(F("[OK] SUCCESS setelah ")); Serial.print(attempt); Serial.print(F(" retry"));
    } else {
        failCount++;
        totalRetriesUsed += attempt;
        Serial.print(F("[FAIL] Gagal permanen setelah ")); Serial.print(MAX_RETRIES); Serial.print(F(" retry"));
    }
    Serial.print(F(" | OK: ")); Serial.print(okCount);
    Serial.print(F(" | FAIL: ")); Serial.print(failCount);
    Serial.print(F(" | Total retry terpakai: ")); Serial.println(totalRetriesUsed);
    Serial.println();

    seq++;

    unsigned long interval = random(SEND_INTERVAL_MIN, SEND_INTERVAL_MAX + 1);
    delay(interval);
}

```

Kode 11.2: src/node/main.cpp — retry terbatas dengan random backoff

```

/*
  LoRa ALOHA + ACK + Random Backoff + Retry
  Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
  Library : LoRa by sandeepmistry v0.8.x
  Environment PlatformIO: gateway

  Pengembangan langsung dari M08B: karena node sekarang mengirim ULANG
  paket yang ACK-nya hilang, gateway bisa menerima SEQ yang SAMA lebih dari
  sekali -- bukan data baru, melainkan retry akibat ACK sebelumnya tidak
  sampai ke node. Gateway membedakan keduanya lewat lastSeq per node: SEQ
  yang sama dengan yang terakhir SUDAH diproses berarti duplicate, dicetak
  sebagai [DUP] dan TIDAK dihitung sebagai data baru -- tetapi tetap dibalas
  ACK, sebab justru itulah yang membuat node akhirnya berhasil.

  Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
  Balasan : "ACK=<id>,SEQ=<n>"

  Mekanisme:
  RX : interrupt DI00 + flag (non-blocking)
  TX : blocking (endPacket), langsung setelah RX selesai diproses,
      dikirim untuk data baru MAUPUN duplicate

  Pin Mapping Dragino Shield v1.2:
  NSS/CS -> D10, DI00 -> D2, RST -> D9
  SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_DURATION      150
#define NODE_COUNT        2 // Node 1 dan Node 2 -> indeks array 1..2

volatile bool rxFlag = false;

unsigned long ledOnTime = 0;
bool ledActive = false;

unsigned long rxCount[NODE_COUNT + 1] = {0}; // data BARU (bukan duplicate) yang diterima
unsigned long dupCount[NODE_COUNT + 1] = {0}; // retry yang dikenali sebagai duplicate
long lastSeq[NODE_COUNT + 1]; // SEQ terakhir yang sudah diproses sebagai data baru
unsigned long lostEst[NODE_COUNT + 1] = {0}; // gagal permanen (semua retry habis)

```

```

void onReceive(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

String getField(const String& data, const String& key) {
    String needle = key + "=";
    int start = data.indexOf(needle);
    if (start == -1) return "";
    start += needle.length();
    int end = data.indexOf(' ', start);
    if (end == -1) end = data.length();
    return data.substring(start, end);
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    for (int i = 0; i <= NODE_COUNT; i++) lastSeq[i] = -1;

    Serial.println(F("=== LoRa ALOHA+ACK+RETRY - GATEWAY ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.onReceive(onReceive);
    LoRa.receive();

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.println(F("Peran: GATEWAY (ALOHA + ACK + Retry) -- kenali duplicate lewat SEQ"));
    Serial.println(F("Menunggu paket dari Node 1 & Node 2...\n"));
}

void loop() {
    updateLED();

    if (!rxFlag) return;
    rxFlag = false;

    String received = "";
    while (LoRa.available()) {
        received += (char)LoRa.read();
    }

    if (!received.startsWith("NODE=")) {
        Serial.print(F("[WARN] Paket tak dikenal (mungkin sisa tabrakan): "));
        Serial.println(received);
        LoRa.receive();
        return;
    }

    int nodeId = getField(received, "NODE").toInt();
    long seq = getField(received, "SEQ").toInt();
    float r1t = getField(received, "R1T").toFloat();
    int r1h = getField(received, "R1H").toInt();
    float r2t = getField(received, "R2T").toFloat();

```



```

int r2h = getField(received, "R2H").toInt();

triggerLED();

bool validNode = (nodeId >= 1 && nodeId <= NODE_COUNT);
bool isDuplicate = validNode && (lastSeq[nodeId] >= 0) && (seq <= lastSeq[nodeId]);

if (isDuplicate) {
    dupCount[nodeId]++;
    Serial.println(F("=== PAKET DITERIMA (DUPLICATE) ==="));
    Serial.print(F(" Node : ")); Serial.println(nodeId);
    Serial.print(F(" SEQ : ")); Serial.println(seq);
    Serial.println(F(" -- sudah pernah diproses, ini retry akibat ACK sebelumnya hilang"));
    Serial.print(F(" Duplicate ke- ")); Serial.print(dupCount[nodeId]);
    Serial.print(F(" untuk Node ")); Serial.println(nodeId);
} else {
    Serial.println(F("=== PAKET DITERIMA (DATA BARU) ==="));
    Serial.print(F(" Node : ")); Serial.println(nodeId);
    Serial.print(F(" SEQ : ")); Serial.println(seq);
    Serial.print(F(" Ruang 1 : ")); Serial.print(r1t, 1); Serial.print(F(" C, "));
    Serial.print(r1h); Serial.println(F(" %"));
    Serial.print(F(" Ruang 2 : ")); Serial.print(r2t, 1); Serial.print(F(" C, "));
    Serial.print(r2h); Serial.println(F(" %"));
    Serial.print(F(" RSSI : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
    Serial.print(F(" SNR : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));
}

if (validNode) {
    rxCount[nodeId]++;

    if (lastSeq[nodeId] >= 0) {
        long gap = seq - lastSeq[nodeId] - 1;
        if (gap > 0) {
            // Retry M09 membuat ini jauh lebih jarang daripada [GAP] M08/M08B --
            // hanya muncul kalau SELURUH percobaan retry untuk satu SEQ gagal.
            lostEst[nodeId] += gap;
            Serial.print(F(" [GAP] SEQ meloncat ")); Serial.print(gap);
            Serial.println(F(" -- gagal permanen (retry habis), bukan sekadar tabrakan sesaat"));
        }
        lastSeq[nodeId] = seq;

        Serial.print(F(" Statistik Node ")); Serial.print(nodeId);
        Serial.print(F(": baru=")); Serial.print(rxCount[nodeId]);
        Serial.print(F(" | duplicate=")); Serial.print(dupCount[nodeId]);
        Serial.print(F(" | gagal permanen (est)=")); Serial.println(lostEst[nodeId]);
    }
}

// ACK dibalas untuk DATA BARU maupun DUPLICATE -- retry hanya berhenti
// kalau node akhirnya menerima ACK, tidak peduli ini percobaan keberapa.
if (validNode) {
    String ack = "ACK=" + String(nodeId) + ",SEQ=" + String(seq);
    LoRa.beginPacket();
    LoRa.print(ack);
    LoRa.endPacket();
    Serial.print(F(" [TX] ")); Serial.println(ack);
}
Serial.println(F("=====\n"));

LoRa.receive();
}

```

Kode 11.3: src/gateway/main.cpp – pengenalan duplicate lewat perbandingan SEQ

```
#!/usr/bin/env python3
"""
LoRa ALOHA + ACK + Retry Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Gateway <-> Node 1 & Node 2 (ACK + random backoff + retry otomatis saat timeout)

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
    pip install pyserial rich

Cara pakai:
    python lora_monitor.py                # port bawaan sesuai OS
```

```
python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.table import Table
    from rich.layout import Layout
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.columns import Columns
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

#
# KONFIGURASI
#
# Port bawaan mengikuti sistem operasi. Di linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --gateway/--n1/--n2.
if sys.platform.startswith("win"):
    DEFAULT_GW_PORT = "COM3"
    DEFAULT_N1_PORT = "COM4"
    DEFAULT_N2_PORT = "COM5"
else:
    DEFAULT_GW_PORT = "/dev/ttyACM0"
    DEFAULT_N1_PORT = "/dev/ttyACM1"
    DEFAULT_N2_PORT = "/dev/ttyACM2"
DEFAULT_BAUD = 115200
LOG_LINES = 16
REFRESH_HZ = 4
GAP_WARN_STREAK = 3

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default gateway/n1/n2 selalu
    mengikuti apa yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
        return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))
    # Pure ALOHA (M08A) tidak punya ACK sama sekali -- node tidak pernah tahu
    # apakah paketnya sampai, jadi baris OK/FAIL/Retry di panel node disembunyikan.
    HAS_ACK = True

#
# DATA STRUCTURES
#

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str
```

```

@dataclass
class GatewayState:
    port: str = DEFAULT_GW_PORT
    connected: bool = False
    rx_total: int = 0
    rx_n1: int = 0
    rx_n2: int = 0
    gap_n1: int = 0
    gap_n2: int = 0
    lost_n1: int = 0
    lost_n2: int = 0
    dup_n1: int = 0
    dup_n2: int = 0
    last_rssi: int = 0
    last_snr: float = 0.0
    rssi_sum: int = 0
    snr_sum: float = 0.0
    rssi_count: int = 0
    cycle: int = 0
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

@dataclass
class NodeState:
    name: str = ""
    port: str = ""
    connected: bool = False
    tx_count: int = 0
    ok_count: int = 0
    fail_count: int = 0
    retry_count: int = 0
    last_seq: int = -1
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.gw = GatewayState()
        self.n1 = NodeState("NODE 1", DEFAULT_N1_PORT)
        self.n2 = NodeState("NODE 2", DEFAULT_N2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

#
# PARSERS
#
# Semua regex dibuat agar aman dipakai lintas modul (M08A/M08B/M09/M10) --
# token yang tidak muncul pada suatu modul (mis. RETRY/SYNC) hanya tidak
# pernah cocok, tanpa memengaruhi parsing token lain.

RE_NODE_LINE = re.compile(r'Node\s+:\s*(\d+)')
RE_SEQ_LINE = re.compile(r'SEQ\s+:\s*(\d+)')
RE_RSSI_LINE = re.compile(r'RSSI\s+:\s*(-?\d+)')
RE_SNR_LINE = re.compile(r'SNR\s+:\s*(-?\d+\.?\d*)')
RE_GAP_LINE = re.compile(r'\[GAP\] SEQ meloncat\s+(\d+)')
RE_STAT_LINE = re.compile(r'Statistik Node\s+(\d+):\s*diterima=(\d+)\s*\|\s*perkiraan hilang=(\d+)')
RE_SYNC_TX = re.compile(r'\[TX\]\s+SYNC=(\d+)')
RE_CYCLE_DONE = re.compile(r'Cycle\s+(\d+)\s+selesai')
# M09: "Statistik Node X: baru=Y | duplicate=Z | gagal permanen (est.)=W"
RE_STAT_RETRY = re.compile(r'Statistik Node\s+(\d+):\s*baru=(\d+)\s*\|\s*duplicate=(\d+)\s*\|\s*gagal permanen\s*(est\.)=(\d+)')
# M10: "--- Cycle N selesai | N1: diterima=Y hilang=Z N2: diterima=Y hilang=Z ---"
RE_STAT_CYCLE = re.compile(r'N(\d+):\s*diterima=(\d+)\s*hilang=(\d+)')

RE_NODE_TX = re.compile(r'\[TX\].*NODE=(\d+),SEQ=(\d+)')
RE_NODE_RETRY = re.compile(r'\[RETRY\s+(\d+)/(\d+)\]')
RE_NODE_OK = re.compile(r'\[OK\]')
RE_NODE_FAIL = re.compile(r'\[FAIL\]')

def parse_gateway_line(line: str, st: GatewayState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    if 'PAKET DITERIMA' in line:
        st.rx_total += 1

    m = RE_RSSI_LINE.search(line)
    if m:
        st.last_rssi = int(m.group(1))
        st.rssi_sum += st.last_rssi
        st.rssi_count += 1

```

```

m = RE_SNR_LINE.search(line)
if m:
    st.last_snr = float(m.group(1))
    st.snr_sum += st.last_snr

m = RE_GAP_LINE.search(line)
if m:
    gap = int(m.group(1))
    anomalies.append(Anomaly(ts, "GATEWAY", "GAP", f"SEQ meloncat {gap} -- indikasi tabrakan/paket hilang"))

m = RE_STAT_LINE.search(line)
if m:
    node_id, rx, lost = int(m.group(1)), int(m.group(2)), int(m.group(3))
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_STAT_RETRY.search(line)
if m:
    node_id, rx, dup, lost = int(m.group(1)), int(m.group(2)), int(m.group(3)), int(m.group(4))
    if node_id == 1:
        st.rx_n1, st.dup_n1, st.lost_n1 = rx, dup, lost
    elif node_id == 2:
        st.rx_n2, st.dup_n2, st.lost_n2 = rx, dup, lost

for node_id_s, rx_s, lost_s in RE_STAT_CYCLE.findall(line):
    node_id, rx, lost = int(node_id_s), int(rx_s), int(lost_s)
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_SYNC_TX.search(line)
if m:
    st.cycle = int(m.group(1))

m = RE_CYCLE_DONE.search(line)
if m:
    st.cycle = int(m.group(1))

def parse_node_line(line: str, st: NodeState, node_label: str, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = RE_NODE_TX.search(line)
    if m:
        seq = int(m.group(2))
        if st.last_seq >= 0 and seq == st.last_seq and 'RETRY' not in line:
            pass # aman, tidak dipakai
        st.last_seq = seq
        if 'RETRY' not in line:
            st.tx_count += 1

    m = RE_NODE_RETRY.search(line)
    if m:
        st.retry_count += 1

    if RE_NODE_OK.search(line):
        st.ok_count += 1
    elif RE_NODE_FAIL.search(line):
        st.fail_count += 1
        if st.fail_count and st.fail_count % 5 == 0:
            anomalies.append(Anomaly(ts, node_label, "FAIL_COUNT",
                f"{node_label} sudah {st.fail_count}x gagal (tanpa ACK)"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
    shared: SharedState, role: str):
    if role == "GW":
        state = shared.gw
    elif node_key == "N1":
        state = shared.n1
    else:
        state = shared.n2

    while shared.running:
        try:

```

```

with serial.Serial(port, baud, timeout=1) as ser:
    with shared.lock:
        state.connected = True
    while shared.running:
        try:
            raw = ser.readline()
            if not raw:
                continue
            line = raw.decode('utf-8', errors='replace').rstrip()
            if not line:
                continue
            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
            with shared.lock:
                state.raw_log.append((ts, line))
                if role == "GW":
                    parse_gateway_line(line, state, shared.anomalies)
                else:
                    parse_node_line(line, state, node_key, shared.anomalies)
            shared.csv_queue.put({
                'ts': ts, 'node': node_key, 'raw': line
            })
        except (serial.SerialException, OSError):
            break
    except (serial.SerialException, OSError):
        with shared.lock:
            state.connected = False
        time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                           'node': row['node'],
                           'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"
    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str) -> str:
    if '[FAIL]' in line: return "red"
    if '[GAP]' in line: return "bold red"
    if '[RETRY]' in line: return "bold yellow"
    if '[BACKOFF]' in line: return "yellow"
    if '[OK]' in line: return "bold green"
    if '[TX]' in line: return "cyan"
    if 'PAKET DITERIMA' in line: return "green"
    if '[WAIT]' in line: return "dim"
    if '[WARN]' in line: return "yellow"
    return "dim white"

def build_gateway_panel(st: GatewayState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" Total diterima : ", style="dim"); t.append(f"{st.rx_total}\n", style="bold white")
    if st.cycle:
        t.append(" Cycle terakhir : ", style="dim"); t.append(f"{st.cycle}\n", style="bold cyan")
    t.append("\n")

    dup_suffix1 = f" dup={st.dup_n1}" if st.dup_n1 else ""

```

```

dup_suffix2 = f" dup={st.dup_n2}" if st.dup_n2 else ""
t.append(" Node 1 ", style="bold cyan")
t.append(f"diterima={st.rx_n1:<4} hilang={st.lost_n1}{dup_suffix1}\n", style="white")
t.append(" Node 2 ", style="bold magenta")
t.append(f"diterima={st.rx_n2:<4} hilang={st.lost_n2}{dup_suffix2}\n\n", style="white")

rssi_avg = (st.rssi_sum / st.rssi_count) if st.rssi_count else 0.0
snr_avg = (st.snr_sum / st.rssi_count) if st.rssi_count else 0.0
t.append(" RSSI terakhir : ", style="dim"); t.append(f"{st.last_rssi} dBm\n", style="white")
t.append(" SNR terakhir : ", style="dim"); t.append(f"{st.last_snr:.2f} dB\n", style="white")
t.append(" RSSI rata-rata : ", style="dim"); t.append(f"{rssi_avg:.1f} dBm\n", style="white")
t.append(" SNR rata-rata : ", style="dim"); t.append(f"{snr_avg:.2f} dB\n\n", style="white")

t.append(" Raw Log\n", style="bold dim")
t.append(" " + "-" * 30 + "\n", style="dim")
for ts, line in list(st.raw_log)[-LOG_LINES:]:
    style = log_line_style(line)
    t.append(f" {ts[-8:]} {line[:40]}\n", style=style)

return Panel(t, title="[bold cyan]GATEWAY[/]",
             border_style="cyan", box=box.ROUNDED)

def build_node_panel(st: NodeState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" TX terkirim : ", style="dim"); t.append(f"{st.tx_count:>6}\n", style="bold white")
    t.append(" SEQ terakhir : ", style="dim"); t.append(f"{st.last_seq:>6}\n\n", style="white")

    if HAS_ACK:
        tot = st.ok_count + st.fail_count
        pct = (st.ok_count / tot * 100) if tot else 0.0
        t.append(" OK : ", style="dim"); t.append(f"{st.ok_count:>6}\n", style="bold green")
        t.append(" FAIL : ", style="dim"); t.append(f"{st.fail_count:>6}\n",
            style="bold red" if st.fail_count else "dim")
        if tot:
            t.append(" Rate : ", style="dim"); t.append(f"{pct:>6.2f}%\n", style=pct_style(st.ok_count,
tot))
        t.append(" Retry : ", style="dim"); t.append(f"{st.retry_count:>6}\n\n",
            style="yellow" if st.retry_count else "dim")
    else:
        t.append(" (Pure ALOHA -- tanpa ACK, node tidak pernah tahu\n", style="dim")
        t.append(" apakah paketnya sampai. Lihat panel GATEWAY untuk\n", style="dim")
        t.append(" statistik diterima/hilang per node.)\n\n", style="dim")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

    return Panel(t, title=f"[bold {color}]{label}[/]",
                 border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                        csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n", style="dim")

    kind_style = {
        'GAP': "bold red",
        'FAIL_COUNT': "bold yellow",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=style)
            t.append(f"{a.detail}\n", style="white")

```

```

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                 border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        gw = GatewayState(
            port=shared.gw.port, connected=shared.gw.connected,
            rx_total=shared.gw.rx_total, rx_n1=shared.gw.rx_n1, rx_n2=shared.gw.rx_n2,
            gap_n1=shared.gw.gap_n1, gap_n2=shared.gw.gap_n2,
            lost_n1=shared.gw.lost_n1, lost_n2=shared.gw.lost_n2,
            dup_n1=shared.gw.dup_n1, dup_n2=shared.gw.dup_n2,
            last_rssi=shared.gw.last_rssi, last_snr=shared.gw.last_snr,
            rssi_sum=shared.gw.rssi_sum, snr_sum=shared.gw.snr_sum,
            rssi_count=shared.gw.rssi_count, cycle=shared.gw.cycle,
            raw_log=deque(shared.gw.raw_log),
        )
        n1 = NodeState(
            name=shared.n1.name, port=shared.n1.port, connected=shared.n1.connected,
            tx_count=shared.n1.tx_count, ok_count=shared.n1.ok_count,
            fail_count=shared.n1.fail_count, retry_count=shared.n1.retry_count,
            last_seq=shared.n1.last_seq,
            raw_log=deque(shared.n1.raw_log),
        )
        n2 = NodeState(
            name=shared.n2.name, port=shared.n2.port, connected=shared.n2.connected,
            tx_count=shared.n2.tx_count, ok_count=shared.n2.ok_count,
            fail_count=shared.n2.fail_count, retry_count=shared.n2.retry_count,
            last_seq=shared.n2.last_seq,
            raw_log=deque(shared.n2.raw_log),
        )
        anom = deque(shared.anomalies)
        t_start = shared.session_start

    node_row = Columns([
        build_gateway_panel(gw),
        build_node_panel(n1, "NODE 1", "green"),
        build_node_panel(n2, "NODE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\
┌────────────────────────────────────────────────────────────────────────────────┐
│ LoRa ALOHA+RETRY Monitor – Evaluasi Praktikum                               │
│ Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz                     │
└────────────────────────────────────────────────────────────────────────────────┘

Tool ini membaca data serial dari 3 board LoRa (Gateway, Node 1, Node 2)
secara bersamaan dan menampilkan dashboard real-time: statistik
diterima/OK/FAIL/RETRY, RSSI/SNR, deteksi anomali (GAP/FAIL beruntun),
serta logging CSV otomatis untuk analisis data penelitian.

PENGGUNAAN:
python lora_monitor.py [opsi...]

OPSI:
--gateway PORT Port COM untuk Gateway (default: {gw})
--n1 PORT Port COM untuk Node 1 (default: {n1})
--n2 PORT Port COM untuk Node 2 (default: {n2})
--baud N Baud rate serial semua port (default: {baud})
--out FILE Nama file CSV output (default: lora_session_YYYYMMDD_HHMMSS.csv)
-h / --help Tampilkan pesan bantuan ini

CONTOH:
python lora_monitor.py
-> Jalankan dengan port & baud rate default

python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
-> Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

python lora_monitor.py --out data_jarak_10m.csv
-> Simpan log ke nama file tertentu

```

```

KONTROL DASHBOARD:
Ctrl+C          Hentikan monitoring & simpan CSV

CATATAN:
* Pastikan semua board sudah di-upload firmware dengan baud {baud}
* Upload Gateway dulu, baru Node 1 dan Node 2, sebelum menjalankan tool ini
* File CSV tersimpan otomatis saat Ctrl+C ditekan
"""
format(
    gw=DEFAULT_GW_PORT,
    n1=DEFAULT_N1_PORT,
    n2=DEFAULT_N2_PORT,
    baud=DEFAULT_BAUD,
)

def parse_args(port_aktif):
    gw_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_GW_PORT
    n1_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_N1_PORT
    n2_default = port_aktif[2] if len(port_aktif) > 2 else DEFAULT_N2_PORT

    p = argparse.ArgumentParser(
        description="LoRa ALOHA + ACK + Retry Monitor Dashboard -- tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f"  python lora_monitor.py\n"
            f"  python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2\n"
            f"  python lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--gateway', default=gw_default,
        metavar='PORT', help=f'Port COM Gateway (default: {gw_default}, dari deteksi port aktif)')
    p.add_argument('--n1', default=n1_default,
        metavar='PORT', help=f'Port COM Node 1 (default: {n1_default}, dari deteksi port aktif)')
    p.add_argument('--n2', default=n2_default,
        metavar='PORT', help=f'Port COM Node 2 (default: {n2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
        metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None,
        metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
    return p, p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    p, args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1

    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
    )
    console.print(
        f"[dim]Dipakai -> "
        f"Gateway=[yellow]{args.gateway}[/] "
        f"N1=[green]{args.n1}[/] "
        f"N2=[magenta]{args.n2}[/] "
        f"Baud=[white]{args.baud}[/][dim]\n"
    )

    shared = SharedState()
    shared.gw.port = args.gateway
    shared.n1.port = args.n1
    shared.n2.port = args.n2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_session_{ts_str}.csv"

    if not no_args:
        console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
        console.print(f"  Gateway : [yellow]{args.gateway}[/]")
        console.print(f"  Node 1  : [green]{args.n1}[/]")
        console.print(f"  Node 2  : [magenta]{args.n2}[/]")
        console.print(f"  Baud    : [white]{args.baud}[/]")

    console.print(f"  CSV      : [dim]{csv_path}[/]")
    console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:

```



```

        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread,
                        args=(args.gateway, args.baud, "GW", shared, "GW"),
                        daemon=True, name="reader-gateway"),
        threading.Thread(target=reader_thread,
                        args=(args.n1, args.baud, "N1", shared, "NODE"),
                        daemon=True, name="reader-n1"),
        threading.Thread(target=reader_thread,
                        args=(args.n2, args.baud, "N2", shared, "NODE"),
                        daemon=True, name="reader-n2"),
        threading.Thread(target=csv_writer_thread,
                        args=(shared, csv_path),
                        daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path),
                 console=console,
                 refresh_per_second=REFRESH_HZ,
                 screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass
    finally:
        shared.running = False
        time.sleep(0.6)
        console.print(f"\n[green]Session selesai. [/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 11.4: lora_monitor.py — dasbor tiga panel dengan perekaman CSV

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 09 (ALOHA + ACK + Retry) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul08c_lora_aloha_retry -e gateway -t upload -t monitor
    pio run -d Modul08c_lora_aloha_retry -e node1 -t upload -t monitor
    pio run -d Modul08c_lora_aloha_retry -e node2 -t upload -t monitor

Skrrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `gateway` (port pertama), `node1` (port kedua), dan `node2`
(port ketiga), lalu menjalankan `pio run` dengan --upload-port sesuai hasil
deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga
board selesai di-upload, skrip bertanya apakah mau langsung menjalankan
lora_monitor.py -- bila ya, dijalankan dengan --gateway ... --n1 ... --n2 ...
persis memakai port yang barusan dipakai upload (bukan deteksi ulang),
supaya tidak mungkin salah pasang.

python3 Modul08c_lora_aloha_retry/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul08c_lora_aloha_retry/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti lora_monitor.py

python3 Modul08c_lora_aloha_retry/upload_auto.py --only gateway
-> upload satu environment saja

python3 Modul08c_lora_aloha_retry/upload_auto.py --gateway /dev/ttyACM0 --node1 /dev/ttyACM1 --node2 /
dev/ttyACM2
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys

```

```

from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["gateway", "node1", "node2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"gateway": "--gateway", "node1": "--n1", "node2": "--n2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port gateway/node1/node2 PERSIS seperti yang baru dipakai upload
    -- bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
    for env, opsi in ENV_KE_OPSI.items():
        if env in port_env:
            cmd += [opsi, port_env[env]]

    print(" $", " ".join(cmd))
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--gateway", metavar="/dev/ttyXXX",
                    help="tiapa port gateway (lewati deteksi otomatis)")
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="tiapa port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="tiapa port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (gateway, node1, atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                    "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau ketiganya"
                    sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"gateway": args.gateway, "node1": args.node1, "node2": args.node2}

```

```

port_env = {}
sisas = list(aktif)
for nama in ENV_URUTAN:
    if manual[nama]:
        port_env[nama] = manual[nama]
    elif sisas:
        port_env[nama] = sisas.pop(0)

print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
for nama in ENV_URUTAN:
    if nama in port_env:
        sumber = "manual" if manual[nama] else "otomatis"
        print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
    else:
        print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
              f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

daftar = [args.only] if args.only else ENV_URUTAN
hilang = [n for n in daftar if n not in port_env]
if hilang:
    sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
            f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

for nama in daftar:
    if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
        sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

print("\nSelesai.")
tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 11.5: upload_auto.py — pemilih port otomatis saat unggah

11.7 Build dan Flash

Gateway lebih dahulu, supaya paket pertama dari node langsung dibalas.

```

pio run -d Modul08c_lora_aloha_retry -e gateway -t upload -t monitor
pio run -d Modul08c_lora_aloha_retry -e node1 -t upload -t monitor
pio run -d Modul08c_lora_aloha_retry -e node2 -t upload -t monitor

```

Monitor dashboard. python3 lora_monitor.py membaca ketiga port sekaligus dan menampilkan panel Gateway, Node 1, dan Node 2 (OK/FAIL/retry, data baru vs duplicate, RSSI dan SNR) di terminal, plus perekaman CSV otomatis. Memerlukan pip install pyserial rich.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ Port ketiga board dicatat lewat `pio device list` (atau `python3 ../tools/deteksi_port.py`) dan diisikan ke `platformio.ini`.
- ☐ Tiga Serial Monitor 115200 baud siap, ketiganya terlihat bersamaan.
- ☐ Penghitung OK, FAIL, dan Total retry terpakai pada kedua node diamati sejak baris pertama.

11.8 Percobaan

11.8.1 EXP-01 — Retry Sehat, Jarak Dekat

Nyalakan ketiga board dan amati sepuluh siklus pertama. Pada jarak dekat, sebagian besar siklus seharusnya berhasil tanpa retry sama sekali.

Expected output — node

```

=== LoRa ALOHA+ACK+RETRY - NODE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
ACK timeout: 2000 ms | Max retry: 3 | Backoff: 200-1500 ms
Peran: NODE (ALOHA + ACK + Random Backoff + Retry)

[TX] NODE=1,SEQ=0,R1T=28.4,R1H=63,R2T=24.7,R2H=71
[OK] SUCCESS setelah 0 retry | OK: 1 | FAIL: 0 | Total retry terpakai: 0

```

Expected output — gateway

```

=== PAKET DITERIMA (DATA BARU) ===
Node      : 1
SEQ       : 0
Ruang 1   : 28.4 C, 63 %
Ruang 2   : 24.7 C, 71 %
RSSI      : -46 dBm
SNR       : 9.75 dB
Statistik Node 1: baru=1 | duplicate=0 | gagal permanen (est.)=0
[TX] ACK=1,SEQ=0
=====

```

Data capture — diukur 90 detik (bukan 10 siklus; lihat `logserial.md`).

Parameter	Hasil
OK/FAIL Node 1 (23 siklus)	23 / 0
OK/FAIL Node 2 (23 siklus)	23 / 0
Rata-rata retry per siklus sukses	Node 1: 3 retry / 23 siklus $\approx$ 0,13 — Node 2: 2 retry / 23 siklus $\approx$ 0,09
duplicate yang tercatat di gateway	Node 1: 1 — Node 2: 1

Tabel 11.5: Hasil pengamatan EXP-01 pada sesi verifikasi perangkat

CHECKPOINT

Terpenuhi. Sebagian besar siklus (21/23 Node 1, 22/23 Node 2) sukses langsung dengan SUCCESS setelah 0 retry; hanya satu siklus per node yang butuh retry ($2\times$ dan $1\times$ untuk Node 1, $2\times$ untuk Node 2) pada jarak dekat.

11.8.2 EXP-02 — Memaksa Retry: M08B vs M08C pada Interval Sama

Persempit interval kirim kedua node (300–500 ms, seperti EXP-02 M08) supaya tabrakan lebih sering terjadi, lalu bandingkan hasil M08B dan M08C pada kondisi yang sama.

Data capture

Parameter	M08B (tanpa retry)	M08C (dengan retry)
Tingkat kegagalan per node (%)	FAIL / (OK+FAIL)	[GAP] permanen / total SEQ
Rata-rata retry terpakai per siklus	—	
Total lalu lintas ACK tambahan	—	

Tabel 11.6: Lembar pengamatan EXP-02 — M08B dibanding M08C

BUKA ABSTRAKSINYA

Di `src/node/main.cpp`, nilai `r1t`, `r1h`, `r2t`, `r2h` dibangkitkan **sekali** di awal `loop()`, sebelum lingkaran `while(true)` retry dimulai. Jelaskan mengapa data itu **tidak** boleh dibangkitkan ulang di dalam lingkaran retry, dan telusuri: apa yang akan rusak di sisi gateway bila data dibangkitkan ulang setiap percobaan? Petunjuk: gateway hanya mencetak isi ruangan untuk data BARU, tidak untuk duplicate.

CHECKPOINT

Tingkat kegagalan permanen M08C harus jauh lebih rendah daripada tingkat [FAIL] M08B pada interval kirim yang sama. Bila keduanya mirip, periksa apakah `MAX_RETRIES` atau rentang backoff sudah benar ter-flash.

11.8.3 EXP-03 — Duplicate Betul-Betul Tidak Dihitung Ganda

Amati log gateway secara khusus mencari baris `PAKET DITERIMA (DUPLICATE)`.

Data capture

Parameter	Hasil
Jumlah (DUPLICATE) per node dalam 5 menit	
Apakah baru= di statistik gateway ikut bertambah saat duplicate diterima?	
Apakah gateway tetap mencetak [TX] ACK=... untuk duplicate?	

Tabel 11.7: Lembar pengamatan EXP-03 — deteksi duplicate

CHECKPOINT

baru= pada statistik gateway **tidak boleh** bertambah ketika (DUPLICATE) dicetak. Bila ikut bertambah, periksa kembali syarat `seq <= lastSeq[nodeId]` pada perbandingan SEQ di gateway/main.cpp.

11.8.4 Verifikasi Perangkat Keras**CATATAN**

Diuji di perangkat pada 22 Agustus 2026 — tiga Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway, node1, node2; port /dev/ttyACM0/1/2). Build dan upload ketiga environment sukses tanpa modifikasi kode. EXP-01 dijalankan (90 detik) dan datanya nyata, termasuk siklus yang benar-benar butuh retry dan backoff untuk sukses — lihat `logserial.md`, yang juga memuat perbandingan langsung M08B vs M08C pada interval bawaan yang sama (retry menurunkan kegagalan permanen dari 4 kejadian menjadi 0). EXP-02 versi modul (interval dipersempit 300–500 ms) dan pengukuran EXP-03 selama 5 menit penuh **belum dijalankan** pada sesi verifikasi ini, diserahkan sebagai latihan praktikum.

11.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, ACK_TIMEOUT 2000 ms, MAX_RETRIES 3, backoff acak 200–1500 ms. Posisi board sama seperti M08/M08B. Interval kirim bawaan (2000–5000 ms acak), tidak diubah.

Board & Port

Peran	Environment	Port	Board
Gateway	gateway	/dev/ttyACM0	Uno asli (2341:0043)
Node 1	node1	/dev/ttyACM1	Uno asli (2341:0043)
Node 2	node2	/dev/ttyACM2	Uno asli (2341:0043)

Tabel 11.8: Log serial Modul 08C: Board & Port

Ketiga aliran serial direkam bersamaan, jendela rekam 90 detik setelah ketiga board di-*upload* ulang dan *boot* bersih.

EXP-01 — Retry Sehat, Jarak Dekat

Siklus normal (langsung sukses, 0 retry):

```
[08:06:27.884] GW | === PAKET DITERIMA (DATA BARU) ===
[08:06:27.888] GW | Node      : 2
[08:06:27.888] GW | SEQ       : 1
```

Dan siklus yang butuh retry — Node 1 mengirim SEQ=0, ACK pertama tidak sampai, node menunggu *backoff* acak lalu mengirim ulang **dua kali** sebelum akhirnya sukses:

```
[08:06:21.265] N1 | [BACKOFF] tunggu 854 ms sebelum retry
[08:06:22.125] N1 | [RETRY 1/3] NODE=1,SEQ=0,R1T=29.9,R1H=72,R2T=26.7,R2H=58
[08:06:24.210] N1 | [BACKOFF] tunggu 1358 ms sebelum retry
[08:06:25.573] N1 | [RETRY 2/3] NODE=1,SEQ=0,R1T=29.9,R1H=72,R2T=26.7,R2H=58
[08:06:25.725] N1 | [OK] SUCCESS setelah 2 retry | OK: 1 | FAIL: 0 | Total retry terpakai: 2
[08:06:25.664] GW | === PAKET DITERIMA (DATA BARU) ===
[08:06:25.665] GW | Node      : 1
[08:06:25.668] GW | SEQ       : 0
```

Perhatikan: R1T=29.9, R1H=72, R2T=26.7, R2H=58 **identik** di ketiga percobaan (TX awal + 2 retry) — data dummy dibangkitkan sekali per SEQ dan dipertahankan sepanjang retry, persis seperti dijelaskan di kode.

Parameter	Hasil (90 detik, bukan 10 siklus)
OK/FAIL Node 1 (23 siklus)	23 / 0 — semua siklus akhirnya sukses
OK/FAIL Node 2 (23 siklus)	23 / 0 — semua siklus akhirnya sukses
Total retry terpakai — Node 1 / Node 2	3 / 2
duplicate tercatat di gateway — Node 1 / Node 2	1 / 1

Tabel 11.9: Log serial Modul 08C: EXP-01 — Retry Sehat, Jarak Dekat

CHECKPOINT terpenuhi. Sebagian besar siklus (21/23 Node 1, 22/23 Node 2) sukses langsung dengan SUCCESS setelah 0 retry. Hanya satu siklus per node yang butuh retry pada sesi ini.

Temuan Tambahan — Perbandingan Langsung dengan M08B (Tanpa vs Dengan Retry)

(Bukan EXP-02 versi README — itu menuntut interval dipersempit 300–500 ms, belum dijalankan. Ini perbandingan pada interval bawaan yang sama, memakai data EXP-01 M08B dan M08C.)

Kedua modul diuji pada interval kirim, jarak, dan posisi board yang **sama** (90 detik, 2000–5000 ms):

Parameter	M08B (tanpa retry)	M08C (dengan retry)
FAIL permanen Node 1 (90 detik)	2	0

Parameter	M08B (tanpa retry)	M08C (dengan retry)
FAIL permanen Node 2 (90 detik)	2	0
Mekanisme pemulihan saat ACK hilang	tidak ada — SEQ lanjut, data hilang permanen	retry otomatis dengan backoff acak, hingga 3×

Tabel 11.10: Log serial Modul 08C: Temuan Tambahan — Perbandingan Langsung dengan M08B (Tanpa vs Dengan Retry)

Retry+backoff pada M08C berhasil **memulihkan seluruh kegagalan** yang pada M08B akan tercatat sebagai [FAIL] permanen — pada sesi ini, tingkat kegagalan permanen turun dari 4 kejadian (M08B, gabungan kedua node) menjadi 0 (M08C). Ini bukti langsung nilai tambah retry dibanding sekadar mengetahui kegagalan (M08B).

Temuan Tambahan — Duplicate Tidak Dihitung Ganda

(Bagian dari EXP-03 versi README — belum diamati sepenuhnya selama 5 menit penuh, tapi datanya konsisten pada jendela 90 detik yang diuji.)

```
[08:07:54.477] GW | Statistik Node 1: baru=25 | duplicate=1 | gagal permanen (est.)=0
```

Parameter	Hasil
Data baru — Node 1 / Node 2 (dari sisi gateway)	25 / 24
Duplicate — Node 1 / Node 2	1 / 1
Apakah baru= ikut bertambah saat duplicate diterima?	tidak — hanya duplicate= yang bertambah
Apakah gateway tetap membalas ACK untuk duplicate?	ya (mengikuti kode; tidak diverifikasi eksplisit lewat log RSSI karena baris duplicate tidak mencetak RSSI/SNR)

Tabel 11.11: Log serial Modul 08C: Temuan Tambahan — Duplicate Tidak Dihitung Ganda

Catatan: baru gateway (25/24) sedikit lebih tinggi daripada jumlah siklus OK di node (23/23) karena jendela rekam masing-masing dari tiga proses capture terpisah tidak mulai/berhenti pada detik yang identik persis — bukan indikasi bug penghitungan.

Ringkasan Verifikasi Hardware

Diuji di perangkat pada 2026-08-22: 3× Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway + node1 + node2, port /dev/ttyACM0/1/2). Build dan upload ketiga environment sukses. Protokol retry+backoff berjalan sesuai desain: node mengulang paket yang sama (data dummy tidak berubah) hingga 3× dengan jeda backoff acak sebelum menyerah, gateway mengenali retry sebagai duplicate lewat `SEQ <= lastSeq` dan tidak menghitungnya sebagai data baru namun tetap membalas ACK. Pada sesi 90 detik ini, retry berhasil memulihkan **seluruh** kegagalan yang di M08B akan permanen (FAIL turun dari 4 menjadi 0). EXP-02 versi README (interval dipersempit) dan pengukuran 5 menit penuh untuk EXP-03 **belum dijalankan** pada sesi ini — diserahkan sebagai latihan praktikum.

Catatan teknis capture — dua kali percobaan upload gateway pada sesi ini sempat menghasilkan output serial yang rusak (byte non-ASCII, bukan baris log yang valid) segera setelah avrdude selesai; unggah ulang tanpa mengubah kode langsung memulihkannya. Ini artefak transisi USB-CDC saat board baru selesai diprogram, bukan bug firmware — bila mengalami ini, unggah ulang environment yang bersangkutan sebelum menyalahkan kode.

11.9 Pengukuran

A. Distribusi jumlah retry per siklus sukses (90 detik, interval bawaan)

Retry terpakai	Siklus — Node 1	Siklus — Node 2
0 (langsung sukses)	21	22
1	1	0
2	1	1
3 (batas; tetap gagal bila ini pun tak cukup)	0	0

Tabel 11.12: Lembar pengukuran A — distribusi retry per siklus sukses

B. M08B dibanding M08C pada interval kirim sama (90 detik, jarak dan posisi board identik)

Interval (ms)	FAIL M08B (N1+N2)	Gagal permanen M08C	Rata-rata retry M08C
2000–5000 (bawaan)	4	0	N1: 0,13/siklus — N2: 0,09/siklus
1000–2000	<i>belum diuji</i>		
300–500	<i>belum diuji — jalankan EXP-02</i>		

Tabel 11.13: Lembar pengukuran B — M08B dibanding M08C

C. Duplicate dibanding data baru (dari sisi gateway, 90 detik)

Node	Data baru	Duplicate	Rasio duplicate/baru (%)
Node 1	25	1	4,0
Node 2	24	1	4,2

Tabel 11.14: Lembar pengukuran C — duplicate dibanding data baru

11.10 Analisis

1. Dari Tabel 11.12, berapa persen siklus yang sukses tanpa retry sama sekali? Bandingkan dengan proporsi yang membutuhkan retry pada interval kirim bawaan.
2. Dari Tabel 11.13, seberapa besar penurunan tingkat kegagalan dari M08B ke M08C pada interval yang sama? Jelaskan mengapa retry tidak pernah membuat kegagalan menjadi nol, hanya menurunkannya.
3. Dari Tabel 11.14, jelaskan mengapa rasio duplicate terhadap data baru naik seiring interval kirim yang dipersempit — kaitkan dengan peluang tabrakan yang lebih tinggi.
4. Hitung tambahan waktu rata-rata yang dibayar per siklus akibat retry (waktu tunggu ACK yang gagal + backoff + kirim ulang), lalu bandingkan dengan waktu siklus sukses tanpa retry.
5. Rancang, tanpa menulis kodenya, skenario ketika `MAX_RETRIES = 3` masih tidak cukup untuk mencapai tingkat keberhasilan yang dapat diterima. Solusi apa dari M10 (bukan menambah `MAX_RETRIES`) yang menyelesaikan akar masalahnya, bukan sekadar mencoba lebih banyak?

11.11 Concept Check

1. Mengapa data dummy (`r1t`, `r1h`, `r2t`, `r2h`) dibangkitkan sekali per SEQ, bukan setiap kali retry?

2. Apa yang salah bila backoff antar-retry memakai jeda **tetap**, bukan acak?
3. Bagaimana gateway membedakan paket duplicate dari data baru, dan mengapa perbandingannya $\leq$, bukan $=$ saja?
4. Mengapa gateway tetap mengirim ACK untuk paket yang dikenali sebagai duplicate?
5. Apa perbedaan antara “gagal” pada M08B dan “gagal permanen” pada M08C?

11.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Backoff yang lebih pintar

CH-1 — Exponential backoff Ganti rentang backoff acak seragam dengan backoff yang jangkauannya melebar tiap percobaan (misalnya percobaan ke- n memakai $\text{random}(0, \text{BACKOFF_MIN_MS} * 2^n)$), lalu bandingkan tingkat keberhasilan dan rata-rata retry dengan versi seragam pada interval kirim yang sama.

CH-2 — Retry adaptif Sesuaikan `MAX_RETRIES` secara otomatis berdasarkan tingkat keberhasilan berjalan (naikkan bila FAIL sering, turunkan bila selalu SUCCESS setelah 0 retry), dan jelaskan trade-off latensi terhadap keandalan yang muncul.

Level 3 — Tantangan · Mengukur ongkos radio sesungguhnya

CH-3 — Statistik gap sesungguhnya Tambahkan penghitung total kirim (asli 1. retry) yang benar-benar mengudara per node di gateway (bukan hanya data baru), lalu hitung *airtime utilization* kanal untuk membandingkan ongkos radio M08B dengan M08C.

CH-4 — Tiga node dengan retry Tambahkan environment node3, sesuaikan `NODE_COUNT` di gateway, dan amati apakah rata-rata retry per siklus naik signifikan ketika jumlah node bertambah pada interval kirim yang sama.

11.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — retry, random backoff, gagal permanen, duplicate.
3. Konfigurasi — `MAX_RETRIES`, rentang backoff, `ACK_TIMEOUT`, kontrak `NODE=` dan `ACK=` (tak berubah dari M08B).
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 11.12, Tabel 11.13, dan Tabel 11.14.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri, khususnya mengenai batas kemampuan retry dan backoff sebelum penjadwalan slot (M10) benar-benar diperlukan.

12. Modul 09 — CSMA/CA: Dengar Dulu, Baru Bicara

LoRa COMMUNICATION LAB · MODUL 09

Listen Before You Talk — CSMA/CA

Arduino Uno + Dragino LoRa Shield v1.2 · topologi bintang, 2 node + gateway · carrier sense + backoff · level Advanced · 1 × 50 menit · folder kode Modul09_loRa_csma_ca

PENGANTAR MODUL · Gambaran Umum

Modul 09 dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat lanjut, dan merupakan langkah keempat dalam menghadapi tabrakan. Tiga modul sebelumnya menanganinya **setelah** terjadi: M08 membiarkannya senyap, M08B membuatnya terlihat lewat ACK, M08C memulihkannya lewat retry. Modul ini yang pertama berusaha **menghindarinya sebelum terjadi** — node mendengarkan kanal lebih dahulu, dan hanya bicara kalau sedang sepi.

12.1 Pendahuluan

Gagasannya juga membongkar asumsi yang dipegang sejak M05: bahwa supaya banyak node bisa berbagi satu kanal, harus ada **satu pihak yang mengatur giliran**. M05 memakai master yang memanggil node satu per satu, M07 memindahkan master itu ke Raspberry Pi — keduanya bergantung pada koordinator pusat. Di sini koordinator itu tidak ada, dan penggantinya adalah **kesopanan yang dijalankan sendiri oleh tiap node**: dengarkan dulu, kalau ada yang sedang bicara, tunggu sebentar dengan lama tunggu acak, lalu coba lagi.

Topologinya tetap *many-to-one* seperti M05 — dua node sensor, satu gateway — dan justru kemiripan itulah yang membuat perbandingannya tajam. Yang hilang cuma satu: gateway tidak pernah lagi berkata “Node 1, sekarang giliranmu.” Ia hanya duduk mendengarkan. Pertanyaan modul ini: bisakah dua node yang tidak pernah saling berjanji tetap menghindari tabrakan, hanya bermodalkan telinga masing-masing?

Prasyaratnya M02 untuk pola interrupt DIO0 dan flag di sisi gateway, M04 untuk pembacaan RSSI per paket, M05 untuk gagasan pengalamatan node di lapisan aplikasi, dan M08 sebagai pembanding langsung — kanal yang sama tanpa carrier sense sama sekali. Yang dibangun di sini adalah *carrier sensing* pada SX1276 (dua cara: ambang RSSI dan CAD), *random exponential backoff* dengan pencacah yang dibekukan selagi kanal terpakai, serta pengukuran **tunda akses kanal** — besaran baru yang belum pernah muncul di modul mana pun sebelumnya.

PENTING

Dua hal sengaja dikembalikan ke keadaan paling polos di modul ini, dan itu keputusan metodologis. Pertama, **tidak ada ACK** — padahal M08B dan M08C sudah memilikinya. Kedua, **payload-nya lebih sederhana**: `NODE=<id>, SEQ=<n>, T=. ., H=. .`, satu sensor per node, bukan format dua ruangan `R1T/R1H/R2T/R2H` yang dipakai arc M08. Alasannya sama untuk keduanya: modul ini mengukur **satu variabel saja**, yaitu efek mendengarkan kanal sebelum bicara. Kalau ACK dan retry ikut menyala, kegagalan yang tersisa akan tertutup oleh pemulihan, dan angka yang terbaca bukan lagi angka carrier sense. Karena itu pembandingnya bukan M08C, melainkan **M08** — dan modul ini bahkan menyediakan pembanding itu di dalam dirinya sendiri lewat `CS_MODE=2` (“telinga dimatikan”), sehingga kedua kondisi dapat diukur pada perangkat, ruangan, dan sesi yang sama.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
05	Tabrakan dicegah lewat polling terpusat — master memanggil satu per satu
08	Penjadwalan dilepas — node kirim bebas, tabrakan senyap diamati
08B	ACK ditempelkan di atas M08 — node tahu SUCCESS/FAILED, belum ada retry
08C	Random backoff + retry — kegagalan dipulihkan, dan Pure ALOHA menjadi lengkap
09 (ini)	Carrier sense — dengar dulu sebelum bicara, tabrakan dihindari sebelum terjadi
10	SYNC + slot waktu — Slotted ALOHA (slot diundi) vs TDMA (slot tetap)

Tabel 12.1: Peta modul pada arc kedua seri LoRa

Kontrak data lab ini. Tiap node membawa **satu** sensor suhu dan kelembaban dan mengirimkannya sebagai `NODE=<id>,SEQ=<n>,T=<suhu>,H=<lembab>`. Bentuk `KEY=VALUE` dipisah koma dipilih alih-alih CSV posisional (`NODE1,TEMP,28.5,HUM,70.2`) karena satu alasan praktis: penerima tidak perlu tahu urutan field, cukup mencari kuncinya dengan `indexOf` dan `substring` — dan bila kelak ada field tambahan (RSSI, tegangan baterai), parser lama tetap jalan tanpa diubah. Identitas sumber ada di field `NODE`, sehingga gateway langsung tahu data ini milik siapa tanpa perlu bertanya. Nomor urut `SEQ` naik untuk **setiap data yang dihasilkan**, termasuk data yang akhirnya dibuang node karena kanal tak kunjung bebas — dengan begitu lubang pada `SEQ` di sisi gateway ikut merekam paket yang bahkan tidak pernah naik ke udara.

12.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menghindari tabrakan tanpa koordinator pusat

1. Menjelaskan tiga bagian CSMA/CA (*carrier sense, multiple access, collision avoidance*) dan menunjukkan baris kode yang mewujudkan masing-masing.
2. Melakukan *carrier sensing* pada SX1276 dengan dua cara — ambang RSSI dan CAD (*Channel Activity Detection*) — serta menjelaskan kapan keduanya memberi jawaban berbeda.
3. Menentukan ambang RSSI kanal-sibuk secara empiris dari pengukuran lantai derau dan kekuatan sinyal node tetangga, bukan menebak angkanya.
4. Menjelaskan mengapa *backoff* harus **acak** dan mengapa jendela kontensi (CW) harus **melebar** setiap kali kanal ditemukan sibuk.
5. Mengukur harga yang dibayar CSMA/CA — tunda akses kanal dan paket yang dibuang — lalu membandingkannya dengan kirim-buta tanpa carrier sense.

Kriteria keberhasilan

- ☐ Kedua node mengirim data suhu dan kelembaban secara mandiri, tanpa perintah apa pun dari gateway.
- ☐ Gateway mencetak setiap paket lengkap dengan identitas node, RSSI, SNR, dan selang dari paket sebelumnya.
- ☐ Serial monitor node menampilkan [CS] kanal SIBUK dan [BACKOFF] ketika node lain sedang mengirim — bukti telinganya benar-benar bekerja.
- ☐ Ketika interval kirim dipersempit, jumlah [BACKOFF], [FREEZE], dan tunda akses naik, tetapi paket yang tiba di gateway tetap utuh terbaca.

12.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Carrier Sense (CS)	Memeriksa kanal sebelum mengirim: adakah orang lain sedang memakai udara di frekuensi ini?
Multiple Access (MA)	Banyak node berbagi satu frekuensi yang sama, tanpa pembagian waktu atau kanal dari pusat.
Collision Avoidance (CA)	Bila kanal sibuk: jangan langsung kirim begitu sepi, tetapi mundur selama waktu acak lebih dulu.
DIFS	Rentang waktu kanal harus terbukti bebas terus-menerus sebelum node berhak mengirim. Satu sampel bebas tidak cukup.
Slot dan Contention Window (CW)	Backoff dihitung dalam satuan slot. Node mengundi angka $0 \dots CW - 1$; makin sering kanal ditemukan sibuk, makin lebar CW.
Freeze	Pencacah backoff berhenti berkurang selama ada yang mengirim, lalu lanjut lagi saat sepi — supaya yang sudah lama antre tidak kehilangan gilirannya.
RSSI sensing	Kanal dianggap terpakai bila kekuatan sinyal terbaca melewati ambang. Murah, tetapi ikut menghitung derau dan sinyal non-LoRa.
CAD	SX1276 mengkorelasikan simbol LoRa; ia mengenali sinyal LoRa bahkan yang tenggelam di bawah lantai derau, dan mengabaikan gangguan yang bukan LoRa.
Hidden node	Dua node yang saling tidak terdengar tetap dapat menabrak di gateway, walau keduanya patuh mendengarkan.
Tunda akses kanal	Selisih waktu antara “data siap” dan “paket benar-benar naik ke udara” — harga yang dibayar demi kesopanan.

Tabel 12.2: Istilah kerja Modul 09

Mengapa CA, bukan CD. Ethernet klasik memakai CSMA/CD — *collision detection* — karena kabel memungkinkan pengirim mendengarkan jalurnya sendiri sambil mengirim, dan langsung berhenti ketika dua sinyal beradu. Radio tidak bisa begitu: pemancar SX1276 membutuhkan penerimanya sendiri, sehingga selama TX berlangsung node **buta total** terhadap keadaan kanal. Karena tabrakan tidak mungkin dideteksi di tengah jalan, satu-satunya strategi yang tersisa adalah **menghindarinya sebelum berangkat**. Itulah sebabnya semua protokol radio berbagi kanal — Wi-Fi, Zigbee, LoRaWAN kelas tertentu — memakai CA, bukan CD.

Mengapa backoff harus acak. Bayangkan backoff bernilai tetap. Dua node yang sama-sama menunggu kanal yang sedang dipakai akan selesai menunggu pada saat yang sama persis, lalu mengirim bersamaan — tabrakan yang justru **diciptakan** oleh mekanisme penghindarnya sendiri. Angka acak memecah simetri itu. Dan ketika kanal berkali-kali ditemukan sibuk (tanda peserta makin banyak atau makin ramai), rentang undiannya dilebarkan supaya peluang dua node mengundi angka yang sama makin kecil — inilah *exponential backoff*.

Batas yang tetap ada. CSMA/CA mengurangi tabrakan, tidak menghapusnya. Dua node yang menyelesaikan DIFS pada milidetik yang sama tetap akan berangkat bersamaan; node yang mulai mengirim tepat setelah tetangganya selesai menyensor juga tidak akan terdeteksi. Dan karena modul ini belum punya ACK, tabrakan yang tetap terjadi tetap **senyap** — node mengira pakatnya berhasil karena kanal terdengar sepi saat ia berangkat.

Sekuens yang diamati



12.4 Topologi



Node	Environment	Peran	Mekanisme TX/RX	Interval kirim
Node 1	node1	Baca sensor, sensing kanal, backoff, kirim	TX blocking; RX kontinu hanya untuk mendengar kanal	acak 2000–5000 ms
Node 2	node2	Sama persis, keputusannya independen	TX blocking; RX kontinu hanya untuk mendengar kanal	acak 2000–5000 ms
Gateway	gateway	Terima, catat, deteksi lubang SEQ	Interrupt DIO0 + flag, tanpa TX sama sekali	—

Tabel 12.3: Peran tiap node Modul 09

Karena gateway tidak pernah memancar, ia tidak pernah ikut membuat kanal sibuk. Setiap kali sebuah node melaporkan [CS] kanal SIBUK, yang ia dengar dipastikan node satunya — bukan gema dari gateway.

12.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3

No	Peralatan	Spesifikasi	Jumlah
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3

Tabel 12.4: Alat dan bahan Modul 09

Sensor suhu dan kelembaban. Nilai T dan H dibangkitkan di dalam firmware (dummy, berbeda titik dasar untuk tiap node) supaya modul bisa dijalankan tanpa sensor fisik — fokus modul ini ada pada akses kanal, bukan pembacaan sensor. Untuk memakai DHT22 sungguhan, satu-satunya yang perlu diubah adalah isi fungsi `bacaSensor()` di `src/node/main.cpp`; tambahkan library-nya ke `lib_deps` dan pasang pin datanya ke pin digital yang **tidak dipakai shield** (D10, D11, D12, D13, D9, dan D2 sudah terpakai SPI, RST, dan DIO0 — gunakan D3–D8). Tidak ada baris lain yang perlu disentuh, sebab seluruh mekanisme CSMA/CA tidak peduli dari mana angkanya datang.

Pemetaan pin Dragino Shield v1.2

Fungsi	Pin Uno
NSS / CS	D10
RST	D9
DIO0	D2
SCK / MOSI / MISO	D13 / D11 / D12

Tabel 12.5: Pemetaan pin Modul 09

Struktur proyek

```

Modul09_lora_csma_ca/
├── platformio.ini      ← tiga environment; CS_MODE memilih RSSI atau CAD
├── lora_monitor.py     ← dashboard 3-panel live + statistik akses kanal + CSV
├── upload_auto.py      ← unggah ketiga board, port dideteksi sendiri
├── logserial.md        ← log serial aktual dari pengujian perangkat
└── src/
    ├── node/main.cpp   ← sensing + backoff + TX (env node1, node2)
    └── gateway/main.cpp ← terima & cetak, deteksi lubang SEQ (env gateway)

```

Parameter CSMA/CA di `src/node/main.cpp`

Konstanta	Nilai bawaan	Arti
RSSI_THRESHOLD	– 95 dBm	Di atas ini kanal dianggap terpakai. Dikalibrasi di EXP-01.
DIFS_MS	30 ms	Kanal harus bebas selama ini sebelum boleh mengirim
SLOT_MS	20 ms	Panjang satu slot backoff
CW_MIN / CW_MAX	4 / 64	Jendela kontensi awal dan batas atasnya
MAX_ATTEMPT	5	Sesudah ini paket dibuang ([DROP]), tidak dipaksakan
CS_MODE	0 (RSSI)	0 = RSSI, 1 = CAD, 2 = telinga dimatikan (pembanding EXP-04) — diatur dari <code>platformio.ini</code>
SEND_INTERVAL_MIN / MAX	2000 / 5000 ms	Jarak antar data. Bisa ditimpa dari <code>platformio.ini</code> untuk EXP-03

Tabel 12.6: Parameter CSMA/CA Modul 09

Waktu udara satu paket modul ini (≈ 26 byte, SF7, BW 125 kHz, CR 4/5, preamble 8) sekitar **60 ms** menurut perhitungan — hitung sendiri angka pastinya sebagai bagian dari Analisis. Seluruh angka di tabel di atas dipilih relatif terhadap angka itu.

12.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul09_lora_csma_ca/](#)

- [Modul09_lora_csma_ca/platformio.ini](#)
- [Modul09_lora_csma_ca/src/node/main.cpp](#)
- [Modul09_lora_csma_ca/src/gateway/main.cpp](#)
- [Modul09_lora_csma_ca/lora_monitor.py](#)
- [Modul09_lora_csma_ca/upload_auto.py](#)

```
[platformio]
default_envs = gateway

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 115200

[env:gateway]
build_src_filter = +<gateway/*.cpp>
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua node memakai source yang sama; nomornya ditentukan build flag.
;
; CS_MODE menentukan CARA node mendengarkan kanal sebelum mengirim:
; -DCS_MODE=0 -> carrier sense berbasis RSSI (bawaan, paling mudah diamati)
; -DCS_MODE=1 -> carrier sense berbasis CAD (Channel Activity Detection)
; -DCS_MODE=2 -> telinga dimatikan, node kirim buta (pembanding Pure ALOHA)
; Ketiganya ada di src/node/main.cpp. EXP-04 membandingkan mode 0 dengan mode 2,
; EXP-05 membandingkan mode 0 dengan mode 1: ganti angkanya di KEDUA node,
; unggah ulang, lalu bandingkan hasilnya.
;
; Interval kirim juga bisa ditimpa dari sini untuk EXP-03, tanpa mengedit source:
; -DSEND_INTERVAL_MIN=300 -DSEND_INTERVAL_MAX=500

[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=1 -DCS_MODE=0
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=2 -DCS_MODE=0
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2
```

Kode 12.1: platformio.ini Modul 09 — CS_MODE dan interval kirim lewat build flag

```
/*
LoRa CSMA/CA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: node1 / node2

Node membawa satu sensor suhu & kelembaban dan mengirim datanya ke gateway
KAPAN SAJA ia mau -- tidak ada master, tidak ada polling, tidak ada giliran
yang dibagikan siapa pun. Satu-satunya aturan yang dipatuhi node adalah
aturan yang ia jalankan sendiri: DENGARKAN KANAL DULU, dan kalau ada yang
sedang memakai udara, MUNDUR SEJENAK secara acak sebelum mencoba lagi.
```

```

Itulah CSMA/CA:
CS (Carrier Sense)    -> radio masuk mode dengar, kanal diperiksa
MA (Multiple Access) -> banyak node berbagi satu frekuensi
CA (Collision Avoidance) -> kalau sibuk: backoff acak, bukan langsung kirim

Belum ada ACK pada modul ini. Node tahu kanal sedang sepi saat ia mengirim,
tetapi tetap tidak tahu apakah pakatnya benar-benar sampai -- itu urusan
modul berikutnya.

Payload  : "NODE=<id>,SEQ=<n>,T=<suhu>,H=<lembab>"
Balasan  : tidak ada

Dua cara mendengar kanal, dipilih lewat build flag CS_MODE:
CS_MODE=0  RSSI -> radio ditahan di RX kontinu, RegRssiValue dibaca
             berulang; kanal dianggap terpakai bila RSSI melewati
             ambang RSSI_THRESHOLD.
CS_MODE=1  CAD  -> SX1276 disuruh masuk mode Channel Activity Detection,
             yang mengkorelasikan simbol LoRa dan bisa mengenali
             sinyal LoRa bahkan di bawah lantai derau. Ditulis
             dengan akses register langsung (lihat M06), sebab
             library sandeepmistry tidak menyediakan CAD.

Mekanisme:
TX  : blocking (endPacket), hanya setelah kanal dinyatakan bebas
RX  : tidak ada callback -- node ini murni pengirim. Radio tetap
      ditahan di mode RX bukan untuk membaca paket, melainkan supaya
      penerimanya hidup dan kanal bisa didengar.

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN      10
#define RST_PIN      9
#define DI00_PIN     2
#define LED_PIN      LED_BUILTIN

#define FREQUENCY     433E6
#define BANDWIDTH     125E3
#define SPREADING_FACTOR 7
#define CODING_RATE    5
#define TX_POWER      17

// Nomor node datang dari build flag (-DNODE_ID=1 atau 2) di platformio.ini,
// sehingga kedua node memakai file source yang sama persis.
#ifndef NODE_ID
#define NODE_ID 1
#endif

// Cara mendengar kanal, diatur dari platformio.ini:
// 0 = RSSI, 1 = CAD, 2 = telinga dimatikan (pembanding Pure ALOHA, EXP-04)
#ifndef CS_MODE
#define CS_MODE 0
#endif

// -----
// Parameter CSMA/CA
// -----
// Waktu udara satu paket modul ini (SF7, BW125 kHz, CR4/5, ~26 byte) sekitar
// 60-70 ms. Seluruh angka di bawah dipilih relatif terhadap angka itu:
// DIFS cukup panjang untuk mengambil beberapa sampel, satu slot backoff cukup
// panjang untuk membedakan kanal yang benar-benar bebas dari sela antar-simbol.
#define RSSI_THRESHOLD -95 // dBm; di atas nilai ini kanal dianggap terpakai
#define RSSI_SAMPLE_MS 2 // jarak antar sampel RSSI
#define DIFS_MS 30 // kanal harus bebas selama ini sebelum boleh TX
#define SLOT_MS 20 // panjang satu slot backoff
#define CW_MIN 4 // contention window awal (jumlah slot)
#define CW_MAX 64 // batas atas contention window
#define MAX_ATTEMPT 5 // sesudah ini paket dibuang, tidak dipaksakan
#define FREEZE_TIMEOUT 2000 // batas menunggu kanal bebas saat backoff beku
#define CAD_TIMEOUT_MS 50 // jaring pengaman bila CadDone tidak pernah naik

// Bisa ditimpa dari platformio.ini (-DSEND_INTERVAL_MIN=300 dst.) supaya
// EXP-03 cukup mengganti build flag, tanpa mengedit file ini.
#ifndef SEND_INTERVAL_MIN
#define SEND_INTERVAL_MIN 2000
#endif

```



```

#ifndef SEND_INTERVAL_MAX
#define SEND_INTERVAL_MAX 5000
#endif

// Nilai dasar sensor dibedakan antar node supaya mudah dikenali di gateway.
#if NODE_ID == 1
#define TEMP_BASE      28.0
#define HUM_BASE       70.0
#else
#define TEMP_BASE      29.0
#define HUM_BASE       68.0
#endif

// --- Register SX1276 yang dipakai mode CAD (datasheet SX1276/77/78/79) ---
#define REG_OP_MODE      0x01
#define REG_IRQ_FLAGS    0x12
#define MODE_LONG_RANGE_MODE 0x80
#define MODE_CAD         0x07
#define IRQ_CAD_DONE_MASK 0x04
#define IRQ_CAD_DETECTED_MASK 0x01

unsigned long seq        = 0;
unsigned long txCount    = 0; // paket yang benar-benar naik ke udara
unsigned long dropCount  = 0; // paket yang dibuang karena kanal tak kunjung bebas
unsigned long busyCount  = 0; // berapa kali kanal ketahuan sedang terpakai
unsigned long delaySum   = 0; // total tunda akses kanal (ms), untuk rata-rata
int lastSenseRssi       = 0; // RSSI terakhir yang terbaca saat menyensor

// -----
// Akses register langsung -- dipakai mode CAD saja
// -----
// Bus SPI yang sama dipakai bergantian dengan library. Aman karena node ini
// tidak memasang interrupt DI00 sama sekali, jadi tidak ada pembacaan register
// yang menyela di tengah transaksi ini.
uint8_t regBaca(uint8_t alamat) {
    SPI.beginTransaction(SPI_Settings(8E6, MSBFIRST, SPI_MODE0));
    digitalWrite(NSS_PIN, LOW);
    SPI.transfer(alamat & 0x7F); // bit 7 = 0 -> baca
    uint8_t nilai = SPI.transfer(0x00);
    digitalWrite(NSS_PIN, HIGH);
    SPI.endTransaction();
    return nilai;
}

void regTulis(uint8_t alamat, uint8_t nilai) {
    SPI.beginTransaction(SPI_Settings(8E6, MSBFIRST, SPI_MODE0));
    digitalWrite(NSS_PIN, LOW);
    SPI.transfer(alamat | 0x80); // bit 7 = 1 -> tulis
    SPI.transfer(nilai);
    digitalWrite(NSS_PIN, HIGH);
    SPI.endTransaction();
}

// -----
// Carrier sense: satu pemeriksaan kanal
// -----
#if CS_MODE == 1
// CAD: radio diminta menganalisis kanal selama beberapa simbol, lalu melapor
// lewat dua flag -- CadDone (analisis selesai) dan CadDetected (ada sinyal
// LoRa di sana). Sesudah itu radio harus dikembalikan sendiri ke mode RX.
bool kanalTerpakai() {
    // CAD harus dimasuki dari STANDBY. Perintah CAD yang dikirim selagi modem
    // masih di RX kontinu tidak pernah dijalankan: CadDone tidak akan pernah
    // naik dan pemeriksaan ini hanya berakhir di timeout.
    LoRa.idle();
    regTulis(REG_IRQ_FLAGS, 0xFF); // bersihkan flag lama
    regTulis(REG_OP_MODE, MODE_LONG_RANGE_MODE | MODE_CAD); // mulai CAD

    unsigned long t0 = millis();
    uint8_t flags = 0;
    do {
        flags = regBaca(REG_IRQ_FLAGS);
    } while (!(flags & IRQ_CAD_DONE_MASK) && (millis() - t0 < CAD_TIMEOUT_MS));

    regTulis(REG_IRQ_FLAGS, 0xFF);
    LoRa.receive(); // kembali mendengar

    return (flags & IRQ_CAD_DETECTED_MASK) != 0;
}
#elseif CS_MODE == 2
// Telinga dimatikan -- node mengirim buta. Ini bukan CSMA/CA lagi melainkan
// Pure ALOHA, dan hanya ada di sini sebagai PEMBANDING untuk EXP-04: berapa

```

```

// sebenarnya harga yang dibayar carrier sense, dan apa yang dibeli dengannya.
bool kanalTerpakai() {
    return false;
}
#else
// RSSI: radio sudah ditahan di RX kontinu, jadi RegRssiValue selalu berisi
// kekuatan sinyal kanal saat ini. Tidak ada paket yang perlu didekode --
// yang diperlukan hanya jawaban "ada energi di frekuensi ini atau tidak".
bool kanalTerpakai() {
    lastSenseRssi = LoRa.rssi();
    return lastSenseRssi > RSSI_THRESHOLD;
}
#endif

// Kanal harus bebas SELAMA satu rentang penuh, bukan sekadar bebas pada satu
// sampel. Satu sampel saja terlalu mudah tertipu sela antar-simbol.
bool kanalBebasSelama(unsigned long durasiMs) {
    #if CS_MODE == 2
        (void)durasiMs; // tanpa carrier sense: berangkat saja, tanpa menunggu
        return true;
    #else
        unsigned long t0 = millis();
        while (millis() - t0 < durasiMs) {
            if (kanalTerpakai()) return false;
        }
    #if CS_MODE == 0
        delay(RSSI_SAMPLE_MS);
    #endif
    }
    return true;
}
#endif

void tungguKanalBebas() {
    unsigned long t0 = millis();
    while (kanalTerpakai() && (millis() - t0 < FREEZE_TIMEOUT)) {
        delay(RSSI_SAMPLE_MS);
    }
}

// -----
// Backoff: hitung mundur slot, DIBEKUKAN selagi kanal terpakai
// -----
// Inilah inti "CA". Pencacah tidak berkurang selama ada yang sedang mengirim,
// sehingga node yang sudah lama menunggu tidak kehilangan antreannya ketika
// tetangganya bicara panjang -- persis perilaku DCF pada 802.11.
void hitungMundurSlot(int slot) {
    while (slot > 0) {
        if (kanalBebasSelama(SLOT_MS)) {
            slot--;
        } else {
            Serial.println(F("[FREEZE] pencacah backoff dibekukan -- kanal terpakai"));
            tungguKanalBebas();
        }
    }
}

// -----
// Satu siklus pengiriman CSMA/CA
// -----
bool kirimCSMA(const String& payload) {
    unsigned long tSiap = millis(); // saat data siap, mulai menghitung tunda akses
    int cw = CW_MIN;

    for (int attempt = 1; attempt <= MAX_ATTEMPT; attempt++) {
        if (kanalBebasSelama(DIFS_MS)) {
            // Tunda akses dihitung SAMPAI SINI saja -- sampai kanal diperoleh.
            // Waktu udara paket bukan bagian dari tunda akses: itu harga transmisi,
            // bukan harga menunggu giliran.
            unsigned long tunda = millis() - tSiap;

            // Wajib sebelum beginPacket() bila CAD dipakai: isTransmitting() di
            // library menguji (RegOpMode & 0x03) == 0x03, dan mode CAD (0x07) lolos
            // uji itu. Bila modem kebetulan masih di CAD, beginPacket() diam-diam
            // gagal, FIFO tidak di-reset, dan yang naik ke udara adalah sampah.
            LoRa.idle();

            digitalWrite(LED_PIN, HIGH);
            LoRa.beginPacket();
            LoRa.print(payload);
            LoRa.endPacket(); // blocking sampai paket habis dikirim
            digitalWrite(LED_PIN, LOW);
            LoRa.receive(); // langsung mendengar lagi

```

```

    txCount++;
    delaySum += tunda;

    Serial.print(F("[TX] "));    Serial.print(payload);
    Serial.print(F(" | attempt=")); Serial.print(attempt);
    Serial.print(F(" | tunda="));  Serial.print(tunda);
    Serial.println(F(" ms"));
    return true;
}

    busyCount++;
    Serial.print(F("[CS] kanal SIBUK"));
    #if CS_MODE == 0
        Serial.print(F(" (RSSI ")); Serial.print(lastSenseRssi); Serial.print(F(" dBm"));
    #elif CS_MODE == 1
        Serial.print(F(" (CAD mendeteksi sinyal LoRa)"));
    #endif
    Serial.println();

    int slot = random(0, cw);    // 0 .. cw-1 slot
    Serial.print(F("[BACKOFF] percobaan ")); Serial.print(attempt);
    Serial.print(F("/"));        Serial.print(MAX_ATTEMPT);
    Serial.print(F(" | CW="));    Serial.print(cw);
    Serial.print(F(" | slot="));  Serial.print(slot);
    Serial.print(F(" -> "));      Serial.print((long)slot * SLOT_MS);
    Serial.println(F(" ms"));

    hitungMundurSlot(slot);

    cw *= 2;                      // contention window melebar tiap kegagalan
    if (cw > CW_MAX) cw = CW_MAX;
}

    dropCount++;
    Serial.print(F("[DROP] SEQ=")); Serial.print(seq);
    Serial.print(F(" dibuang -- kanal tetap sibuk setelah "));
    Serial.print(MAX_ATTEMPT); Serial.println(F(" percobaan"));
    return false;
}

// -----
// Sensor
// -----
// Data masih dibangkitkan (dummy) supaya modul bisa dijalankan tanpa sensor
// fisik. Untuk memakai DHT22 sungguhan, cukup GANTI ISI FUNGSI INI -- tidak
// ada baris lain di file ini yang perlu diubah, sebab seluruh mekanisme
// CSMA/CA tidak peduli dari mana angkanya datang.
void bacaSensor(float& suhu, float& lembab) {
    suhu = TEMP_BASE + (random(-30, 31) / 10.0);    // +-3.0 C
    lembab = HUM_BASE + (random(-50, 51) / 10.0);    // +-5.0 %
}

// -----
// Kalibrasi: seperti apa kanal ini ketika tidak ada siapa-siapa?
// -----
// Dipanggil sekali saat menyala, selagi node lain (mudah-mudahan) belum
// mengirim apa-apa. Angka inilah bahan EXP-01: ambang kanal-sibuk yang masuk
// akal ada di antara lantai derau ini dan RSSI node tetangga saat mengirim.
void laporLantaiDerau() {
    const int JUMLAH_SAMPEL = 200;
    int minimum = 0;
    int maksimum = -200;
    long jumlah = 0;

    for (int i = 0; i < JUMLAH_SAMPEL; i++) {
        int r = LoRa.rssi();
        if (r < minimum) minimum = r;
        if (r > maksimum) maksimum = r;
        jumlah += r;
        delay(5);
    }

    Serial.print(F("[KALIBRASI] lantai derau ")); Serial.print(JUMLAH_SAMPEL);
    Serial.print(F(" sampel: min ")); Serial.print(minimum);
    Serial.print(F(" | rata-rata ")); Serial.print((int)(jumlah / JUMLAH_SAMPEL));
    Serial.print(F(" | maks ")); Serial.print(maksimum);
    Serial.println(F(" dBm"));
    #if CS_MODE == 0
        Serial.print(F("[KALIBRASI] ambang terpakai sekarang: "));
        Serial.print(RSSI_THRESHOLD); Serial.println(F(" dBm -- lihat EXP-01"));
    #elif CS_MODE == 1

```

```

    Serial.println(F("[KALIBRASI] mode CAD: angka di atas hanya rujukan, CAD tidak memakai ambang"));
#else
    Serial.println(F("[KALIBRASI] carrier sense MATI: angka di atas tidak dipakai sama sekali"));
#endif
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    randomSeed(analogRead(A0) + NODE_ID);

    Serial.print(F("=== LoRa CSMA/CA - NODE "));
    Serial.print(NODE_ID);
    Serial.println(F(" ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);

    // Tidak ada LoRa.onReceive() di sini: node ini tidak pernah membaca paket.
    // Radio ditahan di RX kontinu semata-mata supaya kanal bisa didengar.
    LoRa.receive();

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("Carrier sense: "));
    #if CS_MODE == 0
        Serial.print(F("RSSI (ambang ")); Serial.print(RSSI_THRESHOLD); Serial.print(F(" dBm)"));
    #elif CS_MODE == 1
        Serial.print(F("CAD (Channel Activity Detection)"));
    #else
        Serial.print(F("MATI -- kirim buta (Pure ALOHA, pembandingan EXP-04)"));
    #endif
    Serial.print(F(" | DIFS ")); Serial.print(DIFS_MS);
    Serial.print(F(" ms | slot ")); Serial.print(SLOT_MS);
    Serial.print(F(" ms | CW ")); Serial.print(CW_MIN);
    Serial.print(F("..")); Serial.println(CW_MAX);
    Serial.println(F("Peran: NODE (CSMA/CA) -- dengar dulu, mundur acak, baru kirim"));
    Serial.println(F("Tanpa ACK: node tahu kanal sepi, tetap tidak tahu paketnya sampai"));

    laporLantaiDerau();
    Serial.println();
}

void loop() {
    float suhu, lembab;
    bacaSensor(suhu, lembab);

    String payload = "NODE=" + String(NODE_ID) +
                    ",SEQ=" + String(seq) +
                    ",T=" + String(suhu, 1) +
                    ",H=" + String(lembab, 1);

    kirimCSMA(payload);

    unsigned long rataTunda = txCount ? (delaySum / txCount) : 0;
    Serial.print(F("[STAT] TX=")); Serial.print(txCount);
    Serial.print(F(" | DROP=")); Serial.print(dropCount);
    Serial.print(F(" | kanal sibuk=")); Serial.print(busyCount);
    Serial.print(F(" | rata-rata tunda=")); Serial.print(rataTunda);
    Serial.println(F(" ms"));
    Serial.println();

    seq++; // naik untuk setiap data yang DIHASILKAN, termasuk yang dibuang,
          // supaya gateway melihat lubang SEQ pada paket yang tidak terkirim

    delay(random(SEND_INTERVAL_MIN, SEND_INTERVAL_MAX + 1));
}

```

Kode 12.2: src/node/main.cpp — carrier sense (RSSI dan CAD), backoff, dan freeze

```

/*
LoRa CSMA/CA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: gateway

Gateway di modul ini SENGAJA pasif. Ia bukan master: tidak memanggil node,
tidak membagi giliran, tidak mengirim SYNC, dan belum membalas ACK. Seluruh
keputusan "kapan boleh bicara" ada di node, dan gateway hanya menyaksikan
hasilnya. Yang dikerjakan gateway hanya tiga hal:

    1. mendengar terus-menerus (interrupt DIO0 + flag, tanpa blocking),
    2. mencatat isi paket beserta RSSI/SNR-nya,
    3. memperkirakan paket hilang dari lompatan nomor urut SEQ.

Karena tidak ada balasan apa pun, arah radio di sini satu arah penuh --
yang berarti gateway tidak pernah ikut membuat kanal sibuk. Inilah yang
membuat pengamatan CSMA/CA di modul ini bersih: setiap kali sebuah node
melaporkan "kanal sibuk", yang ia dengar pasti node satunya.

Payload : "NODE=<id>,SEQ=<n>,T=<suhu>,H=<lembab>"
Balasan : tidak ada

Pin Mapping Dragino Shield v1.2:
    NSS/CS -> D10, DIO0 -> D2, RST -> D9
    SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DIO0_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_DURATION      150
#define NODE_COUNT        2 // Node 1 dan Node 2 -> indeks array 1..2

volatile bool rxFlag = false;

unsigned long ledOnTime = 0;
bool ledActive = false;

unsigned long rxTotal = 0;
unsigned long rxCount[NODE_COUNT + 1] = {0};
long lastSeq[NODE_COUNT + 1];
unsigned long lostEst[NODE_COUNT + 1] = {0};
unsigned long lastRxTime = 0;

void onReceive(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

String getField(const String& data, const String& key) {
    String needle = key + "=";
    int start = data.indexOf(needle);
    if (start == -1) return "";
    start += needle.length();
    int end = data.indexOf(',', start);
    if (end == -1) end = data.length();
    return data.substring(start, end);
}

```

```

void setup() {
  Serial.begin(115200);
  while (!Serial);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  for (int i = 0; i <= NODE_COUNT; i++) lastSeq[i] = -1;

  Serial.println(F("=== LoRa CSMA/CA - GATEWAY ==="));
  Serial.print(F("Init LoRa ... "));

  LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
  if (!LoRa.begin(FREQUENCY)) {
    Serial.println(F("GAGAL! Cek shield/kabel."));
    while (true);
  }

  LoRa.setSpreadingFactor(SPREADING_FACTOR);
  LoRa.setSignalBandwidth(BANDWIDTH);
  LoRa.setCodingRate4(CODING_RATE);
  LoRa.setTxPower(TX_POWER);
  LoRa.onReceive(onReceive);
  LoRa.receive();

  Serial.println(F("OK"));
  Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
  Serial.println(F("Peran: GATEWAY (CSMA/CA) -- hanya mendengar, tanpa polling, tanpa ACK"));
  Serial.println(F("Menunggu paket dari Node 1 & Node 2...\n"));
}

void loop() {
  updateLED();

  if (!rxFlag) return;
  rxFlag = false;

  unsigned long now = millis();

  String received = "";
  while (LoRa.available()) {
    received += (char)LoRa.read();
  }

  if (!received.startsWith("NODE=")) {
    // Paket yang tetap lolos header tetapi isinya kacau biasanya sisa tabrakan
    // atau interferensi -- dicetak apa adanya supaya bisa dihitung praktikan.
    Serial.print(F("[WARN] Paket tak dikenal (mungkin sisa tabrakan): "));
    Serial.println(received);
    LoRa.receive();
    return;
  }

  String fNode = getField(received, "NODE");
  String fSeq = getField(received, "SEQ");
  String fT = getField(received, "T");
  String fH = getField(received, "H");

  // Paket yang rusak di udara kadang masih menyisakan awalan "NODE=" utuh
  // sementara sisanya hancur. Tanpa pemeriksaan ini, field yang hilang dibaca
  // toInt() sebagai 0 -- dan SEQ palsu bernilai 0 membuat perkiraan kehilangan
  // meledak ratusan paket pada paket berikutnya yang sah. Satu paket rusak
  // tidak boleh merusak seluruh statistik sesi.
  if (fNode.length() == 0 || fSeq.length() == 0 ||
      fT.length() == 0 || fH.length() == 0) {
    Serial.print(F("[WARN] Paket cacat (field tidak lengkap): "));
    Serial.println(received);
    LoRa.receive();
    return;
  }

  int nodeId = fNode.toInt();
  long seq = fSeq.toInt();
  float suhu = fT.toFloat();
  float lembab = fH.toFloat();

  triggerLED();
  rxTotal++;

  Serial.println(F("=== PAKET DITERIMA ==="));
  Serial.print(F(" Node : ")); Serial.println(nodeId);

```

```

Serial.print(F(" SEQ      : ")); Serial.println(seq);
Serial.print(F(" Suhu      : ")); Serial.print(suhu, 1); Serial.println(F(" C"));
Serial.print(F(" Lembab      : ")); Serial.print(lembab, 1); Serial.println(F(" %"));
Serial.print(F(" RSSI       : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
Serial.print(F(" SNR        : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));

// Selang antar-kedatangan: pada kanal yang benar-benar diatur CSMA/CA, dua
// paket tidak pernah tiba nyaris bersamaan -- yang datang belakangan
// seharusnya sudah menunggu lebih dulu di sisi node.
if (lastRxTime > 0) {
    Serial.print(F(" Selang : ")); Serial.print(now - lastRxTime);
    Serial.println(F(" ms dari paket sebelumnya"));
}
lastRxTime = now;

if (nodeId >= 1 && nodeId <= NODE_COUNT) {
    rxCount[nodeId]++;

    if (lastSeq[nodeId] >= 0) {
        long gap = seq - lastSeq[nodeId] - 1;
        if (gap > 0) {
            lostEst[nodeId] += gap;
            Serial.print(F(" [GAP] SEQ meloncat "));
            Serial.print(gap);
            Serial.println(F(" -- paket dibuang node atau bertabrakan"));
        }
    }
    lastSeq[nodeId] = seq;

    Serial.print(F(" Statistik Node ")); Serial.print(nodeId);
    Serial.print(F(": diterima=")); Serial.print(rxCount[nodeId]);
    Serial.print(F(" | perkiraan hilang=")); Serial.println(lostEst[nodeId]);
}

Serial.print(F(" Total diterima gateway: ")); Serial.println(rxTotal);
Serial.println(F("=====\\n"));

LoRa.receive();
}

```

Kode 12.3: src/gateway/main.cpp — terima, catat selang, dan tolak paket cacat

```

#!/usr/bin/env python3
"""
LoRa CSMA/CA Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Gateway <-> Node 1 & Node 2 (node mendengar kanal dulu, backoff acak, tanpa ACK)

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
pip install pyserial rich

Cara pakai:
python lora_monitor.py                                # port bawaan sesuai OS
python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys
from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

```

```

try:
    from rich.live      import Live
    from rich.table     import Table
    from rich.layout    import Layout
    from rich.panel     import Panel
    from rich.text      import Text
    from rich.console   import Console, Group
    from rich.columns   import Columns
    from rich           import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# -----
# KONFIGURASI
# -----
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --gateway/--n1/--n2.
if sys.platform.startswith("win"):
    DEFAULT_GW_PORT = "COM3"
    DEFAULT_N1_PORT = "COM4"
    DEFAULT_N2_PORT = "COM5"
else:
    DEFAULT_GW_PORT = "/dev/ttyACM0"
    DEFAULT_N1_PORT = "/dev/ttyACM1"
    DEFAULT_N2_PORT = "/dev/ttyACM2"
DEFAULT_BAUD      = 115200
LOG_LINES        = 16
REFRESH_HZ       = 4
GAP_WARN_STREAK  = 3

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, menurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default gateway/n1/n2 selalu
    mengikuti apa yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
        return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))
    # CSMA/CA (M08) belum punya ACK -- node tahu kanal sepi saat ia mengirim,
    # tetapi tetap tidak tahu apakah pakatnya sampai. Baris OK/FAIL/Retry di panel
    # node diganti statistik akses kanal: sibuk, backoff, freeze, drop.
    HAS_ACK = False
    # Ambang peringatan: node yang membuang paket sebanyak ini dianggap anomali.
    DROP_WARN_STREAK = 3

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class GatewayState:
    port: str = DEFAULT_GW_PORT
    connected: bool = False
    rx_total: int = 0
    rx_n1: int = 0
    rx_n2: int = 0
    gap_n1: int = 0
    gap_n2: int = 0
    lost_n1: int = 0
    lost_n2: int = 0
    dup_n1: int = 0
    dup_n2: int = 0
    last_rssi: int = 0
    last_snr: float = 0.0
    rssi_sum: int = 0
    snr_sum: float = 0.0
    rssi_count: int = 0

```



```

cycle:         int = 0
raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

@dataclass
class NodeState:
    name:        str = ""
    port:        str = ""
    connected:   bool = False
    tx_count:    int = 0
    ok_count:    int = 0
    fail_count:  int = 0
    retry_count: int = 0
    last_seq:    int = -1
    busy_count:  int = 0 # [CS] kanal SIBUK
    backoff_count: int = 0 # [BACKOFF] berapa kali mundur acak
    freeze_count: int = 0 # [FREEZE] pencacah backoff dibekukan
    drop_count:  int = 0 # [DROP] paket dibuang, kanal tak kunjung bebas
    last_cw:     int = 0 # contention window terakhir yang dipakai
    last_delay:  int = 0 # tunda akses kanal paket terakhir (ms)
    avg_delay:   int = 0 # rata-rata tunda akses kanal (ms), dari [STAT]
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.gw = GatewayState()
        self.n1 = NodeState("NODE 1", DEFAULT_N1_PORT)
        self.n2 = NodeState("NODE 2", DEFAULT_N2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

#
# PARSERS
#
# Semua regex dibuat agar aman dipakai lintas modul (M08A/M08B/M09/M10) --
# token yang tidak muncul pada suatu modul (mis. RETRY/SYNC) hanya tidak
# pernah cocok, tanpa memengaruhi parsing token lain.

RE_NODE_LINE = re.compile(r'Node\s+:\s*(\d+)')
RE_SEQ_LINE = re.compile(r'SEQ\s+:\s*(\d+)')
RE_RSSI_LINE = re.compile(r'RSSI\s+:\s*(-?\d+)')
RE_SNR_LINE = re.compile(r'SNR\s+:\s*(-?\d+.\d+)')
RE_GAP_LINE = re.compile(r'\[GAP\] SEQ meloncat\s+(\d+)')
RE_STAT_LINE = re.compile(r'Statistik Node\s+(\d+):\s*diterima=(\d+)\s*|\s*perkiraan hilang=(\d+)')
RE_SYNC_TX = re.compile(r'\[TX\]\s+SYNC=(\d+)')
RE_CYCLE_DONE = re.compile(r'Cycle\s+(\d+)\s+selesai')
# M09: "Statistik Node X: baru=Y | duplicate=Z | gagal permanen (est.)=W"
RE_STAT_RETRY = re.compile(r'Statistik Node\s+(\d+):\s*baru=(\d+)\s*|\s*duplicate=(\d+)\s*|\s*gagal permanen\s*(est\.)=(\d+)')
# M10: "--- Cycle N selesai | N1: diterima=Y hilang=Z N2: diterima=Y hilang=Z ---"
RE_STAT_CYCLE = re.compile(r'N(\d+):\s*diterima=(\d+)\s*hilang=(\d+)')

RE_NODE_TX = re.compile(r'\[TX\].*NODE=(\d+),SEQ=(\d+)')
# M08 CSMA/CA -- baris khas akses kanal di sisi node
RE_NODE_TUNDA = re.compile(r'tunda=(\d+)\s*ms')
RE_NODE_BUSY = re.compile(r'\[CS\] kanal SIBUK')
RE_NODE_BACKOFF = re.compile(r'\[BACKOFF\].*CW=(\d+)\s*|\s*slot=(\d+)')
RE_NODE_FREEZE = re.compile(r'\[FREEZE\]')
RE_NODE_DROP = re.compile(r'\[DROP\] SEQ=(\d+)')
RE_NODE_STAT = re.compile(r'\[STAT\].*rata-rata tunda=(\d+)\s*ms')
RE_NODE_RETRY = re.compile(r'\[RETRY\s+(\d+)/(\d+)\s*')
RE_NODE_OK = re.compile(r'\[OK\]')
RE_NODE_FAIL = re.compile(r'\[FAIL\]')

def parse_gateway_line(line: str, st: GatewayState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    if 'PAKET DITERIMA' in line:
        st.rx_total += 1

    m = RE_RSSI_LINE.search(line)
    if m:
        st.last_rssi = int(m.group(1))
        st.rssi_sum += st.last_rssi
        st.rssi_count += 1

    m = RE_SNR_LINE.search(line)
    if m:
        st.last_snr = float(m.group(1))
        st.snr_sum += st.last_snr

```

```

m = RE_GAP_LINE.search(line)
if m:
    gap = int(m.group(1))
    anomalies.append(Anomaly(ts, "GATEWAY", "GAP", f"SEQ meloncat {gap} -- indikasi tabrakan/paket hilang"))

m = RE_STAT_LINE.search(line)
if m:
    node_id, rx, lost = int(m.group(1)), int(m.group(2)), int(m.group(3))
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_STAT_RETRY.search(line)
if m:
    node_id, rx, dup, lost = int(m.group(1)), int(m.group(2)), int(m.group(3)), int(m.group(4))
    if node_id == 1:
        st.rx_n1, st.dup_n1, st.lost_n1 = rx, dup, lost
    elif node_id == 2:
        st.rx_n2, st.dup_n2, st.lost_n2 = rx, dup, lost

for node_id_s, rx_s, lost_s in RE_STAT_CYCLE.findall(line):
    node_id, rx, lost = int(node_id_s), int(rx_s), int(lost_s)
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_SYNC_TX.search(line)
if m:
    st.cycle = int(m.group(1))

m = RE_CYCLE_DONE.search(line)
if m:
    st.cycle = int(m.group(1))

def parse_node_line(line: str, st: NodeState, node_label: str, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = RE_NODE_TX.search(line)
    if m:
        seq = int(m.group(2))
        if st.last_seq >= 0 and seq == st.last_seq and '[RETRY]' not in line:
            pass # aman, tidak dipakai
        st.last_seq = seq
        if '[RETRY]' not in line:
            st.tx_count += 1

    m = RE_NODE_TUNDA.search(line)
    if m:
        st.last_delay = int(m.group(1))

    m = RE_NODE_RETRY.search(line)
    if m:
        st.retry_count += 1

    if RE_NODE_BUSY.search(line):
        st.busy_count += 1

    m = RE_NODE_BACKOFF.search(line)
    if m:
        st.backoff_count += 1
        st.last_cw = int(m.group(1))

    if RE_NODE_FREEZE.search(line):
        st.freeze_count += 1

    m = RE_NODE_DROP.search(line)
    if m:
        st.drop_count += 1
        anomalies.append(Anomaly(ts, node_label, "DROP",
            f"SEQ={m.group(1)} dibuang -- kanal tetap sibuk sampai percobaan terakhir"))

    m = RE_NODE_STAT.search(line)
    if m:
        st.avg_delay = int(m.group(1))

    if RE_NODE_OK.search(line):
        st.ok_count += 1
    elif RE_NODE_FAIL.search(line):

```

```

        st.fail_count += 1
        if st.fail_count and st.fail_count % 5 == 0:
            anomalies.append(Anomaly(ts, node_label, "FAIL_COUNT",
                                     f"{node_label} sudah {st.fail_count}x gagal (tanpa ACK)"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
                  shared: SharedState, role: str):
    if role == "GW":
        state = shared.gw
    elif node_key == "N1":
        state = shared.n1
    else:
        state = shared.n2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:
                                state.raw_log.append((ts, line))
                                if role == "GW":
                                    parse_gateway_line(line, state, shared.anomalies)
                                else:
                                    parse_node_line(line, state, node_key, shared.anomalies)
                            shared.csv_queue.put({
                                'ts': ts, 'node': node_key, 'raw': line
                            })
                        except (serial.SerialException, OSError):
                            break
                    except (serial.SerialException, OSError):
                        with shared.lock:
                            state.connected = False
                        time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                           'node': row['node'],
                           'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"
    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str) -> str:
    if '[FAIL]' in line: return "red"
    if '[GAP]' in line: return "bold red"
    if '[RETRY]' in line: return "bold yellow"

```

```

if '[BACKOFF]' in line: return "yellow"
if '[DROP]' in line: return "bold red"
if '[FREEZE]' in line: return "magenta"
if '[CS]' in line: return "yellow"
if '[KALIBRASI]' in line: return "bold blue"
if '[STAT]' in line: return "dim cyan"
if '[OK]' in line: return "bold green"
if '[TX]' in line: return "cyan"
if 'PAKET DITERIMA' in line: return "green"
if '[WAIT]' in line: return "dim"
if '[WARN]' in line: return "yellow"
return "dim white"

def build_gateway_panel(st: GatewayState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" Total diterima : ", style="dim"); t.append(f"{st.rx_total}\n", style="bold white")
    if st.cycle:
        t.append(" Cycle terakhir : ", style="dim"); t.append(f"{st.cycle}\n", style="bold cyan")
    t.append("\n")

    dup_suffix1 = f" dup={st.dup_n1}" if st.dup_n1 else ""
    dup_suffix2 = f" dup={st.dup_n2}" if st.dup_n2 else ""
    t.append(" Node 1 ", style="bold cyan")
    t.append(f"diterima={st.rx_n1:<4} hilang={st.lost_n1}{dup_suffix1}\n", style="white")
    t.append(" Node 2 ", style="bold magenta")
    t.append(f"diterima={st.rx_n2:<4} hilang={st.lost_n2}{dup_suffix2}\n\n", style="white")

    rssi_avg = (st.rssi_sum / st.rssi_count) if st.rssi_count else 0.0
    snr_avg = (st.snr_sum / st.rssi_count) if st.rssi_count else 0.0
    t.append(" RSSI terakhir : ", style="dim"); t.append(f"{st.last_rssi} dBm\n", style="white")
    t.append(" SNR terakhir : ", style="dim"); t.append(f"{st.last_snr:.2f} dB\n", style="white")
    t.append(" RSSI rata-rata : ", style="dim"); t.append(f"{rssi_avg:.1f} dBm\n", style="white")
    t.append(" SNR rata-rata : ", style="dim"); t.append(f"{snr_avg:.2f} dB\n\n", style="white")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:40]}\n", style=style)

    return Panel(t, title="[bold cyan]GATEWAY[/]",
                border_style="cyan", box=box.ROUNDED)

def build_node_panel(st: NodeState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" TX terkirim : ", style="dim"); t.append(f"{st.tx_count:>6}\n", style="bold white")
    t.append(" SEQ terakhir : ", style="dim"); t.append(f"{st.last_seq:>6}\n\n", style="white")

    if HAS_ACK:
        tot = st.ok_count + st.fail_count
        pct = (st.ok_count / tot * 100) if tot else 0.0
        t.append(" OK : ", style="dim"); t.append(f"{st.ok_count:>6}\n", style="bold green")
        t.append(" FAIL : ", style="dim"); t.append(f"{st.fail_count:>6}\n",
            style="bold red" if st.fail_count else "dim")
        if tot:
            t.append(" Rate : ", style="dim"); t.append(f"{pct:>6.2f}%\n", style=pct_style(st.ok_count,
tot))
        t.append(" Retry : ", style="dim"); t.append(f"{st.retry_count:>6}\n\n",
            style="yellow" if st.retry_count else "dim")
    else:
        total = st.tx_count + st.drop_count
        t.append(" Akses kanal (CSMA/CA)\n", style="bold dim")
        t.append(" Kanal sibuk : ", style="dim")
        t.append(f"{st.busy_count:>6}\n", style="yellow" if st.busy_count else "dim")
        t.append(" Backoff : ", style="dim")
        t.append(f"{st.backoff_count:>6}\n", style="yellow" if st.backoff_count else "dim")
        t.append(" Freeze : ", style="dim")
        t.append(f"{st.freeze_count:>6}\n", style="magenta" if st.freeze_count else "dim")
        t.append(" DROP : ", style="dim")
        t.append(f"{st.drop_count:>6}\n", style="bold red" if st.drop_count else "dim")
        if total:
            kirim_pct = st.tx_count / total * 100
            t.append(" Terkirim : ", style="dim")
            t.append(f"{kirim_pct:>6.2f}%\n", style=pct_style(st.tx_count, total))

```

```

        t.append(" CW terakhir : ", style="dim"); t.append(f"{st.last_cw:>6}\n", style="white")
        t.append(" Tunda akhir : ", style="dim"); t.append(f"{st.last_delay:>6} ms\n", style="white")
        t.append(" Tunda rata2 : ", style="dim"); t.append(f"{st.avg_delay:>6} ms\n\n", style="white")
        t.append(" (tanpa ACK -- lihat panel GATEWAY untuk\n", style="dim")
        t.append(" statistik diterima/hilang per node.)\n\n", style="dim")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

    return Panel(t, title=f"[bold {color}]{label}[/]",
                border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                        csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'GAP': "bold red",
        'DROP': "bold red",
        'FAIL_COUNT': "bold yellow",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=style)
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        gw = GatewayState(
            port=shared.gw.port, connected=shared.gw.connected,
            rx_total=shared.gw.rx_total, rx_n1=shared.gw.rx_n1, rx_n2=shared.gw.rx_n2,
            gap_n1=shared.gw.gap_n1, gap_n2=shared.gw.gap_n2,
            lost_n1=shared.gw.lost_n1, lost_n2=shared.gw.lost_n2,
            dup_n1=shared.gw.dup_n1, dup_n2=shared.gw.dup_n2,
            last_rssi=shared.gw.last_rssi, last_snr=shared.gw.last_snr,
            rssi_sum=shared.gw.rssi_sum, snr_sum=shared.gw.snr_sum,
            rssi_count=shared.gw.rssi_count, cycle=shared.gw.cycle,
            raw_log=deque(shared.gw.raw_log),
        )
        n1 = NodeState(
            name=shared.n1.name, port=shared.n1.port, connected=shared.n1.connected,
            tx_count=shared.n1.tx_count, ok_count=shared.n1.ok_count,
            fail_count=shared.n1.fail_count, retry_count=shared.n1.retry_count,
            last_seq=shared.n1.last_seq,
            busy_count=shared.n1.busy_count, backoff_count=shared.n1.backoff_count,
            freeze_count=shared.n1.freeze_count, drop_count=shared.n1.drop_count,
            last_cw=shared.n1.last_cw, last_delay=shared.n1.last_delay,
            avg_delay=shared.n1.avg_delay,
            raw_log=deque(shared.n1.raw_log),
        )
        n2 = NodeState(
            name=shared.n2.name, port=shared.n2.port, connected=shared.n2.connected,
            tx_count=shared.n2.tx_count, ok_count=shared.n2.ok_count,
            fail_count=shared.n2.fail_count, retry_count=shared.n2.retry_count,
            last_seq=shared.n2.last_seq,
            busy_count=shared.n2.busy_count, backoff_count=shared.n2.backoff_count,
            freeze_count=shared.n2.freeze_count, drop_count=shared.n2.drop_count,
            last_cw=shared.n2.last_cw, last_delay=shared.n2.last_delay,

```

```

        avg_delay=shared.n2.avg_delay,
        raw_log=deque(shared.n2.raw_log),
    )
    anom = deque(shared.anomalies)
    t_start = shared.session_start

    node_row = Columns([
        build_gateway_panel(gw),
        build_node_panel(n1, "NODE 1", "green"),
        build_node_panel(n2, "NODE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

# -----
# ENTRY POINT
# -----

BANNER = """\
    LoRa CSMA/CA Monitor – Evaluasi Praktikum
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz
    """

Tool ini membaca data serial dari 3 board LoRa (Gateway, Node 1, Node 2)
secara bersamaan dan menampilkan dashboard real-time: statistik
diterima per node, akses kanal (sibuk/backoff/freeze/drop) dan tunda
akses, RSSI/SNR, deteksi anomali (GAP/DROP),
serta logging CSV otomatis untuk analisis data penelitian.

PENGGUNAAN:
    python lora_monitor.py [opsi...]

OPSI:
    --gateway PORT Port COM untuk Gateway          (default: {gw})
    --n1 PORT Port COM untuk Node 1                  (default: {n1})
    --n2 PORT Port COM untuk Node 2                  (default: {n2})
    --baud N Baud rate serial semua port             (default: {baud})
    --out FILE Nama file CSV output                  (default: lora_session_YYYYMMDD_HHMMSS.csv)
    -h / --help Tampilkan pesan bantuan ini

CONTOH:
    python lora_monitor.py
    -> Jalankan dengan port & baud rate default

    python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
    -> Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

    python lora_monitor.py --out data_jarak_10m.csv
    -> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
    Ctrl+C Hentikan monitoring & simpan CSV

CATATAN:
    * Pastikan semua board sudah di-upload firmware dengan baud {baud}
    * Upload Gateway dulu, baru Node 1 dan Node 2, sebelum menjalankan tool ini
    * File CSV tersimpan otomatis saat Ctrl+C ditekan
"""

def parse_args(port_aktif):
    gw_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_GW_PORT
    n1_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_N1_PORT
    n2_default = port_aktif[2] if len(port_aktif) > 2 else DEFAULT_N2_PORT

    p = argparse.ArgumentParser(
        description="LoRa CSMA/CA Monitor Dashboard -- tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f"python lora_monitor.py\n"
            f"python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2\n"
            f"python lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--gateway', default=gw_default,
        metavar='PORT', help=f'Port COM Gateway (default: {gw_default}, dari deteksi port

```

```

aktif)')
    p.add_argument('--n1', default=n1_default,
                  metavar='PORT', help=f'Port COM Node 1 (default: {n1_default}, dari deteksi port aktif)')
    p.add_argument('--n2', default=n2_default,
                  metavar='PORT', help=f'Port COM Node 2 (default: {n2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
                  metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None,
                  metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
    return p, p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    p, args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1

    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
    )
    console.print(
        f"[dim]Dipakai -> "
        f"Gateway=[yellow]{args.gateway}[/] "
        f"N1=[green]{args.n1}[/] "
        f"N2=[magenta]{args.n2}[/] "
        f"Baud=[white]{args.baud}[/][dim]\n"
    )

    shared = SharedState()
    shared.gw.port = args.gateway
    shared.n1.port = args.n1
    shared.n2.port = args.n2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_session_{ts_str}.csv"

    if not no_args:
        console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
        console.print(f" Gateway : [yellow]{args.gateway}[/]")
        console.print(f" Node 1 : [green]{args.n1}[/]")
        console.print(f" Node 2 : [magenta]{args.n2}[/]")
        console.print(f" Baud : [white]{args.baud}[/]")

    console.print(f" CSV : [dim]{csv_path}[/]")
    console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:
        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread,
                        args=(args.gateway, args.baud, "GW", shared, "GW"),
                        daemon=True, name="reader-gateway"),
        threading.Thread(target=reader_thread,
                        args=(args.n1, args.baud, "N1", shared, "NODE"),
                        daemon=True, name="reader-n1"),
        threading.Thread(target=reader_thread,
                        args=(args.n2, args.baud, "N2", shared, "NODE"),
                        daemon=True, name="reader-n2"),
        threading.Thread(target=csv_writer_thread,
                        args=(shared, csv_path),
                        daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path),
                 console=console,
                 refresh_per_second=REFRESH_HZ,
                 screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass

```

```

finally:
    shared.running = False
    time.sleep(0.6)
    console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]" )

if __name__ == "__main__":
    main()

```

Kode 12.4: lora_monitor.py — dasbor tiga panel dengan statistik akses kanal

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 08 (CSMA/CA) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul09_lora_csma_ca -e gateway -t upload -t monitor
    pio run -d Modul09_lora_csma_ca -e node1 -t upload -t monitor
    pio run -d Modul09_lora_csma_ca -e node2 -t upload -t monitor

Skrip ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment `gateway` (port pertama), `node1` (port kedua), dan `node2`
(port ketiga), lalu menjalankan `pio run` dengan --upload-port sesuai hasil
deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga
board selesai di-upload, skrip bertanya apakah mau langsung menjalankan
lora_monitor.py -- bila ya, dijalankan dengan --gateway ... --n1 ... --n2 ...
persis memakai port yang barusan dipakai upload (bukan deteksi ulang),
supaya tidak mungkin salah pasang.

python3 Modul09_lora_csma_ca/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul09_lora_csma_ca/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti lora_monitor.py

python3 Modul09_lora_csma_ca/upload_auto.py --only gateway
-> upload satu environment saja

python3 Modul09_lora_csma_ca/upload_auto.py --gateway /dev/ttyACM0 --node1 /dev/ttyACM1 --node2 /dev/
ttyACM2
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["gateway", "node1", "node2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"gateway": "--gateway", "node1": "--n1", "node2": "--n2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print("  $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):

```



```

"""Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
dengan port gateway/node1/node2 PERSIS seperti yang baru dipakai upload
-- bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
barusan di-flash.
"""
if not sys.stdin.isatty():
    return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

try:
    jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
except (EOFError, KeyboardInterrupt):
    print()
    return
if jawab not in ("", "y", "ya"):
    return

cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
for env, opsi in ENV_KE_OPSI.items():
    if env in port_env:
        cmd += [opsi, port_env[env]]

print(" $", " ".join(cmd))
try:
    subprocess.call(cmd)
except KeyboardInterrupt:
    # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
    # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
    # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
    # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
    print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--gateway", metavar="/dev/ttyXXX",
                    help="timpa port gateway (lewati deteksi otomatis)")
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="timpa port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="timpa port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (gateway, node1, atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                        "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau ketiganya"
                        "sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"gateway": args.gateway, "node1": args.node1, "node2": args.node2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {' '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
                  f" - sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {' '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai.")
    tanya_lalu_monitor(port_env)

```

```
if __name__ == "__main__":
    main()
```

Kode 12.5: upload_auto.py — unggah ketiga board dengan port terdeteksi sendiri

12.7 Build dan Flash

Gateway lebih dahulu, supaya paket pertama dari node langsung tertangkap.

```
pio run -d Modul09_lora_csma_ca -e gateway -t upload -t monitor
pio run -d Modul09_lora_csma_ca -e node1 -t upload -t monitor
pio run -d Modul09_lora_csma_ca -e node2 -t upload -t monitor
```

Atau otomatis, tanpa mengedit port di platformio.ini:

```
python3 Modul09_lora_csma_ca/upload_auto.py
```

Monitor dashboard. python3 lora_monitor.py membaca ketiga port sekaligus dan menampilkan panel Gateway, Node 1, dan Node 2. Panel node modul ini menampilkan statistik akses kanal: berapa kali kanal ditemukan sibuk, berapa kali backoff dan freeze terjadi, berapa paket dibuang, serta tunda akses terakhir dan rata-ratanya. Memerlukan pip install pyserial rich.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ Port ketiga board dicatat lewat `pio device list` (atau `python3 ../tools/deteksi_port.py`) dan diisikan ke platformio.ini.
- ☐ Tiga Serial Monitor 115200 baud siap, ketiganya terlihat bersamaan.
- ☐ `NODE_ID` pada node1 dan node2 sudah benar, dicek dari baris pembuka `NODE 1` atau `NODE 2`.
- ☐ Baris `Carrier sense`: pada kedua node menunjukkan mode dan ambang yang sama.

12.8 Percobaan

12.8.1 EXP-01 — Kalibrasi Telinga: Berapa Nilai “Sibuk”?

Ambang – 95 dBm di kode hanyalah tebakan awal. Sebelum mempercayai keputusan node, ukur dulu seperti apa kanal ini sebenarnya.

1. Nyalakan **satu node saja**, board lain dimatikan. Saat menyala, node mengambil 200 sampel RSSI kanal kosong dan melaporkannya sendiri.

```
[KALIBRASI] rantai derau 200 sampel: min -119 | rata-rata -112 | maks -104 dBm
[KALIBRASI] ambang terpakai sekarang: -95 dBm -- lihat EXP-01
```

Angka di atas hanya bentuk barisnya; nilai sesungguhnya bergantung pada lingkungan masing-masing.

2. Nyalakan node kedua. Pada serial node pertama, baca nilai RSSI yang tercetak pada baris `[CS] kanal SIBUK (RSSI ...)` — itulah kekuatan sinyal tetangga saat sedang mengirim.
3. Tentukan ambang di tengah-tengah kedua nilai itu, lalu isikan ke `RSSI_THRESHOLD` dan unggah ulang kedua node. Baris `[KALIBRASI]` kedua akan mengonfirmasi ambang baru sudah terpasang.

Data capture

Parameter	Verifikasi 23-08-2026	Hasil sendiri
Lantai derau — min / rata-rata / maks (dBm)	– 117 / – 110 / – 105	
RSSI saat node tetangga mengirim	– 29 ... – 33	

Parameter	Verifikasi 23-08-2026	Hasil sendiri
Selisih keduanya (dB)	≈ 80	
RSSI_THRESHOLD yang dipilih	- 95 (bawaan, sudah di tengah rentang)	

Tabel 12.7: Lembar pengamatan EXP-01 — kalibrasi ambang carrier sense

CHECKPOINT

Terpenuhi pada verifikasi. Selisihnya ternyata sangat lebar — sekitar **80 dB** antara kanal kosong (≈ -110 dBm) dan kanal terpakai (≈ -30 dBm) pada jarak meja — sehingga ambang bawaan - 95 dBm aman di tengah rentang dan tidak perlu diubah. Selisih sebesar itu tidak dijamin di ruangan lain; tetap ukur sendiri. Selisih antara kanal kosong dan kanal terpakai harus jelas (belasan hingga puluhan dB pada jarak meja). Bila selisihnya kecil, jarak antar-board terlalu jauh atau antenna belum terpasang benar. Ambang yang terlalu tinggi membuat node tuli (semua dianggap sepi); terlalu rendah membuatnya paranoid (semua dianggap sibuk, semua paket berakhir [DROP]).

12.8.2 EXP-02 — Dua Node Sopan

Nyalakan ketiga board dengan parameter bawaan dan amati beberapa menit.

Expected output — node

```

=== LoRa CSMA/CA - NODE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
Carrier sense: RSSI (ambang -95 dBm) | DIFS 30 ms | slot 20 ms | CW 4..64
Peran: NODE (CSMA/CA) -- dengar dulu, mundur acak, baru kirim
Tanpa ACK: node tahu kanal sepi, tetap tidak tahu paketnya sampai
[KALIBRASI] lantai derau 200 sampel: min -117 | rata-rata -110 | maks -105 dBm
[KALIBRASI] ambang terpakai sekarang: -95 dBm -- lihat EXP-01

[TX] NODE=1,SEQ=0,T=27.1,H=70.5 | attempt=1 | tunda=31 ms
[STAT] TX=1 | DROP=0 | kanal sibuk=0 | rata-rata tunda=31 ms

```

Ketika tetangganya sedang bicara:

```

[CS] kanal SIBUK (RSSI -47 dBm)
[BACKOFF] percobaan 1/5 | CW=4 | slot=2 -> 40 ms
[FREEZE] pencacah backoff dibekukan -- kanal terpakai
[TX] NODE=2,SEQ=3,T=29.2,H=67.8 | attempt=2 | tunda=118 ms

```

Expected output — gateway

```

=== PAKET DITERIMA ===
Node      : 1
SEQ       : 0
Suhu      : 27.1 C
Lembab    : 70.5 %
RSSI      : -45 dBm
SNR       : 9.25 dB
Statistik Node 1: diterima=1 | perkiraan hilang=0
Total diterima gateway: 1
=====

=== PAKET DITERIMA ===
Node      : 2
SEQ       : 0
Suhu      : 27.1 C
Lembab    : 71.0 %
RSSI      : -55 dBm
SNR       : 9.50 dB
Selang    : 136 ms dari paket sebelumnya
Statistik Node 2: diterima=1 | perkiraan hilang=0
Total diterima gateway: 2
=====

```

Data capture — amati 3 menit.

Parameter	Verifikasi 23-08-2026	Hasil sendiri
Paket diterima gateway — N1 / N2	49 / 49 (dari 50 / 49 dikirim)	
Jumlah [CS] kanal SIBUK — N1 / N2	1 / 2	
Jumlah [BACKOFF] — N1 / N2	1 / 2	
Jumlah [DROP] — N1 / N2	0 / 0	
Tunda akses rata-rata (ms) — N1 / N2	31,7 / 34,3	
Selang antar-paket terkecil di gateway (ms)	116	

Tabel 12.8: Lembar pengamatan EXP-02 — dua node sopan

CHECKPOINT

Terpenuhi. Pada interval bawaan (2–5 detik) kanal jarang berebut: 97 dari 99 paket berangkat dengan attempt=1 dan tunda $\approx$ DIFS_MS (30 ms). Tiga kali sepanjang 3 menit sebuah node menemukan kanal terpakai, mundur, lalu berhasil pada percobaan kedua. Yang wajib muncul minimal beberapa kali adalah baris [CS] kanal SIBUK — itulah bukti telinga node benar-benar mendengar tetangganya. Bila tidak pernah muncul sama sekali sepanjang 3 menit, ambang RSSI kemungkinan masih terlalu tinggi; ulangi EXP-01.

12.8.3 EXP-03 — Memaksa Berebut

Perkecil interval kirim pada **kedua** node menjadi 300–500 ms lewat build flag di `platformio.ini` (tidak perlu mengedit source), seperti Kode 12.6.

```
build_flags = -DNODE_ID=1 -DCS_MODE=0 -DSEND_INTERVAL_MIN=300 -DSEND_INTERVAL_MAX=500
```

Kode 12.6: Build flag EXP-03 — interval kirim dipersempit

Unggah ulang kedua node, lalu amati kembali. Beban kanal kini jauh melewati kapasitasnya (dua paket $\sim$ 60 ms setiap $\sim$ 400 ms per node), sehingga mekanisme CA dipaksa bekerja keras.

Data capture — amati 3 menit.

Parameter	Verifikasi 23-08-2026	Hasil sendiri
Interval kirim yang dipakai (ms)	300–500	
Paket diterima gateway — N1 / N2	351 / 343 (dari 352 / 348 dikirim)	
Jumlah [BACKOFF] — N1 / N2	32 / 67	
Jumlah [FREEZE] — N1 / N2	26 / 46	
Jumlah [DROP] — N1 / N2	0 / 0	
CW terbesar yang sempat terpakai	16	
Tunda akses rata-rata (ms) — N1 / N2	37,2 / 46,1 (maks 238 / 257)	

Parameter	Verifikasi 23-08-2026	Hasil sendiri
Jumlah [GAP] di gateway — N1 / N2	0 / 4	

Tabel 12.9: Lembar pengamatan EXP-03 — memaksa berebut kanal

BUKA ABSTRAKSINYA

Di `src/node/main.cpp`, fungsi `hitungMundurSlot()` **tidak** sekadar `delay(slot * SLOT_MS)`. Pencacahnya berhenti berkurang selama kanal terpakai dan lanjut lagi saat sepi. Ganti sementara isi fungsi itu dengan `delay((long)slot * SLOT_MS)` biasa, unggah ulang, lalu ulangi EXP-03 dan bandingkan jumlah [DROP]-nya. Jelaskan: node mana yang dirugikan oleh backoff yang *tidak* dibekukan, dan mengapa?

CHECKPOINT

Terpenuhi sebagian. [BACKOFF] melonjak dari 3 (EXP-02) menjadi 99, tunda akses rata-rata naik ~ 30 %, dan CW sempat melebar sampai 16. Tetapi [DROP] **tetap nol**: dengan dua node, lima percobaan selalu cukup untuk memperoleh kanal. Untuk benar-benar memunculkan [DROP], perkecil MAX_ATTEMPT atau tambahkan node ketiga (CH-2). Yang paling penting justru ini: dari 700 paket yang naik ke udara, gateway mencatat **0 paket cacat** — CA memindahkan kegagalan dari “rusak di udara” menjadi “tertunda” (dan, pada beban lebih tinggi, “dibuang sebelum berangkat”) — kegagalan yang setidaknya **diketahui pengirimnya**.

12.8.4 EXP-04 — Matikan Telinganya

Ini percobaan pembandingan yang paling menentukan. Pada **kedua** node, matikan telinganya lewat build flag — pertahankan interval sempit dari EXP-03.

```
build_flags = -DNODE_ID=1 -DCS_MODE=2 -DSEND_INTERVAL_MIN=300 -DSEND_INTERVAL_MAX=500
```

Kode 12.7: Build flag EXP-04 — carrier sense dimatikan (CS_MODE=2)

Node kini mengirim buta — persis Pure ALOHA. Unggah ulang kedua node, amati 3 menit, lalu **kembalikan CS_MODE=0**. Baris pembuka node akan menegaskan mode yang sedang aktif: `Carrier sense: MATI -- kirim buta`.

Data capture — hasil verifikasi 23 Agustus 2026, masing-masing 3 menit, beban kirim setara.

Parameter	Dengan CS (EXP-03)	Tanpa CS (EXP-04)
Paket dikirim node — total	700	757
Paket diterima gateway — total	695	581
Paket hilang	5 (0,7 %)	176 (23,2 %)
Jumlah [GAP] di gateway — total	4	102
[WARN] paket cacat di gateway	0	37
Tunda akses rata-rata (ms)	37–46	0
Paket dibuang node ([DROP])	0	0 (tidak ada mekanismenya)

Tabel 12.10: Hasil EXP-04 — CSMA/CA dibanding kirim buta

CHECKPOINT

Terpenuhi. Tanpa carrier sense tunda akses turun ke nol — dan satu dari setiap empat paket lenyap. Inilah pertukaran inti modul ini: CSMA/CA **membeli keandalan dengan waktu**, sekitar 40 ms per paket untuk menekan kehilangan dari 23 % menjadi di bawah 1 %. Perhatikan juga bahwa tanpa carrier sense, node mengirim **lebih banyak** paket (757 vs 700, karena tidak pernah menunggu) namun gateway justru menerima **lebih sedikit**. Bila kedua kolom nyaris sama, beban kanal belum cukup tinggi — persempit lagi intervalnya.

Tabrakan terlihat langsung di sini. Pada sesi tanpa carrier sense, gateway mencetak paket yang awalnya masih terbaca sementara sisanya hancur — inilah wujud tabrakan yang selama EXP-03 tidak pernah muncul sama sekali.

[WARN] Paket cacat (field tidak lengkap): NODE=2,SEQ=1,T=28.t,??,a?2

12.8.5 EXP-05 — RSSI dibanding CAD

Ubah -DCS_MODE=0 menjadi -DCS_MODE=1 pada **kedua** node di platformio.ini, unggah ulang, lalu ulangi EXP-03 dengan interval yang sama persis.

Data capture — hasil verifikasi 23 Agustus 2026, beban kirim setara.

Parameter	CS_MODE=0 (RSSI)	CS_MODE=1 (CAD)
Jumlah [CS] kanal SIBUK — total	99	107
Jumlah [FREEZE] — total	72	96
Jumlah [DROP] — total	0	0
Tunda akses rata-rata (ms) — N1 / N2	37,2 / 46,1	43,3 / 41,3
Paket dikirim / diterima gateway	700 / 695	699 / 694

Tabel 12.11: Hasil EXP-05 — carrier sense berbasis RSSI dibanding CAD

CHECKPOINT

Terpenuhi. CAD tidak memerlukan ambang sama sekali (perhatikan: baris pembuka node tidak lagi menyebut angka dBm), sebab ia mengenali bentuk simbol LoRa, bukan sekadar besar energi. Hasil akhirnya praktis setara — selisih satu paket dari ~ 700 — tetapi CAD melaporkan kanal sibuk **lebih sering** (107 vs 99). Pikirkan mana dari keduanya yang sedang keliru: apakah CAD terlalu waspada, atau justru ambang - 95 dBm yang melewatkan sinyal lemah?

BUKA ABSTRAKSINYA

Untuk yang mengutak-atik mode ini. Library sandeepmistry/LoRa tidak menyediakan CAD sama sekali, jadi kanalTerpakai() versi CAD menulis register SX1276 langsung. Dua jebakan yang ditemukan saat modul ini diuji — keduanya sudah diperbaiki di kode, dan keduanya layak ditelusuri sendiri di src/node/main.cpp:

1. CAD **harus** dimasuki dari STANDBY. Perintah CAD dari mode RX kontinu tidak pernah dijalankan: CadDone tak pernah naik dan tiap pemeriksaan berakhir di timeout.
2. isTransmitting() milik library menguji (RegOpMode & 0x03) == 0x03, dan mode CAD (0x07) **lolos uji itu**. Bila modem masih di CAD saat beginPacket() dipanggil, beginPacket() gagal diam-diam tanpa me-reset FIFO, dan yang naik ke udara adalah sampah. Rinciannya di logserial.md.

12.8.6 Verifikasi Perangkat Keras

CATATAN

Diuji di perangkat pada 23 Agustus 2026 — tiga Arduino Uno asli (2341:0043) + Dragino LoRa Shield v1.2 (gateway, node1, node2 pada /dev/ttyACM0/1/2, cocok dengan platformio.ini bawaan). Ketiga environment dibangun dan diunggah tanpa modifikasi. **EXP-01 sampai EXP-05 seluruhnya dijalankan**, masing-masing jendela rekam 180 detik, dan angka pada semua tabel Data capture di atas serta bagian Pengukuran adalah hasil ukur nyata dari sesi itu — log lengkap beserta cuplikan serial ketiga board ada di logserial.md.

Dua perbaikan lahir dari pengujian ini dan sudah masuk ke kode: gateway kini menolak paket yang field-nya tidak lengkap (tanpa itu, satu paket rusak akibat tabrakan membuat perkiraan kehilangan melonjak ke 1177 dari kenyataan ~ 30), dan jalur CAD kini masuk lewat STANDBY serta memanggil LoRa.idle() sebelum beginPacket() (tanpa itu CAD tidak pernah mendeteksi apa pun **dan** merusak transmisi node sendiri). Keduanya diuraikan di logserial.md.

Satu hal yang **tidak** berhasil dibuktikan: [DROP] tetap nol di seluruh sesi. Dengan dua node, MAX_ATTEMPT=5 selalu cukup. Untuk melihat paket benar-benar dibuang, perkecil MAX_ATTEMPT atau tambahkan node ketiga (CH-2).

12.8.7 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat, direkam **2026-08-23**. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm. Ketiga board di satu meja dengan jarak berbeda-beda — Node 1 lebih dekat ke gateway daripada Node 2 (lihat RSSI di bawah).

Board & Port

Peran	Environment	Port	Board
Gateway	gateway	/dev/ttyACM0	Uno asli (2341:0043)
Node 1	node1	/dev/ttyACM1	Uno asli (2341:0043)
Node 2	node2	/dev/ttyACM2	Uno asli (2341:0043)

Tabel 12.12: Log serial Modul 09: Board & Port

Ketiga aliran serial direkam bersamaan (tiga thread pyserial, satu per port), masing-masing sesi **180 detik** dihitung sejak perekaman dimulai — mencakup boot ketiga board. Parameter CSMA/CA memakai nilai bawaan modul: DIFS 30 ms, slot 20 ms, CW 4...64, MAX_ATTEMPT 5, ambang RSSI -95 dBm.

Ringkasan keempat sesi

Sesi	Carrier sense	Interval kirim	Dikirim node	Tiba Hilang di gateway	[GAP]	[WARN]
EXP-02	RSSI	2000–5000 ms	99	98 1 (1,0%)	1	0
EXP-03	RSSI	300–500 ms	700	695 4 (0,6%)	4	0
EXP-04	mati	300–500 ms	757	581 176 (23,2%)	102	37
EXP-05	CAD	300–500 ms	699	694 4 (0,6%)	4	2

Tabel 12.13: Log serial Modul 09: Ringkasan keempat sesi

Perkiraan kehilangan dari lompatan SEQ di gateway (62 + 113 = 175 pada EXP-04) cocok dengan selisih kirim-terima yang sebenarnya (757 – 581 = 176) — selisih satu paket berasal dari paket terakhir yang belum sempat tercatat saat perekaman berhenti.

Statistik akses kanal per node

Sesi	Node	TX	[CS] sibuk	[BACKOFF]	[FREEZE]	[DROP]	CW maks	Tunda akses med / rata / maks (ms)
EXP-02	1	50	1	1	1	0	4	31 / 31,7 / 116
EXP-02	2	49	2	2	2	0	4	32 / 34,3 / 87
EXP-03	1	352	32	32	26	0	8	32 / 37,2 / 238
EXP-03	2	348	67	67	46	0	16	44 / 46,1 / 257
EXP-04	1	377	0	0	0	0	—	0 / 0 / 0
EXP-04	2	380	0	0	0	0	—	0 / 0 / 0
EXP-05	1	350	56	56	53	0	8	41 / 43,3 / 240
EXP-05	2	349	51	51	43	0	16	39 / 41,3 / 299

Tabel 12.14: Log serial Modul 09: Statistik akses kanal per node

[DROP] **not di seluruh sesi**: bahkan pada beban tertinggi, lima percobaan selalu cukup untuk memperoleh kanal. Untuk memaksa [DROP] muncul, perkecil MAX_ATTEMPT atau tambah node ketiga (lihat CH-2).

RSSI & SNR di gateway

Sesi	Node 1	Node 2
EXP-02	-44,3 dBm / 9,38 dB	-55,8 dBm / 9,63 dB
EXP-03	-44,7 dBm / 9,77 dB	-54,5 dBm / 9,73 dB
EXP-04	-43,3 dBm / 8,58 dB	-54,0 dBm / 9,63 dB
EXP-05	-44,5 dBm / 9,77 dB	-56,1 dBm / 9,85 dB

Tabel 12.15: Log serial Modul 09: RSSI & SNR di gateway

Selisih ± 11 dB antar node murni posisi fisik di meja. SNR Node 1 yang turun $\sim 1,2$ dB khusus di EXP-04 adalah jejak tabrakan: paket yang tetap berhasil didekode pun sebagian tercemar sinyal node lain.

EXP-02 – Boot dan peristiwa backoff pertama

Cuplikan 1,5 detik pertama. Ketiga board menyala hampir bersamaan; kedua node melaporkan lantai derau sekitar -110 dBm. Perhatikan Node 2: pengukuran kalibrasinya sendiri sudah “tercemar” transmisi Node 1 (maks -30 dBm), lalu ia langsung mendeteksi kanal sibuk, mundur 2 slot, membekukan pencacahnya, dan baru mengirim 136 ms setelah paket Node 1 tiba.

```
[11:12:28.633] N1 | === LoRa CSMA/CA - NODE 1 ===
[11:12:28.649] N1 | Init LoRa ... OK
[11:12:28.653] N1 | Freq: 433.00 MHz
[11:12:28.658] N1 | Carrier sense: RSSI (ambang -95 dBm) | DIFS 30 ms | slot 20 ms | CW 4..64
[11:12:28.666] N1 | Peran: NODE (CSMA/CA) -- dengar dulu, mundur acak, baru kirim
[11:12:28.670] N1 | Tanpa ACK: node tahu kanal sepi, tetap tidak tahu paketnya sampai
[11:12:28.687] GW | === LoRa CSMA/CA - GATEWAY ===
[11:12:28.707] GW | Init LoRa ... OK
[11:12:28.707] GW | Freq: 433.00 MHz
[11:12:28.715] GW | Peran: GATEWAY (CSMA/CA) -- hanya mendengar, tanpa polling, tanpa ACK
[11:12:28.715] N2 | === LoRa CSMA/CA - NODE 2 ===
[11:12:28.718] GW | Menunggu paket dari Node 1 & Node 2...
```



```

[11:12:28.718] GW |
[11:12:28.735] N2 | Init LoRa ... OK
[11:12:28.735] N2 | Freq: 433.00 MHz
[11:12:28.743] N2 | Carrier sense: RSSI (ambang -95 dBm) | DIFS 30 ms | slot 20 ms | CW 4..64
[11:12:28.747] N2 | Peran: NODE (CSMA/CA) -- dengar dulu, mundur acak, baru kirim
[11:12:28.755] N2 | Tanpa ACK: node tahu kanal sepi, tetap tidak tahu pakatnya sampai
[11:12:29.677] N1 | [KALIBRASI] lantai derau 200 sampel: min -117 | rata-rata -110 | maks -105 dBm
[11:12:29.685] N1 | [KALIBRASI] ambang terpakai sekarang: -95 dBm -- lihat EXP-01
[11:12:29.686] N1 |
[11:12:29.763] N2 | [KALIBRASI] lantai derau 200 sampel: min -118 | rata-rata -110 | maks -30 dBm
[11:12:29.767] N2 | [KALIBRASI] ambang terpakai sekarang: -95 dBm -- lihat EXP-01
[11:12:29.767] N2 |
[11:12:29.771] N2 | [CS] kanal SIBUK (RSSI -33 dBm)
[11:12:29.775] N2 | [BACKOFF] percobaan 1/5 | CW=4 | slot=2 -> 40 ms
[11:12:29.776] N1 | [TX] NODE=1,SEQ=0,T=27.1,H=70.5 | attempt=1 | tunda=31 ms
[11:12:29.779] GW | === PAKET DITERIMA ===
[11:12:29.779] N2 | [FREEZE] pencacah backoff dibekukan -- kanal terpakai
[11:12:29.780] GW | Node : 1
[11:12:29.780] GW | SEQ : 0
[11:12:29.783] N1 | [STAT] TX=1 | DROP=0 | kanal sibuk=0 | rata-rata tunda=31 ms
[11:12:29.784] GW | Suhu : 27.1 C
[11:12:29.784] GW | Lembab : 70.5 %
[11:12:29.784] GW | RSSI : -45 dBm
[11:12:29.784] N1 |
[11:12:29.788] GW | SNR : 9.25 dB
[11:12:29.792] GW | Statistik Node 1: diterima=1 | perkiraan hilang=0
[11:12:29.796] GW | Total diterima gateway: 1
[11:12:29.796] GW | =====
[11:12:29.796] GW |
[11:12:29.915] GW | === PAKET DITERIMA ===
[11:12:29.915] GW | Node : 2
[11:12:29.915] N2 | [TX] NODE=2,SEQ=0,T=27.1,H=71.0 | attempt=2 | tunda=84 ms
[11:12:29.918] N2 | [STAT] TX=1 | DROP=0 | kanal sibuk=1 | rata-rata tunda=84 ms
[11:12:29.918] N2 |
[11:12:29.919] GW | SEQ : 0
[11:12:29.919] GW | Suhu : 27.1 C
[11:12:29.919] GW | Lembab : 71.0 %
[11:12:29.923] GW | RSSI : -55 dBm
[11:12:29.923] GW | SNR : 9.50 dB
[11:12:29.927] GW | Selang : 136 ms dari paket sebelumnya
[11:12:29.931] GW | Statistik Node 2: diterima=1 | perkiraan hilang=0
[11:12:29.935] GW | Total diterima gateway: 2
[11:12:29.935] GW | =====
[11:12:29.935] GW |

```

Baris kuncinya:

```

[11:12:29.771] N2 | [CS] kanal SIBUK (RSSI -33 dBm) <- telinga bekerja
[11:12:29.775] N2 | [BACKOFF] percobaan 1/5 | CW=4 | slot=2 -> 40 ms
[11:12:29.776] N1 | [TX] NODE=1,SEQ=0,... | attempt=1 | tunda=31 ms
[11:12:29.779] N2 | [FREEZE] pencacah backoff dibekukan <- N1 masih mengudara
[11:12:29.915] N2 | [TX] NODE=2,SEQ=0,... | attempt=2 | tunda=84 ms
[11:12:29.927] GW | Selang : 136 ms dari paket sebelumnya

```

Tunda akses Node 1 = 31 ms, yaitu DIFS saja (kanal memang sepi). Tunda akses Node 2 = 84 ms: DIFS + backoff + waktu beku menunggu Node 1 selesai.

EXP-03 – Beban tinggi: contention window melebar

Interval kirim dipersempit menjadi 300–500 ms pada kedua node. Cuplikan berikut memperlihatkan CW menggandakan diri 4 → 8 → 16 karena kanal ditemukan sibuk tiga kali berturut-turut, sampai akhirnya Node 2 memperoleh kanal pada percobaan ke-4 dengan tunda 155 ms:

```

[11:02:03.950] N2 | [BACKOFF] percobaan 1/5 | CW=4 | slot=0 -> 0 ms
[11:02:03.954] N2 | [CS] kanal SIBUK (RSSI -31 dBm)
[11:02:03.958] N2 | [BACKOFF] percobaan 2/5 | CW=8 | slot=0 -> 0 ms
[11:02:03.962] N2 | [CS] kanal SIBUK (RSSI -32 dBm)
[11:02:03.966] N2 | [BACKOFF] percobaan 3/5 | CW=16 | slot=3 -> 60 ms
[11:02:03.970] N2 | [FREEZE] pencacah backoff dibekukan -- kanal terpakai
[11:02:04.009] GW | === PAKET DITERIMA ===
[11:02:04.010] N1 | [TX] NODE=1,SEQ=231,T=29.1,H=67.5 | attempt=1 | tunda=31 ms
[11:02:04.013] GW | Node : 1
[11:02:04.013] GW | SEQ : 231
[11:02:04.014] GW | Suhu : 29.1 C
[11:02:04.014] N1 | [STAT] TX=232 | DROP=0 | kanal sibuk=19 | rata-rata tunda=37 ms
[11:02:04.014] N1 |
[11:02:04.018] GW | Lembab : 67.5 %
[11:02:04.018] GW | RSSI : -45 dBm

```

```

[11:02:04.021] GW | SNR      : 9.75 dB
[11:02:04.026] GW | Selang   : 409 ms dari paket sebelumnya
[11:02:04.030] GW | Statistik Node 1: diterima=232 | perkiraan hilang=0
[11:02:04.030] GW | Total diterima gateway: 457
[11:02:04.034] GW | =====
[11:02:04.034] GW |
[11:02:04.166] N2 | [TX] NODE=2,SEQ=228,T=29.7,H=72.1 | attempt=4 | tunda=155 ms
[11:02:04.169] GW | === PAKET DITERIMA ===
[11:02:04.169] GW | Node      : 2
[11:02:04.169] GW | SEQ       : 228
[11:02:04.171] N2 | [STAT] TX=229 | DROP=0 | kanal sibuk=47 | rata-rata tunda=44 ms
[11:02:04.171] N2 |
[11:02:04.173] GW | Suhu      : 29.7 C
[11:02:04.173] GW | Lembab    : 72.1 %
[11:02:04.173] GW | RSSI      : -55 dBm
[11:02:04.177] GW | SNR       : 9.50 dB
[11:02:04.181] GW | Selang    : 155 ms dari paket sebelumnya
[11:02:04.186] GW | Statistik Node 2: diterima=226 | perkiraan hilang=3
[11:02:04.189] GW | Total diterima gateway: 458
[11:02:04.189] GW | =====
[11:02:04.190] GW |

```

Meski kanal diperebutkan sepanjang sesi, **tidak satu pun [WARN]** muncul di gateway: setiap paket yang berhasil naik ke udara tiba utuh dan terbaca penuh.

EXP-04 — Telinga dimatikan (pembanding Pure ALOHA)

Kedua node dibangun dengan `-DCS_MODE=2`, interval tetap 300–500 ms. Tunda akses jatuh ke **0 ms** — dan konsekuensinya langsung terlihat di gateway. Cuplikan di bawah menangkap satu paket Node 2 yang bertabrakan di udara: awalnya masih utuh terbaca (NODE=2, SEQ=1, T=28.), sisanya hancur.

```

[10:56:40.865] N2 | [TX] NODE=2,SEQ=1,T=28.4,H=72.1 | attempt=1 | tunda=0 ms
[10:56:40.866] GW | [WARN] Paket cacat (field tidak lengkap): NODE=2,SEQ=1,T=28.t,??,a?2
[10:56:40.869] N2 | [STAT] TX=2 | DROP=0 | kanal sibuk=0 | rata-rata tunda=0 ms
[10:56:40.869] N2 |
[10:56:40.913] N1 | [TX] NODE=1,SEQ=1,T=30.4,H=72.0 | attempt=1 | tunda=0 ms
[10:56:40.917] N1 | [STAT] TX=2 | DROP=0 | kanal sibuk=0 | rata-rata tunda=0 ms
[10:56:40.917] N1 |

```

Sepanjang sesi ini gateway mencatat **102 [GAP] dan 37 paket cacat** — bandingkan dengan 4 [GAP] dan 0 cacat pada EXP-03 yang mengirim jumlah paket serupa.

Paket cacat semacam inilah yang memaksa gateway memeriksa kelengkapan field sebelum mempercayai isinya. Tanpa pemeriksaan itu, SEQ yang hilang dibaca sebagai 0 dan satu paket rusak saja sanggup merusak statistik seluruh sesi — pada percobaan pertama sebelum perbaikan, perkiraan kehilangan Node 2 melonjak ke **1177** padahal kenyataannya sekitar 30:

```

[10:54:09.727] GW | Selang   : 387 ms dari paket sebelumnya
[10:54:09.735] GW | [GAP] SEQ meloncat 272 -- paket dibuang node atau bertabrakan
[10:54:09.739] GW | Statistik Node 2: diterima=233 | perkiraan hilang=1177

```

EXP-05 — Carrier sense berbasis CAD

Kedua node dibangun dengan `-DCS_MODE=1`. Node tidak lagi memakai ambang dBm apa pun; SX1276 sendiri yang memutuskan ada-tidaknya simbol LoRa di kanal.

```

[11:09:12.395] N1 | [CS] kanal SIBUK (CAD mendeteksi sinyal LoRa)
[11:09:12.400] N1 | [BACKOFF] percobaan 1/5 | CW=4 | slot=1 -> 20 ms
[11:09:12.407] N1 | [FREEZE] pencacah backoff dibekukan -- kanal terpakai
[11:09:12.419] GW | === PAKET DITERIMA ===
[11:09:12.422] N2 | [TX] NODE=2,SEQ=19,T=31.4,H=65.2 | attempt=1 | tunda=31 ms
[11:09:12.423] GW | Node      : 2
[11:09:12.423] GW | SEQ       : 19
[11:09:12.423] GW | Suhu      : 31.4 C
[11:09:12.426] N2 | [STAT] TX=20 | DROP=0 | kanal sibuk=5 | rata-rata tunda=46 ms
[11:09:12.426] N2 |
[11:09:12.427] GW | Lembab    : 65.2 %
[11:09:12.427] GW | RSSI      : -56 dBm
[11:09:12.431] GW | SNR       : 9.75 dB
[11:09:12.435] GW | Selang    : 412 ms dari paket sebelumnya
[11:09:12.439] GW | Statistik Node 2: diterima=20 | perkiraan hilang=0
[11:09:12.439] GW | Total diterima gateway: 39
[11:09:12.443] GW | =====
[11:09:12.443] GW |
[11:09:12.537] GW | === PAKET DITERIMA ===

```

```

[11:09:12.538] GW | Node : 1
[11:09:12.538] GW | SEQ : 19
[11:09:12.538] N1 | [TX] NODE=1,SEQ=19,T=30.0,H=65.7 | attempt=2 | tunda=79 ms
[11:09:12.542] GW | Suhu : 30.0 C
[11:09:12.542] GW | Lembab : 65.7 %
[11:09:12.542] N1 | [STAT] TX=20 | DROP=0 | kanal sibuk=1 | rata-rata tunda=32 ms
[11:09:12.542] N1 |
[11:09:12.546] GW | RSSI : -44 dBm
[11:09:12.546] GW | SNR : 9.50 dB
[11:09:12.550] GW | Selang : 116 ms dari paket sebelumnya
[11:09:12.554] GW | Statistik Node 1: diterima=20 | perkiraan hilang=0
[11:09:12.559] GW | Total diterima gateway: 40
[11:09:12.559] GW | =====
[11:09:12.559] GW |

```

Hasil akhirnya setara RSSI (694 vs 695 paket tiba, sama-sama 4 hilang), tetapi CAD **melaporkan kanal sibuk lebih sering**: 107 kali berbanding 99 kali pada beban yang sama persis. Ini konsisten dengan sifat CAD yang mengenali sinyal LoRa sampai di bawah lantai derau, sementara ambang RSSI -95 dBm melewati sinyal lemah.

Satu catatan penting untuk yang mengutak-atik mode ini: pada percobaan pertama, CAD sama sekali tidak pernah mendeteksi apa pun ([CS] kanal SIBUK = 0) **dan** merusak transmisi node itu sendiri — 103 dari 440 paket tiba dalam keadaan cacat. Dua sebabnya, keduanya sudah diperbaiki di kode sekarang:

1. **CAD harus dimasuki dari STANDBY.** Perintah CAD yang dikirim selagi modem masih di RX kontinu tidak pernah dijalankan; CadDone tidak pernah naik dan setiap pemeriksaan hanya berakhir di timeout 50 ms (terlihat dari tunda akses yang konstan 50 ms).
2. **isTransmitting() di library menganggap mode CAD sebagai “sedang TX”.** Fungsi itu menguji (RegOpMode & 0x03) == 0x03, dan mode CAD bernilai 0x07 lolos uji tersebut. Bila modem kebetulan masih berada di CAD saat beginPacket() dipanggil, beginPacket() gagal diam-diam tanpa mereset FIFO — dan yang naik ke udara adalah sampah. Karena itu LoRa.idle() dipanggil sebelum beginPacket().

Kedua jebakan itu tidak akan pernah terlihat dari dokumentasi library, sebab library ini memang tidak menyediakan CAD sama sekali.

12.9 Pengukuran

A. Beban kanal terhadap perilaku CSMA/CA. Baris bertanda OK adalah hasil verifikasi 23 Agustus 2026; baris kosong dijalankan sendiri.

Interval (ms)	Diterima gateway (3 menit)	[BACKOFF]	[DROP]	Tunda rata-rata (ms)
2000–5000 (OK)	98 dari 99 dikirim	3	0	31,7 / 34,3
1000–2000				
300–500 (OK)	695 dari 700 dikirim	99	0	37,2 / 46,1

Tabel 12.16: Lembar pengukuran A — beban kanal terhadap perilaku CSMA/CA

B. RSSI, SNR, dan tunda akses per node — hasil verifikasi pada beban tinggi (interval 300–500 ms, sesi EXP-03).

Node	RSSI rata-rata (dBm)	SNR rata-rata (dB)	Tunda akses rata-rata (ms)	[GAP]
Node 1	– 44,7	9,77	37,2 (maks 238)	0
Node 2	– 54,5	9,73	46,1 (maks 257)	4

Tabel 12.17: Lembar pengukuran B — RSSI, SNR, dan tunda akses per node

Selisih ± 10 dB antar node murni posisi fisik di meja. Menarik: node yang sinyalnya lebih lemah di gateway (Node 2) justru yang lebih sering menemukan kanal sibuk dan paling banyak mundur — periksa apakah pola itu terulang pada pengukuran sendiri.

C. CSMA/CA dibanding kirim buta (dari EXP-04)

Metrik	Dengan CS	Tanpa CS	Selisih
Paket dikirim node	700	757	+57 tanpa CS
Paket tiba di gateway	695	581	– 114 tanpa CS
Paket hilang (perkiraan dari SEQ)	4 (0,6 %)	175 (23,1 %)	44× lebih banyak
Paket cacat di gateway	0	37	—
Tunda akses rata-rata	~ 41 ms	0 ms	harga yang dibayar
Paket dibuang sebelum berangkat	0	0	—

Tabel 12.18: Lembar pengukuran C — CSMA/CA dibanding kirim buta

12.10 Analisis

1. Hitung waktu udara satu paket modul ini pada SF7, BW 125 kHz, CR 4/5, lalu bandingkan dengan DIFS_MS dan SLOT_MS. Apakah satu slot cukup panjang untuk membedakan kanal yang benar-benar bebas dari sela antar-simbol? Bila SLOT_MS dibuat 5 ms, apa yang diperkirakan terjadi?
2. Dari Tabel 12.18, hitung berapa milidetik tunda akses yang harus dibayar untuk setiap satu paket yang berhasil diselamatkan dari tabrakan. Di aplikasi seperti apa harga itu murah, dan di aplikasi seperti apa terlalu mahal?
3. Pada verifikasi dengan dua node, [DROP] tidak pernah muncul sekali pun — lima percobaan selalu cukup. Perkirakan (dengan hitungan, bukan tebakan) berapa node atau berapa MAX_ATTEMPT yang diperlukan agar [DROP] mulai terjadi pada interval 300–500 ms, lalu uji perkiraan itu. Jelaskan juga mengapa node lebih baik membuang paket daripada terus mencoba tanpa batas.
4. Node menaikkan SEQ juga untuk paket yang dibuangnya sendiri. Jelaskan apa yang akan hilang dari kemampuan analisis gateway seandainya SEQ hanya naik saat paket benar-benar terkirim.
5. Gateway menolak paket yang field-nya tidak lengkap. Baca logserial.md bagian EXP-04: tanpa pemeriksaan itu, satu paket rusak membuat perkiraan kehilangan melonjak dari ~ 30 menjadi 1177. Telusuri persis bagaimana satu paket cacat bisa menghasilkan angka sebesar itu, lalu usulkan satu pemeriksaan tambahan yang membuat statistik gateway lebih tahan banting.
6. Gateway modul ini tidak pernah memancar sama sekali. Bila kelak ia mulai mengirim ACK (modul berikutnya), sebutkan **dua** hal yang berubah bagi mekanisme carrier sense di node.

TODO

Penomoran daftar Analisis pada berkas sumber (Modul09_lora_csma_ca/README.md) melompat: butir bernomor 6 ditulis sebelum butir bernomor 5. Urutan isinya dipertahankan apa adanya di sini dan penomorannya dibuat otomatis, sehingga butir “gateway menolak paket cacat” kini menjadi nomor 5 dan “gateway tidak pernah memancar” menjadi nomor 6. Mohon dikonfirmasi apakah urutan itu memang yang dikehendaki, atau kedua butir perlu ditukar posisinya.

12.11 Concept Check

1. Apa beda mendasar CSMA/CA di modul ini dengan polling terjadwal pada M05 — siapa yang memegang keputusan “kapan boleh bicara” pada masing-masing?
2. Mengapa radio tidak bisa memakai *collision detection* seperti Ethernet, sehingga harus puas dengan *collision avoidance*?
3. Mengapa backoff harus acak? Apa yang terjadi bila semua node memakai lama tunggu tetap yang sama?

4. Mengapa jendela kontensi dilebarkan setiap kali kanal ditemukan sibuk, alih-alih dipertahankan tetap?
5. Jelaskan masalah *hidden node*: dua node patuh mendengarkan, tetapi paketnya tetap bertabrakan di gateway. Bagaimana ini bisa terjadi, dan apakah menaikkan MAX_ATTEMPT menolong?
6. CAD dan RSSI sama-sama menjawab “kanal sibuk atau tidak”. Sebutkan satu keadaan di mana keduanya memberi jawaban berbeda, dan mana yang lebih tepat pada keadaan itu.

12.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Telinga yang mengkalibrasi dirinya sendiri

CH-1 — Ambang adaptif Buat node mengukur lantai derau sendiri saat `setup()` (rata-rata puluhan sampel RSSI pada kanal kosong), lalu menetapkan `RSSI_THRESHOLD` sebagai lantai derau ditambah margin tetap. Bandingkan hasilnya dengan ambang manual dari EXP-01.

CH-2 — Node ketiga Tambahkan environment `node3` dan jalankan EXP-03 dengan tiga node. Ukur bagaimana tunda akses rata-rata dan `[DROP]` berubah ketika jumlah peserta naik dari dua menjadi tiga pada beban yang sama.

Level 3 — Tantangan · Prioritas dan pemetaan kesibukan kanal

CH-3 — Prioritas lewat DIFS Beri Node 1 rentang tunggu yang lebih pendek daripada Node 2 (misalnya DIFS 20 ms vs 40 ms), lalu buktikan dari data gateway bahwa Node 1 memperoleh porsi kanal yang lebih besar. Ini adalah versi sederhana dari mekanisme prioritas 802.11e.

CH-4 — Laporkan kesibukan Sisipkan jumlah backoff dan tunda akses paket ini ke dalam payload (`,B0=<n>,DLY=<ms>`), lalu buat gateway mencetak peta kesibukan kanal dari sudut pandang masing-masing node. Perhatikan bahwa parser gateway tidak perlu diubah untuk field yang tidak dikenalnya — buktikan klaim itu sebelum menambahkan pembacaannya.

12.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — CS/MA/CA, DIFS, contention window, exponential backoff, freeze, hidden node.
3. Konfigurasi — format payload, parameter CSMA/CA yang dipakai, ambang RSSI hasil kalibrasi EXP-01, parameter radio.
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-05 beserta checkpoint).
5. Data pengukuran — Tabel 12.16, Tabel 12.17, dan Tabel 12.18.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri, khususnya mengenai harga yang dibayar (tunda akses dan paket yang dibuang) untuk keandalan yang diperoleh, dan mengapa jaringan nyata tetap memilih membayarnya.

13. Modul 10 — Slot Terjadwal: Slotted ALOHA dan TDMA

LoRa COMMUNICATION LAB · MODUL 10

Take a Number — Slot Terjadwal: Slotted ALOHA dan TDMA

Arduino Uno + Dragino LoRa Shield v1.2 · topologi bintang, 2 node + gateway · interrupt + SYNC + slot · level Advanced · 1 × 50 menit · folder kode Modul10_lora_slotted_aloha_tdma

PENGANTAR MODUL · Gambaran Umum

Modul 10 dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat lanjut, dan menutup arc akses kanal yang dimulai di M08. Misinya mengganti akar masalah, bukan menambalnya lagi: M08C menaikkan keandalan dengan mencoba ulang setelah gagal, dan M09 menurunkan peluang tabrakan dengan mendengarkan kanal lebih dahulu — tetapi pada keduanya tabrakan tetap **mungkin** terjadi. Modul ini justru **mencegah** tabrakan sejak awal dengan membagi waktu menjadi slot-slot tetap dan memberi tiap node jatah bicaranya sendiri.

13.1 Pendahuluan

Carrier sense pun tidak menjamin apa-apa bila dua node kebetulan mendapati kanal sepi pada saat yang sama, atau bila keduanya tidak saling mendengar. Gagasan slot ini sama tuanya dengan Pure ALOHA, dikembangkan tak lama setelahnya dengan nama **Slotted ALOHA** (Roberts, 1972).

PENTING

Modul ini sebenarnya memuat dua protokol, dan namanya berbeda. Mode Random (A) adalah Slotted ALOHA — lebih tepatnya **Framed Slotted ALOHA** , karena SYNC dari gateway membuka satu *frame* berisi SLOT_COUNT slot dan tiap node mengundi satu slot di dalamnya (varian yang sama dipakai anti-collision RFID EPC Gen2). Waktu sudah berslot, tetapi node masih **mengundi** , sehingga tabrakan berkurang tanpa pernah hilang. Mode Assigned (B) melangkah satu tingkat lebih jauh dan berhenti menjadi ALOHA sama sekali — begitu tiap node punya slot tetap miliknya sendiri, yang berlaku adalah **TDMA** (Time Division Multiple Access), dan tabrakan hilang secara struktural, bukan berkurang secara statistik. Perbedaan itulah yang diukur di EXP-01 dan EXP-02.

M05 dan M07 juga mencegah tabrakan, tetapi lewat **master yang memanggil satu per satu** (polling terpusat) — master harus aktif bertanya, node hanya menjawab. Modul ini mencegah tabrakan dengan cara yang berbeda: gateway hanya menyiarkan detak waktu bersama (SYNC), dan setiap node **sendiri** yang menghitung kapan gilirannya lalu mengirim tanpa diminta. Perbedaan ini penting — inilah yang membedakan penjadwalan **terpusat aktif** (M05 dan M07, gateway mengatur giliran tiap saat) dari penjadwalan **terdesentralisasi pasif** (M10, gateway hanya menjaga detak, node yang menghitung sendiri). Dua percobaan pada modul ini, Mode Assigned dan Mode Random, menunjukkan bahwa slot saja tidak otomatis menghapus tabrakan — **kepastian** jadwal itulah yang menghapusnya, dan Mode Random sengaja dibiarkan tanpa kepastian itu untuk membuktikannya.

Prasyaratnya adalah M08C untuk kontrak NODE= dan ACK= serta pola tunggu-ACK-dengan-timeout, serta M05 untuk gagasan bahwa penjadwalan (bukan retry) adalah solusi struktural terhadap tabrakan pada kanal bersama. Yang dibangun di sini adalah siaran waktu bersama (SYNC=<cycle>), penghitungan slot di sisi node berdasarkan saat SYNC diterima, dan dua mode pemilihan slot yang dibandingkan langsung. **Retry dan random backoff dari M08C sengaja dihapus** — bukan lupa, melainkan digantikan sepenuhnya oleh slot: satu percobaan per siklus, dan siklus berikutnya yang mengambil alih peran “coba lagi”.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
05	Tabrakan dicegah lewat polling terpusat — master memanggil satu per satu
08	Penjadwalan dilepas — node kirim bebas, tabrakan senyap diamati
08B	ACK ditempelkan di atas M08 — node tahu SUCCESS/FAILED, belum ada retry
08C	Random backoff + retry — kegagalan dipulihkan, dan Pure ALOHA menjadi lengkap
09	Carrier sense — dengar dulu sebelum bicara, tabrakan dihindari sebelum terjadi
10 (ini)	Slot waktu bersama — Slotted ALOHA bila slot diundi, TDMA bila slot tetap

Tabel 13.1: Peta modul pada arc kedua seri LoRa

Kontrak data lab ini. Payload data dan format ACK **identik** dengan M08B dan M08C: `NODE=<id>,SEQ=<n>,R1T=.,R1H=.,R2T=.,R2H=.` dibalas `ACK=<id>,SEQ=<n>`. Yang baru adalah siaran `SYNC=<cycle>` dari gateway di awal tiap siklus, dan **hilangnya** retry: SEQ kini naik di **setiap** siklus tanpa peduli hasilnya — persis seperti M08B, bukan seperti M08C yang menahan SEQ selama retry berlangsung. Alasannya sederhana: pada modul ini tidak ada lagi retry dalam satu siklus untuk ditahan; data yang gagal pada satu slot betul-betul dianggap selesai, dan pembacaan sensor berikutnya sudah menunggu di siklus sesudahnya.

13.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Mencegah tabrakan lewat pembagian waktu

1. Menjelaskan mekanisme slot waktu: waktu dibagi slot tetap, tiap node mengirim hanya di dalam jatah slotnya, dihitung dari referensi waktu bersama (SYNC).
2. Membedakan penjadwalan terpusat aktif (M05, master memanggil) dari penjadwalan terdesentralisasi pasif (M10, gateway hanya menyiarkan detak waktu).
3. Menerapkan dan membandingkan dua strategi pemilihan slot: **Assigned** (tetap, Mode B → TDMA) dan **Random** (diundi tiap siklus, Mode A → Slotted ALOHA), serta menjelaskan mengapa hanya salah satunya yang menghapus tabrakan secara struktural.
4. Menjelaskan mengapa vulnerable period Slotted ALOHA hanya **satu kali** waktu udara paket, bukan dua kali seperti Pure ALOHA (M08), dan menghubungkannya dengan throughput teoretis puncak 36,8 %.
5. Menjelaskan mengapa retry dan random backoff (M08C) tidak lagi diperlukan begitu slot terjadwal dengan pasti, dan kapan kombinasi keduanya (slot + retry) tetap masuk akal.

Kriteria keberhasilan

- ☐ Kedua node menerima SYNC dan mencetak `[TX] cycle=... slot=...` pada waktu yang konsisten relatif terhadap SYNC yang diterima.
- ☐ Pada Mode A (Random / Slotted ALOHA), `[FAIL]` dan `[GAP]` **muncul**, dan proporsi siklus “slot sama” mendekati $1/\text{SLOT_COUNT}$.
- ☐ Pada Mode B (Assigned / TDMA), `[GAP]` nihil dan tiap node konsisten memakai slotnya sendiri.
- ☐ Kedua mode diukur pada sesi yang sama sehingga angkanya dapat dibandingkan langsung.
- ☐ Gateway mencetak ringkasan `--- Cycle N selesai ---` di akhir tiap siklus dengan statistik per node.

13.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Slot	Jendela waktu tetap (SLOT_DURATION_MS) tempat satu node boleh mengirim. Modul ini memakai 2 slot per siklus.
SYNC	Siaran gateway di awal siklus, menjadi referensi waktu bersama bagi seluruh node.
Mode Assigned (B) = TDMA	Tiap node memakai slot tetap (NODE_ID - 1). Tidak ada dua node berbagi slot yang sama, sehingga skema ini bukan lagi ALOHA.
Mode Random (A) = Slotted ALOHA	Tiap node mengundi ulang slotnya setiap siklus. Dua node bisa kebetulan memilih slot yang sama — inilah ALOHA berslot yang sesungguhnya.
Vulnerable period (slotted)	Hanya satu kali waktu udara paket — sebab paket lain yang boleh mengirim wajib menunggu batas slot berikutnya, tidak bisa mulai di tengah slot yang sedang berjalan.
Throughput teoretis	$S = Ge^{-G}$, puncak $\approx 36,8\%$ pada $G = 1$ — dua kali lipat Pure ALOHA (18,4 %) karena vulnerable period lebih sempit.

Tabel 13.2: Istilah kerja Modul 10

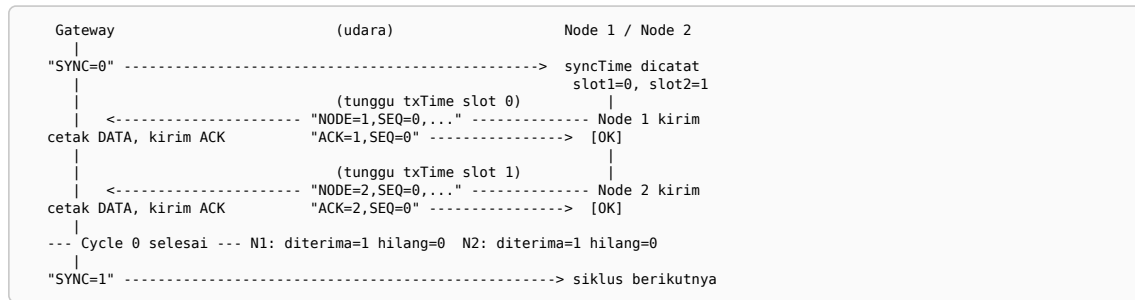
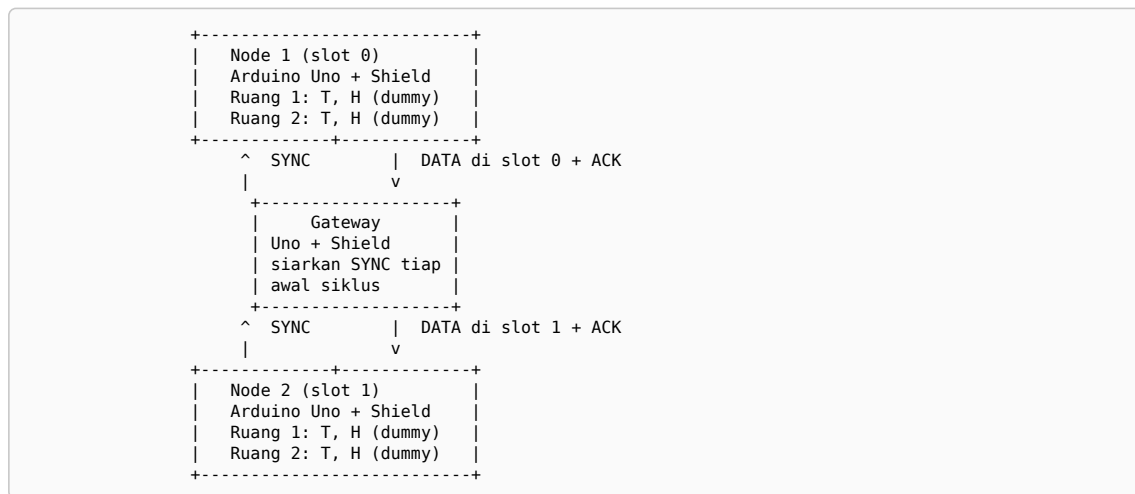
Mengapa vulnerable period Slotted ALOHA setengah dari Pure ALOHA. Pada M08, paket B dapat mulai kapan saja selama paket A mengudara, sehingga rentang rawan selebar dua kali waktu udara. Pada Slotted ALOHA, setiap node **wajib** menunggu batas slot berikutnya sebelum boleh mengirim — tidak ada yang bisa “menyela di tengah”. Akibatnya, dua paket hanya bertabrakan bila keduanya mulai pada **slot yang sama persis**, bukan pada rentang waktu mana pun yang saling tumpang tindih. Itulah sebabnya throughput puncaknya dua kali lipat Pure ALOHA meski sama-sama tanpa carrier sense.

Satu hal yang tetap hilang di kedua mode: retransmisi. Slotted ALOHA klasik mengirim ulang paket yang bertabrakan pada slot berikutnya; modul ini sengaja menghapus retry M08C sepenuhnya, sehingga paket yang gagal betul-betul hilang dan digantikan pembacaan baru pada siklus berikutnya. Jadi Mode A adalah Slotted ALOHA **tanpa** pemulihan — sejajar dengan M08 yang ALOHA tanpa umpan balik. Menambahkan retry ke dalamnya adalah bahan Challenge CH-2.

Kapan sebuah skema berhenti disebut ALOHA. Ciri keluarga ALOHA adalah **keacakan**: node memutuskan sendiri kapan mengirim, dan tabrakan ditangani setelah terjadi, bukan dicegah. Slotted ALOHA mempersempit peluang tabrakan dengan menyerempakkan awal pengiriman, tetapi tetap mengundi — itu sebabnya throughput puncaknya berhenti di 36,8 %. Begitu undian diganti jadwal tetap, tidak ada lagi yang diacak, tidak ada lagi tabrakan yang perlu ditangani, dan namanya berubah menjadi TDMA. Jadi tabel Pengukuran modul ini sebenarnya membandingkan **dua protokol berbeda**, bukan dua setelan dari protokol yang sama.

Mengapa Mode Assigned menghapus tabrakan, sedangkan Mode Random tidak. Dengan SLOT_COUNT sama dengan jumlah node (2 slot, 2 node) dan tiap node memakai slot tetap yang berbeda, **tidak ada** kombinasi kejadian yang membuat keduanya memilih slot sama — tabrakan antar-node dihapus secara struktural, selama SYNC diterima dengan benar oleh keduanya. Mode Random sengaja meniadakan kepastian itu: tiap siklus, node kedua punya peluang $1/\text{SLOT_COUNT}$ menabrak pilihan node pertama — **50 %** pada modul ini yang hanya punya 2 slot, dan pengukuran EXP-01 mendapat 44,9 %. Peluang itu turun begitu slot diperbanyak (33 % untuk 3 slot, 25 % untuk 4), tetapi tidak pernah menjadi nol selama slotnya masih diundi.

Mengapa retry M08C dihapus, bukan digabung begitu saja. Menggabungkan retry-dengan-backoff ke dalam slot yang sudah terjadwal akan merusak jadwal itu sendiri — backoff acak bisa mendorong pengiriman ulang melampaui batas slot node itu sendiri dan masuk ke slot node lain. Solusi yang benar bukan memaksakan retry ke dalam slot, melainkan membiarkan slot berikutnya (siklus SYNC berikutnya) mengambil alih perannya.

Sekuens yang diamati (Mode B, dua siklus)**13.4 Topologi**

Node	Environment	Peran	Slot (Mode B)	Slot (Mode A)
Node 1	node1	Kirim dummy Ruang 1+2 pada slotnya	tetap 0	diundi tiap siklus
Node 2	node2	Kirim dummy Ruang 1+2 pada slotnya	tetap 1	diundi tiap siklus
Gateway	gateway	Siarkan SYNC, dengarkan tiap slot, balas ACK	—	—

Tabel 13.3: Peran tiap node Modul 10

13.5 Alat yang Digunakan

Modul ini dijalankan di atas Arduino Uno (ATmega328P) dengan Dragino LoRa Shield v1.2 (SX1276), memakai PlatformIO dan library LoRa karya sandeepmistry.

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	3
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	3
3	Antena SMA	wajib terpasang sebelum diberi daya	3
4	Kabel USB tipe B	kabel data	3

Tabel 13.4: Alat dan bahan Modul 10

Struktur proyek

```

Modul10_lora_slotted_aloha_tdma/
├── platformio.ini      ← [slot] mode = 1 (Slotted ALOHA) / 0 (TDMA), count = jumlah slot
├── lora_monitor.py     ← dashboard 3-panel live (Gateway/Node1/Node2) + CSV
├── upload_auto.py      ← deteksi port otomatis saat unggah
├── logserial.md        ← cuplikan log serial aktual dari pengujian perangkat
├── src/
│   ├── node/main.cpp   ← dummy Ruang 1+2, hitung slot dari SYNC (env node1, node2)
│   └── gateway/main.cpp ← siarkan SYNC, terima tiap slot, balas ACK (env gateway)

```

13.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul10_lora_slotted_aloha_tdma/](#)

- [Modul10_lora_slotted_aloha_tdma/platformio.ini](#)
- [Modul10_lora_slotted_aloha_tdma/src/node/main.cpp](#)
- [Modul10_lora_slotted_aloha_tdma/src/gateway/main.cpp](#)
- [Modul10_lora_slotted_aloha_tdma/lora_monitor.py](#)
- [Modul10_lora_slotted_aloha_tdma/upload_auto.py](#)

```

[platformio]
default_envs = gateway

; — MODE SLOT: satu-satunya baris yang diganti antara EXP-01 dan EXP-02 —
; Keduanya memakai infrastruktur yang sama persis (SYNC, slot, guard time);
; yang berbeda hanya kebijakan siapa memakai slot mana -- dan justru di situlah
; protokolnya berganti nama:
;
; mode = 1 -> Mode A, slot diundi tiap siklus = Slotted ALOHA (EXP-01)
; mode = 0 -> Mode B, slot tetap per node = TDMA (EXP-02)
;
; Sesudah mengubahnya, unggah ulang KEDUA node (gateway tidak perlu: firmware
; gateway tidak tahu-menahu soal cara node memilih slot).
[slot]
mode = 1

; Jumlah slot per siklus. Dibaca gateway DAN kedua node dari satu tempat ini,
; sehingga ketiganya mustahil berbeda. Dipakai untuk tabel D (peluang tabrakan
; 1/count) dan Challenge CH-1/CH-4. Panjang satu siklus = count x 800 ms.
count = 2

[env]
platform = atmelavr
board = uno
framework = arduino
lib_deps =
    sandeepmistry/LoRa@^0.8.0

; BOARD BERCAMPUR (asli + klon) TIDAK MEMENGARUHI FIRMWARE.
; Keduanya ATmega328P dengan protokol unggah "arduino" yang sama, dan berkas
; .hex yang dihasilkan identik. Yang berbeda hanya nama port: Uno asli muncul
; sebagai /dev/ttyACM*, klon ber-bridge CH340/CH343/FTDI sebagai /dev/ttyUSB*,
; sedangkan Windows memakai COMx untuk keduanya.
;
; Jalankan `python3 ../tools/deteksi_port.py` untuk melihat port mana milik
; jenis board yang mana, lalu sesuaikan baris upload_port/monitor_port di
; bawah. Nilai di bawah hanya contoh untuk tiga Uno asli.
monitor_speed = 115200

[env:gateway]
build_src_filter = +<gateway/*.cpp>
build_flags = -DSLOT_COUNT=${slot.count}
upload_port = /dev/ttyACM0
monitor_port = /dev/ttyACM0

; Kedua node memakai source yang sama; nomor node dan mode slot sama-sama
; ditentukan build flag, sehingga src/node/main.cpp tidak perlu disentuh.
[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=1 -DSLOT_MODE_RANDOM=${slot.mode} -DSLOT_COUNT=${slot.count}
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]

```

```

build_src_filter = +<node/*.cpp>
build_flags = -DNODE_ID=2 -DSLOT_MODE_RANDOM=${slot.mode} -DSLOT_COUNT=${slot.count}
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2

```

Kode 13.1: platformio.ini Modul 10 — bagian [slot] memilih mode dan jumlah slot

```

/*
LoRa Slotted ALOHA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: node1 / node2

Pengembangan dari M09, tapi mengganti "coba lagi dengan jeda acak" (backoff)
dengan "bicara hanya pada jatah waktu sendiri" (slot). Gateway menyiarkan
"SYNC=<cycle>" di awal tiap siklus; node memakai saat SYNC tiba sebagai
referensi waktu bersama, lalu menghitung kapan gilirannya:

    txTime = waktuTerimaSync + slotIndex * SLOT_DURATION_MS + SLOT_GUARD_MS

Dua mode pemilihan slot (SLOT_MODE_RANDOM di bawah):
Mode B (default, 0) - Assigned : slotIndex = NODE_ID - 1, tetap selamanya
Mode A (1)          - Random   : slotIndex diundi ulang tiap siklus

Tidak ada lagi retry/backoff seperti M09 -- kalau satu siklus gagal, node
tidak mengejarnya lagi; SYNC berikutnya membuka giliran baru dengan data
yang lebih baru. Penjadwalan menggantikan peran retry sepenuhnya.

Payload  : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Balasan  : "ACK=<id>,SEQ=<n>"

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

// Nomor node datang dari build flag (-DNODE_ID=1 atau 2) di platformio.ini,
// sehingga kedua node memakai file source yang sama persis.
#ifndef NODE_ID
#define NODE_ID 1
#endif

// SLOT_COUNT dan SLOT_DURATION_MS HARUS SAMA PERSIS dengan gateway/main.cpp
// -- keduanya tidak pernah dipertukarkan lewat radio, hanya disepakati lewat
// kode, persis seperti parameter radio (SF, BW, CR) yang juga tidak dikirim.
#ifndef SLOT_COUNT
#define SLOT_COUNT        2    // Node 1 -> slot 0 ("ganjil"), Node 2 -> slot 1 ("genap")
#endif
#define SLOT_DURATION_MS  800  // cukup untuk TX DATA + ACK + margin pada SF7
#define SLOT_GUARD_MS     50   // jeda kecil dari awal slot sebelum kirim

// 0 = Mode B (Assigned Slot, tanpa tabrakan bila SYNC benar)
// 1 = Mode A (Random Slot, masih bisa tabrakan bila dua node memilih slot sama)
// Ganti nilainya untuk EXP-02 lalu unggah ulang KEDUA node.
#ifndef SLOT_MODE_RANDOM
#define SLOT_MODE_RANDOM  0
#endif

#define ROOM1_TEMP_BASE  27.0
#define ROOM1_HUM_BASE   60
#define ROOM2_TEMP_BASE  25.0
#define ROOM2_HUM_BASE   68

unsigned long seq = 0;

```

```

unsigned long okCount = 0;
unsigned long failCount = 0;

volatile bool rxFlag = false;

void onPacket(int size) {
    if (size > 0) rxFlag = true;
}

void transmit(const String& msg) {
    LoRa.beginPacket();
    LoRa.print(msg);
    LoRa.endPacket();
}

float dummyTemp(float base) {
    return base + (random(-30, 31) / 10.0); // +-3.0 C
}

int dummyHum(int base) {
    return base + random(-15, 16); // +-15 %
}

// Blok sampai paket "SYNC=<cycle>" tiba. Paket lain (DATA/ACK milik siklus
// sebelumnya, atau milik node lain) dibaca lalu diabaikan -- radio LoRa tidak
// beralamat, sehingga node mendengar SEMUA lalu lintas, bukan hanya miliknya.
void waitForSync(unsigned long& syncTime, long& cycleNum) {
    LoRa.receive();
    unsigned long lastHeartbeat = millis();

    while (true) {
        if (rxFlag) {
            rxFlag = false;
            String msg = "";
            while (LoRa.available()) {
                msg += (char)LoRa.read();
            }

            if (msg.startsWith("SYNC=")) {
                cycleNum = msg.substring(5).toInt();
                syncTime = millis();
                return;
            }
            LoRa.receive();
        }

        if (millis() - lastHeartbeat > 3000) {
            Serial.println(F("[WAIT] Menunggu SYNC dari gateway..."));
            lastHeartbeat = millis();
        }
    }
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    randomSeed(analogRead(A0));

    Serial.print(F("=== LoRa SLOTTED ALOHA - NODE "));
    Serial.print(NODE_ID);
    Serial.println(F(" =="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DIO0_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.onReceive(onPacket);

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("Slot: ")); Serial.print(SLOT_COUNT);

```

```

    Serial.print(F(" x ")); Serial.print(SLOT_DURATION_MS); Serial.println(F(" ms"));
    #if SLOT_MODE_RANDOM
        Serial.println(F("Mode : A (Random Slot) -- slot diundi tiap siklus"));
    #else
        Serial.print(F("Mode : B (Assigned Slot) -- tetap di slot "));
        Serial.println(NODE_ID - 1);
    #endif
    Serial.println(F("Menunggu SYNC pertama dari gateway...\n"));
}

void loop() {
    unsigned long syncTime;
    long cycleNum;
    waitForSync(syncTime, cycleNum);

    #if SLOT_MODE_RANDOM
        int slotIndex = random(0, SLOT_COUNT);
    #else
        int slotIndex = NODE_ID - 1;
    #endif

    unsigned long txTime = syncTime + (unsigned long)slotIndex * SLOT_DURATION_MS + SLOT_GUARD_MS;
    while (millis() < txTime) { /* tunggu giliran slot */ }

    float r1t = dummyTemp(ROOM1_TEMP_BASE);
    int r1h = dummyHum(ROOM1_HUM_BASE);
    float r2t = dummyTemp(ROOM2_TEMP_BASE);
    int r2h = dummyHum(ROOM2_HUM_BASE);

    String payload = "NODE=" + String(NODE_ID) +
        ",SEQ=" + String(seq) +
        ",R1T=" + String(r1t, 1) +
        ",R1H=" + String(r1h) +
        ",R2T=" + String(r2t, 1) +
        ",R2H=" + String(r2h);
    String expectedAck = "ACK=" + String(NODE_ID) + ",SEQ=" + String(seq);

    digitalWrite(LED_PIN, HIGH);
    transmit(payload);
    digitalWrite(LED_PIN, LOW);

    Serial.print(F("[TX] cycle=")); Serial.print(cycleNum);
    Serial.print(F(" slot=")); Serial.print(slotIndex);
    Serial.print(F(" | ")); Serial.println(payload);

    // Tunggu ACK hanya sampai batas AKHIR slot sendiri -- tidak boleh
    // menyerobot waktu slot node lain.
    unsigned long slotEnd = syncTime + (unsigned long)(slotIndex + 1) * SLOT_DURATION_MS;
    rxFlag = false;
    LoRa.receive();

    bool gotAck = false;
    while (millis() < slotEnd) {
        if (rxFlag) {
            rxFlag = false;
            String reply = "";
            while (LoRa.available()) {
                reply += (char)LoRa.read();
            }
            if (reply == expectedAck) {
                gotAck = true;
                break;
            }
            // Bukan ACK yang ditunggu (mis. SYNC/DATA node lain yang ikut
            // terdengar) -- abaikan, tetap tunggu sampai batas slot.
        }
    }

    if (gotAck) {
        okCount++;
        Serial.print(F("[OK] ACK diterima"));
    } else {
        failCount++;
        Serial.print(F("[FAIL] Tidak ada ACK dalam slot ini"));
    }
    Serial.print(F(" | OK: ")); Serial.print(okCount);
    Serial.print(F(" | FAIL: ")); Serial.println(failCount);
    Serial.println();

    seq++;
    // Tidak ada delay/backoff -- loop berikutnya langsung menunggu SYNC baru.
}

```

Kode 13.2: src/node/main.cpp — hitung slot dari SYNC, dua mode pemilihan slot

```

/*
LoRa Slotted ALOHA - Dragino LoRa Shield v1.2 + Arduino Uno (ATmega328P)
Library : LoRa by sandeepmistry v0.8.x
Environment PlatformIO: gateway

Gateway kini punya peran baru dibanding M08/M08B/M09: ia bukan sekadar
pendengar pasif, melainkan penentu waktu bersama. Tiap siklus dimulai
dengan menyiarkan "SYNC=<cycle>", lalu gateway mendengarkan selama
SLOT_COUNT x SLOT_DURATION_MS -- jendela waktu yang cukup untuk seluruh
node mengirim pada slotnya masing-masing -- sebelum menyiarkan SYNC
berikutnya. Tidak ada polling seperti M05; gateway tidak memanggil node
satu per satu, ia hanya menjaga detak waktu, dan node sendiri yang tahu
kapan gilirannya berdasarkan SYNC itu.

Payload : "NODE=<id>,SEQ=<n>,R1T=<c>,R1H=<pct>,R2T=<c>,R2H=<pct>"
Balasan : "ACK=<id>,SEQ=<n>"

Pin Mapping Dragino Shield v1.2:
NSS/CS -> D10, DI00 -> D2, RST -> D9
SCK -> D13, MOSI -> D11, MISO -> D12
*/

#include <Arduino.h>
#include <SPI.h>
#include <LoRa.h>

#define NSS_PIN          10
#define RST_PIN          9
#define DI00_PIN         2
#define LED_PIN          LED_BUILTIN

#define FREQUENCY         433E6
#define BANDWIDTH         125E3
#define SPREADING_FACTOR  7
#define CODING_RATE       5
#define TX_POWER          17

#define LED_DURATION      150
#define NODE_COUNT        2 // Node 1 dan Node 2 -> indeks array 1..2

// HARUS SAMA PERSIS dengan src/node/main.cpp di kedua node. Nilainya datang
// dari [slot] count di platformio.ini supaya gateway dan kedua node tidak
// mungkin berbeda -- ketiganya membaca angka yang sama.
#ifndef SLOT_COUNT
#define SLOT_COUNT        2
#endif
#define SLOT_DURATION_MS  800

volatile bool rxFlag = false;

unsigned long ledOnTime = 0;
bool ledActive = false;

unsigned long cycleCount = 0;
unsigned long rxCount[NODE_COUNT + 1] = {0};
long lastSeq[NODE_COUNT + 1];
unsigned long lostEst[NODE_COUNT + 1] = {0};

void onPacket(int size) {
    if (size > 0) rxFlag = true;
}

void updateLED() {
    if (ledActive && (millis() - ledOnTime >= LED_DURATION)) {
        digitalWrite(LED_PIN, LOW);
        ledActive = false;
    }
}

void triggerLED() {
    digitalWrite(LED_PIN, HIGH);
    ledOnTime = millis();
    ledActive = true;
}

String getField(const String& data, const String& key) {
    String needle = key + "=";
    int start = data.indexOf(needle);

```

```

    if (start == -1) return "";
    start += needle.length();
    int end = data.indexOf(',', start);
    if (end == -1) end = data.length();
    return data.substring(start, end);
}

void broadcastSync() {
    String syncMsg = "SYNC=" + String(cycleCount);
    LoRa.beginPacket();
    LoRa.print(syncMsg);
    LoRa.endPacket();
    Serial.print(F("[TX] ")); Serial.println(syncMsg);
    LoRa.receive();
}

void setup() {
    Serial.begin(115200);
    while (!Serial);

    pinMode(LED_PIN, OUTPUT);
    digitalWrite(LED_PIN, LOW);

    for (int i = 0; i <= NODE_COUNT; i++) lastSeq[i] = -1;

    Serial.println(F("=== LoRa SLOTTED ALOHA - GATEWAY ==="));
    Serial.print(F("Init LoRa ... "));

    LoRa.setPins(NSS_PIN, RST_PIN, DI00_PIN);
    if (!LoRa.begin(FREQUENCY)) {
        Serial.println(F("GAGAL! Cek shield/kabel."));
        while (true);
    }

    LoRa.setSpreadingFactor(SPREADING_FACTOR);
    LoRa.setSignalBandwidth(BANDWIDTH);
    LoRa.setCodingRate4(CODING_RATE);
    LoRa.setTxPower(TX_POWER);
    LoRa.onReceive(onPacket);

    Serial.println(F("OK"));
    Serial.print(F("Freq: ")); Serial.print(FREQUENCY / 1E6); Serial.println(F(" MHz"));
    Serial.print(F("Slot: ")); Serial.print(SLOT_COUNT);
    Serial.print(F(" x ")); Serial.print(SLOT_DURATION_MS); Serial.println(F(" ms"));
    Serial.println(F("Peran: GATEWAY (Slotted ALOHA) -- siarkan SYNC, dengar tiap slot\n"));
}

void loop() {
    broadcastSync();

    unsigned long cycleStart = millis();
    unsigned long cycleEnd = cycleStart + (unsigned long)SLOT_COUNT * SLOT_DURATION_MS;

    while (millis() < cycleEnd) {
        updateLED();

        if (!rxFlag) continue;
        rxFlag = false;

        String received = "";
        while (LoRa.available()) {
            received += (char)LoRa.read();
        }

        if (!received.startsWith("NODE=")) {
            // Kemungkinan gema SYNC sendiri atau paket asing -- abaikan.
            LoRa.receive();
            continue;
        }

        int nodeId = getField(received, "NODE").toInt();
        long seq = getField(received, "SEQ").toInt();
        float r1t = getField(received, "R1T").toFloat();
        int r1h = getField(received, "R1H").toInt();
        float r2t = getField(received, "R2T").toFloat();
        int r2h = getField(received, "R2H").toInt();

        int slotObserved = (millis() - cycleStart) / SLOT_DURATION_MS;

        triggerLED();

        Serial.println(F("=== PAKET DITERIMA ==="));
    }
}

```

```

Serial.print(F(" Node : ")); Serial.println(nodeId);
Serial.print(F(" Slot : ")); Serial.println(slotObserved);
Serial.print(F(" SEQ : ")); Serial.println(seq);
Serial.print(F(" Ruang 1 : ")); Serial.print(r1t, 1); Serial.print(F(" C, "));
Serial.print(r1h); Serial.println(F(" %"));
Serial.print(F(" Ruang 2 : ")); Serial.print(r2t, 1); Serial.print(F(" C, "));
Serial.print(r2h); Serial.println(F(" %"));
Serial.print(F(" RSSI : ")); Serial.print(LoRa.packetRssi()); Serial.println(F(" dBm"));
Serial.print(F(" SNR : ")); Serial.print(LoRa.packetSnr()); Serial.println(F(" dB"));

if (nodeId >= 1 && nodeId <= NODE_COUNT) {
    rxCount[nodeId]++;

    if (lastSeq[nodeId] >= 0) {
        long gap = seq - lastSeq[nodeId] - 1;
        if (gap > 0) {
            // Tidak ada retry di M10 -- satu SEQ yang gagal berarti hilang
            // permanen untuk siklus itu, bukan sekadar tabrakan sesaat.
            lostEst[nodeId] += gap;
            Serial.print(F(" [GAP] SEQ meloncat ")); Serial.print(gap);
            Serial.println(F(" -- data hilang pada siklus tersebut (mis. tabrakan slot)"));
        }
        lastSeq[nodeId] = seq;
    }

    // Balas ACK, lalu kembali dengarkan sisa slot / siklus.
    String ack = "ACK=" + String(nodeId) + ",SEQ=" + String(seq);
    LoRa.beginPacket();
    LoRa.print(ack);
    LoRa.endPacket();
    Serial.print(F(" [TX] ")); Serial.println(ack);
    Serial.println(F("=====\n"));

    LoRa.receive();
}

Serial.print(F("--- Cycle ")); Serial.print(cycleCount); Serial.print(F(" selesai | "));
for (int i = 1; i <= NODE_COUNT; i++) {
    Serial.print(F("N")); Serial.print(i);
    Serial.print(F(": diterima=")); Serial.print(rxCount[i]);
    Serial.print(F(" hilang=")); Serial.print(lostEst[i]);
    Serial.print(F(" "));
}
Serial.println(F("---\n"));

cycleCount++;
}

```

Kode 13.3: src/gateway/main.cpp — siarkan SYNC, dengarkan tiap slot, balas ACK

```

#!/usr/bin/env python3
"""
LoRa Slotted ALOHA Monitor Dashboard
=====
Full-terminal live dashboard untuk monitoring komunikasi
LoRa Gateway <-> Node 1 & Node 2 (gateway menyiarkan SYNC, tiap node bicara pada slotnya sendiri)

Kompatibel: Windows (PowerShell / Windows Terminal) & Linux/Mac

Kebutuhan:
    pip install pyserial rich

Cara pakai:
    python lora_monitor.py                                # port bawaan sesuai OS
    python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
    python lora_monitor.py --out data_eksperimen.csv
"""

import threading
import queue
import csv
import time
import argparse
import glob
import re
import os
import sys

```



```

from datetime import datetime
from collections import deque
from dataclasses import dataclass, field

try:
    import serial
except ImportError:
    print("ERROR: pyserial tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

try:
    from rich.live import Live
    from rich.table import Table
    from rich.layout import Layout
    from rich.panel import Panel
    from rich.text import Text
    from rich.console import Console, Group
    from rich.columns import Columns
    from rich import box
except ImportError:
    print("ERROR: rich tidak terinstall. Jalankan: pip install pyserial rich")
    sys.exit(1)

# -----
# KONFIGURASI
# -----
# Port bawaan mengikuti sistem operasi. Di Linux/macOS, Uno asli muncul sebagai
# /dev/ttyACM* dan klon ber-bridge CH340 sebagai /dev/ttyUSB*; di Windows
# keduanya sama-sama COMx. Jalankan tools/deteksi_port.py untuk mengetahui port
# mana milik board yang mana, lalu berikan lewat --gateway/--n1/--n2.
if sys.platform.startswith("win"):
    DEFAULT_GW_PORT = "COM3"
    DEFAULT_N1_PORT = "COM4"
    DEFAULT_N2_PORT = "COM5"
else:
    DEFAULT_GW_PORT = "/dev/ttyACM0"
    DEFAULT_N1_PORT = "/dev/ttyACM1"
    DEFAULT_N2_PORT = "/dev/ttyACM2"
DEFAULT_BAUD = 115200
LOG_LINES = 16
REFRESH_HZ = 4
GAP_WARN_STREAK = 3

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Uno asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI
    sebagai /dev/ttyUSB* -- pola yang sama dipakai tools/deteksi_port.py.
    Dipanggil tiap program start supaya default gateway/n1/n2 selalu
    mengikuti apa yang benar-benar tersambung, bukan port tetap di atas.
    """
    if sys.platform.startswith("win"):
        return [] # deteksi berbasis /dev/tty* tidak berlaku di Windows (COMx)
        return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))
    # Pure ALOHA (M08A) tidak punya ACK sama sekali -- node tidak pernah tahu
    # apakah paketnya sampai, jadi baris OK/FAIL/Retry di panel node disembunyikan.
    HAS_ACK = True

# -----
# DATA STRUCTURES
# -----

@dataclass
class Anomaly:
    ts: str
    node: str
    kind: str
    detail: str

@dataclass
class GatewayState:
    port: str = DEFAULT_GW_PORT
    connected: bool = False
    rx_total: int = 0
    rx_n1: int = 0
    rx_n2: int = 0
    gap_n1: int = 0
    gap_n2: int = 0
    lost_n1: int = 0
    lost_n2: int = 0
    dup_n1: int = 0

```

```

dup_n2:         int = 0
last_rssi:      int = 0
last_snr:       float = 0.0
rssi_sum:       int = 0
snr_sum:        float = 0.0
rssi_count:     int = 0
cycle:          int = 0
raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

@dataclass
class NodeState:
    name:         str = ""
    port:         str = ""
    connected:    bool = False
    tx_count:     int = 0
    ok_count:     int = 0
    fail_count:   int = 0
    retry_count:  int = 0
    last_seq:     int = -1
    raw_log: deque = field(default_factory=lambda: deque(maxlen=300))

class SharedState:
    def __init__(self):
        self.lock = threading.Lock()
        self.gw = GatewayState()
        self.n1 = NodeState("NODE 1", DEFAULT_N1_PORT)
        self.n2 = NodeState("NODE 2", DEFAULT_N2_PORT)
        self.anomalies = deque(maxlen=60)
        self.csv_queue = queue.Queue()
        self.running = True
        self.session_start = datetime.now()

# -----
# PARSERS
# -----
# Semua regex dibuat agar aman dipakai lintas modul (M08A/M08B/M09/M10) --
# token yang tidak muncul pada suatu modul (mis. RETRY/SYNC) hanya tidak
# pernah cocok, tanpa memengaruhi parsing token lain.

RE_NODE_LINE = re.compile(r'Node\s+:\s*(\d+)')
RE_SEQ_LINE = re.compile(r'SEQ\s+:\s*(\d+)')
RE_RSSI_LINE = re.compile(r'RSSI\s+:\s*(-?\d+)')
RE_SNR_LINE = re.compile(r'SNR\s+:\s*(-?\d+\.\d+)')
RE_GAP_LINE = re.compile(r'\[GAP\] SEQ meloncat\s+(\d+)')
RE_STAT_LINE = re.compile(r'Statistik Node\s+(\d+):\s*diterima=(\d+)\s*\|\s*perkiraan hilang=(\d+)')
RE_SYNC_TX = re.compile(r'\[TX\]\s+SYNC=(\d+)')
RE_CYCLE_DONE = re.compile(r'Cycle\s+(\d+)\s+selesai')
# M09: "Statistik Node X: baru=Y | duplicate=Z | gagal permanen (est.)=W"
RE_STAT_RETRY = re.compile(r'Statistik Node\s+(\d+):\s*baru=(\d+)\s*\|\s*duplicate=(\d+)\s*\|\s*gagal permanen\s*(est\.)=(\d+)')
# M10: "--- Cycle N selesai | N1: diterima=Y hilang=Z N2: diterima=Y hilang=Z ---"
RE_STAT_CYCLE = re.compile(r'N(\d+):\s*diterima=(\d+)\s*hilang=(\d+)')

RE_NODE_TX = re.compile(r'\[TX\].*NODE=(\d+),SEQ=(\d+)')
RE_NODE_RETRY = re.compile(r'\[RETRY\s+(\d+)/(\d+)\]')
RE_NODE_OK = re.compile(r'\[OK\]')
RE_NODE_FAIL = re.compile(r'\[FAIL\]')

def parse_gateway_line(line: str, st: GatewayState, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    if 'PAKET DITERIMA' in line:
        st.rx_total += 1

    m = RE_RSSI_LINE.search(line)
    if m:
        st.last_rssi = int(m.group(1))
        st.rssi_sum += st.last_rssi
        st.rssi_count += 1

    m = RE_SNR_LINE.search(line)
    if m:
        st.last_snr = float(m.group(1))
        st.snr_sum += st.last_snr

    m = RE_GAP_LINE.search(line)
    if m:
        gap = int(m.group(1))
        anomalies.append(Anomaly(ts, "GATEWAY", "GAP", f"SEQ meloncat {gap} -- indikasi tabrakan/paket hilang"))

    m = RE_STAT_LINE.search(line)

```

```

if m:
    node_id, rx, lost = int(m.group(1)), int(m.group(2)), int(m.group(3))
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_STAT_RETRY.search(line)
if m:
    node_id, rx, dup, lost = int(m.group(1)), int(m.group(2)), int(m.group(3)), int(m.group(4))
    if node_id == 1:
        st.rx_n1, st.dup_n1, st.lost_n1 = rx, dup, lost
    elif node_id == 2:
        st.rx_n2, st.dup_n2, st.lost_n2 = rx, dup, lost

for node_id_s, rx_s, lost_s in RE_STAT_CYCLE.findall(line):
    node_id, rx, lost = int(node_id_s), int(rx_s), int(lost_s)
    if node_id == 1:
        st.rx_n1, st.lost_n1 = rx, lost
    elif node_id == 2:
        st.rx_n2, st.lost_n2 = rx, lost

m = RE_SYNC_TX.search(line)
if m:
    st.cycle = int(m.group(1))

m = RE_CYCLE_DONE.search(line)
if m:
    st.cycle = int(m.group(1))

def parse_node_line(line: str, st: NodeState, node_label: str, anomalies: deque):
    ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]

    m = RE_NODE_TX.search(line)
    if m:
        seq = int(m.group(2))
        if st.last_seq >= 0 and seq == st.last_seq and '[RETRY]' not in line:
            pass # aman, tidak dipakai
        st.last_seq = seq
        if '[RETRY]' not in line:
            st.tx_count += 1

    m = RE_NODE_RETRY.search(line)
    if m:
        st.retry_count += 1

    if RE_NODE_OK.search(line):
        st.ok_count += 1
    elif RE_NODE_FAIL.search(line):
        st.fail_count += 1
        if st.fail_count and st.fail_count % 5 == 0:
            anomalies.append(Anomaly(ts, node_label, "FAIL_COUNT",
                                     f"{node_label} sudah {st.fail_count}x gagal (tanpa ACK)"))

# -----
# READER THREADS
# -----

def reader_thread(port: str, baud: int, node_key: str,
                  shared: SharedState, role: str):
    if role == "GW":
        state = shared.gw
    elif node_key == "N1":
        state = shared.n1
    else:
        state = shared.n2

    while shared.running:
        try:
            with serial.Serial(port, baud, timeout=1) as ser:
                with shared.lock:
                    state.connected = True
                    while shared.running:
                        try:
                            raw = ser.readline()
                            if not raw:
                                continue
                            line = raw.decode('utf-8', errors='replace').rstrip()
                            if not line:
                                continue
                            ts = datetime.now().strftime("%H:%M:%S.%f")[:-3]
                            with shared.lock:

```

```

        state.raw_log.append((ts, line))
        if role == "GW":
            parse_gateway_line(line, state, shared.anomalies)
        else:
            parse_node_line(line, state, node_key, shared.anomalies)
        shared.csv_queue.put({
            'ts': ts, 'node': node_key, 'raw': line
        })
    except (serial.SerialException, OSError):
        break
except (serial.SerialException, OSError):
    with shared.lock:
        state.connected = False
    time.sleep(2)

def csv_writer_thread(shared: SharedState, filepath: str):
    with open(filepath, 'w', newline='', encoding='utf-8') as f:
        w = csv.DictWriter(f, fieldnames=['timestamp', 'node', 'raw_line'])
        w.writeheader()
        while shared.running or not shared.csv_queue.empty():
            try:
                row = shared.csv_queue.get(timeout=0.5)
                w.writerow({'timestamp': row['ts'],
                           'node': row['node'],
                           'raw_line': row['raw']})
                f.flush()
            except queue.Empty:
                continue

# -----
# RICH RENDERERS
# -----

def conn_badge(connected: bool, port: str) -> Text:
    if connected:
        return Text(f" CONNECTED: {port} ", style="bold black on green")
    return Text(f" DISCONNECTED: {port} ", style="bold white on red")

def pct_style(ok: int, total: int) -> str:
    if total == 0: return "dim"
    r = ok / total
    if r >= 0.97: return "bold green"
    if r >= 0.90: return "bold yellow"
    return "bold red"

def log_line_style(line: str) -> str:
    if '[FAIL]' in line: return "red"
    if '[GAP]' in line: return "bold red"
    if '[RETRY]' in line: return "bold yellow"
    if '[BACKOFF]' in line: return "yellow"
    if '[OK]' in line: return "bold green"
    if '[TX]' in line: return "cyan"
    if 'PAKET DITERIMA' in line: return "green"
    if '[WAIT]' in line: return "dim"
    if '[WARN]' in line: return "yellow"
    return "dim white"

def build_gateway_panel(st: GatewayState) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" Total diterima : ", style="dim"); t.append(f"{st.rx_total}\n", style="bold white")
    if st.cycle:
        t.append(" Cycle terakhir : ", style="dim"); t.append(f"{st.cycle}\n", style="bold cyan")
    t.append("\n")

    dup_suffix1 = f" dup={st.dup_n1}" if st.dup_n1 else ""
    dup_suffix2 = f" dup={st.dup_n2}" if st.dup_n2 else ""
    t.append(" Node 1 ", style="bold cyan")
    t.append(f"diterima={st.rx_n1:<4} hilang={st.lost_n1}{dup_suffix1}\n", style="white")
    t.append(" Node 2 ", style="bold magenta")
    t.append(f"diterima={st.rx_n2:<4} hilang={st.lost_n2}{dup_suffix2}\n\n", style="white")

    rssi_avg = (st.rssi_sum / st.rssi_count) if st.rssi_count else 0.0
    snr_avg = (st.snr_sum / st.rssi_count) if st.rssi_count else 0.0
    t.append(" RSSI terakhir : ", style="dim"); t.append(f"{st.last_rssi} dBm\n", style="white")
    t.append(" SNR terakhir : ", style="dim"); t.append(f"{st.last_snr:.2f} dB\n", style="white")
    t.append(" RSSI rata-rata : ", style="dim"); t.append(f"{rssi_avg:.1f} dBm\n", style="white")
    t.append(" SNR rata-rata : ", style="dim"); t.append(f"{snr_avg:.2f} dB\n\n", style="white")

```

```

t.append(" Raw Log\n", style="bold dim")
t.append(" " + "-" * 30 + "\n", style="dim")
for ts, line in list(st.raw_log)[-LOG_LINES:]:
    style = log_line_style(line)
    t.append(f" {ts[-8:]} {line[:40]}\n", style=style)

return Panel(t, title="[bold cyan]GATEWAY[/]",
             border_style="cyan", box=box.ROUNDED)

def build_node_panel(st: NodeState, label: str, color: str) -> Panel:
    t = Text()
    t.append("\n")
    t.append(conn_badge(st.connected, st.port))
    t.append("\n\n")

    t.append(" TX terkirim : ", style="dim"); t.append(f"{st.tx_count:>6}\n", style="bold white")
    t.append(" SEQ terakhir : ", style="dim"); t.append(f"{st.last_seq:>6}\n\n", style="white")

    if HAS_ACK:
        tot = st.ok_count + st.fail_count
        pct = (st.ok_count / tot * 100) if tot else 0.0
        t.append(" OK : ", style="dim"); t.append(f"{st.ok_count:>6}\n", style="bold green")
        t.append(" FAIL : ", style="dim"); t.append(f"{st.fail_count:>6}\n",
            style="bold red" if st.fail_count else "dim")
        if tot:
            t.append(" Rate : ", style="dim"); t.append(f"{pct:>6.2f}%\n", style=pct_style(st.ok_count,
tot))
        t.append(" Retry : ", style="dim"); t.append(f"{st.retry_count:>6}\n\n",
            style="yellow" if st.retry_count else "dim")
    else:
        t.append(" (Pure ALOHA -- tanpa ACK, node tidak pernah tahu\n", style="dim")
        t.append(" apakah paketnya sampai. Lihat panel GATEWAY untuk\n", style="dim")
        t.append(" statistik diterima/hilang per node.)\n\n", style="dim")

    t.append(" Raw Log\n", style="bold dim")
    t.append(" " + "-" * 30 + "\n", style="dim")
    for ts, line in list(st.raw_log)[-LOG_LINES:]:
        style = log_line_style(line)
        t.append(f" {ts[-8:]} {line[:36]}\n", style=style)

    return Panel(t, title=f"[bold {color}]{label}[/]",
                border_style=color, box=box.ROUNDED)

def build_anomaly_panel(anomalies: deque, session_start: datetime,
                        csv_path: str) -> Panel:
    elapsed = datetime.now() - session_start
    h = int(elapsed.total_seconds() // 3600)
    m = int((elapsed.total_seconds() % 3600) // 60)
    s = int(elapsed.total_seconds() % 60)

    t = Text()
    t.append(f" Session: {h:02d}:{m:02d}:{s:02d}", style="dim")
    t.append(f" CSV: {os.path.basename(csv_path)}", style="dim")
    t.append(f" Ctrl+C untuk keluar\n\n", style="dim")

    kind_style = {
        'GAP': "bold red",
        'FAIL_COUNT': "bold yellow",
    }
    anom_list = list(anomalies)
    if not anom_list:
        t.append(" Tidak ada anomali terdeteksi", style="bold green")
    else:
        for a in reversed(anom_list[-5:]):
            style = kind_style.get(a.kind, "white")
            t.append(f" [{a.ts}] ", style="dim")
            t.append(f"[{a.node}] ", style="bold white")
            t.append(f"{a.kind}: ", style=style)
            t.append(f"{a.detail}\n", style="white")

    return Panel(t, title="[bold red]ANOMALI DETECTOR[/]",
                border_style="red", box=box.ROUNDED)

# -----
# LAYOUT BUILDER
# -----

def build_layout(shared: SharedState, csv_path: str):
    with shared.lock:
        gw = GatewayState(
            port=shared.gw.port, connected=shared.gw.connected,
            rx_total=shared.gw.rx_total, rx_n1=shared.gw.rx_n1, rx_n2=shared.gw.rx_n2,

```

```

        gap_n1=shared.gw.gap_n1, gap_n2=shared.gw.gap_n2,
        lost_n1=shared.gw.lost_n1, lost_n2=shared.gw.lost_n2,
        dup_n1=shared.gw.dup_n1, dup_n2=shared.gw.dup_n2,
        last_rssi=shared.gw.last_rssi, last_snr=shared.gw.last_snr,
        rssi_sum=shared.gw.rssi_sum, snr_sum=shared.gw.snr_sum,
        rssi_count=shared.gw.rssi_count, cycle=shared.gw.cycle,
        raw_log=deque(shared.gw.raw_log),
    )
    n1 = NodeState(
        name=shared.n1.name, port=shared.n1.port, connected=shared.n1.connected,
        tx_count=shared.n1.tx_count, ok_count=shared.n1.ok_count,
        fail_count=shared.n1.fail_count, retry_count=shared.n1.retry_count,
        last_seq=shared.n1.last_seq,
        raw_log=deque(shared.n1.raw_log),
    )
    n2 = NodeState(
        name=shared.n2.name, port=shared.n2.port, connected=shared.n2.connected,
        tx_count=shared.n2.tx_count, ok_count=shared.n2.ok_count,
        fail_count=shared.n2.fail_count, retry_count=shared.n2.retry_count,
        last_seq=shared.n2.last_seq,
        raw_log=deque(shared.n2.raw_log),
    )
    anom = deque(shared.anomalies)
    t_start = shared.session_start

    node_row = Columns([
        build_gateway_panel(gw),
        build_node_panel(n1, "NODE 1", "green"),
        build_node_panel(n2, "NODE 2", "magenta"),
    ], equal=True, expand=True)

    return Group(node_row, build_anomaly_panel(anom, t_start, csv_path))

#
# ENTRY POINT
#

BANNER = """\
    LoRa SLOTTED ALOHA Monitor – Evaluasi Praktikum
    Dragino LoRa Shield v1.2 · SX1276 · Arduino Uno · 433 MHz
"""

Tool ini membaca data serial dari 3 board LoRa (Gateway, Node 1, Node 2)
secara bersamaan dan menampilkan dashboard real-time: statistik
diterima/OK/FAIL/SLOT, RSSI/SNR, deteksi anomali (GAP/FAIL beruntun),
serta logging CSV otomatis untuk analisis data penelitian.

PENGUNAAN:
python lora_monitor.py [opsi...]

OPSI:
--gateway PORT Port COM untuk Gateway (default: {gw})
--n1 PORT Port COM untuk Node 1 (default: {n1})
--n2 PORT Port COM untuk Node 2 (default: {n2})
--baud N Baud rate serial semua port (default: {baud})
--out FILE Nama file CSV output (default: lora_session_YYYYMMDD_HHMMSS.csv)
-h / --help Tampilkan pesan bantuan ini

CONTOH:
python lora_monitor.py
-> Jalankan dengan port & baud rate default

python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2
-> Tentukan port secara eksplisit (Windows: COM3, COM4, COM5)

python lora_monitor.py --out data_jarak_10m.csv
-> Simpan log ke nama file tertentu

KONTROL DASHBOARD:
Ctrl+C Hentikan monitoring & simpan CSV

CATATAN:
* Pastikan semua board sudah di-upload firmware dengan baud {baud}
* Upload Gateway dulu, baru Node 1 dan Node 2, sebelum menjalankan tool ini
* File CSV tersimpan otomatis saat Ctrl+C ditekan
"""
format(
    gw=DEFAULT_GW_PORT,
    n1=DEFAULT_N1_PORT,
    n2=DEFAULT_N2_PORT,
    baud=DEFAULT_BAUD,
)

```

```

def parse_args(port_aktif):
    gw_default = port_aktif[0] if len(port_aktif) > 0 else DEFAULT_GW_PORT
    n1_default = port_aktif[1] if len(port_aktif) > 1 else DEFAULT_N1_PORT
    n2_default = port_aktif[2] if len(port_aktif) > 2 else DEFAULT_N2_PORT

    p = argparse.ArgumentParser(
        description="LoRa Slotted ALOHA Monitor Dashboard -- tool evaluasi praktikum LoRa.",
        formatter_class=argparse.RawDescriptionHelpFormatter,
        epilog=(
            "Contoh penggunaan:\n"
            f"  python lora_monitor.py\n"
            f"  python lora_monitor.py --gateway /dev/ttyACM0 --n1 /dev/ttyACM1 --n2 /dev/ttyACM2\n"
            f"  python lora_monitor.py --out data_eksperimen.csv\n"
        ),
    )
    p.add_argument('--gateway', default=gw_default,
        metavar='PORT', help=f'Port COM Gateway (default: {gw_default}, dari deteksi port aktif)')
    p.add_argument('--n1', default=n1_default,
        metavar='PORT', help=f'Port COM Node 1 (default: {n1_default}, dari deteksi port aktif)')
    p.add_argument('--n2', default=n2_default,
        metavar='PORT', help=f'Port COM Node 2 (default: {n2_default}, dari deteksi port aktif)')
    p.add_argument('--baud', default=DEFAULT_BAUD, type=int,
        metavar='N', help=f'Baud rate serial (default: {DEFAULT_BAUD})')
    p.add_argument('--out', default=None,
        metavar='FILE', help='Nama file CSV output (default: auto timestamp)')
    return p, p.parse_args()

def main():
    port_aktif = deteksi_port_aktif()
    p, args = parse_args(port_aktif)
    no_args = len(sys.argv) == 1

    console = Console()

    if no_args:
        console.print(BANNER)

    console.print(
        f"[dim]Port aktif terdeteksi: {'', '.join(port_aktif) if port_aktif else '(tidak ada -- memakai default tetap)'}[/]"
    )
    console.print(
        f"[dim]Dipakai -> "
        f"Gateway=[yellow]{args.gateway}[/] "
        f"N1=[green]{args.n1}[/] "
        f"N2=[magenta]{args.n2}[/] "
        f"Baud=[white]{args.baud}[/][dim]\n"
    )

    shared = SharedState()
    shared.gw.port = args.gateway
    shared.n1.port = args.n1
    shared.n2.port = args.n2

    ts_str = shared.session_start.strftime("%Y%m%d_%H%M%S")
    csv_path = args.out or f"lora_session_{ts_str}.csv"

    if not no_args:
        console.print("\n[bold cyan]LoRa Monitor Dashboard[/]")
        console.print(f"  Gateway : [yellow]{args.gateway}[/]")
        console.print(f"  Node 1  : [green]{args.n1}[/]")
        console.print(f"  Node 2  : [magenta]{args.n2}[/]")
        console.print(f"  Baud    : [white]{args.baud}[/]")

    console.print(f"  CSV      : [dim]{csv_path}[/]")
    console.print("\nTekan [bold]Enter[/] untuk mulai dashboard, [bold]Ctrl+C[/] untuk batal...\n")
    try:
        input()
    except KeyboardInterrupt:
        return

    threads = [
        threading.Thread(target=reader_thread,
            args=(args.gateway, args.baud, "GW", shared, "GW"),
            daemon=True, name="reader-gateway"),
        threading.Thread(target=reader_thread,
            args=(args.n1, args.baud, "N1", shared, "NODE"),
            daemon=True, name="reader-n1"),
        threading.Thread(target=reader_thread,
            args=(args.n2, args.baud, "N2", shared, "NODE"),

```

```

        daemon=True, name="reader-n2"),
        threading.Thread(target=csv_writer_thread,
            args=(shared, csv_path),
            daemon=True, name="csv-writer"),
    ]
    for t in threads:
        t.start()

    try:
        with Live(build_layout(shared, csv_path),
            console=console,
            refresh_per_second=REFRESH_HZ,
            screen=True) as live:
            while True:
                time.sleep(1.0 / REFRESH_HZ)
                live.update(build_layout(shared, csv_path))
    except KeyboardInterrupt:
        pass
    finally:
        shared.running = False
        time.sleep(0.6)
        console.print(f"\n[green]Session selesai.[/] Data di: [bold]{csv_path}[/]")

if __name__ == "__main__":
    main()

```

Kode 13.4: lora_monitor.py — dasbor tiga panel dengan perekaman CSV

```

#!/usr/bin/env python3
"""Upload otomatis ke Modul 10 (Slotted ALOHA) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul10_lora_slotted_aloha_tdma -e gateway -t upload -t monitor
    pio run -d Modul10_lora_slotted_aloha_tdma -e node1 -t upload -t monitor
    pio run -d Modul10_lora_slotted_aloha_tdma -e node2 -t upload -t monitor

Skrup ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya
ke environment 'gateway' (port pertama), 'node1' (port kedua), dan 'node2'
(port ketiga), lalu menjalankan 'pio run' dengan --upload-port sesuai hasil
deteksi -- platformio.ini tidak perlu diedit tiap kali port berganti nama.

Bawaannya HANYA upload (tanpa membuka pio device monitor). Sesudah ketiga
board selesai di-upload, skrip bertanya apakah mau langsung menjalankan
lora_monitor.py -- bila ya, dijalankan dengan --gateway ... --n1 ... --n2 ...
persis memakai port yang barusan dipakai upload (bukan deteksi ulang),
supaya tidak mungkin salah pasang.

python3 Modul10_lora_slotted_aloha_tdma/upload_auto.py
-> upload ketiga board, port otomatis, lalu tanya mau monitor atau tidak

python3 Modul10_lora_slotted_aloha_tdma/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu (Ctrl-C untuk
    lanjut ke board berikutnya) -- bukan sekaligus seperti lora_monitor.py

python3 Modul10_lora_slotted_aloha_tdma/upload_auto.py --only gateway
-> upload satu environment saja

python3 Modul10_lora_slotted_aloha_tdma/upload_auto.py --gateway /dev/ttyACM0 --node1 /dev/ttyACM1 --
node2 /dev/ttyACM2
-> timpa hasil deteksi otomatis secara manual
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["gateway", "node1", "node2"]
# Nama environment PlatformIO -> nama opsi CLI yang dipakai lora_monitor.py
ENV_KE_OPSI = {"gateway": "--gateway", "node1": "--n1", "node2": "--n2"}

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py dan lora_monitor.py: Uno
    asli muncul sebagai /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI

```



```

sebagai /dev/ttyUSB*.
"""
return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print(" $", " ".join(cmd))
    return subprocess.call(cmd) == 0

def tanya_lalu_monitor(port_env):
    """Tanya apakah mau langsung memantau, lalu jalankan lora_monitor.py
    dengan port gateway/node1/node2 PERSIS seperti yang baru dipakai upload
    -- bukan hasil deteksi ulang, supaya tidak mungkin meleset dari yang
    barusan di-flash.
    """
    if not sys.stdin.isatty():
        return # dipanggil non-interaktif (mis. dari skrip lain) -- jangan menunggu input

    try:
        jawab = input("\nJalankan lora_monitor.py sekarang? [Y/n] ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print()
        return
    if jawab not in ("", "y", "ya"):
        return

    cmd = [sys.executable, str(PROJECT_DIR / "lora_monitor.py")]
    for env, opsi in ENV_KE_OPSI.items():
        if env in port_env:
            cmd += [opsi, port_env[env]]

    print(" $", " ".join(cmd))
    try:
        subprocess.call(cmd)
    except KeyboardInterrupt:
        # Ctrl-C dikirim ke seluruh process group, jadi proses induk ini pun
        # ikut menerima SIGINT selagi menunggu (os.waitpid) lora_monitor.py
        # yang sudah membersihkan dirinya sendiri -- cukup diam, jangan
        # biarkan traceback ikut tercetak di atas ringkasan yang sudah rapi.
        print()

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--gateway", metavar="/dev/ttyXXX",
                    help="tima port gateway (lewati deteksi otomatis)")
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="tima port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="tima port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (gateway, node1, atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload "
                    "(bawaan: upload saja -- pakai lora_monitor.py untuk memantau ketiganya"
                    sekaligus)")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"gateway": args.gateway, "node1": args.node1, "node2": args.node2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {' '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"

```

```

        print(f" -> {nama:<9} = {port_env[nama]} ({sumber})")
    else:
        print(f" -> {nama:<9} = TIDAK ADA PORT TERSISA "
              f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

daftar = [args.only] if args.only else ENV_URUTAN
hilang = [n for n in daftar if n not in port_env]
if hilang:
    sys.exit(f"\nTidak bisa lanjut: port untuk {' '.join(hilang)} tidak ditemukan.\n"
            f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

for nama in daftar:
    if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
        sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

print("\nSelesai.")
tanya_lalu_monitor(port_env)

if __name__ == "__main__":
    main()

```

Kode 13.5: upload_auto.py — pemilih port otomatis saat unggah

13.7 Build dan Flash

Gateway lebih dahulu, supaya kedua node langsung mendapat SYNC pertama begitu menyala.

```

pio run -d Modul10_lora_slotted_aloha_tdma -e gateway -t upload -t monitor
pio run -d Modul10_lora_slotted_aloha_tdma -e node1 -t upload -t monitor
pio run -d Modul10_lora_slotted_aloha_tdma -e node2 -t upload -t monitor

```

Monitor dashboard. python3 lora_monitor.py membaca ketiga port sekaligus dan menampilkan panel Gateway, Node 1, dan Node 2 (diterima dan hilang per node, cycle terakhir, RSSI dan SNR) di terminal, plus perekaman CSV otomatis. Memerlukan pip install pyserial rich.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga shield.
- ☐ Port ketiga board dicatat lewat pio device list (atau python3 ../tools/deteksi_port.py) dan diisikan ke platformio.ini.
- ☐ Tiga Serial Monitor 115200 baud siap, ketiganya terlihat bersamaan.
- ☐ [slot] mode di platformio.ini bernilai 1 (Mode A / Slotted ALOHA) untuk EXP-01 — tidak perlu menyentuh src/node/main.cpp.
- ☐ SLOT_COUNT dan SLOT_DURATION_MS sama persis di gateway dan node (bawaan: 2 dan 800).

13.8 Percobaan

Kedua percobaan pertama memakai **infrastruktur yang sama persis** — SYNC, slot, guard time, firmware gateway yang identik. Yang berganti hanya satu baris di platformio.ini, seperti pada Kode 13.6.

```

[slot]
mode = 1 ; EXP-01: Mode A, slot diundi -> Slotted ALOHA
mode = 0 ; EXP-02: Mode B, slot tetap -> TDMA

```

Kode 13.6: Pemilihan mode slot di platformio.ini

Urutannya sengaja mengikuti sejarah: undian dulu, kepastian belakangan. Gateway **tidak perlu** di-flash ulang saat berpindah mode — ia tidak tahu-menahu bagaimana node memilih slotnya.

13.8.1 EXP-01 — Mode A (Random Slot) = Slotted ALOHA

Pastikan mode = 1, unggah gateway lalu kedua node, dan rekam minimal 60 siklus.

Expected output — node

```

=== LoRa SLOTTED ALOHA - NODE 1 ===
Init LoRa ... OK
Freq: 433.00 MHz
Slot: 2 x 800 ms
Mode : A (Random Slot) -- slot diundi tiap siklus
Menunggu SYNC pertama dari gateway...

[TX] cycle=0 slot=1 | NODE=1,SEQ=0,R1T=26.5,R1H=54,R2T=23.2,R2H=59
[FAIL] Tidak ada ACK dalam slot ini | OK: 0 | FAIL: 1
[TX] cycle=1 slot=0 | NODE=1,SEQ=1,R1T=27.4,R1H=57,R2T=23.6,R2H=57
[OK] ACK diterima | OK: 1 | FAIL: 1

```

Nomor slot berubah-ubah tiap siklus — itulah undiannya. [FAIL] yang muncul sesekali adalah tabrakan: kedua node kebetulan mengundi slot yang sama.

Data capture — diukur 120 detik / 72 siklus.

Parameter	Hasil
Siklus yang terekam di kedua node	69
Siklus kedua node mengundi slot sama	31 (44,9 %) — teori 1/ SLOT_COUNT = 50 %
Node 1: OK / FAIL	58 / 15
Node 2: OK / FAIL	41 / 28
[GAP] di gateway	30
Paket diterima gateway	101 dari 144 kemungkinan (70 %)

Tabel 13.5: Hasil pengamatan EXP-01 — Mode A (Slotted ALOHA)

Hubungan tabrakan dengan kegagalan — inilah inti EXP-01, dan datanya tidak menyisakan keraguan.

Kejadian	Keduanya sukses	N1 saja gagal	N2 saja gagal	Keduanya gagal
Slot berbeda (38 siklus)	38	0	0	0
Slot sama (31 siklus)	0	3	16	12

Tabel 13.6: Hubungan tabrakan slot dengan kegagalan pada Mode A

Dibaca begini: selama kedua node mengundi slot yang berbeda, **tidak ada satu pun kegagalan** — 38 dari 38 siklus mulus. Begitu keduanya mengundi slot yang sama, **tidak pernah keduanya selamat**. Jadi di modul ini tabrakan bukan salah satu penyebab kegagalan, melainkan **satu-satunya** penyebabnya.

Efek capture. Dari 31 tabrakan, 19 di antaranya masih menyelamatkan satu paket — dan yang selamat hampir selalu Node 1 (16 : 3). Penerima LoRa mampu mengunci sinyal yang lebih kuat dan mengabaikan yang lebih lemah, sehingga tabrakan tidak selalu berarti dua-duanya hancur. Catat RSSI kedua node di gateway, lalu jelaskan mengapa pemenangnya konsisten.

CHECKPOINT

[FAIL] **harus** muncul di kedua node, dan proporsi siklus “slot sama” harus mendekati 50 % (bukan 25 %: dengan 2 slot, peluang node kedua menabrak pilihan node pertama adalah 1 dari 2). Kalau [FAIL] nihil sama sekali, hampir pasti mode masih 0 — periksa baris Mode : di pembuka serial node.

BUKA ABSTRAKSINYA

Dua kali sepanjang sesi, gateway mencetak paket yang **terbaca tetapi isinya cacat**:

```
Node      : 2
SEQ       : 0
Ruang 1   : 0.0 C, 0 %
RSSI      : -34 dBm   SNR : -4.50 dB
```

Paket itu lolos karena `LoRa.enableCrc()` **tidak** dipanggil di modul ini, sehingga radio tidak membuang payload yang rusak (bandingkan dengan Bab 8. yang mengaktifkannya). Akibatnya bukan sekadar satu baris aneh: SEQ terbaca 0 padahal aslinya sudah puluhan, dan penghitung hilang milik gateway langsung melonjak dari 24 menjadi 89. Telusuri di `src/gateway/main.cpp` bagaimana hilang dihitung, jelaskan mengapa satu paket cacat bisa merusak seluruh statistik, lalu tentukan: angka mana yang layak dipercaya sebagai ukuran kegagalan — hilang di gateway, atau FAIL di node?

13.8.2 EXP-02 — Mode B (Assigned Slot) = TDMA

Ubah mode menjadi 0, unggah ulang **kedua node** (gateway biarkan), lalu rekam dengan durasi yang sama.

Expected output — node

```
Mode : B (Assigned Slot) -- tetap di slot 0

[TX] cycle=0 slot=0 | NODE=1,SEQ=0,R1T=28.4,R1H=63,R2T=24.7,R2H=71
[OK] ACK diterima | OK: 1 | FAIL: 0
```

Data capture — diukur 120 detik / 72 siklus, sesi yang sama dengan EXP-01.

Parameter	Hasil
Siklus kedua node memakai slot sama	0 — mustahil secara konstruksi
Node 1: OK / FAIL	73 / 0
Node 2: OK / FAIL	69 / 2
[GAP] di gateway	0
Paket diterima gateway	146 (N1 = 72, N2 = 71)

Tabel 13.7: Hasil pengamatan EXP-02 — Mode B (TDMA)

CHECKPOINT

Terpenuhi. Nol [GAP], nol paket hilang di gateway, sepanjang 72 siklus penuh.

Dua **[FAIL]** yang **bukan tabrakan**. Node 2 mencatat 2 kegagalan, tetapi gateway mencatat **nol** paket hilang untuk Node 2 — jadi datanya sampai, dan yang hilang adalah **ACK balasannya**. Kegagalan jenis ini tidak bisa dihapus oleh penjadwalan slot secanggih apa pun, sebab penyebabnya bukan tabrakan melainkan link radio arah balik. Bandingkan dengan kasus serupa di M08B (Tabel 10.13).

13.8.3 EXP-03 — Slot Observed dibanding Assigned

Bandingkan kolom Slot yang dicetak gateway dengan slot yang seharusnya (0 untuk Node 1, 1 untuk Node 2 pada Mode B).

Data capture

Parameter	Hasil
Apakah Slot di gateway selalu cocok dengan assignment (Mode B)?	ya — Node 1 selalu slot 0, Node 2 selalu slot 1, tanpa kecuali

Parameter	Hasil
Durasi rata-rata satu siklus	$\approx 1,63$ detik — dekat teori ($\text{SLOT_COUNT} \times \text{SLOT_DURATION_MS} = 1600$ ms)
Apakah durasi siklus stabil?	ya — 72 siklus berturut-turut dalam 120 detik, tanpa pelebaran nyata

Tabel 13.8: Hasil pengamatan EXP-03 — slot teramati dibanding slot assignment

CHECKPOINT

Terpenuhi. Slot yang teramati selalu sama dengan slot yang di-assign, dan panjang siklus stabil.

13.8.4 Verifikasi Perangkat Keras

CATATAN

Diuji di perangkat pada 23 Agustus 2026 — tiga Arduino Uno asli + Dragino LoRa Shield v1.2 (gateway /dev/ttyACM0, node1 /dev/ttyACM1, node2 /dev/ttyACM2). **Kedua mode dijalankan pada sesi yang sama**, masing-masing 120 detik / 72 siklus, dengan firmware gateway yang tidak diubah di antaranya — hanya [slot] mode di platformio.ini yang diganti lalu kedua node di-flash ulang. Seluruh angka pada EXP-01, EXP-02, dan EXP-03 berasal dari sesi itu. Tabel 13.19 diukur pada hari yang sama lewat dua sesi tambahan dengan SLOT_COUNT 3 (150 detik / 61 siklus) dan 4 (190 detik / 58 siklus), keduanya pada Mode A. Log mentah keempat sesi ada di logserial.md.

13.8.5 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen.

Hasil aktual dari perangkat. Baud **115200**, frekuensi **433 MHz**, SF7 / BW 125 kHz / CR 4/5 / 17 dBm, SLOT_COUNT=2, SLOT_DURATION_MS=800, SLOT_GUARD_MS=50.

Kedua mode direkam pada sesi yang sama, masing-masing 120 detik / 72 siklus, dengan firmware gateway yang sama sekali tidak diubah di antaranya — hanya [slot] mode di platformio.ini yang diganti lalu kedua node di-flash ulang. Itu sebabnya angkanya boleh dibandingkan langsung.

Board & Port

Peran	Environment	Port	Board
Gateway	gateway	/dev/ttyACM0	Uno asli (2341:0043)
Node 1	node1	/dev/ttyACM1	Uno asli (2341:0043)
Node 2	node2	/dev/ttyACM2	Uno asli (2341:0043)

Tabel 13.9: Log serial Modul 10: Board & Port

EXP-01 — Mode A (Random Slot) = Slotted ALOHA

Node mengundi slotnya tiap siklus, dan nomor slot di log ikut berubah-ubah:

```
[21:41:34] [ 1.6] Mode : A (Random Slot) -- slot diundi tiap siklus
[21:41:35] [ 2.6] [TX] cycle=0 slot=1 | NODE=1,SEQ=0,R1T=26.5,R1H=54,R2T=23.2,R2H=59
[21:41:36] [ 3.3] [FAIL] Tidak ada ACK dalam slot ini | OK: 0 | FAIL: 1
[21:41:36] [ 3.4] [TX] cycle=1 slot=0 | NODE=1,SEQ=1,R1T=27.4,R1H=57,R2T=23.6,R2H=57
[21:41:36] [ 3.5] [OK] ACK diterima | OK: 1 | FAIL: 1
...
[21:41:54] [ 22.1] [FAIL] Tidak ada ACK dalam slot ini | OK: 11 | FAIL: 2
[21:41:56] [ 23.8] [TX] cycle=13 slot=1 | NODE=1,SEQ=13,...
[21:41:56] [ 23.9] [OK] ACK diterima | OK: 12 | FAIL: 2
```

Gateway melihat akibatnya sebagai SEQ yang meloncat:

```
[21:41:46] [ 13.3] [GAP] SEQ meloncat 1 -- data hilang pada siklus tersebut (mis. tabrakan slot)
[21:41:53] [ 20.6] [GAP] SEQ meloncat 2 -- data hilang pada siklus tersebut (mis. tabrakan slot)
```

Rekapitulasi 120 detik / 72 siklus

Parameter	Hasil
Siklus terekam di kedua node	69
Siklus kedua node mengundi slot sama	31 (44,9 %) — teori 1/SLOT_COUNT = 50 %
Node 1: OK / FAIL	58 / 15
Node 2: OK / FAIL	41 / 28
[GAP] gateway	30
Paket diterima gateway	101 dari 144 kemungkinan (70 %)
Slot yang dipilih Node 1 (slot 0 / slot 1)	33 / 40
Slot yang dipilih Node 2 (slot 0 / slot 1)	35 / 35

Tabel 13.10: Log serial Modul 10: EXP-01 — Mode A (Random Slot) = Slotted ALOHA

Tabrakan adalah satu-satunya penyebab kegagalan. Hasil pencocokan slot kedua node siklus demi siklus:

Kejadian	Jumlah	Keduanya sukses	N1 saja gagal	N2 saja gagal	Keduanya gagal
Slot berbeda	38	38	0	0	0
Slot sama	31	0	3	16	12

Tabel 13.11: Log serial Modul 10: EXP-01 — Mode A (Random Slot) = Slotted ALOHA

Tidak ada satu pun kegagalan ketika slotnya berbeda, dan tidak pernah keduanya selamat ketika slotnya sama. Dari 31 tabrakan, 19 masih menyisakan satu paket yang selamat — dan pemenangnya hampir selalu Node 1 (16 : 3), yaitu **efek capture**: penerima mengunci sinyal yang lebih kuat.

Dua paket cacat yang lolos. Tabrakan tidak selalu berakhir sunyi; dua kali gateway menerima paket yang terbaca tetapi isinya rusak:

```
[21:43:21] [ 108.8] === PAKET DITERIMA ===
[21:43:21] [ 108.8] Node      : 2
[21:43:21] [ 108.8] SEQ       : 0
[21:43:21] [ 108.8] Ruang 1  : 0.0 C, 0 %
[21:43:21] [ 108.8] Ruang 2  : 0.0 C, 57 %
[21:43:21] [ 108.8] RSSI     : -34 dBm
[21:43:21] [ 108.8] SNR      : -4.50 dB
```

Paket itu lolos karena modul ini tidak memanggil `LoRa.enableCrc()` (bandingkan M07B yang mengaktifkannya). Akibatnya tidak berhenti di satu baris aneh: SEQ terbaca 0 padahal aslinya sudah puluhan, dan penghitung hilang untuk Node 2 melonjak dari 24 (siklus 60) menjadi 89 di akhir sesi — jauh melebihi jumlah paket yang benar-benar dikirim. **Angka FAIL di sisi node adalah ukuran kegagalan yang jujur di sesi ini, bukan hilang di gateway.** Perbaikannya ada di Challenge CH-5.

EXP-02 — Mode B (Assigned Slot) = TDMA

Slot tidak lagi diundi; tiap node terkunci di slotnya sendiri sepanjang sesi.

```
[21:45:08] [ 1.6] Mode : B (Assigned Slot) -- tetap di slot 0
[21:47:06] [ 117.6] --- Cycle 70 selesai | N1: diterima=71 hilang=0 N2: diterima=70 hilang=0 ---
[21:47:08] [ 119.2] --- Cycle 71 selesai | N1: diterima=72 hilang=0 N2: diterima=71 hilang=0 ---
```

Rekapitulasi 120 detik / 72 siklus

Parameter	Hasil
Siklus kedua node memakai slot sama	0
Slot yang dipakai Node 1 / Node 2	selalu 0 / selalu 1
Node 1: OK / FAIL	73 / 0
Node 2: OK / FAIL	69 / 2
[GAP] gateway	0
Paket diterima gateway	146 (N1 = 72, N2 = 71)
hilang menurut gateway	0 untuk kedua node

Tabel 13.12: Log serial Modul 10: EXP-02 — Mode B (Assigned Slot) = TDMA

Dua [FAIL] yang bukan tabrakan. Node 2 mencatat 2 kegagalan, tetapi gateway mencatat **nol** paket hilang untuk Node 2 — datanya sampai, yang hilang adalah **ACK balasannya**. Kegagalan jenis ini tidak dapat dihapus oleh penjadwalan slot, sebab penyebabnya link arah balik, bukan tabrakan. Kasus yang sama pernah tertangkap di M08B.

Perbandingan langsung

Ukuran	Mode A — Slotted ALOHA	Mode B — TDMA
Siklus “slot sama”	31 dari 69 (44,9 %)	0
Node 1 OK / FAIL	58 / 15	73 / 0
Node 2 OK / FAIL	41 / 28	69 / 2
[GAP] gateway	30	0
Paket diterima gateway	101	146
Keberhasilan keseluruhan	70 %	≈99 %

Tabel 13.13: Log serial Modul 10: Perbandingan langsung

Selisihnya bukan soal setelan, melainkan soal protokol: mengundi slot menyisakan peluang tabrakan 1/ SLOTT_COUNT yang tidak pernah nol, sedangkan slot tetap menghapusnya secara konstruksi. Yang tersisa di Mode B hanyalah kegagalan yang memang di luar jangkauan penjadwalan — ACK yang hilang di udara.

Tabel D — sapuan SLOTT_COUNT (Mode A)

EXP-01 diulang dengan jumlah slot 3 dan 4, mengubah [slot] count di platformio.ini lalu mem-flash ketiga board (gateway ikut, sebab jendela dengarnya memanjang).

SLOTT_COUNT	Durasi sesi	Siklus	Panjang siklus	Tabrakan teori	Tabrakan terukur	Keberhasilan	Throughput
2	116 detik	72	1,63 detik	50 %	31/69 = 44,9 %	99/142 = 69,7 %	0,85 paket/detik
3	146 detik	61	2,43 detik	33,3 %	20/61 = 32,8 %	101/122 = 82,8 %	0,69 paket/detik
4	184 detik	58	3,23 detik	25 %	16/55 = 29,1 %	95/113 = 84,1 %	0,52 paket/detik

Tabel 13.14: Log serial Modul 10: Tabel D — sapuan SLOTT_COUNT (Mode A)

Distribusi slot yang diundi juga merata, seperti yang diharapkan dari `random(0, SLOTT_COUNT)`:

SLOT_COUNT	Node 1	Node 2
3	16 / 22 / 23	17 / 21 / 23
4	12 / 15 / 14 / 16	14 / 15 / 7 / 20

Tabel 13.15: Log serial Modul 10: Tabel D — sapuan SLOT_COUNT (Mode A)

Dua kesimpulan yang berlawanan arah: peluang tabrakan turun persis seperti ramalan $1/\text{SLOT_COUNT}$ (selisih $-5,1$, $-0,5$, dan $+4,1$ poin persen terhadap teori, semuanya di dalam simpangan baku binomial ≈ 6 poin untuk sampel sebesar ini), **tetapi** paket sukses per detik justru turun dari 0,85 menjadi 0,52 karena tiap slot tambahan memperpanjang siklus 800 ms sementara node tetap mengirim satu paket per siklus.

Ringkasan Verifikasi Hardware

Diuji di perangkat pada **2026-08-23**: 3× Arduino Uno asli + Dragino LoRa Shield v1.2 (/dev/ttyACM0/1/2). Kedua mode dijalankan berurutan pada sesi yang sama, masing-masing 120 detik / 72 siklus. Perpindahan mode dilakukan lewat [slot] mode di platformio.ini (build flag SLOT_MODE_RANDOM), tanpa menyentuh src/node/main.cpp dan tanpa mem-flash ulang gateway. Seluruh angka pada README bagian Percobaan dan Pengukuran berasal dari sesi ini.

Tabel D diukur pada hari yang sama lewat dua sesi tambahan (SLOT_COUNT 3 dan 4). Yang **belum** dijalankan dan diserahkan sebagai pekerjaan praktikum: seluruh Challenge, termasuk CH-5 (LoRa.enableCrc()) yang lahir dari temuan paket cacat di EXP-01.

13.9 Pengukuran

A. Slotted ALOHA dibanding TDMA (120 detik / 72 siklus, sesi yang sama)

Ukuran	Mode A — Slotted ALOHA	Mode B — TDMA
Siklus “slot sama”	31 dari 69 (44,9 %)	0
Node 1 OK / FAIL	58 / 15	73 / 0
Node 2 OK / FAIL	41 / 28	69 / 2
Paket diterima gateway	101	146
[GAP] gateway	30	0
Keberhasilan keseluruhan	70 %	≈ 99 %

Tabel 13.16: Lembar pengukuran A — Slotted ALOHA dibanding TDMA

B. Distribusi slot Mode A

Slot terpilih	Node 1	Node 2
0	33	35
1	40	35

Tabel 13.17: Lembar pengukuran B — distribusi slot pada Mode A

Undian yang seimbang: masing-masing mendekati 50/50 dari ± 70 siklus, seperti yang diharapkan dari `random(0, SLOT_COUNT)`.

C. Tabrakan dan akibatnya (Mode A)

Kejadian	Jumlah	Keduanya sukses	Satu selamat (capture)	Keduanya gagal
Slot berbeda	38	38	—	0

Kejadian	Jumlah	Keduanya sukses	Satu selamat (capture)	Keduanya gagal
Slot sama	31	0	19	12

Tabel 13.18: Lembar pengukuran C — tabrakan dan akibatnya pada Mode A

D. Pengaruh jumlah slot — EXP-01 diulang dengan SLOT_COUNT 3 dan 4. Cukup ubah [slot] count di platformio.ini; angka itu dibaca gateway **dan** kedua node, jadi ketiganya mustahil berbeda. Ketiga board perlu di-flash ulang, sebab jendela dengar gateway ikut memanjang.

SLOT_COUNT	Panjang siklus	Teori $1/N$	Tabrakan terukur	Keberhasilan	Throughput
2	1,63 detik	50 %	44,9 % (31/69)	69,7 % (99/142)	0,85 paket/detik
3	2,43 detik	33,3 %	32,8 % (20/61)	82,8 % (101/122)	0,69 paket/detik
4	3,23 detik	25 %	29,1 % (16/55)	84,1 % (95/113)	0,52 paket/detik

Tabel 13.19: Lembar pengukuran D — pengaruh jumlah slot pada Mode A

Tiga titik ini sudah cukup untuk melihat dua hal sekaligus, dan keduanya berlawanan arah.

1. **Teori $1/N$ / SLOT_COUNT terbukti.** Selisih terukur terhadap teori berturut-turut – 5,1, – 0,5, dan +4,1 poin persen — semuanya di dalam sebaran wajar untuk sampel 55–69 siklus (simpangan baku binomial ≈ 6 poin). Memperbanyak slot memang menurunkan peluang tabrakan, persis sebesar yang diramalkan.
2. **Tetapi throughput justru turun.** Keberhasilan naik dari 69,7 % ke 84,1 %, sementara paket sukses per detik **turun** dari 0,85 menjadi 0,52 — sebab tiap slot tambahan memperpanjang siklus 800 ms, sedangkan dua node tetap hanya mengirim satu paket per siklus. Slot ke-3 dan ke-4 dibayar dengan waktu yang menganggur.

Dua node dengan empat slot berarti separuh jatah waktu terbuang. Inilah alasan Slotted ALOHA tidak selesai hanya dengan memperbanyak slot, dan mengapa TDMA — yang jumlah slotnya **pas** sebanyak node — mengalahkan keduanya: pada EXP-02 keberhasilannya ≈ 99 % dengan siklus yang tetap 1,63 detik.

13.10 Analisis

1. Dari Tabel 13.16 dan Tabel 13.18, hitung: berapa persen kegagalan Mode A yang dapat dijelaskan oleh tabrakan slot semata? Bandingkan dengan Mode B yang tabrakannya nol. Apakah tersisa kegagalan yang **bukan** akibat tabrakan — dan dari mana asalnya?
2. Dari Tabel 13.17, hitung proporsi siklus yang kedua node kebetulan memilih slot sama, lalu bandingkan dengan $1/N$ / SLOT_COUNT yang diprediksi teori.
3. Bandingkan [GAP] Mode A modul ini dengan [GAP] M08 (Pure ALOHA) pada kepadatan kirim yang sebanding. Jelaskan mengapa Slotted ALOHA — bahkan pada Mode Random — tetap menghasilkan tabrakan yang lebih jarang.
4. Modul ini menghapus retry M08C sepenuhnya. Hitung dari data sendiri: pada Mode B, apakah keandalan yang dicapai tanpa retry ini sama atau lebih baik daripada M08C (dengan retry) pada kondisi kanal yang sebanding? Jelaskan.
5. Rancang, tanpa menulis kodenya, cara menambah node ketiga pada Mode B tanpa membuatnya berbagi slot dengan Node 1 atau Node 2. Apa yang harus berubah pada SLOT_COUNT, dan apa akibatnya terhadap panjang satu siklus penuh?

13.11 Concept Check

1. Apa bedanya penjadwalan M05 (master memanggil) dengan penjadwalan M10 (gateway hanya menyiarkan SYNC)?
2. Mengapa vulnerable period Slotted ALOHA hanya satu kali waktu udara, sedangkan Pure ALOHA dua kali?
3. Mengapa Mode Assigned menghapus tabrakan struktural, sedangkan Mode Random tidak — walau keduanya sama-sama memakai slot?
4. Mengapa retry dan random backoff M08C dihapus pada modul ini, bukan digabungkan begitu saja ke dalam slot?
5. Apa yang terjadi pada seluruh sistem bila SYNC dari gateway hilang di jalan pada satu siklus?

13.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Memperbesar jaringan dan memulihkan Mode Random

CH-1 — Node ketiga Tambahkan environment node3, naikan SLOT_COUNT menjadi 3 pada gateway dan ketiga node, lalu ukur ulang Tabel 13.16 dan Tabel 13.17 pada Mode B dan Mode A dengan tiga node.

CH-2 — Backoff hanya untuk Mode Random Tambahkan random backoff kecil (dalam batas satu slot, bukan lintas-slot) khusus untuk Mode A: bila ACK tidak diterima, node mengundi ulang slot untuk siklus berikutnya alih-alih memakai slot terakhir yang gagal. Bandingkan hasilnya dengan Mode A tanpa strategi ini.

CH-3 — Deteksi drift Tambahkan pencatatan selisih waktu kedatangan tiap paket terhadap awal slot yang diharapkan di gateway, lalu jalankan modul selama beberapa menit untuk melihat apakah selisih itu melebar seiring waktu antar-SYNC.

Level 3 — Tantangan · Statistik yang jujur dan jadwal yang optimal

CH-5 — Buang paket cacat dengan CRC Tambahkan `LoRa.enableCrc()` di gateway dan kedua node (lihat Bab 8. yang sudah memakainya), lalu ulangi EXP-01. Hitung: berapa paket yang tadinya “diterima” kini hilang sama sekali, apakah baris Ruang 1 : 0.0 C masih muncul, dan apakah penghitung hilang di gateway berhenti melonjak. Jelaskan mengapa membuang paket cacat justru membuat statistiknya lebih jujur.

CH-4 — Cari jumlah slot yang optimal Tabel 13.19 sudah menunjukkan bahwa menambah slot menaikkan keberhasilan tetapi menurunkan throughput untuk 2 node. Lanjutkan dengan node ketiga (CH-1), ukur ulang ketiga nilai SLOT_COUNT, lalu tentukan: untuk n node, berapa SLOT_COUNT yang menghasilkan paket-sukses-per-detik tertinggi, dan mengapa jawabannya tidak selalu sama dengan n ?

TODO

Urutan penomoran Challenge pada berkas sumber (`Modul10_lora_slotted_aloha_tdma/README.md`) tidak berurutan: CH-5 ditulis sebelum CH-4. Urutan isinya dipertahankan apa adanya di sini agar cocok dengan sumber. Mohon dikonfirmasi apakah penomorannya perlu ditukar, atau urutan penyajiannya yang perlu disesuaikan.

13.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.

2. Dasar teori ringkas — slot, SYNC, vulnerable period slotted dibanding pure, Mode Assigned dibanding Random.
3. Konfigurasi — SLOT_COUNT, SLOT_DURATION_MS, SLOT_GUARD_MS, kontrak SYNC=, NODE=, dan ACK=.
4. Hasil eksperimen — log serial ketiga board (EXP-01 sampai EXP-03 beserta checkpoint).
5. Data pengukuran — Tabel 13.16 sampai Tabel 13.19.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri: rangkuman seluruh arc M08 → M10 — apa yang diperbaiki tiap modul, dan pilihan mana (retry atau slot) yang lebih tepat untuk skenario nyata seperti apa.

14. Modul 11 — LoRaWAN: Protokol Mengambil Alih

LoRa COMMUNICATION LAB · MODUL 11

Let the Protocol Take Over — LoRaWAN dan ChirpStack

2 × Arduino Uno + LoRa Shield v1.2 dan Raspberry Pi 5 + LoRa GPS HAT v1.4 · topologi bintang, 2 node + server · LoRaWAN Class A (MCCI LMIC) · level Advanced · 1 × 50 menit · folder kode Modul11_lorawan_chirpstack

PENGANTAR MODUL · Gambaran Umum

Modul 11 dirancang untuk satu pertemuan (1 × 50 menit) pada tingkat lanjut, dan membuka arc ketiga lab ini. Sepuluh modul sebelumnya membangun sendiri semua yang menyerupai jaringan: alamat node di M05, ACK di M04 dan M08B, retry di M08C, penjadwalan di M10. Semuanya buatan tangan, semuanya di lapisan aplikasi, dan semuanya berhenti bekerja begitu ada node ketiga atau gateway kedua. Modul ini menyerahkan seluruh pekerjaan itu kepada **protokol** — LoRaWAN — lalu memakai sisa waktunya untuk satu pertanyaan: apa yang sebenarnya didapat sebagai gantinya, dan apa yang diserahkan.

14.1 Pendahuluan

Yang didapat: join dengan kunci (OTAA), alamat perangkat yang diberikan server (DevAddr), enkripsi payload ujung-ke-ujung, penomoran frame anti-ulang (FCnt), dan dua jendela downlink terjadwal. Yang diserahkan: kendali atas kapan node boleh bicara, dan kemampuan membaca isi paket dari gateway — sebab **gateway LoRaWAN tidak memegang kunci aplikasi**. Pemisahan peran itulah inti modul ini, dan cara membuktikannya sederhana: `single_chan_pkt_fwd.py` mencetak DevAddr dan FCnt tiap paket, tetapi tidak pernah bisa mencetak $T=27.4$, $H=68$. Angka itu baru muncul di ChirpStack.

Prasyaratnya M06 — driver register SX1276 di Python — karena gateway modul ini adalah kelanjutan langsung dari Modul06_rpi_lora_python/src/receiver.py, dengan register yang sama dan hanya beberapa tambahan (sync word LoRaWAN, header eksplisit, IQ terbalik untuk downlink). Prasyarat keduanya M08–M10 (arc akses kanal), karena gateway kanal tunggal di sini setengah-dupleks: saat mengirim downlink ia tuli, dan uplink yang datang tepat pada saat itu hilang — persis tabrakan yang diukur di M08, kini dari sisi yang lain.

Peta modul LoRa

Modul	Fokus (yang ditumpuk di atas modul sebelumnya)
06	Library dilepas — register SX1276 dipegang langsung dari Python
07	Penjadwal pindah ke gateway Linux, firmware node tidak berubah
08–08C	Akses kanal tanpa penjadwal: ALOHA telanjang, lalu ACK, lalu retry acak
09	Carrier sense — dengar dulu sebelum bicara (CSMA/CA)
10	Slot waktu bersama: Slotted ALOHA (Mode A) dan TDMA (Mode B)
11 (ini)	Seluruh lapisan MAC buatan tangan diganti protokol: LoRaWAN + ChirpStack

Tabel 14.1: Kedudukan Modul 11 terhadap modul sebelumnya

Kontrak data modul ini. Payload sengaja dibuat **sesederhana mungkin dan tanpa encoding**: ASCII polos $T=<\text{suhu}>$, $H=<\text{kelembapan}>$, dikirim *unconfirmed* di FPort 1. Pengemasan biner yang hemat byte adalah bahan modul berikutnya; di sini fokusnya tetap pada alur **Register Device** → **OTAA Join** → **Uplink** → **data terlihat di ChirpStack**. Rentang dummy tiap node dibedakan supaya asal data langsung terbaca di layar tanpa mencocokkan DevEUI.

Node	Lokasi	Temperature	Humidity	Contoh payload
Node 1	Ruangan 1	25–30 °C	60–75 %	T=27.4,H=68
Node 2	Ruangan 2	28–35 °C	40–65 %	T=31.2,H=52

Tabel 14.2: Rentang data dummy tiap node Modul 11

14.2 Capaian Pembelajaran

Setelah menyelesaikan modul ini, praktikan mampu:

Menyerahkan lapisan MAC kepada protokol standar

1. Membedakan peran **gateway**, **network server**, dan **application server** pada LoRaWAN, serta membuktikan dari log bahwa gateway meneruskan paket tanpa dapat membaca isinya.
2. Mendaftarkan perangkat OTAA di ChirpStack (DevEUI, JoinEUI, AppKey) dan menjelaskan mengapa DevEUI ditulis terbalik urutan bytenya di firmware sementara AppKey tidak.
3. Menjelaskan sekuens **OTAA join**: JoinRequest berisi DevNonce dan MIC, JoinAccept dibalas di jendela RX1 pada detik ke-5, lalu NwkSKey dan AppSKey diturunkan di kedua sisi dan DevAddr dipakai untuk seluruh uplink berikutnya.
4. Menjelaskan keterbatasan **single channel gateway** (satu frekuensi, satu SF, setengah-dupleks) dan menerapkan penguncian kanal di sisi node agar tetap dapat join.
5. Membaca uplink kedua node di ChirpStack dan lewat integrasi MQTT, serta menafsirkan FCnt, RSSI, SNR, dan asal data berdasarkan rentang dummy tiap ruangan.

Kriteria keberhasilan

- ☐ Gateway muncul **online** di ChirpStack dan mencetak [RX] ... JoinRequest DevEUI=... saat node dinyalakan.
- ☐ Kedua node mencetak [JOIN] BERHASIL beserta DevAddr yang berbeda, dan status keduanya di ChirpStack berubah menjadi ter-aktivasi.
- ☐ Uplink kedua node terlihat berselang-seling di Applications → praktikum-wsn → Devices → Events, dengan payload $T=...$, $H=...$ yang rentangnya sesuai ruangnya.
- ☐ FCnt tiap node naik satu per satu tanpa lompat besar selama pengamatan berlangsung.

14.3 Dasar Teori (Secukupnya)

Istilah	Definisi kerja di lab ini
Gateway	Penerus paket. Menangkap apa pun yang mengudara pada kanalnya, membungkusnya dalam UDP, mengirimkannya ke network server. Tidak punya kunci aplikasi, tidak tahu isi paket, tidak memutuskan apa pun.
Network server (ChirpStack)	Otak jaringan. Memeriksa MIC, menghapus duplikat dari banyak gateway, mengelola FCnt, menjawab join, dan menjadwalkan downlink.
Application server	Tujuan akhir data. Pada ChirpStack v4 menyatu dengan network server, dan pintu keluarnya adalah integrasi MQTT yang dipakai <code>uplink_listen.py</code> .

Istilah	Definisi kerja di lab ini
OTAA	<i>Over-The-Air Activation</i> . Perangkat join memakai DevEUI + JoinEUI + AppKey; kunci sesi diturunkan saat join, tidak pernah dikirim di udara.
ABP	Alternatifnya: kunci sesi ditanam langsung, tanpa join. Lebih mudah, tetapi tidak dipakai modul ini justru karena join-lah yang mau diamati.
DevEUI / JoinEUI	Identitas 64-bit perangkat dan server join. Di firmware LMIC ditulis LSB-first ; di layar ChirpStack tampil MSB-first.
AppKey	Rahasia 128-bit. Ditulis MSB-first di kedua sisi (tidak dibalik). Dari sinilah NwkSKey dan AppSKey diturunkan.
DevAddr	Alamat 32-bit yang diberikan server saat join. Bandingkan dengan NODE_ID M05–M10 yang ditentukan sendiri lewat build flag.
FCnt	Penghitung frame. Server menolak frame dengan FCnt yang sudah lewat — inilah anti-replay yang di M08C harus dibuat sendiri lewat SEQ.
FPort	Nomor port aplikasi (1–223). Memisahkan jenis pesan dalam satu perangkat, tanpa perlu menaruh penanda di dalam payload.
Class A	Kelas paling hemat daya: node bicara kapan saja (seperti ALOHA di M08), lalu membuka dua jendela RX pendek sesudahnya. Server hanya boleh menjawab di jendela itu.
RX1 / RX2	Jendela pertama: 1 detik sesudah uplink (5 detik untuk JoinAccept), frekuensi dan SF sama dengan uplink. Jendela kedua: 2 detik sesudahnya, di frekuensi tetap (EU433: 434,665 MHz SF12).
Sync word 0x34	Penanda jaringan publik LoRaWAN. Modul 01–10 memakai 0x12 (privat) — itu sebabnya paket M01–M10 dan paket LoRaWAN tidak saling terdengar walau frekuensinya sama.
Semtech UDP (GWMP)	Protokol antara gateway dan server: PUSH_DATA membawa uplink, PULL_DATA menjaga jalur balik tetap terbuka, PULL_RESP membawa downlink beserta tmst — waktu persis paket harus mengudara.

Tabel 14.3: Istilah kerja Modul 11

Apa yang sebelumnya dibuat tangan, dan siapa yang mengerjakannya sekarang

Kemampuan	Dibuat sendiri di	Disediakan LoRaWAN sebagai
Alamat node	NODE=1 (M05, M08–M10)	DevAddr dari server + DevEUI global
Balasan terima	ACK=<id>, SEQ=<n> (M04, M08B)	Confirmed uplink (satu argumen di LMIC_setTxData2)
Nomor urut dan anti-duplikat	SEQ (M08C)	FCnt, diperiksa server
Kirim ulang	retry + random backoff (M08C)	Retry LMIC + ADR (dimatikan di modul ini)
Giliran bicara	SYNC + slot (M10)	Duty cycle + jendela RX1/RX2
Kerahasiaan	tidak ada — semua terbaca di udara	AES-128: AppSKey (payload) dan NwkSKey (MIC)

Tabel 14.4: Lapisan buatan tangan M04–M10 dan padanannya di LoRaWAN

Gateway hanya melihat amplop. Inilah yang membedakan gateway LoRaWAN dari gateway M07 yang membaca isi paket dengan bebas. Jalankan `single_chan_pkt_fwd.py` dan bandingkan dua barisnya:

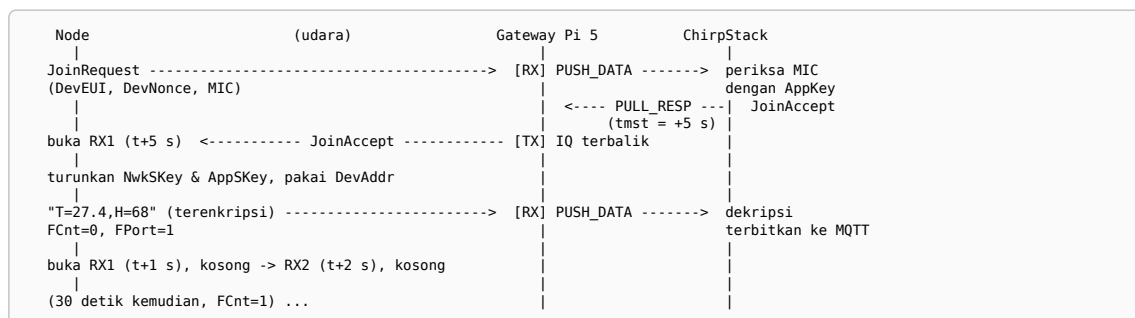
gateway mencetak `UnconfirmedDataUp DevAddr=01D9B47E FCnt=3 FPort=1` (payload terenkripsi), sementara `uplink_listen.py` — yang menerima data setelah melewati network server — mencetak `T=31.8,H=53`. Data yang sama, dua sudut pandang, dan yang membedakannya hanya kepemilikan kunci.

Keterbatasan single channel gateway. Gateway sungguhan memakai konsentrator SX1301 atau SX1302 yang mendengar 8 kanal dikali seluruh SF sekaligus. Modul ini memakai SX1276 — satu modem, satu frekuensi, satu SF pada satu waktu. Tiga akibat yang harus disadari, dan ketiganya terlihat di percobaan:

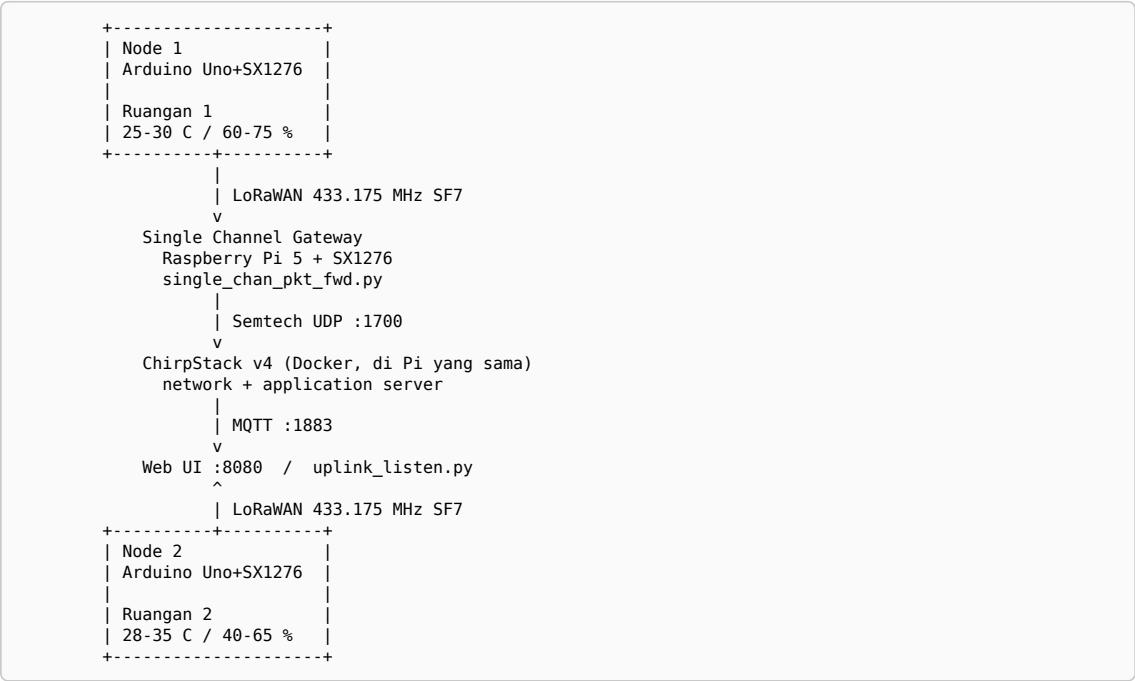
1. **Node wajib dikunci.** Rencana kanal EU433 punya tiga kanal wajib (433,175 / 433,375 / 433,575 MHz). Node yang menyebar uplink ke ketiganya akan kehilangan dua dari tiga pakatnya. Karena itu `lockSingleChannel()` di firmware mematikan semua kanal kecuali satu.
2. **Setengah-dupleks.** Saat menembakkan downlink, gateway tidak mendengarkan. Uplink node lain yang datang tepat pada saat itu hilang tanpa jejak — versi LoRaWAN dari tabrakan M08.
3. **Downlink dijadwalkan dari user space.** Jendela RX1 hanya terbuka beberapa puluh milidetik. Python tidak sepresisi pencacah keras konsentrator, karena itu node memakai `LMIC_setClockError(MAX_CLOCK_ERROR * 20 / 100)` untuk melebarkan jendelanya.

EU433 di atas rencana kanal EU868. Perangkat keras lab ini 433 MHz, dan ChirpStack memang punya region eu433 (aktif secara bawaan; yang perlu diubah hanya awalan topik MQTT gateway bridge — lihat `gateway/docker-compose.override.yml`). Di sisi node, MCCI LMIC tidak menyediakan `CFG_eu433`, jadi dipakai `CFG_eu868` lalu frekuensi kanalnya ditimpa ke 433,175 MHz. Ini sah karena EU433 dan EU868 memakai **tabel datarate yang sama persis** (DR0 = SF12 ... DR5 = SF7 pada BW125), sehingga DR5 di node berarti DR5 yang sama di server. Yang berbeda hanya frekuensi, dan itulah satu-satunya yang ditimpa.

Sekuens yang diamati (join lalu dua uplink)



14.4 Topologi



Perangkat	Environment / skrip	Peran
Node 1	node1	Join OTAA, kirim dummy Ruangan 1 tiap 30 detik
Node 2	node2	Join OTAA, kirim dummy Ruangan 2 tiap 30 detik
Raspberry Pi 5	gateway/single_chan_pkt_fwd.py	Menangkap paket, meneruskan ke ChirpStack, menembakkan downlink
Raspberry Pi 5	ChirpStack v4 (Docker)	Network + application server, web UI, broker MQTT

Tabel 14.5: Peran tiap perangkat Modul 11

Perhatikan bahwa **tidak ada environment gateway** di platformio.ini modul ini — gateway bukan lagi Arduino, seperti halnya master M07 yang juga sudah pindah ke Raspberry Pi.

14.5 Alat yang Digunakan

No	Peralatan	Spesifikasi	Jumlah
1	Arduino Uno	ATmega328P	2
2	Dragino LoRa Shield	v1.2, SX1276, 433 MHz	2
3	Raspberry Pi 5	4/8 GB, Raspberry Pi OS 64-bit	1
4	Dragino LoRa GPS HAT	v1.4, SX1276, 433 MHz	1
5	Antena SMA	wajib terpasang sebelum diberi daya	3
6	Kabel USB tipe B	kabel data	2

Tabel 14.6: Alat dan bahan Modul 11

Pemetaan pin — node (Dragino LoRa Shield v1.2)

Pin Uno	Fungsi	Catatan
D10	NSS / CS	sama seperti M01–M10
D9	RST	sama seperti M01–M10
D2	DIO0	RxDone / TxDone
D6	DIO1	BARU di modul ini. RxTimeout — tanpa ini LMIC tidak pernah tahu kapan harus berhenti menunggu downlink. Sudah tersambung dari pabrik pada shield v1.2.
D11/D12/D13	MOSI/MISO/SCK	SPI

Tabel 14.7: Pemetaan pin sisi node Modul 11

Pemetaan pin — gateway (Dragino LoRa GPS HAT v1.4), sama seperti M06 dan M07.

WiringPi	BCM	Fungsi
GPIO6	25	LoRa_NSS
GPIO0	17	RESET
GPIO7	4	DIO0

Tabel 14.8: Pemetaan pin sisi gateway Modul 11

Struktur proyek

```

Modul11_lorawan_chirpstack/
├── platformio.ini
├── upload_auto.py           ← upload kedua node, port dideteksi sendiri
├── logserial.md             ← cuplikan log aktual dari pengujian perangkat
├── src/node/
│   ├── main.cpp             ← LMIC OTAA + dummy sensor (env node1, node2)
│   └── lorawan_keys.h       ← DevEUI/JoinEUI/AppKey per node
├── gateway/
│   ├── setup_chirpstack.sh  ← seluruhnya dijalankan DI RASPBERRY PI
│   ├── siapkan_gateway.sh   ← pasang Docker + ChirpStack v4 (sekali jalan)
│   ├── lorawan-gateway.service ← siapkan Pi satu kelompok (kanal + kunci + EUI)
│   ├── simpan_image_docker.sh ← unit systemd, bila gateway ingin jalan otomatis
│   ├── docker-compose.override.yml ← bungkus image ChirpStack untuk dipindah
│   ├── single_chan_pkt_fwd.py ← satu-satunya perubahan: region EU433
│   ├── uplink_listen.py     ← gateway kanal tunggal (lanjutan M06)
│   ├── provision_lab.py     ← baca uplink kedua node lewat MQTT + CSV
│   └── requirements.txt     ← (asisten) daftarkan ulang isi server lewat API

```

14.6 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [Modul11_lorawan_chirpstack/](#)

- [Modul11_lorawan_chirpstack/platformio.ini](#)
- [Modul11_lorawan_chirpstack/src/node/main.cpp](#)
- [Modul11_lorawan_chirpstack/src/node/lorawan_keys.h](#)
- [Modul11_lorawan_chirpstack/upload_auto.py](#)
- [Modul11_lorawan_chirpstack/gateway/requirements.txt](#)
- [Modul11_lorawan_chirpstack/gateway/single_chan_pkt_fwd.py](#)
- [Modul11_lorawan_chirpstack/gateway/uplink_listen.py](#)
- [Modul11_lorawan_chirpstack/gateway/provision_lab.py](#)
- [Modul11_lorawan_chirpstack/gateway/setup_chirpstack.sh](#)
- [Modul11_lorawan_chirpstack/gateway/siapkan_gateway.sh](#)
- [Modul11_lorawan_chirpstack/gateway/simpan_image_docker.sh](#)
- [Modul11_lorawan_chirpstack/gateway/docker-compose.override.yml](#)
- [Modul11_lorawan_chirpstack/gateway/lorawan-gateway.service](#)

14.6.1 Sisi Node (Arduino)

```

; Modul 11 - LoRaWAN OTAA ke ChirpStack lewat single channel gateway.
;
; Hanya KEDUA NODE yang dibangun di sini. Gateway modul ini adalah Raspberry
; Pi 5 yang menjalankan gateway/single_chan_pkt_fwd.py -- tidak ada environment
; PlatformIO untuknya, sama seperti M07 yang masternya juga Python.
;
;   pio run -d Modul11_lorawan_chirpstack -e node1 -t upload
;   pio run -d Modul11_lorawan_chirpstack -e node2 -t upload
;   python3 gateway/single_chan_pkt_fwd.py          # di Raspberry Pi
;
; ATAU cukup: python3 Modul11_lorawan_chirpstack/upload_auto.py

[platformio]
default_envs = node1

[env]
platform = atmelavr
board = uno
framework = arduino

; MCCI LoRaWAN LMIC -- stack LoRaWAN 1.0.3 Class A lengkap (join, MIC, enkripsi,
; frame counter, jendela RX1/RX2). Berbeda dari library sandeepmistry/LoRa yang
; dipakai M01-M10: yang itu hanya lapisan PHY (kirim byte), yang ini lapisan MAC.
lib_deps =
    mcci-catena/MCCI LoRaWAN LMIC library@^4.1.1

; KONFIGURASI LMIC LEWAT BUILD FLAG.
; ARDUINO_LMIC_PROJECT_CONFIG_H_SUPPRESS mematikan header konfigurasi bawaan
; library, sehingga seluruh pilihan ditentukan di sini dan terlihat jelas.
;
;   CFG_eu868          : rencana kanal dinamis (frekuensi kanal boleh diatur
;                       sendiri lewat LMIC_setupChannel) -- dipakai untuk
;                       MEMAKSA kanal tunggal 433.175 MHz. Lihat catatan
;                       "EU433 di atas rencana EU868" pada README bagian 3.
;   CFG_sx1276_radio   : radio pada Dragino LoRa Shield v1.2.
;   DISABLE_PING/BEACONS : Class B dimatikan -- modul ini Class A. Menghemat
;                       ~1 KB flash dan RAM yang sangat berharga di ATmega328P.
;   LMIC_ENABLE_arbitrary_clock_error : mengizinkan LMIC_setClockError() dengan
;                                       nilai besar, supaya jendela RX node dibuka lebih lebar
;                                       dan toleran terhadap gateway Python yang penjadwalan
;                                       downlink-nya tidak sepresisi konsentrator SX1301.
build_flags =
    -D ARDUINO_LMIC_PROJECT_CONFIG_H_SUPPRESS
    -D CFG_eu868=1
    -D CFG_sx1276_radio=1
    -D DISABLE_PING
    -D DISABLE_BEACONS
    -D LMIC_ENABLE_arbitrary_clock_error=1
    -D DISABLE_LMIC_FAILURE_TO
    -D LMIC_PRINTF_TO=Serial

; NOMOR KELOMPOK. Satu-satunya baris yang perlu diubah tiap kelompok bila
; beberapa meja bekerja bersamaan di satu ruangan. Nilainya menentukan kanal
; EU433 sekaligus DevEUI/AppKey kedua node:
;
;   1 -> 433.175 MHz, DevEUI 00112233445566xx, AppKey 00112233...F1/F2
;   2 -> 433.375 MHz, DevEUI 00112233445577xx, AppKey 01112233...F1/F2
;   3 -> 433.575 MHz, DevEUI 00112233445588xx, AppKey 02112233...F1/F2
;
; Frekuensi ini HARUS sama dengan --freq pada gateway/single_chan_pkt_fwd.py.
; Lampiran B di README menjelaskan alurnya untuk beberapa kelompok sekaligus.
    -D KELOMPOK=1

monitor_speed = 115200

; Kedua node memakai source yang sama; nomornya ditentukan build flag, dan
; NODE_ID itulah yang memilih blok kunci OTAA di src/node/lorawan_keys.h
; sekaligus rentang dummy sensornya.
[env:node1]
build_src_filter = +<node/*.cpp>
build_flags = ${env.build_flags} -D NODE_ID=1
upload_port = /dev/ttyACM1
monitor_port = /dev/ttyACM1

[env:node2]
build_src_filter = +<node/*.cpp>
build_flags = ${env.build_flags} -D NODE_ID=2
upload_port = /dev/ttyACM2
monitor_port = /dev/ttyACM2

```

Kode 14.1: platformio.ini Modul 11 — environment node1 dan node2, flag KELOMPOK

```

/*
LoRaWAN Class A - Node OTAA
Arduino Uno (ATmega328P) + Dragino LoRa Shield v1.2 (SX1276)
Library : MCCI LoRaWAN LMIC 4.1.x
Environment PlatformIO: node1 / node2

Modul pertama di lab ini yang TIDAK mengirim byte mentah. Sembilan modul
sebelumnya menaruh string apa adanya di udara ("NODE=1,SEQ=3,..."), dan semua
yang menyerupai jaringan -- alamat, ACK, penjadwalan -- dibangun sendiri di
lapisan aplikasi. Di sini pekerjaan itu diserahkan ke protokol: LMIC yang
mengurus join, alamat perangkat (DevAddr), MIC, enkripsi payload, frame
counter, dan dua jendela RX. Yang tersisa untuk kode ini hanyalah "kirim 11
byte ini", persis seperti pada sensor komersial.

Alur satu perangkat:

OTAA join : kirim JoinRequest (DevEUI + JoinEUI + MIC dari AppKey)
            -> ChirpStack membalas JoinAccept di jendela RX1 (5 detik)
            -> NwkSKey & AppSKey diturunkan di kedua sisi, DevAddr dipakai
Uplink     : tiap TX_INTERVAL detik, "T=xx.x,H=xx" unconfirmed di FPort 1
Downlink   : dicek otomatis di RX1/RX2 sesudah tiap uplink (Class A)

Payload sengaja ASCII dan tanpa encoding apa pun ("T=27.4,H=68") supaya di
ChirpStack terlihat langsung tanpa codec. Pengemasan biner yang hemat adalah
bahan modul berikutnya, bukan modul ini.

Dummy sensor dibedakan rentangnya per node, sehingga asal data langsung
terbaca di layar ChirpStack tanpa perlu melihat DevEUI:

Node 1 (Ruangan 1) : suhu 25-30 C, kelembapan 60-75 %
Node 2 (Ruangan 2) : suhu 28-35 C, kelembapan 40-65 %

KANAL TUNGGAL. Gateway modul ini hanya mendengar SATU frekuensi pada SATU
spreading factor (433.175 MHz, SF7BW125). Karena itu seluruh rencana kanal
LoRaWAN dilumpuhkan menjadi satu kanal saja -- lihat lockSingleChannel().

Pin Mapping Dragino Shield v1.2 (sama seperti M01-M10, ditambah DI01):
NSS/CS -> D10, RST -> D9, DI00 -> D2, DI01 -> D6, DI02 -> D7
SCK -> D13, MOSI -> D11, MISO -> D12

DI01 WAJIB pada modul ini, dan inilah bedanya dengan M01-M10. Library
sandeepmistry hanya perlu DI00 (RxDone/TxDone). LMIC juga perlu DI01, yaitu
RxTimeout: sinyal "jendela RX sudah lewat, tidak ada apa-apa" -- tanpa itu
LMIC tidak pernah tahu kapan harus menyerah menunggu downlink. Pada shield
Dragino v1.2, DI01 sudah tersambung ke D6 dari pabrik, tidak perlu kabel.
*/

#include <Arduino.h>
#include <lmic.h>
#include <hal/hal.h>
#include <SPI.h>

#include "lorawan_keys.h"

// — Identitas node —————
#ifndef NODE_ID
#define NODE_ID 1
#endif

#if NODE_ID == 1
#define ROOM_NAME "Ruangan 1"
#define T_MIN_X10 250 // 25.0 C
#define T_MAX_X10 300 // 30.0 C
#define H_MIN 60
#define H_MAX 75
#else
#define ROOM_NAME "Ruangan 2"
#define T_MIN_X10 280 // 28.0 C
#define T_MAX_X10 350 // 35.0 C
#define H_MIN 40
#define H_MAX 65
#endif

// — Parameter LoRaWAN —————
#define TX_INTERVAL 30 // detik antar-uplink (dihitung dari selesainya TX)
#define FPORT 1 // port aplikasi; 1-223 bebas dipakai aplikasi

// Kanal tunggal yang didengar gateway. HARUS sama persis dengan frekuensi

```

```
// yang dipakai gateway (--freq pada single_chan_pkt_fwd.py).
//
// Tiap kelompok memakai satu kanal wajib EU433 sendiri, supaya tiga meja di
// satu ruangan tidak saling menimpa. Nomornya datang dari -DKELOMPOK di
// platformio.ini, dan menggeser DevEUI/AppKey juga -- lihat lorawan_keys.h.
#ifndef KELOMPOK
#define KELOMPOK 1
#endif

#if KELOMPOK == 1
#define SC_FREQ_HZ    433175000UL
#define SC_FREQ_TXT   "433.175"
#elif KELOMPOK == 2
#define SC_FREQ_HZ    433375000UL
#define SC_FREQ_TXT   "433.375"
#elif KELOMPOK == 3
#define SC_FREQ_HZ    433575000UL
#define SC_FREQ_TXT   "433.575"
#else
#error "KELOMPOK harus 1, 2, atau 3 -- satu kanal EU433 untuk tiap kelompok"
#endif
#define SC_DR          DR_SF7           // SF7BW125 = DR5 pada rencana EU433/EU868

// Jendela RX2 menurut EU433. Gateway kanal tunggal tidak pernah memakainya
// (ChirpStack menjawab di RX1), tetapi nilai bawaan LMIC adalah RX2 EU868
// 869.525 MHz -- di luar band perangkat keras 433 MHz, jadi tetap dibetulkan.
#define SC_RX2_FREQ_HZ 434665000UL

// — Pemetaan pin shield untuk LMIC —————
const lmic_pinmap lmic_pins = {
    .nss = 10,
    .rxtx = LMIC_UNUSED_PIN,    // shield tidak punya switch RX/TX terpisah
    .rst = 9,
    .dio = {2, 6, 7},          // DI00 -> D2, DI01 -> D6, DI02 -> D7
    .rxtx_rx_active = 0,
    .rssi_cal = 10,
    .spi_freq = 1000000,
};

// — Kunci OTAA: LMIC memanggil ketiga fungsi ini saat menyusun JoinRequest ———
void os_getArtEui(u1_t *buf) { memcpy_P(buf, APPEUI, 8); }
void os_getDevEui(u1_t *buf) { memcpy_P(buf, DEVEUI, 8); }
void os_getDevKey(u1_t *buf) { memcpy_P(buf, APPKEY, 16); }

static osjob_t sendjob;
static uint8_t payload[16];
static uint16_t txCount = 0;

// =====
// Kanal tunggal
// =====
/*
Rencana kanal EU868 bawaan LMIC punya tiga kanal wajib (868.1/868.3/868.5) dan
menyebarkan uplink ke ketiganya secara acak. Di depan gateway kanal tunggal,
dua dari tiga uplink akan hilang tanpa jejak -- gejala yang membingungkan
karena tidak ada pesan galat sama sekali.

Karena itu kanal 0 didefinisikan ulang ke 433.175 MHz dan kanal 1-8
dimatikan, sehingga hanya ada satu kemungkinan frekuensi. Datarate juga
dikunci di SF7 dan ADR dimatikan, supaya server tidak pernah memindahkan
node ke SF lain yang tidak didengar gateway.

Fungsi ini dipanggil TIGA KALI, dan ketiganya diperlukan:

1. di setup()      -- keadaan awal
2. di EV_JOINING   -- WAJIB. LMIC_startJoining() memanggil initJoinLoop()
                    yang MENGEMBALIKAN tabel kanal ke tiga kanal bawaan
                    EU868 (868.1/868.3/868.5). Tanpa pemanggilan ulang di
                    sini, node mengirim JoinRequest di 868 MHz dan gateway
                    tidak mendengar apa pun -- tanpa satu pun pesan galat.
3. di EV_JOINED    -- JoinAccept boleh membawa daftar kanal tambahan
                    (CFList) yang otomatis dipasang LMIC.
*/
static void lockSingleChannel() {
    LMIC_setupChannel(0, SC_FREQ_HZ, DR_RANGE_MAP(DR_SF12, DR_SF7), BAND_CENTI);
    for (u1_t ch = 1; ch < 9; ch++) {
        LMIC_disableChannel(ch);
    }
    LMIC.dn2Freq = SC_RX2_FREQ_HZ;    // RX2 sesuai EU433, bukan bawaan EU868
    LMIC.dn2Dr = DR_SF12;
    LMIC_setAdrMode(0);               // server tidak boleh memindahkan DR
    LMIC_setLinkCheckMode(0);         // tanpa link check: hemat, dan tidak relevan di 1 kanal
}
```

```

    LMIC_setDrTxpow(SC_DR, 14);
}

// =====
// Payload dummy
// =====
static uint8_t buildPayload() {
    int t10 = random(T_MIN_X10, T_MAX_X10 + 1); // suhu x10, supaya tanpa float
    int h = random(H_MIN, H_MAX + 1);

    // Format sengaja seragam untuk kedua node: "T=27.4,H=68"
    char buf[16];
    snprintf(buf, sizeof(buf), "T=%d.%d,H=%d", t10 / 10, t10 % 10, h);

    uint8_t len = strlen(buf);
    memcpy(payload, buf, len);
    return len;
}

static void do_send(osjob_t *j) {
    if (LMIC.opmode & OP_TXRXPEND) {
        Serial.println(F("[TX] dilewati - masih ada TX/RX berjalan"));
        return;
    }

    uint8_t len = buildPayload();
    payload[len] = '\0';

    // Argumen terakhir 0 = unconfirmed uplink: node tidak menunggu ACK dari
    // server. Bandingkan dengan ACK buatan sendiri di M04/M08B -- di sini
    // confirmed uplink cukup diganti angka 1, dan seluruh mekanismenya sudah
    // disediakan protokol.
    LMIC_setTxData2(FPORT, payload, len, 0);

    txCount++;
    Serial.print(F("[TX #]"));
    Serial.print(txCount);
    Serial.print(F("] FPort="));
    Serial.print(FPORT);
    Serial.print(F(" \\"));
    Serial.print((char *)payload);
    Serial.println(F("\\" -> antre di LMIC));
}

// =====
// Event LMIC
// =====
void onEvent(ev_t ev) {
    switch (ev) {
        case EV_JOINING:
            // initJoinLoop() baru saja mengembalikan tabel kanal ke bawaan
            // EU868 -- kunci ulang ke kanal tunggal 433.175 MHz.
            lockSingleChannel();
            Serial.println(F("[JOIN] mengirim JoinRequest ..."));
            break;

        case EV_JOINED: {
            u4_t netid = 0;
            devaddr_t devaddr = 0;
            u1_t nwkey[16];
            u1_t artkey[16];
            LMIC_getSessionKeys(&netid, &devaddr, nwkey, artkey);

            Serial.println(F("[JOIN] BERHASIL"));
            Serial.print(F("  DevAddr : "));
            Serial.println(devaddr, HEX);
            Serial.print(F("  NetID   : "));
            Serial.println(netid, HEX);

            // JoinAccept boleh membawa CFlst -> kunci ulang ke satu kanal.
            lockSingleChannel();

            do_send(&sendjob);
            break;
        }

        case EV_JOIN_FAILED:
            Serial.println(F("[JOIN] GAGAL - LMIC akan mencoba lagi"));
            break;

        case EV_REJOIN_FAILED:
            Serial.println(F("[JOIN] rejoin gagal"));
    }
}

```

```

        break;

    case EV_TXSTART:
        // Pengaman terakhir, tepat sebelum os_radio(RADIO_TX): LMIC sudah
        // selesai memilih kanal dan datarate, dan di sinilah keduanya
        // dipaksa kembali ke kanal tunggal gateway. Diperlukan karena
        // percobaan join berikutnya selalu menurunkan DR (SF7 -> SF8 ->
        // ... -> SF12), sementara gateway hanya mendengar SF7.
        LMIC.freq = SC_FREQ_HZ;
        if (LMIC.datarate != SC_DR) {
            LMIC.datarate = SC_DR;
            LMIC.rps = setCr(updr2rps(SC_DR), (cr_t)LMIC.errcr);
            LMIC.dndr = SC_DR; // RX1 memakai DR yang sama dengan uplink
        }
        Serial.print(F("[TX] radio: "));
        Serial.print(LMIC.freq / 1000);
        Serial.print(F(" kHz DR"));
        Serial.println(LMIC.datarate);
        break;

    case EV_TXCOMPLETE:
        Serial.print(F("[TX] selesai (FCntUp=)"));
        Serial.print(LMIC.seqnoUp);
        Serial.println(F(""));

        if (LMIC.txrxFlags & TXRX_ACK) {
            Serial.println(F(" ACK dari server diterima"));
        }
        if (LMIC.dataLen > 0) {
            Serial.print(F(" [RX] downlink "));
            Serial.print(LMIC.dataLen);
            Serial.print(F(" byte, RSSI="));
            Serial.print(LMIC.rssi);
            Serial.print(F(" dBm SNR="));
            Serial.println(LMIC.snr / 4);
        }

        os_setTimedCallback(&sendjob,
                           os_getTime() + sec2osticks(TX_INTERVAL),
                           do_send);

        break;

    case EV_JOIN_TXCOMPLETE:
        Serial.println(F("[JOIN] tidak ada JoinAccept di RX1/RX2"));
        break;

    default:
        Serial.print(F("[EV] "));
        Serial.println((unsigned)ev);
        break;
}

}

// =====
void setup() {
    Serial.begin(115200);
    while (!Serial && millis() < 3000) { }

    Serial.println();
    Serial.print(F("=== LoRaWAN NODE "));
    Serial.print(NODE_ID);
    Serial.print(F(" - " ROOM_NAME " (Kelompok)"));
    Serial.print(KELOMPOK);
    Serial.println(F(") ==="));
    Serial.print(F("Kanal : " SC_FREQ_TXT " MHz SF7BW125 (kanal tunggal)\r\nInterval: "));
    Serial.print(TX_INTERVAL);
    Serial.println(F(" detik"));

    // Seed acak dari pin analog mengambang -- cukup untuk dummy sensor, dan
    // membuat kedua node tidak menghasilkan deret angka yang identik.
    randomSeed(analogRead(A0) + NODE_ID * 977);

    os_init();
    LMIC_reset();

    // Toleransi jam. Jendela RX node dilebarkan 20% supaya tetap terbuka saat
    // downlink dari gateway Python datang beberapa milidetik lebih cepat atau
    // lambat dari jadwal. Konsentrator SX1301 asli tidak memerlukan ini;
    // gateway kanal tunggal yang menjadwalkan dari user space memerlukannya.
    LMIC_setClockError(MAX_CLOCK_ERROR * 20 / 100);

    lockSingleChannel();

```

```

    Serial.println(F("Menyusun JoinRequest OTAA ..."));
    LMIC_startJoining();
}

void loop() {
    os_runloop_once();
}

```

Kode 14.2: src/node/main.cpp — LMIC OTAA, penguncian kanal, dan dummy sensor

```

/*
Kunci OTAA Modul 11 -- dipilih oleh build flag -DNODE_ID (platformio.ini).

Tiga identitas yang dipakai LoRaWAN 1.0.x saat join OTAA:

DevEUI (8 byte) identitas unik perangkat, seperti nomor seri
JoinEUI (8 byte) dulu bernama AppEUI -- identitas server join.
ChirpStack tidak memakainya untuk memilih perangkat,
jadi nol semua adalah nilai yang lazim di lab.
AppKey (16 byte) rahasia bersama. Dari kunci inilah NwkSKey dan AppSKey
diturunkan saat join berhasil. Nilai ini HARUS sama
persis dengan yang didaftarkan di ChirpStack.

URUTAN BYTE ADALAH JEBAKAN UTAMA MODUL INI:

DevEUI dan JoinEUI ditulis LSB-first (little endian) di sini, karena itulah
yang diminta os_getDevEui()/os_getArtEui(). ChirpStack menampilkan keduanya
MSB-first (seperti dibaca manusia). Jadi DevEUI yang di layar ChirpStack
tertulis 0011223344556601 harus dibalik urutan bytenya menjadi
{ 0x01, 0x66, 0x55, 0x44, 0x33, 0x22, 0x11, 0x00 } di bawah.

AppKey ditulis MSB-first -- TIDAK dibalik. Persis seperti di ChirpStack.

Salah balik = MIC join request tidak cocok, dan gejalanya paling menyedihkan
di seluruh modul ini: gateway tetap menerima paket, ChirpStack tetap mencatat
frame masuk, tetapi perangkat tidak pernah join. Lihat README EXP-02.

Nilai di bawah adalah kunci LAB -- boleh dipakai apa adanya. Untuk perangkat
sungguhan, AppKey wajib acak dan tidak pernah masuk repository.

NOMOR KELOMPOK IKUT MENENTUKAN KUNCI. Bila beberapa kelompok bekerja di satu
ruangan, kunci yang sama persis di semua meja berbahaya: gateway kelompok
sebelah ikut mendengar JoinRequest node Anda, dan karena AppKey-nya cocok,
server mereka akan menjawabnya -- node Anda bisa join ke jaringan kelompok
lain tanpa satu pun pesan galat. Karena itu -DKELOMPOK=<n> di platformio.ini
menggeser dua byte:

DevEUI : 0011223344 55 KK NN      KK = 66 (klp 1), 77 (klp 2), 88 (klp 3)
NN = 01 (Node 1), 02 (Node 2)
AppKey : KK 112233445566778899AABBCCDDEE FN
KK = 00 (klp 1), 01 (klp 2), 02 (klp 3)
FN = F1 (Node 1), F2 (Node 2)

Kelompok 1 memakai nilai yang sama persis dengan contoh di README, sehingga
seluruh tabel dan log di modul ini tetap berlaku apa adanya.
*/
#define LORAWAN_KEYS_H
#define LORAWAN_KEYS_H

#include <Arduino.h>

#ifndef KELOMPOK
#define KELOMPOK 1
#endif

#if NODE_ID != 1 && NODE_ID != 2
#error "NODE_ID harus 1 atau 2 -- diatur lewat build_flags di platformio.ini"
#endif
#if KELOMPOK < 1 || KELOMPOK > 3
#error "KELOMPOK harus 1, 2, atau 3 -- satu kanal EU433 untuk tiap kelompok"
#endif

// Byte pembeda kelompok, dihitung compiler (bukan saat program berjalan).
#define EUI_BYTE_KELOMPOK (0x66 + (KELOMPOK - 1) * 0x11)
#define KEY_BYTE_KELOMPOK (KELOMPOK - 1)

// DevEUI di ChirpStack : 0011223344 55 <KK> <0N> -- ditulis LSB-first di sini

```

```
static const u1_t PROGMEM DEVEUI[8] = { NODE_ID, EUI_BYTE_KELompok,
                                         0x55, 0x44, 0x33, 0x22, 0x11, 0x00 };

// JoinEUI di ChirpStack: 0000000000000000
static const u1_t PROGMEM APPEUI[8] = { 0x00, 0x00, 0x00, 0x00,
                                         0x00, 0x00, 0x00, 0x00 };

// AppKey di ChirpStack : <KK> 112233445566778899AABBCCDDEE <FN> -- MSB, tidak dibalik
static const u1_t PROGMEM APPKEY[16] = { KEY_BYTE_KELompok, 0x11, 0x22, 0x33,
                                         0x44, 0x55, 0x66, 0x77,
                                         0x88, 0x99, 0xAA, 0xBB,
                                         0xCC, 0xDD, 0xEE, 0xFF + NODE_ID };

#endif // LORAWAN_KEYS_H
```

Kode 14.3: src/node/lorawan_keys.h — DevEUI, JoinEUI, dan AppKey per node

```
#!/usr/bin/env python3
"""Upload otomatis ke Modul 11 (LoRaWAN OTAA) — port serial dideteksi sendiri.

Alternatif untuk menjalankan manual:
    pio run -d Modul11_lorawan_chirpstack -e node1 -t upload -t monitor
    pio run -d Modul11_lorawan_chirpstack -e node2 -t upload -t monitor

Skrup ini memindai port /dev/ttyACM*/ttyUSB* yang sedang aktif, memetakannya ke
environment `node1` (port pertama) dan `node2` (port kedua), lalu menjalankan
`pio run` dengan --upload-port sesuai hasil deteksi -- platformio.ini tidak
perlu diedit tiap kali port berganti nama.

BERBEDA DARI MODUL SEBELUMNYA: modul ini hanya punya dua environment. Gateway
bukan Arduino melainkan Raspberry Pi yang menjalankan
gateway/single_chan_pkt_fwd.py, jadi tidak ada yang perlu di-upload untuknya.

python3 Modul11_lorawan_chirpstack/upload_auto.py
-> upload kedua node, port otomatis

python3 Modul11_lorawan_chirpstack/upload_auto.py --monitor
-> upload lalu buka pio device monitor satu per satu
    (Ctrl-C untuk lanjut ke board berikutnya)

python3 Modul11_lorawan_chirpstack/upload_auto.py --only node2
-> upload satu environment saja

python3 Modul11_lorawan_chirpstack/upload_auto.py --node1 /dev/ttyACM0 --node2 /dev/ttyACM1
-> timpa hasil deteksi otomatis secara manual

URUTAN YANG BENAR: nyalakan gateway di Raspberry Pi LEBIH DAHULU, baru unggah
node. JoinRequest yang dikirim saat gateway belum mendengar akan hilang tanpa
jejak, dan LMIC baru mencoba lagi setelah jeda yang makin panjang.
"""
import argparse
import glob
import subprocess
import sys
from pathlib import Path

PROJECT_DIR = Path(__file__).resolve().parent
ENV_URUTAN = ["node1", "node2"]

def deteksi_port_aktif():
    """Port serial USB yang sedang tersambung, terurut nama device.

    Pola yang sama dipakai tools/deteksi_port.py: Uno asli muncul sebagai
    /dev/ttyACM*, klon ber-bridge CH340/CP2102/FTDI sebagai /dev/ttyUSB*.
    """
    return sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*"))

def jalankan_pio(env, port, monitor):
    """Panggil `pio run` untuk satu environment dengan port hasil deteksi."""
    cmd = ["pio", "run", "-d", str(PROJECT_DIR), "-e", env, "--upload-port", port]
    if monitor:
        cmd += ["--monitor-port", port, "-t", "upload", "-t", "monitor"]
    else:
        cmd += ["-t", "upload"]
    print(f"\n=== {env.upper()} -> {port} ===")
    print("  $", " ".join(cmd))
```



```

return subprocess.call(cmd) == 0

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--node1", metavar="/dev/ttyXXX",
                    help="tampa port node1 (lewati deteksi otomatis)")
    ap.add_argument("--node2", metavar="/dev/ttyXXX",
                    help="tampa port node2 (lewati deteksi otomatis)")
    ap.add_argument("--only", choices=ENV_URUTAN,
                    help="upload satu environment saja (node1 atau node2)")
    ap.add_argument("--monitor", action="store_true",
                    help="buka pio device monitor per board sesudah upload")
    args = ap.parse_args()

    aktif = deteksi_port_aktif()
    manual = {"node1": args.node1, "node2": args.node2}

    port_env = {}
    sisa = list(aktif)
    for nama in ENV_URUTAN:
        if manual[nama]:
            port_env[nama] = manual[nama]
        elif sisa:
            port_env[nama] = sisa.pop(0)

    print(f"Port aktif terdeteksi: {'', '.join(aktif) if aktif else '(tidak ada)'}")
    for nama in ENV_URUTAN:
        if nama in port_env:
            sumber = "manual" if manual[nama] else "otomatis"
            print(f" -> {nama:<6} = {port_env[nama]} ({sumber})")
        else:
            print(f" -> {nama:<6} = TIDAK ADA PORT TERSISA "
                  f"- sambungkan board atau tentukan --{nama} /dev/ttyXXX")

    daftar = [args.only] if args.only else ENV_URUTAN
    hilang = [n for n in daftar if n not in port_env]
    if hilang:
        sys.exit(f"\nTidak bisa lanjut: port untuk {'', '.join(hilang)} tidak ditemukan.\n"
                 f"Sambungkan board-nya atau tentukan lewat --{hilang[0]} /dev/ttyXXX")

    for nama in daftar:
        if not jalankan_pio(nama, port_env[nama], monitor=args.monitor):
            sys.exit(f"\nGagal pada environment '{nama}' - proses dihentikan.")

    print("\nSelesai. Pantau hasilnya di ChirpStack "
          "(Applications > praktikum-wsn > Devices > <node> > Events;\n"
          "kelompok selain 1: praktikum-wsn-k<n>), atau di Raspberry Pi OS:\n"
          "python3 gateway/uplink_listen.py")

if __name__ == "__main__":
    main()

```

Kode 14.4: upload_auto.py — unggah kedua node dengan port terdeteksi sendiri

14.6.2 Sisi Gateway dan Server (Raspberry Pi)

```

# Modul 11 - dependensi Raspberry Pi (jalankan DI Raspberry Pi, bukan di PC)
#
# pip3 install -r Modul11_lorawan_chirpstack/gateway/requirements.txt
#
# Bila pip menolak karena "externally-managed-environment" (Debian 12+/Pi OS
# Bookworm ke atas), pasang lewat apt yang sudah disediakan Raspberry Pi OS:
# sudo apt install python3-spidev python3-rpi-lgpio python3-paho-mqtt
#
# spidev : akses /dev/spidev0.0 untuk bicara ke SX1276 (packet forwarder)
# rpi-lgpio : kendali NSS/RESET dan pembacaan DIO0 (packet forwarder)
# paho-mqtt : berlangganan uplink dari ChirpStack (uplink_listen.py)
#
# RASPBERRY PI 5. RPi.GPIO yang asli TIDAK bekerja di Pi 5 -- chip GPIO barunya
# (RPi) tidak dikenali. rpi-lgpio adalah pengganti drop-in dengan nama modul
# yang sama, sehingga `import RPi.GPIO as GPIO` tetap berjalan apa adanya.
# Jangan memasang keduanya sekaligus: nama pakatnya sama dan saling menimpa.
spidev>=3.5
rpi-lgpio>=0.6
paho-mqtt>=1.6

```

Kode 14.5: gateway/requirements.txt — paket Python sisi Raspberry Pi

```
#!/usr/bin/env python3
"""Modul 11 - Single Channel LoRaWAN Gateway untuk Raspberry Pi 5 (SX1276).

Menjembatani udara dan ChirpStack: paket LoRa yang tertangkap SX1276 dibungkus
protokol Semtech UDP (GWMP v2) dan dikirim ke chirpstack-gateway-bridge di
port 1700; balasan downlink dari server ditembakkan kembali ke udara tepat
pada jendela RX perangkat.

Driver radionya adalah lanjutan langsung dari Modul06_rpi_lora_python/src/
receiver.py -- register yang sama, gaya yang sama. Yang ditambahkan hanyalah
apa yang membuat sebuah paket LoRa menjadi paket LoRaWAN:

    sync word 0x34      penanda jaringan publik LoRaWAN (0x12 = privat, M01-M10)
    explicit header     panjang payload ikut dikirim, sebab paket LoRaWAN
                        panjangnya berubah-ubah
    IQ terbalik         hanya untuk downlink -- lihat _stage_tx()
    CRC                 aktif saat RX (uplink), mati saat TX (downlink)

SATU KANAL, SATU SF. Konsentrator gateway sungguhan (SX1301/SX1302) mendengar
8 kanal x seluruh SF sekaligus. SX1276 hanya punya satu modem: satu frekuensi,
satu spreading factor. Konsekuensinya harus disadari sejak awal -- lihat README
bagian 3: node WAJIB dikunci ke kanal dan DR yang sama dengan skrip ini.

    python3 single_chan_pkt_fwd.py          # server di Pi ini juga
    python3 single_chan_pkt_fwd.py --server 192.168.1.45
    python3 single_chan_pkt_fwd.py --verbose # cetak hex tiap paket

Pemetaan pin (Dragino LoRa GPS HAT - WiringPi -> BCM), sama seperti M06/M07:
    LoRa_NSS -> BCM 25    RESET -> BCM 17    DI00 -> BCM 4

Prasyarat:
    sudo raspi-config      # Interface Options > SPI > Yes, lalu reboot
    pip3 install -r requirements.txt
"""

import argparse
import base64
import json
import os
import random
import select
import signal
import socket
import struct
import sys
import time
from datetime import datetime, timezone

import spidev
import RPi.GPIO as GPIO

# -- Pin (BCM) -----
NSS_PIN = 25
RST_PIN = 17
DI00_PIN = 4

# -- Parameter radio: HARUS sama dengan node -----
FREQUENCY_HZ = 433175000 # kanal 0 rencana EU433
SPREADING_FACTOR = 7 # SF7BW125 = DR5
BANDWIDTH_HZ = 125000
CODING_RATE = 5 # 4/5
TX_POWER_DBM = 17
LORAWAN_SYNC_WORD = 0x34 # jaringan publik. 0x12 dipakai M01-M10 (privat)
PREAMBLE_SYMBOLS = 8 # LoRaWAN mewajibkan 8

# -- Semtech UDP protocol (GWMP) -----
PROTOCOL_VERSION = 2
PKT_PUSH_DATA = 0x00
PKT_PUSH_ACK = 0x01
PKT_PULL_DATA = 0x02
PKT_PULL_RESP = 0x03
PKT_PULL_ACK = 0x04
PKT_TX_ACK = 0x05

KEEPALIVE_S = 5 # PULL_DATA: menjaga "lubang" NAT tetap terbuka
STAT_INTERVAL_S = 30

# Downlink dijadwalkan dari user space, bukan dari pencacah keras konsentrator.
```

```

# STAGE_LEAD_US : berapa lama sebelum jadwal radio dilepas dari RX untuk
# diisi payload (selama itu gateway tuli -- sependek mungkin)
# TX_START_LEAD_US : kompensasi waktu tulis register OP_MODE + ramp-up PLL
STAGE_LEAD_US = 20000
TX_START_LEAD_US = 500

# -- Register SX1276 -----
REG_FIFO = 0x00
REG_OP_MODE = 0x01
REG_FRF_MSB = 0x06
REG_FRF_MID = 0x07
REG_FRF_LSB = 0x08
REG_PA_CONFIG = 0x09
REG_LNA = 0x0C
REG_FIFO_ADDR_PTR = 0x0D
REG_FIFO_TX_BASE_ADDR = 0x0E
REG_FIFO_RX_BASE_ADDR = 0x0F
REG_FIFO_RX_CURRENT_ADDR = 0x10
REG_IRQ_FLAGS_MASK = 0x11
REG_IRQ_FLAGS = 0x12
REG_RX_NB_BYTES = 0x13
REG_PKT_SNR_VALUE = 0x19
REG_PKT_RSSI_VALUE = 0x1A
REG_MODEM_CONFIG_1 = 0x1D
REG_MODEM_CONFIG_2 = 0x1E
REG_PREAMBLE_MSB = 0x20
REG_PREAMBLE_LSB = 0x21
REG_PAYLOAD_LENGTH = 0x22
REG_MAX_PAYLOAD_LENGTH = 0x23
REG_MODEM_CONFIG_3 = 0x26
REG_DETECTION_OPTIMIZE = 0x31
REG_INVERT_IQ = 0x33
REG_DETECTION_THRESHOLD = 0x37
REG_SYNC_WORD = 0x39
REG_INVERT_IQ2 = 0x3B
REG_DIO_MAPPING_1 = 0x40
REG_VERSION = 0x42
REG_PA_DAC = 0x4D

MODE_LONG_RANGE = 0x80
MODE_SLEEP = 0x00
MODE_STDBY = 0x01
MODE_TX = 0x03
MODE_RX_CONTINUOUS = 0x05

IRQ_RX_TIMEOUT = 0x80
IRQ_RX_DONE = 0x40
IRQ_CRC_ERROR = 0x20
IRQ_TX_DONE = 0x08

PA_BOOST = 0x80

# Nilai register IQ. Uplink memakai IQ normal, downlink memakai IQ terbalik --
# begitulah LoRaWAN mencegah node saling mendengar sesama node, dan gateway
# mendengar gema sesama gateway.
IQ_NORMAL = (0x27, 0x1D)
IQ_INVERTED = (0x66, 0x19)

_spi = spidev.SpiDev()

# Kanal RX gateway, diisi lora_begin(). Disimpan tersendiri karena downlink
# boleh memakai frekuensi dan SF yang BERBEDA (RX2 EU433 = 434.665 MHz SF12),
# dan sesudah TX radio harus dikembalikan ke kanal ini.
_rx_freq_hz = FREQUENCY_HZ
_rx_sf = SPREADING_FACTOR

# =====
# Lapisan SPI - identik dengan M06
# =====
def _read_reg(addr):
    GPIO.output(NSS_PIN, GPIO.LOW)
    val = _spi.xfer2([addr & 0x7F, 0x00])[1]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return val

def _write_reg(addr, value):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([(addr | 0x80) & 0xFF, value & 0xFF])
    GPIO.output(NSS_PIN, GPIO.HIGH)

```

```

def _read_fifo(length):
    """Baca FIFO sekali jalan (burst) -- jauh lebih cepat daripada per byte."""
    GPIO.output(NSS_PIN, GPIO.LOW)
    data = _spi.xfer2([REG_FIFO & 0x7F] + [0x00] * length)[1:]
    GPIO.output(NSS_PIN, GPIO.HIGH)
    return bytes(data)

def _write_fifo(data):
    GPIO.output(NSS_PIN, GPIO.LOW)
    _spi.xfer2([REG_FIFO | 0x80] + list(data))
    GPIO.output(NSS_PIN, GPIO.HIGH)

# =====
# Konfigurasi radio
# =====

def _set_mode(mode):
    _write_reg(REG_OP_MODE, MODE_LONG_RANGE | mode)

def _set_frequency(freq_hz):
    frf = int(round(freq_hz / 32e6 * (1 << 19)))
    _write_reg(REG_FRF_MSB, (frf >> 16) & 0xFF)
    _write_reg(REG_FRF_MID, (frf >> 8) & 0xFF)
    _write_reg(REG_FRF_LSB, frf & 0xFF)

def _set_spreading_factor(sf):
    sf = max(6, min(12, int(sf)))
    # SF11/SF12 pada BW125 wajib LowDataRateOptimize supaya simbol tidak melar
    ldro = 0x08 if sf >= 11 else 0x00
    _write_reg(REG_DETECTION_OPTIMIZE, 0xC5 if sf == 6 else 0xC3)
    _write_reg(REG_DETECTION_THRESHOLD, 0x0C if sf == 6 else 0x0A)
    _write_reg(REG_MODEM_CONFIG_3, ldro | 0x04) # 0x04 = AGC otomatis
    _write_reg(REG_MODEM_CONFIG_2,
        (_read_reg(REG_MODEM_CONFIG_2) & 0x0F) | ((sf << 4) & 0xF0))

def _set_bandwidth(bw_hz):
    table = [7.8e3, 10.4e3, 15.6e3, 20.8e3, 31.25e3, 41.7e3, 62.5e3,
        125e3, 250e3, 500e3]
    idx = min(range(len(table)), key=lambda i: abs(table[i] - bw_hz))
    _write_reg(REG_MODEM_CONFIG_1,
        (_read_reg(REG_MODEM_CONFIG_1) & 0x0F) | (idx << 4))

def _set_coding_rate(denom):
    cr = max(5, min(8, int(denom))) - 4
    _write_reg(REG_MODEM_CONFIG_1,
        (_read_reg(REG_MODEM_CONFIG_1) & 0xF1) | (cr << 1))

def _set_explicit_header():
    """Header eksplisit: panjang payload ikut dikirim di dalam header.

    M01-M10 tidak pernah mempersoalkan ini karena panjang paketnya seragam.
    Paket LoRaWAN panjangnya berubah-ubah (JoinRequest 23 byte, uplink data
    bisa belasan sampai ratusan byte), jadi penerima harus diberi tahu.
    """
    _write_reg(REG_MODEM_CONFIG_1, _read_reg(REG_MODEM_CONFIG_1) & 0xFE)

def _set_crc(enabled):
    cfg = _read_reg(REG_MODEM_CONFIG_2)
    _write_reg(REG_MODEM_CONFIG_2, (cfg | 0x04) if enabled else (cfg & ~0x04))

def _set_invert_iq(inverted):
    reg33, reg3b = IQ_INVERTED if inverted else IQ_NORMAL
    _write_reg(REG_INVERT_IQ, reg33)
    _write_reg(REG_INVERT_IQ2, reg3b)

def _set_tx_power(dbm):
    dbm = max(2, min(17, int(dbm)))
    _write_reg(REG_PA_DAC, 0x84) # mode normal (bukan +20 dBm)
    _write_reg(REG_PA_CONFIG, PA_BOOST | (dbm - 2)) # HAT hanya punya PA_BOOST

def lora_begin(freq_hz, sf, bw_hz, cr):

```

```

global _rx_freq_hz, _rx_sf
_rx_freq_hz, _rx_sf = freq_hz, sf

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(NSS_PIN, GPIO.OUT, initial=GPIO.HIGH)
GPIO.setup(RST_PIN, GPIO.OUT, initial=GPIO.HIGH)
GPIO.setup(DI00_PIN, GPIO.IN)

_spi.open(0, 0)
_spi.max_speed_hz = 5_000_000
_spi.mode = 0b00

GPIO.output(RST_PIN, GPIO.LOW)
time.sleep(0.01)
GPIO.output(RST_PIN, GPIO.HIGH)
time.sleep(0.01)

if _read_reg(REG_VERSION) != 0x12:
    return False

_set_mode(MODE_SLEEP)                # berpindah ke modem LoRa hanya boleh saat SLEEP
time.sleep(0.01)
_set_frequency(freq_hz)
_write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
_write_reg(REG_FIFO_RX_BASE_ADDR, 0x00)
_write_reg(REG_LNA, 0x23)            # LNA gain maksimum + boost
_write_reg(REG_MAX_PAYLOAD_LENGTH, 0x80)
_write_reg(REG_SYNC_WORD, LORAWAN_SYNC_WORD)
_write_reg(REG_PREAMBLE_MSB, 0x00)
_write_reg(REG_PREAMBLE_LSB, PREAMBLE_SYMBOLS)
_set_mode(MODE_STDBY)

_set_bandwidth(bw_hz)
_set_coding_rate(cr)
_set_spreading_factor(sf)
_set_explicit_header()
_set_tx_power(TX_POWER_DBM)
return True

def start_rx():
    """Kembali mendengarkan uplink: kanal gateway, IQ normal, CRC, RX continuous.

    Frekuensi dan SF WAJIB ikut dikembalikan, bukan hanya mode. Downlink RX2
    memakai 434.665 MHz SF12, dan bila hanya mode yang diubah, gateway tetap
    mendengarkan di kanal RX2 itu sesudahnya -- tuli terhadap seluruh uplink,
    tanpa satu pun pesan galat. Gejalanya: semua berjalan normal sampai
    downlink RX2 yang pertama, lalu sunyi.
    """
    _set_mode(MODE_STDBY)
    _set_frequency(_rx_freq_hz)
    _set_spreading_factor(_rx_sf)
    _set_bandwidth(BANDWIDTH_HZ)
    _set_coding_rate(CODING_RATE)
    _set_explicit_header()
    _set_crc(True)
    _set_invert_iq(False)
    _write_reg(REG_DIO_MAPPING_1, 0x00)    # DI00 = RxDone
    _write_reg(REG_IRQ_FLAGS, 0xFF)
    _write_reg(REG_FIFO_ADDR_PTR, 0x00)
    _set_mode(MODE_RX_CONTINUOUS)

def packet_rssi_snr():
    raw_snr = _read_reg(REG_PKT_SNR_VALUE)
    if raw_snr > 127:
        raw_snr -= 256
    snr = raw_snr / 4.0

    rssi = _read_reg(REG_PKT_RSSI_VALUE) - (164 if FREQUENCY_HZ < 868e6 else 157)
    if snr < 0:
        # Di bawah lantai derau, pembacaan register terlalu optimistis;
        # koreksi standar Semtech adalah menambahkan SNR-nya.
        rssi += snr
    return int(round(rssi)), snr

# =====
# Pembacaan amplop LoRaWAN (TANPA dekripsi)
# =====
MTYPE_NAMES = {

```

```

0: "JoinRequest", 1: "JoinAccept", 2: "UnconfirmedDataUp",
3: "UnconfirmedDataDown", 4: "ConfirmedDataUp", 5: "ConfirmedDataDown",
6: "RFU", 7: "Proprietary",
}

def describe_phy(payload):
    """Ringkas isi header PHY, untuk memperlihatkan apa yang gateway TAHU.

    Gateway LoRaWAN adalah tukang pos: ia membaca amplop (jenis pesan, DevAddr,
    frame counter) tetapi TIDAK memegang AppSKey, sehingga isi payload tetap
    terenkripsi baginya. Inilah bukti praktis pemisahan peran gateway dan
    network server -- lihat README bagian 3.
    """
    if not payload:
        return "kosong"
    mtype = payload[0] >> 5
    name = MTYPE_NAMES.get(mtype, "?")

    if mtype == 0 and len(payload) >= 23:
        # JoinRequest: JoinEUI(8, LSB) DevEUI(8, LSB) DevNonce(2) MIC(4)
        deveui = payload[9:17][::-1].hex().upper()
        devnonce = int.from_bytes(payload[17:19], "little")
        return f"{name} DevEUI={deveui} DevNonce={devnonce}"

    if mtype in (2, 4) and len(payload) >= 12:
        devaddr = payload[1:5][::-1].hex().upper()
        fctrl = payload[5]
        fcnt = int.from_bytes(payload[6:8], "little")
        idx = 8 + (fctrl & 0x0F) # lewati F0pts bila ada
        fport = payload[idx] if len(payload) > idx + 4 else None
        port_txt = f" FPort={fport}" if fport is not None else ""
        return f"{name} DevAddr={devaddr} FCnt={fcnt}{port_txt} (payload terenkripsi)"

    return f"{name} ({len(payload)} byte)"

# =====
# Utilitas
# =====
def micros():
    """Pencacah mikrodetik 32-bit, meniru pencacah keras konsentrator.

    Nilai inilah yang dikirim sebagai 'tmst' dan yang dipakai server untuk
    menentukan kapan downlink harus mengudara (tmst uplink + 1 detik untuk
    RX1 data, + 5 detik untuk JoinAccept).
    """
    return (time.monotonic_ns() // 1000) & 0xFFFFFFFF

def us_until(target):
    """Selisih ke target, memperhitungkan pencacah yang berputar di 2^32."""
    return ((target - micros() + 0x80000000) & 0xFFFFFFFF) - 0x80000000

def datr_to_sf(datr):
    """'SF7BW125' -> 7"""
    try:
        return int(str(datr).upper().split("SF")[1].split("BW")[0])
    except (IndexError, ValueError):
        return SPREADING_FACTOR

def default_gateway_id():
    """EUI-64 dari MAC antarmuka jaringan, cara baku gateway sungguhan.

    MAC 48-bit disisipi FF:FE di tengah: b8:27:eb:12:34:56 -> b827ebfffe123456
    """
    for iface in ("eth0", "wlan0", "end0"):
        path = f"/sys/class/net/{iface}/address"
        if os.path.exists(path):
            with open(path) as f:
                mac = f.read().strip().replace(":", "")
            if len(mac) == 12:
                return (mac[:6] + "fffe" + mac[6:]).lower()
    return "0102030405060708"

def iso_now():
    """Cap waktu untuk rxpk: ISO 8601 'expanded', presisi milidetik."""
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%S.%f")[:-3] + "Z"

```

```

def stat_time_now():
    """Cap waktu untuk pesan stat -- FORMATNYA BERBEDA dari rxpk, dan ini
    jebakan yang sunyi.

    chirpstack-gateway-bridge mengurai waktu di dalam stat dengan pola lama
    "2006-01-02 15:04:05 MST", bukan ISO 8601 seperti pada rxpk. Bila diberi
    ISO 8601, seluruh pesan stat ditolak dengan "could not handle packet" di
    log bridge -- sementara uplink tetap mengalir normal, sehingga satu-satunya
    gejala yang terlihat adalah gateway yang selamanya tampak OFFLINE di
    ChirpStack meski datanya jelas-jelas masuk.
    """
    return datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S GMT")

def log(pesan):
    """Cetak dengan cap waktu lokal, supaya baris gateway bisa dicocokkan
    dengan Serial Monitor kedua node saat menyusun laporan."""
    print(f"{datetime.now():%H:%M:%S.%f"}[:-3] + f" {pesan}", flush=True)

# =====
# Packet forwarder
# =====
class Forwarder:
    def __init__(self, args):
        self.args = args
        self.gw_id = bytes.fromhex(args.gateway_id)
        self.addr = (args.server, args.port)

        self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.sock.setblocking(False)

        self.last_pull = 0.0
        self.last_stat = 0.0
        self.pending_tx = None # downlink yang sedang menunggu jadwalnya

        self.rx_nb = self.rx_ok = self.rx_fw = self.dw_nb = self.tx_nb = 0

    # -- UDP -----
    def _send(self, kind, payload=b"", with_eui=True):
        token = struct.pack("<H", random.getrandbits(16))
        pkt = bytes([PROTOCOL_VERSION]) + token + bytes([kind])
        pkt += (self.gw_id if with_eui else b"") + payload
        try:
            self.sock.sendto(pkt, self.addr)
        except OSError as e:
            log(f"[UDP] gagal kirim: {e}")

    def push_data(self, obj):
        self._send(PKT_PUSH_DATA, json.dumps(obj).encode())

    def pull_data(self):
        self._send(PKT_PULL_DATA)

    def tx_ack(self, token, error="NONE"):
        body = json.dumps({"txpk_ack": {"error": error}}).encode()
        pkt = bytes([PROTOCOL_VERSION]) + token + bytes([PKT_TX_ACK]) + self.gw_id + body
        try:
            self.sock.sendto(pkt, self.addr)
        except OSError:
            pass

    def send_stat(self):
        self.push_data({"stat": {
            "time": stat_time_now(), "lati": 0.0, "long": 0.0, "alti": 0,
            "rxnb": self.rx_nb, "rxok": self.rx_ok, "rxfw": self.rx_fw,
            "ackr": 100.0, "dwnb": self.dw_nb, "txnb": self.tx_nb,
        }})

    # -- Uplink -----
    def handle_rx(self, tmst):
        irq = _read_reg(REG_IRQ_FLAGS)
        _write_reg(REG_IRQ_FLAGS, 0xFF)
        self.rx_nb += 1

        if irq & IRQ_CRC_ERROR:
            log("[RX] CRC salah - paket dibuang (bukan paket kita, atau tabrakan)")
            return

        length = _read_reg(REG_RX_NB_BYTES)
        _write_reg(REG_FIFO_ADDR_PTR, _read_reg(REG_FIFO_RX_CURRENT_ADDR))

```

```

payload = _read_fifo(length)
rssi, snr = packet_rssi_snr()
self.rx_ok += 1

log(f"[RX] {len(payload):3d} B RSSI={rssi:4d} dBm SNR={snr:5.1f} dB "
    f"tmst={tmst}\n          {describe_phy(payload)}")
if self.args.verbose:
    print(f"          hex: {payload.hex().upper()}")

self.push_data({"rxpk": [{
    "time": iso_now(),
    "tmst": tmst,
    "chan": 0,
    "rfch": 0,
    "freq": self.args.freq / 1e6,
    "stat": 1,
    "modu": "LORA",
    "datr": f"SF{self.args.sf}BW{int(BANDWIDTH_HZ / 1000)}",
    "codr": f"4/{CODING_RATE}",
    "rssi": rssi,
    "lsnr": round(snr, 1),
    "size": len(payload),
    "data": base64.b64encode(payload).decode(),
}]}))
self.rx_fw += 1

# -- Downlink -----
def handle_pull_resp(self, token, body):
    try:
        txpk = json.loads(body.decode())["txpk"]
    except (ValueError, KeyError, UnicodeDecodeError) as e:
        log(f"[TX] PULL_RESP tidak terbaca: {e}")
        self.tx_ack(token, "TX_FREQ")
        return

    data = base64.b64decode(txpk["data"])
    imme = txpk.get("imme", False)
    target = (micros() + 2000) & 0xFFFFFFFF if imme else int(txpk.get("tmst", micros())) & 0xFFFFFFFF

    if self.pending_tx is not None:
        log("[TX] masih ada downlink lain menunggu - yang ini ditolak")
        self.tx_ack(token, "COLLISION_PACKET")
        return

    delay_us = us_until(target)
    log(f"[TX] downlink dijadwalkan: {len(data)} B {txpk.get('freq')} MHz "
        f"{txpk.get('datr')} dalam {delay_us / 1000:.0f} ms\n"
        f"          {describe_phy(data)}")

    if delay_us < STAGE_LEAD_US:
        # Terlambat: jendela RX node keburu lewat sebelum radio siap.
        log("[TX] TERLAMBAT - jadwal sudah lewat, downlink dibuang")
        self.tx_ack(token, "T00_LATE")
        return

    self.pending_tx = {
        "data": data, "target": target, "token": token, "staged": False,
        "freq": int(float(txpk.get("freq", self.args.freq / 1e6)) * 1e6),
        "sf": datr_to_sf(txpk.get("datr", f"SF{self.args.sf}BW125")),
        "ncrc": txpk.get("ncrc", True),
    }

def _stage_tx(self, tx):
    """Siapkan radio + isi FIFO lebih awal, supaya saat jadwalnya tiba yang
    tersisa hanya satu penulisan register: OP_MODE = TX."""
    _set_mode(MODE_STDBY)
    _write_reg(REG_IRQ_FLAGS, 0xFF)
    _set_frequency(tx["freq"])
    _set_spreading_factor(tx["sf"])
    _set_bandwidth(BANDWIDTH_HZ)
    _set_coding_rate(CODING_RATE)
    _set_explicit_header()
    _set_crc(not tx["ncrc"]) # downlink LoRaWAN tidak memakai CRC PHY
    _set_invert_iq(True) # WAJIB: node hanya mendengar IQ terbalik
    _write_reg(REG_SYNC_WORD, LORAWAN_SYNC_WORD)
    _write_reg(REG_PREAMBLE_MSB, 0x00)
    _write_reg(REG_PREAMBLE_LSB, PREAMBLE_SYMBOLS)
    _set_tx_power(TX_POWER_DBM)
    _write_reg(REG_FIFO_TX_BASE_ADDR, 0x00)
    _write_reg(REG_FIFO_ADDR_PTR, 0x00)
    _write_fifo(tx["data"])
    _write_reg(REG_PAYLOAD_LENGTH, len(tx["data"]))

```



```

_write_reg(REG_DIO_MAPPING_1, 0x40) # DIO0 = TxDone
tx["staged"] = True

def _fire_tx(self, tx):
    _set_mode(MODE_TX)
    # Dicatat SEBELUM menunggu TxDone: sesudahnya, yang terukur bukan lagi
    # ketepatan jadwal melainkan lama paket mengudara.
    meleset_us = -us_until(tx["target"])
    t0 = time.monotonic()
    while not (_read_reg(REG_IRQ_FLAGS) & IRQ_TX_DONE):
        if time.monotonic() - t0 > 3:
            log("[TX] TxDone tidak pernah datang - radio dikembalikan ke RX")
            break
        time.sleep(0.0002)
    _write_reg(REG_IRQ_FLAGS, 0xFF)

    self.tx_nb += 1
    self.dw_nb += 1
    log(f"[TX] terkirim (meleset {meleset_us / 1000:+.1f} ms dari jadwal)")
    self.tx_ack(tx["token"])
    start_rx()

# -- Loop utama -----
def run(self):
    print(f"Gateway EUI : {self.gw_id.hex().upper()}")
    print(f"Server      : {self.args.server}:{self.args.port} (Semtech UDP)")
    print(f"Kanal          : {self.args.freq / 1e6:.3f} MHz SF{self.args.sf}BW125 CR4/{CODING_RATE}")
    print(f"Sync word      : 0x{LORAWAN_SYNC_WORD:02X} (LoRaWAN publik)")
    print(f"\nDaftarkan Gateway EUI di atas ke ChirpStack, lalu tunggu uplink.\n"
          "Ctrl-C untuk berhenti.\n")

    start_rx()
    self.pull_data()
    self.last_pull = self.last_stat = time.monotonic()

    while True:
        now = time.monotonic()

        # 1. Uplink dari udara. DIO0 naik saat RxDone; dibaca dengan polling
        #    cepat supaya cap waktunya sedekat mungkin dengan akhir paket.
        if self.pending_tx is None and GPIO.input(DIO0_PIN):
            self.handle_rx(micros())

        # 2. Downlink yang sudah dijadwalkan
        tx = self.pending_tx
        if tx is not None:
            sisa = us_until(tx["target"])
            if not tx["staged"] and sisa <= STAGE_LEAD_US:
                self._stage_tx(tx)
            elif tx["staged"] and sisa <= TX_START_LEAD_US:
                self._fire_tx(tx)
                self.pending_tx = None

        # 3. Balasan dari server
        r, _, _ = select.select([self.sock], [], [], 0)
        if r:
            try:
                pkt, _ = self.sock.recvfrom(2048)
            except OSError:
                pkt = b""
            if len(pkt) >= 4:
                kind = pkt[3]
                if kind == PKT_PULL_RESP:
                    self.handle_pull_resp(pkt[1:3], pkt[4:])
                elif kind in (PKT_PUSH_ACK, PKT_PULL_ACK) and self.args.verbose:
                    nama = "PUSH_ACK" if kind == PKT_PUSH_ACK else "PULL_ACK"
                    log(f"[UDP] {nama}")

        # 4. Pemeliharaan
        if now - self.last_pull >= KEEPALIVE_S:
            self.pull_data()
            self.last_pull = now
        if now - self.last_stat >= STAT_INTERVAL_S:
            self.send_stat()
            self.last_stat = now

        time.sleep(0.0002)

def main():
    global FREQUENCY_HZ

```

```

ap = argparse.ArgumentParser(description=__doc__,
                             formatter_class=argparse.RawDescriptionHelpFormatter)
ap.add_argument("--server", default="127.0.0.1",
                help="alamat chirstack-gateway-bridge (bawaan: 127.0.0.1)")
ap.add_argument("--port", type=int, default=1700, help="port UDP (bawaan: 1700)")
ap.add_argument("--freq", type=float, default=FREQUENCY_HZ,
                help="frekuensi kanal dalam Hz (bawaan: 433175000)")
ap.add_argument("--sf", type=int, default=SPREADING_FACTOR,
                help="spreading factor (bawaan: 7)")
ap.add_argument("--gateway-id", default=None,
                help="EUI-64 gateway, 16 digit hex (bawaan: diturunkan dari MAC)")
ap.add_argument("--verbose", action="store_true", help="cetak hex tiap paket")
args = ap.parse_args()

if args.gateway_id is None:
    args.gateway_id = default_gateway_id()
args.gateway_id = args.gateway_id.replace(":", "").replace("-", "").lower()
if len(args.gateway_id) != 16:
    sys.exit("Gateway ID harus 16 digit hex (EUI-64).")

FREQUENCY_HZ = args.freq

print("=== SINGLE CHANNEL LoRaWAN GATEWAY ===")
print("Init SX1276 ... ", end="", flush=True)
if not lora_begin(args.freq, args.sf, BANDWIDTH_HZ, CODING_RATE):
    print("GAGAL")
    print("REG_VERSION bukan 0x12. Periksa HAT terpasang rapat dan SPI aktif:")
    print("  sudo raspi-config > Interface Options > SPI > Yes")
    sys.exit(1)
print("OK")

# systemd menghentikan layanan dengan SIGTERM, bukan Ctrl-C. Tanpa baris
# ini Python langsung keluar dan blok finally di bawah tidak pernah jalan --
# radio ditinggal dalam mode RX dan pin GPIO tidak dilepas. Diubah menjadi
# KeyboardInterrupt supaya keduanya berakhir lewat jalan yang sama.
signal.signal(signal.SIGTERM, lambda *_: (_ for _ in ()).throw(KeyboardInterrupt))

fwd = Forwarder(args)
try:
    fwd.run()
except KeyboardInterrupt:
    print("\n\nDihentikan.")
finally:
    _set_mode(MODE_SLEEP)
    _spi.close()
    GPIO.cleanup()
    print("Radio dimatikan.")

if __name__ == "__main__":
    main()

```

Kode 14.6: gateway/single_chan_pkt_fwd.py — gateway kanal tunggal, lanjutan M06

```

#!/usr/bin/env python3
"""Modul 11 - dengarkan uplink dari ChirpStack lewat MQTT, tanpa membuka web UI.

Web UI ChirpStack sudah menampilkan semua ini, tetapi hanya satu perangkat per
layar. Skrip ini berlangganan seluruh aplikasi sekaligus, jadi uplink Node 1
dan Node 2 terlihat berselang-seling di satu terminal -- sekaligus memperlihatkan
bahwa integrasi MQTT-lah pintu keluar data LoRaWAN menuju aplikasi sungguhan.
Setiap baris juga dicatat ke CSV, seperti modul-modul sebelumnya.

python3 uplink_listen.py # broker di Pi ini juga
python3 uplink_listen.py --host 192.168.1.45
python3 uplink_listen.py --csv sesi_praktikum.csv

Topik yang dilanggan (ChirpStack v4):
application/+device/+event/up

Prasyarat: pip3 install paho-mqtt
"""

import argparse
import base64
import csv
import json
import re

```

```

import sys
from datetime import datetime

try:
    import paho.mqtt.client as mqtt
except ImportError:
    sys.exit("paho-mqtt belum terpasang: pip3 install paho-mqtt")

# "T=27.4,H=68" -- format payload Modul 11, sengaja ASCII polos
POLA = re.compile(r"T=(-?\d+(?:\.\d+)?),H=(\d+(?:\.\d+)?)")

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--host", default="127.0.0.1", help="broker MQTT (bawaan: 127.0.0.1)")
    ap.add_argument("--port", type=int, default=1883)
    ap.add_argument("--csv", default=None,
                    help="berkas CSV keluaran (bawaan: lorawan_<tanggal>.csv)")
    args = ap.parse_args()

    nama_csv = args.csv or f"lorawan_{datetime.now():%Y%m%d_%H%M%S}.csv"
    berkas = open(nama_csv, "w", newline="")
    tulis = csv.writer(berkas)
    tulis.writerow(["waktu", "device", "dev_eui", "fcnt", "payload",
                   "suhu_c", "kelembapan_pct", "rssi_dbm", "snr_db"])

    print(f"Broker : {args.host}:{args.port}")
    print(f"Topik : application+/device+/event/up")
    print(f"CSV : {nama_csv}")
    print(f"Ctrl-C untuk berhenti.\n")
    print(f"{'waktu':8} {'device':10} {'FCnt':>5} {'payload':14} "
          f"{'suhu':>6} {'RH':>5} {'RSSI':>6} {'SNR':>6}")
    print("-" * 72)

    def on_connect(client, userdata, flags, rc, properties=None):
        if rc != 0:
            print(f"Gagal terhubung ke broker (rc={rc})")
            return
        client.subscribe("application+/device+/event/up")

    def on_message(client, userdata, msg):
        try:
            ev = json.loads(msg.payload.decode())
        except ValueError:
            return

        info = ev.get("deviceInfo", {})
        nama = info.get("deviceName", "?")
        eui = info.get("devEui", "?")
        fcnt = ev.get("fCnt", 0)
        raw = base64.b64decode(ev.get("data", "")) if ev.get("data") else b""
        teks = raw.decode("ascii", errors="replace")

        rx = (ev.get("rxInfo") or [{}])[0]
        rssi = rx.get("rssi", 0)
        snr = rx.get("snr", 0.0)

        cocok = POLA.search(teks)
        suhu = cocok.group(1) if cocok else ""
        rh = cocok.group(2) if cocok else ""

        # Frame tanpa payload aplikasi (jawaban perintah MAC dari server) tetap
        # ditampilkan -- justru berguna dilihat -- tetapi tidak dipaksa masuk
        # kolom suhu/kelembapan.
        tampil = teks if teks else "(MAC saja)"
        suhu_txt = f"{suhu:>5}C" if suhu else "    -"
        rh_txt = f"{rh:>4}%" if rh else "    -"

        jam = datetime.now().strftime("%H:%M:%S")
        print(f"{jam} {nama:10} {fcnt:5d} {tampil:14} "
              f"{'suhu_txt':14} {'rh_txt':14} {'rssi:5} dBm {snr:5.1f} dB")

        tulis.writerow([datetime.now().isoformat(timespec="seconds"), nama, eui,
                        fcnt, teks, suhu, rh, rssi, snr])
        berkas.flush()

    # paho-mqtt 2.x mewajibkan versi API callback disebutkan; 1.x belum mengenalnya.
    try:
        client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    except AttributeError:
        client = mqtt.Client()

```

```

client.on_connect = on_connect
client.on_message = on_message

try:
    client.connect(args.host, args.port, 60)
except OSError as e:
    sys.exit(f"Tidak bisa menghubungi broker {args.host}:{args.port} -- {e}\n"
            f"Pastikan ChirpStack sudah berjalan (docker compose ps).")

try:
    client.loop_forever()
except KeyboardInterrupt:
    print(f"\n\nDihentikan. Log tersimpan di {nama_csv}")
finally:
    berkas.close()

if __name__ == "__main__":
    main()

```

Kode 14.7: gateway/uplink_listen.py — baca uplink terdekripsi lewat MQTT ke CSV

```

#!/usr/bin/env python3
"""Modul 11 - daftarkan gateway, device profile, aplikasi, dan kedua node ke
ChirpStack lewat REST API. Aman dijalankan berulang (yang sudah ada dilewati).

PRAKTIKAN TIDAK MEMAKAI SKRIP INI. Pendaftaran perangkat adalah inti EXP-01 dan
dikerjakan sendiri lewat web UI -- di situlah terlihat bahwa DevEUI/AppKey di
server harus cocok dengan yang tertanam di firmware. Skrip ini untuk asisten:
menyiapkan ulang lab dengan cepat, atau memastikan isi server memang sama
persis dengan yang tertulis di README ketika ada yang tidak jalan.

# buat token dulu (sekali saja):
cd ~/chirpstack-docker
sudo docker compose exec chirpstack chirpstack -c /etc/chirpstack \
    create-api-key --name praktikum

python3 provision_lab.py --token eyJ0eXAi... --gateway-id 2CCF67FFFE53AC11
python3 provision_lab.py --token ... --gateway-id ... --kelompok 2

DevEUI dan AppKey dihitung dari nomor kelompok dengan aturan yang sama persis
seperti src/node/lorawan_keys.h -- ubah salah satunya saja dan join akan gagal
dengan gejala paling menyedihkan: paket sampai, tetapi perangkat tidak pernah
ter-aktivasi.

Kelompok 1 memakai nama aplikasi dan perangkat tanpa akhiran, sama seperti
contoh di README; kelompok 2 dan 3 diberi akhiran -k2 / -k3 supaya ketiganya
bisa hidup berdampingan di satu server bila network server dipakai bersama.
"""

import argparse
import json
import sys
import urllib.error
import urllib.request

# Aturan penomorannya sama dengan src/node/lorawan_keys.h:
# DevEUI : 0011223344 55 KK NN   KK = 66/77/88 untuk kelompok 1/2/3
# AppKey : KK 112233445566778899AABBCCDDEE FN   KK = 00/01/02, FN = F1/F2
RUANGAN = {
    1: ("Ruangan 1", "suhu 25-30 C, RH 60-75 %"),
    2: ("Ruangan 2", "suhu 28-35 C, RH 40-65 %"),
}
KANAL_HZ = {1: 433175000, 2: 433375000, 3: 433575000}
KANAL_KELOMPOK = {k: f"{hz / 1e6:.3f} MHz" for k, hz in KANAL_HZ.items()}

def daftar_node(kelompok):
    """Susun DevEUI, AppKey, dan nama untuk kedua node satu kelompok."""
    awalan = "" if kelompok == 1 else f"k{kelompok}-"
    hasil = []
    for node in (1, 2):
        ruang, rentang = RUANGAN[node]
        hasil.append({
            "name": f"{awalan}node{node}-{ruang.lower().replace(' ', '-')}",
            "dev_eui": f"001122334455{0x66 + (kelompok - 1) * 0x11:02x}{node:02x}",
            "app_key": f"{kelompok - 1:02x}112233445566778899aabbccddeef{node}",

```

```

        "desc": f"Arduino Uno + SX1276 - {ruang} ({rentang}), "
                f"kelompok {kelompok}, {KANAL_KELOMPOK[kelompok]}",
    })
    return hasil

# Codec opsional: mengubah "T=27.4,H=68" menjadi objek JSON, supaya kolom
# "Object" di ChirpStack menampilkan angka yang sudah terbaca. Payload di
# udara tetap ASCII polos -- codec ini murni pekerjaan server.
CODEC_JS = """function decodeUplink(input) {
    var s = String.fromCharCode.apply(null, input.bytes);
    var m = s.match(/T=(-?\d+(?:\.\d+)?),H=(\d+(?:\.\d+)?)/);
    if (!m) { return { data: { raw: s } }; }
    return { data: { temperature: parseFloat(m[1]), humidity: parseFloat(m[2]), raw: s } };
}

function encodeDownlink(input) {
    return { bytes: [] };
}
"""

class API:
    def __init__(self, base, token):
        self.base = base.rstrip("/")
        self.token = token

    def call(self, method, path, body=None):
        data = json.dumps(body).encode() if body is not None else None
        req = urllib.request.Request(self.base + path, data=data, method=method)
        req.add_header("Content-Type", "application/json")
        req.add_header("Grpc-Metadata-Authorization", f"Bearer {self.token}")
        try:
            with urllib.request.urlopen(req, timeout=15) as r:
                raw = r.read().decode()
                return json.loads(raw) if raw.strip() else {}
        except urllib.error.HTTPError as e:
            detail = e.read().decode()
            # ChirpStack menolak objek kembar dengan beberapa cara berbeda:
            # 409, pesan "already exists", atau -- untuk gateway dengan EUI yang
            # sudah terdaftar -- 500 berisi pelanggaran unique constraint dari
            # PostgreSQL. Ketiganya berarti hal yang sama bagi skrip ini:
            # objeknya sudah ada, lanjut saja.
            if (e.code == 409 or "already exists" in detail
                or "duplicate key" in detail):
                print(f"    (sudah ada, dilewati: {method} {path.split('?')[0]})")
                return None
            raise SystemExit(f"{method} {path} gagal ({e.code}): {detail}")
        except urllib.error.URLError as e:
            raise SystemExit(f"Tidak bisa menghubungi {self.base}: {e.reason}")

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--token", required=True, help="API key ChirpStack")
    ap.add_argument("--gateway-id", required=True,
                    help="EUI gateway, seperti yang dicetak single_chan_pkt_fwd.py")
    ap.add_argument("--api", default="http://127.0.0.1:8090",
                    help="alamat chirpstack-rest-api (bawaan: http://127.0.0.1:8090)")
    ap.add_argument("--kelompok", type=int, default=1, choices=(1, 2, 3),
                    help="nomor kelompok: menentukan DevEUI, AppKey, dan kanal "
                    "(bawaan: 1). HARUS sama dengan -DKELOMPOK di platformio.ini")
    ap.add_argument("--app-name", default=None,
                    help="nama aplikasi (bawaan: praktikum-wsn untuk kelompok 1, "
                    "praktikum-wsn-k<n> untuk kelompok lain)")
    ap.add_argument("--profile-name", default="uno-sx1276-eu433-otaa")
    args = ap.parse_args()

    if args.app_name is None:
        args.app_name = ("praktikum-wsn" if args.kelompok == 1
                        else f"praktikum-wsn-k{args.kelompok}")
    nodes = daftar_node(args.kelompok)

    api = API(args.api, args.token)
    print(f"Kelompok      : {args.kelompok} (kanal {KANAL_KELOMPOK[args.kelompok]})")

    tenants = api.call("GET", "/api/tenants?limit=10")
    if not tenants or not tenants.get("result"):
        raise SystemExit("Tidak ada tenant di ChirpStack -- server baru dipasang?")
    tenant_id = tenants["result"][0]["id"]
    print(f"Tenant              : {tenants['result'][0]['name']} ({tenant_id})")

```

```

gw_id = args.gateway_id.lower()
api.call("POST", "/api/gateways", {"gateway": {
    "gatewayId": gw_id,
    "name": f"single-chan-pi5-k{args.kelompok}",
    "description": "Raspberry Pi 5 + Dragino LoRa GPS HAT (SX1276), "
        f"{KANAL_KELOMPOK[args.kelompok]} SF7 - kelompok {args.kelompok}",
    "tenantId": tenant_id,
    "statsInterval": 30,
    "location": {"latitude": 0, "longitude": 0, "altitude": 0},
}})
print(f"Gateway          : {gw_id}")

# Device profile: EU433, LoRaWAN 1.0.3, OTAA, ADR dimatikan (kanal tunggal)
profiles = api.call("GET", f"/api/device-profiles?limit=100&tenantId={tenant_id}")
prof_id = next((p["id"] for p in profiles.get("result", []))
               if p["name"] == args.profile_name, None)
if prof_id is None:
    res = api.call("POST", "/api/device-profiles", {"deviceProfile": {
        "tenantId": tenant_id,
        "name": args.profile_name,
        "description": "Profil Modul 11 - kanal tunggal EU433 SF7BW125",
        "region": "EU433",
        "macVersion": "LORAWAN_1_0_3",
        "regParamsRevision": "A",
        "adrAlgorithmId": "default",
        "supportsOtaa": True,
        "uplinkInterval": 60,
        "deviceStatusReqInterval": 1,
        "flushQueueOnActivate": True,
        "payloadCodecRuntime": "JS",
        "payloadCodecScript": CODEC_JS,
    }})
    prof_id = res["id"]
print(f"Device profile   : {args.profile_name} ({prof_id})")

apps = api.call("GET", f"/api/applications?limit=100&tenantId={tenant_id}")
app_id = next((a["id"] for a in apps.get("result", []))
               if a["name"] == args.app_name, None)
if app_id is None:
    res = api.call("POST", "/api/applications", {"application": {
        "tenantId": tenant_id,
        "name": args.app_name,
        "description": "Praktikum WSN/IoT - LoRaWAN Modul 11",
    }})
    app_id = res["id"]
print(f"Application      : {args.app_name} ({app_id})")

for n in nodes:
    api.call("POST", "/api/devices", {"device": {
        "devEui": n["dev_eui"],
        "name": n["name"],
        "description": n["desc"],
        "applicationId": app_id,
        "deviceProfileId": prof_id,
        "isDisabled": False,
        "skipFcmtCheck": False,
    }})
    # LoRaWAN 1.0.x: AppKey disimpan ChirpStack di field nwkKey.
    api.call("POST", f"/api/devices/{n['dev_eui']}/keys", {"deviceKeys": {
        "devEui": n["dev_eui"],
        "nwkKey": n["app_key"],
    }})
    print(f"Device          : {n['name']} DevEUI={n['dev_eui']}")

print("\nSelesai. Jalankan gateway pada kanal kelompok ini:")
print(f"  python3 single_chan_pkt_fwd.py --freq {KANAL_HZ[args.kelompok]}")
print("  lalu pantau di web UI:")
print("  Applications > " + args.app_name + " > Devices > <node> > Events")

if __name__ == "__main__":
    main()

```

Kode 14.8: gateway/provision_lab.py — pendaftaran ulang isi server lewat REST API

```

#!/usr/bin/env bash
# Modul 11 - pasang ChirpStack v4 di Raspberry Pi 5 (sekali jalan, aman diulang).

```

```
#
#   bash Modul11_lorawan_chirpstack/gateway/setup_chirpstack.sh
#
# Yang dilakukan:
#   1. memasang Docker bila belum ada (skrip resmi get.docker.com)
#   2. mengambil chirpstack-docker resmi ke ~/chirpstack-docker
#   3. menyalin docker-compose.override.yml (EU433) ke dalamnya
#   4. menyalakan seluruh service dan menunggu API siap
#
# Sesudah selesai, buka http://<ip-pi>:8080 -- login awal admin / admin.
set -euo pipefail

TARGET="${HOME}/chirpstack-docker"
SRC_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"

if ! command -v docker >/dev/null 2>&1; then
    echo "==> Docker belum ada, memasang ..."
    curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
    sudo sh /tmp/get-docker.sh
    sudo usermod -aG docker "$USER"
    echo "    (log out/in sekali supaya bisa memakai docker tanpa sudo)"
fi

DOCKER="docker"
docker info >/dev/null 2>&1 || DOCKER="sudo docker"

if [ ! -d "$TARGET" ]; then
    echo "==> Mengambil chirpstack-docker ..."
    git clone https://github.com/chirpstack/chirpstack-docker.git "$TARGET"
fi

echo "==> Memasang override EU433 ..."
cp "$SRC_DIR/docker-compose.override.yml" "$TARGET/docker-compose.override.yml"

echo "==> Menyalakan ChirpStack ..."
cd "$TARGET"
$DOCKER compose up -d

echo -n "==> Menunggu API siap "
for _ in $(seq 1 60); do
    if curl -sf -o /dev/null "http://127.0.0.1:8080"; then
        echo " siap."
        break
    fi
    echo -n "."
    sleep 2
done

IP=$(hostname -I | awk '{print $1}')
cat <<TXT

ChirpStack berjalan.

    Web UI   : http://${IP}:8080      (admin / admin)
    UDP      : ${IP}:1700            (untuk single_chan_pkt_fwd.py)
    MQTT     : ${IP}:1883            (untuk uplink_listen.py)

Langkah berikutnya ada di README bagian 6 (EXP-01):
    1. daftarkan gateway memakai EUI yang dicetak single_chan_pkt_fwd.py
    2. buat Device Profile region EU433, LoRaWAN 1.0.3, OTAA
    3. daftarkan Node 1 dan Node 2 beserta DevEUI + AppKey-nya

Belum pernah memakai Docker? Lampiran A di README modul ini berisi lima
perintah yang dipakai, cara membacanya, dan apa yang harus dilakukan bila
ada yang tidak beres.
TXT
```

Kode 14.9: gateway/setup_chirpstack.sh — pasang Docker dan ChirpStack v4

```
#!/usr/bin/env bash
# Modul 11 - siapkan satu Raspberry Pi menjadi gateway kelompok (sekali jalan).
#
#   bash Modul11_lorawan_chirpstack/gateway/siapkan_gateway.sh --kelompok 2 \
#   --server 192.168.1.45
#
# Dipakai saat beberapa kelompok bekerja bersamaan: tiap Pi menjadi gateway
# kelompoknya sendiri, sementara ChirpStack cukup SATU untuk sekelas -- persis
# seperti LoRaWAN sungguhan, yang network server-nya memang milik bersama.
```

```

#
# --kelompok N 1, 2, atau 3. Menentukan kanal EU433 yang didengar gateway
#                (433.175 / 433.375 / 433.575 MHz) dan harus SAMA dengan
#                -DKELOMPOK di platformio.ini kedua node.
# --server IP    alamat ChirpStack. Kosongkan bila server dipasang di Pi ini
#                juga -- pakai --dengan-server.
# --dengan-server pasang ChirpStack di Pi ini (memanggil setup_chirpstack.sh).
# --token TOKEN  bila diisi, sekaligus daftarkan gateway + kedua node ke server
#                lewat REST API. Tanpa ini, pendaftaran dikerjakan praktikan
#                lewat web UI (itu memang inti EXP-01).
# --service      pasang layanan systemd, sehingga gateway ikut hidup sendiri
#                setiap Pi dinyalakan. Untuk demo yang ditinggal jalan; saat
#                praktikum justru lebih baik dijalankan manual dari terminal
#                supaya baris [RX]/[TX] terlihat langsung.
#
# Yang dikerjakan: pasang dependensi Python, nyalakan SPI bila belum, hitung
# Gateway EUI, lalu cetak perintah persis untuk menjalankan gateway kelompok itu.
set -euo pipefail

SRC_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
KELOMPOK=1
SERVER=""
DENGAN_SERVER=0
TOKEN=""
SERVICE=0

while [ $# -gt 0 ]; do
    case "$1" in
        --kelompok) KELOMPOK="$2"; shift 2 ;;
        --server) SERVER="$2"; shift 2 ;;
        --dengan-server) DENGAN_SERVER=1; shift ;;
        --token) TOKEN="$2"; shift 2 ;;
        --service) SERVICE=1; shift ;;
        -h|--help) sed -n '2,/^\$/p' "${BASH_SOURCE[0]}"; exit 0 ;;
        *) echo "Ops! tidak dikenal: $1"; exit 1 ;;
    esac
done

case "$KELOMPOK" in
    1) FREQ=433175000 ;;
    2) FREQ=433375000 ;;
    3) FREQ=433575000 ;;
    *) echo "--kelompok harus 1, 2, atau 3 (EU433 hanya punya tiga kanal wajib).";
       echo "Kelompok keempat boleh berbagi kanal dengan kelompok 1, asalkan";
       echo "DevEUI dan AppKey-nya berbeda -- lihat Lampiran B di README.";
       exit 1 ;;
esac

echo "=== Menyiapkan gateway kelompok ${KELOMPOK} (kanal ${FREQ} Hz) ==="

# — 1. Dependensi —
# Dipasang lewat apt, bukan pip: Raspberry Pi OS Bookworm ke atas menolak pip
# di luar virtualenv (externally-managed-environment).
echo "==> Memasang dependensi Python ..."
sudo apt-get update -qq
sudo apt-get install -y -qq python3-spidev python3-rpi-lgpio python3-paho-mqtt

# — 2. SPI —
if [ ! -e /dev/spidev0.0 ]; then
    echo "==> SPI belum aktif, menyalakan ..."
    sudo raspi-config nonint do_spi 0
    echo "    SPI baru berlaku setelah REBOOT. Jalankan ulang skrip ini sesudahnya."
    exit 0
fi
echo "==> SPI aktif (/dev/spidev0.0)"

# — 3. Server (opsional) —
if [ "$DENGAN_SERVER" = "1" ]; then
    bash "$SRC_DIR/setup_chirpstack.sh"
    SERVER="${SERVER:-127.0.0.1}"
fi
SERVER="${SERVER:-127.0.0.1}"

# — 4. Gateway EUI —
# Diambil dari fungsi yang sama yang dipakai gateway itu sendiri, supaya tidak
# mungkin berbeda dengan yang nanti tercetak saat dijalankan.
EUI=$(cd "$SRC_DIR" && python3 -c \
    'import single_chan_pkt_fwd as g; print(g.default_gateway_id().upper())')

# — 5. Pendaftaran otomatis (opsional) —
if [ -n "$TOKEN" ]; then
    echo "==> Mendaftarkan gateway + kedua node ke ${SERVER} ..."

```



```

python3 "$SRC_DIR/provision_lab.py" --token "$TOKEN" --gateway-id "$EUI" \
--kelompok "$KELOMPOK" --api "http://${SERVER}:8090"
fi

# — 6. Layanan systemd (opsional) —————
if [ "$SERVICE" = "1" ]; then
    echo "=> Memasang layanan systemd lorawan-gateway ..."

    # Frekuensi dan server ditaruh terpisah dari berkas unit, supaya ganti
    # kanal cukup mengubah satu baris di sini lalu restart.
    sudo tee /etc/default/lorawan-gateway >/dev/null <<TXT
# Modul 11 - konfigurasi layanan lorawan-gateway (kelompok ${KELOMPOK}).
# Sesudah mengubah berkas ini: sudo systemctl restart lorawan-gateway
SERVER=${SERVER}
FREQ=${FREQ}
OPSI=
TXT

    sed -e "s|__USER__|${USER}|g" -e "s|__DIR__|${SRC_DIR}|g" \
        "$SRC_DIR/lorawan-gateway.service" \
        | sudo tee /etc/systemd/system/lorawan-gateway.service >/dev/null

    # Skrip yang barangkali sedang jalan manual harus dihentikan dulu: dua
    # proses yang memperebutkan SPI yang sama akan saling mengacaukan radio.
    pkill -f single_chan_pkt_fwd || true
    sleep 1

    sudo systemctl daemon-reload
    sudo systemctl enable --now lorawan-gateway
    sleep 3
    systemctl --no-pager --lines=0 status lorawan-gateway || true
    echo "    Lihat keluarannya: journalctl -fu lorawan-gateway"
fi

# — 7. Ringkasan —————
CATATAN_SERVICE=""
if [ "$SERVICE" = "1" ]; then
    CATATAN_SERVICE=" (layanan systemd sudah berjalan -- hentikan dulu dengan
'sudo systemctl stop lorawan-gateway' sebelum menjalankan manual, sebab
dua proses tidak boleh berebut SPI yang sama)
"
fi
DEV_KK=$(printf '%02x' $((0x66 + (KELOMPOK - 1) * 0x11)))
KEY_KK=$(printf '%02x' $((KELOMPOK - 1)))
cat <<TXT

-----
KELOMPOK ${KELOMPOK} — siap dijalankan
-----

Gateway EUI : ${EUI}
Kanal       : ${FREQ} Hz  SF7BW125
Server      : ${SERVER}

Jalankan gateway:
cd ${SRC_DIR}
python3 single_chan_pkt_fwd.py --server ${SERVER} --freq ${FREQ}
${CATATAN_SERVICE}
Pantau data yang sudah didekripsi server:
python3 uplink_listen.py --host ${SERVER}

Di sisi node — platformio.ini, ubah SATU baris:
-D KELOMPOK=${KELOMPOK}

Nilai yang harus cocok di ChirpStack (EXP-01):
DevEUI Node 1 : 001122334455${DEV_KK}01
DevEUI Node 2 : 001122334455${DEV_KK}02
AppKey Node 1 : ${KEY_KK}112233445566778899aabbccddeef1
AppKey Node 2 : ${KEY_KK}112233445566778899aabbccddeef2
-----
TXT

```

Kode 14.10: gateway/siapkan_gateway.sh — siapkan Pi satu kelompok

```

#!/usr/bin/env bash
# Modul 11 - bungkus image Docker ChirpStack ke satu berkas, untuk dipindahkan
# lewat flashdisk ke Raspberry Pi lain tanpa mengunduh ulang.
#
# Berguna bila internet lab lemah dan ada beberapa Pi yang masing-masing perlu

```

```

# menjalankan ChirpStack sendiri. Yang dihemat adalah unduhan image (ratusan MB);
# Docker engine-nya sendiri tetap harus terpasang di Pi tujuan.
#
# # di Pi yang ChirpStack-nya sudah jalan:
# bash simpan_image_docker.sh
# bash simpan_image_docker.sh --keluar /media/pi/FLASHDISK/chirpstack.tar.gz
#
# # di Pi tujuan (Docker sudah terpasang):
# bash simpan_image_docker.sh --muat /media/pi/FLASHDISK/chirpstack.tar.gz
# cd ~/chirpstack-docker && docker compose up -d
#
# Daftar image diambil langsung dari docker-compose.yml, jadi tidak mungkin
# ketinggalan bila suatu saat ChirpStack menambah komponen.
set -euo pipefail

COMPOSE_DIR="${HOME}/chirpstack-docker"
KELUAR="${HOME}/chirpstack-images.tar.gz"
MUAT=""

while [ $# -gt 0 ]; do
    case "$1" in
        --keluar) KELUAR="$2"; shift 2 ;;
        --muat) MUAT="$2"; shift 2 ;;
        --compose-dir) COMPOSE_DIR="$2"; shift 2 ;;
        -h|--help) sed -n '2,/~/p' "${BASH_SOURCE[0]}"; exit 0 ;;
        *) echo "Opsi tidak dikenal: $1"; exit 1 ;;
    esac
done

DOCKER="docker"
docker info >/dev/null 2>&1 || DOCKER="sudo docker"

# pigz memakai seluruh inti prosesor; gzip biasa hanya satu. Di Raspberry Pi 5
# selisihnya terasa untuk arsip ratusan MB.
if command -v pigz >/dev/null 2>&1; then
    KOMPRES="pigz"; DEKOMPRES="pigz -d"
else
    KOMPRES="gzip"; DEKOMPRES="gzip -d"
fi

# — Memuat —
if [ -n "$MUAT" ]; then
    [ -f "$MUAT" ] || { echo "Berkas tidak ditemukan: $MUAT"; exit 1; }
    echo "==> Memuat image dari $MUAT (memakai $KOMPRES) ..."
    $DEKOMPRES -c "$MUAT" | $DOCKER load
    echo
    echo "Selesai. Lanjutkan dengan:"
    echo "  cd ~/chirpstack-docker && docker compose up -d"
    exit 0
fi

# — Menyimpan —
if [ ! -f "$COMPOSE_DIR/docker-compose.yml" ]; then
    echo "docker-compose.yml tidak ada di $COMPOSE_DIR."
    echo "Jalankan setup_chirpstack.sh dulu, atau tunjuk foldernya dengan --compose-dir."
    exit 1
fi

mapfile -t IMAGES <<(cd "$COMPOSE_DIR" && $DOCKER compose config --images | sort -u)
echo "==> ${#IMAGES[@]} image akan dibungkus:"
printf '    %s\n' "${IMAGES[@]}"

echo "==> Menulis $KELUAR ..."
$DOCKER save "${IMAGES[@]}" | $KOMPRES > "$KELUAR"

echo
echo "Selesai: $(du -h "$KELUAR" | cut -f1) -> $KELUAR"
echo "Salin ke flashdisk, lalu di Pi tujuan (Docker sudah terpasang):"
echo "  bash simpan_image_docker.sh --muat <berkas>"

```

Kode 14.11: gateway/simpan_image_docker.sh — bungkus dan muat ulang image ChirpStack

```

# Modul 11 - penyesuaian chirpstack-docker untuk band EU433.
#
# Berkas ini DITARUH DI DALAM folder chirpstack-docker hasil clone, di samping
# docker-compose.yml bawaannya. Docker Compose otomatis membaca
# docker-compose.override.yml dan menggabungkannya -- repo resmi ChirpStack
# tidak perlu diubah sama sekali, sehingga `git pull` di sana tetap bersih.

```

```
#
# YANG DIUBAH HANYA SATU HAL: awalan topik MQTT, dari eu868 menjadi eu433.
#
# Awalan itu bukan sekadar label. chirpstack-gateway-bridge menerbitkan tiap
# uplink ke topik "<region>/gateway/<GatewayEUI>/event/up", dan dari awalan
# itulah ChirpStack menyimpulkan rencana frekuensi mana yang berlaku bagi
# gateway tersebut. Bila dibiarkan eu868 sementara radionya 433 MHz, uplink
# tetap sampai tetapi ditolak dengan alasan frekuensi tidak dikenal.
#
# Region eu433 sendiri sudah aktif secara bawaan di chirpstack.toml bawaan
# repo (lihat enabled_regions), jadi tidak ada yang perlu ditambahkan di sana.

services:
  chirpstack-gateway-bridge:
    environment:
      - INTEGRATION__MQTT__EVENT_TOPIC_TEMPLATE=eu433/gateway/{{ .GatewayID }}/event/{{ .EventType }}
      - INTEGRATION__MQTT__STATE_TOPIC_TEMPLATE=eu433/gateway/{{ .GatewayID }}/state/{{ .StateType }}
      - INTEGRATION__MQTT__COMMAND_TOPIC_TEMPLATE=eu433/gateway/{{ .GatewayID }}/command/#
```

Kode 14.12: gateway/docker-compose.override.yml — satu-satunya perubahan: region EU433

```
# Modul 11 - menjalankan single_chan_pkt_fwd.py sebagai layanan systemd,
# supaya gateway ikut hidup sendiri setiap Raspberry Pi dinyalakan.
#
# Berkas ini adalah TEMPLATE: __USER__ dan __DIR__ diisi oleh
# siapan_gateway.sh --service. Untuk memasang sendiri secara manual:
#
#   sed -e "s|__USER__|${USER}|" -e "s|__DIR__|${PWD}|" lorawan-gateway.service \
#   | sudo tee /etc/systemd/system/lorawan-gateway.service
#   sudo systemctl daemon-reload
#   sudo systemctl enable --now lorawan-gateway
#
# Frekuensi dan alamat server TIDAK ditulis di sini, melainkan di
# /etc/default/lorawan-gateway -- sehingga satu berkas unit yang sama dipakai
# kelompok mana pun, dan mengubah kanal tidak perlu menyentuh systemd:
#
#   SERVER=192.168.1.45
#   FREQ=433375000
#   OPSI=
#
# Melihat keluarannya (pengganti terminal): journalctl -fu lorawan-gateway

[Unit]
Description=LoRaWAN single channel gateway (Modul 11) - SX1276 ke ChirpStack
Documentation=file://__DIR__/../README.md
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=__USER__
WorkingDirectory=__DIR__
EnvironmentFile=/etc/default/lorawan-gateway

# -u mematikan buffering Python, supaya tiap baris langsung masuk journal
# dan bukan menumpuk sampai proses berhenti.
#
# $OPSI ditulis TANPA kurung kurawal: systemd memecahnya menjadi beberapa
# argumen bila lebih dari satu opsi, sedangkan ${SERVER} dan ${FREQ}
# memakai kurawal karena masing-masing memang satu nilai utuh.
ExecStart=/usr/bin/python3 -u single_chan_pkt_fwd.py --server ${SERVER} --freq ${FREQ} $OPSI

# Radio bisa gagal diinisialisasi bila HAT belum siap saat boot; coba lagi
# alih-alih menyerah. StartLimit menjaga supaya kegagalan permanen (HAT lepas,
# SPI mati) tidak berubah menjadi loop tanpa henti.
Restart=on-failure
RestartSec=10
StartLimitIntervalSec=300
StartLimitBurst=10

[Install]
WantedBy=multi-user.target
```

Kode 14.13: gateway/lorawan-gateway.service — unit systemd untuk gateway otomatis

14.7 Perintah Deploy

```
# 1. DI RASPBERRY PI – server, sekali saja (belum pernah pakai Docker?
#   baca Lampiran A di akhir bab ini lebih dulu – cukup lima perintah)
bash Modul11_lorawan_chirpstack/gateway/setup_chirpstack.sh
pip3 install -r Modul11_lorawan_chirpstack/gateway/requirements.txt
# Raspberry Pi OS Bookworm ke atas menolak pip di luar venv; pakai apt:
#   sudo apt install python3-spidev python3-rpi-lgpio python3-paho-mqtt

# 2. DI RASPBERRY PI – gateway; catat Gateway EUI yang tercetak
python3 Modul11_lorawan_chirpstack/gateway/single_chan_pkt_fwd.py

# 3. DI PC – kedua node, SETELAH gateway berjalan
python3 Modul11_lorawan_chirpstack/upload_auto.py --monitor

# 4. DI RASPBERRY PI – pantau data yang sudah didekripsi server
python3 Modul11_lorawan_chirpstack/gateway/uplink_listen.py
```

Bila **beberapa kelompok bekerja bersamaan** di satu ruangan, langkah 1 dan 2 diganti satu perintah `siapkan_gateway.sh --kelompok <n>` dan hanya satu Pi yang perlu menjalankan ChirpStack — lihat Lampiran B pada bab ini.

Pre-flight checklist

- ☐ Antena terpasang pada ketiga radio (dua shield dan satu HAT).
- ☐ SPI aktif di Raspberry Pi (`ls /dev/spidev0.0`), dan `rpi-lgpio` terpasang — **bukan** RPi.GPIO asli (Pi 5).
- ☐ `docker compose ps` di `~/chirpstack-docker` menunjukkan seluruh service Up (cara membacanya: Lampiran A).
- ☐ Web UI ChirpStack terbuka di `http://<ip-pi>:8080` (admin / admin).
- ☐ Gateway EUI hasil `single_chan_pkt_fwd.py` sudah didaftarkan dan berstatus online.
- ☐ DevEUI dan AppKey di ChirpStack **sama persis** dengan `src/node/lorawan_keys.h` (ingat urutan byte DevEUI).
- ☐ Gateway dijalankan **lebih dahulu**, baru node dinyalakan.

14.8 Percobaan

14.8.1 EXP-01 — Register Device: Menyiapkan Jaringan Sebelum Ada Satu Pun Paket

Jalankan server, lalu daftarkan tiga hal lewat web UI: gateway, device profile, dan kedua device. Urutan ini tidak boleh dibalik — device tidak bisa dibuat tanpa profile, dan uplink tidak akan diterima dari gateway yang tidak dikenal.

1. **Gateway** — Gateways → Add gateway. Isi *Gateway EUI* dengan nilai yang dicetak `single_chan_pkt_fwd.py` saat dijalankan.
2. **Device profile** — Device profiles → Add. Region **EU433**, MAC version **LoRaWAN 1.0.3**, Regional parameters **A**, aktifkan **Device supports OTAA**.
3. **Application** → Add, lalu **Add device** dua kali dengan DevEUI dari `lorawan_keys.h`, dan isi *Application key* dengan AppKey masing-masing node.

Expected output — gateway (sebelum node dinyalakan)

```
=== SINGLE CHANNEL LoRaWAN GATEWAY ===
Init SX1276 ... OK
Gateway EUI : 2CCF67FFFE53AC11
Server      : 127.0.0.1:1700 (Semtech UDP)
Kanal       : 433.175 MHz SF7BW125 CR4/5
Sync word   : 0x34 (LoRaWAN publik)
```

Data capture

Parameter	Hasil
Gateway EUI yang dipakai	2CCF67FFFE53AC11 (diturunkan dari MAC eth0 Pi)

Parameter	Hasil
Status gateway di ChirpStack setelah $\pm$ 30 detik	online — lastSeenAt terisi dan maju tiap 30 detik, berasal dari pesan stat (bukan dari uplink; lihat Lampiran A.6)
Region pada device profile	EU433 , LoRaWAN 1.0.3, Regional parameters A, OTAA
DevEUI Node 1 / Node 2 di ChirpStack	0011223344556601 / 0011223344556602

Tabel 14.9: Hasil pengamatan EXP-01 — pendaftaran perangkat

CHECKPOINT

Gateway harus berubah menjadi **online** di ChirpStack sebelum node dinyalakan. Statusnya berasal dari pesan stat yang dikirim gateway tiap 30 detik; kalau tetap offline, periksa --server menunjuk ke IP yang benar dan port 1700/UDP tidak diblokir.

14.8.2 EXP-02 — OTAA Join, dan Satu Kesalahan yang Sengaja Dibuat

Nyalakan Node 1 saja lebih dahulu (buka Serial Monitor 115200), amati sampai [JOIN] BERHASIL, baru nyalakan Node 2.

Expected output — node

```

=== LoRaWAN NODE 1 - Ruang 1 (Kelompok 1) ===
Kanal : 433.175 MHz SF7BW125 (kanal tunggal)
Interval: 30 detik
Menyusun JoinRequest OTAA ...
[JOIN] mengirim JoinRequest ...
[TX] radio: 433175 kHz DR5
[JOIN] BERHASIL
  DevAddr : 1EBD973
  NetID : 0
[TX] radio: 433175 kHz DR5
[TX #1] FPort=1 "T=25.0,H=66" -> antre di LMIC
[TX] selesai (FCntUp=1)

```

DevAddr berbeda di tiap sesi — server memberi yang baru setiap kali perangkat join.

Expected output — gateway

```

17:18:43.397 [RX] 23 B RSSI=-66 dBm SNR= 9.8 dB tms=1631694015
JoinRequest DevEUI=0011223344556601 DevNonce=23600
17:18:43.611 [TX] downlink dijadwalkan: 17 B 433.175 MHz SF7BW125 dalam 4786 ms
JoinAccept (17 byte)
17:18:48.443 [TX] terkirim (meleset -0.2 ms dari jadwal)

```

Sesudah join berhasil, **buat kesalahannya dengan sengaja**: balik urutan byte DEVEUI di lorawan_keys.h (tulis MSB-first seperti tampilan ChirpStack), unggah ulang, dan amati apa yang terjadi di kedua sisi.

Data capture

Parameter	Node 1	Node 2
Percobaan join yang berhasil	ke-1	ke-1
Waktu mengirim JoinRequest sampai BERHASIL	8,2 detik	6,5 detik
DevAddr yang diberikan server	01EBD973	01DC98E3
RSSI / SNR JoinRequest di gateway	- 66 dBm / 9,8 dB	- 64 dBm / 9,8 dB

Parameter	Node 1	Node 2
Jeda JoinAccept yang dijadwalkan server (ms)	4786	4786
Ketelitian penembakan JoinAccept	- 0,2 ms	- 0,4 ms
DevEUI terbalik: gateway tetap menerima paket?		
DevEUI terbalik: ChirpStack mencatat join?		

Tabel 14.10: Hasil pengamatan EXP-02 — OTAA join

Sebagian besar dari 6–8 detik itu **bukan** waktu udara: LMIC menunda JoinRequest pertama beberapa detik secara acak (supaya sekumpulan node yang menyala bersamaan tidak serentak bicara), lalu JoinAccept baru boleh dikirim 5 detik sesudah uplink. Waktu udara pakatnya sendiri hanya puluhan milidetik.

BUKA ABSTRAKSINYA

Di `src/node/main.cpp`, `lockSingleChannel()` dipanggil **tiga kali**, dan salah satunya ada di dalam `case EV_JOINING`. Hapus yang di `EV_JOINING` saja, unggah ulang, lalu baca baris `[TX] radio: ... kHz`. Jelaskan angka yang muncul, dan telusuri di sumber LMIC mengapa penguncian kanal yang dilakukan di `setup()` bisa hilang begitu saja. Petunjuk: `LMICeulike_initJoinLoop()` memanggil `LMICbandplan_initDefaultChannels()`.

CHECKPOINT

DevAddr kedua node harus **berbeda**, dan keduanya diberikan server, bukan ditentukan firmware. Bila salah satu node tidak pernah join sementara yang lain lancar, periksa AppKey-nya di ChirpStack sebelum menyalahkan radio.

14.8.3 EXP-03 — Uplink Dua Node: Mengenali Asal Data Tanpa Melihat DevEUI

Biarkan kedua node berjalan minimal 5 menit. Buka Applications → Devices → Events di ChirpStack untuk kedua perangkat, dan jalankan `uplink_listen.py` di Raspberry Pi.

Expected output — `uplink_listen.py`

waktu	device	FCnt	payload	suhu	RH	RSSI	SNR
17:18:48	node1-ruangan-1	0	T=25.0,H=66	25.0C	66%	-67 dBm	10.0 dB
17:19:32	node2-ruangan-2	0	T=32.1,H=47	32.1C	47%	-64 dBm	11.8 dB
17:20:00	node1-ruangan-1	3	T=26.0,H=61	26.0C	61%	-67 dBm	9.8 dB
17:20:11	node2-ruangan-2	2	T=33.7,H=44	33.7C	44%	-63 dBm	9.8 dB

Data capture (isi dari CSV `uplink_listen.py` atau tabel Events)

Parameter	Node 1	Node 2
Uplink dikirim (dari [TX #n] di serial)	10	9
Uplink yang sampai ke ChirpStack	9	9
Rentang suhu yang terlihat	25,0–28,5 °C (spesifikasi 25–30)	31,1–34,4 °C (spesifikasi 28–35)
Rentang kelembapan yang terlihat	60–71 % (spesifikasi 60–75)	40–65 % (spesifikasi 40–65)
FCnt awal → akhir	0 → 10	0 → 9

Parameter	Node 1	Node 2
Ada FCnt yang terlewat? Berapa?	ya — FCnt 2 (satu uplink hilang)	tidak ada
RSSI rata-rata / SNR rata-rata	– 66,1 dBm / 9,7 dB	– 62,8 dBm / 10,0 dB

Tabel 14.11: Hasil pengamatan EXP-03 — uplink dua node

Satu frame per node **tidak** dihitung di tabel ini: uplink 15 byte tanpa FPort yang muncul beberapa detik sesudah join. Itu jawaban otomatis LMIC atas perintah MAC dari server, bukan data aplikasi — lihat `logserial.md`.

CHECKPOINT

Tanpa melihat kolom nama perangkat, praktikan harus bisa menebak node asal tiap baris hanya dari angkanya. Bila suhu Node 2 pernah terbaca di bawah 28 °C, berarti yang sedang dibaca adalah baris Node 1.

14.8.4 EXP-04 — Kanal Tunggal: Harga yang Dibayar

Amati apa yang hilang karena gateway hanya punya satu modem. Dua pengamatan, keduanya ada di log yang sudah dikumpulkan.

1. **Setengah-dupleks.** Cari di log gateway momen [TX] terkirim dan periksa apakah ada uplink node lain yang jatuh dalam jendela ± 100 ms di sekitarnya. Bandingkan dengan [TX] selesai (FCntUp=...) di kedua node — uplink yang tidak pernah muncul di sisi gateway adalah uplink yang hilang.
2. **RX2 sebagai cadangan.** Bila server terlambat mengirim jawabannya, gateway mencetak [TX] TERLAMBAT - jadwal sudah lewat lalu ChirpStack mencoba lagi lewat RX2 (434,665 MHz SF12). Cari kejadian ini di log sendiri.

Data capture

Parameter	Hasil
Jumlah [TX] TERLAMBAT selama sesi	0 (pada sesi lain, saat uplink pertama sesudah join, pernah terjadi 1×)
Jumlah downlink yang akhirnya lewat RX2	0 (pada sesi tersebut: 1, di 434,665 MHz SF12 — dan berhasil diterima node)
Ketelitian jadwal downlink, terbaik / terburuk	– 0,2 ms / – 0,4 ms dari 4 downlink
Uplink node yang tidak sampai ke gateway	1 dari 19 — Node 1 FCnt=2, saat gateway sedang menangani JoinRequest Node 2
Bila kedua node dinyalakan bersamaan	1 JoinRequest hilang , node itu baru join 69 detik kemudian

Tabel 14.12: Hasil pengamatan EXP-04 — harga gateway kanal tunggal

Kejadian FCnt yang melompat itu justru bahan analisis paling berharga di modul ini: cocokkan cap waktu [TX #n] di serial node dengan log gateway pada detik yang sama, dan akan terlihat gateway sedang sibuk dengan node lain. `logserial.md` memuat kedua log yang sudah disejajarkan.

BUKA ABSTRAKSINYA

Digateway/single_chan_pkt_fwd.py, fungsi start_rx() mengembalikan **frekuensi dan SF**, bukan hanya mode radio. Hapus dua baris _set_frequency dan _set_spreading_factor di sana, jalankan sampai ada satu downlink RX2, lalu jelaskan mengapa sesudah itu gateway tidak pernah menerima apa pun lagi — dan mengapa tidak ada satu pun pesan galat yang muncul.

14.8.5 Verifikasi Perangkat Keras

CATATAN

Diuji di perangkat pada 22 Agustus 2026 — dua Arduino Uno asli + Dragino LoRa Shield v1.2 (/dev/ttyACM1, /dev/ttyACM2), Raspberry Pi 5 (Debian 13)

1. Dragino LoRa GPS HAT v1.4 sebagai gateway, ChirpStack v4 di Docker pada Pi yang sama, region EU433. Build kedua environment sukses (RAM 69,8 %, Flash 75,0 % dari ATmega328P). EXP-02, EXP-03, dan EXP-04 dijalankan dengan kode yang persis seperti di repositori ini, dan datanya nyata — kedua node **join OTAA pada percobaan pertama, 18 dari 19 uplink** sampai ke aplikasi selama jendela rekam 5,5 menit (yang satu hilang beserta penyebabnya terdokumentasi di logserial.md), dan seluruh downlink meleset kurang dari setengah milidetik dari jadwal server. Skema pembagian kelompok (Lampiran B) diuji terpisah dengan menjalankan kelompok 2 di 433,375 MHz.

Yang **belum** dijalankan pada sesi ini dan memang menjadi pekerjaan praktikan: EXP-01 lewat web UI (di sesi ini pendaftaran dilakukan dengan gateway/provision_lab.py supaya cepat direproduksi), percobaan DevEUI sengaja dibalik di EXP-02, kedua kotak “Buka abstraksinya”, dan seluruh Challenge.

14.8.6 Log Serial Terverifikasi

Log serial lengkap hasil uji pada board nyata, bukan contoh. Bagian ini hanya dicetak pada edisi dosen. Hasil aktual dari perangkat. Node: baud **115200**, LoRaWAN Class A OTAA, kanal tunggal **433.175 MHz SF7BW125 CR4/5** (kelompok 1), interval uplink 30 detik, payload ASCII T=.,H=.. di FPort 1 (unconfirmed). Gateway: Raspberry Pi 5 + Dragino LoRa GPS HAT v1.4 menjalankan single_chan_pkt_fwd.py, ChirpStack v4 (Docker) di Pi yang sama, region EU433.

Board & Port

Peran	Environment / skrip	Port	Board
Node 1 (Ruangan 1)	node1	/dev/ttyACM1	Uno asli (2341:0043) + Dragino LoRa Shield v1.2
Node 2 (Ruangan 2)	node2	/dev/ttyACM2	Uno asli (2341:0043) + Dragino LoRa Shield v1.2
Gateway	gateway/single_chan_pkt_fwd.py	—	Raspberry Pi 5 +

Peran	Environment / skrip	Port	Board
			LoRa GPS HAT v1.4, Gateway EUI 2CCF67FFFE53AC11
Server	ChirpStack v4 (Docker)	:8080 UI, :1700/udp, :1883 MQTT	Raspberry Pi 5 yang sama

Tabel 14.13: Log serial Modul 11: Board & Port

Sesi rekam: **17:18:38–17:24:18**, Node 1 dinyalakan lebih dahulu dan Node 2 menyusul 45 detik kemudian — persis urutan yang diminta README, supaya kedua JoinRequest tidak berebut satu-satunya kanal yang didengar gateway.

EXP-02 — OTAA Join

Sisi node (Node 1), dari boot sampai uplink pertama:

```
[17:18:40] [ 1.5] === LoRaWAN NODE 1 - Ruangan 1 (Kelompok 1) ===
[17:18:40] [ 1.5] Kanal : 433.175 MHz SF7BW125 (kanal tunggal)
[17:18:40] [ 1.5] Interval: 30 detik
[17:18:40] [ 1.6] Menyusun JoinRequest OTAA ...
[17:18:40] [ 1.6] [JOIN] mengirim JoinRequest ...
[17:18:43] [ 4.7] [TX] radio: 433175 kHz DR5
[17:18:48] [ 9.8] [JOIN] BERHASIL
[17:18:48] [ 9.8] DevAddr : 1EBD973
[17:18:48] [ 9.8] NetID : 0
[17:18:48] [ 9.8] [TX] radio: 433175 kHz DR5
[17:18:48] [ 9.8] [TX #1] FPort=1 "T=25.0,H=66" -> antre di LMIC
[17:18:49] [10.9] [TX] selesai (FCntUp=1)
```

Kejadian yang sama dari sisi gateway — perhatikan jeda **4786 ms** yang ditentukan server untuk JoinAccept (jendela RX1 join = 5 detik dikurangi waktu pemrosesan), dan ketelitian penembakannya:

```
17:18:43.397 [RX] 23 B RSSI= -66 dBm SNR= 9.8 dB tmst=1631694015
JoinRequest DevEUI=0011223344556601 DevNonce=23600
17:18:43.611 [TX] downlink dijadwalkan: 17 B 433.175 MHz SF7BW125 dalam 4786 ms
JoinAccept (17 byte)
17:18:48.443 [TX] terkirim (meleset -0.2 ms dari jadwal)
17:18:48.524 [RX] 24 B RSSI= -67 dBm SNR= 10.0 dB tmst=1636821687
UnconfirmedDataUp DevAddr=01EBD973 FCnt=0 FPort=1 (payload terenkripsi)
17:18:48.853 [TX] downlink dijadwalkan: 13 B 433.175 MHz SF7BW125 dalam 671 ms
UnconfirmedDataDown (13 byte)
17:18:49.565 [TX] terkirim (meleset -0.2 ms dari jadwal)
17:18:54.669 [RX] 15 B RSSI= -67 dBm SNR= 9.5 dB tmst=1642965879
UnconfirmedDataUp DevAddr=01EBD973 FCnt=1 (payload terenkripsi)
```

Node 2, join pada percobaan pertama juga, 45 detik sesudahnya:

```
[17:19:25] [ 1.7] [JOIN] mengirim JoinRequest ...
[17:19:26] [ 3.1] [TX] radio: 433175 kHz DR5
[17:19:31] [ 8.2] [JOIN] BERHASIL
[17:19:31] [ 8.2] DevAddr : 1DC98E3
```

Tiga hal yang terbaca dari cuplikan di atas.

1. **DevAddr diberikan server, dan berbeda untuk tiap node** (01EBD973 dan 01DC98E3) — bandingkan NODE_ID M05–M10 yang ditentukan build flag. Nilainya juga berganti setiap kali join diulang, jadi jangjan heran bila berbeda dari sesi Anda.
2. **Ada uplink yang tidak pernah diminta kode aplikasi.** Baris FCnt=1 berukuran 15 byte, tanpa FPort, dan tidak didahului [TX #n] di sisi node. Itu jawaban otomatis LMIC atas perintah MAC

(DevStatusReq) yang dititipkan server pada downlink 13 byte sebelumnya — lapisan MAC bekerja sendiri tanpa sepengetahuan loop().

3. **Gateway tidak tahu isinya.** Setiap baris uplink diakhiri (payload terenkripsi); gateway hanya membaca amplopnya (DevAddr, FCnt, FPort).

EXP-03 — Uplink dua node, 5 menit

Keluaran uplink_listen.py (data yang **sudah** didekripsi network server, lewat MQTT):

waktu	device	FCnt	payload	suhu	RH	RSSI	SNR
17:18:48	node1-ruangan-1	0	T=25.0,H=66	25.0C	66%	-67 dBm	10.0 dB
17:19:32	node2-ruangan-2	0	T=32.1,H=47	32.1C	47%	-64 dBm	11.8 dB
17:20:00	node1-ruangan-1	3	T=26.0,H=61	26.0C	61%	-67 dBm	9.8 dB
17:20:11	node2-ruangan-2	2	T=33.7,H=44	33.7C	44%	-63 dBm	9.8 dB

Rekapitulasi jendela rekam (hanya frame ber-payload; frame MAC dihitung terpisah):

Node	DevAddr	Uplink dikirim (serial)	Diterima gateway	Sampai ChirpStack	RSSI rata-rata	SNR rata-rata
Node 1	01EBD973	10	9	9	106,1 106,1 dBm % (FCnt 2 hilang)	9,7 dB
Node 2	01DC98E3	9	9	9	062,8 062,8 dBm %	10,0 dB

Tabel 14.14: Log serial Modul 11: EXP-03 — Uplink dua node, 5 menit

Rentang dummy yang terlihat di server — cukup untuk mengenali asal data tanpa melihat nama perangkat:

Node	Spesifikasi	Terukur di sesi ini
Node 1 (Ruangan 1)	25–30 °C, 60–75 %	25,0–28,5 °C, 60–71 %
Node 2 (Ruangan 2)	28–35 °C, 40–65 %	31,1–34,4 °C, 40–65 %

Tabel 14.15: Log serial Modul 11: EXP-03 — Uplink dua node, 5 menit

Statistik gateway sepanjang sesi: **0** CRC salah, **0** [TX] TERLAMBAT, seluruh downlink meleset **-0,2 s.d. -0,4 ms** dari tmst yang diminta server.

Satu uplink yang benar-benar hilang

FCnt Node 1 melompat dari 1 ke 3 — uplink FCnt=2 tidak pernah sampai. Penyebabnya terbaca dengan mencocokkan cap waktu kedua log. Node 1 mulai memancar pada 17:19:27, kurang dari setengah detik sesudah gateway menyelesaikan penerimaan JoinRequest Node 2:

node 1 :	[17:19:27] [48.7] [TX] radio: 433175 kHz DR5
	[17:19:27] [48.7] [TX #2] FPort=1 "T=26.0,H=61" -> antre di LMIC
gateway:	17:19:26.825 [RX] 23 B ... JoinRequest DevEUI=0011223344556602
	17:19:27.039 [TX] downlink dijadwalkan: 17 B ... JoinAccept (17 byte)
	(tidak ada [RX] apa pun dari Node 1 di sini)
	17:19:31.871 [TX] terkirim (meleset -0.4 ms dari jadwal)
	17:20:00.071 [RX] 24 B ... DevAddr=01EBD973 FCnt=3 FPort=1

Node 1 mengirim tanpa tahu apa-apa, gateway tidak menerimanya, dan **tidak ada satu pun pesan galat di kedua sisi** — persis tabrakan senyap yang diukur di M08, kini pada jaringan yang sudah berprotokol. Yang berbeda dari M08 hanyalah akibatnya terlihat: FCnt yang melompat langsung menunjukkan ada frame yang hilang, tanpa perlu menghitung manual seperti dulu. Inilah bahan EXP-04.

Catatan: dua node dinyalakan bersamaan

Pada sesi percobaan lain kedua node di-*reset* pada detik yang sama. Hasilnya: Node 1 join mulus, sedangkan JoinRequest pertama Node 2 tidak pernah sampai ke gateway dan baru berhasil pada percobaan kedua, **69 detik** kemudian (LMIC menunda percobaan berikutnya makin lama tiap kali gagal):

```
[ 1.7] [JOIN] mengirim JoinRequest ...
[ 9.0] [TX] radio: 433175 kHz DR5
[ 16.7] [JOIN] tidak ada JoinAccept di RX1/RX2
[ 78.3] [TX] radio: 433175 kHz DR5
[ 83.4] [JOIN] BERHASIL
```

Inilah harga gateway kanal tunggal yang setengah-dupleks: selama ia menembakkan JoinAccept untuk satu node, ia tuli terhadap node lain. Karena itu README meminta node dinyalakan **satu per satu**, dan bukan karena firmware-nya rewel.

Uji skema multi-kelompok (Lampiran B)

Pembagian kanal per kelompok diuji dengan menjalankan kelompok 2 secara penuh: gateway dipindah ke 433.375 MHz (--freq 433375000), node di-flash dengan -D KELOMPOK=2.

```
node : === LoRaWAN NODE 2 - Ruang 2 (Kelompok 2) ===
      Kanal : 433.375 MHz SF7BW125 (kanal tunggal)
      [ 76.3] [TX] radio: 433375 kHz DR5
      [ 81.4] [JOIN] BERHASIL
      [ 81.4] DevAddr : D1C237

gateway: 17:12:27.838 [RX] 23 B RSSI= -65 dBm SNR= 9.5 dB
          JoinRequest DevEUI=0011223344557702 DevNonce=12184
          17:12:28.055 [TX] downlink dijadwalkan: 17 B 433.375 MHz SF7BW125 dalam 4783 ms
          17:12:32.884 [TX] terkirim (meleset -0.3 ms dari jadwal)
```

DevEUI bergeser sendiri ke ...7702 mengikuti nomor kelompok, dan uplink-nya masuk ke Application **praktikum-wsn-k2** yang terpisah dari praktikum-wsn milik kelompok 1 (diperiksa lewat API: devAddr 00d1c237, fCntUp 4). Join berhasil pada percobaan kedua; percobaan pertamanya tidak diterima node meski JoinAccept sudah ditembakkan tepat waktu — kejadian yang sesekali muncul juga pada kelompok 1, dan memang sifat gateway kanal tunggal.

Ringkasan Verifikasi Hardware

Diuji di perangkat pada **2026-08-22**: 2× Arduino Uno asli + Dragino LoRa Shield v1.2 (port /dev/ttyACM1 dan /dev/ttyACM2), Raspberry Pi 5 (Debian 13, Docker) + Dragino LoRa GPS HAT v1.4, ChirpStack v4 region EU433. Build kedua environment sukses (RAM 69,8 %, Flash 75,0 % dari ATmega328P). Seluruh alur modul berjalan pada perangkat sungguhan dengan kode yang persis seperti di repositori ini: gateway online di ChirpStack, kedua node **join OTAA pada percobaan pertama**, dan 18 dari 19 uplink sampai ke aplikasi selama jendela rekam 5,5 menit — satu yang hilang terdokumentasi di atas beserta penyebabnya. Skema multi-kelompok diuji terpisah untuk kelompok 2.

EXP-01 (pendaftaran lewat web UI), percobaan DevEUI sengaja dibalik pada EXP-02, kedua kotak **Buka abstraksinya**, dan seluruh Challenge belum dijalankan — semuanya adalah pekerjaan praktikan.

Dua temuan dari pengujian ini yang sudah diperbaiki di kode, dan keduanya menjadi bahan kotak **Buka abstraksinya** di README:

1. LMIC_startJoining() mengembalikan tabel kanal ke bawaan EU868, sehingga penguncian kanal tunggal harus **diulang di EV_JOINING** — tanpa itu node mengirim JoinRequest di 868.1 MHz dan gateway diam tanpa pesan galat apa pun.
2. start_rx() di gateway harus mengembalikan **frekuensi dan SF**, bukan hanya mode radio. Sesudah satu downlink RX2 (434.665 MHz SF12), gateway yang tidak mengembalikan keduanya akan tetap mendengarkan di kanal RX2 dan tuli selamanya terhadap seluruh uplink.
3. Pesan **stat** memakai format cap waktu yang **berbeda** dari rxpk: 2006-01-02 15:04:05 MST, bukan ISO 8601. Dengan format yang salah, gateway bridge menolak seluruh stat (could not handle packet di log-nya) sementara uplink tetap mengalir normal — gejalanya hanya satu: gateway selamanya tampak **offline** di ChirpStack meski datanya jelas masuk. Ditemukan saat menguji reboot pada 2026-08-23 dan diperbaiki lewat stat_time_now().

Uji reboot (Lampiran A.5 dan C.4)

Raspberry Pi di-*reboot* pada 2026-08-23 pukul 02:12:52 untuk memastikan seluruh lab bangkit sendiri:

Peristiwa	Waktu
Kernel selesai boot	02:13:21 (29 detik sesudah perintah)
SSH menjawab	detik ke-40
lorawan-gateway.service aktif	02:13:33
docker.service aktif	02:13:38
Web UI ChirpStack menjawab HTTP 200	detik ke-48

Tabel 14.16: Log serial Modul 11: Uji reboot (Lampiran A.5 dan C.4)

Ketujuh container kembali Up sendiri, radio gateway terinisialisasi (Init SX1276 ... OK), kedua aplikasi masih ada, dan sesi OTAA kedua node utuh — fCntUp 307 dan 306, melanjutkan hitungan sebelum reboot, sehingga tidak ada join ulang.

14.9 Pengukuran

A. Join OTAA

Node	Join ke-	Waktu join (s)	DevAddr	RSSI JoinRequest	Jeda JoinAccept (ms)
Node 1	1	8,2	01EBD973	– 66 dBm	4786
Node 2	1	6,5	01DC98E3	– 64 dBm	4786

Tabel 14.17: Lembar pengukuran A — join OTAA

B. Uplink 5 menit

Node	Dikirim	Diterima gateway	Muncul di ChirpStack	Loss (%)	RSSI rata-rata	SNR rata-rata
Node 1	10	9	9	10	– 66,1 dBm	9,7 dB
Node 2	9	9	9	0	– 62,8 dBm	10,0 dB

Tabel 14.18: Lembar pengukuran B — uplink selama 5 menit

C. Rentang data dummy (bukti asal data)

Node	Suhu min	Suhu maks	RH min	RH maks	Sesuai spesifikasi?
Node 1 (25–30 °C, 60–75 %)	25,0	28,5	60	71	ya
Node 2 (28–35 °C, 40–65 %)	31,1	34,4	40	65	ya

Tabel 14.19: Lembar pengukuran C — rentang data dummy

14.10 Analisis

1. Dari Tabel 14.18, hitung loss tiap node. Bandingkan dengan loss M08 (Pure ALOHA) pada kepadatan kirim yang sebanding. Apakah LoRaWAN menghapus tabrakan, atau hanya memindahkan penanganannya? Dukung jawaban dengan data.
2. Gateway mencetak DevAddr dan FCnt untuk tiap uplink, tetapi tidak pernah mencetak suhu. Jelaskan secara teknis kunci mana yang dipegang siapa, dan mengapa pemisahan ini masuk akal untuk jaringan dengan ribuan gateway milik pihak lain.

3. Dari Tabel 14.17, bandingkan jeda JoinAccept (± 5 detik) dengan jeda downlink data (± 1 detik). Mengapa spesifikasi LoRaWAN memberi waktu jauh lebih panjang khusus untuk JoinAccept?
4. Modul ini mematikan ADR (`LMIC_setAdrMode(0)`). Jelaskan apa yang akan dilakukan server bila ADR aktif dan SNR uplink tinggi, lalu jelaskan mengapa hal itu justru mematikan komunikasi pada gateway kanal tunggal.
5. Bandingkan FCnt LoRaWAN dengan SEQ yang dibuat sendiri di M08C. Sebutkan dua hal yang dilakukan FCnt tetapi tidak dilakukan SEQ, dan jelaskan akibatnya bila keduanya tidak ada.

14.11 Concept Check

1. Mengapa DevEUI ditulis terbalik urutan bytenya di firmware sementara AppKey tidak? Apa gejalanya bila keduanya tertukar?
2. Apa beda peran gateway di modul ini dengan peran gateway di M07? Sebutkan satu hal yang bisa dilakukan gateway M07 tetapi tidak boleh dilakukan gateway LoRaWAN.
3. Node ini Class A. Apa yang terjadi bila server ingin mengirim perintah ke node tepat setelah node selesai membuka jendela RX-nya?
4. Sync word LoRaWAN 0×34 , sedangkan M01–M10 memakai 0×12 . Apa yang terjadi bila node M08 dan node M11 dinyalakan bersamaan di frekuensi yang sama?
5. OTAA menurunkan kunci sesi saat join; ABP menanamnya langsung. Sebutkan satu keunggulan OTAA yang tidak dimiliki ABP, dan satu situasi nyata yang membuat orang tetap memilih ABP.

14.12 Challenge (Tugas Modifikasi)

Modifikasi kode, bukan sekadar menjelaskan hasil.

Level 2 — Modifikasi · Memakai fitur protokol yang sudah tersedia

CH-1 — Confirmed uplink Ubah argumen terakhir `LMIC_setTxData2()` menjadi 1, unggah ulang satu node saja, lalu bandingkan log kedua node: berapa downlink tambahan yang muncul di gateway, dan apa yang terjadi pada tingkat keberhasilan node yang satunya?

CH-3 — Downlink Kirim downlink dari ChirpStack (Devices $\rightarrow$ Queue) berisi satu byte, dan tampilkan isinya di Serial Monitor node. Ukur berapa lama pesan itu menunggu sebelum benar-benar mengudara, dan jelaskan mengapa.

Level 3 — Tantangan · Mengukur keuntungan menyerahkan MAC ke protokol

CH-2 — Payload biner Ganti payload ASCII `T=27.4,H=68` (11 byte) dengan 3 byte biner: suhu dikali 10 sebagai `int16`, kelembapan sebagai `uint8`. Tulis decoder-nya di device profile ChirpStack, lalu hitung penghematan waktu udara memakai kalkulator airtime LoRa.

CH-4 — Node ketiga tanpa menyentuh gateway Tambahkan node3 dengan rentang dummy Ruang 3 sendiri. Catat apa saja yang perlu diubah di gateway (jawabannya: tidak ada) dan bandingkan dengan usaha menambah node ketiga di M10.

14.13 Laporan

Deliverable

1. Misi dan capaian pembelajaran.
2. Dasar teori ringkas — peran gateway, network server, dan application server; OTAA; Class A; keterbatasan kanal tunggal.
3. Konfigurasi — Gateway EUI, region, device profile, DevEUI dan AppKey kedua node, parameter radio.

4. Hasil eksperimen — log gateway, log serial kedua node, dan tangkapan layar ChirpStack (EXP-01 sampai EXP-04 beserta checkpoint).
5. Data pengukuran — Tabel 14.17, Tabel 14.18, dan Tabel 14.19.
6. Analisis dan concept check.
7. Challenge — minimal CH-1.
8. Kesimpulan yang disusun sendiri: bandingkan seluruh lapisan buatan tangan M04–M10 dengan yang disediakan LoRaWAN, dan tentukan untuk skenario seperti apa membangun sendiri masih lebih tepat.

14.14 Lampiran A — Docker Seperlunya

Modul ini satu-satunya di lab yang memakai Docker, dan Docker **tidak** perlu dipelajari sebagai bahan kuliah. Lampiran ini hanya berisi yang benar-benar dipakai: lima perintah, cara membaca keluarannya, dan apa yang harus dilakukan kalau ada yang tidak beres.

14.14.1 A.1 Kenapa Harus Ada Docker

ChirpStack bukan satu program. Supaya berjalan, ia memerlukan lima hal sekaligus.

Komponen	Tugasnya di modul ini
chirpstack	network + application server: memeriksa MIC, mengelola join, FCnt, downlink
chirpstack-gateway-bridge	menerima UDP dari <code>single_chan_pkt_fwd.py</code> di port 1700
postgres	menyimpan gateway, device, DevEUI/AppKey, dan sesi OTAA
redis	menyimpan keadaan sementara tiap perangkat
mosquitto	broker MQTT: pintu keluar data ke <code>uplink_listen.py</code>

Tabel 14.20: Komponen ChirpStack v4 dan tugasnya

Memasang kelimaanya satu per satu berarti mengurus lima versi, lima berkas konfigurasi, dan lima cara menyalakan — dan sebagian besar waktu praktikum akan habis di situ, bukan di LoRaWAN-nya.

Analogi yang mungkin lebih dekat: Docker itu seperti memakai **modul jadi** (modul relay, modul step-down) alih-alih merakitnya sendiri dari komponen. Kotaknya sudah berisi semua yang diperlukan dengan nilai yang sudah benar; tinggal memberi daya dan menyambungkan pin yang tepat.

Istilah Docker	Padanan yang mungkin lebih akrab
image	modul yang masih di dalam kemasan — belum menyala, isinya tetap
container	modul yang sedang menyala di atas meja
volume	EEPROM di dalam modul: isinya tidak hilang saat modul dimatikan
<code>docker-compose.yml</code>	skema rangkaian: modul apa saja, port mana disambung ke mana
<code>docker compose up -d</code>	menyalakan seluruh rangkaian sekaligus

Tabel 14.21: Istilah Docker dan padanannya

14.14.2 A.2 Lima Perintah yang Dipakai

Semuanya dijalankan **dari dalam folder** `~/chirpstack-docker` — perintah `docker compose` membaca `docker-compose.yml` di folder tempat ia dipanggil, jadi kalau dijalankan dari folder lain akan berkata tidak menemukan apa-apa.

```
cd ~/chirpstack-docker
```

Perintah	Artinya	Kapan dipakai
<code>docker compose ps</code>	apa saja yang sedang menyala	pertama kali dicek saat ada yang aneh
<code>docker compose up -d</code>	nyalakan semuanya (-d = jalan di latar belakang)	awal praktikum; aman diulang berkali-kali
<code>docker compose stop</code>	matikan sementara — data tetap aman	selesai praktikum
<code>docker compose logs -f chirpstack</code>	lihat “serial monitor” milik satu service	saat join gagal dan penyebabnya belum jelas
<code>docker compose restart</code>	matikan lalu nyalakan lagi	setelah mengubah berkas konfigurasi

Tabel 14.22: Lima perintah Docker yang dipakai modul ini

Ctrl-C pada `logs -f` hanya menutup tampilannya, **tidak** mematikan server.

PERINGATAN

Satu perintah lagi yang sebaiknya diketahui justru supaya tidak salah ketik:

```
docker compose down -v      # HAPUS container BESERTA datanya
```

-v itulah yang membuang volume — seluruh gateway, device, dan sesi OTAA yang sudah didaftarkan ikut hilang, dan EXP-01 harus diulang dari nol. Tanpa -v, down hanya membongkar container dan data tetap utuh.

14.14.3 A.3 Membaca docker compose ps

SERVICE	STATUS	PORTS
chirpstack	Up 43 minutes	0.0.0.0:8080->8080/tcp
chirpstack-gateway-bridge	Up 43 minutes	0.0.0.0:1700->1700/udp
chirpstack-gateway-bridge-basicstation	Up 43 minutes	0.0.0.0:3001->3001/tcp
chirpstack-rest-api	Up 43 minutes	0.0.0.0:8090->8090/tcp
mosquitto	Up 43 minutes	0.0.0.0:1883->1883/tcp
postgres	Up 43 minutes	5432/tcp
redis	Up 43 minutes	6379/tcp

Yang perlu dilihat hanya kolom **STATUS**: semuanya harus Up. Exited berarti mati; Restarting berarti gagal menyala berulang-ulang — dan penyebabnya selalu ada di `docker compose logs <nama service>`.

Port	Dipakai oleh	Terlihat di
8080	web UI ChirpStack	browser
1700/udp	<code>single_chan_pkt_fwd.py</code>	opsi --server pada skrip gateway
1883	<code>uplink_listen.py</code>	opsi --host
8090	<code>provision_lab.py</code> (REST API)	opsi --api
3001	protokol BasicStation	tidak dipakai modul ini, biarkan saja

Tabel 14.23: Port yang dipakai tiap komponen

postgres dan redis tidak punya 0.0.0.0: di depan portnya: keduanya hanya bisa dihubungi container lain, tidak dari jaringan. Itu memang disengaja.

14.14.4 A.4 Perlu sudo atau Tidak

`setup_chirpstack.sh` memasukkan pengguna pi ke grup docker, tetapi keanggotaan grup baru berlaku **setelah login ulang**. Jadi:

- Baru selesai memasang, masih di sesi SSH yang sama → muncul `permission denied ... /var/run/docker.sock`. Pakai `sudo docker compose ...`, atau keluar lalu SSH lagi.
- Sesi SSH baru → `docker compose ...` tanpa `sudo` sudah bisa. Di Pi lab ini sudah diuji dan memang bisa.

14.14.5 A.5 Sesudah Raspberry Pi Dimatikan atau Reboot

Yang terjadi	Keterangan
ChirpStack hidup sendiri	layanan Docker berstatus <code>enabled</code> , dan ketujuh container memakai kebijakan <code>restart: unless-stopped</code>
Data tetap utuh	gateway, device, DevEUI/AppKey, dan sesi OTAA tersimpan di volume; kedua node tidak perlu join ulang
<code>single_chan_pkt_fwd.py</code> tidak hidup sendiri	skrip gateway dijalankan dari terminal, jadi harus dijalankan lagi setiap kali Pi dinyalakan

Tabel 14.24: Perilaku sistem sesudah Raspberry Pi dimatikan

Satu pengecualian yang sering membingungkan: bila container pernah dihentikan **manual** dengan `docker compose stop`, ia tidak akan ikut hidup pada boot berikutnya — itu memang arti kata *unless-stopped*. Nyalakan lagi dengan `docker compose up -d`.

Diukur di Pi lab ini: sesudah `docker compose down` lalu `docker compose up -d`, web UI sudah menjawab **2 detik** kemudian, dan kedua device masih terdaftar lengkap — bukti bahwa membongkar container tidak menyentuh data.

Diuji dengan reboot sungguhan pada 23 Agustus 2026. Dari perintah `reboot` sampai layanan kembali: SSH menjawab pada detik ke-**40**, web UI ChirpStack pada detik ke-**48**, dan ketujuh container berstatus `Up` tanpa satu pun perintah diketik. Seluruh data selamat — kedua aplikasi masih ada, dan sesi OTAA kedua node utuh (`fCntUp` 307 dan 306 melanjutkan hitungan sebelum reboot), sehingga node tidak perlu join ulang.

14.14.6 A.6 Kalau Ada yang Tidak Beres

Gejala	Sebab yang paling sering	Tindakan
Web UI tidak terbuka dari laptop	Pi dan laptop beda jaringan, atau alamat IP-nya berubah	<code>hostname -I</code> di Pi, lalu buka <code>http://<ip>:8080</code>
<code>permission denied ... docker.sock</code>	belum login ulang sesudah pemasangan	keluar dan SSH lagi, atau pakai <code>sudo</code>
Satu service <code>Restarting</code> terus	konfigurasi salah atau port bentrok	<code>docker compose logs <service></code> — bacanya dari bawah
<code>port is already allocated</code>	ada program lain memakai port yang sama	matikan program itu, atau ubah nomor port di <code>docker-compose.override.yml</code>
Gateway tetap offline padahal skrip gateway jalan	hampir selalu bukan Docker: --server salah alamat, atau Gateway EUI yang didaftarkan berbeda	cocokkan EUI yang dicetak skrip dengan yang ada di UI
Uplink sampai ke gateway tetapi tidak muncul di UI	region gateway bridge bukan <code>eu433</code>	pastikan <code>docker-compose.override.yml</code> ada di <code>~/chirpstack-docker</code> , lalu <code>docker compose up -d</code>

Gejala	Sebab yang paling sering	Tindakan
Gateway tetap offline padahal uplink jelas masuk	pesan stat ditolak gateway bridge, biasanya karena format waktunya salah	docker compose logs chirpstack-gateway-bridge lalu cari could not handle

Tabel 14.25: Gejala umum dan cara menanganinya

CATATAN

Kenapa “offline” bisa berdampungan dengan data yang masuk. Status online sebuah gateway di ChirpStack tidak ditentukan oleh uplink yang diteruskannya, melainkan oleh pesan **stat** yang dikirim gateway tiap 30 detik. Keduanya berjalan di jalur yang terpisah, jadi sangat mungkin data mengalir sempurna sementara gateway tampak mati. Modul ini pernah mengalaminya persis: stat memakai cap waktu ISO 8601 seperti pada rxpk, padahal gateway bridge mengurainya dengan pola lama 2006-01-02 15:04:05 MST — seluruh stat ditolak diam-diam selama berjam-jam. Sudah diperbaiki di `single_chan_pkt_fwd.py` (fungsi `stat_time_now()`), dan inilah alasan fungsi itu ada terpisah dari `iso_now()`.

Kalau benar-benar buntu, membangun ulang dari nol memakan waktu kurang dari satu menit — tetapi **seluruh pendaftaran EXP-01 hilang** dan harus diulang.

```
cd ~/chirpstack-docker
docker compose down -v
docker compose up -d
```

14.14.7 A.7 Tiga Hal yang Sebaiknya Tidak Dilakukan

PERINGATAN

1. **Jangan** menjalankan `docker system prune -a --volumes`. Perintah itu menghapus semua volume yang tidak terpakai — termasuk basis data ChirpStack bila container sedang mati.
2. **Jangan** mengedit `docker-compose.yml` bawaan repo ChirpStack. Semua penyesuaian lab ini ada di `docker-compose.override.yml`, dan Docker Compose otomatis menggabungkan keduanya — sehingga repo resminya tetap bersih dan bisa diperbarui.
3. **Jangan** menghapus folder `~/chirpstack-docker` selagi container menyala. Hentikan dulu dengan `docker compose down`, baru hapus.

Sebagai gambaran ruang: seluruh image modul ini berukuran sekitar 640 MB dan volume datanya puluhan MB — jauh di bawah kapasitas kartu memori Raspberry Pi lab ini.

14.15 Lampiran B — Menjalankan Lab untuk Beberapa Kelompok Sekaligus

Tiga kelompok, tiga Raspberry Pi, satu ruangan. Yang perlu diputuskan ada dua: siapa yang menjalankan server, dan bagaimana ketiga kelompok tidak saling mengganggu di udara.

14.15.1 B.1 Satu Server untuk Sekelas

Network server adalah **infrastruktur bersama**; itu memang bentuk LoRaWAN di dunia nyata. Satu gateway melayani banyak perangkat, dan satu server melayani banyak gateway. Lab ini sebaiknya mengikuti bentuk yang sama.

```
Kelompok 1: Pi + HAT
Kelompok 2: Pi + HAT
Kelompok 3: Pi + HAT
└── UDP :1700 → satu Pi berisi ChirpStack (Docker)
```

	Server bersama (dianjurkan)	Server sendiri tiap kelompok
Docker	hanya di satu Pi	di ketiga Pi
Yang dipasang di Pi kelompok	dua paket apt saja	Docker + 637 MB image
Kesiapan	menit	bergantung unduhan, dikali tiga
Pemisahan data antar-kelompok	lewat Application masing-masing	terpisah total
Mirip praktik nyata	ya	tidak

Tabel 14.26: Server bersama dibanding server per kelompok

Pi mana yang menjadi server bebas — boleh salah satu Pi kelompok, boleh Pi dosen. Yang penting alamat IP-nya tetap dan diketahui semua kelompok.

14.15.2 B.2 Yang Membedakan Tiap Kelompok

Bila ketiga kelompok memakai frekuensi **dan** kunci yang sama, akan terjadi hal yang jauh lebih membingungkan daripada sekadar tabrakan.

PERINGATAN

Gateway kelompok B ikut mendengar JoinRequest node kelompok A. Karena AppKey-nya sama persis, MIC-nya sah di mata server, dan server pun mengirim JoinAccept. Node kelompok A bisa berakhir join ke jaringan kelompok B — datanya muncul di dashboard yang salah, tanpa satu pun pesan galat di sisi mana pun.

Karena itu nomor kelompok menggeser tiga hal sekaligus, dan semuanya dihitung dari **satu** angka.

Kelompok	Kanal (EU433)	DevEUI Node 1 / Node 2	AppKey (awalan)	Application
1	433,175 MHz	0011223344556601 / ...6602	00...f1 / 00...f2	praktikum-wsn
2	433,375 MHz	0011223344557701 / ...7702	01...f1 / 01...f2	praktikum-wsn-k2
3	433,575 MHz	0011223344558801 / ...8802	02...f1 / 02...f2	praktikum-wsn-k3

Tabel 14.27: Pembagian kanal dan kunci tiap kelompok

Ketiga frekuensi itu bukan pilihan bebas: itulah **tiga kanal wajib rencana EU433**, yang sudah dikenal ChirpStack tanpa perlu konfigurasi tambahan. Kelompok 1 memakai nilai yang sama persis dengan seluruh contoh di modul ini, jadi tabel dan log di bab ini tetap berlaku apa adanya.

14.15.3 B.3 Langkah Tiap Kelompok

1 — Di Raspberry Pi kelompok: satu perintah, mengurus dependensi, SPI, dan menghitung Gateway EUI.

```
bash Modul11_lorawan_chirpstack/gateway/siapkan_gateway.sh \
  --kelompok 2 --server 192.168.1.45
```

Tambahkan `--dengan-server` pada Pi yang sekaligus menjalankan ChirpStack. Keluarannya berisi Gateway EUI, perintah menjalankan gateway, serta DevEUI dan AppKey yang harus didaftarkan — salin apa adanya.

```
Gateway EUI : 2CCF67FFFE53AC11
Kanal      : 433375000 Hz SF7BW125
Jalankan gateway:
python3 single_chan_pkt_fwd.py --server 192.168.1.45 --freq 433375000
DevEUI Node 1 : 0011223344557701
AppKey Node 1 : 01112233445566778899aabbccddeef1
```

2 — **Di sisi node:** ubah **satu** baris di `platformio.ini`, lalu unggah ulang kedua node.

```
-D KELOMPOK=2
```

Baris itu menentukan kanal sekaligus DevEUI dan AppKey; tidak ada berkas lain yang perlu disentuh. Periksa hasilnya di baris pembuka Serial Monitor.

```
=== LoRaWAN NODE 1 - Ruangan 1 (Kelompok 2) ===
Kanal : 433.375 MHz SF7BW125 (kanal tunggal)
```

3 — **Di ChirpStack:** tiap kelompok mendaftarkan gateway dan kedua node-nya sendiri lewat web UI (ini inti EXP-01, jangan dilewati). Asisten yang ingin menyiapkan atau memulihkan seluruh kelas dengan cepat bisa memakai:

```
python3 gateway/provision_lab.py --token <API-KEY> \
--gateway-id <EUI-KELOMPOK> --kelompok 2
```

14.15.4 B.4 Hal-Hal yang Perlu Diantisipasi

- **Jarak antar-meja.** Tiga kanal itu hanya berjarak 200 kHz, sedangkan bandwidth tiap sinyal 125 kHz dan node memancar 17 dBm. Dua papan yang berjarak beberapa sentimeter tetap dapat saling menulikan penerimaan meski beda kanal. Letakkan tiap kelompok di meja yang berbeda.
- **Lebih dari tiga kelompok.** EU433 hanya menyediakan tiga kanal wajib. Kelompok keempat boleh berbagi kanal dengan kelompok 1 **asalkan DevEUI dan AppKey-nya berbeda** — konsekuensinya hanya tabrakan yang lebih sering, bukan data yang tertukar. Skrip sengaja menolak `--kelompok 4` supaya keputusan itu diambil sadar, bukan tidak sengaja.
- **Satu Pi, satu radio.** Satu Raspberry Pi tidak bisa menjadi gateway dua kelompok sekaligus: SX1276 hanya punya satu modem, jadi satu frekuensi pada satu waktu.
- **Gateway EUI berbeda sendirinya.** EUI diturunkan dari MAC tiap Pi, jadi ketiga kelompok otomatis punya EUI berbeda tanpa diatur — tetapi berarti tiap kelompok wajib mendaftarkan EUI Pi-nya sendiri, bukan menyalin punya kelompok lain.

14.15.5 B.5 Kalau Tetap Ingin Meng-clone Kartu SD

Cara ini masuk akal hanya bila internet lab tidak dapat diandalkan. Jangan memakai `dd` mentah: kartu 128 GB akan disalin seluruhnya termasuk ruang kosong. Pakai **rpi-clone** (berbasis `rsync`, hanya data terpakai) atau `dd` + **PiShrink**.

Sesudah itu, **tiap hasil clone wajib dibereskan** — kalau tidak, ketiga Pi akan bentrok di jaringan.

```
sudo hostnamectl set-hostname pi-gw2          # 1. nama host jangan kembar
sudo rm /etc/ssh/ssh_host_*                    # 2. kunci host SSH jangan kembar
sudo dpkg-reconfigure openssh-server
sudo rm -f /etc/machine-id /var/lib/dbus/machine-id # 3. machine-id kembar membuat
sudo systemd-machine-id-setup                  # DHCP memberi IP yang sama
```

Keempat: basis data ChirpStack ikut tersalin, jadi isinya masih Gateway EUI Pi induk. Jalankan `siapkan_gateway.sh --kelompok <n> --token ...` di tiap clone untuk mendaftarkan EUI Pi tersebut.

14.15.6 B.6 Memindahkan Image Lewat Flashdisk

Bila beberapa Pi masing-masing perlu menjalankan ChirpStack sendiri sementara internet lab lemah, yang berat bukan pemasangan Docker-nya melainkan **unduhannya image**-nya. Image itu bisa dibungkus sekali di Pi yang sudah jalan, lalu dipindahkan.

```
# di Pi yang ChirpStack-nya sudah berjalan
bash gateway/simpan_image_docker.sh --keluar /media/pi/FLASHDISK/chirpstack.tar.gz

# di Pi tujuan, sesudah Docker terpasang
bash gateway/simpan_image_docker.sh --muat /media/pi/FLASHDISK/chirpstack.tar.gz
cd ~/chirpstack-docker && docker compose up -d
```

Daftar image tidak diketik manual, melainkan dibaca dari `docker-compose.yml` (`docker compose config --images`), sehingga tidak mungkin ketinggalan bila ChirpStack suatu saat menambah komponen. Kompresinya memakai `pigz` bila tersedia — Raspberry Pi 5 punya empat inti, dan `gzip` biasa hanya memakai satu.

Diukur di Pi lab ini: **6 image, 637 MB di dalam Docker → 165 MB** setelah dibungkus, **6,5 detik** untuk menyimpan dan **10,7 detik** untuk memuat kembali.

Dua hal yang **tidak** diselesaikan cara ini:

1. **Docker engine-nya sendiri tetap harus terpasang** di Pi tujuan. Yang dihemat hanya unduhan image. Bila Pi tujuan benar-benar tanpa internet sama sekali, siapkan paket `.deb`-nya lebih dahulu (`apt-get install --download-only`) dari mesin yang terhubung.
2. **Data tidak ikut terbawa.** Arsip ini berisi image, bukan volume — Pi tujuan akan menyala dengan basis data kosong, tanpa gateway dan tanpa device. Pendaftarannya diulang lewat web UI (EXP-01) atau `provision_lab.py`.

14.15.7 B.7 Yang Sudah Diuji

Pemindahan image B.6 diuji di perangkat pada 22 Agustus 2026, kedua arahnya: `docker save` menghasilkan arsip 165 MB dalam 6,5 detik, dan `--muat` mengembalikan keenam image (`Loaded image: chirpstack/chirpstack:4` dan seterusnya) dalam 10,7 detik.

Skema kelompok ini diuji di perangkat pada 22 Agustus 2026 dengan menjalankan kelompok 2 secara penuh: gateway dipindah ke 433,375 MHz, node di-flash dengan `-D KELOMPOK=2`, dan hasilnya node mengirim di 433,375 MHz ([TX] radio: 433375 kHz DR5), join OTAA dengan DevEUI 0011223344557702, mendapat DevAddr 00D1C237, lalu uplink-nya masuk ke Application **praktikum-wsn-k2** yang terpisah dari `praktikum-wsn` milik kelompok 1. Join berhasil pada percobaan kedua; percobaan pertamanya tidak diterima node meski JoinAccept sudah ditembakkan tepat waktu — kejadian yang sama sesekali muncul juga pada kelompok 1 dan memang sifat gateway kanal tunggal, bukan akibat pembagian kelompok.

14.16 Lampiran C — Menjalankan Gateway Otomatis Lewat systemd

`single_chan_pkt_fwd.py` yang dijalankan dari terminal akan mati begitu SSH ditutup atau Pi dimatikan. Untuk praktikum itu justru tepat — baris [RX] dan [TX] yang mengalir di layar adalah bahan EXP-02 sampai EXP-04. Tetapi untuk **demo yang ditinggal jalan** atau lab yang Pi-nya sering dimatikan, gateway sebaiknya hidup sendiri seperti ChirpStack.

14.16.1 C.1 Memasang

```
bash Modul11_lorawan_chirpstack/gateway/siapkan_gateway.sh \
--kelompok 1 --server 127.0.0.1 --service
```

Yang terpasang ada dua berkas, dan pemisahannya disengaja.

Berkas	Isi	Kapan disentuh
<code>/etc/systemd/system/lorawan-gateway.service</code>	cara menjalankan: pengguna, folder kerja, perintah	hampir tidak pernah
<code>/etc/default/lorawan-gateway</code>	SERVER, FREQ, OPSI	tiap ganti kanal atau alamat server

Tabel 14.28: Dua berkas yang dipasang oleh opsi `--service`

Jadi kelompok yang pindah kanal cukup mengubah satu baris.

```
sudo nano /etc/default/lorawan-gateway # FREQ=433375000
sudo systemctl restart lorawan-gateway
```

14.16.2 C.2 Perintah Harian

Perintah	Artinya
<code>systemctl status lorawan-gateway</code>	hidup atau tidak, plus beberapa baris terakhir
<code>journalctl -fu lorawan-gateway</code>	pengganti terminal — [RX] dan [TX] mengalir seperti biasa
<code>sudo systemctl stop lorawan-gateway</code>	matikan, misalnya sebelum menjalankan manual
<code>sudo systemctl start lorawan-gateway</code>	nyalakan lagi
<code>sudo systemctl disable --now lorawan-gateway</code>	matikan sekaligus cabut dari daftar boot

Tabel 14.29: Perintah systemd yang dipakai sehari-hari

PENTING

Folder repo pindah atau berganti nama? Berkas unit menyimpan lokasi skrip sebagai `WorkingDirectory`, jadi layanan akan gagal menyala dengan `status=200/CHDIR`. Perbaikannya satu perintah: jalankan ulang `siapkan_gateway.sh --kelompok <n> --service` dari lokasi yang baru — berkas unitnya ditulis ulang dengan path yang benar.

Jangan menjalankan skrip manual selagi layanan hidup. Keduanya akan berebut `/dev/spidev0.0` dan register `SX1276` yang sama, dan gejalanya bukan pesan galat melainkan paket yang hilang serta downlink yang kacau. Hentikan layanannya dulu.

14.16.3 C.3 Perilaku yang Sudah Diatur

- **Ikut hidup saat boot** (`WantedBy=multi-user.target`, `enable` dijalankan oleh skrip).
- **Bangun sendiri bila jatuh** — `Restart=on-failure` dengan jeda 10 detik, berguna bila HAT belum siap saat boot. Batasnya 10 percobaan per 5 menit, supaya kegagalan permanen (HAT lepas, SPI mati) tidak menjadi loop tanpa henti; sesudah itu layanan berhenti dan alasannya terbaca di `journalctl`.
- **Berhenti dengan bersih.** `systemd` mengirim `SIGTERM`, dan skrip menanganinya sama seperti `Ctrl-C`: radio dikembalikan ke mode sleep, SPI ditutup, GPIO dilepas. Di `journal` terlihat sebagai Dihentikan. lalu Radio dimatikan.

14.16.4 C.4 Yang Sudah Diuji

Diuji di perangkat pada 22 Agustus 2026: layanan terpasang, berstatus `enabled` dan `active`, dan uplink kedua node muncul di `journalctl` persis seperti saat dijalankan manual. `systemctl restart` pulih sendiri tanpa satu pun traceback, dan `systemctl stop` menghasilkan penutupan bersih (Dihentikan. → Radio dimatikan. → Deactivated successfully). Reboot sungguhan diuji pada 23 Agustus 2026: layanan tegak sendiri **12 detik** sesudah kernel selesai boot, radio terinisialisasi (`Init SX1276 ... OK`), dan gateway kembali tercatat aktif di ChirpStack — tanpa satu pun perintah diketik. Rincian waktunya di Lampiran A.5.

14.17 Lampiran D — Menyiapkan Raspberry Pi 5 Lain dari Nol

Resep lengkap dari Pi yang baru selesai di-*flash* sampai ChirpStack berjalan. Ada dua jalur: D.2 bila Pi punya internet yang layak, D.3 bila tidak — memakai arsip image yang sudah dibungkus (`docker save`) sehingga tidak ada unduhan ratusan MB dari Docker Hub.

14.17.1 D.1 Yang Perlu Ada Lebih Dulu

- Raspberry Pi OS **64-bit** (lab ini diuji di Debian 13 / Pi OS berbasis trixie), SSH aktif.
- Dragino LoRa GPS HAT terpasang dan SPI menyala. `siapkan_gateway.sh` akan menyalakan SPI sendiri bila belum, tetapi butuh satu kali reboot sesudahnya.
- Repositori lab ini ada di Pi tersebut:

```
git clone <url-repo> ~/Documents/WSN-IOT-prak-Lora
cd ~/Documents/WSN-IOT-prak-Lora
```

Bila Pi tidak terhubung internet sama sekali, salin foldernya lewat flashdisk.

14.17.2 D.2 Jalur Cepat — Pi Punya Internet

```
bash Modul11_lorawan_chirpstack/gateway/setup_chirpstack.sh
```

Satu perintah itu memasang Docker (lewat skrip resmi `get.docker.com`), mengambil `chirpstack-docker`, memasang penyesuaian EU433, lalu menyalakan seluruh service. Sesudah selesai, lanjut ke D.4.

Satu hal yang sering membingungkan sesudah pemasangan pertama: perintah `docker` masih menolak dengan `permission denied ... docker.sock`. Penyebabnya keanggotaan grup `docker` baru berlaku pada sesi login berikutnya — keluar dari SSH lalu masuk lagi, atau sementara pakai `sudo`.

14.17.3 D.3 Jalur Tanpa Unduhan Besar — Image dari Arsip

Docker engine-nya tetap perlu dipasang sekali (langkah 1); yang dihemat adalah unduhan **image** yang jauh lebih besar.

1 — Pasang Docker. Bila Pi punya internet sebentar saja, ini cukup:

```
curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
sudo sh /tmp/get-docker.sh
sudo usermod -aG docker "$USER"      # keluar-masuk SSH sekali sesudah ini
```

Bila benar-benar tanpa internet, siapkan pakatnya lebih dulu dari mesin yang terhubung (`sudo apt-get install --download-only docker.io`), salin berkas `.deb`-nya, lalu `sudo dpkg -i *.deb`.

2 — Ambil arsip image. Berkasnya (165 MB, berisi keenam image) ada di pCloud dengan kode publik `XZ2LUy7Zku2jHxEVvXmYmoQPqWw1GVbQaI7k`.

Lewat browser cukup klik unduh. Dari terminal Pi, tautan unduhan langsungnya harus diminta dulu ke API pCloud — tautan itu **kedaluwarsa dalam beberapa jam**, jadi jangan disimpan, mintalah baru setiap kali.

```
CODE=XZ2LUy7Zku2jHxEVvXmYmoQPqWw1GVbQaI7k
URL=$(curl -s "https://eapi.pcloud.com/getpublicdownload?code=$CODE" \
| python3 -c 'import sys,json; d=json.load(sys.stdin); print("https://"+d["hosts"][0]+d["path"])' )
wget -O chirpstack-images.tar.gz "$URL"
```

`eapi.pcloud.com` dipakai karena tautannya beralamat `e.pcloud.link` (server Eropa); untuk tautan `u.pcloud.link` gantilah menjadi `api.pcloud.com`.

Periksa hasil unduhan sebelum dipakai.

```
sha256sum chirpstack-images.tar.gz
# aa10fae4a3688d7552e667128e74e2d8063af26fa003c15e5cac07ee633821a4
```

Alternatifnya tanpa internet sama sekali: salin berkas yang sama lewat flashdisk dari Pi yang sudah jalan — lihat Lampiran B.6.

3 — Muat image ke Docker:

```
bash Modul11_lorawan_chirpstack/gateway/simpan_image_docker.sh --muat chirpstack-images.tar.gz
```

4 — Siapkan folder ChirpStack. `setup_chirpstack.sh` biasanya meng-*clone* `chirpstack-docker` dari GitHub. Bila Pi tidak punya internet, salin saja foldernya (hanya ± 850 KB, berisi berkas konfigurasi) dari

Pi yang sudah jalan ke ~/chirpstack-docker. Skripnya memeriksa keberadaan folder itu dan melewati proses clone bila sudah ada.

```
bash Modul11_lorawan_chirpstack/gateway/setup_chirpstack.sh
```

Image yang diperlukan sudah ada di Docker lokal, jadi `docker compose up -d` tidak akan menariknya lagi dari internet.

14.17.4 D.4 Menjadikan Pi Itu Gateway Kelompok

```
bash Modul11_lorawan_chirpstack/gateway/siapkan_gateway.sh --kelompok 2 --server 192.168.1.45
```

Ganti `--server` dengan alamat Pi yang menjalankan ChirpStack — boleh Pi ini sendiri (127.0.0.1) bila D.2 atau D.3 dijalankan di sini juga. Tambahkan `--service` bila gateway ingin hidup otomatis tiap Pi menyala (Lampiran C). Keluarannya memuat Gateway EUI serta DevEUI dan AppKey kelompok tersebut, yang tinggal didaftarkan di ChirpStack (EXP-01).

Sesudah itu Pi baru ini setara dengan Pi lab: web UI di `http://<ip>:8080` (bila menjalankan server sendiri), gateway mendengarkan di kanal kelompoknya, dan node tinggal diunggah dengan `-D KELOMPOK=<n>` yang cocok.

14.17.5 D.5 Yang Sudah Diuji

Pemasangan Docker lewat `get.docker.com` dijalankan di perangkat pada 22 Agustus 2026 di Raspberry Pi 5 dengan Debian 13 (hasil: Docker 29.7.2, Compose v5.5.0), diikuti `setup_chirpstack.sh` sampai ChirpStack melayani web UI. Pembungkusan dan pemuatan ulang image diuji dua arah (Lampiran B.6). Tautan pCloud di D.3 diuji pada 23 Agustus 2026: API-nya menghasilkan tautan unduhan yang sah, berkasnya berukuran 172.701.363 byte, dan potongan awal yang diunduh identik dengan arsip asli.

Yang **belum** diuji: menjalankan seluruh rangkaian D.3 di Raspberry Pi kedua yang benar-benar bersih — lab ini baru punya satu Pi.

A. Lampiran A — Perkakas Lintas Modul

Seluruh perkakas yang dipakai pada lebih dari satu modul berada di folder `tools/` pada akar repositori praktikum. Perkakas yang hanya dipakai satu modul — `monitor_serial.py`, `lora_monitor.py`, `upload_auto.py`, `cek_radio.py` — dimuat lengkap pada bagian “Kode Program” bab modul yang bersangkutan, sebab isinya memang berbeda-beda mengikuti format log tiap modul.

A.1 Mengenali Port Sebelum Mengunggah

Lab ini memakai Arduino Uno asli maupun klon secara bercampur. Firmware keduanya identik; yang berbeda hanya chip jembatan USB-ke-serial, dan itu hanya mengubah nama port di sistem operasi (lihat Tabel P.13 pada bab Pendahuluan). Skrip `tools/deteksi_port.py` membaca VID dan PID tiap port serial lalu melaporkan jenis board-nya, sehingga `upload_port` di `platformio.ini` dapat diisi dengan pasti, bukan ditebak.

```
python3 tools/deteksi_port.py          # daftar port + jenis board
python3 tools/deteksi_port.py --ini    # potongan platformio.ini siap tempel
```

Tanpa argumen, skrip mencetak daftar port beserta jenis jembatannya.

Port	VID:PID	Jenis	Jembatan USB
/dev/ttyACM0	2341:0043	Uno asli	ATmega16U2 (Arduino LLC)
/dev/ttyACM1	2341:0043	Uno asli	ATmega16U2 (Arduino LLC)
/dev/ttyUSB0	1a86:7523	klon	CH340/CH341

Dengan `--ini`, keluarannya berupa potongan `platformio.ini` yang tinggal disalin ke berkas modul yang sedang dikerjakan.

PENTING

Jalankan skrip ini **sebelum** mengunggah pada setiap modul yang memakai lebih dari satu board. Port di tiap `platformio.ini` yang disertakan repositori masih memakai nilai contoh untuk tiga Uno asli, dan nilai itu hampir pasti berbeda di komputer lain.

PERINGATAN

Skrip serial yang menyatel jalur DTR/RTS **sebelum** `open()` ditolak oleh CDC ATmega16U2 pada Uno asli dengan [Errno 110] Connection timed out, sementara pada klon CH340 hal itu lolos. Seluruh `monitor_serial.py` pada seri ini sudah tidak menyentuh jalur tersebut. Bila menulis skrip serial sendiri, buka port apa adanya — jangan mengatur DTR/RTS sebelum membukanya.

A.2 Kode Program

KODE SUMBER

Seluruh berkas di bawah dimuat lengkap pada bab ini. Salinan daringnya: [tools/](#)

- [tools/deteksi_port.py](#)

```
#!/usr/bin/env python3
"""Kenali jenis board pada tiap port serial, untuk lab dengan board bercampur.

Arduino Uno asli dan klon memakai mikrokontroler yang sama (ATmega328P) dan
firmware yang sama persis; yang berbeda hanya chip jembatan USB-ke-serial di
atas board. Perbedaan itu tidak menyentuh program sama sekali, tetapi mengubah
**nama port** di sistem operasi - dan itulah satu-satunya hal yang perlu
disesuaikan pada platformio.ini.
```



```
python3 tools/deteksi_port.py
python3 tools/deteksi_port.py --ini      # cetak baris siap tempel

Tanpa argumen, skrip mencetak daftar port beserta jenis jembatannya. Dengan
--ini, keluarannya berupa potongan platformio.ini yang tinggal disalin.
"""
import argparse
import glob
import os

# VID:PID jembatan USB yang lazim ditemui pada board Uno dan turunannya
JEMBATAN = {
    "2341:0043": ("Uno asli", "ATmega16U2 (Arduino LLC)"),
    "2341:0001": ("Uno asli", "ATmega8U2 (Arduino LLC)"),
    "2a03:0043": ("Uno asli", "ATmega16U2 (Arduino SRL)"),
    "1a86:7523": ("klon", "CH340/CH341"),
    "1a86:55d3": ("klon", "CH343"),
    "0403:6001": ("klon", "FTDI FT232"),
    "10c4:ea60": ("klon", "Silicon Labs CP2102"),
}

def baca(path):
    try:
        with open(path) as f:
            return f.read().strip()
    except OSError:
        return ""

def daftar_port():
    """Kumpulkan port serial USB beserta VID:PID-nya dari sysfs."""
    hasil = []
    for dev in sorted(glob.glob("/dev/ttyACM*") + glob.glob("/dev/ttyUSB*")):
        nama = os.path.basename(dev)
        # Jalur device di sysfs berbeda kedalamannya antara CDC-ACM dan
        # konverter USB-serial, jadi ditelusuri ke atas sampai VID ditemukan.
        d = os.path.realpath(f"/sys/class/tty/{nama}/device")
        vid = pid = ""
        for _ in range(4):
            d = os.path.dirname(d)
            vid, pid = baca(f"{d}/idVendor"), baca(f"{d}/idProduct")
            if vid and pid:
                break
        kunci = f"{vid}:{pid}".lower()
        jenis, chip = JEMBATAN.get(kunci, ("tidak dikenal", "-"))
        hasil.append((dev, kunci if vid else "?", jenis, chip))
    return hasil

def main():
    ap = argparse.ArgumentParser(description=__doc__,
                                formatter_class=argparse.RawDescriptionHelpFormatter)
    ap.add_argument("--ini", action="store_true",
                    help="cetak potongan platformio.ini siap tempel")
    args = ap.parse_args()

    port = daftar_port()
    if not port:
        print("Tidak ada port serial USB yang terdeteksi.")
        print("Periksa kabel data (bukan charge-only) dan hak akses grup dialout.")
        return

    if args.ini:
        print("; Tempel ke environment yang sesuai pada platformio.ini")
        for i, (dev, _, jenis, _) in enumerate(port):
            print(f"; board {i + 1} - {jenis}")
            print(f"upload_port = {dev}")
            print(f"monitor_port = {dev}")
        return

    print(f"{'Port':<16} {'VID:PID':<12} {'Jenis':<14} Jembatan USB")
    print("-" * 62)
    for dev, vp, jenis, chip in port:
        print(f"{dev:<16} {vp:<12} {jenis:<14} {chip}")
    print()
    print("Firmware kedua jenis board IDENTIK - keduanya ATmega328P dengan")
    print("protokol unggah 'arduino'. Yang berbeda hanya nama port di atas,")
    print("jadi cukup sesuaikan upload_port/monitor_port pada platformio.ini.")

```

```
if __name__ == "__main__":
    main()
```

Kode A.1: tools/deteksi_port.py — kenali jenis board pada tiap port serial

A.3 Peta Perkakas Tiap Modul

Modul	Perkakas pemantau	Perkakas lain
01-04	monitor_serial.py	upload_auto.py
05	monitor_serial.py, lora_monitor.py	upload_auto.py
06	— (keluaran langsung ke terminal Pi)	—
07, 07B	lora_monitor.py	cek_radio.py, upload_auto.py
08, 08B, 08C	lora_monitor.py	upload_auto.py
09, 10	lora_monitor.py	upload_auto.py
11	gateway/uplink_listen.py	gateway/provision_lab.py, upload_auto.py
semua	—	tools/deteksi_port.py

Tabel A.1: Perkakas yang tersedia pada tiap modul

Perkakas yang memerlukan pustaka tambahan hanya dua: `lora_monitor.py` (memerlukan `pyserial` dan `rich`) serta `gateway/uplink_listen.py` (memerlukan `paho-mqtt`). Sisanya berjalan dengan Python 3 standar, kecuali `monitor_serial.py` yang memerlukan `pyserial`.

A.4 Salinan Daring

Seluruh berkas yang dimuat di buku ini tersedia pada repositori praktikum github.com/hendrieepis/WSN-IOT-prak-Lora. Folder perkakas lintas modul: `tools/`. Aset pendukung yang tidak dimuat di buku — skematik shield dan HAT, user manual Dragino, serta berkas `logserial.md` tiap modul — juga berada di repositori yang sama.